Prof. Dr. Thomas Allweyer
Hochschule Kaiserslautern

Skript zur Vorlesung

Software-Engineering

Version 2.0

Oktober 2026

©2026 Thomas Allweyer

thomas.allweyer@hs-kl.de

Dieses Skript dient ausschließlich der Nutzung durch die Teilnehmer der vom Autor abgehaltenen Lehrveranstaltung „Software-Engineering“ zur Vor- und Nachbereitung. Die über diesen Teilnehmerkreis hinausgehende Weitergabe, Vervielfältigung und insbesondere die Bereitstellung in elektronischen Medien ist ohne schriftliche Genehmigung durch den Autor nicht zulässig.

Inhalt

Inhalt ... 3

1 Einführung ... 8
1.1 Kleine und große Projekte ... 8
1.2 Zunehmende Anforderungen an die Software-Entwicklung ... 10
1.3 Erfolgreiche und weniger erfolgreiche Software-Entwicklung ... 10
1.4 Zum Begriff Software-Engineering ... 11
1.5 Ziele des Software-Engineering im Spannungsfeld ... 12
1.6 Vorgehensmodelle zur Software-Entwicklung ... 13
1.6.1 Plangetriebene Vorgehensmodelle ... 14
1.6.2 Agile Vorgehensmodelle ... 15
1.7 Braucht man im KI-Zeitalter noch Software-Engineering? ... 17

2 Anforderungen ... 19
2.1 Problemraum und Lösungsraum ... 19
2.2 Arten von Anforderungen ... 21
2.2.1 Funktionale Anforderungen ... 21
2.2.2 Qualitätsanforderungen (nicht-funktionale Anforderungen) ... 21
2.2.3 Randbedingungen ... 23
2.3 Erhebung und Management von Anforderungen ... 23
2.4 Dokumente zur Anforderungsspezifikation ... 25
2.5 Nachverfolgbarkeit von Anforderungen ... 27
2.6 Qualitätsszenarien ... 28

3 UML-Modellierungswerkzeuge ... 31

4 Use-Cases und Use-Case-Diagramme ... 32
4.1 Einführendes Beispiel ... 32
4.2 Beschreibung von Use-Cases ... 33
4.3 Include-Beziehung ... 34
4.4 Extend-Beziehung ... 36
4.5 Ablauf bei „include“ und „extend“ ... 37
4.6 Verwendung von „include“ und „extend“ ... 39
4.7 Generalisierung von Use-Cases ... 40
4.8 Kommunikation mit externen Systemen ... 41
4.9 Generalisierung von Akteuren ... 41
4.10 Beispiel: Use-Case-Diagramm der Autovermietung ... 42
4.11 Geeignete Größe für Use-Cases ... 43
4.12 Vorgehen beim Erstellen von Use-Cases ... 44
4.13 Kriterien für gute Use-Cases ... 44
4.14 Hinweise zu den Use-Case-Dokumentationen ... 46

4.15 Fallstudie: Geräteverwaltung ... 47

5 Modellierung von Abläufen ... 50
5.1 Entscheidungen ... 51
5.2 Zusammenführungen ... 53
5.3 Weitere Elemente von BPMN-Diagrammen ... 55
5.4 Ereignisse ... 57
5.5 Paralleler Gateway ... 59
5.6 Inklusiver Gateway ... 60
5.7 Unterschiedliche Arten von Endereignissen ... 60
5.8 Unterprozesse ... 61
5.9 Objektflüsse ... 62
5.10 Einsatzmöglichkeiten für Prozessmodelle ... 63
5.11 Modellierung eines Geschäftsprozesses ... 64
5.12 Ablauf innerhalb eines Use-Case ... 65
5.13 Kontrollfluss von Programmcode ... 69
5.14 Mehr zu BPMN ... 69

6 Klassendiagramme ... 70
6.1 Einführendes Beispiel ... 70
6.2 Modellierung von Objekten ... 73
6.3 Assoziationen ... 75
6.4 Verbundene Objekte ... 78
6.5 Multiplizitäten ... 78
6.6 Multiplizitäten lesen und anwenden ... 81
6.7 Implementierung mehrwertiger Assoziationen ... 83
6.8 Wahl der Navigationsrichtung ... 86
6.9 Eigenschaften von Assoziationen ... 87
6.10 Mehrere Assoziationen zwischen zwei Klassen ... 90

7 Klassendiagramme – weitere Konzepte ... 92
7.1 Weitere Angaben zu Attributen und Methoden ... 92
7.2 Aufzählungstypen ... 94
7.3 Reflexive Assoziationen ... 94
7.4 Aggregationen ... 96
7.5 Kompositionen ... 99
7.6 Assoziative Klassen ... 101
7.7 Mehrfachassoziationen ... 102

8 Klassendiagramme – Vererbung ... 105
8.1 Generalisierung und Spezialisierung ... 105
8.2 Vererben von Methoden und Attributen ... 106
8.3 Substitutionsprinzip ... 109
8.4 Sichtbarkeiten bei Vererbung ... 110
8.5 Vererben von Assoziationen ... 110

8.6 Mehrstufige Vererbungshierarchie ... 112
8.7 Abstrakte Klassen ... 113
8.8 Mehrfachvererbung ... 116
8.9 Schnittstellen ... 117

9 Sequenzdiagramme ... 120
9.1 Einführendes Beispiel ... 120
9.2 Bedingte und mehrfache Methodenaufrufe ... 123
9.3 Darstellung weiterer Aspekte in Sequenzdiagrammen ... 126
9.4 Einsatz von Sequenzdiagrammen ... 129

10 Klassen- und Sequenzdiagramme in der Analyse ... 130
10.1 Entwicklung von Analysediagrammen ... 130
10.2 Beispiel für ein Analysediagramm ... 131
10.3 Verfeinerung des Klassendiagramms ... 136
10.4 Analysemuster ... 136
10.4.1 Analysemuster Liste ... 136
10.4.2 Analysemuster Typ-Exemplar ... 136
10.4.3 Analysemuster Historie ... 137
10.4.4 Analysemuster Gruppe und Gruppenhistorie ... 138
10.4.5 Analysemuster Wechselnde Rollen ... 139
10.4.6 Analysemuster Baugruppe ... 141
10.4.7 Analysemuster Stückliste ... 141
10.4.8 Analysemuster Verwendungsnachweis ... 142

11 Analysediagramm – Fallstudie Geräteverwaltung ... 144
11.1 Räume und Mitarbeiter ... 144
11.2 Geräte ... 145
11.3 Reservierungen ... 147
11.4 Ausleihe ... 147
11.5 Verfügbarkeit ermitteln ... 149
11.6 Überblick über die Entity-Klassen ... 150
11.7 Paketdiagramm ... 151

12 Zustandsdiagramme ... 154
12.1 Zustandsdiagramm der Klasse Gerät ... 154
12.2 Implementierung von Zustandsdiagrammen ... 156
12.3 Angaben an Transitionen ... 158
12.4 Weitere Implementierungsmöglichkeiten ... 160
12.5 Zustandsdiagramme – weitere Konzepte ... 163

13 Oberflächenentwurf ... 166
13.1 Hauptnavigation ... 167
13.2 Anlegen eines Objekts ... 169
13.3 Finden eines Objekts ... 171

13.4 Bearbeiten eines Objekts ... 173
13.5 Zuordnung von Objekten ... 173
13.6 Umsetzung von Vererbungsstrukturen ... 176
13.7 Fallbeispiel Geräteverwaltung ... 178

14 Architektur ... 185
14.1 Begriff der Software-Architektur ... 185
14.2 Strukturierung mit Hilfe von Paketen ... 187
14.3 Lose Kopplung ... 188
14.4 Komponenten und Komponentendiagramme ... 192
14.5 Modellierung der physischen Struktur ... 194
14.6 Architekturstile ... 195
14.6.1 Pipes and Filters ... 196
14.6.2 Repository („Shared Data") ... 197
14.6.3 Objektorientiert ... 198
14.6.4 Layered (Schichtenarchitekturen) ... 198
14.6.5 Nutzung von Architekturstilen ... 200

15 Entwurf ... 201
15.1 Entwurfsprinzipien ... 201
15.2 Entwurf der Fachkonzept-Schicht ... 203
15.2.1 Zusammenfassen von Klassen ... 204
15.2.2 Vererbung reduzieren ... 205
15.2.3 Stereotypen ... 208
15.2.4 Objektverwaltung ... 209
15.2.5 Verfeinerung von Assoziationen ... 211
15.2.6 Verfeinerung von Attributen und Methoden ... 216

16 Entwurfsmuster ... 218
16.1 Fassade ... 218
16.2 Singleton ... 220
16.3 Strategie ... 221
16.4 Schablonenmethode ... 222
16.5 Fabrikmethode ... 223
16.6 Kompositum ... 224
16.7 Dekorierer ... 226
16.8 Beobachter ... 230

17 Persistenz ... 232
17.1 Abbildung der Objektstrukturen auf Datenbanktabellen ... 232
17.1.1 Abbildung einfacher Klassen ... 232
17.1.2 Umsetzung von 1:1-Assoziationen ... 233
17.1.3 Umsetzung von 1:*-Assoziationen ... 233
17.1.4 Umsetzung von *:*-Assoziationen ... 235
17.1.5 Umsetzung von Vererbungsbeziehungen ... 236

17.2 Abspeichern und Materialisieren von Objekten ... 238
17.3 Lazy Instantiation ... 238
17.4 Proxy-Muster ... 240
17.5 Entwurf der Persistenzschicht ... 242
18 Testen ... 246
18.1 Grundlagen ... 246
18.1.1 Fehler ... 246
18.1.2 Test-Stufen ... 247
18.1.3 Unterschiedliche Sichten auf das Testobjekt ... 247
18.1.4 Weitere Begriffe ... 248
18.2 Testfallexplosion ... 248
18.3 Äquivalenzklassenbildung ... 251
18.3.1 Beispiel zur Äquivalenzklassenbildung ... 251
18.3.2 Abhängigkeiten der Parameter ... 253
18.3.3 Äquivalenzklassen und Objektorientierung ... 254
18.4 Code-Überdeckung beim Testen ... 254
18.4.1 Anweisungsüberdeckung ... 255
18.4.2 Zweigüberdeckung ... 256
18.4.3 Pfadüberdeckung ... 256
18.4.4 Bedingungsüberdeckung ... 258
18.4.5 Datenflussorientierte Überdeckungen ... 260
18.5 Testplanung und -durchführung ... 262
18.5.1 Testobjekte und Leistungsmerkmale ... 262
18.5.2 Teststrategie ... 263
18.5.3 Testabläufe ... 265
18.5.4 Test-Ende-Kriterien ... 265
18.5.5 Testdokumentation ... 266
18.6 Testautomatisierung ... 268
18.6.1 JUnit-Tests schreiben ... 268
18.6.2 JUnit-Tests ausführen ... 270
18.6.3 Assert-Methoden ... 271
18.6.4 Test-Fixtures ... 272
18.6.5 Exceptions in Tests ... 273
18.6.6 Fazit JUnit ... 274
19 Quellenverzeichnis ... 275

1 Einführung

1.1 Kleine und große Projekte

In den Grundlagenveranstaltungen zur Programmierung erstellt man meist kleine, ganz gut überschaubare Programme. In der Praxis geht es jedoch darum, umfassende Software-Produkte zu erstellen, die um ein Vielfaches größer und komplexer sind. Solche Produkte erfordern die koordinierte Zusammenarbeit mehrerer Entwicklerinnen und Entwickler über einen längeren Zeitraum hinweg. Es leuchtet ein, dass man sich hier nicht einfach an seine Entwicklungsumgebung setzen und losprogrammieren kann. Stattdessen muss man sich Gedanken darüber machen, was das Produkt eigentlich können soll, wie es aufgebaut sein muss, wie die Arbeit aufgeteilt wird, usw.

Je größer das Team, desto größer ist auch der Aufwand für die Kommunikation und die Koordination der Beteiligten. Dies wird deutlich, wenn man betrachtet, mit wie vielen Kollegen sich die Mitglieder unterschiedlich großer Teams abstimmen müssen. Geht man davon aus, dass jeder gelegentlich mit jedem anderen sprechen muss, so kann man die Zahl der Kommunikationskanäle innerhalb eines Teams bestimmen.

Beträgt die Teamgröße *n*, so bestehen für das erste Teammitglied *n-1* Kommunikationsbeziehungen – eine zu jedem anderen Teammitglied. Für das zweite Teammitglied kommen weitere *n-2* Kommunikationsbeziehungen hinzu, da die Kommunikationsbeziehung mit dem ersten Teammitglied bereits bei der Zählung dessen Beziehungen berücksichtigt wurde. Die Zahl der Kommunikationsbeziehungen ist somit $(n\text{-}1) + (n\text{-}2) + \ldots + 1 = \frac{n*(n-1)}{2}$.
Einige Beispiele sind in Abbildung 1 dargestellt.

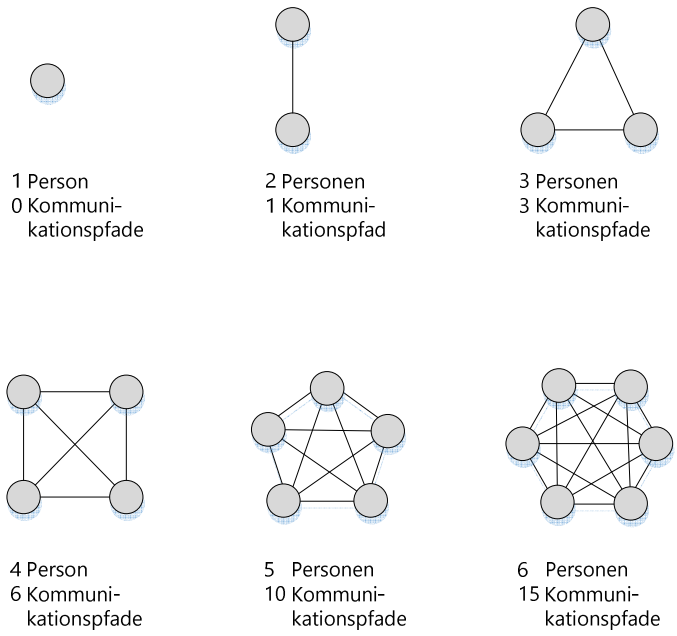


Abbildung 1: Wachstum des Kommunikationsbedarfs

Ab vier Personen ist die Zahl der Kommunikationspfade größer als die Zahl der Personen. Der Kommunikationsbedarf wächst somit überproportional mit der Teamgröße.

Mit einem größeren Software-Produkt steigt aber nicht nur der Kommunikationsbedarf, sondern auch der gesamte Erstellungsaufwand überproportional (Abbildung 2). Kennt man den Aufwand, den ein kleines Projekt verursacht hat, darf man diesen Aufwand also nicht einfach linear extrapolieren. Ein doppelt so großes Produkt erfordert mehr als den doppelten Aufwand.

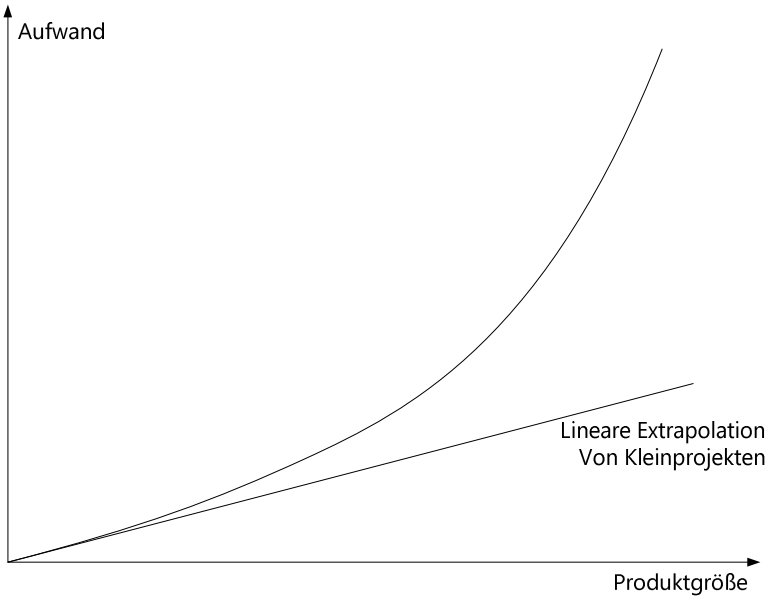


Abbildung 2: Überproportionaler Anstieg des Aufwandes

Wie misst man Produktgröße und Aufwand, und in welchen Einheiten gibt man sie an?

Als Aufwand können die Gesamtkosten betrachtet werden, die für die Entwicklung des Produktes anfallen. Da bei einer reinen Softwareentwicklung die Personalkosten den überwiegenden Anteil der Software ausmachen, wird oft auch nur die anfallende Arbeit betrachtet. Sie kann in Personentagen, Personenmonaten etc. angegeben werden. Ein Personentag bedeutet, dass eine Person einen Tag lang arbeitet. Arbeitet ein dreiköpfiges Team fünf Tage lang an einer Aufgabe, dann ist der entstandene Aufwand

3 Personen x 5 Tage = 15 Personentage

Denselben Aufwand hätte man entsprechend auch, wenn fünf Personen drei Tage lang arbeiten.

Schwieriger zu fassen ist die Größe eines Softwareproduktes. Für einen einfachen Vergleich wird häufig einfach die Zahl der Code-Zeilen (Lines of Code) herangezogen. Ein genauer Vergleich ist damit allerdings nicht möglich, denn die Zahl der Code-Zeilen hängt von sehr viel verschiedenen Faktoren ab, wie z. B. der Programmiersprache und dem Programmierstil. So können mehrere Anweisungen in eine Zeile geschrieben werden, oder sie werden über mehrere Zeilen verteilt. Auch führt etwa ein schlechter Programmierstil, bei dem viel Code

per „Copy and Paste“ dupliziert und dann verändert wird, zu einer höheren Zahl an Code-Zeilen als ein besserer Programmierstil.

Daher gibt es verschiedene Ansätze, ein anderes Maß für die Größe eines Software-Produktes zu finden, beispielsweise indem man die von der Software bereitgestellte Funktionalität mit Hilfe sogenannter Funktionspunkte (Function Points) bewertet. Zusätzlich werden technische Faktoren berücksichtigt, z. B. ob es sich um eine für viele tausend Nutzer skalierbare Software handelt.

1.2 Zunehmende Anforderungen an die Software-Entwicklung

In Zeiten der ständig wachsenden Vernetzung und Digitalisierung hat man es mit immer größeren Software-Systemen zu tun. Bestand die Steuerungssoftware des seit den 1980er-Jahren eingesetzten amerikanischen Space-Shuttle laut [Cand15] aus ca. 400.000 Codezeilen, so kann die Software, die sich in einem modernen Auto befindet, durchaus 100 Millionen Codezeilen umfassen. Und auch ein Betriebssystem besteht heute aus vielen Millionen Zeilen, im Falle von Windows 7 sind es etwa 40 Millionen.

Mit der Größe der Software steigt auch ihre Komplexität. Daher ist es erforderlich, sie geeignet zu strukturieren und in verschiedene Komponenten aufzuteilen. Auch die erforderlichen Tests der Software werden aufwändiger und schwieriger.

Zunehmend wird Software für sicherheitskritische Aufgaben eingesetzt. Man denke nur an medizintechnische Software, z. B. zur Kontrolle lebenserhaltender Systeme, oder an die Software zur Steuerung eines autonomen Fahrzeugs. Zur Entwicklung solcher Programme sind Methoden und Vorgehensweisen erforderlich, die eine hohe Zuverlässigkeit und Sicherheit gewährleisten.

Nicht zuletzt ändern sich die Anforderungen, die eine Software erfüllen muss, häufig bereits während der Entwicklung. Daher muss das Entwicklungsteam in der Lage sein, flexibel auf Änderungen zu reagieren.

Noch gar nicht berücksichtigt ist hierbei der Einsatz von Künstlicher Intelligenz (KI). Zum einen erhöht die Integration von KI-Kompontenen die Komplexität der Systeme weiter. Zum anderen kann mit Hilfe von KI sehr viel Code in kurzer Zeit generiert werden. Hierdurch werden die Entwickler zwar von vielen Routine-Aufgaben entlastet, doch stellen das Zusammenspiel mit KI-Agenten und die Sicherstellung qualitativ hochwertiger und sicherer Software ganz neue Herausforderungen dar.

1.3 Erfolgreiche und weniger erfolgreiche Software-Entwicklung

Dass die Entwicklung großer und komplexer Softwaresysteme schwierig ist, zeigt sich auch daran, dass längst nicht alle Projekte erfolgreich sind. Eine regelmäßig wiederholte Untersuchung zu diesem Thema trägt den bezeichnenden Titel „Chaos Report“. Im Jahr 2015 wurden die folgenden Zahlen ermittelt [HaWo15]:

- 29% aller untersuchten Projekte waren erfolgreich.
- Bei 52% wurde der Zeit- oder Kostenrahmen deutlich überschritten, oder das Ergebnis entsprach nicht den Erwartungen.
- 19% wurden gar als gescheitert gewertet, d. h. sie wurden komplett abgebrochen.

Auch wenn die Zahlen dieser Untersuchung in der Fachwelt nicht unumstritten sind, macht jeder, der an einem großen Software-Projekt mitarbeitet, die Erfahrung, dass ein solches Projekt schwierig zu meistern ist.

Woran liegt es, dass viele Projekte Probleme bekommen oder gar scheitern? Ein Vergleich verschiedener Studien und Veröffentlichungen identifizierte die folgenden sieben Hauptgründe für das Scheitern von IT-Projekten [Gull11]:

1. Schlechte Projektplanung und -leitung
2. Unzureichende Kommunikation
3. Fehlendes Management von Änderungen, Risiko, Finanzen und Performance
4. Mangelnde Abstimmung mit Auftraggebern und Benutzern
5. Unzureichende Unterstützung durch die Unternehmensleitung
6. Fehlende soziale Kompetenzen, Anpassungsfähigkeit und Erfahrung der Teammitglieder
7. Schlechte oder fehlende Methoden und Werkzeuge

Es fällt auf, dass keiner der genannten Gründe mit technischen Problemen, Programmierkenntnissen oder ähnlichem zu tun hat. Die Hauptschwierigkeiten bei der Softwareentwicklung hängen vielmehr mit Planung, Management, Kommunikation, Zusammenarbeit und Methoden zusammen. Es werden daher geeignete Ansätze und Vorgehensweisen benötigt, um diesen Schwierigkeiten zu begegnen. Und genau damit beschäftigt sich das Software-Engineering.

1.4 Zum Begriff Software-Engineering

Der Begriff „Engineering“ deutet darauf hin, dass Software systematisch und ingenieurmäßig entwickelt werden sollte – ähnlich wie Gebäude oder Maschinen. Wenn ein Haus gebaut wird, fängt man nicht einfach an, darauf los zu mauern. Stattdessen wird zunächst der Bauherr nach seinen Wünschen befragt, und es wird ein Bauplan gezeichnet. Der Architekt greift dabei auf bewährte Architekturprinzipien zurück. Auch wird beispielsweise geplant, wann und in welcher Reihenfolge die beteiligten Firmen und Handwerker ihre jeweiligen Arbeiten durchführen sollen. In vergleichbarer Weise kann man bei der Softwareentwicklung nicht einfach darauf los programmieren ohne vorher den Auftraggeber zu befragen, die Software zu entwerfen, das Vorgehen zu planen usw.

Der Begriff „Software-Engineering“ kann definiert werden als „the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software” [BoFa14]. Übersetzt handelt es sich bei Software-Engineering also um:

Die Anwendung eines

- systematischen,
- disziplinierten,
- quantifizierbaren

Ansatzes

- zur Entwicklung
- zum Betrieb
- und zur Wartung

von Software.

Auch wenn es im Folgenden vor allem um die eigentliche Entwicklung von Software geht, sind systematische Vorgehensweisen für die Zeit nach der Fertigstellung, d. h. für den Betrieb und die Wartung, genauso wichtig.

Im ersten Teil der obigen Definition taucht der Begriff „quantifizierbar" auf. Es geht also darum, Software-Entwicklung auch zahlenmäßig erfassbar und messbar zu machen. Zum Beispiel sollte es möglich sein, den für eine geplante Software benötigten Arbeitsaufwand verlässlich zu schätzen.

1.5 Ziele des Software-Engineering im Spannungsfeld

Im oben erwähnten Chaos-Report wurden nur solche Softwareprojekte als erfolgreich gewertet, die nicht nur die gewünschte Funktionalität lieferten, sondern auch noch rechtzeitig fertig wurden und im Kostenrahmen blieben. Zudem muss selbstverständlich auch die Qualität stimmen.

Die vier genannten Zieldimensionen Funktionalität, Qualität, Zeit und Kosten stehen in einem Spannungsverhältnis zueinander. Möchte man etwa mehr Funktionalität bei gleichbleibender Qualität entwickeln, so wird dies länger dauern und mehr kosten. Strebt man bei unveränderter Funktionalität eine höhere Qualität an, so wird dies ebenfalls zu längeren Entwicklungszeiten und höheren Kosten führen, da man beispielsweise Zeit und Geld für zusätzliche Tests benötigt.

Verlangt man von einem Entwicklungsteam, dieselbe Funktionalität schneller und zu niedrigeren Kosten zu entwickeln, dürfte dies dazu führen, dass die Qualität sinkt – z. B. weil Zeit und Geld beim Testen eingespart werden. Die Verbesserung bei einem Kriterium führt daher häufig zur Verschlechterung bei den anderen Kriterien.

Abbildung 3 visualisiert diesen Zusammenhang in Form eines „Spannungsvierecks". Man kann sich vorstellen, die Fläche des Vierecks sei unveränderlich. Zieht man dann an einer Ecke nach außen, um das Projekt in der betreffenden Dimension zu verbessern, dann rutschen die

anderen Ecken auf den Achsen nach innen, d. h. in den betreffenden Dimensionen treten Verschlechterungen ein.

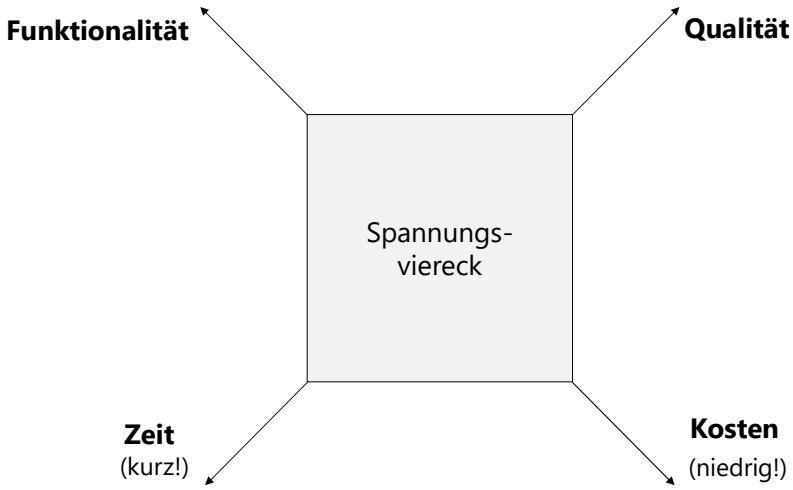


Abbildung 3: Ziele des Software-Engineering im Spannungsviereck (in Anlehnung an [Tiem18).

Die Fläche des Vierecks symbolisiert die Produktivität eines Software-Entwicklungsteams. Wünschenswert ist es, die Fläche des Vierecks und damit die Produktivität insgesamt zu erhöhen, so dass gleichzeitige Verbesserungen auf allen vier Achsen möglich sind. Mit Hilfe eines besseren Software-Engineerings kann dies gelingen. Ein Team, das bessere Methoden und Vorgehensweisen einsetzt, kann in kürzerer Zeit und zu niedrigeren Kosten mehr Funktionalität mit einer höheren Qualität entwickeln als ein anderes Team, das über schlechtere Methoden und Vorgehensweisen verfügt.

1.6 Vorgehensmodelle zur Software-Entwicklung

Im Laufe der Zeit wurden verschiedene Vorgehensmodelle zur Entwicklung von Software entwickelt. Ein solches Vorgehensmodell gibt dem Entwicklungsteam einen Rahmen für seine Arbeit vor. Gelegentlich wird auch vom „Software-Entwicklungs-Prozess" gesprochen. Typische Inhalte eines Vorgehensmodells sind:

- Aktivitäten,
  z. B. Anforderungsanalyse, Entwurf, Implementierung, Testen
- Produkte,
  z. B. verschiedene Dokumente, Quellcode, Installationsroutinen
- Rollen,
  z. B. Projektleiter, Entwickler, Tester
- Regeln
- Methoden und Werkzeuge

Es existiert eine Vielzahl von unterschiedlichen Vorgehensmodellen. Wichtig ist insbesondere die Unterscheidung zwischen plangetriebenen und agilen Vorgehensmodellen.

1.6.1 Plangetriebene Vorgehensmodelle

Plangetriebene Vorgehensmodelle sind gekennzeichnet durch:

- Die genaue und vollständige Ermittlung der Anforderungen zu Beginn des Projektes
- Einen detaillierten Vertrag zwischen Auftraggeber und der Softwareentwicklungsfirma
- Ein sequenzielles Vorgehen, bei dem die einzelnen Schritte nacheinander durchgeführt werden
- Die komplette zeitliche Planung des Projektes zu Beginn
- Die erstmalige Bereitstellung einer laufenden Software am Ende des Projektes
- Eine umfangreiche und vollständige Dokumentation
- Recht genaue Vorgaben

Ein typischer Vertreter eines plangetriebenen Vorgehensmodells ist das Wasserfallmodell (Abbildung 4).

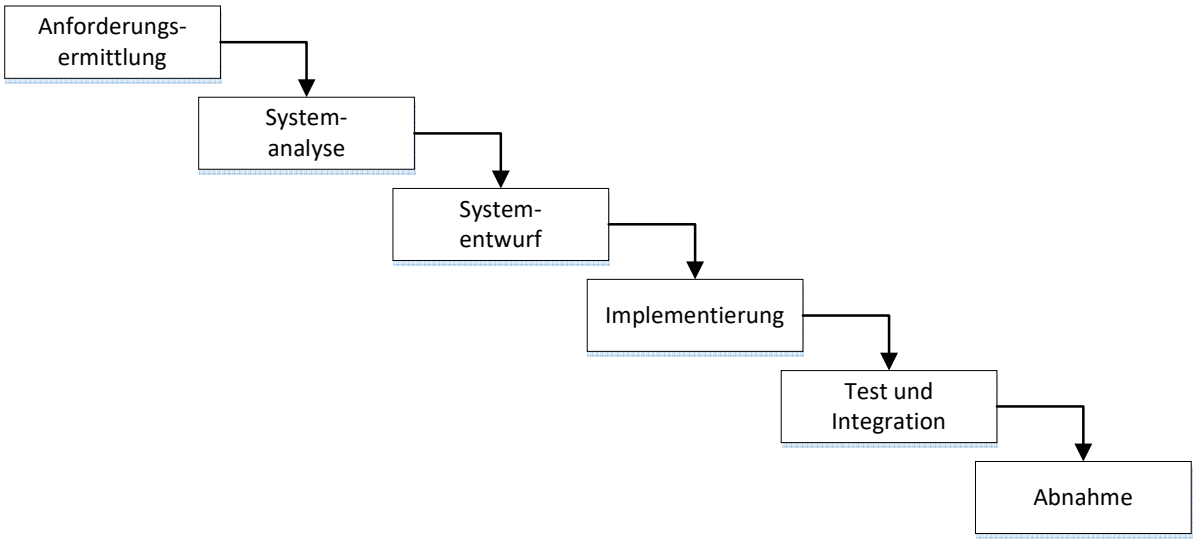


Abbildung 4: Struktur des Wasserfallmodells (nach [Goll 11])

Aus der Abbildung wird deutlich, wie der Name entstanden ist: Ähnlich wie bei einem mehrstufigen Wasserfall das Wasser von einem Becken ins andere fließt, fließen die Ergebnisse jeder Phase in die nächste Phase. Dabei darf jede Phase erst beginnen, wenn die vorangehende Phase komplett abgeschlossen ist.

So darf beispielsweise erst dann mit dem Systementwurf, d. h. der softwaretechnisch ausgerichteten Spezifikation der Komponenten, begonnen werden, wenn die Anforderungen des Kunden vollständig analysiert worden sind. Der Grund dafür lässt sich leicht nachvollziehen: Beginnt man mit dem Entwurf zu früh, so werden die noch nicht genau analysierten Anforderungen nicht berücksichtigt. Entsprechend werden sie in den nächsten Phasen auch nicht umgesetzt. Unter Umständen führt dies dazu, dass erst ganz am Ende am fertigen System festgestellt wird, dass die Kundenanforderungen nicht erfüllt sind. Aufwändige und teure Nacharbeiten sind die Folge.

Sind während eines Projektes Änderungen erforderlich, weil sich z. B. die Kundenwünsche geändert haben, so muss man in die entsprechende Phase zurückspringen, die dort erstellten Produkte (z. B. eine Anforderungsspezifikation) ändern und dann die Produkte der nachfolgenden Phasen (z. B. Entwurfsdokumente und Quellcode) entsprechend anpassen.

Änderungen führen also unter Umständen zu einem recht hohen Aufwand. Dies ist auch einer der Kritikpunkte an den klassischen Vorgehensmodellen. Da ständige Änderungen heute in vielen Projekten eher die Regel als die Ausnahme sind, gewannen in den vergangenen Jahren agile Vorgehensmodelle zunehmend an Bedeutung.

1.6.2 Agile Vorgehensmodelle

Agile Vorgehensmodelle sind gekennzeichnet durch:

- Die genaue Ausarbeitung weniger grundlegenden Anforderungen zu Beginn.
- Die Ermittlung und Ausarbeitung weiterer Anforderungen während des Projektes.
- Ein enges Vertrauensverhältnis mit dem Kunden und die ständige Kommunikation mit ihm.
- Ein iteratives Vorgehen, bei dem die einzelnen Schritte immer aufs Neue wiederholt werden, wobei der Funktionsumfang der Software immer größer wird.
- Eine genaue zeitliche Planung immer nur für die jeweils nächste Iteration
- Die frühzeitige Bereitstellung einer lauffähigen Software, deren Funktionsumfang dann von Iteration zu Iteration erweitert wird.
- Eine Reduktion der Dokumentation auf das Nötigste und den Ersatz vieler Dokumente durch direkte Kommunikation.
- Eine hohe Eigenverantwortung und Selbstdisziplin der Mitarbeiter.

Eine typische agile Vorgehensweise, die sehr weit verbreitet ist, heißt „Scrum“. Abbildung 5 enthält einige zentrale Elemente von Scrum:

- Sprint
  Ein Sprint ist eine Iteration vorher festgelegter Dauer (meist zwischen ein und vier Wochen), in der die eigentliche Entwicklung stattfindet und die vorher festgelegten Aufgaben abgearbeitet werden. Das Team darf dabei nicht von außen durch Änderungswünsche o. ä. gestört werden.
- Daily Scrum
  Ein fünfzehnminütiges Treffen, das jeden Tag zur selben Uhrzeit durchgeführt wird. Jedes Teammitglied stellt kurz vor, was es seit dem letzten Tag gemacht hat, was es bis zum nächsten Tag machen will und wo es Unterstützung benötigt.
- Product-Backlog
  Dabei handelt es sich um eine Sammlung von Anforderungen, die noch umgesetzt werden sollen. Meist sind sie auf Karteikarten notiert, die sich leicht umsortieren, ergänzen und entfernen lassen, so dass man auf Änderungen der Anforderungen reagieren kann.

- Selected Backlog
  Vor Beginn jedes Sprints wählt das Team die Anforderungen aus dem Product-Backlog aus, die es in diesem Sprint bearbeiten wird.
- Sprint-Backlog
  Das Sprint-Backlog enthält die Aufgaben, die durchgeführt werden müssen, um die Anforderungen aus dem Selected-Backlog umzusetzen. Sie werden an einer Wandtafel aufgehängt und je nach Zustand (offen, in Arbeit, abgeschlossen) an eine andere Stelle umgehängt, so dass der Arbeitsfortschritt auf einen Blick sichtbar wird.
- Ausführbare Software
  Nach jedem Sprint steht eine neue Version der Software zur Verfügung, die bereits ausführbar ist und prinzipiell auch ausgeliefert werden könnte. Mit jedem Sprint kommt mehr Funktionalität hinzu.

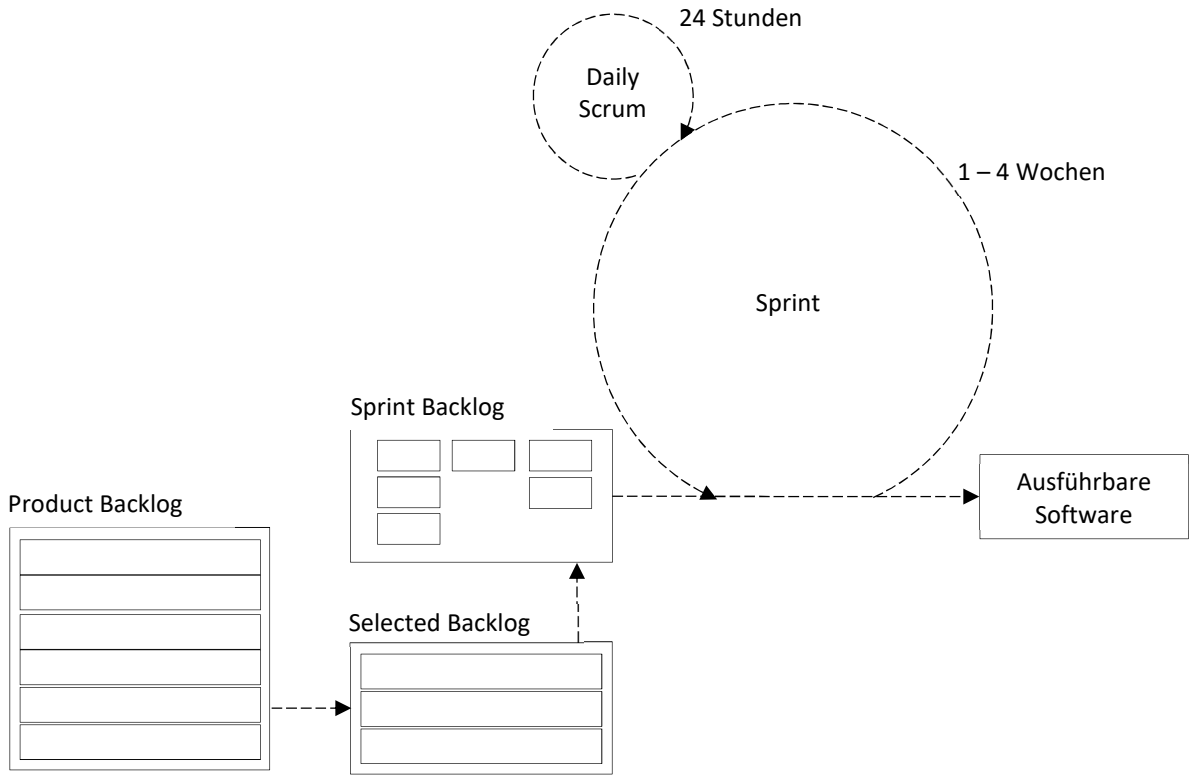


Abbildung 5: Scrum im Überblick (nach [Wird22])

Diese Vorgehensweise ermöglicht eine bessere Anpassung an geänderte Anforderungen, u. a. durch Veränderung der Anforderungen im Product-Backlog. Außerdem steht nach jedem Sprint eine funktionsfähige Software zur Verfügung. Der Kunde kann diese direkt ausprobieren und feststellen, ob die Software seinen Vorstellungen entspricht. Wenn nicht, kann sie im nächsten Sprint überarbeitet werden. Durch die kurzen Iterationszyklen ist der Aufwand hierfür wesentlich geringer, als wenn man erst am Ende feststellt, dass die Wünsche des Kunden nicht so gut getroffen sind.

Trotz der Popularität agiler Vorgehensmodelle sind plangetriebene Vorgehensmodelle keineswegs überflüssig geworden. Je nach Projekt-Typ eignen sich eher plangetriebene oder

agile Verfahren. Häufig werden auch Elemente unterschiedlicher Vorgehensweisen miteinander kombiniert.

Die Gliederung der Veranstaltung „Software-Engineering" und damit auch des vorliegenden Skripts orientiert sich im Wesentlichen an den Phasen eines plangetriebenen, wasserfallartigen Vorgehens. Die vermittelten Inhalte sind jedoch genauso für agile Vorgehensweisen relevant. Man sollte sich daher bewusst sein, dass die vorgestellten Methoden und Verfahren in einem konkreten Projekt nicht zwangsläufig strikt nacheinander angewendet werden müssen, sondern ggf. auch parallel eingesetzt oder mehrfach wiederholt werden können.

Eine ausführlichere Diskussion von Vorgehensmodellen für die Software-Entwicklung erfolgt in der folgenden Lehrveranstaltung „Projektmanagement". Dort werden auch weitere wichtige Aspekte für die Planung und Steuerung von Projekten zur Entwicklung von Software besprochen, wie z. B. die Schätzung des benötigten Aufwandes, die zeitliche Planung oder die Überwachung des Projektfortschritts.

1.7 Braucht man im KI-Zeitalter noch Software-Engineering?

Als die erste Version dieses Skripts geschrieben wurde, gab es noch keine Large-Language-Models, wie sie heute zur Verfügung stehen und in großem Umfang für die Software-Entwicklung eingesetzt werden. Doch auch wenn die zitierten Beispiele immer komplexerer Software und fehlgeschlagener Projekte aus der Vor-KI-Ära stammen, lassen sich die entsprechenden Tendenzen nach wie vor beobachten. Auch wenn Künstliche Intelligenz heute ganz erstaunliche Möglichkeiten bietet, geraten nach wie vor viele IT-Projekte in Schwierigkeiten.

Seit es KI-Systeme gibt, die programmieren können, wird diskutiert, wie sich die Rolle und das Aufgabenspektrum von Informatikerinnen und Informatikern verändert. Dabei gilt generell, dass man als IT-Expertin oder -Experte erst einmal selbst die Grundlagen verstehen muss, bevor man eine Künstliche Intelligenz mit Aufgaben der Software-Entwicklung betraut. Hierdurch unterscheiden sich Informatiker von reinen Vibe-Codern, die keine Ahnung davon haben, was die KI unter der Haube tut, und deswegen bei anspruchsvolleren Entwicklungen früher oder später an ihre Grenzen kommen.

Schon aus diesem Grund ist es wichtig, die Grundlagen der Software-Entwicklung zu beherrschen, zu denen auch die Methoden des Software-Engineering gehören.

Und auch wenn man in der Praxis immer größere Anteile der Entwicklung mit Hilfe von KI automatisisert, so tragen die in diesem Skript beschriebenen Methoden des Software-Engineering wesentlich dazu bei, dass man in der Lage ist, mit der KI so zu kommunizieren, dass am Ende auch ein sinnvolles und gewünschtes Ergebnis herauskommt. Hierfür muss man beispielsweise wissen, wie Anforderungen an ein Software-System dokumentiert werden. Ebenso sollte man die fachlichen Zusammenhänge sowie Abläufe und Regeln analysieren und beschreiben können, die von der Software unterstützt werden müssen. Und diese

Beschreibungen sollten sowohl von Menschen verstanden als auch von KI-Systemen sinnvoll verarbeitet werden können.

2 Anforderungen

Im einleitenden Kapitel war bereits mehrfach die Rede von der Ermittlung und Analyse von Anforderungen. Auftraggeber und künftige Benutzer wollen mit der zu entwickelnden Software bestimmte Ziele erreichen, bestimmte Aufgaben durchführen, und sie haben verschiedene Wünsche, wie das System beschaffen sein soll.

Die Ausführungen in diesem Kapitel orientieren sich im Wesentlichen an [Eber22].

Unter einer Anforderung versteht man:

- Eine Eigenschaft der Software, die ein Benutzer benötigt, um ein Problem zu lösen oder ein Ziel zu erreichen.
- Eine Eigenschaft der Software, die benötigt wird um einen Vertrag, eine Norm, ein Gesetz, eine Spezifikation o.ä. zu erfüllen.

Anforderungen sollten möglichst dokumentiert werden.

2.1 Problemraum und Lösungsraum

Ein nützliches Prinzip beim Umgang mit Anforderungen ist die strikte Trennung zwischen Problem und Lösung. Man spricht auch von Anforderungen aus dem „Problemraum" und dem „Lösungsraum" [Eber22]:

- Problemraum:
  Was will bzw. braucht der Kunde?
- Lösungsraum:
  Wie sollen die Kundenbedürfnisse durch das Produkt erfüllt werden?

Diese Unterscheidung ist wichtig, denn konkrete Anforderungen des Lösungsraums dürfen bei Bedarf geändert werden, Anforderungen des Problemraums hingegen nicht – bzw. nur vom Kunden seöbst.

Zum Beispiel könnte der Kunde die Anforderung haben, dass die Daten einer Anwendung über ein Smartphone abrufbar sind. Daraufhin wird entschieden, eine für kleine Bildschirme optimierte Webseite zu erstellen. Stellt sich im Laufe des Projektes heraus, dass es besser wäre, eine eigene Smartphone-App zu entwickeln, so ist dies ohne weiteres möglich.

Verwischt man bei der Dokumentation der Anforderungen den Unterschied zwischen Problem und Lösung, so ist später nicht mehr klar, ob es sich bei der Forderung nach einer mobilen Website um eine Kundenanforderung handelt, oder ob es eine abgeleitete Anforderung ist, die auch geändert werden kann. Denn dann ist es zulässig, die mobile Website durch eine App zu ersetzen.

Anforderungen des Problemraums stammen vom Markt, d. h. dem auftraggebenden Kunden oder – bei einer Standardsoftware – allen potenziellen Kunden. Entsprechend kann man sie

als Marktanforderungen oder Kundenanforderungen bezeichnen. Sie beantworten die Frage, warum das Produkt bestimmte Eigenschaften aufweisen soll.

Die Anforderungen des Lösungsraums lassen sich unterteilen in Produktanforderungen, die sich auf das gesamte Softwareprodukt beziehen, und Komponentenanforderungen, die einzelne Produktkomponenten betreffen. Produktanforderungen beantworten die Frage, was umgesetzt werden soll, und die Komponentenanforderungen beschreiben, wie die Produktanforderungen realisiert werden sollen.

Abbildung 6 stellt den verschiedenen Typen von Anforderungen Beispiele gegenüber, die sich aufeinander beziehen.

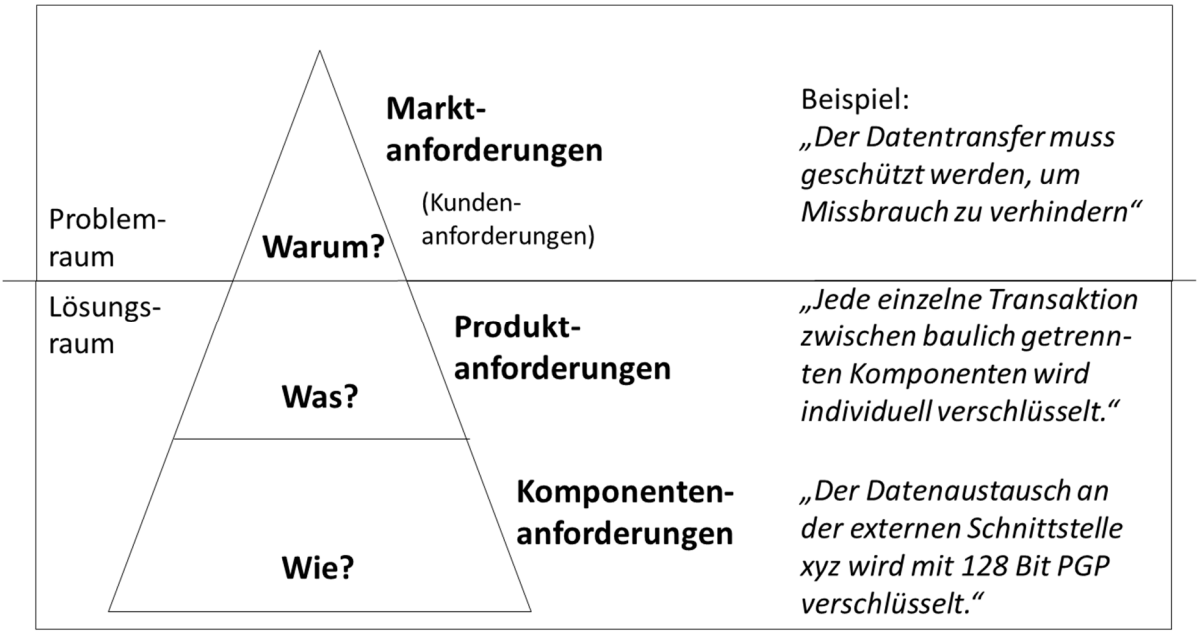


Abbildung 6: Problem- und Lösungsraum (nach [Eber22])

Dabei kann die Zuordnung einer Anforderung zu einer dieser Kategorien je nach Fall unterschiedlich sein. So wurde die Anforderung nach einer 128 Bit PGP-Verschlüsselung für den Datenaustausch in Abbildung 6 als Komponentenanforderung eingestuft. Nun könnte es aber sein, dass der Kunde explizit genau diese Art der Verschlüsselung gefordert hat. In diesem Fall würde es sich bei derselben Anforderung um eine Marktanforderung handeln (die hier zugleich auch eine Komponentenanforderung ist).

Am Rande sei bemerkt, dass es in einem solchen Fall durchaus sinnvoll ist, beim Kunden genau nachzufragen, ob und warum es genau diese spezielle Verschlüsselung sein muss. Möglicherweise hat auch der Kunde das eigentliche Problem und eine mögliche Lösungsvariante miteinander vermischt, was die Möglichkeit für alternative – eventuell sogar besser geeignete – Lösungsvarianten verbaut.

2.2 Arten von Anforderungen

Man kann drei grundsätzliche Arten von Anforderungen unterscheiden (vgl. Abbildung 7):

1. Funktionale Anforderungen
2. Qualitätsanforderungen
3. Randbedingungen

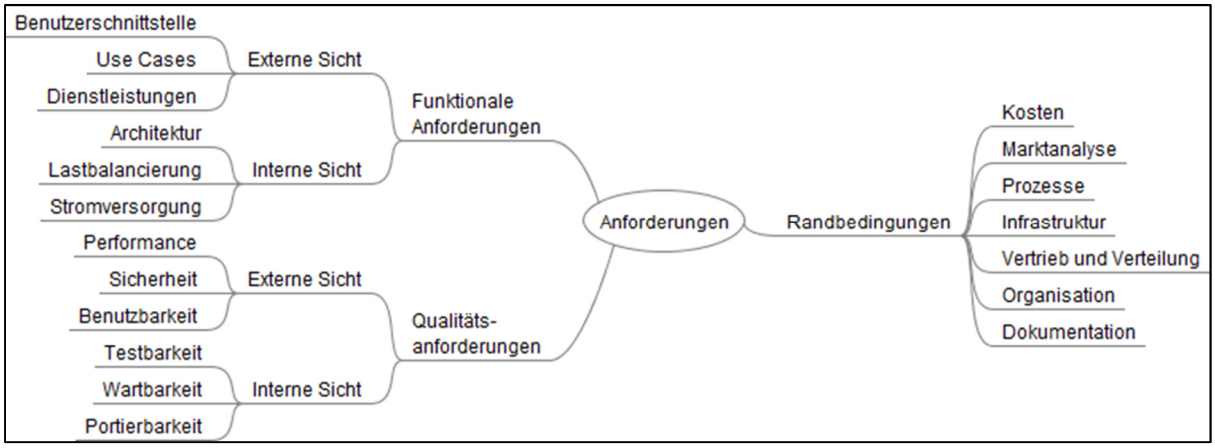


Abbildung 7: Arten von Anforderungen (nach [Eber22])

2.2.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche Funktionen ein System bereitstellen soll, d. h. sie beschreiben, *was* das System tun soll.

Beispiele für funktionale Anforderungen:

> *„Der Benutzer kann ein Bild hochladen"*
>
> *„Der Datenaustausch an der externen Schnittstelle xyz wird mit 128 Bit PGP verschlüsselt."*

Zur Beschreibung von funktionalen Anforderungen werden u. a. Beschreibungen von Szenarien, Benutzerschnittstellen, Ablaufbeschreibungen, Use-Cases oder User Stories eingesetzt.

Use-Cases (Anwendungsfälle) und User Stories dienen beide der Beschreibung des Systemverhaltens aus Anwendersicht, wobei Use-Cases etwas umfangreicher sind. Sie werden im folgenden Kapitel behandelt. Die knapperen User Stories werden vor allem in agilen Vorgehensmodellen eingesetzt. Sie sind Gegenstand der Lehrveranstaltung Projektmanagement.

2.2.2 Qualitätsanforderungen (nicht-funktionale Anforderungen)

Qualitätsanforderungen ergänzen die funktionalen Anforderungen um qualitative Eigenschaften, wie z. B. Geschwindigkeit, Sicherheit oder Zuverlässigkeit.

Beispiel einer Qualitätsanforderung:

> *„Die Verschlüsselung und Entschlüsselung einer Transaktion muss innerhalb von 1 Millisekunde abgeschlossen sein."*

In Abgrenzung zu den funktionalen Anforderungen werden Qualitätsanforderungen häufig auch explizit als „nicht-funktionale“ Anforderungen bezeichnet.

Während die Funktionalität für jedes System völlig anders sein kann, gibt es für Qualitätsanforderungen typische Kategorien, die bei vielen Systemen eine Rolle spielen. Die internationale Norm ISO/IEC 25010 definiert ein Softwarequalitätsmodell mit insgesamt acht Kategorien von Qualitätsmerkmalen, die jeweils mehrere Unterkategorien enthalten (Abbildung 8). Dieses Modell kann als Grundlage zum Aufstellen von Qualitätsanforderungen für ein konkretes Softwareprodukt dienen.

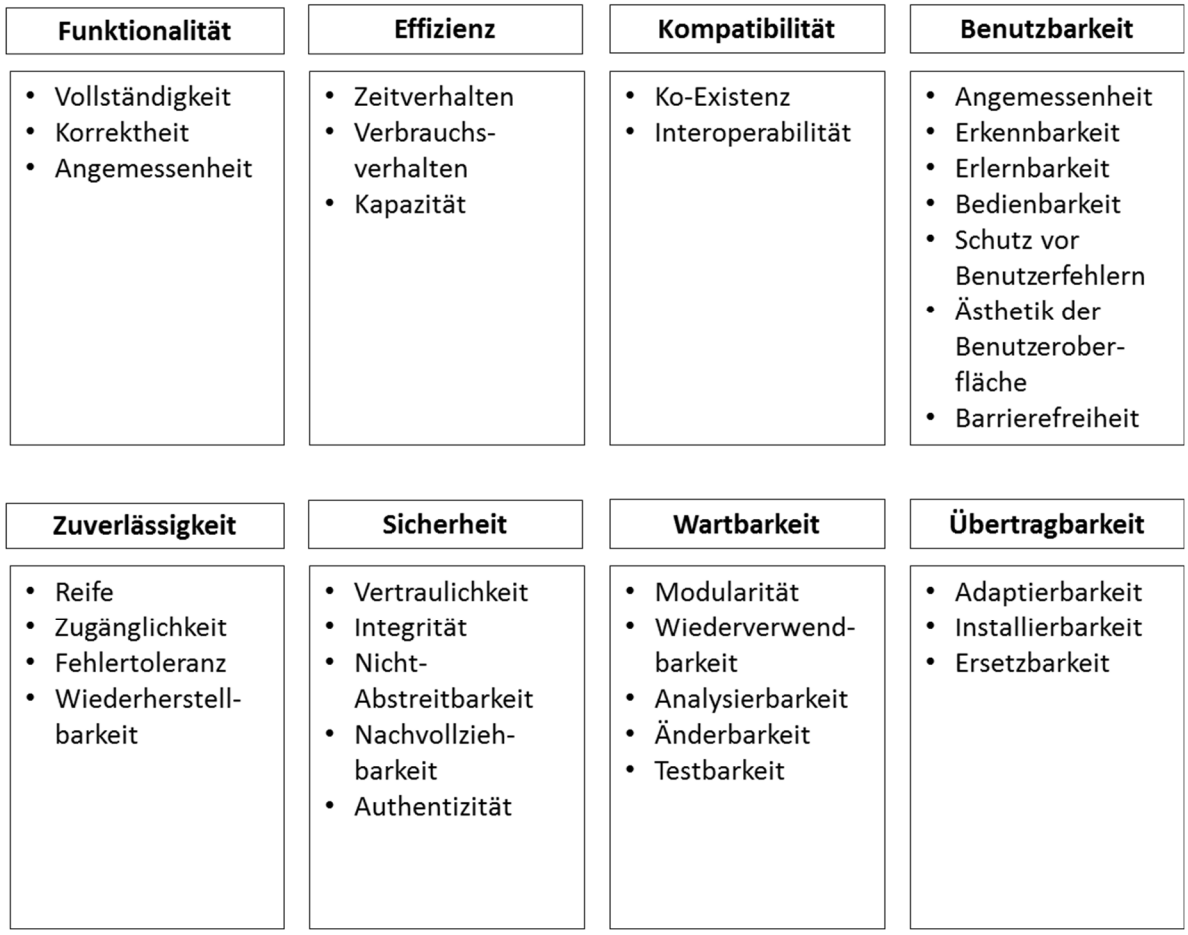


Abbildung 8: Qualitätsmerkmale gemäß ISO 25010 (nach [Tiem18, übersetzt)

Funktionale Anforderungen sind vergleichsweise einfach zu spezifizieren und zu testen. Man muss einfach beschreiben was das System tun soll. Am fertigen System kann man dann ausprobieren, ob es das auch tut.

Im Gegensatz dazu sind Qualitätsanforderungen wesentlich schwerer zu spezifizieren und zu testen. Bei der Forderung nach einem bestimmten Zeitverhalten ist das noch vergleichsweise einfach. Die oben genannten Anforderungen nach Ver- und Entschlüsselung innerhalb einer Millisekunde war leicht zu formulieren, und eine Zeitmessung lässt sich auch ganz gut durchführen.

Doch hängt das Zeitverhalten beispielsweise auch von der Auslastung des Systems ab. Streng genommen müsste man also auch definieren, bei welcher Auslastung die Zeit eingehalten werden muss. Und wie definiert man die Auslastung genau?

Noch schwieriger wird es bei anderen der in Abbildung 8 genannten Kriterien. Wie spezifiziert und überprüft man beispielsweise, wie gut die Änderbarkeit eines Systems sein muss?

Die Realisierung vieler Qualitätsanforderungen ist aufwändig und zieht sich oft quer durch alle Komponenten. Es gibt starke Wechselwirkungen mit der System-Architektur und umgebenden Systemen. Schließlich sind verschiedene Qualitätsanforderungen voneinander abhängig, und auch zu den funktionalen Anforderungen können Abhängigkeiten bestehen.

Qualitätsanforderungen sind also verhältnismäßig schwierig aufzustellen und umzusetzen. Daher werden sie in der Praxis oft nicht oder nur sehr vage spezifiziert. In der Folge werden sie zu Beginn nicht ausreichend berücksichtigt. Die später erforderlichen Korrekturen sind dann sehr aufwändig.

Daher sollten Qualitätsanforderungen trotz der genannten Schwierigkeiten auf keinen Fall vernachlässigt werden. Eine Möglichkeit zur Formulierung dieser Anforderungen sind Qualitätsszenarien. Sie werden in Abschnitt 2.6 beschrieben.

2.2.3 Randbedingungen

Die dritte Art von Anforderungen bilden die verschiedenen Arten von Randbedingungen. Sie schränken die Art und Weise ein, wie das System realisiert wird. Hierbei kann es sich etwa um Kosten, die Geschäftsprozesse des Kunden, einzuhaltende Gesetze o. ä. handeln.

Beispiel für eine Randbedingung:

> „*Die Verschlüsselung einer Transaktion muss den gesetzlichen Anforderungen des Bundesamts für Sicherheit in der Informationstechnik (BSI) genügen.*"

2.3 Erhebung und Management von Anforderungen

Warum ist es wichtig, sich intensiv mit der Erhebung, der Analyse und dem Management von Anforderungen zu beschäftigen? Tut man es nicht, kann es teuer werden.

Macht man einen Fehler bei der Anforderungsermittlung, und wird dieser Fehler während des gesamten Entwicklungsprojekts nicht entdeckt, so gelangt der Fehler in die ausgelieferte Software. Wird er dann erst während des Betriebs entdeckt, so ist der Aufwand zu seiner Behebung sehr groß, da ja sämtliche Dokumente und der Quellcode überarbeitet werden müssen und die Software erneut getestet werden muss.

Wird der Fehler bereits sehr früh entdeckt, sind die Kosten zu seiner Behebung wesentlich niedriger, da ja noch kein Code geschrieben wurde und evtl. nur einige Änderungen in einem Anforderungsdokument durchgeführt werden müssen.

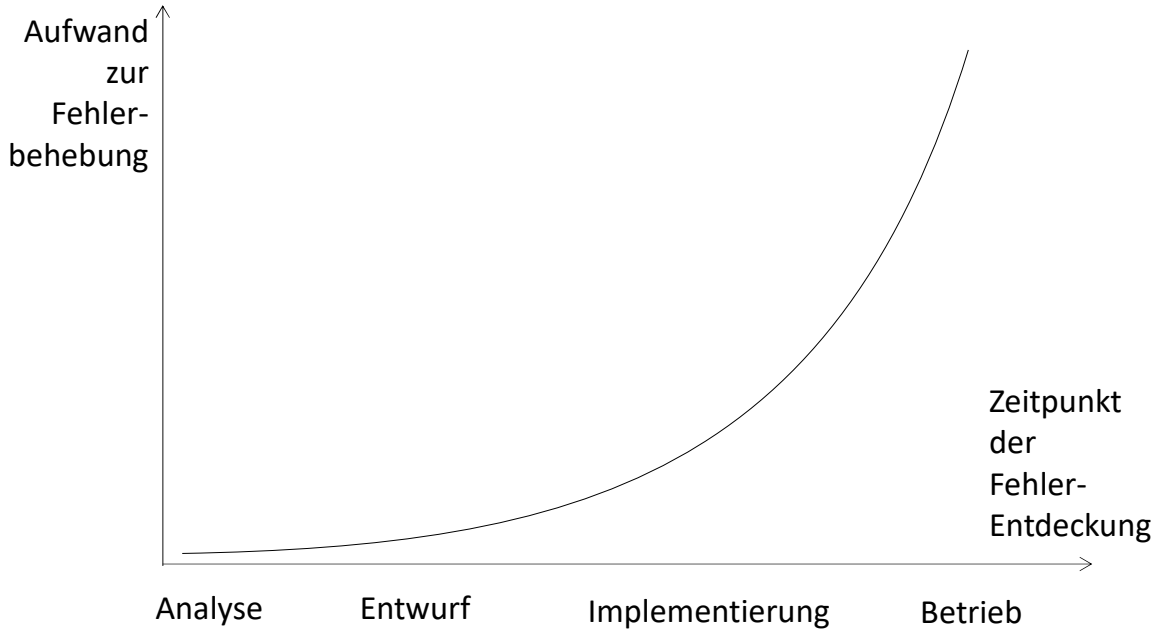


Abbildung 9: Aufwand einer Fehlerbehebung in Abhängigkeit vom Zeitpunkt der Entdeckung

Abbildung 9 zeigt den typischen Verlauf des Aufwandes zur Behebung eines Fehlers in Abhängigkeit von dem Zeitpunkt, zu dem er entdeckt wird. Es lohnt sich also, durch eine genaue Anforderungsanalyse möglichst viele Fehler möglichst früh zu entdecken und somit die Fehlerbehebungskosten möglichst niedrig zu halten.

Der in Abbildung 9 gezeigte Verlauf gilt in dieser Form vor allem für Projekte mit einem wasserfallartigen Vorgehen. Agile Projekte mit ihren kurzen Iterationszyklen haben den Vorteil, dass man schon frühzeitig eine laufende Software hat, die von Anwendern getestet werden kann. Fehler bei der Ermittlung von Anforderungen werden somit meist auch früher gefunden. Doch auch bei einer agilen Vorgehensweise spielt der angemessene Umgang mit Anforderungen eine wichtige Rolle. Auch in einer zwei- oder dreiwöchigen Iteration sollten die Anforderungen möglichst korrekt umgesetzt werden, so dass in der folgenden Iteration nicht zu viele Überarbeitungen nötig sind.

Die typischen Aufgabengebiete des Anforderungsmanagements (engl. Requirements Engineering) werden in Abbildung 10 gezeigt. Zur **Ermittlung** der Anforderungen kommen verschiedene Verfahren zum Einsatz, wie z. B. Interviews, Workshops oder die Analyse relevanter Dokumente. Die erhobenen Anforderungen müssen genauer **analysiert** werden, z. B. hinsichtlich ihrer Zusammenhänge und Abhängigkeiten, Details, abzuleitenden Produkt- und Komponentenanforderungen usw. Zur **Dokumentation** kommen verschiedene Arten von Dokumenten zum Einsatz, wie z. B. Lasten- und Pflichtenheft. Diese werden weiter unten beschrieben. Verschiedene Aspekte können zudem mit Hilfe grafischer Modelle dargestellt werden, z. B. den in späteren Kapiteln behandelten Use-Case-Diagrammen oder Klassendiagrammen.

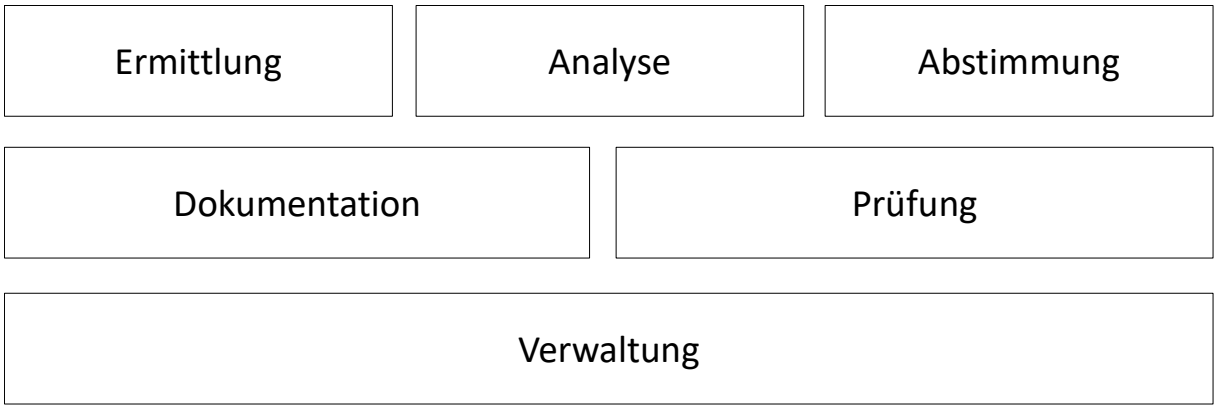


Abbildung 10: Aktivitäten des Anforderungsmanagements (nach [Eber22])

Schließlich werden die dokumentierten Anforderungen mit dem Kunden und im Team **abgestimmt**, um sicherzustellen, dass ein einheitliches Verständnis besteht und die Anforderungen in einer sinnvollen Reihenfolge implementiert werden. Vor allem bei größeren Projekten stellt die **Verwaltung** der Vielzahl von Anforderungen, ihren Abhängigkeiten und Änderungen eine Herausforderung dar.

2.4 Dokumente zur Anforderungsspezifikation

Anforderungen werden häufig in sogenannten Lastenheften und Pflichtenheften dokumentiert. Während das Lastenheft vor allem die Markt- oder Kundenanforderungen enthält und damit die Inhalte des Problemraums beschreibt, stammen die Inhalte des Pflichtenhefts oder Fachkonzepts aus dem Lösungsraum und beschreiben, wie die Anforderungen aus dem Lastenheft umgesetzt werden sollen. Im Pflichtenheft finden sich daher die Produktanforderungen und Teile der Komponentenanforderungen.

Die Trennung von Lasten- und Pflichtenheft spiegelt das oben angesprochene Prinzip der konsequenten Trennung zwischen Problem und Lösung wider.

In vielen Fällen wird das Lastenheft vom Auftraggeber erstellt. Es kann als Leistungsbeschreibung in den Vertrag zwischen Kunde und Softwareentwicklungsfirma aufgenommen werden. Das Pflichtenheft hingegen wird oftmals vom Auftragnehmer erstellt. Er beschreibt darin, wie er die Anforderungen des Pflichtenhefts umsetzen möchte. Abbildung 11 stellt die Abgrenzung der beiden Arten von Anforderungsdokumenten noch einmal im Überblick dar.

Im englischsprachigen Raum werden die beiden Arten von Dokumenten meist in einem Dokument zusammengefasst, der Anforderungsspezifikation (Requirements Specification). Eine mögliche Gliederung enthält der IEEE-Standard 830 (Tabelle 1).

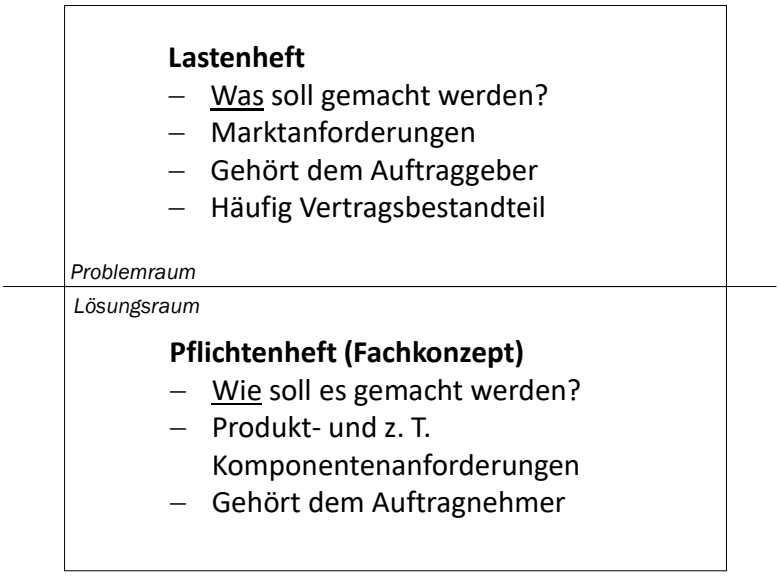


Abbildung 11: Lasten- und Pflichtenheft (nach [Eber22])

Kapitel	*Inhalt*
	Titel, Autor, Empfänger, Identifizierung, Änderungshistorie, Inhaltsverzeichnis
1	**Einführung**
1.1	Zweck (Darstellung der Vision)
1.2	Markt (z. B. Marktsegmente, Bedarf, Design, Alleinstellungsmerkmale)
1.3	Glossar (Definitionen, Abkürzungen)
1.4	Referenzen
1.5	Systemübersicht (System- und Kontextabgrenzung)
2	**Beschreibung**
2.1	Produktsicht (Systemschnittstellen, Benutzung, Hard- und Software-schnittstellen)
2.2	Funktionen (z. B. als Use-Cases)
2.3	Benutzer (z. B. verschiedene Benutzergruppen, Profile, Szenarien)
2.4	Einschränkungen (externe Vorgaben, z. B. Protokolle, Hardware Algorithmen)
2.5	Qualitätsanforderungen (externe Vorgaben, z. B. Zuverlässigkeit, Sicherheit)
2.6	Annahmen (nicht durch den Kunden definiert)
3	**Spezifische Anforderungen**
3.1	Funktionale Anforderungen (z. B. Gültigkeitsprüfungen, Abnahmekriterien)
3.2	Architektur (z. B. Datenmodelle, Systemmodelle)
3.3	Einschränkungen (intern und extern, messbar, Abnahmekriterien)
3.4	Qualitätsanforderungen (intern und extern, messbar, Abnahmekriterien)
3.5	Standards
	Anhänge, Index

Tabelle 1: Gliederung einer Anforderungsspezifikation gemäß IEEE-Standard 870 (nach [Eber22])

Beispiele für Lasten- und Pflichtenhefte finden sich u. a. in [Balz09] Kapitel III.7 (Fallstudie Seminarorganisation), Kapitel III.8 (Fallstudie Fensterheber) und [Vect23] (Lastenheft Aufzug).

Die genannten Dokumenttypen kommen hauptsächlich in klassischen Vorgehensmodellen zum Einsatz. Agile Ansätze verwenden eher Karteikarten zum Notieren der Anforderungen, die dann an Wandtafeln geheftet werden. Diese Karteikarten lassen sich leicht umsortieren, ergänzen und gemäß Bearbeitungsfortschritt umhängen (vgl. Abschnitt 1.6.2 auf S. 15).

2.5 Nachverfolgbarkeit von Anforderungen

Wird eine Anforderung ins Lastenheft aufgenommen, so schlägt sich diese Anforderung in den Dokumenten und Produkten nieder, die im weiteren Projektverlauf erstellt werden. So können z. B. aus einer Marktanforderung im Lastenheft ein oder mehrere Produkt- oder Komponentenanforderungen im Pflichtenheft entstehen. Diese führen dazu, dass im technisch ausgerichteten Systementwurfsdokument bestimmte Festlegungen getroffen werden und dass schließlich der erforderliche Programmcode geschrieben wird.

Jede Anforderung hinterlässt also ihre Spuren im Projekt (vgl. Abbildung 12). Was passiert nun, wenn sich eine Anforderung ändert, nachdem sie schon in anderen Dokumenten und vielleicht auch schon im Code umgesetzt wurde? Damit die Änderungen auch richtig umgesetzt werden, müssen alle Stellen gefunden werden, die auf die zu ändernde Anforderung zurückzuführen sind.

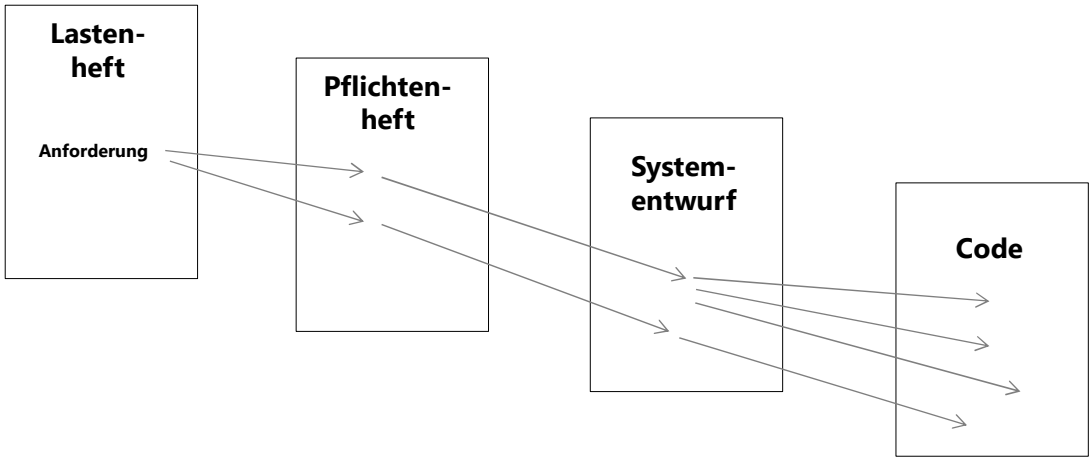


Abbildung 12: Nachverfolgbarkeit von Anforderungen über verschiedene Produkte hinweg

Man kann sich vorstellen, dass dies – insbesondere in größeren Projekten – sehr mühsam und aufwändig ist. Daher gibt es verschiedene Ansätze, die zu einer besseren Nachverfolgbarkeit (engl. Traceability) von Anforderungen beitragen.

Hilfreich sind zum einen geeignete, einheitliche Benennungen und Nummerierungen von Anforderungen. Werden in den verschiedenen Dokumenten die Anforderungen nach einem einheitlichen Schema nummeriert, dann lassen sich darüber die voneinander abhängigen Inhalte finden.

Bei komplizierteren Abhängigkeiten – wenn z. B. eine Anforderung mehrere Auswirkungen an verschiedenen Stellen in einem anderen Dokument hat, gerät man mit der reinen Zuordnung mittels Nummern und Bezeichnungen an Grenzen. In diesem Fall kann man beispielsweise in

jedem Abschnitt explizite Verweise auf die zugrunde liegenden Anforderungen eines vorherigen Dokuments einfügen.

Teilweise werden solche Abhängigkeiten auch in Form einer Abhängigkeitsmatrix dargestellt. Dabei werden die Anforderungen des einen Dokuments in den Zeilen und die abhängigen Anforderungen des zweiten Dokuments in den Spalten eingetragen. In den Feldern der Matrix wird jeweils ein x eingetragen, wenn eine Abhängigkeit zwischen den Anforderungen der jeweiligen Zeile und Spalte besteht (Tabelle 2).

	Anforderung B1	Anforderung B2	Anforderung B3	Anforderung B4	Anforderung B5	…
Anforderung A1		x		x		
Anforderung A2	x				x	
Anforderung A3					x	
Anforderung A4			x			
…						

Tabelle 2: Abhängigkeitsmatrix

Ein Nachteil von expliziten Verweisen in den Dokumenten und Abhängigkeitsmatrizen ist der Pflegeaufwand. Bei sämtlichen Änderungen müssen die Verweise bzw. die Einträge in den Matrizen aktualisiert werden. Dies wird leicht vergessen, worunter die Nachverfolgbarkeit leidet.

In großen Projekten mit umfangreichen und komplexen Anforderungen werden daher zum Teil spezielle Requirements-Management-Tools eingesetzt. Die Anforderungen werden dann nicht mehr in gewöhnlichen Dokumenten abgelegt, sondern als Einträge in einer Datenbank. Meist können diese Tools auch die Abhängigkeiten verwalten und beispielsweise bei einer Änderung herausfinden, welche anderen Anforderungen davon betroffen sind.

2.6 Qualitätsszenarien

Es wurde bereits erläutert, dass insbesondere viele Arten von Qualitätsanforderungen schwierig zu spezifizieren und zu überprüfen sind. In diesem Abschnitt wird ein in [Zörn21] beschriebener Ansatz zur Beschreibung von Qualitätsanforderungen mit Hilfe von Szenarien vorgestellt.

Der Ansatz wird anhand des Beispiels einer Schach-Software demonstriert. Die Dokumentation dieses Systems mit den hier vorgestellten Qualitätsszenarien findet sich im Internet [Zörn25].

Die Qualitätsanforderungen für dieses System werden zunächst in Form von Qualitätszielen formuliert (Tabelle 3).

Qualitätsmerkmal	*Motivation und Erläuterung*
Zugängliches Beispiel (Analysierbarkeit)	Da DokChess in erster Linie als Anschauungsmaterial für Architekten und Entwickler dient, erschließen sich Entwurf und Implementierung rasch.
Einladende Experimentierplattform (Änderbarkeit)	Alternative Algorithmen und Strategien, etwa zur Bewertung einer Schachstellung, können leicht implementiert und in die Lösung integriert werden.
Bestehende Frontends nutzen (Interoperabilität)	DokChess lässt sich mit angemessenem Aufwand in bestehende grafische Schach-Frontends einbinden.
Akzeptable Spielstärke (Attraktivität)	DokChess spielt stark genug, um schwache Gegner sicher zu schlagen und Gelegenheitsspieler zumindest zu fordern.
Schnelles Antworten auf Züge (Effizienz)	Da DokChess in Seminaren und Vorträgen live demonstriert wird, erfolgt die Berechnung der Spielzüge rasch.

Tabelle 3: Qualitätsziele für eine Schach-Software (Quelle: [Zörn25]).

Die Formulierungen der Qualitätsanforderungen sind noch recht allgemein gehalten und lassen sich schwer überprüfen. Was bedeutet es genau, dass alternative Algorithmen „leicht implementiert" werden können, oder dass man das System „mit angemessenem Aufwand" in bestehende Frontends einbinden kann?

Da es schwierig ist, genaue und allgemeingültige Kriterien für derartige Ziele zu formulieren, kann man sich konkrete, realistische Szenarien überlegen, an denen deutlich wird, was die Erfüllung dieser Ziele im Einzelfall bedeuten kann. Damit bekommt man zwar keine eindeutigen Maßstäbe, mit denen man die Erfüllung eines bestimmten Kriteriums für alle möglichen Fälle bewerten kann. Doch erhält man ganz gute Anhaltspunkte für eine Einschätzung.

Beim Aufstellen geeigneter Szenarien wird man dazu gezwungen, nachzudenken, was ein bestimmtes Qualitätsmerkmal in einem bestimmten Fall für konkrete Konsequenzen hätte. So muss eine gute Änderbarkeit nicht bedeuten, dass jede beliebige Art von Änderung möglich sein soll, sondern vor allem sinnvolle und wahrscheinliche Änderungen. Die Qualitätsszenarien sind dann auch am fertigen System überprüfbar.

Tabelle 4 enthält einige Qualitätsszenarien, die sich z. T. auf die o. g. Qualitätsziele beziehen. Die „leichte Implementierbarkeit" alternativer Algorithmen wird beispielsweise in Form der Szenarien Nr. 2 und 3 konkretisiert: So soll es etwa möglich sein, eine neue Stellungsbewertung ohne Änderung und Neu-Compilierung vorhandenden Codes einzufügen. Dies gibt den Entwicklern klare Hinweise, was sie bzgl. Änderbarkeit tun müssen. Und es kann anschließend festgestellt werden, ob das Szenario so wie gefordert funktioniert.

Nr	*Szenario*	*Qualitätsmerkmal*
1	Ein Interessierter mit Grundkenntnissen in UML und Schach möchte einen Einstieg in die Architektur von DokChess finden. Lösungsstrategie und Entwurf erschließen sich ihm innerhalb von 15 Minuten.	Analysierbarkeit
2	Ein Entwickler implementiert eine neue Stellungsbewertung. Er kann sie ohne Änderung und ohne Übersetzung vorhandenen Codes in bestehende Strategien integrieren.	Änderbarkeit
3	Ein Entwickler implementiert eine figurenzentrierte Bitboard-Repräsentation der Spielsituation. Der Aufwand dazu beträgt – inklusive des Austauschs der bestehenden, feldzentrierten Darstellung durch die neue – maximal eine Woche.	Änderbarkeit
4	Ein schwacher Spieler zieht in einer Partie gegen die Engine eine Figur ungedeckt und frei von Sinn auf ein von der Engine angegriffenes Feld. Die Engine im Anschluss am Zug nimmt die "eingestellte" Figur.	Attraktivität
5	In einer Partie ergibt sich für die Engine ein Matt in 2 Zügen. Die Engine zieht sicher zum Sieg.	Attraktivität
6	Während einer Partie antwortet die Engine auf gegnerische Züge innerhalb von 5 Sekunden mit einem Zug.	Effizienz
7	Ein Benutzer will DokChess mit einem Schachfrontend verwenden, das ein von der Lösung implementiertes Kommunikationsprotokoll unterstützt. Das Einbinden erfordert keinerlei Programmieraufwand, die Konfiguration innerhalb des Frontends ist innerhalb von 10 Minuten durchgeführt und getestet.	Interoperabilität
8	Ein Java-Programmierer will DokChess mit einem Schachfrontend verwenden, das das Einbinden von Engines erlaubt, aber keines der implementierten Protokolle unterstützt. Das neue Protokoll kann ohne Änderung am bestehenden Code implementiert und die Engine anschließend wie gewohnt eingebunden werden.	Interoperabilität
9	Der Engine wird im Spielverlauf ein unzulässiger Gegenzug präsentiert. Die Engine lehnt den Zug ab, erlaubt im Anschluss die Eingabe eines anderen Zuges und spielt fehlerfrei weiter.	Effizienz

Tabelle 4: Auswahl einiger Qualitätsszenarien für eine Schach-Software (Quelle: [Zörn25]).

3 UML-Modellierungswerkzeuge

Im vorliegenden Kurs werden viele Sachverhalte grafisch mit Hilfe von UML dargestellt. „UML" steht für Unified Modeling Language, einem Standard für die Modellierung von Software-Systemen, insbesondere im Zusammenhang mit objektorientierter Programmierung [OMG17]. Dieser Standard definiert eine Reihe von Diagrammtypen zur Darstellung unterschiedlicher Aspekte einer Software, wie z. B. die Benutzersicht, die statische Struktur oder dynamische Abläufe. Die wichtigsten Diagrammtypen werden in diesem Skript beschrieben.

Es gibt verschiedene Arten von Werkzeugen, mit denen man UML-Diagramme erstellen kann.

1. **Handzeichnungen.** Möchte man einen Sachverhalt kurz skizzieren und mit Kollegen diskutieren, genügt oft eine kleine Skizze an einer Tafel oder auf Papier.
2. **Grafikwerkzeuge.** Sie sind am flexibelsten und können für unterschiedliche Arten von Diagrammen eingesetzt werden. Manche haben auch eine Symbolbibliothek für UML-Diagramme. Andererseits bieten sie meist keine spezielle UML-Unterstützung. So wird etwa nicht verhindert, dass man gegen die Regeln der UML verstößt. Ein Beispiel eines kostenlosen Tools, das UML-Symbole enthält, ist draw.io, das man sowohl online im Browser benutzen kann, als auch in einer Desktop-App.
3. **UML-Modellierungswerkzeuge.** Diese Tools sind speziell auf die UML-Modellierung ausgelegt und bieten hierfür zahlreiche Unterstützungsfunktionen. Ein Beispiel ist „Visual Paradigm", dessen Community-Edition für nicht-kommerzielle Zwecke kostenlos genutzt werden darf (empfohlen wird die Desktop-Version, nicht die Online-Version).
   Hierbei ist zu beachten, dass die in Kapitel 5 für die Modellierung von Abläufen verwendete Notation „BPMN" nicht zur UML gehört und daher in speziellen UML-Modellierungswerkzeugen oft nicht verfügbar ist. Doch auch hierfür gibt es spezielle Tools, die zum Teil frei verfügbar sind, wie z. B. der „Bizagi Modeler".
4. **Text-to-Model-Werkzeuge.** Anstatt die Diagramme zu zeichnen, gibt es auch die Möglichkeit, sie in Form von Text gemäß einer bestimmten Syntax zu definieren. Daraus wird dann das grafische Diagramm generiert. Wenn man sich einmal mit der – relativ einfachen – Sprache vertraut gemacht hat, geht das Modellieren recht schnell. Zudem handelt es sich bei den erstellten Dateien um reine Textdateien, die gut zusammen mit dem Code in eine Versionsverwaltung eingestellt werden können.
   Zu dieser Klasse von UML-Werkzeugen gehört „PlantUML", das ebenfalls kostenlos verwendet werden kann.
   Die Ablaufprogrammierung mit BPMN in Kapitel 5 wird von PlantUML und ähnlichen Tools allerdings nicht unterstützt.

4 Use-Cases und Use-Case-Diagramme

In diesem Kapitel erlernen Sie die Anwendung von Use-Cases und UML Use-Case-Diagrammen zur Beschreibung eines Systems aus Sicht des Benutzers. Sie lassen sich insbesondere für die Dokumentation von funktionalen Anforderungen einsetzen.

4.1 Einführendes Beispiel

Use-Cases und UML Use-Case-Diagramme beschreiben die Funktionalität eines Software-Systems aus Benutzersicht. Sie beantworten die Frage „Wer macht was mit dem System?". Die deutsche Übersetzung von „Use-Case" ist „Anwendungsfall". Allerdings wird auch im Deutschen meist der Originalbegriff „Use-Case" benutzt, weshalb auch in diesem Skript der englische Begriff verwendet wird.

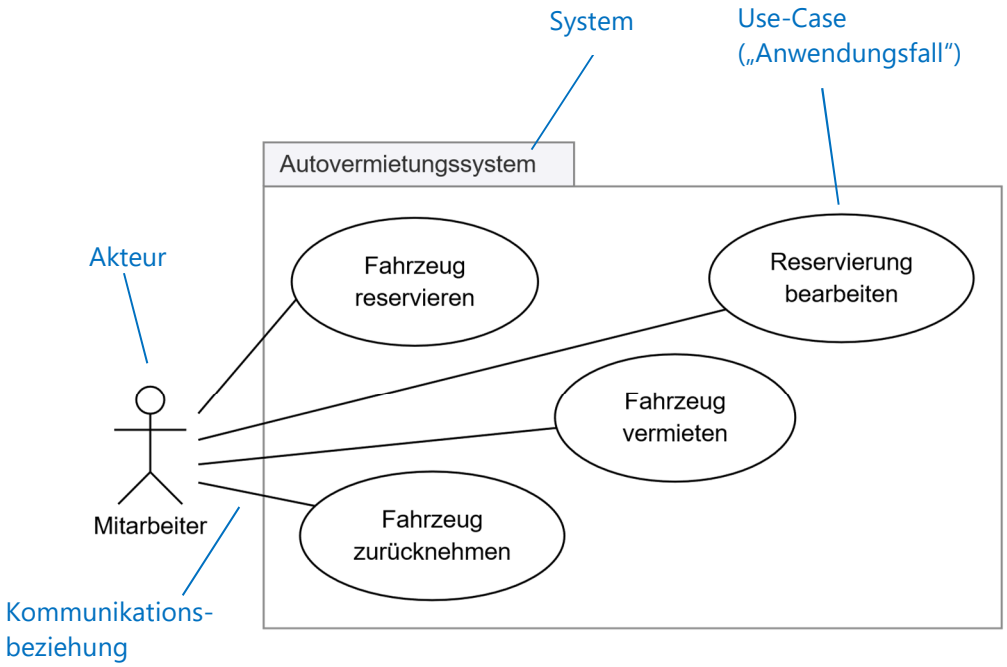


Abbildung 13: Beispiel für ein Use-Case-Diagramm

Abbildung 13 zeigt ein einfaches Beispiel eines Use-Case-Diagramms für ein Autovermietungssystem. Es enthält die folgenden Elemente:

- **System**
  Die Grenzen des zu entwickelnden Systems werden als Rechteck dargestellt, der Name des Systems als Überschrift eingetragen. Da sich ein Use-Case-Diagramm zumeist auf ein einzelnes System bezieht, werden die Systemgrenzen in der Praxis auch oft weggelassen.

- **Akteur**
  Hierbei handelt es sich um eine Rolle, die ein Benutzer gegenüber dem System einnimmt. Akteure werden als Strichmännchen dargestellt. Neben menschlichen Akteuren gibt es auch Systemakteure, d. h. externe Systeme, mit denen das zu entwickelnde System interagiert.

- **Use-Case**
  Ein Use-Case oder Anwendungsfall wird als Ellipse abgebildet. Er stellt eine konkrete, abgeschlossene, mit dem System durchgeführte Aufgabe aus Anwendersicht dar. Dabei kann ein Use-Case aus mehreren Einzelaktivitäten bestehen. „Abgeschlossene Aufgabe" bedeutet, dass ein Use-Case in der Regel ohne Unterbrechung durchgeführt wird, bevor der Benutzer etwas anderes mit dem System macht.

- **Kommunikationsbeziehung**
  Eine solche Beziehung verbindet einen Akteur mit einem Use-Case, mit dem der Akteur kommuniziert. In der Regel bedeutet dies, dass der Akteur den Use-Case durchführt. Es könnte aber auch einmal sein, dass der Akteur nur an dem Use-Case beteiligt ist und z. B. irgendwelche Eingaben machen muss.

4.2 Beschreibung von Use-Cases

Ein Use-Case-Diagramm gibt einen Überblick über das, was man mit dem System machen kann. Um zu dokumentieren wie die verschiedenen Aufgaben im Einzelnen durchgeführt werden sollen, wird für jeden Use-Case noch eine detaillierte Beschreibung benötigt. Wie eine solche Use-Case-Beschreibung aussieht, ist im UML-Standard nicht festgelegt.

Hierfür kann man andere Diagrammtypen heranziehen. Beispielsweise kann man die nacheinander durchzuführenden Einzelschritte in Form eines Ablaufdiagramms darstellen (vgl. Kapitel 5).

Zumeist werden Use-Cases aber in Form von Text beschrieben. Tabelle 5 zeigt ein Schema für eine solche Use-Case-Beschreibung. Dieses Schema ist nicht vom UML-Standard vorgegeben, wird aber in dieser oder ähnlicher Form häufig in der Praxis eingesetzt.

Name	*Name des Use-Case, typischerweise Substantiv + Verb.*
Vorbedingungen	*Gibt es Voraussetzungen im System, damit der Use-Case überhaupt gestartet werden kann?*
Nachbedingung im Erfolgsfall	*Was hat sich geändert, wenn der Use-Case erfolgreich durchgeführt wurde?*
Nachbedingung bei Fehlschlag	*Was ist das Ergebnis, wenn der Use-Case nicht erfolgreich abgeschlossen werden konnte? Insbesondere: Änderungen rückgängig gemacht, System ist wieder bereit für neue Aktionen.*
Akteure	*Dieselben wie im Use-Case-Diagramm.*
Auslösendes Ereignis	*Wodurch wird der Use-Case gestartet?* *Häufig: Der Akteur startet den Use-Case*
Beschreibung	*Ablauf im erfolgreichen Normalfall („Happy Path"),* *1. Schritt 1* *2. Schritt 2* *3. …*
Erweiterungen / Alternativen	*Mögliche Abweichungen und Ergänzungen gegenüber dem Normalfall* *a) Zu Schritt 2: …* *b) Zu Schritt 3: …* *c) …*

Tabelle 5: Schema zur Beschreibung eines Use-Case

Damit kann nun beispielsweise der Use-Case „Fahrzeug reservieren“ aus Abbildung 13 beschrieben werden (Tabelle 6).

Name	Fahrzeug reservieren
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Neue Reservierung angelegt
Nachbedingung bei Fehlschlag	Keine neue Reservierung angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	Mitarbeiter wählt „Fahrzeug reservieren“
Beschreibung	1. Der Mitarbeiter gibt Fahrzeugklasse und Zeitraum an. 2. Das System zeigt verfügbare Modelle und Preise an. 3. Der Mitarbeiter wählt das gewünschte Modell aus. 4. Der Mitarbeiter gibt den Kundennamen ein. 5. Das System zeigt passende Kunden an. 6. Der Mitarbeiter wählt den betreffenden Kunden aus. 7. Das System zeigt die Kundendaten an. 8. Der Mitarbeiter bestätigt die Reservierung.
Erweiterungen / Alternativen	a) Bis Schritt 8 ist jederzeit ein Abbruch möglich b) Zu Schritt 2: Es ist kein Fahrzeug verfügbar. Die Eingaben können geändert werden. c) Zu Schritt 4 bis 7: Der Kunde ist noch nicht im System. Der Mitarbeiter gibt die Kundendaten ein (Vorname, Nachname, Geburtsdatum, Adresse, Ausweisnummer) und bestätigt die Eingabe. Falls die Daten unvollständig sind, weist das System darauf hin und fordert zur Ergänzung auf.

Tabelle 6: Beschreibung des Use-Case „Fahrzeug reservieren“

4.3 Include-Beziehung

Im oben beschriebenen Use-Case „Fahrzeug reservieren“ werden in Schritt 1 und 2 die verfügbaren Modelle und Preise der gewünschten Fahrzeugklasse angezeigt. Die Fahrzeugsuche kann aber selbst wieder als ein eigener Ablauf und damit als separater Use-Case betrachtet werden. Wenn nun „Fahrzeug reservieren“ durchgeführt wird, muss der Use-Case „Fahrzeug suchen“ aufgerufen werden. Der Use-Case „Fahrzeug reservieren“ schließt also „Fahrzeug suchen“ ein. Dies ist in Abbildung 14 mit Hilfe einer sogenannten include-Beziehung modelliert.

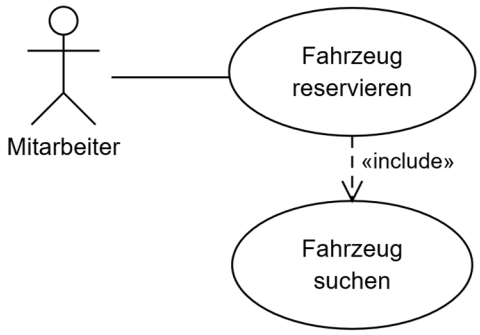


Abbildung 14: Include-Beziehung

In der Beschreibung des Use-Case „Fahrzeug reservieren“ wird auf den eingeschlossenen Use-Case verwiesen (Tabelle 7).

Name	Fahrzeug reservieren
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Neue Reservierung angelegt
Nachbedingung bei Fehlschlag	Keine neue Reservierung angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	Mitarbeiter wählt „Fahrzeug reservieren“
Beschreibung	1. → Use-Case „Fahrzeug suchen“ 2. Der Mitarbeiter gibt den Kundennamen ein. 3. …
Erweiterungen / Alternativen	a) Bis Schritt 6 ist jederzeit …

Tabelle 7: Verweis auf den eingeschlossenen Use-Case „Fahrzeug suchen“

Wird nun der Use-Case „Fahrzeug reservieren“ gestartet, so wird als erstes der Use-Case „Fahrzeug suchen“ aufgerufen und durchgeführt. Wenn dieser abgeschlossen ist, wird „Fahrzeug reservieren“ mit Schritt 2 fortgesetzt. Was innerhalb des aufgerufenen Use-Case passiert, wird in einer eigenen Beschreibung dokumentiert (Tabelle 8).

Name	Fahrzeug suchen
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Ein Fahrzeug ausgewählt
Nachbedingung bei Fehlschlag	Kein Fahrzeug gefunden
Akteure	Mitarbeiter
Auslösendes Ereignis	Fahrzeugsuche im aufrufenden Use-Case gestartet
Beschreibung	1. Der Mitarbeiter gibt Fahrzeugklasse und Zeitraum an 2. Das System zeigt verfügbare Modelle und Preise an 3. Der Mitarbeiter wählt das gewünschte Modell aus
Erweiterungen / Alternativen	a) Es ist jederzeit ein Abbruch möglich b) Zu Schritt 2: Kein geeignetes Fahrzeug gefunden, die Eingaben können geändert werden.

Tabelle 8: Beschreibung des Use-Case „Fahrzeug suchen“

Als auslösendes Ereignis ist hier der Start durch den aufrufenden Use-Case genannt. Die Fahrzeugsuche könnte ja auch noch aus anderen Use-Cases heraus aufgerufen werden. Die include-Beziehung ermöglicht somit auch die Mehrfachverwendung von Use-Cases.

Falls man die Fahrzeugsuche auch direkt aufrufen kann, d. h. nicht aus einem anderen Use-Case heraus, so verbindet man sie im Diagramm zusätzlich direkt mit dem Akteur und formuliert das auslösende Ereignis so, dass auch der direkte Start ohne aufrufenden Use-Case berücksichtigt ist, z. B. „Mitarbeiter startet Fahrzeugsuche (direkt oder über aufrufenden Use-Case)“.

4.4 Extend-Beziehung

Die Beschreibung des Use-Case „Fahrzeug reservieren“ in Tabelle 6 enthält noch einen weiteren Schritt, den man in einen eigenen Use-Case auslagern könnte: Das Anlegen eines neuen Kunden. Allerdings passt hier die include-Beziehung nicht, denn im Normalfall wird beim Reservieren eines Fahrzeugs kein neuer Kunde angelegt. Das Anlegen eines Kunden ist in der Beschreibung ja unter „Erweiterungen/Alternativen“ aufgeführt.

Stellt man „Kunde anlegen“ als eigenen Use-Case dar, so erweitert dieser den Use-Case „Fahrzeug reservieren“. Im Use-Case-Diagramm steht hierfür die extend-Beziehung zur Verfügung (Abbildung 15). Im Gegensatz zu „Fahrzeug suchen“ wird „Kunde anlegen“ bei der Reservierung eines Fahrzeuges im Normalfall nicht durchgeführt, sondern nur im Sonderfall.

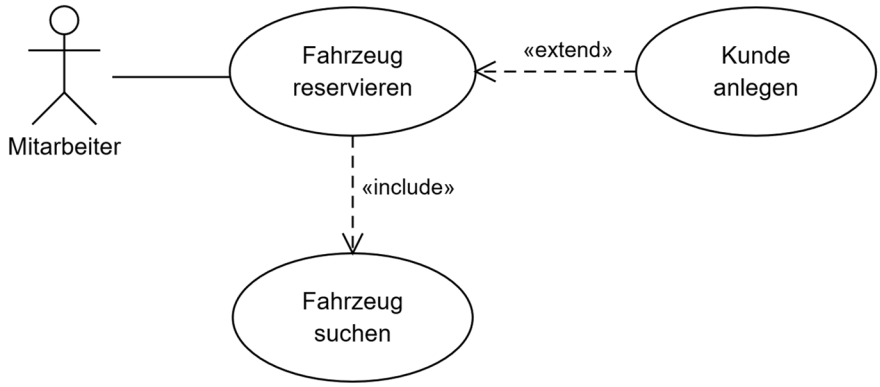


Abbildung 15: Extend-Beziehung

Entsprechend findet sich der Verweis auf den erweiternden Use-Case in der Beschreibung unter „Erweiterungen/Alternativen“ (Tabelle 9). Die Beschreibung für den erweiternden Use-Case selbst findet sich in Tabelle 10.

Name	Fahrzeug reservieren
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Neue Reservierung angelegt
Nachbedingung bei Fehlschlag	Keine neue Reservierung angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	Mitarbeiter wählt „Fahrzeug reservieren“
Beschreibung	1. → Use-Case „Fahrzeug suchen“ 2. Der Mitarbeiter gibt den Kundennamen ein. 3. Das System zeigt passende Kunden an. 4. Der Mitarbeiter wählt den betreffenden Kunden aus. 5. Das System zeigt die Kundendaten an. 6. Der Mitarbeiter bestätigt die Reservierung.
Erweiterungen / Alternativen	a) Bis Schritt 6 ist jederzeit ein Abbruch möglich b) Zu Schritt 2 bis 5: → Use-Case „Kunde anlegen“

Tabelle 9: Verweis auf den erweiternden Use-Case „Kunde anlegen“

Name	Kunde anlegen
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Neuer Kunde angelegt
Nachbedingung bei Fehlschlag	Kein neuer Kunde angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	„Neuer Kunde“ im aufrufenden Use-Case gewählt
Beschreibung	1. Der Mitarbeiter gibt die Kundendaten ein (Vorname, Nachname, Geburtsdatum, Adresse, Ausweisnummer). 2. Der Mitarbeiter bestätigt das Anlegen des neuen Kunden.
Erweiterungen / Alternativen	a) Bis Schritt 2 ist ein Abbruch möglich b) Zu Schritt 2: Falls die Daten unvollständig sind, weist das System darauf hin und fordert zur Ergänzung auf.

Tabelle 10: Beschreibung des Use-Case „Kunde anlegen“

Zu beachten sind die Pfeilrichtungen (vgl. Abbildung 16). Der include-Pfeil geht vom einschließenden Haupt-Use-Case zum eingeschlossenen Use-Case. Der extend-Pfeil hingegen verläuft vom erweiternden Use-Case zum erweiterten Haupt-Use-Case. Man merkt sich die Pfeilrichtungen am besten, indem man die Beziehung gemäß ihrem Namen in Form eines Aussagesatzes formuliert:

- „Fahrzeug reservieren“ schließt ein (includes) „Fahrzeug suchen“.
- „Kunde anlegen“ erweitert (extends) „Fahrzeug reservieren“.

Der Pfeil verläuft dann vom zuerst genannten Use-Case zum zweitgenannten.

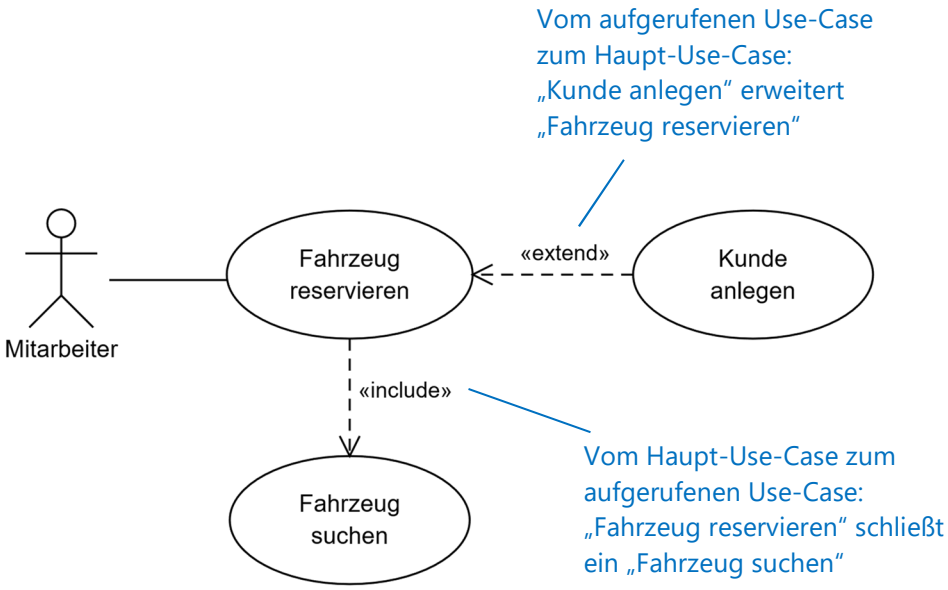


Abbildung 16: Entgegengesetzte Pfeilrichtungen für „include“ und „extend“

4.5 Ablauf bei „include“ und „extend“

In Tabelle 11 wird der Ablauf bei einer include-Beziehung anhand der Use-Case-Beschreibungen visualisiert. Wird der Use-Case „Fahrzeug reservieren“ gestartet, so werden die in der Beschreibung aufgeführten Schritte nacheinander ausgeführt. Bereits der erste Schritt in dem Beispiel enthält den Aufruf des eingeschlossenen Use-Case „Fahrzeug suchen“. Daher springt

man bei der Abarbeitung an dieser Stelle zur Beschreibung von „Fahrzeug suchen“ und führt zunächst die dort aufgeführten Schritte aus. Ist „Fahrzeug suchen“ abgeschlossen, so erfolgt ein Rücksprung in den aufrufenden Use-Case „Fahrzeug suchen“, wo es mit Schritt 2 weitergeht.

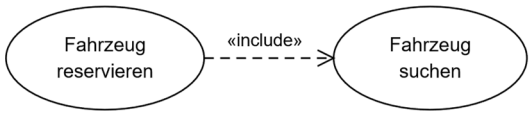
		
Name	Fahrzeug reservieren	Fahrzeug suchen
...	...	...
Beschreibung Ablauf im Normalfall	1. → Use-Case „Fahrzeug suchen“ 2. Der Mitarbeiter gibt den Kundennamen ein. 3. Das System zeigt passende Kunden an. 4. Der Mitarbeiter wählt den betreffenden Kunden aus. 5. Das System zeigt die Kundendaten an. 6. Der Mitarbeiter bestätigt die Reservierung.	1. Der Mitarbeiter gibt Fahrzeugklasse und Zeitraum an 2. Das System zeigt verfügbare Modelle und Preise an 3. Der Mitarbeiter wählt das gewünschte Modell aus

Tabelle 11: Ablauf bei „include“-Beziehung

Der eingeschlossene Use-Case wird also im Normalfall mit durchgeführt. Natürlich könnte es eine Ausnahme geben, die dazu führt, dass der eingeschlossene Use-Case nicht mit ausgeführt wird, z. B. einen Abbruch des Benutzers vor der Fahrzeugsuche. Insofern wird der eingeschlossene Use-Case nicht in jedem Fall mit durchgeführt, aber beim normalen Ablauf schon.

Im Gegensatz dazu wird der erweiternde Use-Case bei einer extend-Beziehung nur in Sonderfällen eingeschlossen. Dies wird in Tabelle 12 gezeigt. Im blau dargestellten Ablauf für den Normalfall wird der Use-Case „Kunde anlegen“ nicht mit ausgeführt, wenn ein Fahrzeug reserviert wird.

Unter „Erweiterungen/Alternativen“ steht unter Punkt b), dass anstelle der Schritte 2 bis 5 aus der Beschreibung der Use-Case „Kunde anlegen“ ausgeführt werden kann. Wenn dieser Sonderfall eintritt, dann ergibt sich der rot dargestellte Ablauf: Wenn im Ablauf von „Fahrzeug reservieren“ Schritt 2 erreicht wird, erfolgt ein Sprung in die Beschreibung von „Kunde anlegen“. Dann werden die dort beschriebenen Schritte durchgeführt. Anschließend wird in den Use-Case „Fahrzeug reservieren“ zurückgesprungen, wo es mit Schritt 6 weitergeht, da „Kunde anlegen“ laut Beschreibung ja die Schritte 2 bis 5 ersetzt.

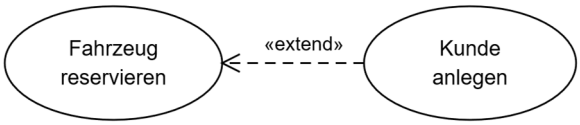
		
Name	Fahrzeug reservieren	Kunde anlegen
...	...	...
Beschreibung Ablauf im Normalfall	1. → Use-Case „Fahrzeug suchen“ 2. Der Mitarbeiter gibt den Kundennamen ein. 3. Das System zeigt passende Kunden an. 4. Der Mitarbeiter wählt den betreffenden Kunden aus. 5. Das System zeigt die Kundendaten an. 6. Der Mitarbeiter bestätigt die Reservierung.	1. Der Mitarbeiter gibt die Kundendaten ein (Vorname, Nachname, Geburtsdatum, Adresse, Ausweisnummer). 2. Der Mitarbeiter bestätigt das Anlegen des neuen Kunden. Ablauf im Sonderfall
Erweiterungen / Alternativen	a) Bis Schritt 6 ist jederzeit ein Abbruch möglich b) Zu Schritt 2 bis 5: → Use-Case „Kunde anlegen“	a) ...

Tabelle 12: Ablauf bei „extend“-Beziehung

4.6 Verwendung von „include“ und „extend“

Wann verwendet man „include“ und wann „extend“?
Man kann einen Teil eines Use-Cases in einen eigenen Use-Case auslagern, wenn ...

- ... es sich um einen eigenständigen, in sich geschlossenen Ablauf handelt,
- ... man ihn eventuell auch ohne den aufrufenden Use-Case selbständig verwenden kann,
- ... man den neuen Use-Case eventuell auch in mehreren anderen Use-Cases wiederverwenden kann.

Die include-Beziehung wird verwendet, wenn der Use-Case im Normalfall mit ausgeführt wird. Der Aufruf des eingeschlossenen Use-Case wird in dem verwendeten Schema unter „Beschreibung“, im sogenannten „Happy Path“ dokumentiert.

Die extend-Beziehung wird verwendet, wenn der aufgerufene Use-Case nur im Sonderfall mit ausgeführt wird. Der Aufruf wird im Schema unter „Erweiterungen/Alternativen“ dokumentiert.

Häufig kann man sich auch darüber streiten, was der Normalfall und was ein Sonderfall ist. Beispielsweise könnte man es beim Reservieren eines Fahrzeugs als Normalfall ansehen, zu-

nächst die Kundendaten zu erfassen (oder zumindest zu aktualisieren). Dann wäre es ein Sonderfall, wenn die Kundendaten schon korrekt im System sind und nicht mehr erfasst oder aktualisiert werden müssen. Man sollte sich daher überlegen, was für den Benutzer wohl der typische und häufigere Fall ist. Falls es sich nicht eindeutig bestimmen lässt, muss man eben eine Entscheidung treffen, was als Normal- und was als Sonderfall angesehen werden soll. Letztlich müssen aus dem Diagramm und den Beschreibungen alle verschiedenen Fälle hervorgehen.

4.7 Generalisierung von Use-Cases

Gelegentlich gibt es mehrere Use-Cases, die im Wesentlichen das Gleiche tun und sich nur in Details voneinander unterscheiden. In diesem Fall kann man diese Use-Cases zu einem einzigen Use-Case zusammenfassen und die unterschiedlichen Möglichkeiten mit Hilfe von Sonderfällen und Alternativen in der Use-Case-Dokumentation beschreiben.

Sollen jedoch die verschiedenen Fälle als eigene Use-Cases dargestellt werden, so kann man eine Generalisierungsbeziehung verwenden. In Abbildung 17 wurde ein allgemeiner Use-Case „Zahlung abwickeln" definiert. Hierfür gibt es zwei Varianten: „Barzahlung abwickeln" und „Kreditkartenzahlung abwickeln". Sie sind als eigene Use-Cases dargestellt, die über Generalisierungs-Beziehungen mit dem allgemeinen Use-Case verbunden sind.

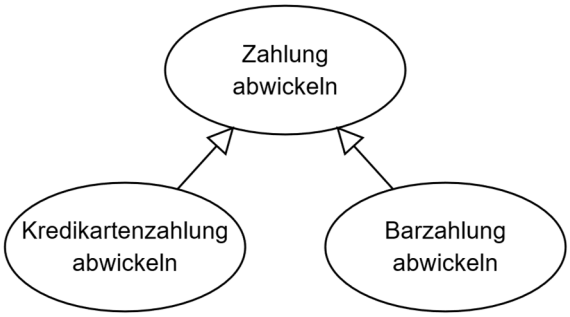


Abbildung 17: Generalisierung von Use-Cases

Der Pfeil läuft von den speziellen Use-Cases zum allgemeinen Use-Case. Die Pfeilspitze wird als nicht ausgefülltes Dreieck gezeichnet. Sie ist etwas größer als gewöhnliche Pfeilspitzen. Dasselbe Symbol wird auch für Generalisierungsbeziehungen in Klassendiagrammen verwendet (vgl. Kapitel 8). Dort hat es eine sehr genau festgelegte Bedeutung: Im Fall der Use-Cases ist hingegen nicht genau festgelegt, welche Eigenschaften der spezielle Fall vom allgemeinen Fall übernimmt. Man kann also selbst entscheiden, wo eine Generalisierung sinnvoll einzusetzen ist.

Da die über eine Generalisierungsbeziehung verbundenen Use-Cases sehr viele Gemeinsamkeiten haben, empfiehlt es sich, die Gemeinsamkeiten in der Dokumentation des allgemeinen Use-Cases zu beschreiben. In den Beschreibungen der spezialisierten Use-Cases braucht dann nur noch angegeben zu werden, wie sie sich vom allgemeinen Fall unterscheiden.

4.8 Kommunikation mit externen Systemen

Wenn das modellierte System mit anderen Systemen kommuniziert, dann wird das andere System ebenfalls als Akteur eingetragen und über Kommunikationsbeziehungen mit den Use-Cases verbunden, an denen es beteiligt ist. So werden im Use-Case „Kreditkartenzahlung abwickeln" Daten mit einem externen Kreditkartensystem ausgetauscht. Neben dem menschlichen Akteur ist damit ein sogenannter Systemakteur an dem Use-Case beteiligt (Abbildung 18).

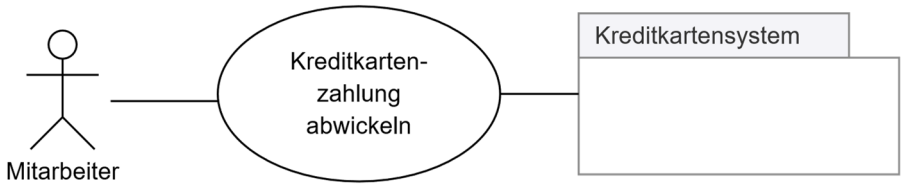


Abbildung 18: Systemakteur

Zwar darf man auch einen Systemakteur mit dem Strichmännchen-Symbol darstellen, doch ist die Darstellung als Rechteck intuitiver – die hier gewählte Darstellung entspricht der für die Systemgrenzen des modellierten Systems (vgl. Abbildung 13).

Mit dem Use-Case „Kreditkartenzahlung abwickeln" sind zwei Akteure verbunden. Dies bedeutet, dass beide Akteure zugleich an der Durchführung dieses Use-Cases beteiligt sind. Weder kann ein Mitarbeiter den Use-Case erfolgreich durchführen, wenn das Kreditkartensystem nicht funktioniert, noch wird eine Zahlung über das Kreditkartensystem ausgeführt, wenn kein Mitarbeiter beteiligt ist.

Es können auch mehrere menschliche Akteure gemeinsam einen Use-Case durchführen, z. B. eine Hilfestellung durch einen Support-Mitarbeiter, der sich auf dem Computer des Benutzers einloggt und gemeinsam mit diesem die notwendigen Schritte zur Problembehebung durchgeht.

Selbstverständlich kann es auch automatisierte Use-Cases geben, an denen kein menschlicher Akteur, sondern nur ein Systemakteur – oder gar kein Akteur – beteiligt ist.

4.9 Generalisierung von Akteuren

Auch für Akteure kann man die Generalisierungsbeziehung verwenden. So wird in Abbildung 19 dargestellt, dass ein Manager ein spezieller Mitarbeiter ist. Dies bedeutet, dass ein Manager auch an die Stelle eines Mitarbeiters treten kann. Wer die Rolle „Manager" hat, kann damit automatisch auch alle Use-Cases ausführen, die dem Akteur „Mitarbeiter" zugeordnet sind. Zusätzlich kann er noch die mit dem Akteur „Manager" verbundenen Use-Cases durchführen.

Möchte man darstellen, dass ein Use-Case wahlweise von unterschiedlichen Akteuren ausgeführt werden kann, so kann man nicht einfach alle diese Akteure mit dem Use-Case verbin-

den. Wie oben beschrieben würde dies bedeuten, dass sie den Use-Case *gemeinsam* durchführen. Stattdessen verbindet man den Use-Case mit nur einem Akteur, der eine Generalisierung der einzelnen Akteure darstellt, die alle den Use-Case ausführen können.

Abbildung 19: Generalisierung von Akteuren

4.10 Beispiel: Use-Case-Diagramm der Autovermietung

Abbildung 20 zeigt das Use-Case-Diagramm für die Autovermietung im Überblick. Zu den bisher besprochenen Use-Cases und Beziehungen sind noch extend-Beziehungen zu „Fahrzeug vermieten“ und zu „Reservierung bearbeiten“ sowie eine include-Beziehung zu „Zahlung abwickeln“ hinzugekommen.

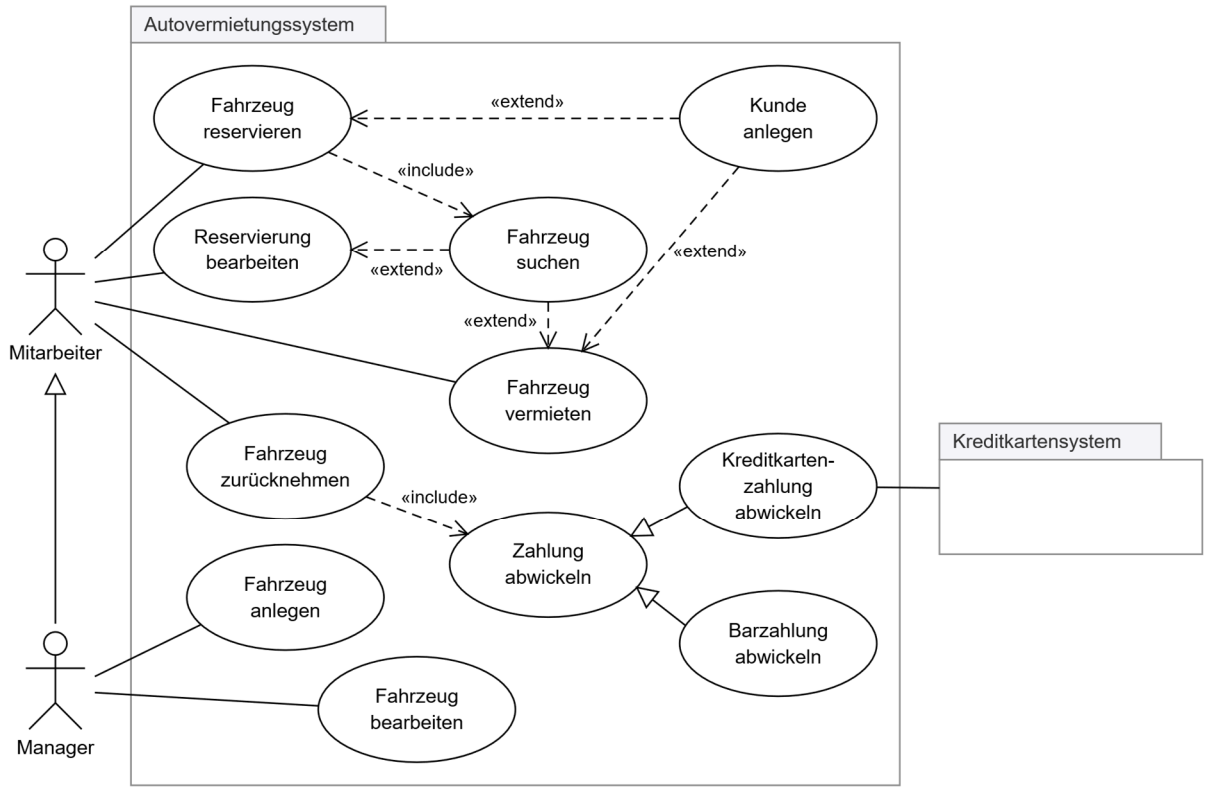


Abbildung 20: Use-Case-Diagramm für das Autovermietungssystem

Außerdem wurden die beiden Use-Cases „Fahrzeug anlegen“ und „Fahrzeug bearbeiten“ hinzugefügt, denn schließlich müssen die für die Fahrzeug-Reservierung und -Ausleihe benötigten Fahrzeugdaten erfasst und bearbeitet werden können. Diese Use-Cases wurden dem Akteur „Manager“ zugeordnet. Wie oben beschrieben kann ein Manager aufgrund der Generalisierungsbeziehung auch die Use-Cases des Mitarbeiters, wie „Fahrzeug reservieren“, „Reservierung bearbeiten“, usw. durchführen.

4.11 Geeignete Größe für Use-Cases

Die UML macht keine genauen Vorschriften, wie umfangreich ein Use-Case sein soll. Es gibt unterschiedliche Arten, wie Use-Cases eingesetzt und strukturiert werden.

In diesem Skript werden System-Use-Cases betrachtet, für die folgendes gelten soll:

- Ein Use-Case ist eine abgeschlossene Aufgabe, die zu einem Zeitpunkt mit dem System in der Regel ohne Unterbrechungen komplett durchgeführt wird.
- Wenn mehrere Akteure beteiligt sind, arbeiten diese gemeinsam und gleichzeitig an dem Use-Case.

Beispiele für zu kleine und zu große Use-Cases:

- „Name eingeben“ oder „Start drücken“ sind zu klein, da es keine abgeschlossenen Aufgaben sind.
- „Dienstfahrzeug beschaffen“ ist zu groß, da in der Regel mehrere Use-Cases erforderlich sind (beantragen, genehmigen, Bestellung aufgeben …).

Häufig stellt sich die Frage, ob man ähnliche und miteinander verwandte Use-Cases zusammenfassen oder separat darstellen soll. Beispielsweise kann man „Kunde anlegen“, „Kunde ändern“ und „Kunde löschen“ als drei Use-Cases darstellen oder in Form eines einzigen Use-Case „Kunde bearbeiten“. Falls man nur einen Use-Case verwendet, müssen die verschiedenen Möglichkeiten aus der Dokumentation hervorgehen.

Es gibt keine allgemeingültige Regel, wann man verwandte Use-Cases zusammenfassen soll. Dies muss für den jeweiligen Fall entschieden werden und hängt davon ab, ob es sehr große Unterschiede zwischen den einzelnen Use-Cases gibt, z. B. umfangreiche Prüfungen beim Anlegen, die beim späteren Bearbeiten eines Kunden nicht benötigt werden. Dann können separate Use-Cases sinnvoll sein. Andererseits sollte man auch darauf achten, dass das Diagramm nicht zu viele Use-Cases enthält und dadurch unübersichtlich wird.

Und schließlich sollte man es einheitlich handhaben: Wenn man nur einen Use-Case „Kunde bearbeiten“ modelliert hat, sollte man dies für andere Daten genauso handhaben und nicht etwa für das Anlegen, Ändern und Löschen von Fahrzeugen mehrere Use-Cases verwenden.

4.12 Vorgehen beim Erstellen von Use-Cases

Zumeist bietet es sich an, zunächst die künftigen Akteure zu ermitteln: Wer benutzt das System und welche Aufgaben führt jeder Akteur durch? Damit gewinnt man schon eine Reihe von Use-Case-Kandidaten. Wenn Modelle der zu unterstützenden Geschäftsprozesse vorliegen, liefern diese ebenfalls eine Reihe von Informationen.

Weitere Use-Case-Kandidaten kann man ermitteln indem man sich die folgenden Fragen stellt:

- Welche Ziele hat das System? Welche Ergebnisse sollen durch die Arbeit mit dem System erzielt werden?
- Welche Ereignisse spielen eine Rolle für das System? Was wird dadurch jeweils ausgelöst?
- Wie kommen die benötigten Daten ins System? Werden für die Eingabe weitere Use-Cases benötigt?

Ein nützliches Prinzip ist es, zunächst den Normalfall zu ermitteln, den sogenannten „Happy Path". Im zweiten Schritt identifiziert man dann die verschiedenen möglichen Sonderfälle, Ausnahmen, Erweiterungen und mehrfach genutzte Use-Cases. Damit wird das Diagramm dann verfeinert, z. B. mit include- und extend-Beziehungen sowie Generalisierungen. Schließlich werden die ermittelten Sachverhalte in den Use-Case-Dokumentationen beschrieben.

4.13 Kriterien für gute Use-Cases

Einige Kriterien für gute Use-Cases sind:

- Die Use-Cases und ihre Dokumentationen sind verständlich für den Auftraggeber.

- Es handelt sich um fachliche Beschreibungen extern wahrnehmbaren Verhaltens.
  Es ist beschrieben, was der Benutzer bei der Bedienung des Systems erlebt. Nicht in die Use-Case-Beschreibungen gehören hingegen Interna der Software oder technische Details.

- Der Funktionsumfang des Systems wird durch die Use-Cases vollständig beschrieben. Was nicht beschrieben ist, wird später auch nicht umgesetzt. Das bedeutet auch, dass alle bei ordnungsgemäßem Betrieb des Systems möglichen Ausnahmen aufzuführen sind. Beispiele für derartige Ausnahmen sind:

  - Ein benötigtes anderes System ist nicht erreichbar.
  - Ein Drucker ist nicht an.
  - Eine Datei ist nicht vorhanden.
  - Der Benutzer bricht eine Aktion ab.
  - Eingaben sind unzulässig oder unvollständig.

- Die Use-Case-Diagramme sind übersichtlich .
  Es geht keineswegs darum, mit Hilfe von include- und extend-Beziehungen alle

Navigationsmöglichkeiten zwischen den verschiedenen Use-Cases komplett zu beschreiben. Ein Use-Case-Diagramm beschreibt keinen Screenflow. Vielmehr sollen die wichtigsten fachlichen Zusammenhänge modelliert werden.
Sehr große Use-Case-Diagramme sollten in kleinere Diagramme aufgeteilt werden.

- Use-Case-Diagramme enthalten keine Abläufe
  Jeder Use-Case enthält selbst einen kleinen, abgeschlossenen Ablauf. Möchte man Use-Case-übergreifende Abläufe oder den Ablauf innerhalb eines Use-Cases grafisch darstellen, so kann man hierfür Ablaufdiagramme verwenden (vgl. Kapitel 5). Wird eine Aktivität innerhalb eines solchen Ablaufdiagramms mit Hilfe des zu entwickelnden Systems durchgeführt, so kann sie einem Use-Case entsprechen.

- Use-Case-Diagramme enthalten keine Datenflüsse.
  Wenn beispielsweise die Use-Cases „Auftrag anlegen" und „Auftrag genehmigen" zu verschiedenen Zeiten und von verschiedenen Personen durchgeführt werden, wird im Use-Case-Diagramm keine Beziehung gezeichnet. Der entsprechende Zusammenhang geht dann nur aus den Dokumentationen hervor.

Die Dokumentation eines Use-Cases sollte ca. eine Druckseite nicht überschreiten, d. h. sie sollte nicht zu detailliert sein. Die Use-Case-Dokumentationen sollten so geschrieben sein, dass sie später als Grundlage für das Testen des Systems genutzt werden können. Hierzu geht man beim Testen die beschriebenen Schritte für den Normalfall und die unterschiedlichen Ausnahmefälle durch. Somit lassen sich die verschiedenen Testfälle direkt aus den Use-Case-Beschreibungen ableiten.

Zu beachten ist weiterhin, dass die hier verwendeten System-Use-Cases immer das Zusammenspiel der Benutzer mit dem System beschreiben. Das bedeutet insbesondere, dass keine Aktionen außerhalb des Systems abgebildet werden. Die von dem System zu unterstützenden Abläufe umfassen natürlich eine Reihe von Aspekten, die außerhalb des Systems erfolgen. Sie sollten im Vorfeld im Zuge einer Geschäftsprozessanalyse und –modellierung betrachtet werden.

Mit den Use-Cases soll die Funktionalität des Systems beschrieben werden. Ein Beispiel für eine eher ungünstige Beschreibung wäre etwa:

> *Der Sachbearbeiter vergleicht jeden Eintrag in dem Reisekostenabrechnungsformular mit dem zugehörigen Beleg und hakt ihn ab. Dann trägt er den Betrag ins System ein.*

Die meisten Informationen betreffen das System überhaupt nicht. Besser wäre folgendes Beispiel:

> *Der Sachbearbeiter trägt in die tabellarische Eingabemaske für jede Position der Reisekostenabrechnung einen Text und den Betrag ein. Das System zeigt jeweils den aktualisierten Gesamtbetrag an.*

Hieraus lässt sich für die Entwicklung wesentlich konkreter entnehmen, wie das System funktionieren soll.

Damit soll keineswegs gesagt werden, dass es unwichtig sei, was der Sachbearbeiter außerhalb des Systems macht. Im Gegenteil: Um sinnvolle Systemfunktionalitäten zu entwerfen ist es sogar sehr wichtig zu verstehen, wie der Benutzer arbeitet. Nur dann kann man herausfinden, welche Unterstützung er von dem zu entwickelnden System benötigt. Die Use-Cases und ihre Dokumentationen sind bereits das Ergebnis dieser Überlegungen. Sie sollten daraufhin überprüft werden, ob sie die tatsächlichen Bedürfnisse der Anwender befriedigen.

4.14 Hinweise zu den Use-Case-Dokumentationen

In der Praxis ergibt sich immer wieder eine Reihe von Fragen bei der Dokumentation der Use-Cases. Hierfür einige Hinweise:

Vorbedingungen

Es sollten keine trivialen Vorbedingungen aufgeführt werden („*System läuft*") und auch keine Bedingungen außerhalb des Systems („*Benutzer hat genug Geld auf dem Konto*"). Vielmehr geht es darum, Bedingungen im System anzugeben, damit der Use-Case überhaupt starten kann.

Z. B. ist die Vorbedingung für „*Rechtschreibung prüfen*" in einer Textverarbeitung, dass ein Dokument geöffnet ist. Dagegen ist „*Gewünschtes Buch vorhanden*" keine Vorbedingung für den Use-Case „*Buch suchen*" – denn man kann die Suche ja auch beginnen, wenn gar nicht bekannt ist, ob ein gewünschtes Buch vorhanden ist. Das erfährt man ja erst als Ergebnis der Suche.

Nachbedingungen

Die Nachbedingung im Erfolgsfall ist einfach das gewünschte Ergebnis. Interessanter ist die Nachbedingung im Falle eines Fehlschlags. Zum einen bedeutet dies natürlich, dass das gewünschte Ergebnis nicht – oder zumindest nicht vollständig – erzielt wurde. Wesentlich interessanter sind aber mögliche sonstige Nachbedingungen, so müssen vielleicht bereits stattgefundene Änderungen rückgängig gemacht worden sein (z. B. Geldrückgabe bei einem Automaten, wenn der Kauf abgebrochen wurde).

Es muss ein sinnvoller, für den Benutzer nachvollziehbarer Zustand hergestellt worden sein. Beispiel: „*Die bereits erfolgten Eingaben sind zwischengespeichert und können später vervollständigt werden.*" Man kann sich natürlich darüber streiten, ob es ein Fehlschlag ist, wenn das gewünschte Ergebnis nur teilweise erreicht wurde, oder ob man es lieber als Möglichkeit bei der Nachbedingung im Erfolgsfall dokumentiert. Wichtig ist vor allem, *dass* es dokumentiert wird.

Auslösendes Ereignis

Beispiele für auslösende Ereignisse sind:

- Benutzeraktion: „*Benutzer wählt ‚neues Spiel'*" oder „*Benutzer wirft Geld ein*".

- Das Eintreten eines bestimmten Zeitpunktes bzw. das Ablaufen einer bestimmten Frist.
- Das Eintreten eines bestimmten Zustandes (z. B. Überschreiten einer bestimmten Temperatur).

Ereignisse, die nicht auf das System einwirken, sollten auch nicht als auslösende Ereignisse genannt werden. Z. B. ist „*Benutzer hat Hunger*" aus Sicht des Systems kein auslösendes Ereignis für den Use-Case „*Pizza-Bestellung erfassen*", denn das System bekommt ja nichts davon mit, dass der Benutzer Hunger hat.

4.15 Fallstudie: Geräteverwaltung

Im vorliegenden Skript wird eine kleine, durchgängige Fallstudie entwickelt, anhand derer die verschiedenen Modelltypen ausprobiert werden können. Hierzu liegt die folgende Beschreibung vor:

- Es soll ein System zur Verwaltung von Geräten in einer Firma, einer Hochschule o. ä. entwickelt werden (z. B. Computer, Beamer, Musikanlagen, Messgeräte, medizinische Geräte etc.)
- Ein Gerät kann entweder fest einem Mitarbeiter und ggf. einem Raum zugeordnet sein, oder es kann von einem Mitarbeiter ausgeliehen werden.
- Die Zuordnungen zu Mitarbeiter und Raum können geändert werden, d. h. ein Gerät wird weitergegeben, oder es zieht um.
- Für die Pflege der Geräte im System ist ein Geräteverwalter zuständig. Er verwaltet auch Informationen über Inspektionen oder Reparaturen der Geräte.
- Um ein Gerät auszuleihen, kann man es zunächst über das System reservieren. Hierbei wird immer ein bestimmter Gerätetyp reserviert, zu dem es mehrere einzelne Geräte geben kann. Das Verleihen und das Zurücknehmen eines Gerätes werden vom Geräteverwalter durchgeführt.
- Ein Verleih ist auch ohne vorherige Reservierung möglich.
- Mitarbeiter können sich Geräte anzeigen lassen, die eigene Gerätenutzung anzeigen lassen (Reservierungen, fest zugeordnete Geräte) und eigene Reservierungen ändern.
- Der Geräteverwalter kann die Gerätenutzung beliebiger Mitarbeiter anzeigen und auch deren Reservierungen ändern.
- Für manche der genannten Aufgaben ist es erforderlich, Geräte oder Mitarbeiter zu suchen. Hierbei handelt es sich um eigene Aufgaben, die an verschiedenen Stellen wiederverwendet werden können.

- Der Geräteverwalter soll auch Statistiken erstellen können (über die Gerätenutzung o. ä.).

Abbildung 21 zeigt ein mögliches Use-Case-Diagramm für das beschriebene Fallbeispiel. Außerdem sind Beschreibungen für die Use-Cases „Gerät reservieren“ (Tabelle 13) und „Gerät verleihen“ (Tabelle 14) angegeben.

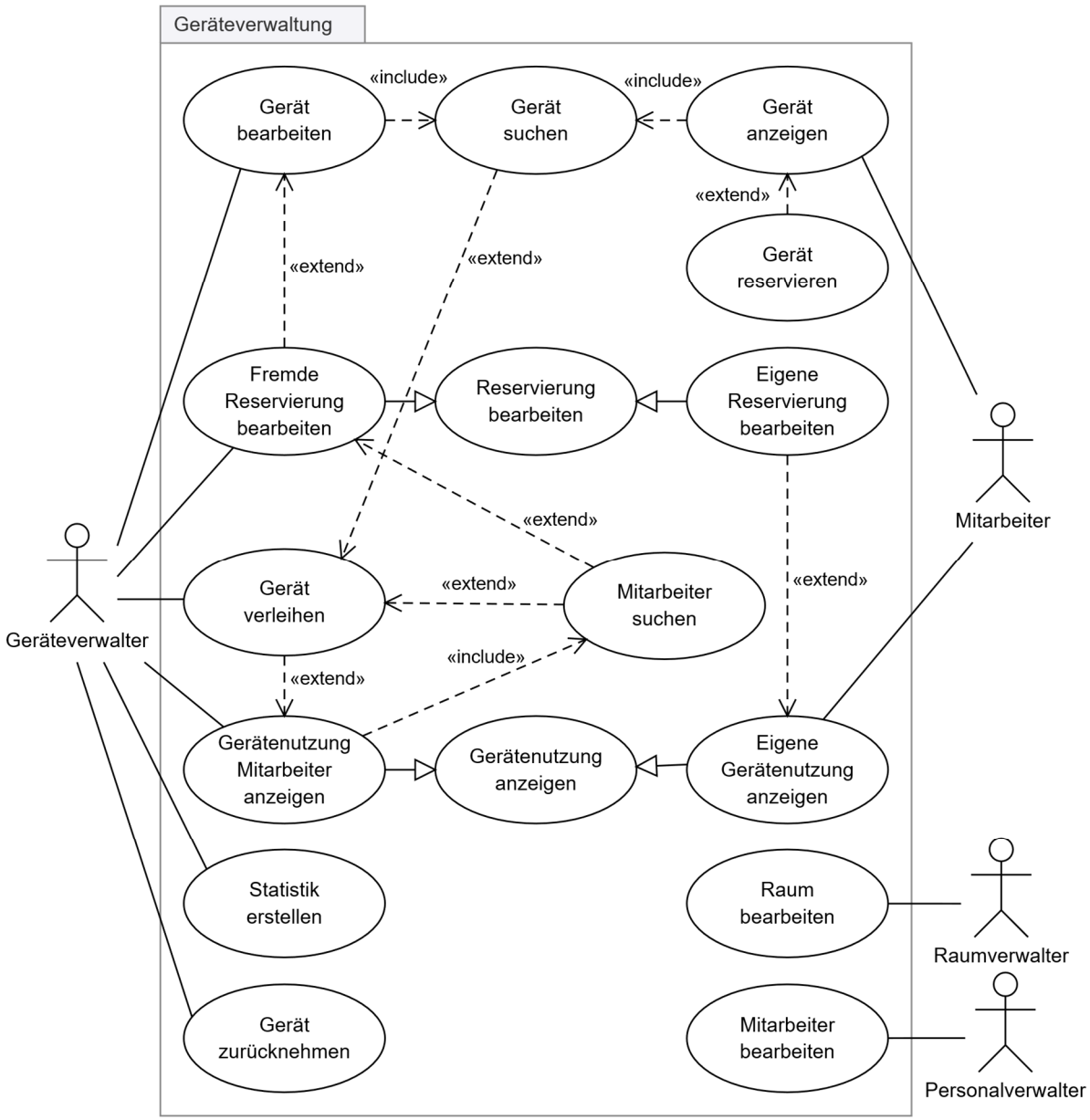


Abbildung 21: Use-Case-Diagramm Geräteverwaltung

Name	Gerät reservieren
Vorbedingungen	Ausleihbarer Gerätetyp wird angezeigt
Nachbed. im Erfolgsfall	Neue Reservierung angelegt
Nachbedingung bei Fehlschlag	Keine neue Reservierung angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	Mitarbeiter wählt in Geräteanzeige „Gerät reservieren“
Beschreibung	1. Der Mitarbeiter gibt den gewünschten Zeitraum an. 2. Das System zeigt an, dass die Reservierung zu diesem Zeitraum möglich ist. 3. Der Mitarbeiter bestätigt die Reservierung.
Erweiterungen / Alternativen	a) Bis Schritt 3 ist ein Abbruch möglich b) Zu Schritt 2: Die Reservierung ist nicht möglich. Falls ein Teil des gewünschten Zeitraums möglich ist, wird dieser angezeigt. Zudem wird angezeigt, wann ein Gerät des gewünschten Typs anschließend wieder verfügbar ist. Der Mitarbeiter kann dann den gewünschten Zeitraum ändern.

Tabelle 13: Beschreibung des Use-Case „Gerät reservieren“

Name	Gerät verleihen
Vorbedingungen	Reservierung in „Gerätenutzung Mitarbeiter anzeigen“ selektiert, falls Verleih ohne Reservierung: keine Vorbedingung
Nachbed. im Erfolgsfall	Gerät verliehen
Nachbedingung bei Fehlschlag	Gerät nicht verliehen, Reservierung ggf. storniert oder geändert
Akteure	Geräteverwalter
Auslösendes Ereignis	Geräteverwalter wählt „Verleihen“
Beschreibung	1. Das System zeigt verfügbare Geräte für reservierten Gerätetyp an. 2. Der Geräteverwalter wählt das zu verleihende Gerät aus. 3. Das System zeigt Mitarbeiter, Gerät, Zeitraum an. 4. Der Geräteverwalter bestätigt die Ausleihe.
Erweiterungen / Alternativen	a) Bis Schritt 4 ist ein Abbruch möglich b) Zu Schritt 1: Falls Verleih ohne Reservierung, → UC Gerät suchen und → UC Mitarbeiter suchen. Rückgabedatum eintragen. c) Zu 3: Rückgabedatum kann geändert werden. d) Zu 1: Kein Gerät verfügbar (z. B. wegen Defekt), Reservierung löschen oder ändern → UC Fremde Reservierung bearbeiten.

Tabelle 14: Beschreibung des Use-Case „Gerät verleihen“

In einem realen Projekt würden für jeden Use-Case aus Abbildung 21 entsprechende Use-Case-Beschreibungen erstellt werden.

Das Geräteverwaltungssystem, dessen funktionale Anforderungen hier aus Benutzersicht beschrieben wurden, wird in den folgenden Kapiteln als durchgängiges Beispiel wieder aufgegriffen.

5 Modellierung von Abläufen

Häufig hat man es in der Software-Entwicklung mit Abläufen zu tun, beispielsweise mit der Abarbeitung von Algorithmen oder den Geschäftsprozessen, die von der Software unterstützt werden sollen. Es gibt verschiedene Arten von Ablaufdiagrammen, die sich in ihrer Grundstruktur ähneln. Im Folgenden wird die standardisierte Notation BPMN (Business-Process-Model-and-Notation) vorgestellt [OMG13]. Zwar enthält auch die in den anderen Kapiteln dieses Buchs verwendete UML mit den sogenannten Aktivitätsdiagrammen eine Ablaufdarstellung, doch ist diese nicht so verbreitet wie BPMN.

Hinweis: Wie bereits in Kapitel 3 erwähnt, unterstützen viele UML-Tools die BPMN nicht, so dass ggf. ein spezielles BPMN-Modellierungstool verwendet werden muss, wie z. B. der kostenfrei nutzbare „Bizagi Modeler".

Das BPMN-Diagramm in Abbildung 22 beinhaltet einen einfachen Ablauf zum Bearbeiten und Korrigieren von Aufgaben auf einer Online-Lernplattform. Zunächst lädt ein Studierender eine Aufgabenbearbeitung hoch. Anschließend wird sie vom Tutor korrigiert. Wurde die Mindestpunktzahl erreicht, so lädt der Tutor seine Korrekturanmerkungen hoch. Wenn der Student diese Korrekturanmerkungen gelesen hat, ist der Ablauf beendet.

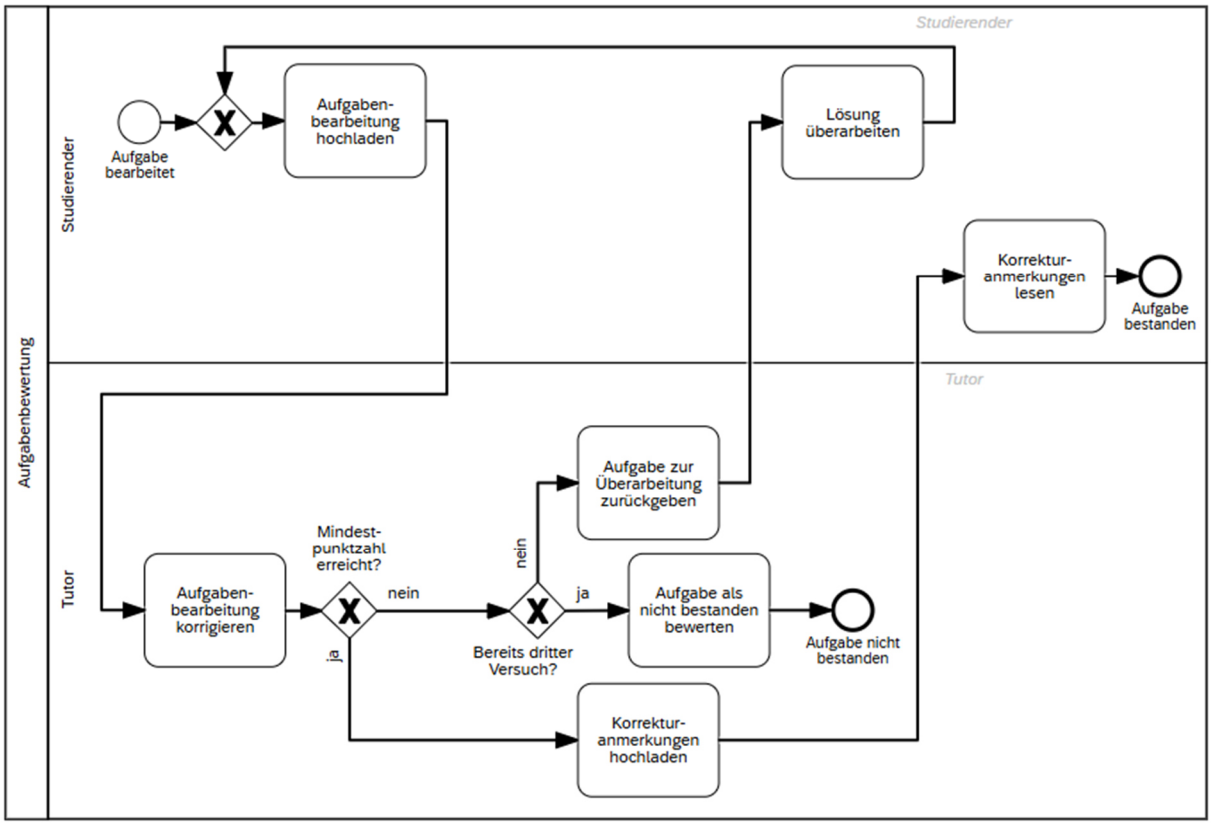


Abbildung 22: Ablauf zur Bearbeitung und Korrektur einer Aufgabe auf einer Online-Lernplattform

Stellt sich bei der Korrektur heraus, dass die Mindestpunktzahl nicht erreicht wurde, so gibt es zwei Möglichkeiten: Entweder handelt es sich bei der hochgeladenen Aufgabenbearbeitung bereits um den dritten Versuch, oder noch nicht. Ist es bereits der dritte Versuch, so bewertet der Tutor die Aufgabe als nicht bestanden, und der Ablauf ist an dieser Stelle beendet.

Handelt es sich noch nicht um den dritten Versuch, so gibt der Tutor die Aufgabe zur Überarbeitung zurück, woraufhin der Student die Lösung überarbeitet. Anschließend führt der Student die Aktion „Aufgabenbearbeitung hochladen" erneut durch, woraufhin sich der beschriebene Ablauf wiederholt – solange, bis die Bearbeitung erfolgreich war und der Student die Korrekturanmerkung gelesen hat, oder bis die Aufgabe als nicht bestanden bewertet wurde.

Der Ablauf besteht aus mehreren Aktivitäten. Eine elementare Aktivität, die man nicht weiter unterteilt, wird auch als Task (Arbeitsschritt) bezeichnet. Die Pfeile stellen den Sequenzfluss dar, also die Reihenfolge, in der die Tasks durchgeführt werden.

Das Diagramm ist in zwei Lanes (Bahnen) eingeteilt. Durch die Platzierung in unterschiedlichen Lanes wird ausgedrückt, wer den jeweiligen Task durchführt. Lanes können – wie hier – unterschiedliche Rollen, aber auch Organisationseinheiten oder IT-Systeme repräsentieren.

Lanes sind optional, d. h. es müssen keine Lanes eingezeichnet werden. Beispielsweise kann man darauf verzichten, wenn alle Tasks nur von einer einzigen Rolle oder einem einzigen System durchgeführt werden.

Der Ablauf beginnt mit einem Kreis, dem Startereignis, und er endet an einem Kreis mit dickem Rand, einem Endereignis. In dem Beispiel sieht man, dass es auch mehrere Endereignisse für einen Ablauf geben kann.

Bei den mit einem „X" versehenen Rauten handelt es sich um „exklusive Gateways", die für Verzweigungen und Zusammenführungen verwendet werden. Ein verzweigender Gateway hat einen eingehenden und mehrere ausgehende Sequenzfluss-Pfeile, an denen jeweils notiert ist, in welchem Fall der betreffende Ausgang gewählt wird. Wie in diesem Beispiel steht an einer Verzweigung oftmals eine Frage, und je nach Antwort wird ein anderer Ausgang gewählt.

Um exklusive Sequenzflüsse zusammenzuführen, wird wiederum ein Gateway mit einem „X" verwendet, der dann mehrere Eingänge und nur einen Ausgang hat.

Das „X" beim exklusiven Gateway kann auch weggelassen werden. Doch es hilft zur besseren Unterscheidung von anderen Arten von Gateways.

5.1 Entscheidungen

Ein verzweigender exklusiver Gateway repräsentiert eine Entscheidung. Er hat einen Eingang und zwei oder mehr Ausgänge (Abbildung 23).

Jeder ausgehende Sequenzfluss muss mit einer Bedingung versehen sein. Dabei ist darauf zu achten, dass für jeden möglichen Fall immer genau eine Bedingung wahr ist. So ist in Abbildung 24 für jeden Wert von „Summe“ eindeutig bestimmbar, welcher Ausgang auszuwählen ist.

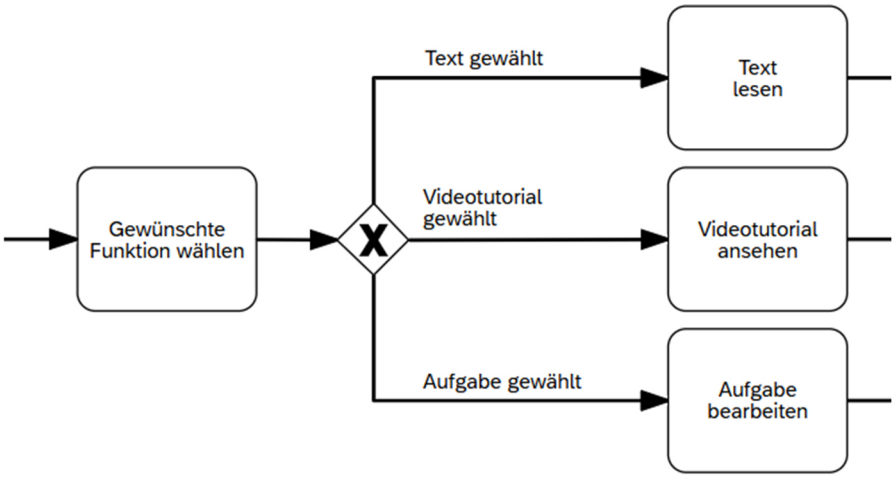


Abbildung 23: Beispiel einer Entscheidung mit drei Ausgängen

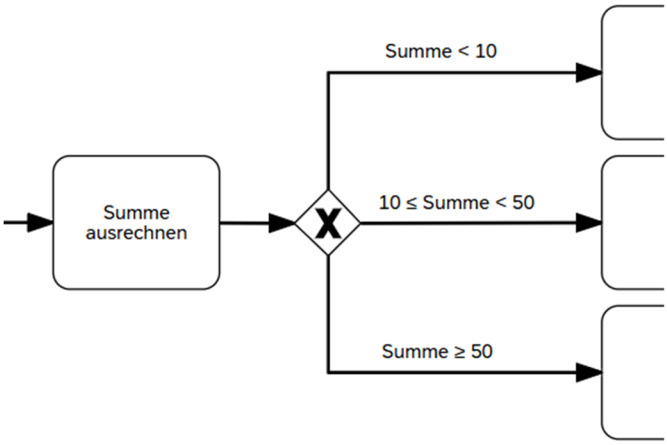


Abbildung 24: Für jeden Wert von „Summe“ ist genau eine Bedingung wahr

In Abbildung 25 ist diese Regel verletzt worden. Für Summen zwischen 10 und 50 ist kein Ausgang definiert.

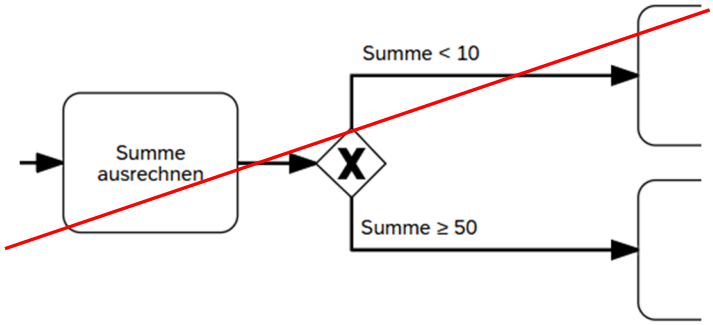


Abbildung 25: Unvollständige Kombination von Bedingungen

Das Beispiel in Abbildung 26 illustriert den gegenteiligen Fall. Hier überlappen die Bedingungen. Hat die Summe einen Wert zwischen 5 und 10, so sind beide Bedingungen wahr und es ist nicht klar, welcher der beiden Ausgänge gewählt werden soll.

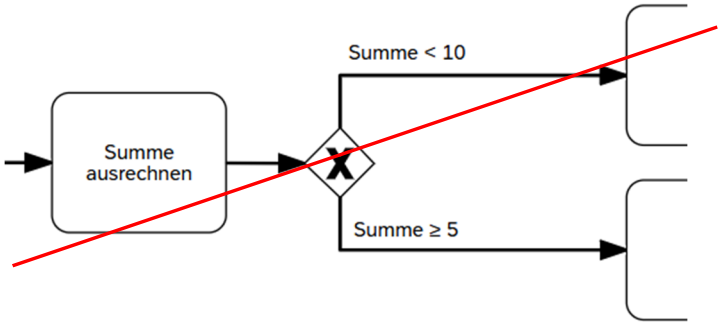


Abbildung 26: Überlappende Bedingungen

Bei einem verzweigenden exklusiven Gateway darf aber per Definition immer nur *genau ein* Ausgang gewählt werden.

Um sicherzustellen, dass auf jeden Fall ein Ausgang gewählt wird (und um Schreibarbeit zu sparen), kann man an einem Ausgang einen Schrägstrich anbringen. Damit wird der Default-Ausgang gekennzeichnet. Dieser wird immer dann gewählt, wenn keine der Bedingungen an den anderen Ausgängen zutrifft (Abbildung 27).

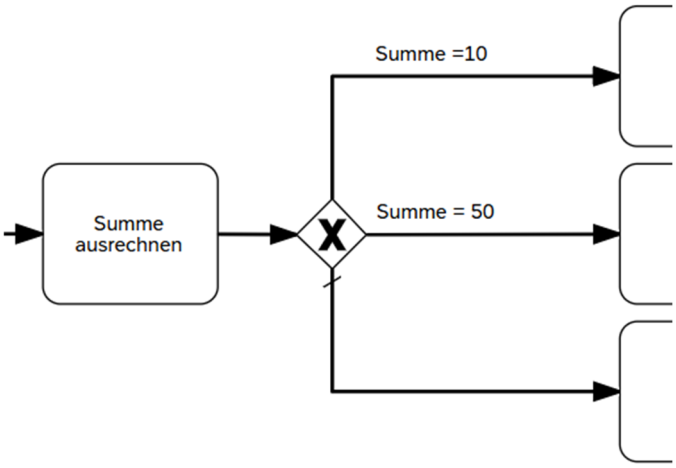


Abbildung 27: Der Default-Ausgang wird gewählt, wenn keine der anderen Bedingungen zutrifft

5.2 Zusammenführungen

Zur Zusammenführung alternativer Pfade wird in der Regel ebenfalls ein Gateway verwendet, wie dies in Abbildung 28 gemacht wurde.

Beim Beispiel in Abbildung 29 münden mehrere Sequenzflüsse in eine Aktivität. Dies ist gleichbedeutend mit dem zusammenführenden Gateway in Abbildung 28. Dennoch wird die Variante mit dem Gateway empfohlen, da hier deutlicher sichtbar wird, dass es sich um eine exklusive Zusammenführung handelt, d. h. der Ablauf wird an dieser Stelle direkt fortgesetzt, sobald eine einzige der drei vorausgehenden Aktivitäten abgeschlossen ist.

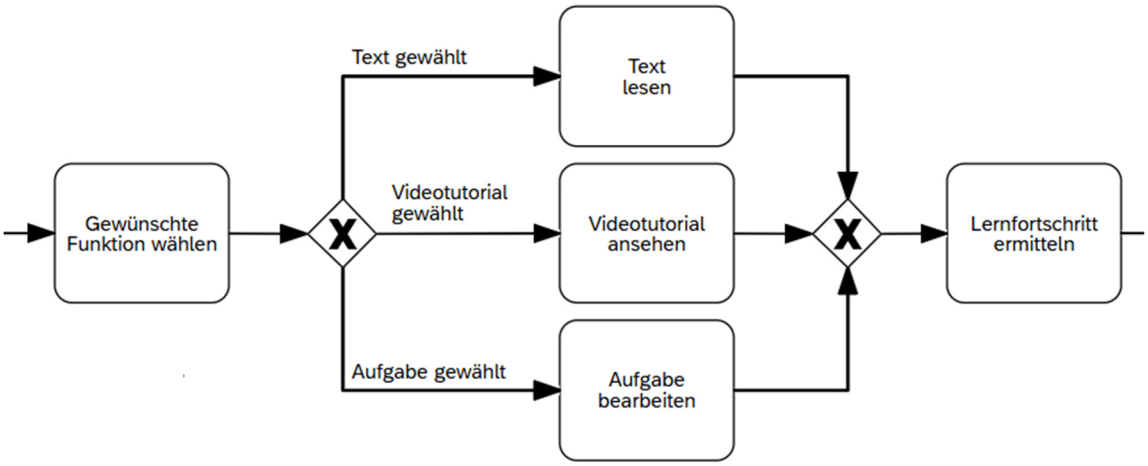


Abbildung 28: Zusammenführung exklusiver Pfade (rechter Gateway)

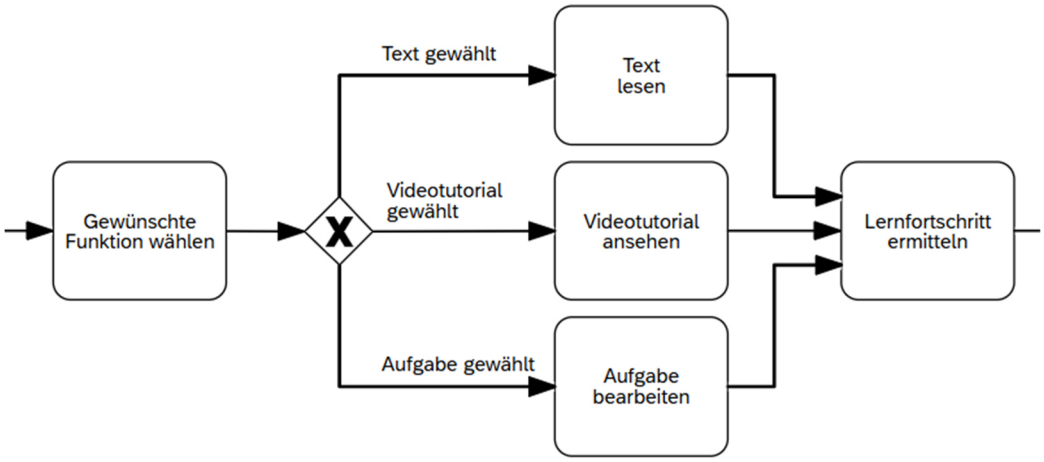


Abbildung 29: Zusammenführung exklusiver Pfade ohne Gateway

Gelegentlich kommt es vor, dass auf eine Zusammenführung direkt wieder eine Verzweigung folgt. Dann ist es zulässig, beides in einem Gateway zusammenzufassen (Abbildung 30).

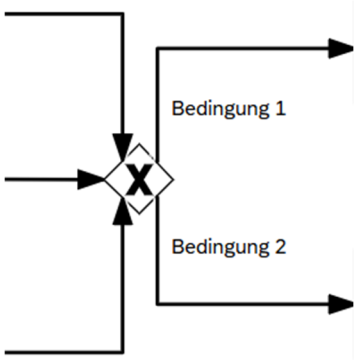


Abbildung 30: Kombination von Zusammenführung und Verzweigung

Das ist etwas kompakter und platzsparender als die separate Modellierung von Zusammenführung und Verzweigung doch oft für den Leser auch weniger gut nachvollziehbar. Daher wird die getrennte Modellierung empfohlen (Abbildung 31).

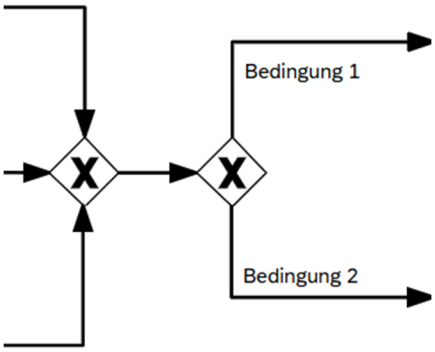


Abbildung 31: Getrennte Modellierung von Zusammenführung und Verzweigung

5.3 Weitere Elemente von BPMN-Diagrammen

Der Bewerbungsprozess in Abbildung 32, der einige weitere Elemente enthält, läuft folgendermaßen ab: Wenn eine Bewerbung eingegangen ist, wird sie vom Sekretariat der Personalabteilung erfasst. Anschließend passieren zwei Dinge parallel: Zum einen prüft der Personalreferent, der ebenfalls zu Personalabteilung gehört, die Bewerbung formal, zum anderen prüft die Fachabteilung die prinzipielle Eignung. Ist der Bewerber prinzipiell geeignet, wird die Bewerbung anschließend inhaltlich geprüft. Ist die prinzipielle Eignung hingegen nicht gegeben, dann wird die inhaltliche Prüfung übersprungen.

Wenn die parallel durchgeführten Aufgaben von Personalreferent und Fachabteilung beide abgeschlossen sind, wird unterschieden, ob beide Prüfungen positiv ausgefallen sind, oder nicht. In letzterem Fall sagt das Sekretariat dem Bewerber ab. Ansonsten lädt das Sekretariat den Bewerber ein, und der Prozess ist beendet. Anschließend wird darauf gewartet, dass der Bewerber den Interviewtermin bestätigt. Wenn das geschehen ist, ist der Prozess ebenfalls beendet.

In diesem Beispiel sind folgende neue Elemente enthalten:

- Paralleler Gateway: Parallele Gateways sind mit einem „+“ gekennzeichnet. Auch parallele Gateways können entweder als Verzweigungen oder als Zusammenführungen eingesetzt werden.
- Nachrichtenempfangendes Zwischenereignis, dargestellt durch einen Kreis mit doppelter Umrandung und einem weißen Briefumschlag-Symbol.
- Senkrechte Lanes: Die Lanes in einem Diagramm können entweder waagrecht (wie in Abbildung 22) oder senkrecht verlaufen.
- Unterteilte Lanes: Wie man am Beispiel der Personalabteilung sieht, können Lanes in weitere Lanes unterteilt werden. So sind die Bereiche Sekretariat und Personalreferent beide Teilbereiche der Personalabteilung.

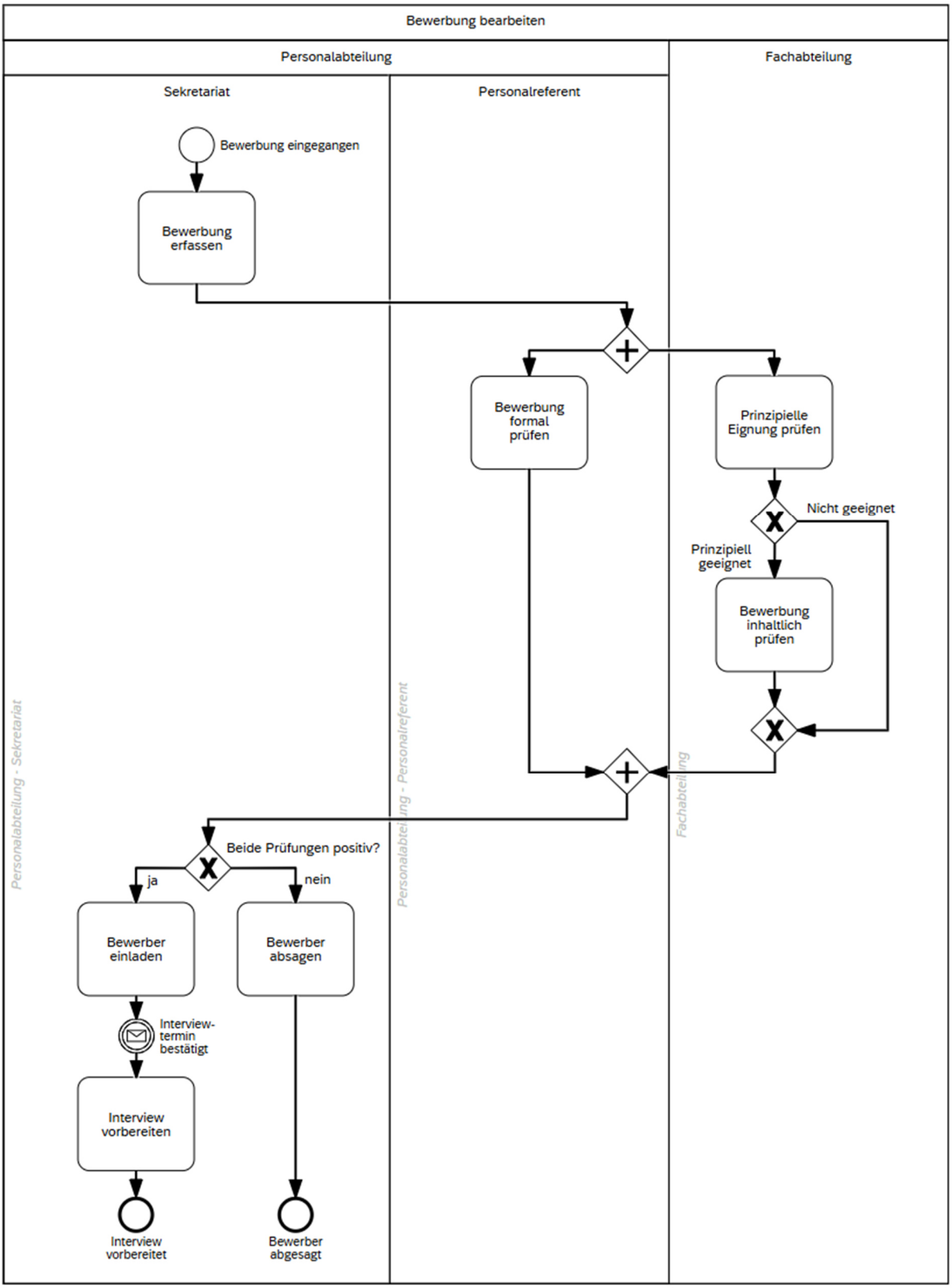


Abbildung 32: Beispiel: Bewerbungsprozess

5.4 Ereignisse

Der Bewerbungsprozess beginnt mit dem Startereignis „Bewerbung eingegangen". Jedes Mal, wenn eine neue Bewerbung eingegangen ist, wird eine neue Instanz des Prozesses gestartet.

Zwischenereignisse sind Ereignisse, die innerhalb des Prozesses auftreten können (Abbildung 33). So wird nach dem Einladen eines Bewerbers auf das Ereignis „Interviewtermin bestätigt" gewartet. Erst wenn dieses Ereignis eingetreten ist, geht der Ablauf mit dem Task „Interview vorbereiten" weiter.

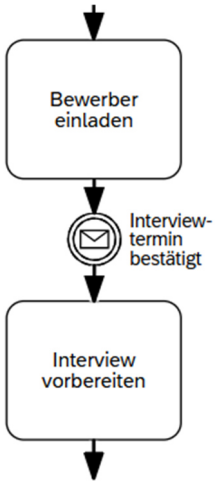


Abbildung 33: Warten auf das Eintreten eines Ereignisses

Zwischenereignisse haben eine doppelte Umrandung. Zudem enthält ein Zwischenereignis in der Regel ein Icon. Das hier verwendete helle Briefumschlag-Symbol kennzeichnet ein nachrichtenempfangendes Ereignis, an dem auf den Eingang einer Nachricht gewartet wird.

Es gibt als Gegenstück auch das nachrichtensendende Ereignis, das ein dunkles Briefumschlagsymbol enthält. Tritt in einem Prozess ein solches Ereignis ein, so wird die betreffende Nachricht verschickt, und der Prozess wird sofort weitergeführt. An sendenden Ereignissen wird also nicht gewartet.

Abbildung 34: Nachrichtensendendes Ereignis

Würde man auch die Aktivitäten des Bewerbers als Prozess modellieren, so könnte man darin das sendende Ereignis „Interviewtermin bestätigt" verwenden. Wenn dann im Bewerberprozess dieses Ereignis ausgelöst wird, wird die Nachricht an das empfangende Ereignis im Bewerbungsprozess verschickt, und der Prozess wird fortgesetzt.

Die Art der Nachricht kann ganz unterschiedlich sein, z. B. E-Mail, Briefpost oder der Aufruf einer Methode.

Nachrichtenereignisse können nicht nur Zwischenereignisse innerhalb von Prozessen sein. Ein empfangendes Nachrichtenereignis kann auch als Startereignis verwendet werden, ein

sendendes Nachrichtenereignis als Endereignis, das beim Abschluss des Prozesss noch eine Nachricht versendet (Abbildung 35). Bei einem nachrichtenempfangenden Startereignis hingegen startet das Eintreffen einer betreffenden Nachricht eine neue Prozessinstanz.

Abbildung 35: Nachrichtenereignisse als empfangende Start- und Zwischenereignisse (links) und als sendende Zwischen- und Endereignisse (rechts)

Auch das Erreichen eines bestimmten Zeitpunkts oder der Ablauf einer festgelegten Zeitspanne stellen Ereignisse dar. Solche zeitlichen Ereignisse werden mit einer Uhr gekennzeichnet. Der Prozess der Gehaltszahlung in Abbildung 36 startet immer am Monatsende.

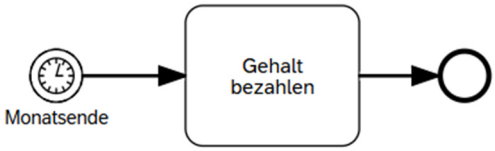


Abbildung 36: Start eines Ablaufs zu einem bestimmten Zeitpunkt

Ebenso wie nachrichtenempfangendee Ereignisse können auch zeitliche Ereignisse als Zwischenereignisse genutzt werden. Es wird dann beim Erreichen eines solchen Ereignisses bis zu einem bestimmten Zeitpunkt oder bis zum Ablauf einer Frist gewartet (Abbildung 37).

Abbildung 37: Zeitliches Zwischenereignis

Eine weitere Art von Ereignis ist das Eintreffen eines Signals. Solch ein Signal kann aus einem anderen Prozess oder von einer anderen Stelle desselben Prozesses gesendet werden. In Abbildung 38 wird das Signal „Mitteilung über Serverabschaltung“ gesendet. Das Symbol hierfür ist ein dunkel ausgefülltes Dreieck. Möchte man in einem anderen Prozess auf dieses Signal reagieren, so modelliert man ein empfangendes Ereignis mit derselben Bezeichnung „Mitteilung Serverabschaltung“.

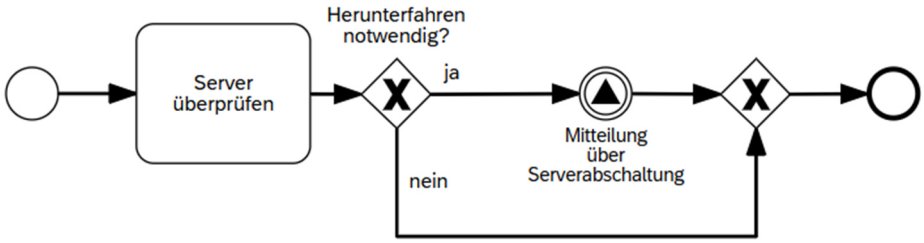


Abbildung 38: Senden eines Signals

In Abbildung 41 wird ein empfangendes Startereignis verwendet. Es ist durch ein helles Dreieck gekennzeichnet. Im Unterschied zu einem empfangenden Zwischenereignis wird hier nicht in einer bereits existierenden Prozessinstanz auf ein Ereignis gewartet, sondern es wird beim Eintreffen des Signals eine neue Prozessinstanz gestartet.

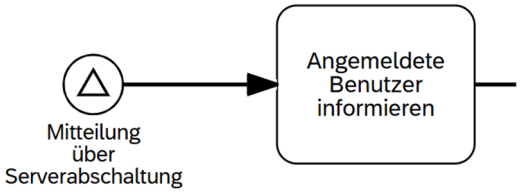


Abbildung 39: Das Empfangen eines Signals startet eine neue Prozessinstanz

Wie bei den Nachrichtenereignissen können Signalereignisse als sendende Start- und Zwischenereignisse verwendet werden, sowie als sendende Zwischen- und Endereignisse (Abbildung 40).

Abbildung 40: Signalereignisse als empfangende Start- und Zwischenereignisse (links) und als sendende Zwischen- und Endereignisse (rechts)

5.5 Paralleler Gateway

Im Gegensatz zum verzweigenden exklusiven Gateway, bei dem immer nur genau ein Ausgang gewählt wird, werden bei dem mit einem Plus-Symbol versehenen verzweigenden parallelen Gateway immer alle Ausgänge zugleich gewählt. Die ausgehenden Pfade werden parallel durchlaufen. „Parallel" muss dabei nicht „gleichzeitig" heißen. Die Aktionen in den parallelen Abschnitten des Prozesses können durchaus nacheinander oder nur teilweise zeitlich überlappend ausgeführt werden. Bevor es an einer folgenden parallelen Zusammenführung weitergeht, müssen aber alle parallelen Abschnitte komplett durchlaufen worden sein.

So wird der Beispielprozess nach der parallelen Zusammenführung erst dann fortgesetzt, wenn die Bewerbung sowohl formal geprüft wurde, als auch hinsichtlich ihrer prinzipiellen Eignung und ggf. inhaltlich geprüft wurde (Abbildung 41).

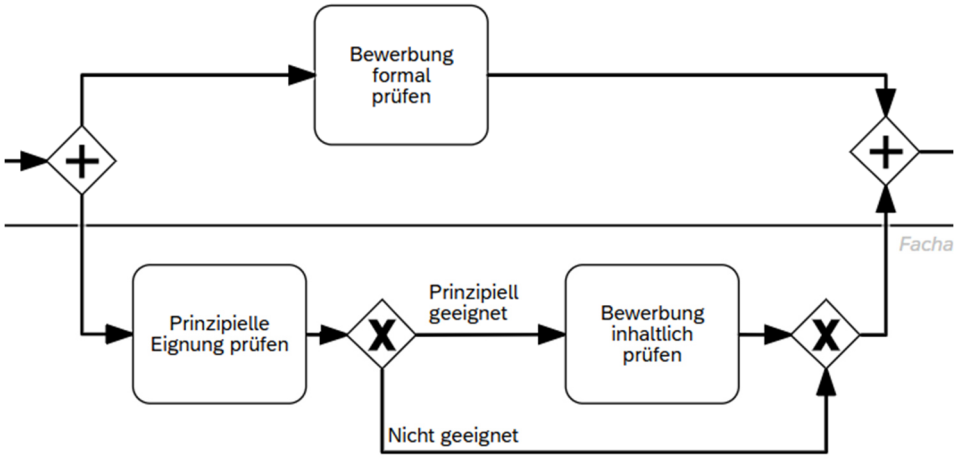


Abbildung 41: Parallele Aufsplittung und Zusammenführung

5.6 Inklusiver Gateway

Modelliert man eine Verzweigung mit Hilfe eines exklusiven Gateway, so wird immer *genau einer* von mehreren Ausgängen selektiert. Bei einer Aufsplittung durch einen parallelen Gateway werden immer *alle* Ausgänge gewählt.

Bei einem inklusiven Gateway – dargestellt als Raute mit einem Kreis darin – wird immer *mindestens ein* Ausgang gewählt. Es können aber auch mehrere Ausgänge zugleich gewählt werden.

Wie beim exklusiven Gateway werden die ausgehenden Kanten mit Bedingungen versehen. Allerdings dürfen diese Bedingungen sich nun überlappen, d. h. es können auch mehrere Bedingungen gleichzeitig zutreffen. In diesem Fall werden all diejenigen Ausgänge zugleich gewählt, deren Bedingungen wahr sind.

In Abbildung 42 wird der untere Ausgang immer gewählt, während der mittlere Ausgang nur gewählt wird, wenn eine Variable a einen Wert größer als 100 hat. Entsprechend wird der obere Ausgang nur gewählt, wenn der Wert der Variablen b größer als 50 ist. Je nach den Werten von a und b werden also ein, zwei oder drei Ausgänge gewählt.

Für die Zusammenführung der entstehenden bedingt parallelen Pfade wird in der Regel wiederum ein inklusiver Gateway verwendet. Dort darf nun nicht mehr davon ausgegangen werden, dass immer alle eingehenden Pfade durchlaufen sein müssen, bevor der Prozess fortgesetzt wird. Wurde aufgrund der Bedingungen etwa nur einer der parallelen Pfade ausgewählt, so muss der Prozess bereits fortgesetzt werden, wenn dieser eine Pfad durchlaufen wurde. Ein Warten auf die anderen Pfade wäre sinnlos, da diese ja nicht ausgewählt wurden.

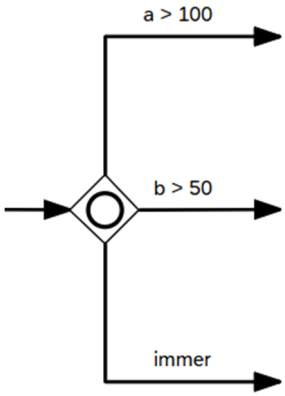


Abbildung 42: Bei einem inklusiven Gateway werden ein oder mehrere Ausgänge gewählt

5.7 Unterschiedliche Arten von Endereignissen

In den bisherigen Ausführungen wurden gewöhnliche Endereignisse verwendet. Wird ein solches Endereignis erreicht, so ist der betreffende Pfad abgeschlossen. Wurden parallelele oder inklusive Verzweigungen verwendet, so kann es jedoch mehrere aktive Pfade geben. Der

Prozess ist erst dann komplett beendet, wenn in jedem aktiven Pfad ein Endereignis erreicht wurde.

In manchen Fällen ist es jedoch gewünscht, dass beim Erreichen eines Endereignisses der gesamte Prozess sofort beendet wird. Hierfür kann man ein sogenanntes Terminierungs-Endereignis verwenden, erkennbar an einem schwarz gefüllten Kreis.

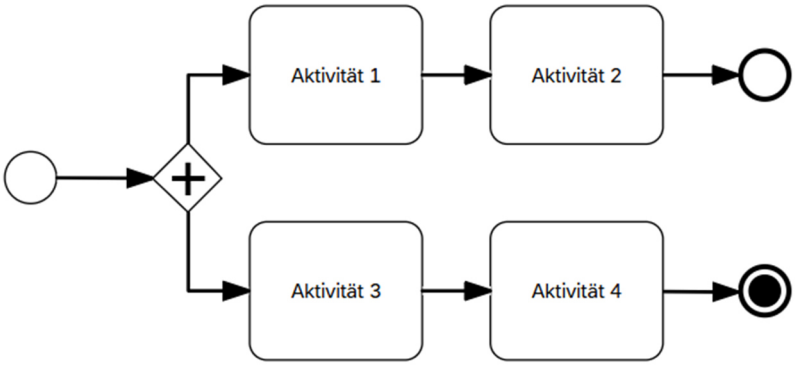


Abbildung 43: Endknoten für die Gesamtaktivität (links) und für einen Zweig (rechts)

Im Beispiel von Abbildung 43 wird der Ablauf durch den parallelen Gateway in zwei parallele Pfade aufgeteilt. Falls der untere Ast zuerst abgearbeitet wird, und im oberen Ast beispielsweise Aktivität 4 noch nicht ausgeführt wurde, so wird das Terminierungsereignis im unteren Pfad erreicht. Da dieses sofort den gesamten Ablauf beendet, wird Aktion 4 nun nicht mehr durchgeführt.

Anders liegt der Fall, wenn der obere Ast zuerst abgearbeitet wird. Beim Erreichen des gewöhnlichen Endereignisses, wird nur dieser obere Ast beendet. Der sonstige Prozess läuft hingegen weiter, d. h. der untere Ast wird noch fertig abgearbeitet.

5.8 Unterprozesse

Eine umfangreichere Aktivität kann selbst noch einmal einen detaillierten Ablauf umfassen. Hierzu kennzeichnet man die Aktion mit einem kleinen Plus-Symbol. Dann weiß man, dass es für diese Aktion ein eigenes BPMN-Diagramm gibt, das den Unterprozess enthält (Abbildung 44). Wird innerhalb des übergeordneten Prozesses die Aktivität „Bewerbung formal prüfen“ erreicht, so wird der Unterprozess durchgeführt. Wenn dieser abgeschlossen ist, ist damit auch die Aktion „Bewerbung formal prüfen“ beendet, und der übergeordnete Prozess wird

fortgesetzt.

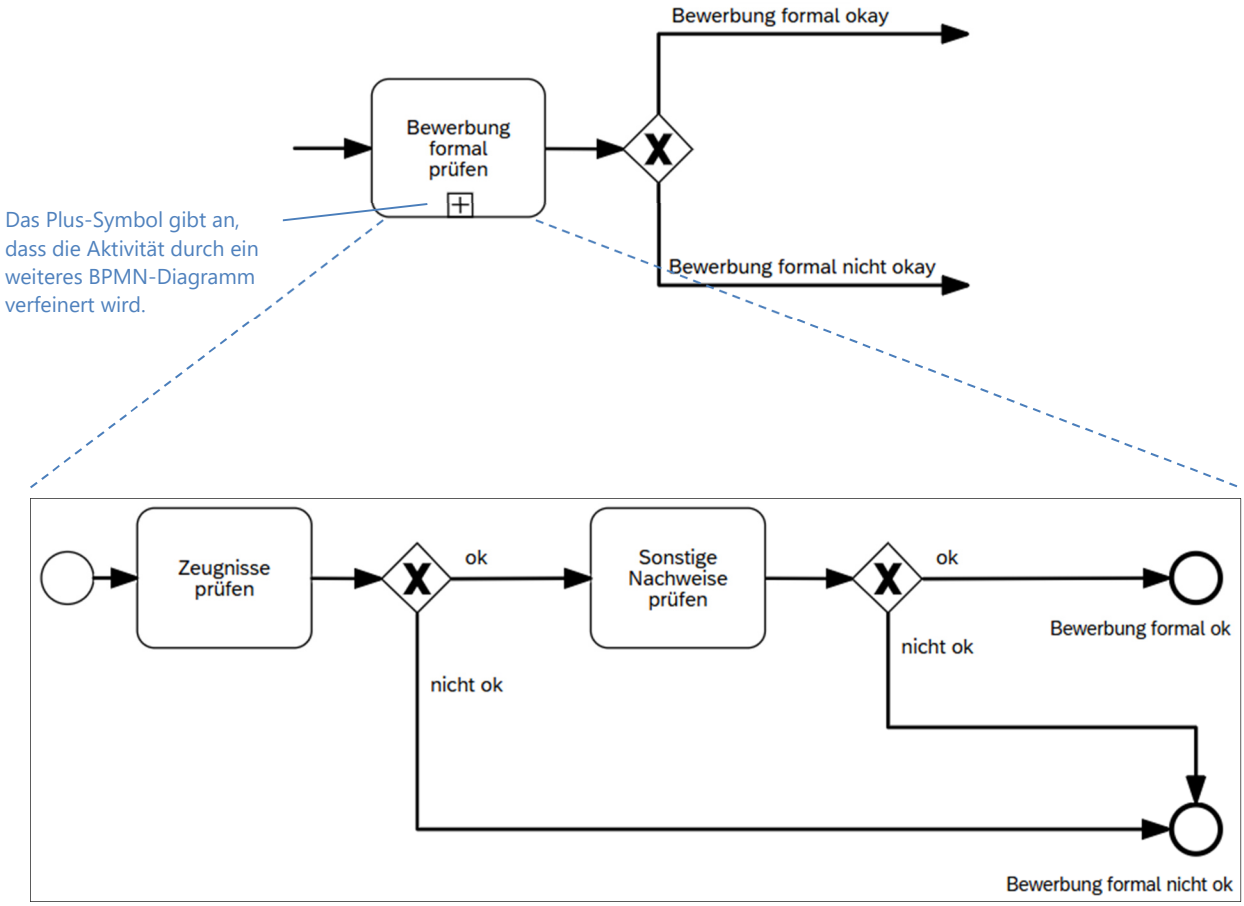


Abbildung 44: Das hinterlegte Diagramm zeigt, was innerhalb einer Aktivität passiert

Manche BPMN-Modellierungswerkzeuge bieten die Möglichkeit, das Diagramm des Unterprozesses mit der Aktivität zu verknüpfen, so dass man das hinterlegte Diagramm per Mausklick öffnen kann.

5.9 Objektflüsse

Gelegentlich möchte man darstellen, welche Daten und Informationen in einem Ablauf erzeugt und verwendet werden. Hierzu kann man die betreffenden Daten-Objekte in das Diagramm einzeichnen.

In Abbildung 45 wird zunächst eine Anfrage erfasst. Hierbei wird ein Objekt vom Typ „Anfrage“ erstellt. Anschließend wird die Aktion „Angebot erstellen“ durchgeführt. Diese Aktion greift auf das Anfrage-Objekt zu und erstellt ein Objekt vom Typ „Angebot“. Dieses Objekt befindet sich zunächst im Zustand „zu prüfen“.

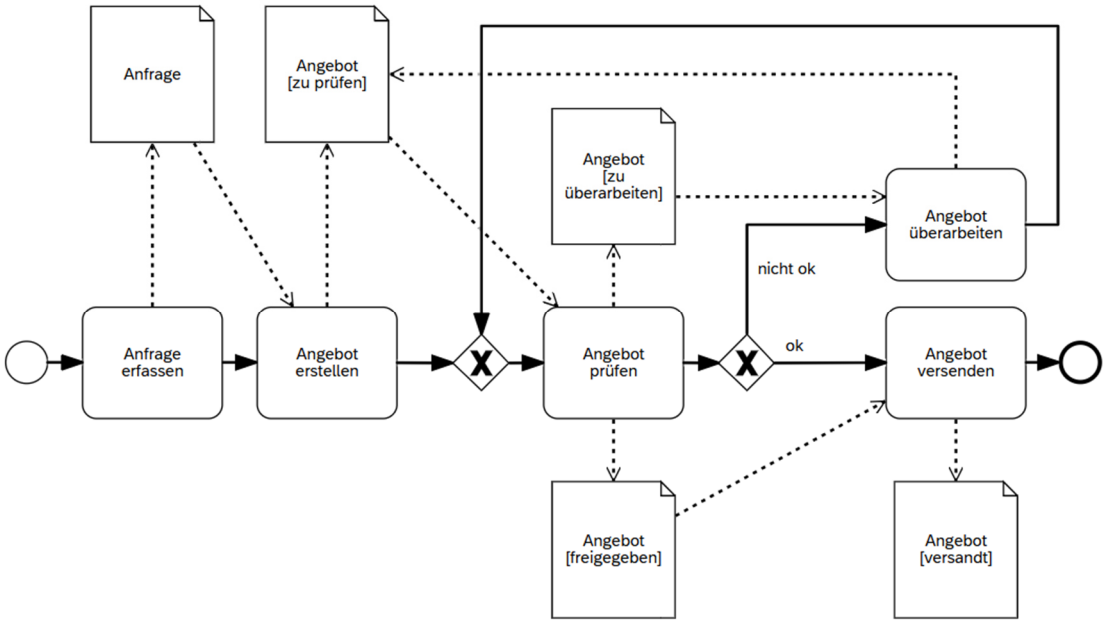


Abbildung 45: BPMN-Diagramm ergänzt um Objektflüsse

Die nächste Aktivität, „Angebot prüfen", greift auf das Angebot-Objekt zu und überführt es in den Zustand „freigegeben" oder „zu überarbeiten". Ein zu überarbeitendes Angebot wird von der Aktivität „Angebot überarbeiten" wieder in den Zustand „erstellt" überführt, woraufhin es erneut geprüft wird. Ist das Angebot freigegeben, so wird es von der nächsten Aktivität versandt und damit in den Zustand „versandt" überführt.

5.10 Einsatzmöglichkeiten für Prozessmodelle

Prozessdiagramme dienen der Darstellung von Abläufen. Im Zusammenhang mit der Softwareentwicklung können Abläufe auf unterschiedlichen Ebenen betrachtet werden:

1. Übergreifende Abläufe (Geschäftsprozesse)
   Hierbei handelt es sich um organisatorische Abläufe, die teilweise mit und teilweise ohne Softwareunterstützung durchgeführt werden können. Sie können auch über verschiedene IT-Systeme hinweg verlaufen. Bei der Durchführung eines Geschäftsprozesses können ggf. mehrere Use-Cases eines Softwaresystems genutzt werden.

2. Abläufe innerhalb einzelner Use-Cases
   Hier kann man Prozessmodelle einsetzen um mögliche Reihenfolgen der Benutzer- und Systemaktionen beim Bearbeiten eines Use-Case zu modellieren und damit ggf. die textuelle Use-Case-Beschreibung zu ergänzen.

3. Abläufe in Programmcode
   Schließlich können Prozessmodelle verwendet werden um den Kontrollfluss beim Ablauf eines Programmes oder einer einzelnen Methode zu visualisieren.

5.11 Modellierung eines Geschäftsprozesses

Abbildung 46 zeigt ein – vereinfachtes – Modell eines Geschäftsprozesses zur Abwicklung eines Auftrags bei einem Händler. Einige der darin enthaltenen Aktivitäten werden möglicherweise manuell ausgeführt, wie das Auslagern und das Versenden der Ware. Andere werden zumeist durch Software unterstützt. Häufig kommen dabei verschiedene Systeme zum Einsatz. So könnten etwa die Auftragserfassung und das Drucken des Lieferscheins mit Hilfe eines Auftragsbearbeitungssystems erfolgen. Zum Erstellen der Rechnung wird vielleicht ein separates System eingesetzt.

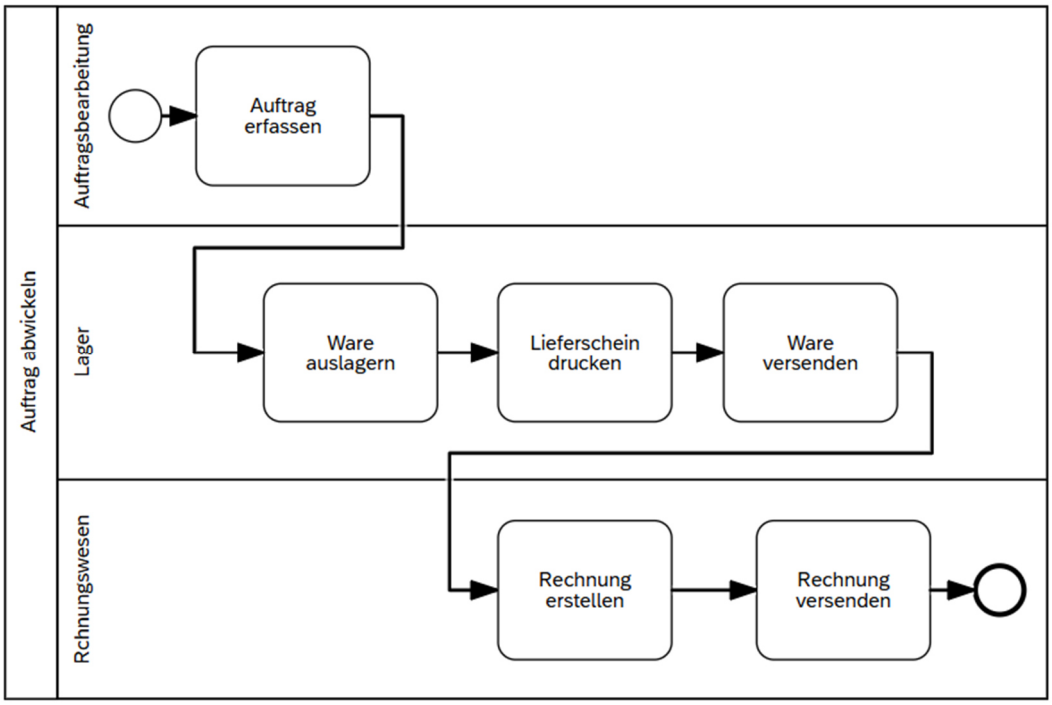


Abbildung 46: Geschäftsprozess zur Auftragsabwicklung

Eine derartige Modellierung von Geschäftsprozessen erfolgt meist im Vorfeld der eigentlichen Software-Entwicklung, um die Abläufe des Unternehmens zu verstehen und zu verbessern. Vielfach ist es erforderlich, die Prozesse zunächst inhaltlich zu verändern, bevor man eine neue Software zu ihrer Unterstützung entwickelt. Ein schlechter Ablauf wird durch Softwareunterstützung möglicherweise schneller, aber nicht unbedingt besser.

Die Reorganisation von Geschäftsprozessen ist nicht Gegenstand dieses Skripts. Man sollte sich aber bewusst sein, dass der Erfolg von Projekten zur Entwicklung und Einführung von Software für Unternehmen mindestens genauso stark von der geeigneten Gestaltung der Geschäftsprozesse abhängt wie von der eigentlichen Software. Letztlich entsteht der Nutzen, den ein Unternehmen von Software hat, durch die Unterstützung der Prozesse – z. B. in Form reduzierter Kosten, höherer Umsätze oder einer geringeren Zahl von Fehlern. Deswegen ist es für Software-Entwickler unerlässlich, zu verstehen, wozu die Anwender die Software einsetzen.

Geschäftsprozessmodelle stellen daher auch eine wichtige Quelle für Anforderungen an die Software dar. So können oftmals aus einem Geschäftsprozess eine Reihe wichtiger Use-Cases

abgeleitet werden. In vielen Unternehmen liegen bereits Prozessmodelle vor, die für die Anforderungsermittlung ausgewertet werden können.

Zur Ableitung von Use-Cases aus Geschäftsprozessmodellen kann man jede darin enthaltene Aktivität anhand folgender Kriterien überprüfen:

- Soll die Aktivität mit dem zu entwickelnden System durchgeführt werden?
- Handelt es sich um einen kompletten Use-Case, oder um den Teil eines umfassenderen Use-Case?
- Kann ein Use-Case für mehrere Aktivitäten verwendet werden?
  So kann man Arbeitsschritte wie „Neuen Kunden anlegen", „Kundenadresse ändern" etc. durch einen gemeinsamen Use-Case „Kunde bearbeiten" erledigen lassen.
- Ist es eventuell sinnvoll, eine umfangreiche Aktion durch mehrere Use-Cases zu realisieren?

Oftmals finden sich aber nicht alle erforderlichen Use-Cases in den Prozessmodellen. So wird im Zusammenhang mit der Auftragsabwicklung unter Umständen für das Anlegen von Produktbeschreibungen und anderen Stammdaten kein Prozessmodell erstellt, weil hierfür kein größerer Ablauf erforderlich ist. Trotzdem sind diese Use-Cases für das Funktionieren des Systems notwendig. Man findet sie, indem man sich für jeden der anderen Use-Cases fragt, wie die dort verwendeten Daten in das System gelangen.

5.12 Ablauf innerhalb eines Use-Case

Auch bei der Durchführung eines Use-Cases findet in der Regel ein kleiner Ablauf statt. Er wird in der Use-Case-Beschreibung dokumentiert, wobei zuerst der Ablauf im Normalfall aufgeführt wird und anschließend mögliche Erweiterungen und Alternativen beschrieben werden (vgl. Abschnitt 4.2). Der Ablauf im Normalfall ist eine rein sequenzielle Abfolge von Einzelschritten. Im Zusammenspiel mit den Erweiterungen und Alternativen ergibt sich unter Umständen eine Vielfalt möglicher Ablaufreihenfolgen. Diese Möglichkeiten können mit Hilfe eines Prozessmodells veranschaulicht werden.

Tabelle 15 enthält noch einmal die Beschreibung des Use-Cases „Fahrzeug reservieren" aus Kapitel 4. Aus dieser Beschreibung wurde das Aktivitätsdiagramm in Abbildung 47 entwickelt.

Durchläuft man in Abbildung 47 die auf der linken Seite angeordneten Aktionen, so erhält man den beschriebenen Ablauf im Normalfall. Zur übersichtlicheren Darstellung wurden jeweils einige zusammengehörige Schritte aus Tabelle 15 zu Aktivitäten zusammengefasst.

Die Erweiterungen und Alternativen aus Tabelle 15 sind rechts vom Normalablauf dargestellt. Trifft jeweils die Bedingung für eine Erweiterung oder Alternative zu, so wird dorthin verzweigt.

Name	Fahrzeug reservieren
Vorbedingungen	Keine
Nachbed. im Erfolgsfall	Neue Reservierung angelegt
Nachbedingung bei Fehlschlag	Keine neue Reservierung angelegt
Akteure	Mitarbeiter
Auslösendes Ereignis	Mitarbeiter wählt „Fahrzeug reservieren“
Beschreibung	1. Der Mitarbeiter gibt Fahrzeugklasse und Zeitraum an. 2. Das System zeigt verfügbare Modelle und Preise an. 3. Der Mitarbeiter wählt das gewünschte Modell aus. 4. Der Mitarbeiter gibt den Kundennamen ein. 5. Das System zeigt passende Kunden an. 6. Der Mitarbeiter wählt den betreffenden Kunden aus. 7. Das System zeigt die Kundendaten an. 8. Der Mitarbeiter bestätigt die Reservierung.
Erweiterungen / Alternativen	a) Bis Schritt 8 ist jederzeit ein Abbruch möglich b) Zu Schritt 2: Es ist kein Fahrzeug verfügbar. Die Eingaben können geändert werden. c) Zu Schritt 4 bis 7: Der Kunde ist noch nicht im System. Der Mitarbeiter gibt die Kundendaten ein (Vorname, Nachname, Geburtsdatum, Adresse, Ausweisnummer) und bestätigt die Eingabe. Falls die Daten unvollständig sind, weist das System darauf hin und fordert zur Ergänzung auf.

Tabelle 15: Beschreibung des Use-Case „Fahrzeug reservieren“ (aus Tabelle 6)

Eine Besonderheit stellt die Modellierung der jederzeitigen Abbruchmöglichkeit dar. Zwar könnte man nach jeder Aktion in einer Verzweigung überprüfen, ob „Abbruch“ gewählt wurde, doch würde dies das Diagramm sehr unübersichtlich machen. Daher wird der vom Benutzer ausgelöste Abbruch als Signal-Ereignis aufgefasst. Rechts unten im Diagramm ist modelliert, dass bei Eintritt dieses Ereignisses ein Terminierungsereigns erreicht wird. Damit wird sofort der gesamte Ablauf abgebrochen.

Der Ablauf innerhalb eines Use-Cases kann in unterschiedlicher Detaillierungsstufe modelliert werden. In Abbildung 47 wurden die zusammengehörigen Aktionen von Benutzer und System zu einigen grundlegenden Aktionen zusammengefasst. Damit erhält man ein recht übersichtliches Modell, das den grundlegenden Ablauf recht gut erkennen lässt.

Man kann das Zusammenspiel von Benutzer und System aber auch ganz detailliert ausmodellieren, wie dies in Abbildung 48 gemacht wurde. Dieses Diagramm ist so detailliert, dass es weitgehend als Ersatz für die textuelle Beschreibung verwendet werden kann.

Andererseits ist das Diagramm deutlich umfangreicher und komplexer als das aus Abbildung 47 und damit auch schwerer verständlich. Meist empfiehlt sich daher eher eine Kombination aus Text und Diagramm.

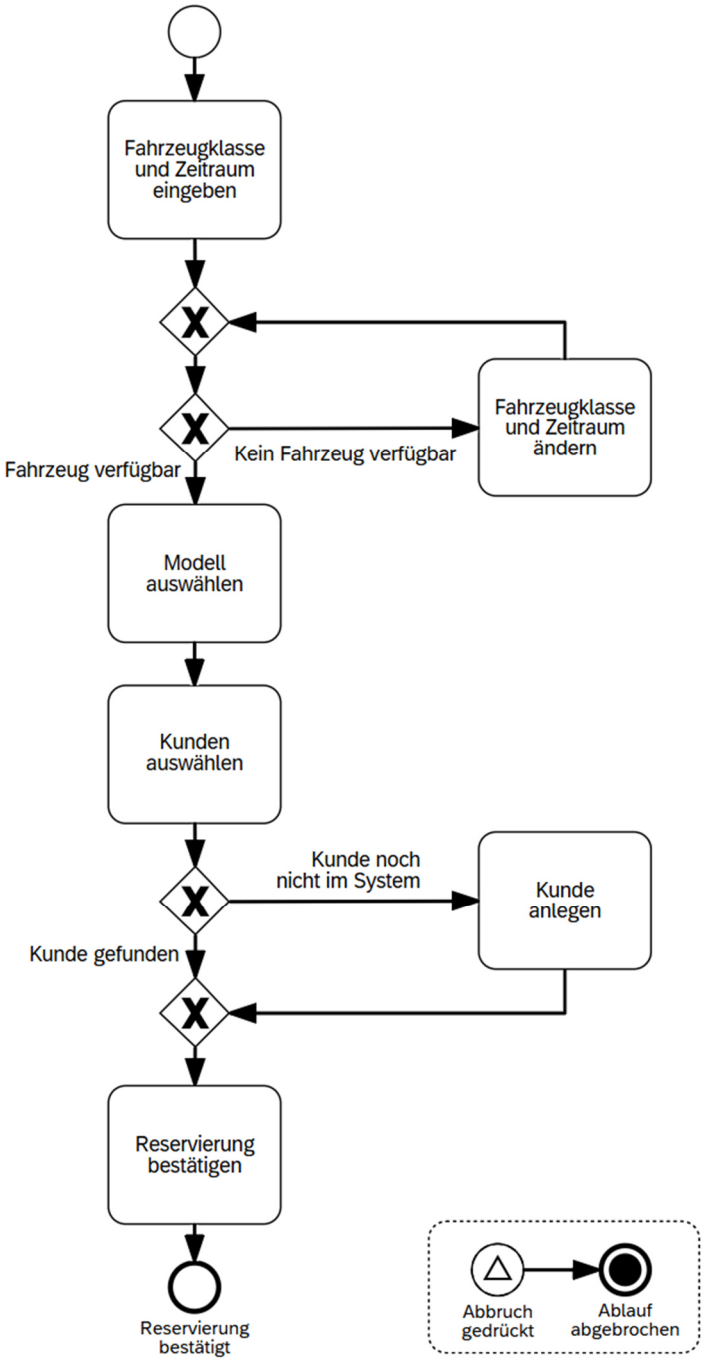


Abbildung 47: Ablauf innerhalb des Use-Case „Fahrzeug reservieren“

Prinzipiell sollten Prozessdiagramme nur dann zur Visualisierung von Abläufen innerhalb von Use-Cases verwendet werden, wenn sie hilfreich für den Leser sind, den betreffenden Ablauf besser zu verstehen. In einfachen Fällen genügt daher die textuelle Beschreibung.

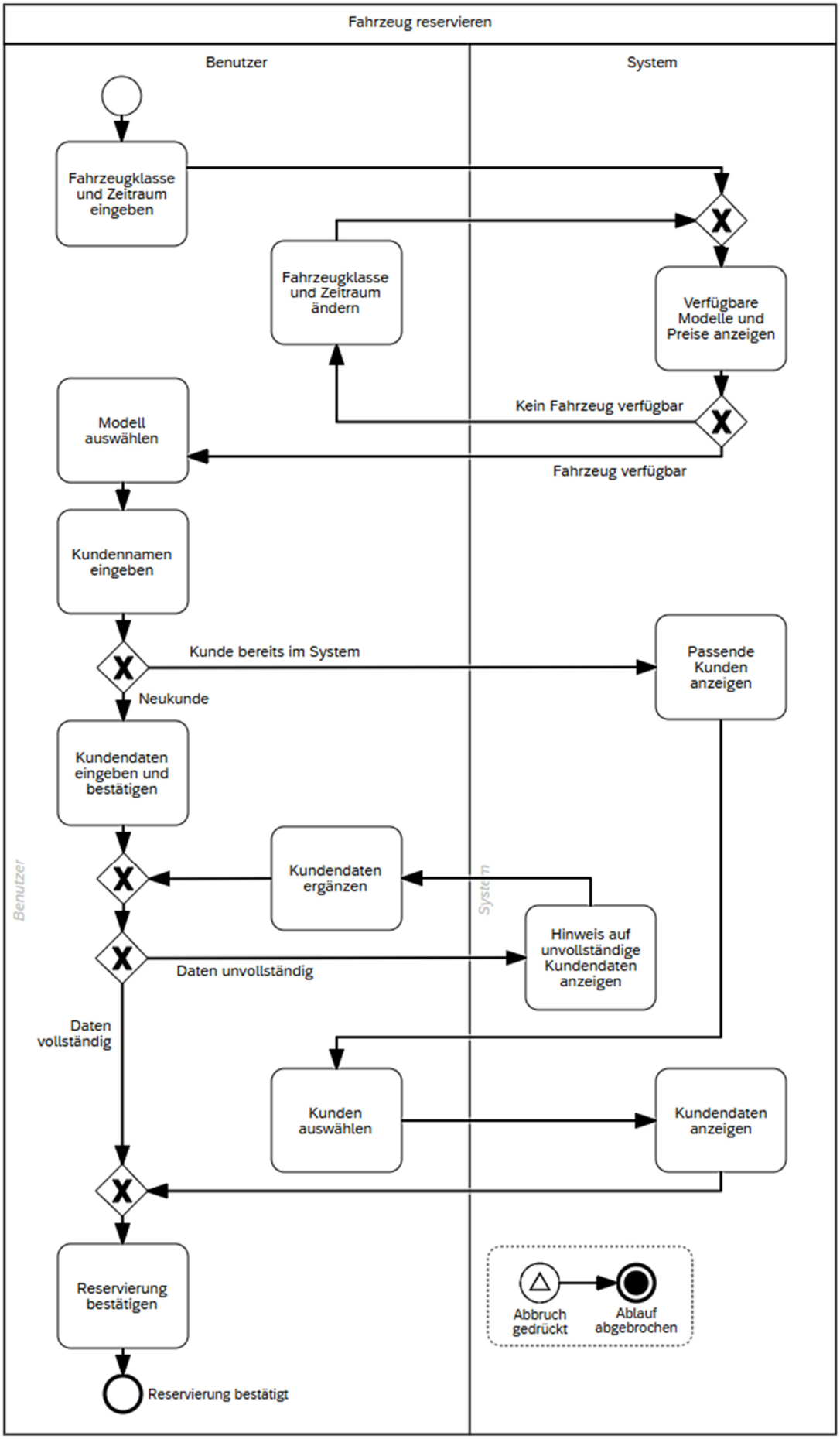


Abbildung 48: Ablauf innerhalb des Use-Case „Fahrzeug reservieren“ – detaillierteres Modell

5.13 Kontrollfluss von Programmcode

Listing 1 enthält einen kleinen Ausschnitt einer Java-Methode mit einer for-Schleife.

```
aktion1();
for(int i=1; i<=100; i++){
    aktion2();
    aktion3();
}
```

Listing 1: Codeausschnitt mit for-Schleife

Der Ablauf dieses Code-Segments kann wie in **Abbildung 49** modelliert werden. Die Inititialisierung der Variablen „i" mit dem Wert 1 und die Inkrementierung um 1 am Ende jedes Schleifendurchlaufs sind explizit als eigene Tasks modelliert worden. Trifft die Bedingung „i<=100" zu, so wird an der Verzweigung der rechte Ausgang gewählt und der Schleifenrumpf mit den Aktionen 2 und 3 und der Inkrementierung von i wird ausgeführt.

Anschließend erfolgt ein Rücksprung zur erneuten Überprüfung der Bedingung. Trifft sie nicht mehr zu, so wird der linke Ausgang der Verzweigung gewählt, der zur Beendigung des Ablaufs führt.

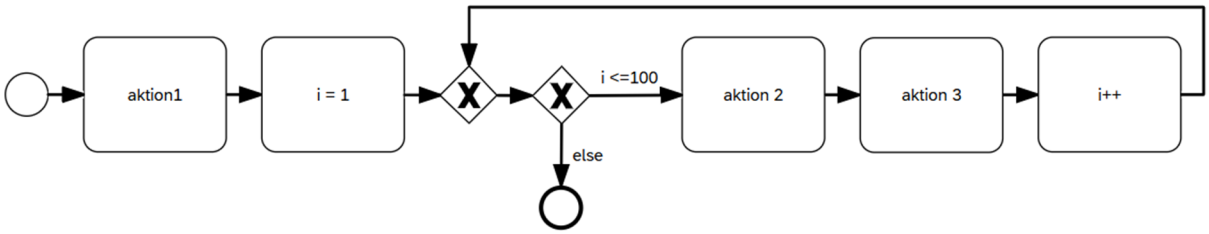


Abbildung 49: Modellierung einer for-Schleife

5.14 Mehr zu BPMN

Die BPMN enthält noch viele weitere Elemente und Konstrukte, mit denen auch komplexe Ablauflogik modelliert werden kann. Sie werden insbesondere eingesetzt, wenn man detaillierte Geschäftsprozess-Modelle erstellen möchte, die von einer Workflow-Engine direkt ausgeführt werden können, um das Zusammenspiel von automatisierten Funktionen und menschlichen Arbeitsschritten zu steuern.

6 Klassendiagramme

UML Klassendiagramme sind ein zentrales Beschreibungsmittel in der objektorientierten Analyse und im objektorientierten Entwurf. Sie beschreiben, welche Daten (Attribute) und welche Funktionen (Methoden) verwendet werden – und insbesondere, wie diese in Form von Objekten und ihren Beziehungen strukturiert werden. Ein wesentlicher Vorteil besteht darin, dass dieselben Konzepte wie in der objektorientierten Programmierung verwendet werden. Ein in einer objektorientierten Sprache geschriebenes Programm lässt sich daher direkt als Klassendiagramm darstellen. Umgekehrt kann man ein UML Klassendiagramm leicht in ein Programmgerüst umwandeln.

Klassendiagramme lassen sich aber auch im Rahmen der Analyse einsetzen um erst einmal die fachlichen Zusammenhänge zu beschreiben, die in dem System umgesetzt werden sollen. Ein solches Klassendiagramm wird meist noch nicht direkt in Code überführt, sondern zunächst im Rahmen des Entwurfs verfeinert und um technische Aspekte erweitert. Da man denselben Diagrammtyp verwendet, ist der Übergang von der Analyse in den Entwurf und die Implementierung sehr einfach. Auch für Analyse-Klassendiagramme gilt, dass man sie prinzipiell in Code umsetzen kann. Das entstehende Programm ist dann in technischer Hinsicht noch nicht optimiert, und es fehlen noch viele Bestandteile, wie z. B. die Benutzungsoberfläche oder die Datenhaltung.

In jedem Fall ist es wichtig, das Konzept der Objektorientierung und den Zusammenhang zwischen Klassendiagrammen und ihrer Implementierung zu verstehen. Im Folgenden wird daher immer der Bezug zur objektorientierten Programmierung hergestellt. Als Sprache wird Java verwendet. Wenn man eine andere objektorientierte Sprache beherrscht, ist die Umsetzung in diese Sprache nicht schwierig.

6.1 Einführendes Beispiel

Es wird die folgende Java-Klasse betrachtet:

```
public class Person {
    private String vorname;
    private String nachname;

    public Person(String vorname, String nachname){
        this.vorname = vorname;
        this.nachname = nachname;
    }

    public String getName(){
        return vorname + " " + nachname;
    }

}
```

Listing 2: Klasse „Person“

In dieser Klasse ist definiert, wie Personen-Objekte aufgebaut sind. Jede Person verfügt über einen Vornamen und einen Nachnamen. Außerdem gibt es einen Konstruktor zum Anlegen

von neuen Personen-Objekten. Diesem Konstruktor übergibt man den Vornamen und den Nachnamen der Person als Parameter. Schließlich gibt es die Methode *„getName( )"*, die einen String zurückgibt, der aus Vor- und Nachname besteht, getrennt durch ein Leerzeichen.

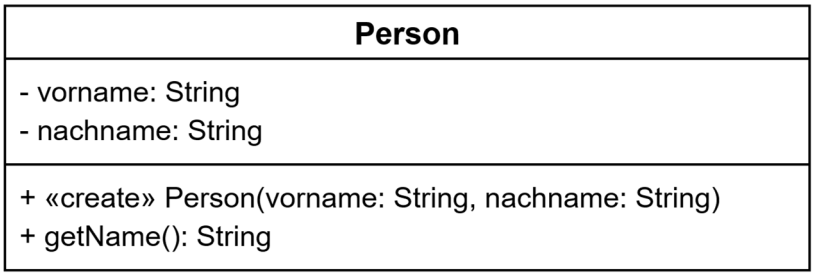


Abbildung 50: Die Klasse „Person" in UML-Darstellung

Im oberen Teil der Klassendarstellung wird der Klassenname angegeben, in der Mitte die Attribute, unten die Operationen oder Methoden.

Für die Attribute sind jeweils die Sichtbarkeit, der Attributname und der Datentyp angegeben. Das Minus-Symbol steht für die Sichtbarkeit *„private"*, d. h. das Attribut ist nur innerhalb der Klasse sichtbar. Man kann die folgenden Sichtbarkeiten darstellen:

Symbol	*Bezeichnung*	*Bedeutung*
-	private	Zugriff nur innerhalb der Klasse
+	public	Zugriff von überall
#	protected	Zugriff nur aus der Klasse und ihren Unterklassen
~	Package	Zugriff nur innerhalb desselben Pakets

Tabelle 16: Darstellung von Sichtbarkeiten

Ist kein Symbol angegeben, so ist nicht definiert, welche Sichtbarkeit das Attribut haben soll.

Wenn ein Attribut einen Default-Wert besitzt, so kann man ihn ebenfalls darstellen, z. B.:

```
- alter: int = 20
```

Bezüglich der Datentypen macht die UML keine Vorgaben. Wenn das Diagramm in eine bestimmte Programmiersprache umgesetzt werden soll, so wird man die Datentypen dieser Sprache verwenden. Ansonsten kann man allgemeine Datentypen verwenden, die es in jeder Sprache gibt, wie String, Integer, Datum. Wenn die Datentypen nicht von Bedeutung sind, z. B. in der Analyse, so kann man sie auch weglassen.

Im unteren Teil des Klassensymbols finden sich die Operationen. Die UML spricht von Operationen anstatt von Methoden. Eine Operation stellt nur die Deklaration dar, also einen Namen, Übergabeparameter usw., aber keine Implementierung. Im Klassendiagramm tauchen nur diese Deklarationen, also die Operationen auf. Eine Methode stellt also die Implementierung einer Operation dar, sie findet sich erst im Programmcode. Im allgemeinen Sprachgebrauch wird jedoch häufig auch für Operationen der Begriff Methode verwendet. Im vorliegenden Skript werden die Begriffe daher ebenfalls synonym gebraucht.

Zunächst wird die Methode „*getName( )*“ betrachtet:

+ getName( ): String

Die Angabe besteht aus der Sichtbarkeit, dem Namen der Methode, der – in diesem Fall leeren – Parameterliste und dem Rückgabetyp. Falls eine Methode keinen Wert zurückgibt, lässt man den Rückgabetyp weg. Die Angabe „*void*“ aus Java muss also nicht übernommen werden.

Der Konstruktor der Klasse Person ist folgendermaßen dargestellt:

+ «create» Person(vorname: String, nachname: String)

Die Angabe *«create»* kennzeichnet die Methode als Konstruktor, d. h. sie erzeugt neue Objekte. In Java heißt der Konstruktor genauso wie die Klasse. Von daher könnte man hier auch auf die *«create»*-Angabe verzichten. Da UML aber unabhängig von Java ist, kann ein Konstruktor auch ganz anders benannt werden, z. B. in einem Analysediagramm. Dann ist die Kennzeichnung ganz nützlich.

Die Parameterliste enthält die beiden benötigten Parameter *vorname* und *nachname*, jeweils mit ihrem Datentyp.

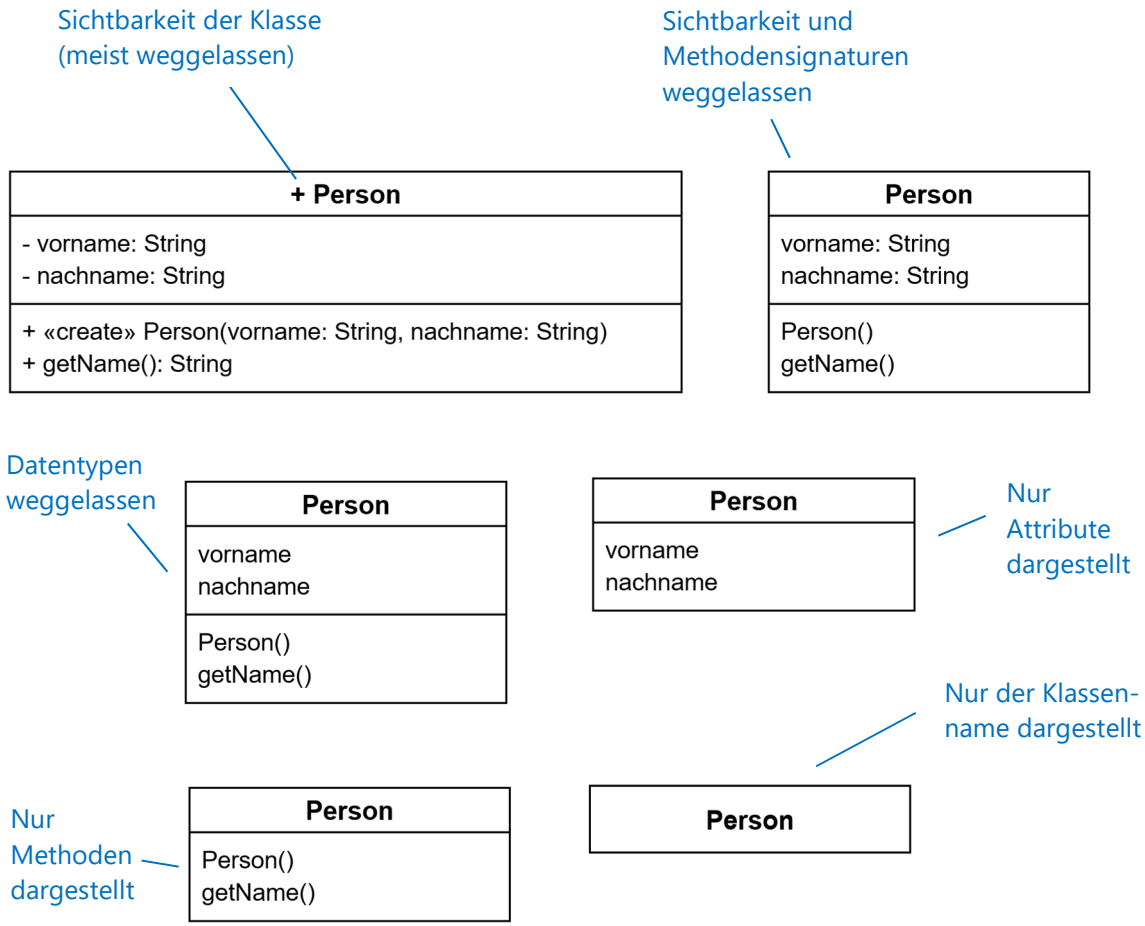


Abbildung 51: Verschiedene Darstellungsmöglichkeiten einer Klasse

Je nach Darstellungszweck kann man in einem Klassendiagramm unterschiedliche Details aufnehmen oder weglassen (Abbildung 51).

So wird man in einem Analysediagramm häufig Details wie Datentypen, Sichtbarkeiten und Parameterlisten weglassen. Im Entwurf und als Dokumentation des Codes sind diese Details eher interessant. Doch auch hier lässt man in Übersichtsdiagrammen oft Details weg, damit die Diagramme übersichtlich bleiben. Man kann auch ein Übersichtsdiagramm ohne Einzelheiten erstellen und zusätzlich noch Detaildiagramme für einzelne Bereiche des Übersichtsdiagramms.

Wie man in Abbildung 51 oben links sieht, kann man auch die Sichtbarkeit der Klasse selbst angeben. Dies wird jedoch eher selten gemacht.

6.2 Modellierung von Objekten

Es wird noch einmal die Klasse Person betrachtet. Möchte man Personen-Objekte anlegen, so ruft man wie im folgenden Beispiel den Konstruktor auf (Listing 3).

```
public class Test {

    public static void main(String[] args) {
        Person p1 = new Person("George", "Orwell");
        Person p2 = new Person("Douglas", "Adams");

        System.out.println(p1.getName());
        System.out.println(p2.getName());
      }
}
```

Listing 3: Anlegen von Objekten der Klasse „Person“

In der *main( )*-Methode der Klasse Test werden zunächst zwei Personenobjekte angelegt und in den Variablen p1 und p2 gespeichert. Anschließend wird von jeder der beiden Personen-Objekte die Methode *„getName( )“* aufgerufen und auf der Konsole ausgegeben.

Die Ausgabe ist dann erwartungsgemäß (Listing 4):

```
    George Orwell
    Douglas Adams
```

Listing 4: Ausgabe der main-Methode aus Listing 3

Auch die Objekte lassen sich in UML darstellen (Abbildung 52).

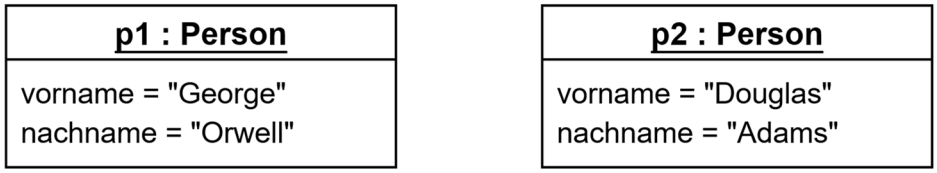


Abbildung 52: Objekte der Klasse „Person“

Die Objektdarstellungen bestehen aus zwei Teilen. Der obere Bereich enthält die Objektbezeichnung und den Klassennamen, der untere die Attribute mit ihren Werten. Die Methoden

werden nicht angegeben. Sie sind in der Klasse definiert und können für jedes Objekt der Klasse aufgerufen werden, wie dies oben für die Methode *„getName( )"* gezeigt wurde.

Die Objektbezeichnung dient zur Unterscheidung, wenn man mehrere Objekte derselben Klasse hat. Sie wird gefolgt von einem Doppelpunkt und dem Namen der Klasse, zu der das Objekt gehört. Wichtig: Bei einem Objekt müssen die Bezeichnung und der Klassenname unterstrichen werden. Hierdurch lassen sich Objekte besser von Klassen unterscheiden.

Die Attribute sind wie in der Klasse angegeben. Da ein Objekt genau über die in seiner Klasse definierten Attribute verfügt, dürfen auch nur diese Attribute eingetragen werden. Im verwendeten Modellierungstool wird auch in Objekten der Datentyp der Attribute mit angegeben, doch kann man ihn weglassen, da er in der Klasse definiert ist. Zwar verfügen alle Objekte einer Klasse über dieselben Attribute, doch ihre Werte unterscheiden sich. Daher werden in Objekten meist die Attributwerte eingetragen, wie dies auch in Abbildung 52 gemacht wurde.

In einem System zur Verwaltung von Personen kann es hunderte oder tausende von Personen-Objekten geben. Diese wird man natürlich nicht alle modellieren. Objektdiagramme erstellt man eigentlich nur, um einzelne Beispiele darzustellen und zu untersuchen.

Im Mittelpunkt steht das Klassendiagramm. Auch bei der Programmierung erstellt man ja Klassen und keine Objekte. Die einzelnen Objekte werden erst zur Laufzeit des Programms durch Aufruf des Konstruktors erzeugt. Tabelle 17 stellt Klassen und Objekte im Modell und im Java-Programm einander gegenüber.

	Modell	*Java-Programm*

Klasse	Im Klassendiagramm:
	Person
	- vorname: String
	- nachname: String
	+ «create» Person(vorname: String, nachname: String)
	+ getName(): String
		Klassendefinition im Programmcode:
		`public class Person {`
		`  //...`
		`}`

Objekte	Im Objektdiagramm:
	p1 : Person
	vorname = "George"
	nachname = "Orwell"
	p2 : Person
	vorname = "Douglas"
	nachname = "Adams"
	(Normalerweise nicht modelliert)
		Objekte werden zur Laufzeit durch Aufruf des Konstruktors erstellt:
		`Person p1 =`
		`new Person("George", "Orwell");`
		`Person p2 =`
		`new Person("Douglas", "Adams");`

Tabelle 17: Klassen und Objekte im Modell und im Java-Programm

Auch wenn normalerweise vor allem Klassendiagramme und kaum Objektdiagramme erstellt werden, sind Objektdiagramme von kleinen Beispielen sehr nützlich um zu verstehen, was ein Klassendiagramm genau bedeutet. Daher werden in diesem Skript öfter Objektdiagramme zur Verdeutlichung verwendet, und es wird empfohlen, bei komplexeren Sachverhalten zumindest kurz ein paar Beispielobjekte zu skizzieren.

Auch für Objekte gibt es verschiedene Möglichkeiten der Darstellung, wobei man je nach Darstellungszweck unterschiedliche Details weglassen kann (Abbildung 53). In jedem Fall müssen Objektbezeichnung und Klassenname unterstrichen werden. Wird nur der Klassenname angegeben, so muss davor ein Doppelpunkt sein. Ansonsten würde es sich um eine Objektbezeichnung handeln.

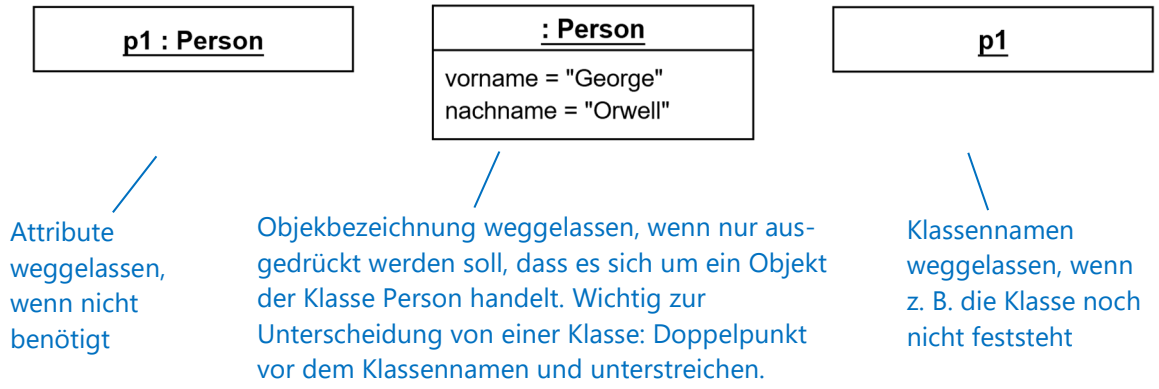


Abbildung 53: Verschiedene Darstellungen von Objekten

6.3 Assoziationen

Zur oben verwendeten Klasse „*Person*" kommt nun noch eine zweite Klasse „*Buch*" hinzu. Zunächst wird wieder der Java-Code betrachtet (Listing 5).

```
public class Buch {
	private String titel;
	private int seitenzahl;
	private Person autor;

	public Buch(String titel, int seitenzahl, Person autor){
		this.titel = titel;
		this.seitenzahl = seitenzahl;
		this.autor = autor;
	}

	public void print(){
		System.out.println(titel + ", " + seitenzahl +
				" S., Autor: " + autor.getName());
	}
}
```

Listing 5: Klasse „Buch"

Die Klasse hat drei Attribute. „*titel*" und „*seitenzahl*" sind vom Typ *String* bzw. *int*. Dabei handelt es sich um grundlegende Datentypen, die einzelne Werte aufnehmen können. „*autor*" hingegen ist vom Typ „*Person*". Hier wurde die oben betrachtete Klasse „*Person*" als Datentyp verwendet. Diesem Attribut kann man also ein Objekt der Klasse „*Person*" zuordnen.

Wenn man dem Konstruktor „*Buch*" aufruft, übergibt man ihm als dritten Parameter das gewünschte Person-Objekt, das dann im Konstruktor dem Attribut „*autor*" zugeordnet wird. Da der Parameter „*autor*" so heißt wie das entsprechende Attribut der Klasse, erfolgt der Zugriff auf das Attribut mittels „*this.autor*".

Bevor man ein Buch anlegt, muss also bereits ein Person-Objekt existieren, das man dann dem Buch zuordnet. Dadurch steht jedes Buch-Objekt immer mit einem Person-Objekt in Beziehung (Abbildung 54).

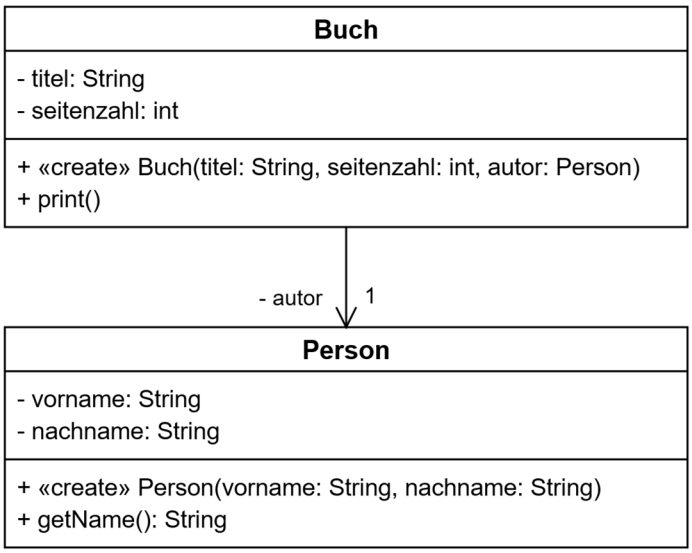


Abbildung 54: Assoziation

Die Assoziation wird als Linie dargestellt. Existiert im Klassendiagramm eine Assoziation zwischen zwei Klassen, so heißt dies zunächst einmal, dass Objekte der beiden Klassen miteinander in Beziehung stehen können. Die Assoziation in Abbildung 54 enthält noch einige Zusatzinformationen: Die Navigationsrichtung, die Rolle und die Multiplizität.

Der Pfeil gibt die mögliche Navigationsrichtung an. Dies bedeutet, dass man direkt vom Buch zum Autor gelangen kann, umgekehrt aber nicht. Das liegt daran, wie die Assoziation implementiert ist: Wie im Code der Klasse „*Buch*“ zu sehen ist (Listing 5), enthält sie das Attribut „*autor*“, dem ein Person-Objekt zugeordnet werden kann. Will man für ein bestimmtes Buch die zugehörige Person herausfinden, so muss man lediglich im Attribut „*autor*“ nachsehen.

Umgekehrt ist das nicht so leicht. Die Implementierung der Klasse „*Person*“ (Listing 2) wurde unverändert übernommen. Dort gibt es kein Attribut vom Typ „*Buch*“.

Die Angabe „*autor*“ bezeichnet die Rolle, die das Person-Objekt in der Beziehung spielt. Es könnte ja unterschiedliche Beziehungen zwischen Büchern und Personen geben: Eine Person könnte auch der Herausgeber oder der Übersetzer eines Buchs sein.

Als Rollenname wird der Name des Attributs verwendet, mit dem die Assoziation in der Klasse „*Buch*“ implementiert ist. Das im Code vorhandene Attribut „*autor*“ wird in Abbildung 54 in der Darstellung der Klasse „*Buch*“ gar nicht in der Liste der Attribute aufgeführt. Dies ist auch nicht nötig, weil es bereits durch die Rollenangabe an der Assoziation dargestellt wird. Dort findet sich auch das Minus-Symbol für die Sichtbarkeit. Der Typ des Attributs „*autor*“ ist die verbundene Klasse „*Person*“.

Wichtig: „autor“ ist als Attribut der Klasse „Buch“ implementiert. Eingetragen wird die Rolle aber stets *am gegenüberliegenden Ende* der Assoziation.

Schließlich findet sich noch die Angabe der Multiplizität, in diesem Fall „1". Dies bedeutet, dass jedem Buch genau eine Person zugeordnet ist. In unserem Fall muss also jedes Buch immer exakt einen Autor haben. Bücher mit mehreren Autoren sind mit diesem Beispiel also nicht möglich, ebenso wenig Bücher ohne einen Autor.

Diese Multiplizität ergibt sich aus der Implementierung der Klasse „*Buch*" (Listing 5). Das Attribut „*autor*" kann nur ein Person-Objekt aufnehmen, nicht aber mehrere. Da durch den Konstruktor beim Anlegen eines Buch-Objekts immer schon gleich ein Person-Objekt als Autor zugeordnet wird und auch keine Möglichkeit vorgesehen ist, dieses wieder zu entfernen, sollte es auch keine Buch-Objekte ohne Autor geben. Um ganz sicherzugehen, müsste man im Konstruktor übrigens noch überprüfen, ob nicht versehentlich der Wert „*null*" anstelle eines Person-Objekts übergeben wurde, denn dann gäbe es doch ein Buch ohne Autor.

Auch für die Multiplizität gilt: Sie wird stets *am gegenüberliegenden Ende der Assoziation* eingetragen. Man betrachtet also ein Buch-Objekt und sieht am anderen Ende der Assoziation wie viele Person-Objekte mit diesem Buch-Objekt in Beziehung stehen.

Abbildung 4.6 zeigt noch einmal den beschriebenen Zusammenhang zwischen der Attribut-Deklaration im Code und der Assoziationsdarstellung im Klassendiagramm.

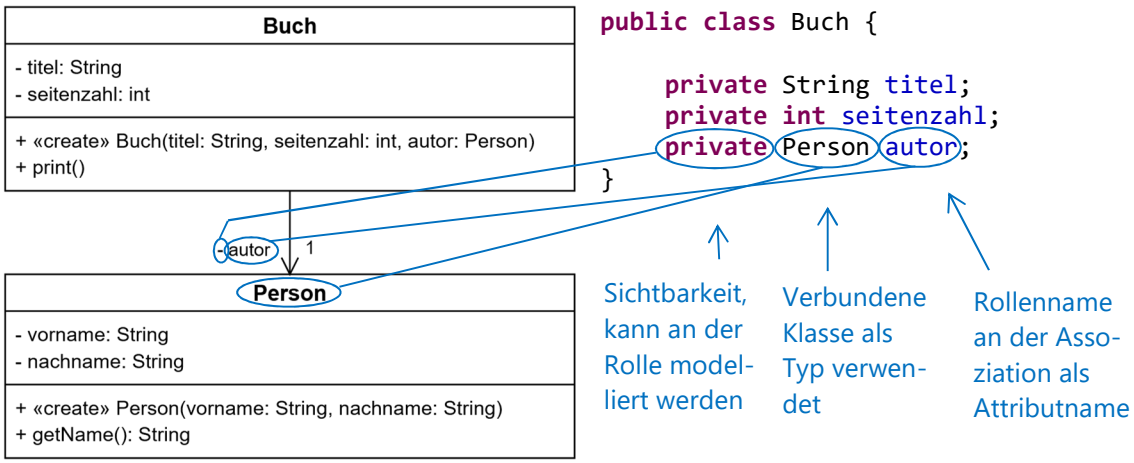


Abbildung 55: Assoziation im Klassendiagramm und im Code

Wann wird ein Attribut im Klassendiagramm einfach in die Attributliste eingetragen und wann wird es zu einer Assoziation? Es wird immer dann eine Assoziation verwendet, wenn die als Typ verwendete Klasse ebenfalls im Klassendiagramm auftaucht – wie die Klasse „*Person*" im obigen Beispiel.

Attribute mit den gängigen Datentypen (*String*, *int*, …) werden stets als gewöhnliche Attribute modelliert. Auch wenn eine Klasse als Typ verwendet wird, die nicht im Modell auftaucht, wird das betreffende Attribut als gewöhnliches Attribut dargestellt. Beispiele sind Klassen aus Klassenbibliotheken, wie *java.time.LocalDate*.

6.4 Verbundene Objekte

Nun sollen einige Buch-Objekte angelegt werden. Da im Konstruktor eine Person als Parameter übergeben werden muss, werden in Listing 6 zunächst zwei Person-Objekte erzeugt. Anschließend werden drei Buch-Objekte angelegt, wobei die ersten beiden *„person1"* und das dritte *„person2"* als Autor übergeben bekommt. Wie bereits erläutert wird das übergebene Personenobjekt im Konstruktor jeweils dem Attribut *„autor"* zugeordnet (Listing 5).

```
public class Test2 {

    public static void main(String[] args) {
        Person person1 = new Person("George", "Orwell");
        Person person2 = new Person("Douglas", "Adams");
        Buch buch1 = new Buch("Farm der Tiere", 132, person1);
        Buch buch2 = new Buch("1984", 383, person1);
        Buch buch3 = new Buch("Per Anhalter durch die Galaxis", 208, person2);
    }
}
```

Listing 6: Anlegen von Buchobjekten

Die erstellten Objekte lassen sich wiederum in einem Objektdiagram darstellen (Abbildung 56).

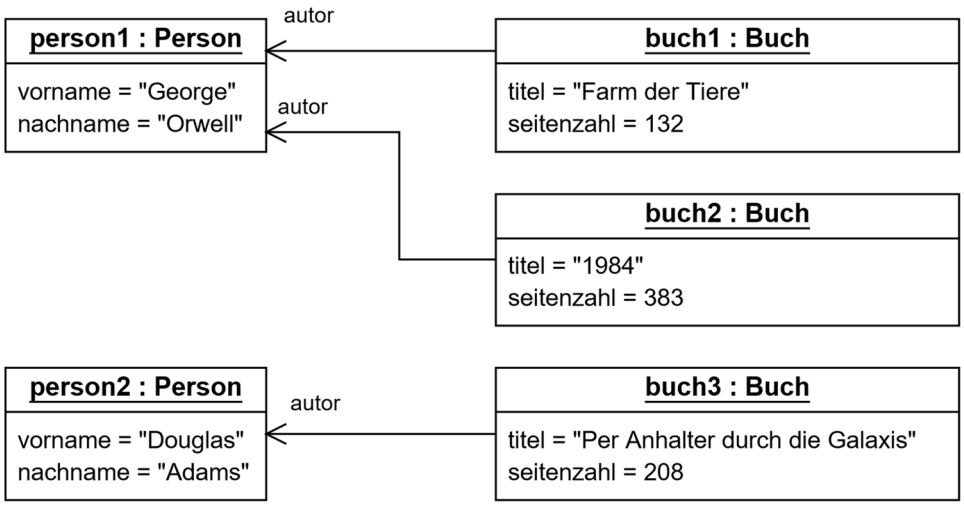


Abbildung 56: Miteinander verbundene Objekte der Klassen „Person" und „Buch"

An dem Beispiel lässt sich noch einmal die Bedeutung der Multiplizität „1" nachvollziehen. Zu jedem Buch muss genau eine Person zugeordnet sein. Entsprechend ist kein Buch-Objekt mit mehreren Person-Objekten verbunden. Ebenso gibt es kein Buch-Objekt ohne zugeordnetes Person-Objekt.

6.5 Multiplizitäten

Möchte man auch Bücher ohne Autor zulassen, muss man die Multiplizität zu „0..1" ändern (Abbildung 57). Jedem Buch-Objekt ist dann entweder kein Autor oder ein Autor zugeordnet. Sinnvollerweise hat die Klasse Buch nun auch einen zweiten Konstruktor, mit dem ein Buch ohne Autor angelegt werden kann.

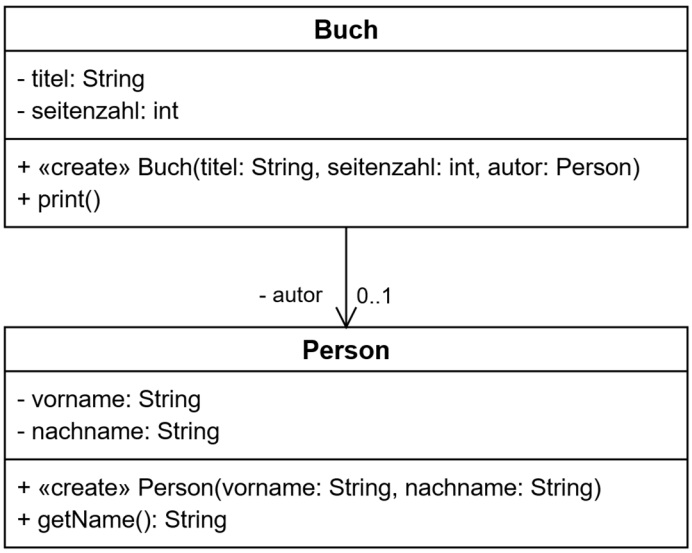


Abbildung 57: Multiplizität 0..1

Im Objektdiagramm können jetzt auch Buch-Objekte auftauchen, die keine Verbindung zu einer Person haben.

Wie schon im Beispiel der Abbildung 56 ersichtlich, kann ein Autor mehrere Bücher geschrieben haben. Dort sind z. B. zwei Bücher mit der Person George Orwell verbunden. Um darzustellen, dass eine Person zu mehreren Büchern zugeordnet sein kann, wird die Multiplizität „*“ verwendet (Abbildung 58). Der Stern steht für „beliebig viele“. Dies schließt auch null mit ein, d. h. es kann auch Personen geben, die mit keinem Buch-Objekt verbunden sind.

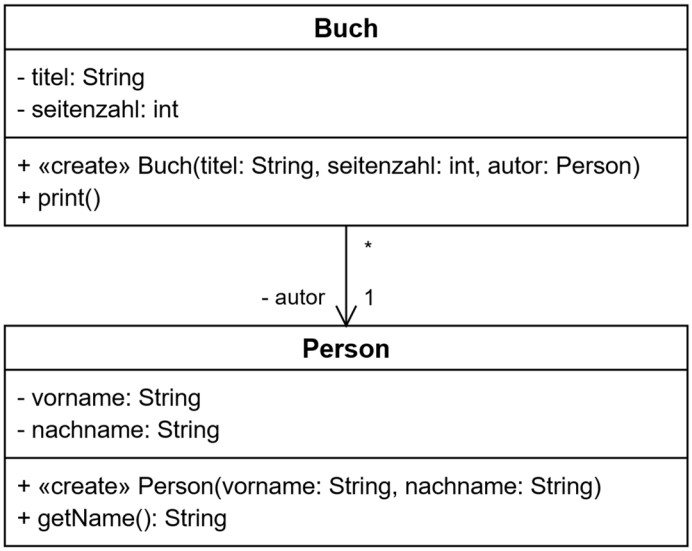


Abbildung 58: Eine Person kann beliebig vielen Büchern zugeordnet sein

Die Multiplizität „*“ entspricht der vorhandenen Implementierung. Man kann ohne weiteres dasselbe Person-Objekt mehreren Büchern als Autor übergeben, wie dies in Listing 6 mit dem Objekt *person1* gemacht wurde. Und natürlich kann man Person-Objekte anlegen, die bei keinem Buch-Objekt eingetragen werden.

Schwieriger wird dies, wenn man auf der nicht-navigierbaren Seite eine 1 einträgt. Nimmt man einmal an, dass eine Zahlung immer zu genau einer Rechnung gehören muss, dann ergibt sich die Darstellung in Abbildung 59.

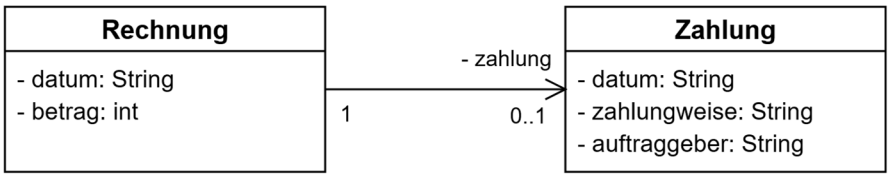


Abbildung 59: Eine Zahlung gehört zu genau einer Rechnung.

Wenn man diese Assoziation implementiert, dann erhält die Klasse „*Rechnung*" ein Attribut „*zahlung*" vom Typ „*Zahlung*". Es hindert einen aber zunächst niemand daran, eine Zahlung bei mehreren Rechnungen einzutragen. Dies könnte man beispielsweise dadurch verhindern, dass man in der Klasse Rechnung keine Möglichkeit vorsieht, ein existierendes Zahlungsobjekt zuzuordnen, sondern dass man jedes Mal, wenn Zahlungsdaten zu einer Rechnung eingetragen werden sollen, ein neues Zahlungsobjekt anlegt (Listing 7). Das neu angelegte Zahlungsobjekt ist dann tatsächlich nur dieser einen Rechnung zugeordnet.

```
public class Rechnung {
    private String datum;
    private String betrag;
    private Zahlung zahlung;

    public void erfasseZahlung(String zahlungsdatum, String zahlungsweise,
                                                        String auftraggeber){
        zahlung = new Zahlung(zahlungsdatum, zahlungsweise, auftraggeber);
    }
}
```

Listing 7: Die Methode erfasseZahlung( ) legt ein neues Zahlungsobjekt an.

Das verhindert andererseits noch nicht, dass man den Konstruktor von Zahlung auch außerhalb der Klasse „*Rechnung*" aufruft. Dann hätte man eine Zahlung, die zu keiner Rechnung gehört, was gemäß der Multiplizität 1 ja nicht erlaubt ist. Diese bedeutet ja „genau eins". Um dies zu verhindern, könnte man die Assoziation auch in die andere Richtung implementieren, d. h. die Klasse Zahlung erhält ein Attribut vom Typ „*Rechnung*", wo immer ein Rechnung-Objekt eingetragen sein muss. Dann hätte man Navigierbarkeit in beide Richtungen, also Pfeile auf beiden Seiten. Andererseits muss auch nicht unbedingt sichergestellt werden, dass man keine Zahlung ohne Rechnung anlegen *kann*, die Multiplizität 1 besagt lediglich, dass man dies nicht tun *darf*.

Im Übrigen wird die Multiplizität an einem nicht navigierbaren Ende nur eingetragen, wenn sie wirklich von Bedeutung ist, ansonsten kann man sie auch einfach weglassen.

Neben 0, 1 und * sind als Multiplizitätsangaben auch andere natürliche Zahlen und Kombinationen möglich. Häufig benötigt werden die folgenden:

1	genau eins
0..1	null oder eins

*	beliebig viele, auch null. Kann alternativ als 0..* angegeben werden
1..*	mindestens eins

Beispiele für andere mögliche Multiplizitätsangaben, die aber seltener verwendet werden:

2	genau zwei
0..4	0, 1, 2, 3 oder 4
2, 4, 6	2, 4 oder 6
0..2, 4, 6..*	alles außer 3 und 5

6.6 Multiplizitäten lesen und anwenden

Aus den bisherigen Ausführungen ergibt sich für das Lesen von Multiplizitäten an einer Assoziation:

1. Man betrachtet ein Objekt der einen Klasse
2. Dann liest man im Klassendiagramm am gegenüberliegenden Ende der Assoziation nach, wie viele Objekte der anderen Klasse mit dem betrachteten Objekt verbunden sein dürfen.

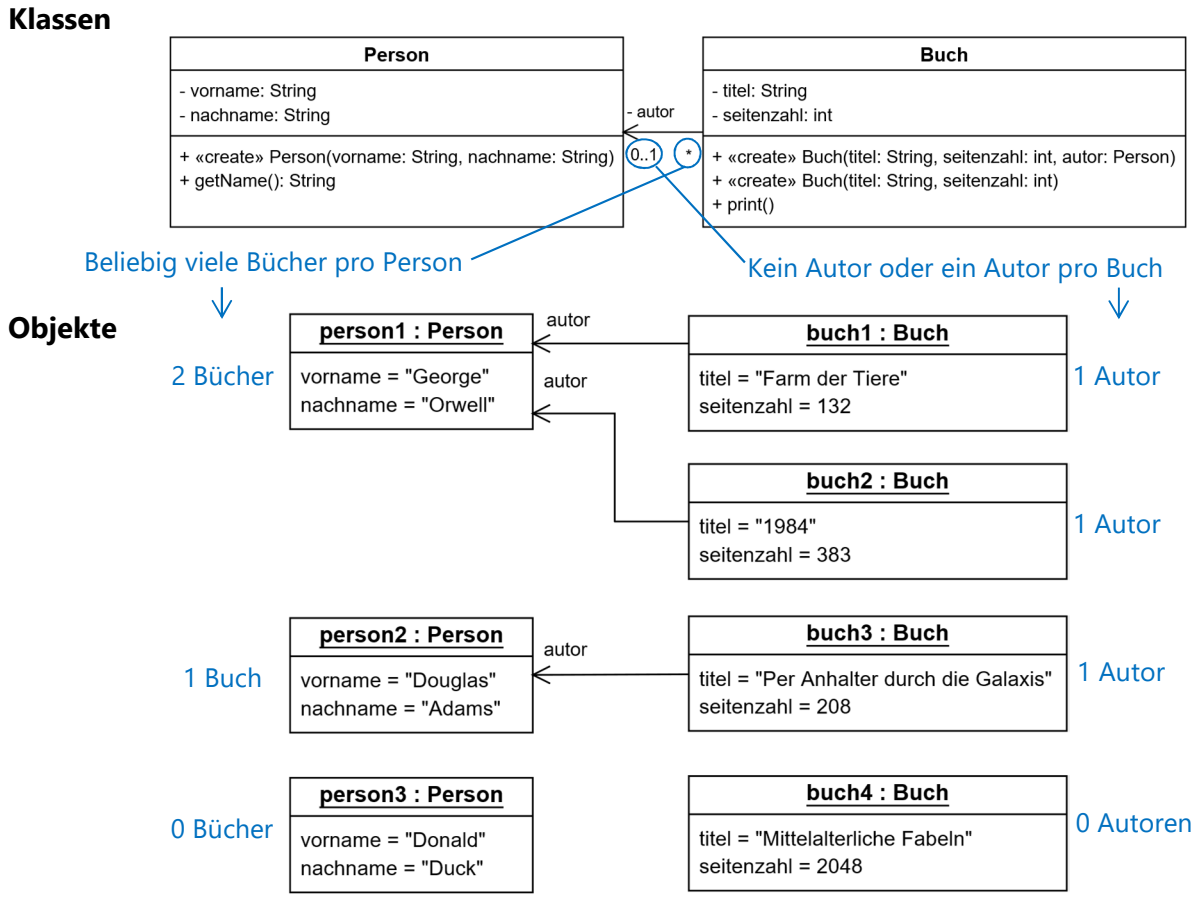


Abbildung 60: Beispiele für Beziehungen von Objekten entsprechend der modellierten Multiplizitäten

Abbildung 60 zeigt einige gültige Beispiele für die Assoziation zwischen Person und Buch. Jedes Objekt der Klasse „*Person*“ darf gemäß der Multiplizität im oben dargestellten Klassendiagramm mit beliebig vielen Buch-Objekten verbunden sein. Betrachtet man z. B. *Person1*, so ist dieses Objekt mit zwei Buch-Objekten verbunden. Dies ist aufgrund der Multiplizität „*“ genauso zulässig wie die beiden anderen Person-Objekte, die einem oder keinem Buch zugeordnet sind.

Betrachtet man ein Buch-Objekt, so sieht man im Klassendiagramm, dass es mit null oder einem Person-Objekt verbunden sein kann. Buch1, Buch2 und Buch3 sind jeweils mit einem, Buch4 mit null Person-Objekten verbunden. Damit sind die im Klassendiagramm modellierten Multiplizitäten alle eingehalten.

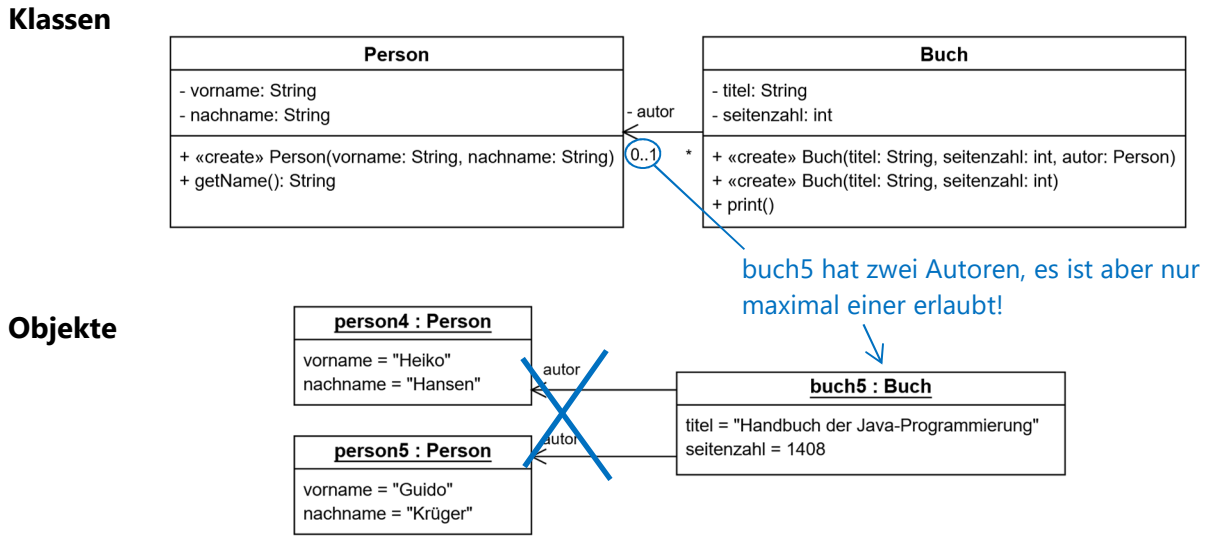


Abbildung 61: Beispiel für die Verletzung von Multiplizitäten

Abbildung 61 hingegen zeigt ein Beispiel, bei dem die Multiplizitäten nicht korrekt berücksichtigt worden sind. Buch5 hat zwei Person-Objekte zugeordnet, obwohl im Klassendiagramm festgelegt ist, dass jedes Buch null oder ein Person-Objekt zugeordnet haben muss. Verwendet man die Implementierung der Klasse Buch aus Listing 5, dann ist der Fall aus Abbildung 61 gar nicht möglich, denn „*autor*“ ist ein einfaches Attribut der Klasse „*Buch*“, dem man nur ein Person-Objekt zuordnen kann.

Nun wird das Beispiel geändert: Jedes Buch kann nun mehrere Autoren haben, es muss aber mindestens einen haben. Außerdem wird gefordert, dass jede Person Autor mindestens eines Buches ist. Die Multiplizitäten auf beiden Seiten sind damit nun „1..*“.

Abbildung 62 zeigt einige Beispiele für Objektverbindungen, die nun erlaubt bzw. verboten sind. Nach wie vor erlaubt sind die beiden Personen mit ein bzw. zwei Büchern. Das Buch5 mit zwei als Autoren zugeordneten Personen ist nun ebenfalls zulässig. Nicht mehr zulässig sind nun aber Personen ohne Bücher und Bücher ohne Personen.

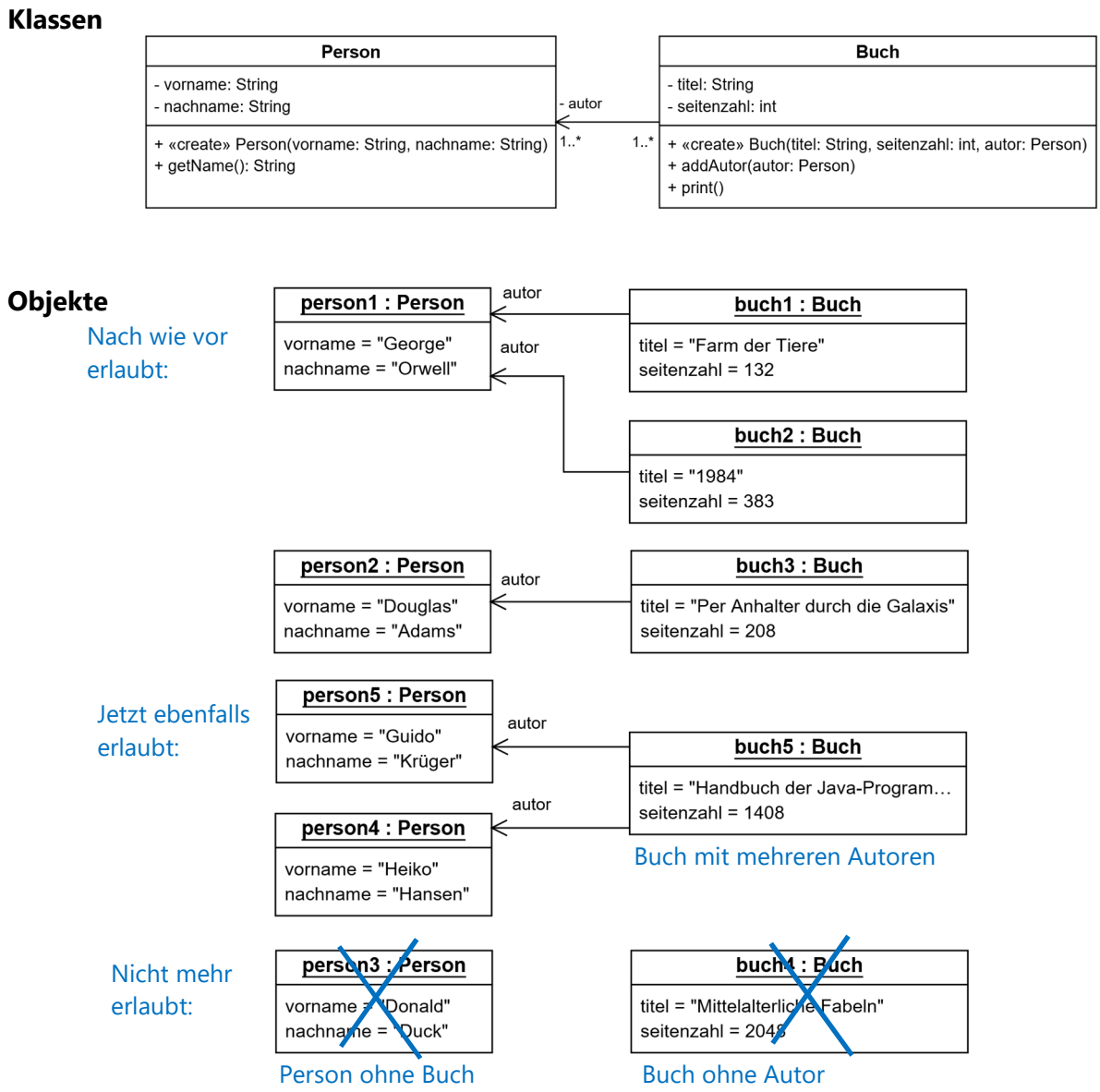


Abbildung 62: Mögliche Objektverbindungen bei Multiplizität "1..*" auf beiden Seiten

Würden die Multiplizitäten auf beiden Seiten in „*“ geändert, so wären alle vorgestellten Beispiele zulässig. Der Stern steht ja für „beliebig viele“, worunter auch die Null fällt, so dass auch Objekte ohne andere zugeordnete Objekte erlaubt sind.

6.7 Implementierung mehrwertiger Assoziationen

Falls man direkt von der Person zum Buch navigieren möchte, wird der Navigationspfeil in die Gegenrichtung eingetragen (Abbildung 63). Für die Implementierung bedeutet dies, dass jedes Person-Objekt die zugehörigen Buch-Objekte speichern muss. Hierzu genügt ein gewöhnliches Attribut vom Typ „*Buch*“ nicht, da dieses ja nur ein Buch aufnehmen kann. Stattdessen wird eine Liste benötigt.

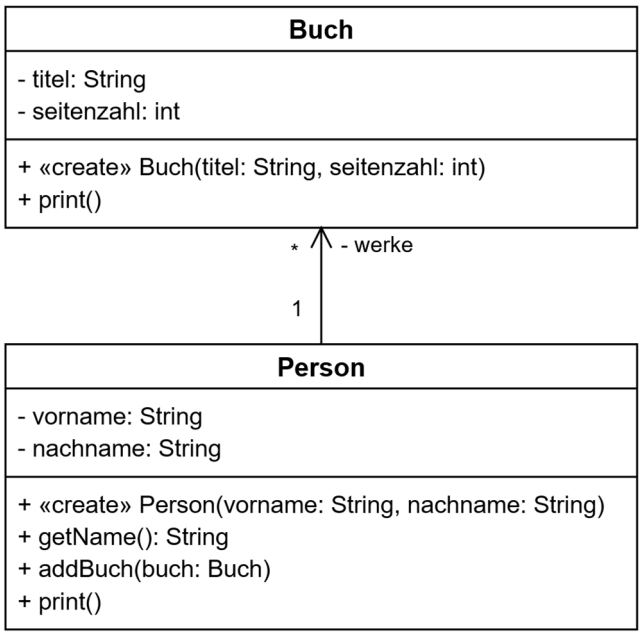


Abbildung 63: Navigierbarkeit von der Person zum Buch

Listing 8 zeigt eine mögliche Implementierung der mehrwertigen Assoziation. Für die Verwaltung von Listen stellt Java verschiedene Collections zur Verfügung. In diesem Fall wird eine *ArrayList* verwendet. Ein gewöhnliches Array „*Buch[ ]*" kann man hier nicht verwenden, da man für ein Array von vornherein die Maximalgröße festlegen muss. Hier sollen aber beliebig viele Bücher möglich sein, weshalb die Maximalgröße der Bücher-Liste einer Person vorher nicht feststeht.

```
import java.util.ArrayList;

public class Person {
	private String vorname;
	private String nachname;
	private ArrayList<Buch> werke = new ArrayList<Buch>();

	public Person(String vorname, String nachname){
		this.vorname = vorname;
		this.nachname = nachname;
	}

	public void addBuch(Buch buch){
		werke.add(buch);
	}

	public void print(){
		System.out.println(vorname + " " + nachname);
		for(Buch b:werke){
			b.print();
		}
	}
}
```

Listing 8: Implementierung einer mehrwertigen Assoziation

Die *ArrayList* muss zunächst mit „*import java.util.ArrayList;*“ aus der Bibliothek *java.util* importiert werden. Als Typ für das Attribut „*werke*“, das die von der Person geschriebenen Bücher aufnehmen soll, wird „*ArrayList<Buch>*“ festgelegt. Mit Hilfe der Angabe *<Buch>* wird die Liste typisiert, d. h. sie kann nur Objekte vom Typ Buch aufnehmen und nicht etwa beliebige andere Objekte. Das Attribut wird mit einer mittels *new ArrayList<Buch>( )* neu angelegten Liste initialisiert.

Mit der Methode „*addBuch( )*“ lässt sich ein Buch-Objekt in die Liste „*werke*“ eintragen. Die Methode „*print( )*“ gibt zunächst Vor- und Nachname der Person aus und ruft dann für jedes Buch in der Liste „*werke*“ die in der Klasse Buch definierte Methode „*print( )*“ auf, so dass alle Bücher der Person aufgelistet werden.

Die Methode „*print( )*“ lässt sich nur deswegen so einfach gestalten, weil die Assoziation in der Richtung von der Person zum Buch implementiert ist. Wäre die Assoziation nur in die andere Richtung navigierbar, dann wäre es sehr mühsam herauszufinden, welche Bücher eine Person geschrieben hat.

In Listing 9 werden zunächst zwei Personen und drei Bücher angelegt. Anschließend werden Buch1 und Buch2 der Person1 und Buch3 der Person2 zugeordnet. Die Objekte und ihre Beziehungen sind in Abbildung 64 dargestellt.

```
Person person1 = new Person("George", "Orwell");
Person person2 = new Person("Douglas", "Adams");

Buch buch1 = new Buch("Farm der Tiere", 132);
Buch buch2 = new Buch("1984", 383);
Buch buch3 = new Buch("Per Anhalter durch die Galaxis", 208);

person1.addBuch(buch1);
person1.addBuch(buch2);
person2.addBuch(buch3);
```

Listing 9: Zuordnen von Büchern zu Personen

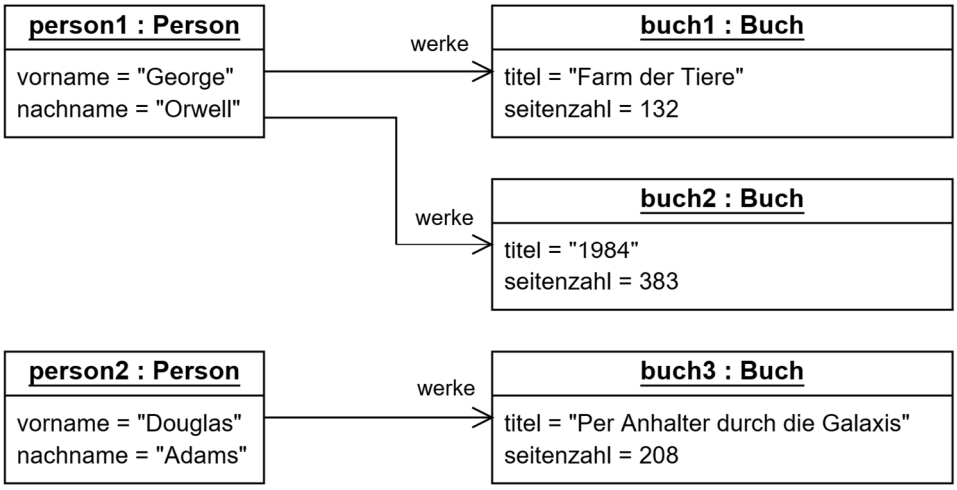


Abbildung 64: Personen und zugeordnete Bücher im Objektdiagramm

6.8 Wahl der Navigationsrichtung

Welche Navigationsrichtung sollte man implementieren? Sinnvollerweise sollte man in die Richtung navigieren können, die häufig benötigt wird. Muss oft von der Person auf die Bücher zugegriffen werden, wie z. B. in der oben beschriebenen *print( )*-Methode, dann ist es sehr hilfreich, wenn man direkt dorthin gelangt. Falls man hingegen oft vom Buch zur Person navigieren muss, ist die andere Navigationsrichtung nützlich.

Falls Zugriffe in beide Richtungen erforderlich sind, gibt es zwei Möglichkeiten: Zum einen kann trotzdem nur die eine Richtung implementiert werden. Wenn der Zugriff in die andere Richtung nötig ist, werden alle Objekte der anderen Klasse durchsucht, ob sie eine Beziehung zum betreffenden Objekt haben. Der Nachteil ist, dass dieser Zugriff recht aufwändig ist.

Alternativ kann man die Assoziation in beide Richtungen implementieren (Abbildung 65), d. h. beide Klassen haben je ein Attribut zum Speichern des oder der verbundenen Objekte der jeweils anderen Klasse. Auch dies ist jedoch mit einem Nachteil verbunden: Beim Aufbau und beim Abbau von Beziehungen müssen die zugeordneten Objekte jeweils auf beiden Seiten eingetragen bzw. entfernt werden.

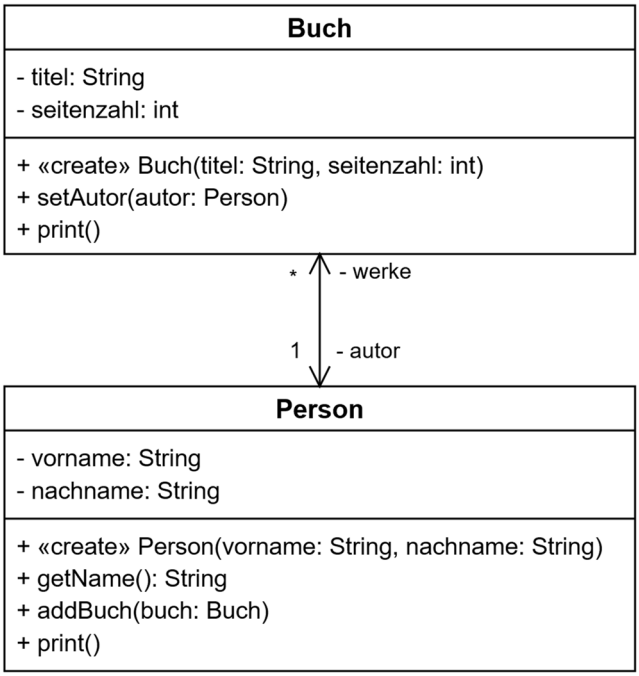


Abbildung 65: Navigierbarkeit in beide Richtungen

Insbesondere in der Analyse, wenn es noch gar nicht um eine konkrete Implementierung geht, spielt die Navigationsrichtung noch gar keine Rolle. In diesem Fall lässt man die Navigationsrichtung weg und trägt nur die Multiplizitäten ein. Dann trägt man auch nur dort Rollen ein, wo sie notwendig sind, um die Bedeutung der betreffenden Beziehung zu verstehen. So ist es in Abbildung 66 von Interesse, dass es sich bei der mit einem Buch verbundenen Person um den Autor und nicht z. B. den Übersetzer handelt. Für die andere Richtung ergibt sich die Bedeutung bereits aus der Rolle „*autor*“: Es handelt sich logischerweise um die Bücher, die die Person geschrieben hat.

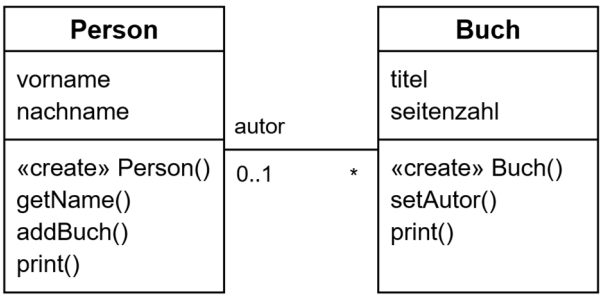


Abbildung 66: In Analysemodellen werden Assoziationen ohne Navigationsrichtungen dargestellt

Man kann auch der Assoziation selbst eine Bezeichnung geben, z. B. könnte man die Assoziation zwischen Person und Buch „hat geschrieben" nennen (Abbildung 67). Normalerweise verwendet man eine Assoziationsbezeichnung aber nur dort, wo die Bedeutung nicht von selbst klar ist.

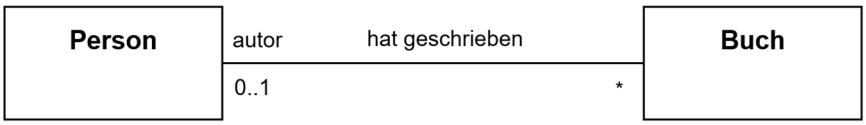


Abbildung 67: Assoziation mit Bezeichnung

Unter Umständen ergibt sich auch die Richtung nicht von selbst. So könnte man sich bei der Beziehung „liefert an" zwischen Hersteller und Händler in Abbildung 68 fragen, ob der Händler den Hersteller beliefert (z. B. mit Rohstoffen) oder der Hersteller den Händler (mit Waren, die dieser weiterverkauft). Daher wurde ein kleiner Pfeil eingefügt, der die Leserichtung angibt. Damit ist klar, dass mit der Assoziation ausgedrückt wird, welcher Hersteller an welchen Händler liefert.

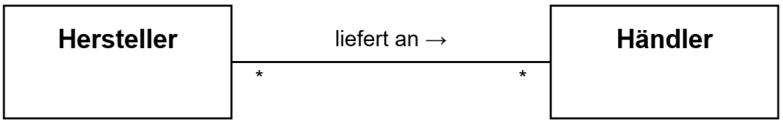


Abbildung 68: Assoziation mit Bezeichnung und Leserichtung

6.9 Eigenschaften von Assoziationen

Wie oft kann dasselbe Objekt an einer mehrwertigen Assoziation teilnehmen? In vielen Fällen ist es klar: Eine bestimmte Person kann ein bestimmtes Buch geschrieben haben oder nicht. Dennoch könnte man in der Implementierung aus Listing 8 dasselbe Buch-Objekt mehrfach in die Liste „*werke*" eines Person-Objekts eintragen, und zwar indem man einfach mehrfach die Methode „*addBuch( )*" aufruft (Listing 10). Damit existiert die Verbindung zwischen den beiden Objekten zweimal (Abbildung 69), was nicht besonders sinnvoll ist. Möchte man z. B. die Zahl der Werke eines Autors ermitteln, indem man die Einträge in „*werke*" zählt, wird das betreffende Buch doppelt gezählt.

```
person2.addBuch(buch3);
person2.addBuch(buch3);
```

Listing 10: Durch mehrfachen Aufruf von "addBuch( )" kann man dieselben Objekte einander mehrmals zuordnen

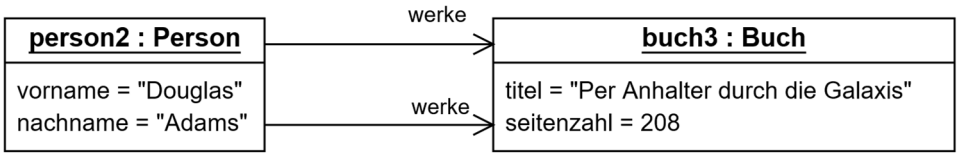


Abbildung 69: Dasselbe Buch ist zweimal in der Liste "werke" eingetragen

Möchte man darstellen, dass jedes Objekt der Klasse „*Buch*" höchstens einmal mit einem Person-Objekt verbunden sein kann, kann man dies durch die Eigenschaft {*unique*} ausdrücken. Sie wird am betreffenden Assoziationsende eingetragen (Abbildung 70).

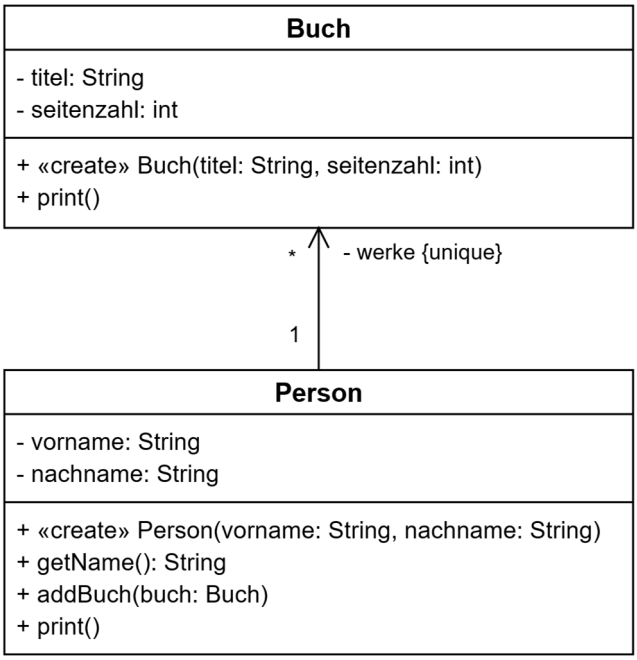


Abbildung 70: Assoziation mit der Eigenschaft {unique}

Um diese Eigenschaft in der Implementierung sicherzustellen, muss man beim Hinzufügen eines Buchs überprüfen, ob sich das Buch nicht schon in der Liste „*werke*" befindet. In Listing 11 wurde die *addBuch( )*-Methode aus Listing 8 um diese Prüfung erweitert.

```
public void addBuch(Buch buch){
	if(!werke.contains(buch)){
		werke.add(buch);
	}
}
```

Listing 11: Die Methode fügt nur dann ein Buch hinzu, wenn es sich noch nicht in der Liste befindet

Anstelle einer *ArrayList* hätte man für „*werke*" auch ein *Set*, z. B. ein *HashSet* oder ein *TreeSet*, verwenden können. Diese Collections erlauben keine doppelten Einträge.

Normalerweise geht man davon aus, dass Assoziationen unique sind. Die Angabe {*unique*} verwendet man daher meist nur dort, wo besonders darauf hingewiesen werden soll.

Abbildung 71 zeigt ein Beispiel einer nicht-uniquen Assoziation. Hier kann jeder Spieler mehrfach an derselben Spielrunde teilnehmen. Kostet jede Teilnahme an einer Spielrunde einen bestimmten Betrag und hat jeder Eintrag in der Spielerliste dieselbe Gewinnchance, so

kann man seine Gewinnchance erhöhen, indem man sich mehrfach an der Spielrunde beteiligt.

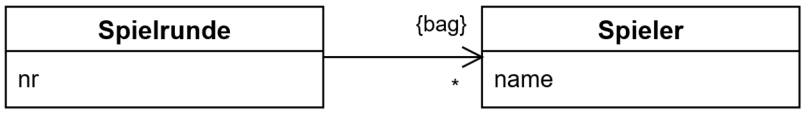


Abbildung 71: Spieler können mehrfach an einer Spielrunde teilnehmen

In vielen Fällen ist es egal, in welche Reihenfolge die Objekte in einer Liste stehen. Beispielsweise gibt es bei der Zuordnung von Büchern zu einem Autor keine bestimmte Reihenfolge zu beachten. Fügt man ein neues Buch hinzu, so kann dies an eine beliebige Stelle in die Liste der zugeordneten Werke eingefügt werden. Für die Ausgabe kann man die Bücher eines Autors ja ohne weiteres ordnen lassen, z. B. alphabetisch nach dem Titel oder nach dem Erscheinungsjahr. In einer Assoziation mit der Eigenschaft {*bag*} ist also keine bestimmte Reihenfolge festgelegt. Wenn Mehrfacheinträge erlaubt sind kann man auch einfach {*non-unique*} angeben. Damit wird keine Aussage über die Reihenfolge gemacht. Insofern ist {*non-unique*} gleichbedeutend mit {*bag*}.

Im Beispiel von Abbildung 72 ist die Reihenfolge hingegen von Bedeutung. Es ist ja nicht nur wichtig, dass ein bestimmter Titel in den Charts ist, sondern auch auf welchem Platz er steht. Mit {*ordered*} wird ausgedrückt, dass die Liste der Titel in den Charts geordnet ist.

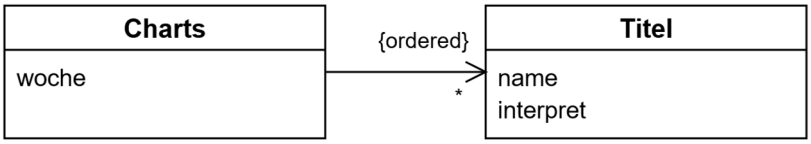


Abbildung 72: Bei der Beziehung von Titeln zu einer Chart-Liste ist die Reihenfolge wichtig

Für die Implementierung bedeutet dies zum einen, dass man eine Datenstruktur benötigt, die eine bestimmte Reihenfolge der Elemente vorsieht. Dies ist bei der *ArrayList* der Fall. Zum anderen muss man beim Einfügen von Objekten auf die Reihenfolge achten. Man kann ein hinzuzufügendes Element nicht mehr einfach an das Ende der Liste hinzufügen. Stattdessen muss man es an der richtigen Stelle einfügen und hierfür ggf. andere Objekte verschieben.

Man könnte natürlich auch auf andere Weise eine Ordnung unter den Titeln erzeugen. So könnte man ein Attribut „*platzierung*" einfügen und die Titel dann einfach nach diesem Attribut sortieren. Andererseits gibt es ja jede Woche neue Charts. Daher ist jeder Titel mit mehreren Charts-Objekten verbunden. Nun kann aber die Platzierung eines Titels in jeder Woche anders sein. Damit genügt das eine Attribut „*platzierung*" im Titel nicht mehr. Stattdessen braucht man für jeden Titel viele Platzierungen – für jedes Charts-Objekt eine andere. Die betreffende Lösung wäre deutlich aufwändiger, da eine eigene Klasse benötigt wird um die Platzierungen aufzunehmen.

Die Eigenschaft {*ordered*} beinhaltet übrigens die Eigenschaft {*unique*}, d. h. jeder Titel ist einem bestimmten Charts-Objekt höchstens einmal zugeordnet. Auch nicht-unique Assoziationen können geordnet oder ungeordnet sein. Bei dem oben verwendeten {*bag*} handelt

es sich um eine ungeordnete Assoziation. Eine nicht-unqiue, geordnete Assoziation wird mit {*sequence*} oder kurz {*seq*} gekennzeichnet.

Das folgende Beispiel kann mit einer Sequence modelliert werden. In einem Verein können sich die Mitglieder auf eine Liste als Helfer für eine bestimmte Aktivität eintragen lassen (z. B. für den Thekendienst im Vereinsheim). Die Helfer kommen in der Reihenfolge der Liste zum Einsatz. Dabei ist es auch möglich, sich mehrfach auf die Liste setzen zu lassen. So könnte sich der erste auf der Liste bereits wieder hinten für einen späteren Einsatz eintragen lassen. Die Liste könnte z. B. so aussehen:

Peter, Paul, Maria, John, Peter, Maria, John, Paul, Peter, …

Die Liste ist also geordnet und kann Mehrfacheinträge enthalten, weshalb die Assoziation in Abbildung 73 als {*sequence*} gekennzeichnet ist.

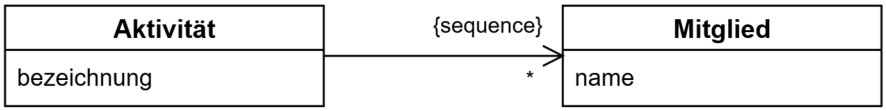


Abbildung 73: Beispiel für eine Sequence

Tabelle 18 fasst die möglichen Eigenschaftsangaben an Assoziationen noch einmal zusammen.

	Geordnet	*Ungeordnet*
Keine Mehrfacheinträge	{ordered}	{unique}
Mehrfacheinträge zulässig	{sequence}, {seq}	{bag}, {non-unique}

Tabelle 18: Eigenschaftsangaben für Assoziationen

6.10 Mehrere Assoziationen zwischen zwei Klassen

Es wurde bereits angesprochen, dass Assoziationen zwischen zwei Klassen unterschiedliche Bedeutungen haben können. Die jeweilige Bedeutung geht aus den Rollennamen oder der Bezeichnung hervor. So hat in dem oben betrachteten Beispiel die Person in der Assoziation zum Buch die Rolle „*autor*". Natürlich hätte man stattdessen auch eine Assoziation mit der Rolle „*herausgeber*" oder „*übersetzer*" modellieren können.

Möchte man in einem Modell sowohl die Autoren als auch die Übersetzer eines Buches berücksichtigen, so modelliert man eben zwei Assoziationen zwischen diesen beiden Klassen (Abbildung 74).

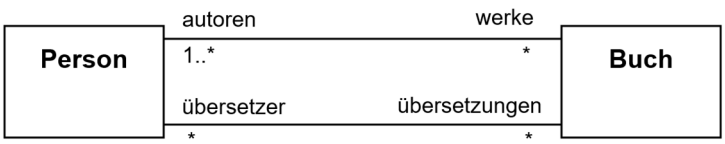


Abbildung 74: Mehrere Assoziationen zwischen denselben Klassen

Wenn man zwei oder mehrere Assoziationen zwischen denselben Klassen hat, ist es wichtig, diese unterscheiden zu können. Auch in einem Analysediagramm, in das man sonst nicht alle Rollen eintragen muss, werden hier die Rollenangaben – oder aber Assoziationsbezeichnungen – zur Unterscheidung benötigt.

7 Klassendiagramme – weitere Konzepte

Die wesentlichen Elemente der Klassendiagramme wurden im vorangehenden Kapitel besprochen – mit Ausnahme der Vererbung, für die es noch einmal ein eigenes Kapitel gibt. Das vorliegende Kapitel behandelt Variationen der bisher besprochenen Themen, wie z. B. Klassenattribute und –methoden, Assoziationen von Klassen mit sich selbst, „ist Teil von"-Beziehungen und Mehrfachassoziationen, bei denen mehr als zwei Klassen beteiligt sind.

7.1 Weitere Angaben zu Attributen und Methoden

In Kapitel 6.1 wurden die wichtigsten Angaben zu Attributen erläutert. Abbildung 75 zeigt einige weitere. In Listing 12 ist die Umsetzung in Java dargestellt.

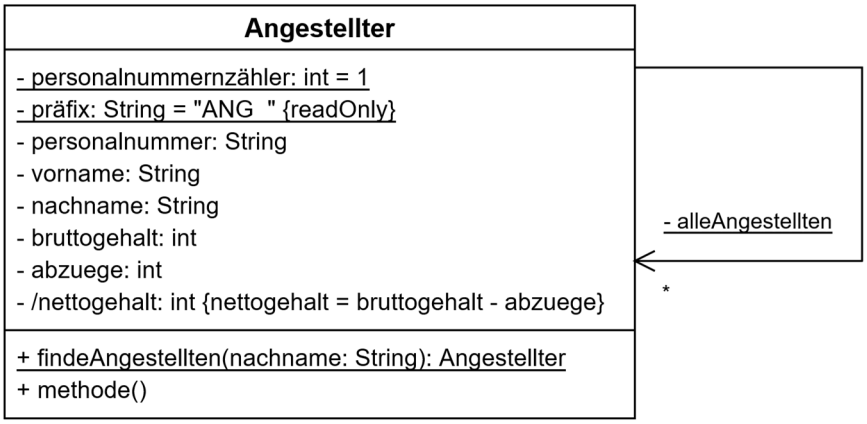


Abbildung 75: Weitere Angaben zu Attributen und Methoden

```
public class Angestellter {
	private static int personalnummernzaehler = 1;
	private static final String praefix="ANG_";
	private String personalnummer;
	private String vorname;
	private String nachname;
	private int bruttogehalt;
	private int abzuege;
	private int nettogehalt; // bruttogehalt-abzuege

	private static ArrayList<Angestellter>
			alleAngestellten = new ArrayList<Angestellter>();

	public static Angestellter findeAngestellten(String nachname){
		// …
	}
}
```

Listing 12: Attribute und Methoden der Klasse Angestellter

Das Attribut „*personalnummernzaehler*" ist unterstrichen. Damit wird ein Klassenattribut gekennzeichnet. Gewöhnliche Attribute existieren für jedes Objekt der Klasse. Jedes Objekt hat eigene Werte für ein solches Attribut, die sich i. d. R. von denen der anderen Objekte unterscheiden. Im Gegensatz dazu existiert ein Klassenattribut nur einmal für die gesamte Klasse, d. h. es gibt nur einen einzigen Wert dafür. Im vorliegenden Beispiel handelt es sich um einen

Zähler, der dazu verwendet wird, jedem neuen Angestelltenobjekt eine neue Nummer zuzuordnen. Am Anfang hat er den ebenfalls in der Klasse dargestellten Default-Wert 1. Klassenattribute werden im Javacode mit „*static*" gekennzeichnet.

Das zweite Attribut, „*praefix*", ist ebenfalls ein Klassenattribut. Außerdem hat es die Eigenschaft „{*readOnly*}". Das bedeutet, dass der Wert nicht geändert werden kann. In dem Beispiel sollen die Personalnummern immer mit demselben Präfix „*ANG_*" beginnen. Dies soll unveränderbar sein. Im Javacode wird diese Unveränderbarkeit durch die Kennzeichnung „*final*" erreicht.

Betrachtet man die Assoziation, die die Klasse mit sich selbst verbindet, so fällt auf, dass der Rollenname ebenfalls unterstrichen ist. Im Code sieht man, dass die Liste aller Angestellten als Klassenattribut implementiert ist. Dies bedeutet, dass die Klasse „*Angestellter*" über eine Liste aller Angestellten-Objekte verfügt. Es ist nicht notwendig und sinnvoll, dass jedes Angestellten-Objekt eine Liste aller existierenden Angestellten-Objekte besitzt, daher ist sie der Klasse selbst zugeordnet.

Eine solche Liste ist z. B. nützlich, wenn man ein Angestellten-Objekt finden will. Hierfür wird die Methode „*findeAngestellten( )*" verwendet. Sie ist ebenfalls unterstrichen, d. h. es handelt sich um eine Klassenmethode. Die Methode wird also normalerweise nicht bei einem einzelnen Objekt aufgerufen, sondern bei der Klasse:

```
Angestellter a = Angestellter.findeAngestellten("Schmidt");
```

Hierzu geht die Klasse alle Angestellten-Objekte in der Liste „*alleAngestellten*" durch und vergleicht den Nachnamen mit dem gesuchten Nachnamen. Klassenmethoden werden in Java ebenso wie Klassenattribute als „*static*" gekennzeichnet.

In Listing 13 sieht man die Anwendung des hier verwendeten Einsatzbeispiels von veränderlichen und unveränderlichen Klassenattributen. Wenn ein neues Angestelltenobjekt angelegt wird, wird die Personalnummer aus dem unveränderlichen Präfix und dem Wert des Personalnummernzählers zusammengesetzt. Anschließend wird der Wert des Personalnummernzählers um eins erhöht, damit das nächste Angestelltenobjekt eine neue Nummer erhält. Die Angestelltenobjekte erhalten somit die Personalnummern „*ANG_1*", „*ANG_2*", usw. Außerdem muss der Konstruktor das neu angelegte Objekt in die Liste „*alleAngestellten*" eintragen, damit es auch von der Methode „*findeAngestellten( )*" gefunden werden kann.

```
public Angestellter(){
      personalnummer = praefix + personalnummernzaehler;
      personalnummernzaehler++;
      alleAngestellten.add(this);
}
```

Listing 13: Im Konstruktor werden die Personalnummer zusammengesetzt, der Zähler inkrementiert und das Objekt in die Liste aller Angestellten eingetragen.

Dem Attribut „*nettogehalt*" in Abbildung 75 ist ein kleiner Schrägstrich vorangestellt, da es sich um ein sogenanntes abgeleitetes Attribut handelt. Den Wert eines abgeleiteten Attributs

kann man immer aus anderen Werten ermitteln. Die Berechnungsvorschrift ist hier als Zusicherung in geschweiften Klammern angegeben. Eigentlich bräuchte man das Attribut „*nettogehalt*" überhaupt nicht, da man seinen Wert ja jederzeit aus „*bruttogehalt*" und „*abzuege*" berechnen kann. Sinnvoll kann ein solches Attribut sein, wenn die Berechnung – anders als in diesem Fall – sehr zeitaufwändig ist und man sie aus Performancegründen nicht bei jedem Zugriff erneut ausführen will.

In Java gibt es keine spezielle Kennzeichnung für ein abgeleitetes Attribut. Im Beispiel ist die Berechnungsvorschrift als Kommentar dokumentiert. Die Verwendung eines abgeleiteten Attributs hat aber an anderer Stelle Konsequenzen: Jedes Mal, wenn sich der Wert eines der zugrunde liegenden Attribute – „*bruttogehalt*" und „*abzuege*" – ändert, muss auch der Wert von „*nettogehalt*" angepasst werden.

7.2 Aufzählungstypen

Soll anstelle eines elementaren Datentyps oder einer Klasse ein Aufzählungstyp verwendet werden, so kann dieser ebenfalls im Klassendiagramm modelliert werden. Er wird ähnlich einer Klasse dargestellt und mit dem Stereotyp «e*numeration*» gekennzeichnet (Abbildung 76). Die Eintragungen im unteren Feld sind keine Attribute, sondern die für den Typ definierten Werte.

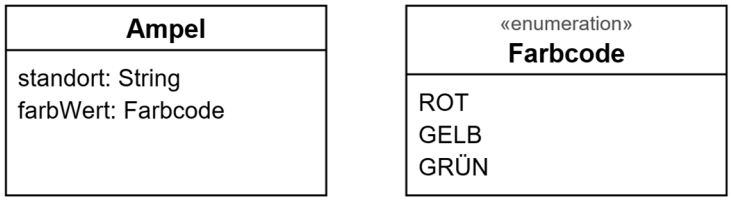


Abbildung 76: Aufzählungstyp (rechts) und seine Verwendung als Datentyp eines Attributs (links)

Der Aufzählungstyp kann dann wie ein gewöhnlicher Datentyp für ein Attribut, einen Parameter oder den Rückgabetyp einer Methode verwendet werden. Ein Attribut mit dem Aufzählungstyp kann man alternativ auch als Assoziation darstellen (Abbildung 77), auch wenn keine Verbindung mit einem Objekt angelegt wird, sondern mit einem Wert.

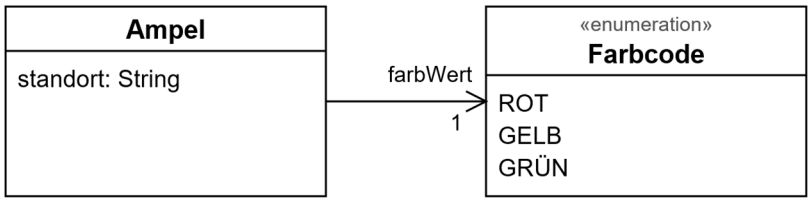


Abbildung 77: Alternative Darstellung der Verwendung eines Aufzählungstyps mittels Assoziation

7.3 Reflexive Assoziationen

Im Gegensatz zu der Assoziation in Abbildung 75, die als Klassenattribut implementiert ist, handelt es sich bei der Assoziation in Abbildung 78 um eine gewöhnliche Assoziation, d. h. es

werden Objekte miteinander verbunden. Da es sich bei Quelle und Ziel der Assoziation jeweils um die Klasse „*Angestellter*" handelt, können hiermit Beziehungen zwischen unterschiedlichen Angestellten-Objekten angelegt werden. Dabei nimmt der eine Angestellte gegenüber dem anderen die Rolle „*chef*" ein. Gemäß den Multiplizitäten kann jeder Angestellte maximal einen Chef haben. Umgekehrt kann jeder Angestellte Chef von beliebig vielen anderen Angestellten sein. Eine Assoziation, die eine Klasse mit sich selbst verbindet, wird reflexive Assoziation genannt.

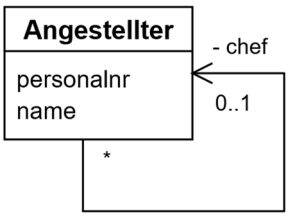


Abbildung 78: Reflexive Assoziation

Auch bei reflexiven Assoziationen ist es wichtig, zumindest eine Rolle einzutragen. Dadurch kann man die beiden Assoziationsenden unterscheiden. Betrachtet man zwei miteinander verbundene Angestellter-Objekte, so genügt es nicht zu wissen, dass zwischen den beiden ein Vorgesetztenverhältnis existiert. Vielmehr ist es auch von Bedeutung, welcher der beiden Angestellten der Chef ist und welcher der Untergebene.

Die Implementierung einer reflexiven Assoziation erfolgt genauso wie die einer gewöhnlichen Assoziation (Listing 14). Die Klasse „*Angestellter*" erhält ein Attribut „*chef*" vom Typ „*Angestellter*". Diesem kann man ein anderes Angestellten-Objekt zuweisen, was von der Methode „*ordneChefZu( )*" erledigt wird.

```
public class Angestellter {

	private int personalnr;
	private String name;
	private Angestellter chef;

	public Angestellter(int personalnr, String name){
		this.personalnr = personalnr;
		this.name = name;
	}

	public void ordneChefZu(Angestellter chef){
		this.chef = chef;
	}
}
```

Listing 14: Implementierung der reflexiven Assoziation

Es lassen sich mehrstufige Hierarchien aufbauen, denn jeder Angestellte kann einerseits einen Chef haben, andererseits selbst Chef von anderen Angestellten sein. In Listing 15 werden zunächst sieben Angestellter-Objekte erstellt. Anschließend werden mehrere Chef-Beziehungen angelegt.

```
Angestellter a1 = new Angestellter(1, "Minnie");
Angestellter a2 = new Angestellter(2, "Donald");
Angestellter a3 = new Angestellter(3, "Dagobert");
Angestellter a4 = new Angestellter(4, "Tick");
Angestellter a5 = new Angestellter(5, "Trick");
Angestellter a6 = new Angestellter(6, "Track");
Angestellter a7 = new Angestellter(7, "Micky");
a1.ordneChefZu(a3);
a2.ordneChefZu(a3);
a4.ordneChefZu(a2);
a5.ordneChefZu(a2);
a6.ordneChefZu(a2);
a7.ordneChefZu(a1);
```

Listing 15: Anlegen einer Hierarchie von Angestellten

Die sich ergebende Hierarchie ist in Abbildung 79 als Objektdiagramm dargestellt. Auch im Objektdiagramm ist es wichtig, die Rollen einzutragen, da sonst nicht klar ist, wer Chef und wer Untergebener ist. Es ist zwar naheliegend, das jeweils weiter oben gezeichnete Objekt als Chef anzusehen, doch hat die geometrische Anordnung im Klassen- und Objektdiagramm eigentlich keine Bedeutung.

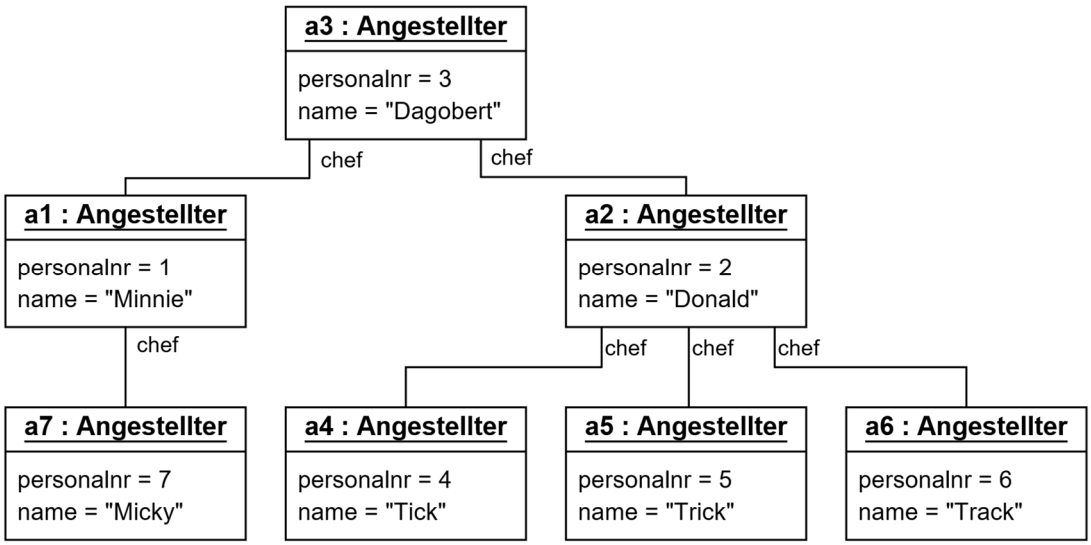


Abbildung 79: Reflexive Assoziation – Objekte

7.4 Aggregationen

Bei gewöhnlichen Assoziationen sind beide beteiligten Klassen gleichberechtigt. Eine besondere Art der Assoziation ist die Aggregation. Hier sind die beiden Klassen nicht mehr gleichberechtigt, sondern eine Klasse ist der anderen übergeordnet. Oft steht die übergeordnete Klasse, das sogenannte Aggregat, für ein zusammengesetztes Objekt. Die untergeordnete Klasse, als Komponente bezeichnet, steht dann für ein Teil, das zu dem Aggregat gehört. Man bezeichnet die Aggregation daher auch als „Ist Teil von"-Beziehung. Gekennzeichnet wird die Aggregation durch eine kleine, nicht ausgefüllte Raute an dem Ende, das sich beim Aggregat befindet.

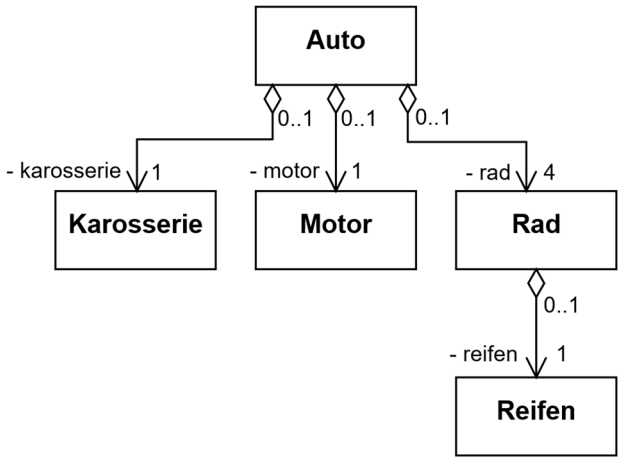


Abbildung 80: Aggregation

In Abbildung 80 besteht ein Auto aus einer Karosserie, einem Motor und vier Rädern. Dabei ist das Rad selbst wieder ein Aggregat, es besteht aus einem Reifen. Weitere Komponenten, wie z. B. die ebenfalls zum Rad gehörende Felge, wurden aus Platzgründen nicht modelliert.

Implementiert wird eine Aggregation zunächst wie eine ganz gewöhnliche Assoziation. Im vorliegenden Fall erhält das Auto ein Attribut vom Typ „Karosserie", eines vom Typ „Motor" und ein „Rad"-Array der Größe 4. Auch die umgekehrten Navigationsrichtungen von den Komponenten zum Aggregat wären möglich.

Die hierarchische Aggregationsbeziehung hat jedoch unter Umständen Konsequenzen für verschiedene Methoden. Z. B. muss man sich beim Kopieren oder Verschieben eines Aggregats überlegen, ob die Komponenten mitkopiert oder verschoben werden sollen.

Prinzipiell können die Komponenten – je nach Multiplizität – auch für sich alleine existieren und auch vom Aggregat entfernt werden. Im Beispiel aus Abbildung 80 ist die Multiplizität am Aggregat jeweils 0..1, d. h. ein Motor oder ein Reifen kann auch für sich existieren, ohne zu einem Aggregat zu gehören.

Es ist auch möglich, dass eine Komponente zu mehr als einem Aggregat gehört. Im Beispiel des Autos wäre dies nicht sinnvoll. Ein Motor kann zur selben Zeit sicherlich nur zu einem Auto gehören.

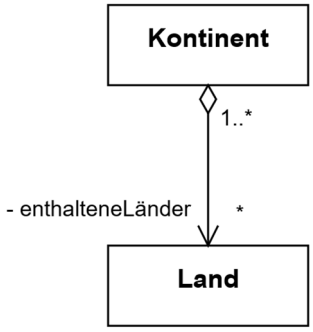


Abbildung 81: Ein Land kann Teil mehrerer Kontinente sein

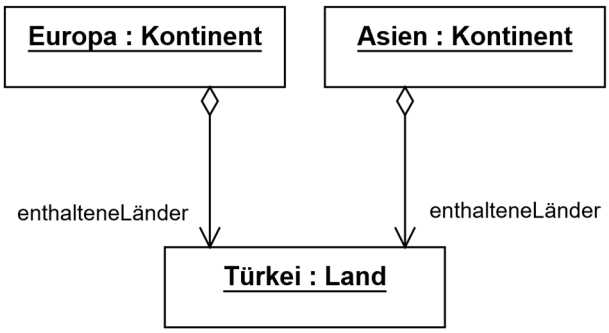


Abbildung 82: Die Türkei gehört zu Europa und zu Asien

Anders im Beispiel von Abbildung 81. Hier ist modelliert, dass Kontinente Länder enthalten können. Dabei kann ein Land durchaus zu mehreren Kontinenten gehören, wie im Objektdiagramm in Abbildung 82. Die Türkei gehört sowohl zu Europa als auch zu Asien.

Wie das Beispiel aus Abbildung 80 zeigt, können Aggregationen mehrstufig sein. Der Reifen kann Teil eines Rades sein, und damit indirekt Teil eines Autos. Durch eine Aggregation wird eine strenge Hierarchie aufgebaut.

Dies bedeutet insbesondere, dass eine Komponente selbst nicht wieder einem zugehörigen Aggregat übergeordnet sein darf. Es dürfen keine Zyklen in der Objektstruktur entstehen.

Das Beispiel in Abbildung 83, bei dem das Auto selbst wieder Teil des enthaltenen Motors ist, ist natürlich inhaltlich unsinnig. Dennoch: Hätte man gewöhnliche Assoziationen verwendet, wäre das Modell formal nicht zu beanstanden. Erst durch die Aggregationen wird im Modell ausgedrückt, dass es sich um eine strenge Hierarchie handelt und keine Zyklen existieren dürfen.

Die in Kapitel 7.3 vorgestellte reflexive Assoziation zwischen Angestellten stellt inhaltlich auch eine Hierarchie dar. Entsprechend liegt es nahe, sie ebenfalls als Aggregation zu modellieren (Abbildung 84).

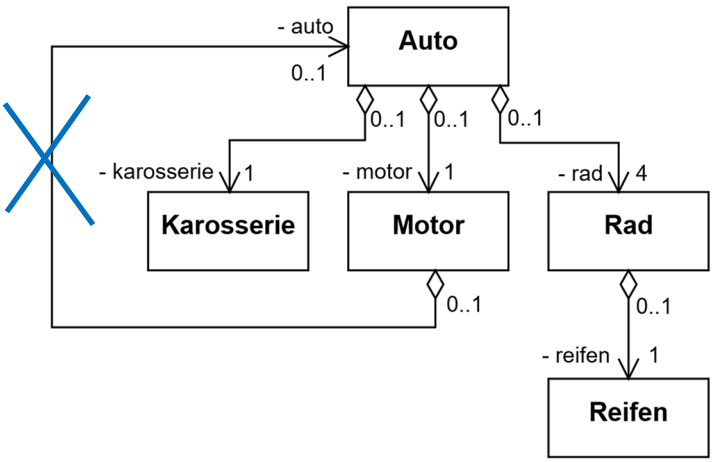


Abbildung 83: Beispiel für einen unzulässigen Zyklus

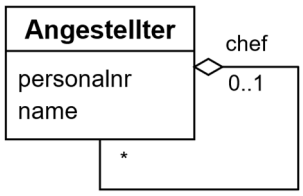


Abbildung 84: Die reflexive Assoziation sieht aus wie ein Zyklus. Verboten sind Zyklen jedoch nur auf Ebene der Objekte.

Andererseits scheint die Verbindung einer Klasse zu sich selbst ebenfalls einen Zyklus darzustellen, wie er für Aggregationen gerade ausgeschlossen wurde. Genau genommen bezieht sich das Zyklusverbot aber nicht auf Klassen, sondern auf Objekte und ihre Beziehungen. Von daher ist das Klassendiagramm in Abbildung 84 in Ordnung. Die Aggregation gibt an, dass unter den Objekten eine strenge Hierarchie bestehen muss, wie das beispielsweise in Abbildung 79 der Fall ist.

7.5 Kompositionen

Eine besondere Form der Aggregation ist die Komposition. Sie wird durch eine gefüllte Raute gekennzeichnet. Damit wird ausgedrückt, dass die Komponente ein untrennbarer Bestandteil des Aggregats ist und ohne dieses auch nicht sinnvoll existieren könnte.

Ein typisches Beispiel für eine Komposition zeigt Abbildung 85. Ein Auftrag besteht in der Regel aus mehreren einzelnen Auftragspositionen. Jede Position bezieht sich auf ein anderes Produkt. Die Auftragsposition enthält insbesondere das Attribut „*menge*", das angibt, wie viele Stück des betreffenden Produkts bestellt worden sind. Ein Beispiel eines Auftrags mit mehreren Positionen ist in Abbildung 86 dargestellt.

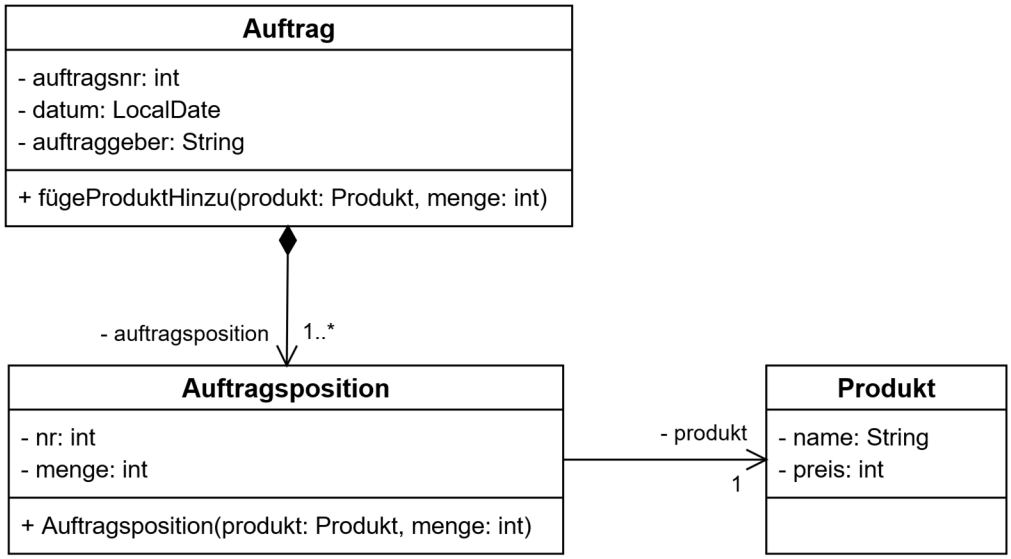


Abbildung 85: Komposition

Auftrag Nr. 4711

Hans Meier 7.10.2026
Amerikastr. 1
66482 Zweibrücken

Position Nr.	*Menge*	*Bezeichnung*	*Einzelpreis*	*Summe*
1	10	Ordner	€ 2,89	€ 28,90
2	1	Locher	€ 12,55	€ 12,55
3	5	Papier (500 Blatt)	€ 3,99	€ 19,95
Gesamtsumme				€ 61,40

Abbildung 86: Beispiel für einen Auftrag mit mehreren Positionen

Eine Auftragsposition ist nur als Teil eines Auftrags von Bedeutung. Man würde kaum auf die Idee kommen, z. B. eine Auftragsposition anzulegen um sie erst anschließend einem Auftrag zuzuordnen. Ebenso wie ein Attribut untrennbarer Teil einer Klasse ist, so ist es in diesem Fall auch die Komponente „*Auftragsposition*".

Dies hat einige Konsequenzen: Zum einen braucht an dem Ende mit der Raute keine Multiplizität angegeben zu werden. Diese ist zwangsläufig 1, denn eine Komponente kann ja kaum untrennbarer Bestandteil von zwei Aggregaten sein. Zum anderen ist das Aggregat für das Anlegen, Ändern und Löschen der Komponenten zuständig. So erfolgt im Beispiel das Anlegen einer neuen Auftragsposition über die Methode „*fuegeProduktHinzu( )*" der Klasse Auftrag. Diese ruft dann den Konstruktor der Auftragsposition auf.

Dieser Konstruktor sollte eigentlich immer nur vom Auftrag aus aufgerufen werden. In Java lässt sich dies jedoch nicht so einfach sicherstellen. Zumindest wurde die Sichtbarkeit des Konstruktors „*Auftragsposition( )*" auf das Package eingeschränkt, so dass kein Aufruf von außerhalb des Packages möglich ist.

Die Implementierung erfolgt zumeist wie bei einer gewöhnlichen Assoziation über entsprechende Attribute bzw. Listen. Alternativ könnte man die Auftragsposition in Java auch als innere Klasse (*Inner Class*) implementieren.

Die Kompositionseigenschaft ist vor allem bei den Methoden zu berücksichtigen. So benötigt das Aggregat Methoden, über die man indirekt Komponenten anlegen und bearbeiten kann. Es sollte möglichst kein direkter Aufruf der Komponentenmethoden durch andere Klassen als das Aggregat erfolgen. Entsprechend sollten möglichst auch keine Assoziationen genutzt werden, über die von anderen Klassen direkt zu den Komponenten navigiert werden kann. Stattdessen sollten derartige Assoziationen zum Aggregat führen. Und schließlich sind beim Kopieren bzw. Löschen des Aggregats auch die Komponenten mit zu kopieren bzw. zu löschen.

7.6 Assoziative Klassen

Mit der Assoziation zwischen Kunde und Konferenz in Abbildung 87 wird ausgedrückt, welche Kunden zu welchen Konferenzen angemeldet sind. Zu der Anmeldung sind aber noch weitere Informationen von Interesse, wie das Anmeldedatum und ob der jeweilige Kunde am Konferenz-Dinner teilnehmen möchte. Hierbei handelt es sich nicht um Attribute des Kunden, denn ein Kunde kann an mehreren Konferenzen angemeldet sein, wobei Anmeldedatum und Konferenz-Dinner-Teilnahme jeweils unterschiedlich sein können. Genauso wenig gehören diese Attribute zur Konferenz, da für jede Konferenz viele Kunden mit unterschiedlichen Anmeldedaten und Konferenz-Dinner-Teilnahmewünschen angemeldet sein können.

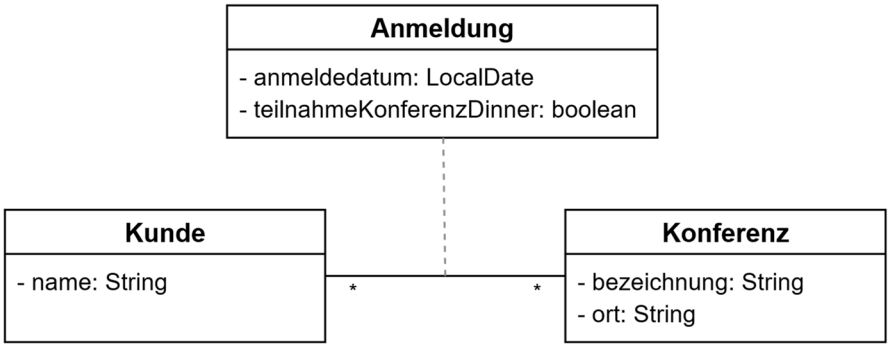


Abbildung 87: Assoziative Klasse „Anmeldung“

Das Anmeldedatum und die Konferenz-Dinner-Teilnahme hängen von der Kombination aus Kunde und Konferenz ab. Ein bestimmtes Anmeldedatum gibt beispielsweise an, wann sich Kunde A bei der Konferenz X angemeldet hat. Die Attribute „*anmeldedatum*“ und „*teilnahmeKonferenzDinner*“ gehören also zur Assoziation. Eine solche Assoziation, die zusätzlich Attribute und Methoden wie eine Klasse hat, heißt assoziative Klasse. Sie wird als Klassensymbol dargestellt, das durch eine gestrichelte Linie mit der Assoziation verbunden ist. Eine assoziative Klasse hat immer eine Bezeichnung, die in das Namensfeld des Klassensymbols eingetragen wird.

Eine direkte Implementierung einer assoziativen Klasse in Java ist nicht möglich. Daher löst man sie zunächst auf in eine normale Klasse und verbindet diese über gewöhnliche Assoziationen mit den zwei beteiligten Klassen (Abbildung 88). Jedes Anmeldung-Objekt repräsentiert die Kombination aus einem Kunden- und einem Konferenz-Objekt. Es bezieht sich also auf genau einen Kunden und eine Konferenz. Daher sind die Multiplizitäten an den Assoziationsenden bei Kunde und Konferenz jeweils 1. Umgekehrt kann es für einen Kunden beliebig viele Anmeldungen geben, da er sich ja bei verschiedenen Konferenzen angemeldet haben kann. Entsprechend gibt es für eine Konferenz mehrere Anmeldungen – eine für jeden angemeldeten Kunden. Die Multiplizitäten auf der anderen Seite sind daher „*“.

Abbildung 88: Aufgelöste assoziative Klasse

Da nun gewöhnliche Assoziationen vorliegen, kann die Implementierung wie gewohnt erfolgen. Einen Unterschied gibt es jedoch zwischen den Varianten von Abbildung 87 und Abbildung 88, und zwar dann, wenn die als assoziative Klasse ausgeprägte Assoziation die Eigenschaft „{*unique*}" hat. Im Normalfall geht man davon aus, dass Assoziationen unique sind, d. h. dass es zwischen zwei Objekten der beiden beteiligten Klassen maximal eine Beziehung geben kann. Dies ist auch für das betrachtete Beispiel zweckmäßig. Dass ein und derselbe Kunde zweimal für dieselbe Konferenz angemeldet ist, ist nicht besonders sinnvoll.

Betrachtet man aber die aufgelösten Klassen aus Abbildung 88, so ist diese Einschränkung dort nicht enthalten. Zu jedem Kunden kann es beliebig viele Anmeldungen geben. Ob diese sich – wie es gedacht ist – jeweils auf andere Konferenzen beziehen, oder ob versehentlich mehrere Anmeldungen des Kunden mit demselben Konferenz-Objekt verbunden sind, das geht nicht aus dem Modell hervor. Die Multiplizitätsangaben beziehen sich ja immer nur auf eine einzelne Assoziation. Um sicherzustellen, dass es für jeden Kunden pro Konferenz maximal eine Anmeldung gibt, müssen aber beide Assoziationen betrachtet werden.

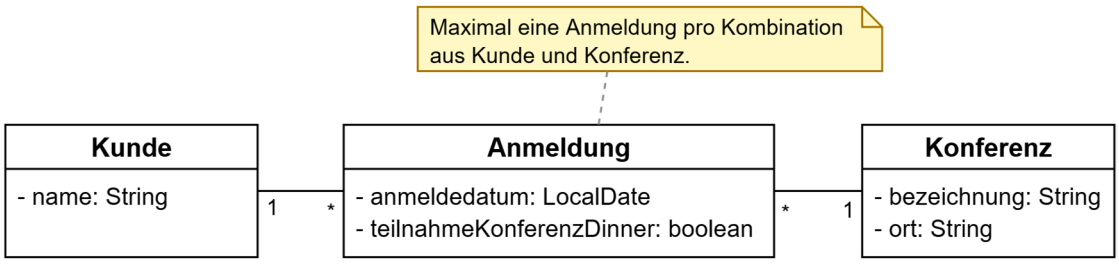


Abbildung 89: Angabe der erforderlichen Zusatzbedingung als Kommentar

Die Zusatzbedingung ist daher in einem erläuternden Text oder im Modell als Anmerkung zu dokumentieren (Abbildung 89). Generell gilt: Wichtige Sachverhalte, die sich nicht direkt im Modell darstellen lassen, sind als Anmerkungen oder Erläuterungen zu dokumentieren.

7.7 Mehrfachassoziationen

Die bisher betrachteten Assoziationen haben immer zwei Klassen miteinander verbunden. Es gibt aber auch die Möglichkeit, drei oder mehr Klassen über Assoziationen miteinander zu verbinden. So verbindet die Assoziation in Abbildung 90 die Klassen Verein, Jahr und Spieler. Man kann damit also abbilden, welcher Spieler in welchem Jahr in welchem Verein gespielt hat.

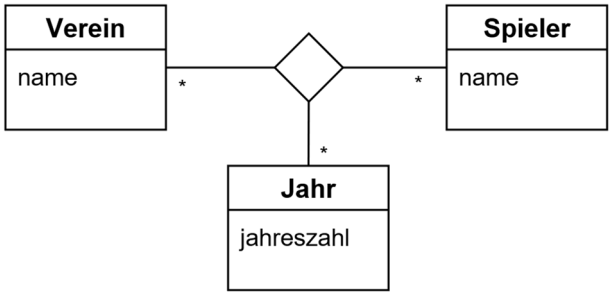


Abbildung 90: Mehrfachassoziation

Eine Mehrfachassoziation wird als Raute dargestellt, die mit den beteiligten Klassen verbunden ist. Die Multiplizität ist folgendermaßen zu lesen: Man wählt je ein Objekt von zwei der beteiligten Klassen. Dann kann man ablesen, mit wie vielen Objekten der dritten Klasse sie verbunden sind.

Im Beispiel aus Abbildung 90 sind alle drei Multiplizitäten „*“. Betrachtet man beispielsweise einen bestimmten Verein und ein bestimmtes Jahr, dann können beliebig viele Spieler zugeordnet sein. Betrachtet man einen bestimmten Spieler und bestimmtes Jahr, dann sind beliebig viele Vereine zugeordnet. Da es möglich ist, dass der Spieler während des betrachteten Jahres den Verein gewechselt hat, ist das sinnvoll. Einer bestimmten Kombination aus Spieler und Verein sind beliebig viele Jahre zugeordnet. Ein Spieler kann ja mehrere Jahre im selben Verein spielen.

Auch Mehrfachassoziationen können assoziative Klassen sein. In Abbildung 91 erhält die assoziative Klasse das Attribut „*tore*“. Damit kann man die Frage beantworten: „Welcher Spieler hat in welchem Jahr in welchem Verein wie viele Tore geschossen?“. Auch hier ist wieder der mögliche Vereinswechsel eines Spielers in einem Jahr berücksichtigt. Man möchte in diesem Fall eben nicht nur wissen, wie viele Tore der Spieler in einem bestimmten Jahr geschossen hat. Vielmehr soll dies auch noch nach den Vereinen unterschieden werden, für die er diese Tore erzielt hat.

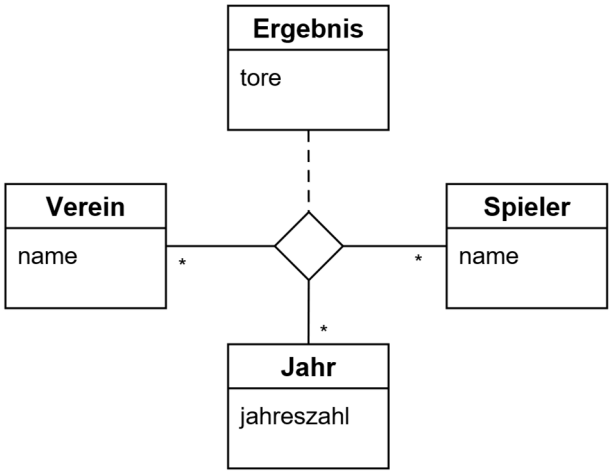


Abbildung 91: Auch Mehrfachassoziationen können assoziative Klassen sein

Auch Mehrfachassoziationen lassen sich nicht direkt in Java implementieren. Daher löst man die Mehrfachassoziation ebenfalls in gewöhnliche Klassen und Assoziationen auf. Auch hier ersetzt man die Assoziation durch eine gewöhnliche Klasse und verbindet sie über normale Assoziationen mit den anderen Klassen. Die Multiplizitäten und die Zusatzbedingung in Abbildung 92 entsprechen denen der Auflösung einer zweiwertigen assoziativen Klasse, wie oben in Kapitel 7.6 beschrieben. Auch wenn die Mehrfachassoziation nicht zugleich eine assoziative Klasse ist, wird sie wie in Abbildung 92 aufgelöst. Die zentrale Klasse enthält dann zunächst eben keine Attribute und Methoden, sondern nur die Assoziationen zu den anderen Klassen.

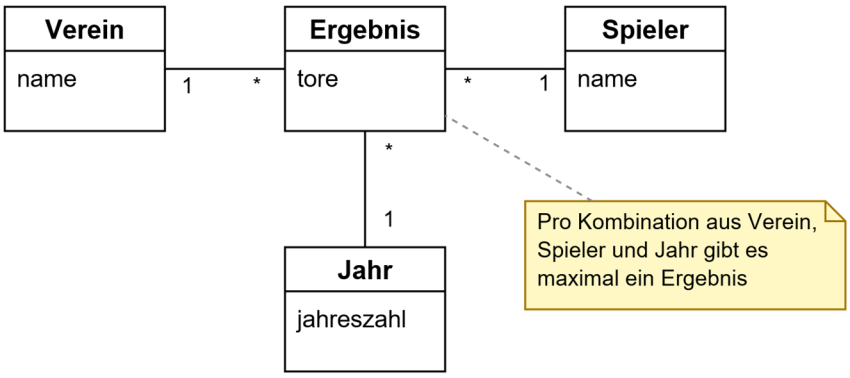


Abbildung 92: Auflösung von Mehrfachassoziationen

8 Klassendiagramme – Vererbung

Die Vererbung ist ein wichtiges Konzept der Objektorientierung. Spezialisierte Klassen erben die Eigenschaften ihrer Oberklassen und ermöglichen so die Wiederverwendung in verschiedenen Zusammenhängen. Es werden Methoden, Attribute und Assoziationen vererbt – auch über mehrere Stufen hinweg. Objekte der Unterklasse können anstelle von Objekten der Oberklasse verwendet werden. Abstrakte Klassen dienen ausschließlich dazu, als Oberklassen eingesetzt zu werden. Eng verwandt ist das Konzept der Schnittstelle (*Interface*). Wenn eine Klasse eine bestimmte Schnittstelle implementiert, wird damit zugesichert, dass sie ein bestimmtes Verhalten aufweist.

8.1 Generalisierung und Spezialisierung

Die Klassen „*Kaufvertrag*" und „*Mietvertrag*" in Abbildung 93 sind Spezialisierungen der Klasse „*Vertrag*". „*Vertrag*" ist die Generalisierung von „*Kaufvertrag*" und „*Mietvertrag*". Kaufvertrag und Mietvertrag werden auch als Unterklassen bezeichnet, Vertrag als Oberklasse.

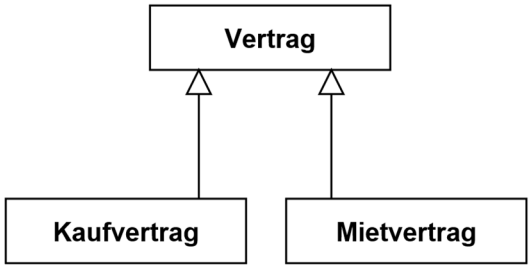


Abbildung 93: Generalisierung/Spezialisierung

Man spricht außerdem von der Generalisierungsbeziehung als „ist ein"-Beziehung. Durch das Modell wird ausgedrückt: Ein Kaufvertrag *ist ein* Vertrag. Und: Ein Mietvertrag *ist ein* Vertrag.

Die Pfeilspitze der Generalisierungsbeziehung besteht aus einem geschlossenen, nicht ausgefüllten Dreieck, das in der Regel etwas größer als eine gewöhnliche Pfeilspitze ist. Die richtige Darstellung ist wichtig, damit man die Generalisierungsbeziehung eindeutig von einer Assoziation mit Navigationspfeil unterscheiden kann. Generalisierungen wurden bereits im Use-Case-Diagramm verwendet (Kapitel 4.7), allerdings ist die Bedeutung im Zusammenhang mit Klassen wesentlich genauer definiert.

Java als objektorientierte Programmiersprache bietet die Möglichkeit zur Spezialisierung von Klassen. Das Diagramm aus Abbildung 93 kann daher direkt umgesetzt werden (Listing 16). Die Spezialisierung wird in der Unterklasse mit Hilfe des Schlüsselworts „*extends*" angegeben. Trotz des Namens hat das übrigens nichts mit der extend-Beziehung in Use-Case-Diagrammen zu tun.

```
public class Vertrag {
}

public class Kaufvertrag extends Vertrag {
```

```
}

public class Mietvertrag extends Vertrag {
}
```

Listing 16: Spezialisierung in Java

8.2 Vererben von Methoden und Attributen

Eine wichtige Eigenschaft von Generalisierungsbeziehungen ist die Vererbung. Die Oberklassen vererben ihre Attribute und Methoden an die Unterklassen. So ist in Abbildung 99 unter anderem modelliert, dass jedes Vertragsobjekt die Attribute *datum*, *vertragsgegenstand* und *bedingungen* enthält. Da ein Kaufvertrag laut Modell ebenfalls ein Vertrag ist, enthält auch jedes Kaufvertragsobjekt die Attribute *datum*, *vertragsgegenstand* und *bedingungen*. Zusätzlich enthält es noch das Attribut *kaufsumme*. Abbildung 95 zeigt ein Objekt der Klasse Vertrag und ein Objekt der Klasse Kaufvertrag. Analog würde ein Objekt der Klasse Mietvertrag die Attribute *datum*, *vertragsgegenstand*, *bedingungen*, *laufzeit* und *monatsrate* enthalten.

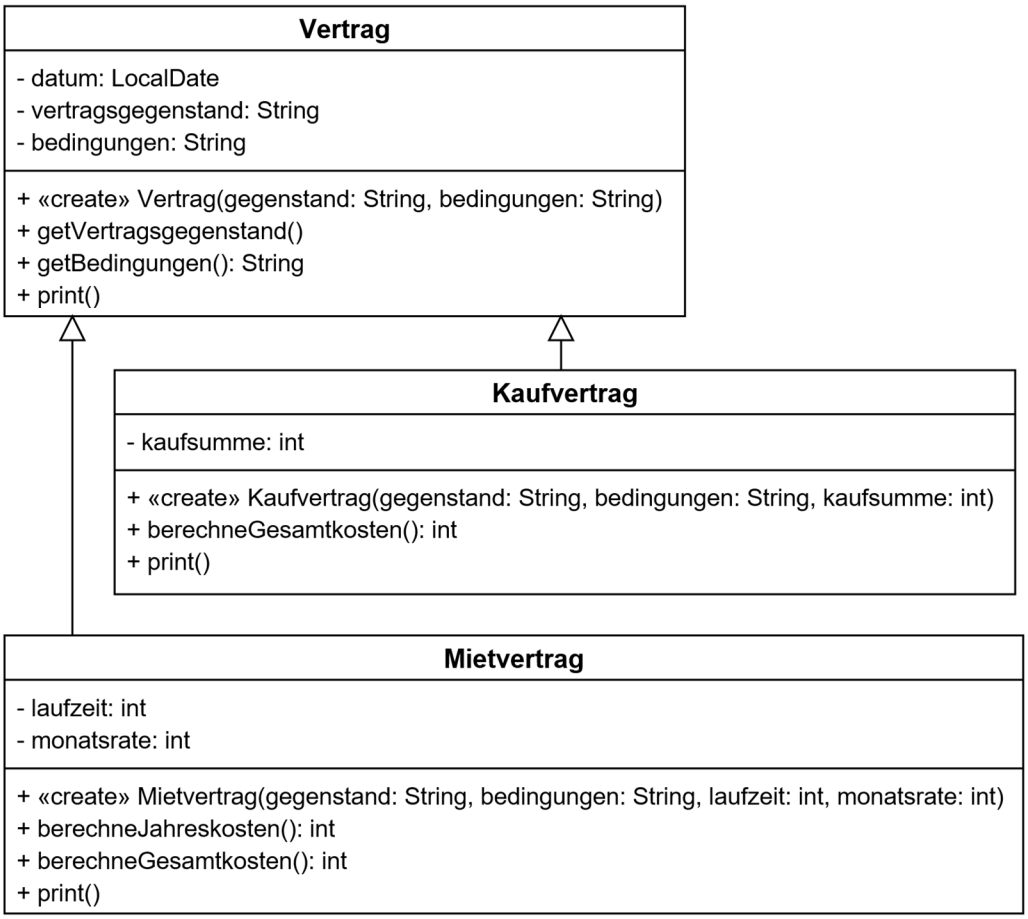


Abbildung 94: Vererbung

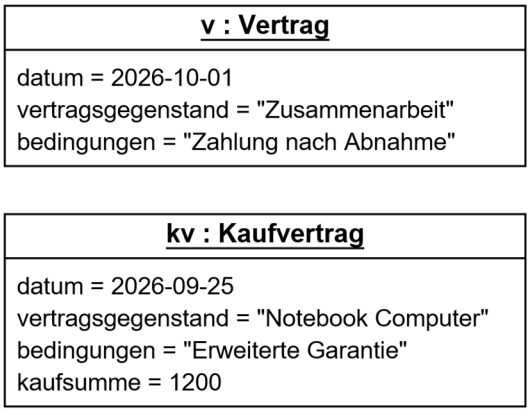


Abbildung 95: Objekte der Ober- und der Unterklasse

Auch die Methoden werden vererbt, d. h. bei einem Objekt der Unterklasse kann man die Methoden der Oberklasse aufrufen und zusätzlich die direkt in der Unterklasse definierten Methoden. Bei einem existierenden Kaufvertrag-Objekt kann man gemäß Abbildung 99 also folgende Methoden aufrufen: *getVertragsgegenstand( )*, *getBedingungen( )*, *print( )*, *berechneGesamtkosten( )*. Die Methode *print( )* ist in der Ober- und in der Unterklasse enthalten. Hierbei überschreibt die Implementierung der Unterklasse die der Oberklasse.

Die Implementierung der beiden Klassen Vertrag und Kaufvertrag ist aus Listing 17 und Listing 18 ersichtlich. Eine Bemerkung zum Konstruktor der Unterklasse: Da in der Oberklasse explizit ein Konstruktor definiert wurde, ist der ansonsten vorhandene parameterlose Default-Konstruktor „*Vertrag( )*" nicht mehr sichtbar. In einem solchen Fall muss die Unterklasse ebenfalls einen expliziten Konstruktor haben. In diesem muss als erstes mit dem Schlüsselwort „*super*" der Konstruktor der Oberklasse aufgerufen werden. Im vorliegenden Fall weist der Konstruktor der Oberklasse den Attributen *vertragsgegenstand* und *bedingungen* ihre Werte zu. Anschließend fährt der Konstruktor der Unterklasse fort und weist dem in der Unterklasse definierten Attribut „*kaufsumme*" einen Wert zu.

Wie bereits erwähnt, überschreibt die Methode „*print( )*" der Klasse Kaufvertrag die gleichnamige Methode der Oberklasse Vertrag. Genau genommen müssen die beiden Methoden nicht nur gleich heißen, sondern die gleiche Signatur haben. Neben dem Namen müssen also auch Zahl und Datentypen der Parameter übereinstimmen. Ruft man bei einem Objekt der Klasse Kaufvertrag die Methode „*print( )*" auf, so wird die Implementierung ausgeführt, die sich direkt in der Klasse Kaufvertrag befindet (Listing 18). Auch diese Methode ruft mit „*super.print()*" zunächst die gleichnamige Methode der Oberklasse auf, bevor sie noch einige zusätzliche Informationen auf der Konsole ausgibt. Im Gegensatz zu dem Konstruktor ist der Aufruf der gleichnamigen Oberklassen-Methode nicht vorgeschrieben.

```
import java.util.Date;

public class Vertrag {
	private Date datum;
	private String vertragsgegenstand;
	private String bedingungen;

	public Vertrag(String gegenstand, String bedingungen){
		this.vertragsgegenstand=gegenstand;
		this.bedingungen=bedingungen;
		this.datum=new Date();
	}

	public String getVertragsgegenstand(){
		return this.vertragsgegenstand;
	}

	public String getBedingungen(){
		return this.bedingungen;
	}

	public void print(){
		System.out.println(datum.toString() + " Gegenstand: " +
		this.vertragsgegenstand + "; Bedingungen: " +
		this.bedingungen);
	}
}
```

Listing 17: Implementierung der Klasse Vertrag

```
public class Kaufvertrag extends Vertrag {
	private int kaufsumme;

	public Kaufvertrag(String gegenstand, String bedingungen, int kaufsumme){
		super(gegenstand, bedingungen);
		this.kaufsumme = kaufsumme;
	}

	public void print(){
		super.print();
		System.out.println("Kaufsumme: " + kaufsumme + ", Gesamtkosten: " +
				berechneGesamtkosten());
	}

	public int berechneGesamtkosten(){
		return this.kaufsumme;
	}
}
```

Listing 18: Implementierung der Klasse Kaufvertrag

In Listing 19 wird ein Objekt der Oberklasse Vertrag und ein Objekt der Unterklasse Kaufvertrag angelegt. Anschließend werden alle zur Verfügung stehenden Methoden aufgerufen. Die aufgerufenen Methoden des Vertragsobjekts *v* sind alle in der Klasse Vertrag implementiert. Für das Objekt der Oberklasse ist es völlig belanglos, dass es noch Unterklassen seiner Klasse gibt.

```
Vertrag v          = new Vertrag("Zusammenarbeit", "Zahlung nach Abnahme");
Kaufvertrag kv     = new Kaufvertrag("Notebook", "Erweiterte Garantie", 1200);

String vGegenstand   = v.getVertragsgegenstand();
String vBedingungen = v.getBedingungen();
v.print();

String kvGegenstand   = kv.getVertragsgegenstand();
String kvBedingungen  = kv.getBedingungen();
kv.print();
int kvGesamtkosten    = kv.berechneGesamtkosten();
```

Listing 19: Objekte von Ober- und Unterklasse anlegen und verwenden

Die ersten beiden Methodenaufrufe „*getVertragsgegenstand( )*“ und „*getBedingungen( )*“ beim Kaufvertragsobjekt *kv* führen dazu, dass die Methodenimplementierungen der Oberklasse aus Listing 17 ausgeführt werden. Bei den folgenden Methodenaufrufen „*print( )*“ und „*berechneGesamtkosten( )*“ werden die Implementierungen der Unterklasse ausgeführt.

Prinzipiell gilt: Wird die Methode eines Objekts aufgerufen, so wird die in seiner Klasse vorhandene Implementierung ausgeführt. Nur bei Methoden, die nicht in der Klasse implementiert sind, wird die Methode der Oberklasse ausgeführt, die über die gleiche Signatur verfügt. Falls sich die Methode auch in der Oberklasse nicht findet, wird die entsprechende Methode aus deren Oberklasse ausgeführt, usw.

8.3 Substitutionsprinzip

Die Methode „*testeVertragsmethoden( )*“ in Listing 20 erwartet einen Parameter vom Typ Vertrag. In der Methode werden die verschiedenen Methoden der Klasse Vertrag aufgerufen. In Listing 21 wird der Methode einmal ein Objekt der Klasse Vertrag („*v*“) und das andere Mal ein Objekt der Unterklasse Kaufvertrag („*kv*“) übergeben. Sie funktioniert ohne weiteres auch mit diesem Objekt. In der Methode „*testeVertragsmethode( )*“ können auch dann nur die in der Oberklasse Vertrag implementierten Methoden aufgerufen werden, wenn es sich um ein Objekt einer Unterklasse handelt. Schließlich steht ja nicht von vornherein fest, zu welcher Klasse das tatsächlich übergebene Objekt gehört. Daher können nur die Gemeinsamkeiten der Oberklasse und all ihrer Unterklassen berücksichtigt werden. Und dies sind eben die Methoden, die in der Oberklasse definiert sind.

```
public void testeVertragsmethoden(Vertrag vertrag){
      String vertragsGegenstand = vertrag.getVertragsgegenstand();
      String vertragsBedingungen = vertrag.getBedingungen();
      vertrag.print();
}
```

Listing 20: Methode mit einem Parameter vom Typ Vertrag

```
Vertrag v     = new Vertrag("Zusammenarbeit",  "Zahlung nach Abnahme");
Kaufvertrag kv = new Kaufvertrag("Notebook", "Erweiterte Garantie", 1200);

testeVertragsmethoden(v);
testeVertragsmethoden(kv);
```

Listing 21: Aufruf der Methode testeVertragsmethoden( ) mit einem Objekt vom Typ der Oberklasse und mit einem Objekt vom Typ der Unterklasse

8.4 Sichtbarkeiten bei Vererbung

Dass die Attribute der Oberklasse Vertrag die Sichtbarkeit „*private*" haben, hat einen Nachteil: Man kann nämlich auch in der Unterklasse nicht direkt auf diese privaten Attribute zugreifen. Obwohl ein Objekt der Klasse Kaufvertrag z. B. über das Attribut vertragsgegenstand verfügt, ist kein direkter Zugriff in der folgenden Form möglich:

```
System.out.println(vertragsgegenstand);
```

Praktischerweise gibt es in der Klasse Vertrag in dem Beispiel eine öffentliche Methode „*getVertragsgegenstand()*", die natürlich auch für ein Objekt der Unterklasse aufgerufen werden kann:

```
System.out.println(getVertragsgegenstand());
```

Im Zusammenhang mit Vererbung ist es sinnvoll, diejenigen Attribute, die in Unterklassen benötigt werden, als „*protected*" zu kennzeichnen. Damit ist der direkte Zugriff aus allen Unterklassen möglich (Abbildung 96und Listing 22).

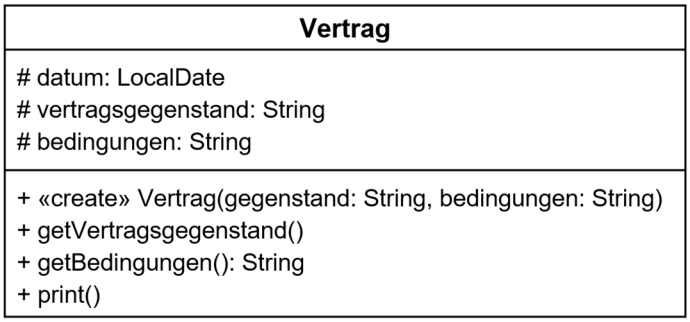


Abbildung 96: Attribute mit der Sichtbarkeit „protected" im Modell

```
import java.util.Date;

public class Vertrag {
	protected Date datum;
	protected String vertragsgegenstand;
	protected String bedingungen;

	// ...

}
```

Listing 22: Attribute mit der Sichtbarkeit „protected" im Code

8.5 Vererben von Assoziationen

Nun wird das Beispiel um zwei Klassen erweitert: Die Klasse Vertrag hat nun eine Assoziation zu einem Anbieter und eine zu einem Kunden (Abbildung 97). Welche Konsequenzen hat dies für die Unterklassen Kaufvertrag und Mietvertrag?

Die Antwort ist einfach, wenn man sich überlegt, wie Assoziationen implementiert werden: als Attribute. Da Attribute vererbt werden, werden folglich auch Assoziationen vererbt.

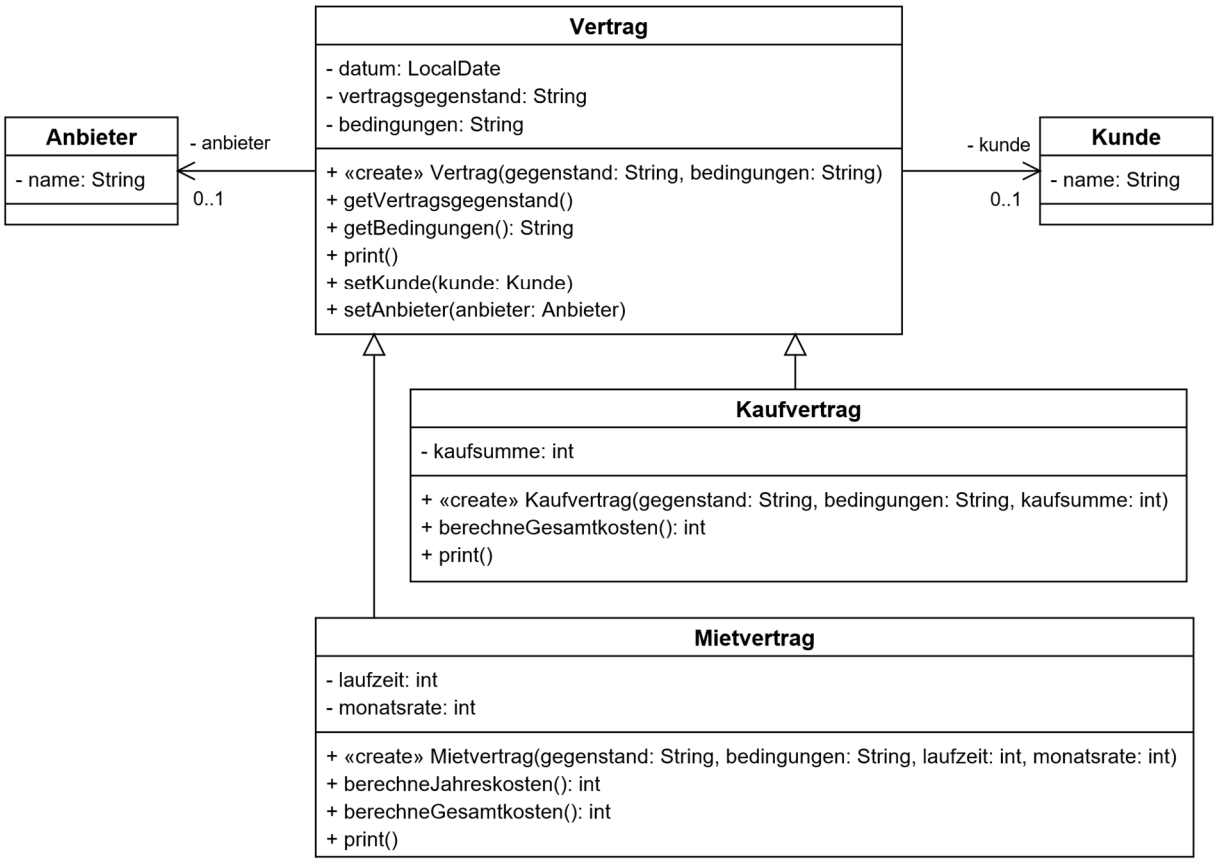


Abbildung 97: Vererbung von Assoziationen

Die Klasse Vertrag erhält ein Attribut „*anbieter*" vom Typ „*Anbieter*" und ein Attribut „*kunde*" vom Typ „*Kunde*" (Listing 23). In Listing 24 wird ein Objekt der Unterklasse Kaufvertrag angelegt, dem dann ein Anbieter-Objekt und ein Kunden-Objekt zugeordnet werden.

```
public class Vertrag {
	// ...
	private Anbieter anbieter;
	private Kunde kunde;

	public void setKunde(Kunde kunde){
		this.kunde = kunde;
	}

	public void setAnbieter(Anbieter anbieter){
		this.anbieter = anbieter;
	}

	// ...
}
```

Listing 23: Implementierung der Assoziationen zu Anbieter und Kunde

```
Kunde k         = new Kunde();
Anbieter a      = new Anbieter();
Kaufvertrag kv  = new Kaufvertrag("Notebook Computer", "Erweiterte Garantie",
                        1200);
kv.setKunde(k);
kv.setAnbieter(a);
```

Listing 24: Zuordnung von Kunde und Anbieter zu einem Objekt der Unterklasse Kaufvertrag

8.6 Mehrstufige Vererbungshierarchie

Wie bereits angesprochen können Unterklassen wiederum Unterklassen haben, wodurch beliebig tiefe Vererbungshierarchien aufgebaut werden können. In Abbildung 98 hat die Klasse Mietvertrag, die Unterklasse von Vertrag ist, selbst noch einmal eine Unterklasse Leasingvertrag.

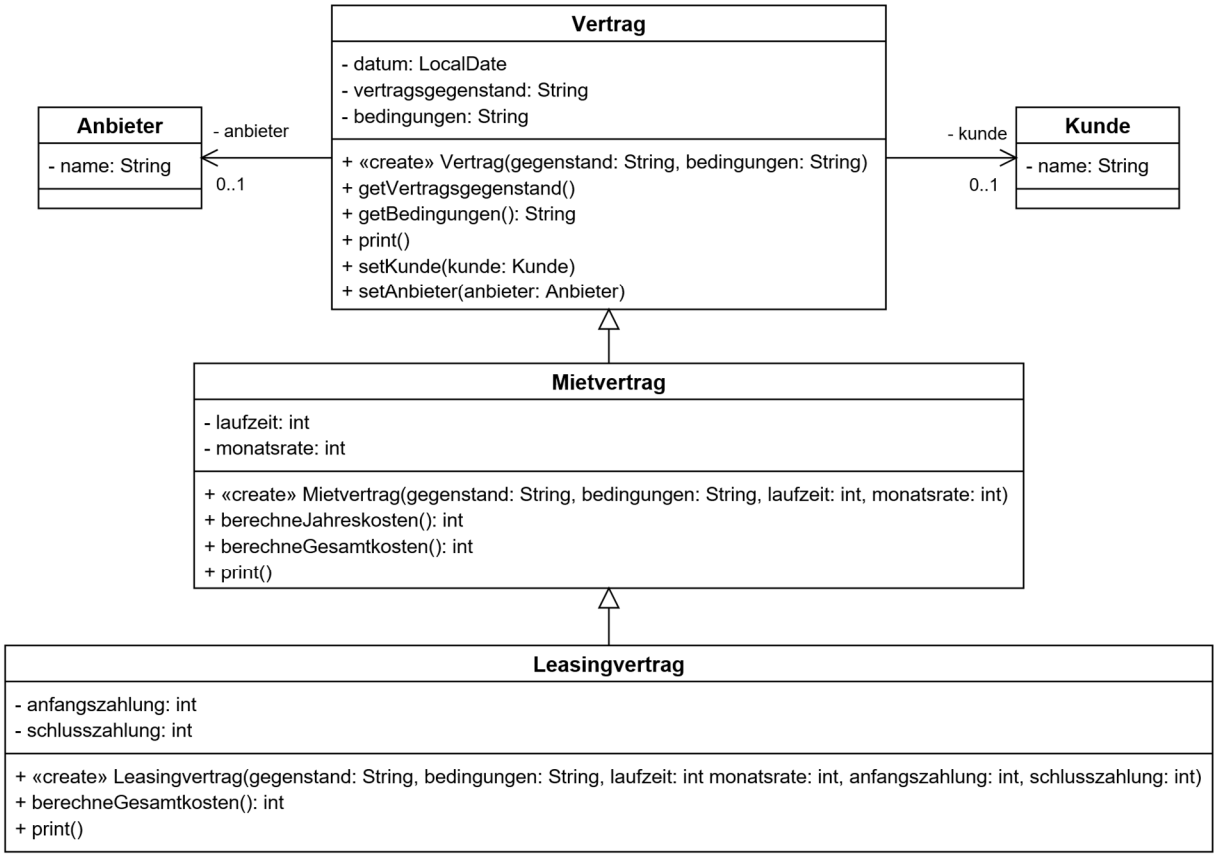


Abbildung 98: Mehrstufige Vererbungshierarchie

Es gilt also: Ein Leasingvertrag ist ein Mietvertrag. Und da ein Mietvertrag ein Vertrag ist, ist auch ein Leasingvertrag ein Vertrag. Entsprechend gilt auch das Substitutionsprinzip: Wo ein Objekt vom Typ Vertrag erwartet wird, kann nicht nur ein Objekt der Klasse Vertrag oder der direkten Unterklasse Mietvertrag verwendet werden, sondern auch ein Objekt der Klasse Leasingvertrag.

Die Vererbung erfolgt ebenfalls über alle Stufen: Jede Klasse verfügt nicht nur über die Attribute und Methoden der direkten Oberklasse sondern auch über diejenigen, die in der Oberklasse der Oberklasse definiert sind, usw. Methoden mit identischen Signaturen (Name, Zahl und Typen der Parameter, Rückgabetyp) direkter oder indirekter Oberklassen werden überschrieben. Beim Aufruf einer Methode wird diese zunächst in der Klasse gesucht, der das Objekt angehört. Wird sie dort nicht gefunden, wird in der nächsthöheren Klasse der Vererbungshierarchie gesucht. Die Vererbungshierarchie wird solange nach oben durchlaufen, bis die betreffende Methode gefunden ist.

8.7 Abstrakte Klassen

In dem bisher betrachteten Beispiel mit unterschiedlichen Arten von Verträgen stellt sich die Frage, ob es überhaupt sinnvoll ist, Objekte der Klasse Vertrag anzulegen. Möglicherweise soll in dem zu entwickelnden System ja jeder Vertrag ein Kaufvertrag oder ein Mietvertrag sein. Ein Objekt, das „nur" Vertrag ist, soll es dann gar nicht geben.

Die Klasse Vertrag dient nur dazu, Gemeinsamkeiten von Kaufverträgen und Mietverträgen aufzunehmen, so dass entsprechende Methoden nicht zweimal implementiert werden müssen. Auch kann man sich das Substitutionsprinzip zunutze machen. Ohne die gemeinsame Oberklasse Vertrag müsste eine Methode, die einen Vertrag als Parameter erwartet, ebenfalls zweimal implementiert werden: Einmal mit einem Parameter vom Typ „Kaufvertrag" und einmal mit einem Parameter vom Typ „Mietvertrag". Auch eine mögliche Erweiterung um weitere Arten von Verträgen wird einfacher: Man braucht den neuen Vertragstyp einfach zur Unterklasse von Vertrag zu machen, und schon stehen die im Vertrag definierten Methoden und Attribute auch für die neue Klasse zur Verfügung.

Möchte man verhindern, dass von einer Klasse Objekte angelegt werden, so kann man sie als abstrakte Klasse kennzeichnen. Es können dann nur noch Objekte der Unterklassen angelegt werden – es sei denn, die betreffende Unterklasse ist selbst abstrakt. Gewöhnliche Klassen, von denen man Objekte anlegen kann, werden im Gegensatz zu abstrakten Klassen als konkrete Klassen bezeichnet.

Abstrakte Klassen werden im Klassendiagramm mit kursiv geschriebenem Namen dargestellt. Da dies – insbesondere bei von Hand gezeichneten Modellen – oft nicht so gut erkennbar ist, kann man stattdessen oder zusätzlich «*abstract*» eintragen (Abbildung 99).

Abbildung 99: Verschiedene Darstellung der abstrakten Klasse „Vertrag"

Eine abstrakte Klasse kann neben gewöhnlichen, konkreten Methoden auch abstrakte Methoden enthalten. Eine abstrakte Methode hat keine Implementierung, d. h. der Methodenrumpf fällt weg. Somit kann auch keine Methodenimplementierung an die Unterklassen vererbt werden. Durch eine abstrakte Methode wird aber definiert, dass jedes Objekt des betreffenden Typs über diese Methode verfügt.

Da es von der abstrakten Klasse keine Objekte gibt, entsteht durch das Fehlen der Implementierung auch kein Problem. Es muss aber sichergestellt sein, dass spätestens in jeder konkreten Unterklasse eine Implementierung vorliegt. Da von konkreten Klassen Objekte gebildet werden können, muss definiert sein, welche Implementierung bei einem Aufruf der Methode

auszuführen ist. Eine abstrakte Methode stellt also eine Verpflichtung für die konkreten Unterklassen dar, die Methode zu implementieren. Wie beim Überschreiben muss die Unterklasse dafür eine Methode mit derselben Signatur enthalten. Da es in der Oberklasse keine Implementierung gibt, die man überschreiben könnte, spricht man davon, die Methode zu implementieren.

Abstrakte Methoden werden ebenfalls durch Kursivschreibung gekennzeichnet. So verfügt die – nunmehr abstrakte – Klasse Vertrag in Abbildung 100 über die abstrakte Methode „*berechneGesamtkosten( )*“. Zur deutlicheren Darstellung kann man ebenfalls «abstract» hinzufügen:

«abstract» berechneGesamtkosten( ): int

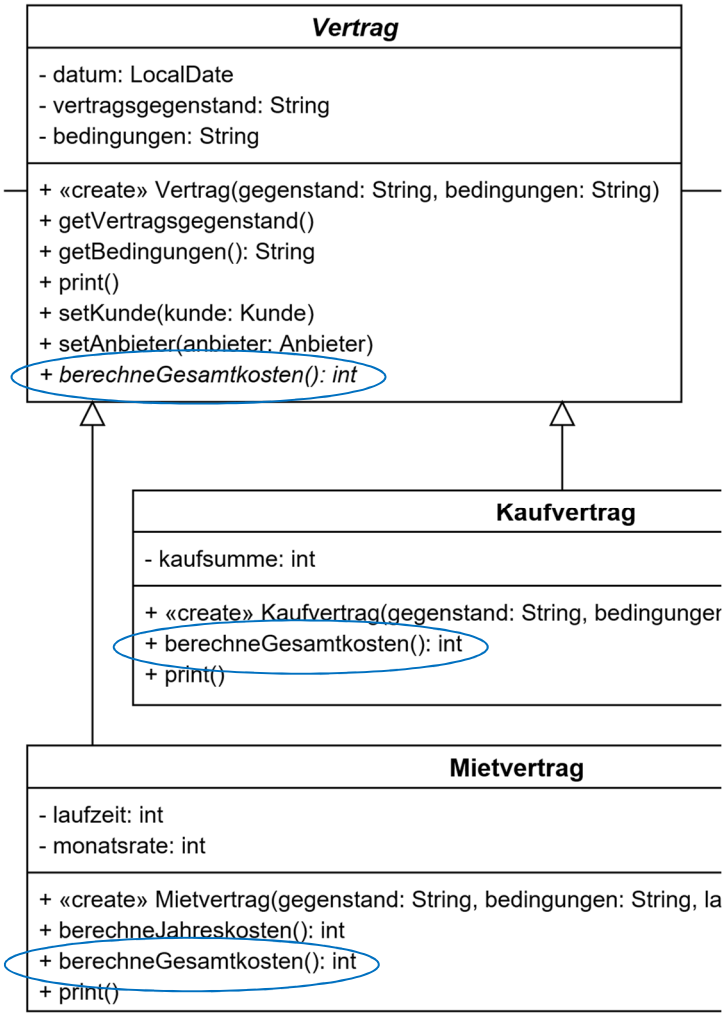


Abbildung 100: Abstrakte Methode. In den Unterklassen sind entsprechende konkrete Methoden erforderlich.

Die konkreten Unterklassen Kaufvertrag und Mietvertrag verfügen entsprechend über die notwendigen Implementierungen der Methode.

Abstrakte Methoden kann es nur in abstrakten Klassen geben. In einer konkreten Klasse träte das Problem auf, dass beim Aufruf der Methode keine Implementierung verfügbar wäre, die ausgeführt werden könnte.

Was ist der Vorteil einer abstrakten Methode? Verwendet man die abstrakte Klasse als Typ, so kann man die entsprechende Methode verwenden, da die konkreten Unterklassen – und damit alle tatsächlich möglichen Objekte des Typs – über Implementierungen verfügen.

In der Klasse Vertrag sind keine Informationen über die Kosten des Vertrags vorhanden. Dadurch ist keine sinnvolle Implementierung der Methode „*berechneGesamtkosten)*" möglich. Dennoch soll sichergestellt werden, dass diese Methode für jeden Vertrag zur Verfügung steht.

Listing 25 zeigt einen Ausschnitt aus der Java-Implementierung der Klasse „*Vertrag*". Sowohl die Klasse als auch die abstrakte Methode werden mit dem Schlüsselwort „*abstract*" gekennzeichnet. Da es für die abstrakte Methode keine Implementierung gibt, entfällt der sonst in geschweiften Klammern enthaltene Methodenrumpf. Die anderen, konkreten Methoden der Klasse Vertrag enthalten wie gewohnt einen Rumpf. Da sie über eine Implementierung verfügen, müssen sie in den Unterklassen nicht überschrieben werden.

```
public abstract class Vertrag {
	// ...
	public abstract int berechneGesamtkosten();
}
```

Listing 25: Abstrakte Klasse mit abstrakter Methode in Java

Listing 26 enthält einen Ausschnitt aus der Klasse Kaufvertrag mit der Implementierung der Methode „*berechneGesamtkosten( )*". Die Implementierung ist unverändert gegenüber der aus Listing 18. Allerdings gab es dort keine Verpflichtung, eine Methode „*berechneGesamtkosten( )*" zur Verfügung zu stellen. Jetzt hingegen erbt die Klasse Kaufvertrag diese Verpflichtung von der abstrakten Methode in der Oberklasse.

```
public class Kaufvertrag extends Vertrag {
	private int kaufsumme;
	// ...
	public int berechneGesamtkosten(){
		return this.kaufsumme;
	}
}
```

Listing 26: Implementierung der abstrakten Methode in der Unterklasse

Wird die Klasse Vertrag nun wieder als Typ verwendet, so können alle Methoden dieser Klasse verwendet werden, sowohl die konkreten Methoden als auch die abstrakte Methode „*berechneGesamtkosten( )*" (Listing 27).

```
public void testeVertragsmethoden(Vertrag vertrag){
	String vertragsGegenstand	= vertrag.getVertragsgegenstand();
	String vertragsBedingungen	= vertrag.getBedingungen();
	int gesamtkosten		= vertrag.berechneGesamtkosten();
	vertrag.print();
}
```

Listing 27: Verwendung der abstrakten Klasse Vertrag als Typ

Da sie abstrakt ist, kann man kein Objekt der Klasse Vertrag selbst mehr anlegen und der Methode „*testeVertragsmethoden*“ übergeben, wie dies in Listing 21 gemacht wurde, wo die Klasse Vertrag konkret war.

In Listing 28 wird „*testeVertragsmethoden( )*“ mit Objekten der beiden Unterklassen aufgerufen. Wenn bei der Ausführung die Methode „*berechneGesamtkosten( )*“ aufgerufen wird, wird jeweils die Implementierung aus der betreffenden Unterklasse ausgeführt. Die Nutzung der Methode „*berechneGesamtkosten( )*“ in „*testeVertragsmethode( )*“ ist nur möglich, weil sie bereits in der Klasse Vertrag als abstrakte Methode definiert wurde.

```
Kaufvertrag kv = new Kaufvertrag("Notebook Computer", "Erweiterte Garantie",
                                                                    1200);
Mietvertrag mv = new Mietvertrag("Computer", "Gemäß AGB", 36, 100);

testeVertragsmethoden(kv);
testeVertragsmethoden(mv);
```

Listing 28: Aufruf der Methode „testeVertragsmethoden“ mit Objekten der Unterklassen von Vertrag

8.8 Mehrfachvererbung

In den bisherigen Beispielen hatte jede Klasse maximal eine direkte Oberklasse. Prinzipiell kann man auch modellieren, dass eine Klasse mehrere direkte Oberklassen hat. So ist in Abbildung 101 eine Klasse „*Finanzkauf*“ enthalten, bei der es sich sowohl um einen Kaufvertrag als auch um eine Kreditfinanzierung handelt. Entsprechend wurde sie sowohl als Unterklasse der Klasse Kaufvertrag als auch als Unterklasse der Klasse Kreditfinanzierung modelliert. Auch die Klasse Leasingvertrag hat zwei Oberklassen: Mietvertrag und Kreditfinanzierung.

Eine solche Mehrfachvererbung kann man zwar in UML modellieren, sie ist aber z. B. in Java nicht erlaubt. Bei den meisten Attributen und Methoden ist die Mehrfachvererbung unproblematisch. So verfügt die Klasse Finanzkauf eben über die Attribute beider Oberklassen. Auch für die Methoden, die in nur einer der Oberklassen vorkommen, ist das kein Problem. Was aber ist mit der Methode „*print( )*“? Finanzkauf erbt „*print( )*“ sowohl von Kaufvertrag als auch von Kreditfinanzierung. Welcher der beiden Implementierungen soll ausgeführt werden, wenn diese Methode bei einem Finanzkauf-Objekt aufgerufen wird? Derartige Probleme werden vermieden, wenn man auf Mehrfachvererbung verzichtet. In Java muss zwangsläufig darauf verzichtet werden.

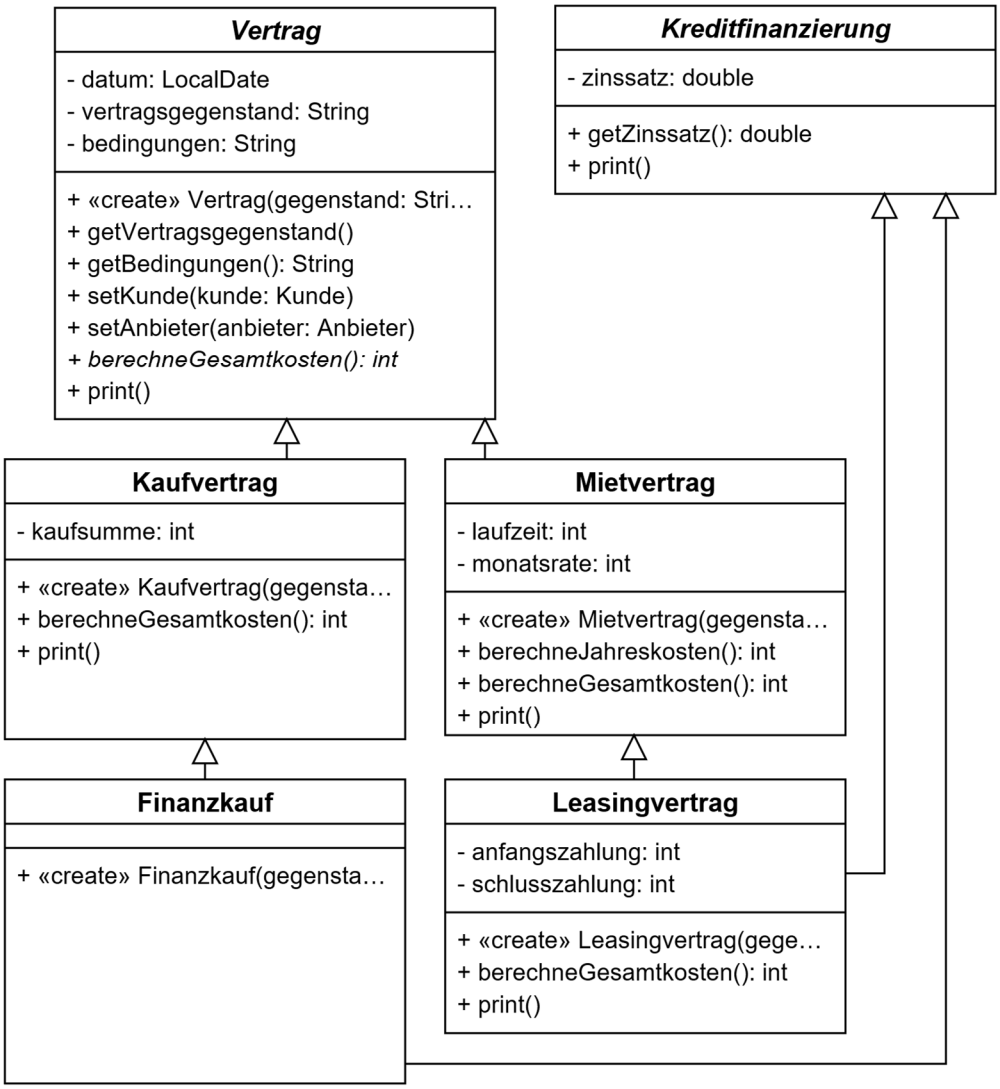


Abbildung 101: Mehrfachvererbung

8.9 Schnittstellen

Das Modell in Abbildung 102 ähnelt stark dem aus Abbildung 101. Allerdings ist Kreditfinanzierung nun ein Interface (Schnittstelle). Ein Interface entspricht einer abstrakten Klasse, die nur abstrakte Methoden enthält. Die implementierenden Klassen erben also nur die Verpflichtung, alle Methoden zu implementieren. Man spricht hier auch nicht von Unterklassen, sondern davon, dass eine Klasse ein Interface implementiert.

Im Gegensatz zur Vererbung kann eine Klasse auch in Java beliebig viele Interfaces implementieren. Dies ist unproblematischer, da keine Implementierungen vererbt werden. Erhält eine Klasse zweimal die Verpflichtung eine bestimmte Methode mit einer bestimmten Signatur zu implementieren, so genügt eine Implementierung dieser Methode um die Verpflichtungen beider Interfaces zu erfüllen.

Auch ein Interface kann als Typ verwendet werden. Dort, wo der Interface-Typ erwartet wird, können dann Objekte der implementierenden Klassen genutzt werden.

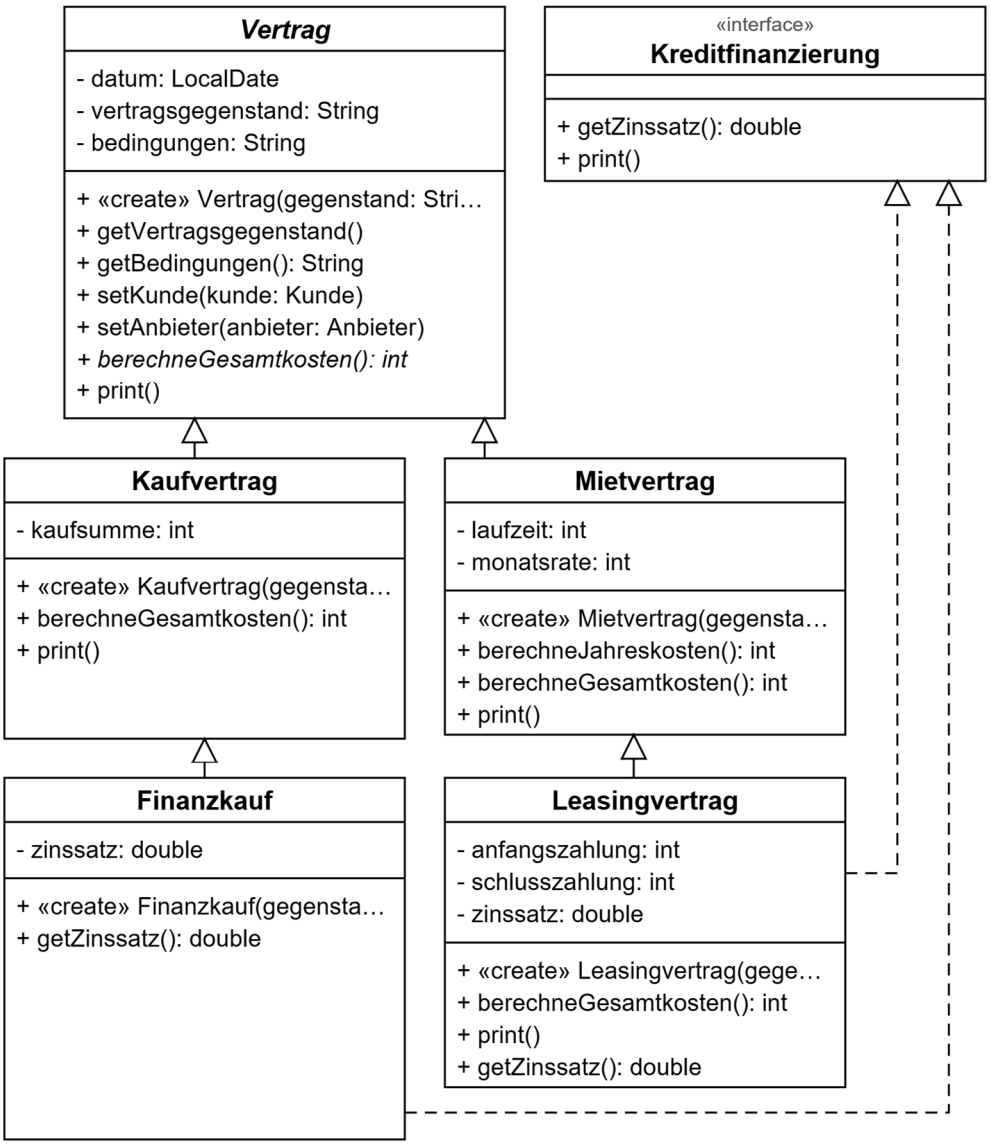


Abbildung 102: Interface „Kreditfinanzierung“

Ein Interface wird mit einem Klassensymbol modelliert und mit dem Stereotyp «*interface*» gekennzeichnet. Die Methoden müssen nicht speziell als abstrakt ausgezeichnet werden, denn ein Interface kann ausschließlich abstrakte Methoden enthalten. Die Implementierungsbeziehung zwischen einer Klasse und einem Interface sieht ähnlich wie die Vererbungsbeziehung aus, jedoch ist die Linie gestrichelt.

Listing 29 zeigt die Definition eines Interface in Java. Hier sind alle Methoden ohne Rumpf. Die Klasse Finanzkauf, die das Interface Kreditfinanzierung implementiert, ist in Listing 30 zu sehen. Die Implementierung wird mit Hilfe des Schlüsselworts „*implements*“ angegeben. Falls mehrere Interfaces implementiert werden, werden diese durch Kommas getrennt aufgeführt. Die Klasse Finanzkauf muss alle Methoden des Interface implementieren. Für die Methode „*getZinssatz( )*“ ist dies dargestellt. Für die Methode „*print( )*“ verfügt Finanzkauf schon über eine Implementierung, nämlich die von der Klasse Kaufvertrag geerbte.

```
public interface Kreditfinanzierung {
      public double getZinssatz();
      public void print();
}
```

Listing 29: Interface in Java

```
public class Finanzkauf extends Kaufvertrag
                        implements Kreditfinanzierung {
      // ...

      public double getZinssatz(){
            return zinssatz;
      }
}
```

Listing 30: Implementierung eines Interface in Java

9 Sequenzdiagramme

Sequenzdiagramme beschreiben das dynamische Verhalten eines objektorientierten Systems, das durch das Zusammenspiel verschiedener Objekte entsteht. Wird eine Methode eines Objekts aufgerufen, so kann diese Methode selbst wieder Methoden anderer Objekte aufrufen. Insgesamt kann durch gegenseitige Methodenaufrufe ein komplexer, nicht mehr ganz einfach zu überblickender Ablauf entstehen.

9.1 Einführendes Beispiel

Zunächst wird das Klassendiagramm in Abbildung 103 betrachtet. Zu einer Reise gehören hier immer eine Hotelbuchung und ein Mietwagen. Der Mietwagen ist von einem bestimmten Fahrzeugtyp. Möchte man den Preis einer Reise berechnen, so muss man den Übernachtungspreis im Hotel und den Mietpreis des Wagens ermitteln.

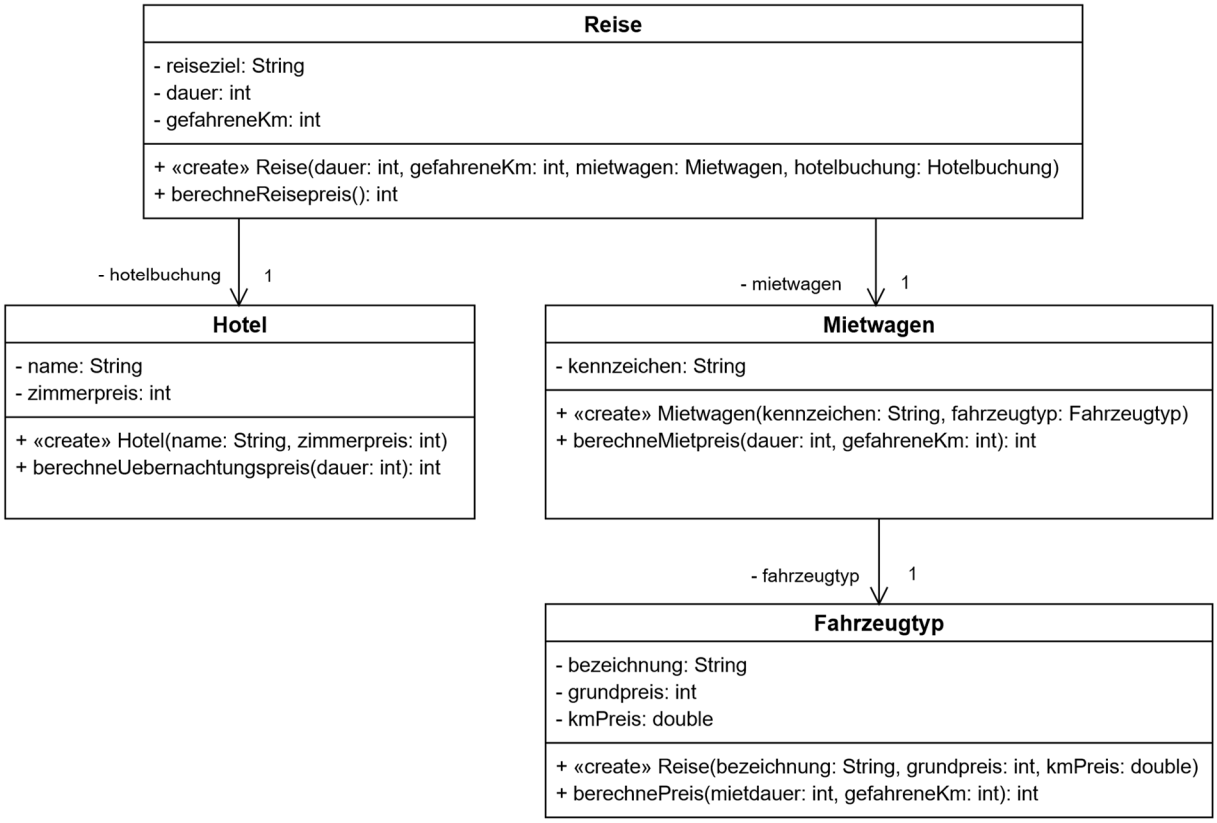


Abbildung 103: Beispiel: Reise

Abbildung 104 zeigt das Objektdiagramm einer Reise mit ihren zugeordneten Objekten.

Es soll nun untersucht werden, welche Methoden welcher Objekte nacheinander aufgerufen werden, wenn die Methode „*berechneReisepreis( )*“ für ein Objekt der Klasse Reise ausgeführt wird. Hierzu wird der Code dieser Methode aus Listing 31 betrachtet. Zunächst wird der Mietwagenpreis berechnet. Hierzu wird die Methode „*berechneMietpreis( )*“ des zugeordneten, im Attribut „*mietwagen*“ gespeicherten Mietwagen-Objekts mit den Parametern „*dauer*“ und „*gefahreneKm*“ aufgerufen.

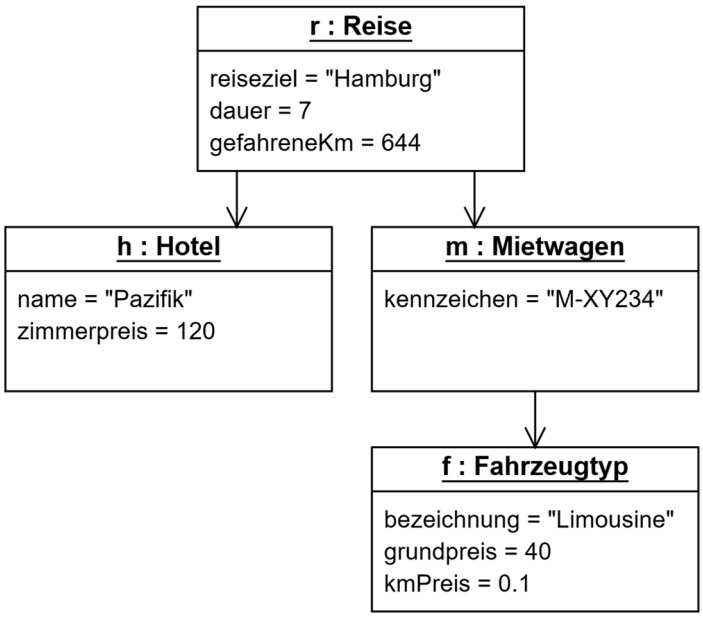


Abbildung 104: Objektdiagramm Reise

```
public class Reise {
   // ...
   public int berechneReisepreis(){
      int mietwagenpreis = mietwagen.berechneMietpreis(dauer, gefahreneKm);
      int hotelpreis = hotelbuchung.berechneUebernachtungspreis(dauer);
      return mietwagenpreis + hotelpreis;
   }
}
```

Listing 31: Methode „berechneReisepreis“ der Klasse Reise

Um nachzuvollziehen, was innerhalb der Methode *„berechneMietpreis( )“* passiert, muss man die Implementierung dieser Methode in der Klasse Mietwagen ansehen (Listing 32). Diese Methode schickt die Nachricht *„berechnePreis(dauer, km)“* an „fahrzeugtyp“ und gibt das erhaltene Ergebnis zurück. Es wird also die Methode *„berechnePreis( )“* der Klasse Fahrzeugtyp aufgerufen.

```
public class Mietwagen {
   // ...
   public int berechneMietpreis(int dauer, int km){
      return fahrzeugtyp.berechnePreis(dauer, km);
   }
}
```

Listing 32: Methode „berechneMietpreis“ der Klasse Mietwagen

Ein Blick in diese Methode zeigt, dass hier aus den übergebenen Parametern und den Attributwerten der Preis berechnet wird (Listing 33). Dabei finden aber keine weiteren Methodenaufrufe statt.

```
public class Fahrzeugtyp {
      // ...
   public int berechnePreis(int mietdauer, int km){
      return mietdauer * grundpreis + (int)(km * kmpreis);
   }
}
```

Listing 33: Methode „berechnePreis( )“ der Klasse Fahrzeugtyp

Somit kann man wieder zur ursprünglichen Methode „*berechneReisepreis( )*“ in Listing 31 zurückgehen. In der zweiten Zeile dieser Methode wird die Nachricht „*berechneUebernachtungspreis(dauer)*“ an das im Attribut „*hotelbuchung*“ hinterlegte Hotel-Objekt geschickt. Diese Methode führt auch nur eine einfache Berechnung aus und gibt den Preis zurück (Listing 34). Zuletzt addiert die Methode „*berechneReisepreis( )*“ den Mietwagenpreis und den Hotelpreis und gibt das Ergebnis zurück. Damit ist der Ablauf beendet.

```
public class Hotel {
   // ...
   public int berechneUebernachtungspreis(int dauer){
      return zimmerpreis * dauer;
   }
}
```

Listing 34: Methode berechneUebernachtungspreis( ) der Klasse Hotel

Das Sequenzdiagramm in Abbildung 105 stellt das beschriebene Zusammenspiel der verschiedenen Methoden grafisch dar. Die zeitliche Reihenfolge verläuft von oben nach unten. Die Objekte sind alle am oberen Rand des Diagramms eingetragen. Von dort läuft jeweils eine gestrichelte Linie nach unten, die sogenannte Lebenslinie. Dort, wo diese Linie verläuft, „lebt“ das betreffende Objekt, d. h. es existiert. Da in dem betrachteten Beispiel keine Objekte neu angelegt oder gelöscht werden, verlaufen alle Lebenslinien durch das gesamte Diagramm.

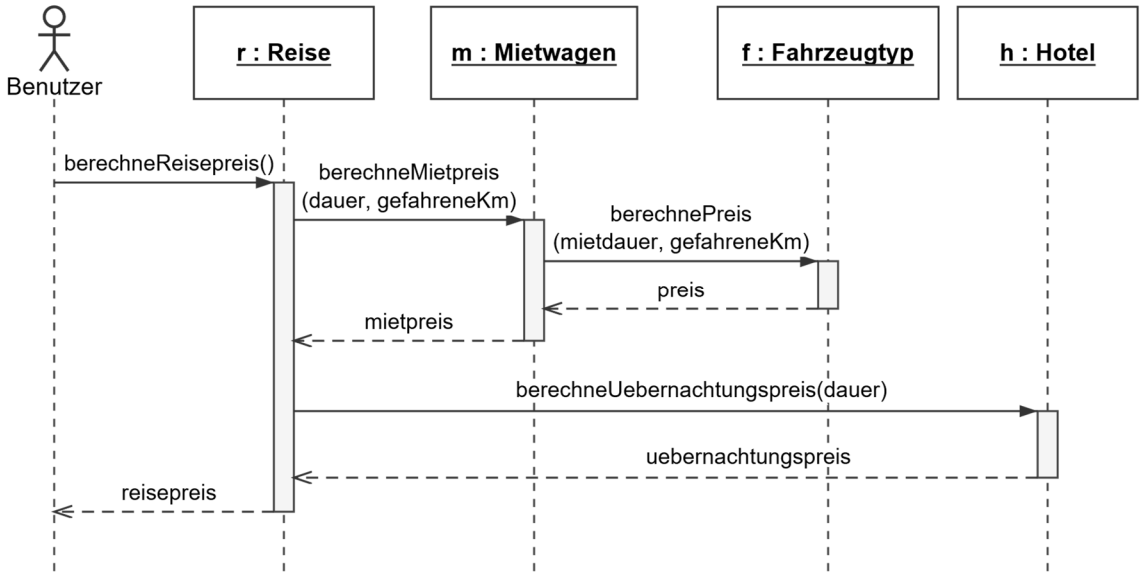


Abbildung 105: Sequenzdiagramm für die Methode „berechneReisepreis“

Der Ablauf beginnt mit dem Aufruf „*berechneReisepreis( )*“ beim Reise-Objekt. Wodurch der Aufruf erfolgt, wird hier nicht näher betrachtet. Daher wurde ein allgemeiner Akteur „Benutzer“ als ursprünglicher Initiator eingezeichnet.

Der nach unten verlaufende Aktivierungsbalken auf der Lebenslinie des Reise-Objekts zeigt die Ausführung der Methode „*berechneReisepreis( )*“ an. Am Ende der Ausführung, ganz

unten im Diagramm, erfolgt die Rückgabe an den Aufrufer, dargestellt durch einen gestrichelten Pfeil nach links und die Angabe des zurückgegebenen Werts, in diesem Fall der Reisepreis.

Wie oben anhand des Quellcodes beschrieben, erfolgen beim Ausführen der Methode „*berechneReisepreis( )*" noch weitere Aufrufe: Zuerst wird die Methode „*berechneMietpreis(dauer, gefahreneKm)*" des Mietwagen-Objekts aufgerufen, anschließend die Methode „*berechneUebernachtungspreis(dauer)*" des Hotel-Objekts.

Die beiden Objekte führen jeweils die aufgerufenen Methoden aus und geben anschließend die Ergebnisse wieder an die noch nicht abgeschlossene Methode „*berechneReisepreis( )*" des Reise-Objekts zurück. Die Methode „*berechneMietpreis( )*" des Mietwagen-Objekts ruft selbst noch die Methode „*berechnePreis(dauer, km)*" des Fahrzeugtyp-Objekts auf.

In anderen UML-Veröffentlichungen und vielen UML-Tools sind die Objektnamen in Sequenzdiagrammen oft nicht unterstrichen. Es spricht jedoch nichts gegen eine Unterstreichung und entspricht der aus den Objektdiagrammen gewohnten Darstellung. Zudem lassen sich Objekte auf diese Weise besser von anderen Arten von Kommunikationspartnern unterscheiden, wie z. B. Klassen (vgl. Abbildung 110) oder ganze Sub-Systeme. Man muss nur wissen, dass die Unterstreichung von Objektbezeichnungen optional ist.

9.2 Bedingte und mehrfache Methodenaufrufe

In dem Klassendiagramm in Abbildung 106 umfasst eine Klausur mehrere Aufgaben. Die Gesamtpunktzahl der Klausur ergibt sich, indem man die Punkte jeder enthaltenen Aufgabe zusammenzählt. Damit man diese Berechnung nicht bei jeder Abfrage der Gesamtpunktzahl neu durchführen muss, wird sie – wenn sie einmal berechnet wurde – im Attribut „*gesamtpunktzahl*" gespeichert. Das Attribut „*gesamtpunktzahl*" könnte man daher auch als abgeleitetes Attribut kennzeichnen (vgl. Kapitel 7.1).

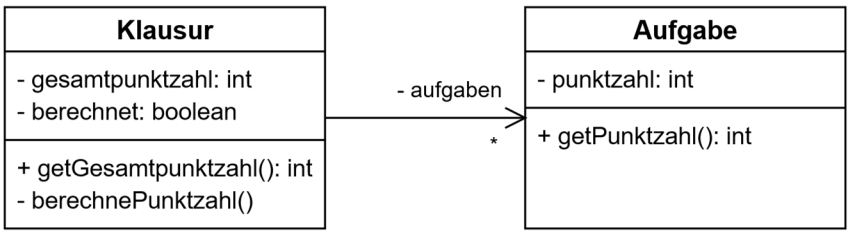


Abbildung 106: Beispiel Klausur

Listing 35 enthält den Java-Code der Klasse „*Klausur*". Modelliert werden soll der Aufruf der Methode „*getGesamtpunktzahl( )*". Hier gibt es mehrere Besonderheiten gegenüber dem vorangehenden Beispiel. Zum einen wird die Methode „*berechnePunktzahl( )*" nicht immer aufgerufen, sondern nur in manchen Fällen, wenn „*berechnet*" den Wert *false* hat. Zum zweiten handelt es sich bei der Methode „*berechnePunktzahl( )*" um eine Methode derselben Klasse. Daher muss der Aufruf-Pfeil nicht zu einem anderen Objekt führen, sondern vom Klausur-Objekt zurück zu sich selbst.

```java
import java.util.List;
import java.util.ArrayList;

public class Klausur {

	private boolean berechnet=false;
	private int gesamtpunktzahl=0;
	private List<Aufgabe> aufgaben = new ArrayList<>();

	public int getGesamtpunktzahl(){
		if(!berechnet){
			berechnePunktzahl();
		}
		return gesamtpunktzahl;
	}

	private void berechnePunktzahl(){
		int punkte;
		for(Aufgabe a:aufgaben){
			punkte = a.getPunktzahl();
			gesamtpunktzahl = gesamtpunktzahl + punkte;
		}
		berechnet = true;
	}
}
```

Listing 35: Implementierung der Klasse „Klausur“

Innerhalb der Methode *„berechnePunktzahl( )“* wird schließlich die Methode *„getPunktzahl( )“* des Aufgabe-Objekts aufgerufen. Dieser Methodenaufruf befindet sich in einer Schleife, d. h. er findet mehrfach statt. Die Nachricht geht an jede der Klausur zugeordnete Aufgabe.

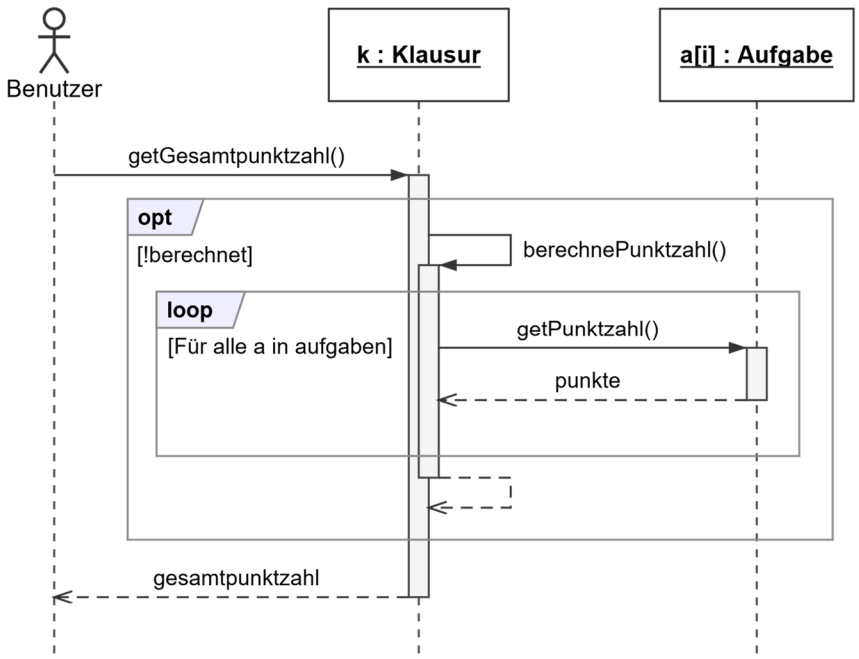


Abbildung 107: Sequenzdiagrammm für die Methode „getGesamtpunktzahl“

Der bedingte Methodenaufruf und die Schleife werden mit Hilfe sogenannter kombinierter Fragmente modelliert (Abbildung 107). Diese werden als Rechtecke dargestellt. Links oben ist jeweils der Typ des Fragments eingetragen, in diesem Fall *„opt“* für „optional“ und *„loop“* für

die Schleife. Darunter ist jeweils in eckigen Klammern eine Bedingung angegeben. Beim „*opt*"-Fragment ist es die Bedingung, die erfüllt sein muss, damit der Nachrichtenaustausch innerhalb des Fragments durchgeführt wird. Bei der Schleife ist es die Wiederholungsbedingung, in diesem Fall informell ausgedrückt.

Wie man sieht, können die Fragmente ineinander geschachtelt sein. Aus Listing 35 geht ja hervor, dass die Schleife, in der die Nachricht „*getPunktzahl( )*" verschickt wird, überhaupt nur erreicht wird, wenn die Bedingung „*!berechnet*" zutrifft.

Auch ein Aufrufpfeil eines Objekts zu sich selbst ist in Abbildung 107 enthalten. Da der Aufruf „*berechnePunktzahl( )*" eine andere Methode derselben Klasse betrifft, endet der Pfeil nicht an einem anderen Objekt, sondern an der Lebenslinie des Ausgangsobjekts selbst.

Dort ist nun ein zweiter Aktivierungsbalken eingezeichnet. Schließlich sind im Klausur-Objekt zwei Methoden aktiv. Zum einen die zu Beginn aufgerufene Methode „*getGesamtpunkt-zahl( )*". Diese ist noch nicht beendet, sie wartet vielmehr darauf, dass sie die Kontrolle von der Methode „*berechnePunktzahl( )*" zurückbekommt. Zum zweiten ist auch diese Methode „*berechnePunktzahl( )*" aktiv.

Normalerweise endet ein Nachrichtenpfeil bei der Lebenslinie eines anderen Objektes. Entsprechend befindet sich der Aktivierungsbalken auf der Lebenslinie dieses anderen Objektes. Bei einer Nachricht an sich selbst sind Nachrichten- und Rückgabepfeil mitsamt der Lebenslinie quasi auf die Lebenslinie des Ursprungsobjekts „zurückgeklappt".

An den Rückgabepfeilen sind jeweils die zurückgegebenen Werte, wie z. B. „*punkte*" oder „*gesamtpunktzahl*". Die Methode „*berechnePunktzahl*" hat den Rückgabetyp „*void*" und gibt somit nichts zurück. Durch den gestrichelten Rückgabepfeil wird dennoch deutlich, wo die Kontrolle wieder zurückgegeben wird.

Bei dem rechts dargestellten Aufgaben-Objekt handelt es sich nicht um ein einzelnes Objekt, sondern um ein „Multiobjekt", das eine ganze Reihe von Objekten repräsentiert, bei denen nacheinander die betreffende Methode aufgerufen wird. Daher wurde die Objektbezeichnung „*a*" mit einer Indexangabe „*[i]*" versehen. Dies wird jedoch von der UML nicht vorgeschrieben. Es ist lediglich eine häufig anzutreffende Darstellungsart.

Neben den bereits verwendeten kombinierten Fragmenten „*opt*" und „*loop*" gibt es noch weiter Typen von Fragmenten. Nützlich sind insbesondere die in Abbildung 108 dargestellten. „*alt*" steht für „Alternativen". Im Gegensatz zum „*opt*"-Fragment, wo es nur die Möglichkeit gibt, dass der Inhalt durchgeführt wird oder nicht, stellt „*alt*" beliebig viele Alternativen zur Verfügung, wie dies bei Verwendung von mehreren „*if-else*"- oder bei „switch-*case*"-Anweisungen erforderlich ist. Die verschiedenen Alternativen werden durch gestrichelte Linien unterteilt. Jedes Segment enthält dann eine Bedingung. Das „*break*"-Fragment kann man verwenden, wenn eine Schleife durch eine break-Anweisung während der Durchführung abgebrochen wird.

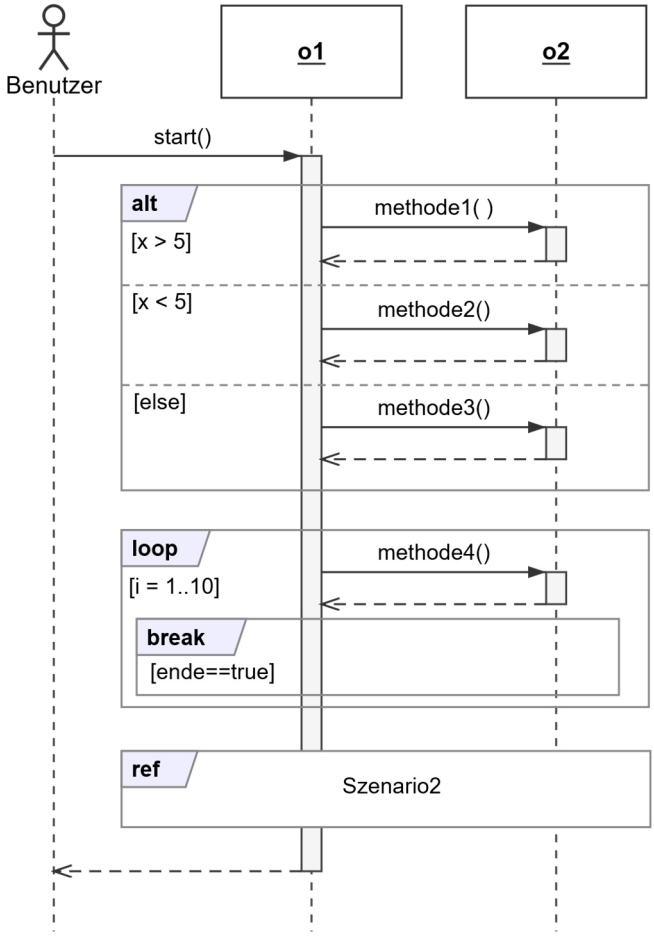


Abbildung 108: Weitere kombinierte Fragmente und Referenz

Bei dem mit „*ref*“ bezeichneten Block handelt es sich nicht um ein gewöhnliches Fragment, sondern um eine Referenz auf ein anderes Diagramm, hier mit der Bezeichnung „*Szenario2*“. Wird dieser Referenz-Block erreicht, wird die in dem anderen Diagramm dargestellte Interaktion aufgerufen. Wenn diese abgeschlossen ist, geht es im ursprünglichen Diagramm weiter. Eine solche Referenz kann man verwenden, wenn das Diagramm zu groß wird. In dem Rechteck wird der Name des aufgerufenen Diagramms eingetragen. Bei dem aufgerufenen Diagramm muss es sich nicht unbedingt um ein Sequenzdiagramm handeln, man kann z. B. auch auf ein Ablaufdiagramm verweisen.

9.3 Darstellung weiterer Aspekte in Sequenzdiagrammen

Abbildung 109 zeigt einen Ausschnitt aus einer Bibliotheksanwendung. Die Klasse „Anwendung“ verfügt über eine Methode „*findeStandortVonTitel( )*“, die dazu dient, in der Bibliothek den Standort eines Buchs mit einem bestimmten Titel zu finden. Dazu muss zunächst das Buch mit dem gesuchten Titel gefunden werden. Hierzu dient die Methode „*findeBuch( )*“. Dabei handelt es sich um eine Klassenmethode, auch als statische Methode bezeichnet. Sie ist in der Klassendarstellung unterstrichen (vgl. Kapitel 7.1). Die Methode wird somit nicht für ein einzelnes Objekt ausgeführt, sondern für die Klasse „*Buch*“. Schließlich hat man ja noch kein

Buch-Objekt, sondern möchte dies von der Suchmethode ja erst geliefert bekommen. Entsprechend wird die Nachricht nicht an ein bestimmtes Buch-Objekt geschickt, sondern an die Klasse „*Buch*“. Wenn es gefunden wurde, wird für dieses Buch-Objekt die Methode „*getStandort( )*“ aufgerufen.

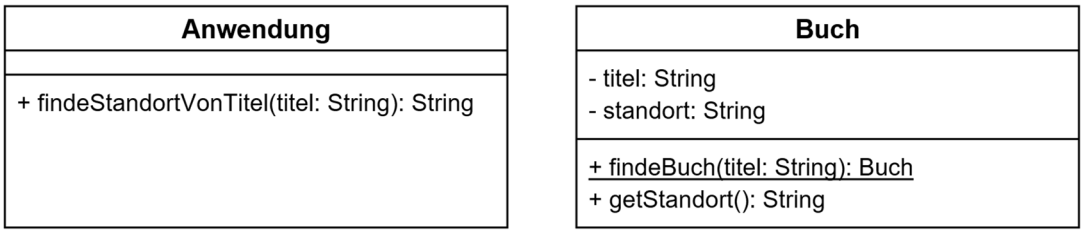


Abbildung 109: Beispiel Bibliotheksanwendung

Abbildung 110 zeigt das Sequenzdiagramm. Für den Parameter „*titel*“ ist hier der Beispielwert „*Harry Potter*“ eingetragen. Da es sich um eine Klassenmethode handelt, ist als Ziel des Aufrufs „*findeBuch( )*“ kein Objekt eingetragen, sondern die Klasse „Buch“ selbst.

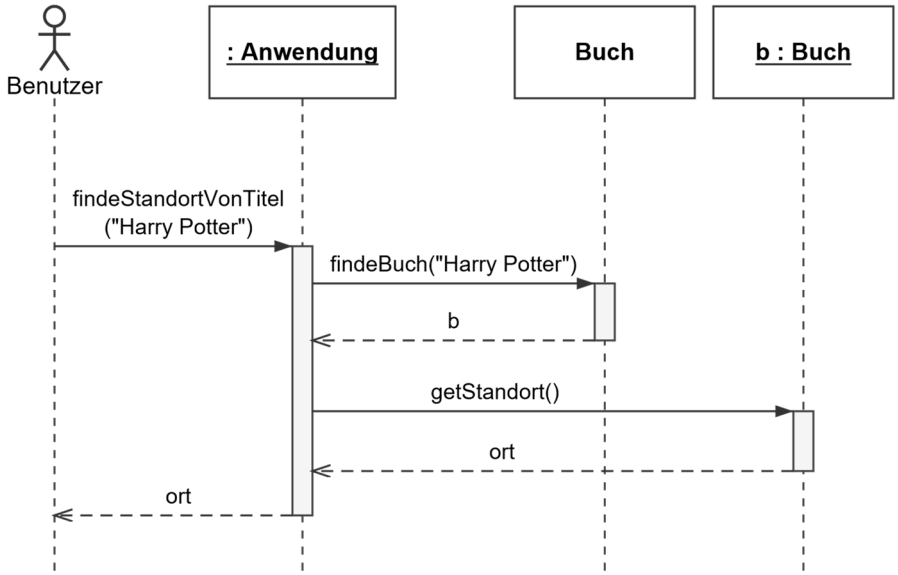


Abbildung 110: Aufruf von Klassenmethoden

Anmerkung: Leider ermöglichen es viele UML-Modellierungstools nicht, Klassen in Sequenzdiagrammen zu verwenden. Dann muss man sich mit einer geeigneten Namenskonvention behelfen, z. B. indem man die Klassenbezeichnung als Name verwendet und als Typ „Class“ (also „Buch : Class“) einträgt oder keinen Typ (also z. B. „Buch :“).

Bei dem Aufruf ‚*findeBuch(„Harry Potter“)*‘ wird für die Rückgabe eine Variable „*b*“ benutzt, die das zurückgegebene Buch-Objekt aufnimmt. „*b*“ wird auch als Bezeichnung für das rechts im Diagramm eingezeichnete Buch-Objekt verwendet. Damit wird ausgedrückt, dass es sich bei dem gezeigten Buch-Objekt um das Objekt handelt, das von „*findeBuch( )*“ zurückgegeben wurde.

Die Methode „*findeStandortVon( )*“ lässt sich also zuerst von „*findeBuch( )*“ das betreffende Buch-Objekt zurückgeben und ruft bei diesem zurückgegebenen Buch-Objekt anschließend die Methode „*getStandort( )*“ auf.

In dem Beispiel wurde vereinfachend angenommen, dass die Methode „*findeBuch( )*“ tatsächlich immer genau ein Buch zurückgibt. In der Praxis findet man mit einer Buchsuche meist mehrere Bücher oder auch einmal kein Buch. Man würde daher eher eine Liste von Büchern zurückgeben (Rückgabetyp z. B. *List<Buch>*) und jedem der enthaltenen Bücher die Nachricht „*getStandort( )*“ schicken.

In den bisherigen Beispielen kommunizierten immer bereits bestehende Objekte. Entsprechend verliefen die Lebenslinien der Objekte immer vom oberen Zeichnungsrand bis ganz nach unten. In Abbildung 111 wird gezeigt, wie man das Anlegen und Löschen von Objekten darstellt.

Das Objekt der Klasse1 existiert schon zu Beginn des Szenarios. Das Objektsymbol ist daher am oberen Rand eingezeichnet, und die Lebenslinie verläuft bis zum unteren Rand des Modells, d. h. das Objekt existiert am Ende des betrachteten Szenarios nach wie vor. Bei diesem Objekt wird nun die Nachricht „*methode1( )*“ aufgerufen.

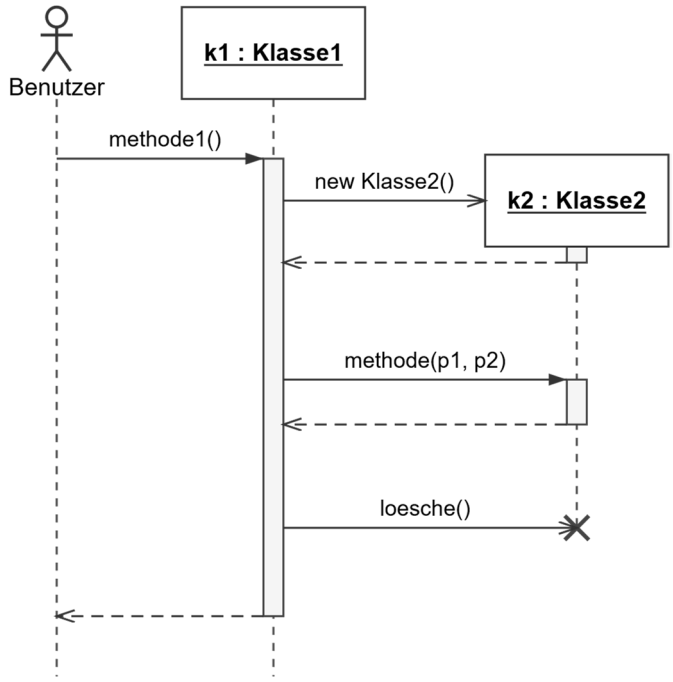


Abbildung 111: Anlegen und Löschen von Objekten im Sequenzdiagramm

Innerhalb von „*methode1( )*“ erfolgt als erstes der der Aufruf des Konstruktors der Klasse2. Das Objekt der Klasse2 existiert zu Beginn noch nicht. Daher wird das Objektsymbol nicht oben am Rand der Zeichnung eingetragen, sondern erst weiter unten, an der Stelle der zeitlichen Reihenfolge, an der das Objekt angelegt wird. Der Pfeil des Konstruktor-Aufrufs endet auch nicht an der Lebenslinie, sondern direkt auf dem Objekt-Symbol. Da der Konstruktor noch verschiedene Aufgaben erledigen – und ggf. auch weitere Nachrichten versenden – kann, ist das neu angelegte Objekt zunächst aktiviert, weshalb am unteren Rand des Objektsymbols direkt ein Aktivierungsbalken beginnt.

Als nächstes erfolgt der Aufruf der „*methode2(p1, p2)*“ beim neu angelegte Objekt. Dies ist eine ganz gewöhnliche Methode. Schließlich wird noch die Methode „*loesche( )*“ des Klasse2

ausgeführt. Methoden zum Löschen von Objekten werden auch als „*Destruktoren*“ bezeichnet. Da das Objekt an dieser Stelle gelöscht wird, endet seine Lebenslinie. Das Löschen wird zudem durch ein großes „X“ markiert. Im Gegensatz zum Anlegen eines Objekts wird das Löschen weniger häufig dargestellt. So werden etwa in Java Objekte nicht explizit gelöscht. Stattdessen werden nicht mehr referenzierte Objekte irgendwann im Hintergrund durch den Garbage Collector entfernt.

9.4 Einsatz von Sequenzdiagrammen

Sequenzdiagramme werden eingesetzt, wenn es darum geht, kompliziertere Nachrichtenaustausch-Sequenzen zu entwickeln, zu verstehen und für andere zu dokumentieren. Use-Case- und Klassendiagramme sollten das Gesamtsystem möglichst vollständig abbilden. Sequenzdiagramme hingegen werden nur dort eingesetzt, wo es einen konkreten Nutzen bringt, weil z. B. ein bestimmter Sachverhalt schwer zu überblicken ist.

Man sollte zudem darauf achten, keine zu komplexen Diagramme zu erstellen. Wenn ein Diagramm zu viele Bedingungen und Alternativen enthält, wird es schwer verständlich. Hier ist es oftmals sinnvoller, nur einen bestimmten Fall in einem Diagramm darzustellen und ggf. für andere Fälle separate Diagramme zu zeichnen. Auch müssen nicht zwangsläufig alle Methodenaufrufe eingezeichnet werden. Man kann sich auch auf die für das Verständnis relevanten Aufrufe beschränken.

Häufig werden Sequenzdiagramme auch von Hand gezeichnet, wenn es nur darum geht, sich das Zusammenspiel von Objekten zu verdeutlichen.

Die UML enthält noch einen weiteren, hier nicht verwendeten, Diagrammtyp zur Dokumentation des Zusammenspiels verschiedener Methodenaufrufe, das Kommunikationsdiagramm. Dabei handelt es sich grob gesagt um ein Objektdiagramm, an dessen Assoziationen die zwischen den Objekten erfolgenden Methodenaufrufe eingetragen sind. Dieser Diagrammtyp eignet sich, wenn man die Interaktionen der Objekte im Zusammenhang mit ihren Verbindungen betrachten möchte.

In Sequenzdiagrammen wird die Verschachtelung der Methodenaufrufe deutlicher sichtbar, und es lassen sich Aspekte wie das Anlegen von Objekten, Schleifen und Alternativen besser darstellen – dafür sieht man nicht direkt, über welche Assoziationen die Aufrufe erfolgen.

10 Klassen- und Sequenzdiagramme in der Analyse

In der Analyse geht es darum, Anforderungen zu analysieren, die fachlichen Rahmenbedingungen zu untersuchen und ein fachliches Konzept zu entwickeln, das noch unabhängig von Implementierungsdetails ist. Entsprechend enthalten Analysediagramme weniger Details. Analysemuster stellen Lösungsmöglichkeiten für häufig auftretende Fragestellungen bereit, wie z. B. die Unterscheidung von Typen und Exemplaren, die Nachvollziehbarkeit von Veränderungen und die Zusammensetzung von Objekten aus anderen Objekten.

10.1 Entwicklung von Analysediagrammen

Im Rahmen der Anforderungsanalyse werden Klassendiagramme verwendet, um die fachlichen Zusammenhänge zu dokumentieren, wie sie in dem System abgebildet werden sollen. Analysediagramme enthalten daher nur fachliche Klassen, die den Anwendungsbereich der Software abbilden, keine softwaretechnischen Klassen. Fachliche Klassen entstammen dem Anwendungsbereich. Beispiele für fachliche Klassen sind „Kunde", „Konto", „Produkt", „Bestellung", …

Keine Rolle in der Analyse spielen hingegen softwaretechnische Klassen, wie z. B. Klassen zur Datenbankanbindung, zur Anzeige von Benutzerdialogen, zur Verwaltung von Objekten oder zur Kommunikation über ein Netzwerk. Um diese kümmert man sich im Entwurf.

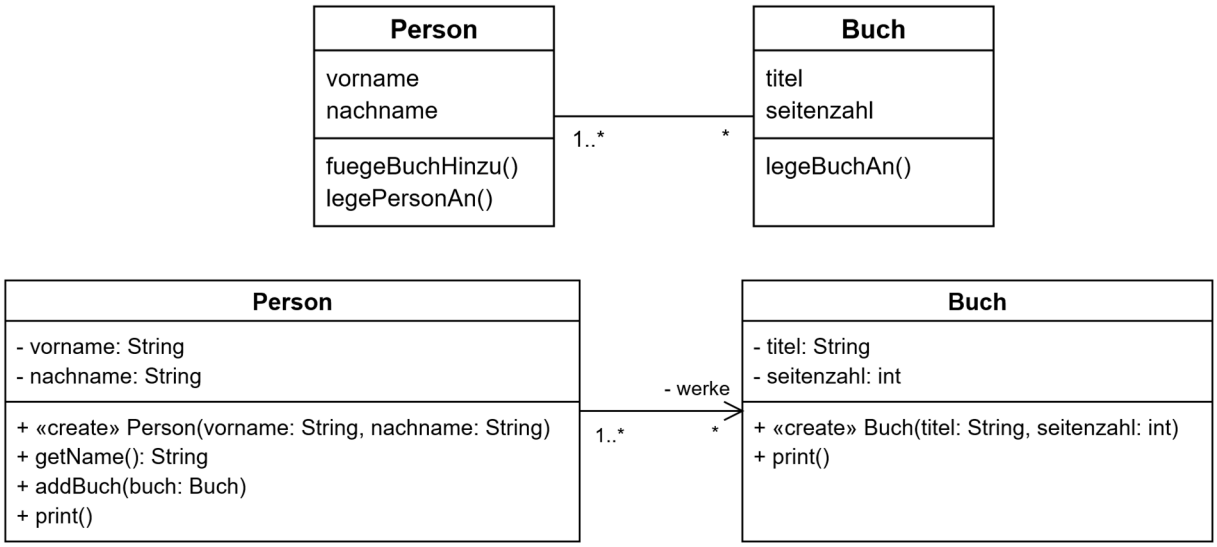


Abbildung 112: Typische Diagramme für den Einsatz in der Analyse (oben) und im Entwurf (unten)

Da man sich in der Analyse noch nicht um die Implementierungsdetails kümmert, werden meist auch weniger Details in die Diagramme aufgenommen. So werden z. B. Sichtbarkeiten, Datentypen, Parameter, Rollen und Navigationsrichtungen häufig weggelassen. Auch nimmt man nur die Methoden auf, die aus fachlicher Sicht eine wichtige Aufgabe erfüllen. Die zahl-

reichen Getter- und Settermethoden, Hilfsmethoden, u. ä. werden hier nicht benötigt. Attribute und Methoden tragen oft eher fachlich orientierte Bezeichnungen, wie z. B. „*legeBuchAn( )*" für einen Konstruktor.

Die obere Darstellung in Abbildung 112 ist typisch für die Analyse, die untere für den Entwurf.

Ein Entwurfsdiagramm ist ein direktes Spiegelbild der Implementierung. Um Code und Diagramm synchron zu halten, ist es hilfreich, ein Werkzeug zu verwenden, das Änderungen im Diagramm in den Code übertragen kann, und umgekehrt.

Auch wenn ein fachliches Diagramm normalerweise nicht direkt implementiert wird, sollte es doch ein korrektes Klassendiagramm sein, das man zumindest prinzipiell implementieren könnte. Ansonsten sind die Diagramme nicht mehr präzise und eindeutig. Dies erschwert dann einerseits die Kommunikation, andererseits die Weiterentwicklung zum Entwurfsdiagramm.

Mögliche Ausgangspunkte und Informationsquellen für die Entwicklung von Analysediagrammen, sind z. B.

- Use-Case-Diagramme und Beschreibungen
- Vorhandene Dokumentationen und Ablaufbeschreibungen
- Vorhandene Systeme, Formulare u. ä.
- Interviews
- Fachquellen (Literatur, technische Beschreibungen, Normen, Gesetze, …)

10.2 Beispiel für ein Analysediagramm

Als Beispiel wird der Use-Case „Noten eintragen" aus einem System zur Verwaltung von Prüfungen in einer Hochschule betrachtet (Tabelle 19). Abbildung 113 zeigt ein Analysediagramm zur Umsetzung dieses Use-Case. Darin wurde das Boundary-Control-Entity-Schema verwendet. Dieses Schema sieht die folgenden verschiedenen Arten von Klassen vor:

- Boundary:
  Eine Boundary-Klasse bündelt die Interaktionsmöglichkeiten des Akteurs, die später durch eine Benutzungsoberfläche bereitgestellt werden müssen. Sie verfügt ausschließlich über Methoden. Typischerweise erstellt man zunächst eine Boundary-Klasse pro Use-Case. Boundary-Klassen sind optional, sie werden in der Analyse verwendet, wenn die Funktionen, die dem Benutzer zur Verfügung gestellt werden, explizit dokumentiert werden sollen.

- Control:
  Eine Control-Klasse leitet Anfragen an andere Klassen weiter, führt Berechnungen u. ä. durch. Sie verfügt selbst meist kaum Attribute. Eine Control-Klasse kann die Objekte anderer Klassen kennen, z. B. kann sie über eine Liste aller Klausuren verfügen. Damit übernimmt sie die Aufgabe der Objektverwaltung. In Kapitel 7.1 wurden derartige Aufgaben, wie z. B. eine Liste aller Objekte oder das Finden von Objekten in der betreffenden Klasse mit Hilfe von Klassenmethoden und -attributen implementiert. Im

Sinne der Trennung von Verantwortlichkeiten ist es insbesondere in größeren Systemen sinnvoll, diese Aufgaben in eine eigene Control-Klasse zu verlagern.

Im ersten Schritt kann man für jeden Use-Case eine Control-Klasse anlegen. Später kann man diese Klassen ggf. noch aufteilen oder zusammenfassen.

- Entity:
  Entity-Klassen beinhalten die eigentlichen Informationen und Methoden zum Bearbeiten dieser Informationen.

Die Zugehörigkeiten zu diesen drei Arten von Klassen wurden in Form von „Stereotypen" in die Klassensymbole eingetragen («*Boundary*», etc.).

Name	Noten eintragen
Vorbedingungen	Es ist eine Klausur selektiert
Nachbed. im Erfolgsfall	Die Noten einer Klausur sind komplett eingetragen und freigegeben.
Nachbedingung bei Fehlschlag	Keine Noten eingetragen, oder Noten einer Klausur teilweise eingegeben und zwischengespeichert.
Akteure	Prüfer
Auslösendes Ereignis	Der Prüfer wählt „Noten eintragen"
Beschreibung	1. Das System zeigt die Teilnehmerliste der Klausur an. 2. Der Prüfer trägt die Note jedes Teilnehmers ein. 3. Der Prüfer gibt die komplette Notenliste frei.
Erweiterungen / Alternativen	a) Bis Schritt 3 ist ein Abbruch möglich. b) Falls ein Teilnehmer fehlt, kann er noch hinzugefügt werden. b) Zu Schritt 3: Falls die Notenliste unvollständig ausgefüllt ist, kann man sie auch speichern und später weiterbearbeiten.

Tabelle 19: Beschreibung des Use-Case „Note eintragen"

Es werden zunächst die Entity-Klassen in Abbildung 113 betrachtet. Wie aus der Use-Case-Beschreibung hervorgeht, gibt es Klausuren, Noten und Teilnehmer. Im Diagramm findet sich eine Klasse „*Klausur*".

Teilnehmer an Klausuren sind in der Hochschule immer Studierende, daher gibt es eine Klasse „*Studierender*". Eine Note bezieht sich normalerweise immer auf eine Klausur und einen Studierenden. Es muss bemerkt werden, dass es sich hierbei zunächst um Annahmen handelt. Wenn derartige Informationen nicht in den Use-Case-Beschreibungen oder anderen Dokumenten enthalten sind, müssen sie durch Rücksprache mit dem Auftraggeber überprüft werden.

Damit eine Teilnehmerliste für eine Klausur angezeigt werden kann, muss dem System bekannt sein, welcher Studierende an welcher Klausur teilnimmt. Dies könnte man durch eine direkte Assoziation zwischen Klausur und Studierender abbilden. Andererseits muss ja anschließend die Note eingetragen werden, die sich auf einen Studierenden und eine Klausur bezieht. Dazu könnte man eine assoziative Klasse verwenden (vgl. Kapitel 7.6), oder man nutzt gleich eine eigene Klasse.

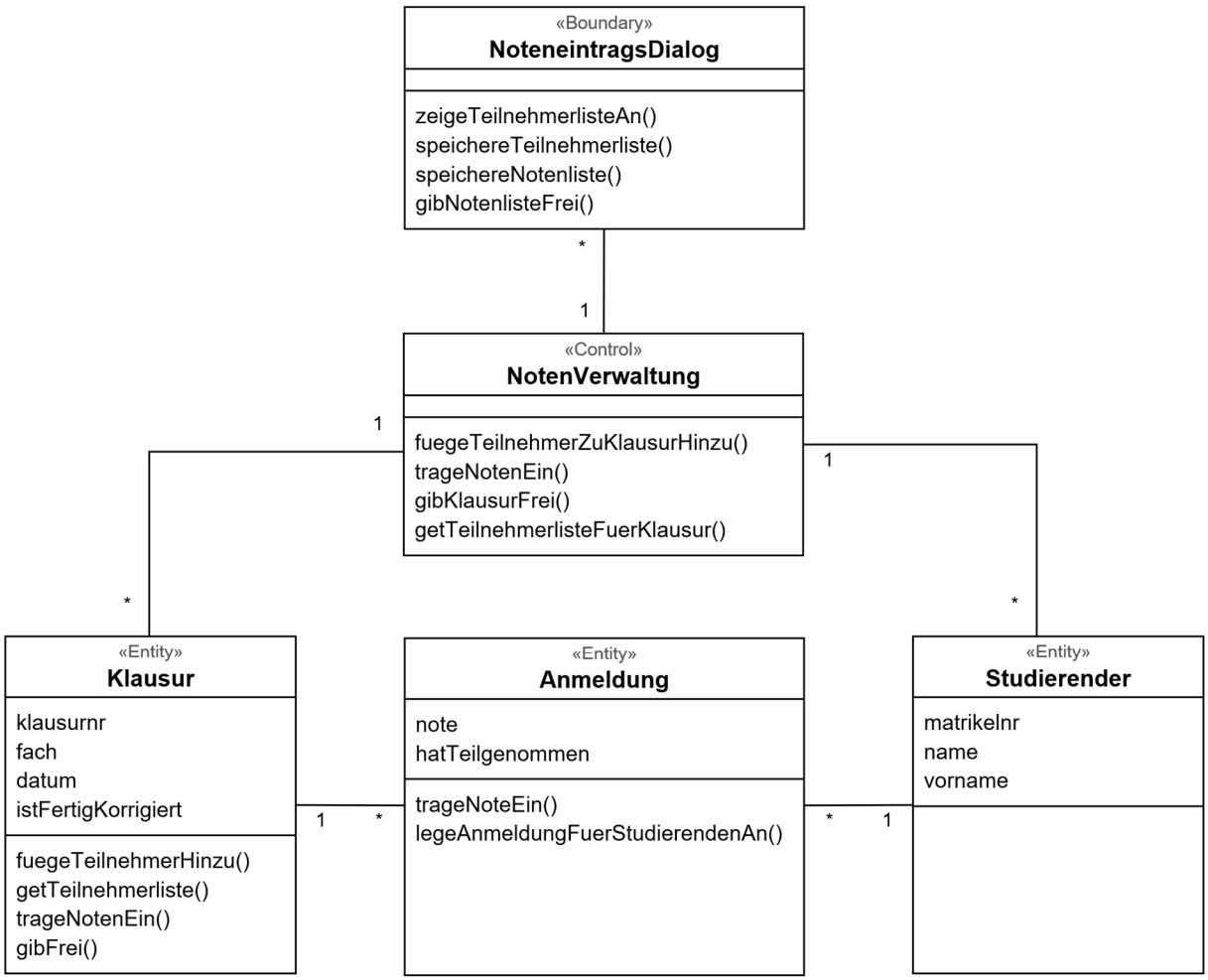


Abbildung 113: Beispiel eines Analysediagramms zur Umsetzung des Use-Case „Noten eintragen“

Im vorliegenden Fall wird eine Klasse „*Anmeldung*“ verwendet. Diese wird auch noch in anderen Use-Cases genutzt: Meldet sich ein Studierender zu einer Prüfung an, so wird ein entsprechendes Anmeldungsbjekt angelegt. Zu Beginn der Prüfung wird eine Anwesenheitskontrolle durchgeführt. Dies wird im Attribut „*hatTeilgenommen*“ vermerkt. Die Information über die Teilnahme ist auch notwendig, um im Use-Case „*Noten eintragen*“ eine Liste der tatsächlichen Teilnehmer anzeigen zu können. In diesem Use-Case werden dann schließlich die Notenwerte in die betreffenden Anmeldungsobjekte eingetragen.

Da Entity-Klassen meist in mehreren Use-Cases genutzt werden, ergibt sich ihre endgültige Struktur erst nachdem man alle diese Use-Cases analysiert hat. In diesem Beispiel gehen die genauen Attribute von Klausur und Studierender auch aus anderen Use-Cases hervor. Lediglich das Attribut „*istFertigKorrigiert*“ der Klasse Klausur wird vom Use-Case „*Noten eintragen*“ gesetzt. Hier erfolgt ein Vermerk, wenn die Notenliste freigegeben wurde.

Die Doppelpunkte hinter den Attributnamen ergeben sich aus einer Eigenschaft des verwendeten Modellierungstools, das kein komplettes Weglassen der Datentypen zulässt. Daher wurde der Eintrag der Datentypen einfach leer gelassen. Der Doppelpunkt wird in dem Tool trotzdem angezeigt.

Die Methoden in der Boundary-Klasse „*NoteneintragsDialog*“ ergeben sich aus der Use-Case-Beschreibung (Tabelle 19). Sie ermöglichen es, die Teilnehmerliste freizugeben, sie nach ggf. erforderlichen Änderungen zu speichern, die noch nicht fertig gestellte Notenliste zu speichern und die Notenliste freizugeben.

Bei der Control-Klasse „*NotenVerwaltung*“ handelt es sich um ein „*Singleton*“ d. h. gibt es nur ein Objekt von dieser Klasse, das die gesamte Koordination der Entity-Klassen im Rahmen des Use-Cases vornimmt (vgl. Kapitel 16.2). Die Boundary-Klasse leitet letztlich nur die Aufrufe an die Control-Klasse weiter, die dann entsprechende Entity-Objekte anlegt, einander zuordnet und ihre Methoden aufruft.

Für die Methoden zur Ausführung der eigentlichen Aufgaben gilt: Methoden sollten in der Klasse untergebracht werden, die gerade genug Informationen hat, um die Aufgabe zu erbringen.

Abbildung 114 zeigt als Beispiel den Nachrichtenaustausch beim Speichern der Teilnehmerliste. Zum Verständnis der Arbeitsweise wurden bereits Parameter eingefügt. Der NoteneintragsDialog ruft für alle neu hinzugefügten Teilnehmereinträge die Methode „*fuegeTeilnehmerZuKlausurHinzu( )*“ auf und übergibt den jeweiligen Namen. Die NotenVerwaltung sucht das entsprechende Objekt der Klasse „*Studierender*“ heraus. Dann schickt sie dem Klausur-Objekt die Nachricht „*fuegeTeilnehmerHinzu( )*“ und übergibt das Studierenden-Objekt als Parameter „*teilnehmer*“. Das Klausurobjekt legt dann ein neues Objekt der Klasse „*Anmeldung*“ an.

Eine Variante dieses Ablaufs ist in Abbildung 115 dargestellt. Hier ist der Ablauf zunächst gleich. Allerdings ruft die NotenVerwaltung nicht mehr „*fuegeTeilnehmerHinzu( )*“ beim Klausur-Objekt auf. Stattdessen legt sie zunächst das neue Anmeldung-Objekt an und übergibt dies dann in der Nachricht „*fuegeAnmeldungHinzu( )*“ an die Klausur. Die betreffenden Methoden müssten natürlich noch ins Klassendiagramm aufgenommen werden.

Diese Variante widerspricht dem oben genannten Prinzip, demzufolge Methoden in der Klasse untergebracht werden sollten, die gerade genug Informationen hat, um die Aufgabe zu erbringen. Im Beispiel hat nämlich die Klasse „*Klausur*“ genug Informationen, um eine neue Anmeldung anzulegen. Es ist also nicht notwendig, dies in der Klasse „*NotenVerwaltung*“ zu erledigen. Diese Klasse wird dadurch zur „allwissenden“ Klasse. Sie muss jetzt nämlich nicht nur die Klasse Klausur und ihre Methodensignaturen kennen, sondern auch die Details, wie Studierende und Klausuren einander zugeordnet sind. Das ist wenig änderungsfreundlich. Falls sich Details bzgl. der Anmeldung ändern, ist nicht mehr nur die Klasse „*Klausur*“, sondern auch die Klasse „*NotenVerwaltung*“ betroffen.

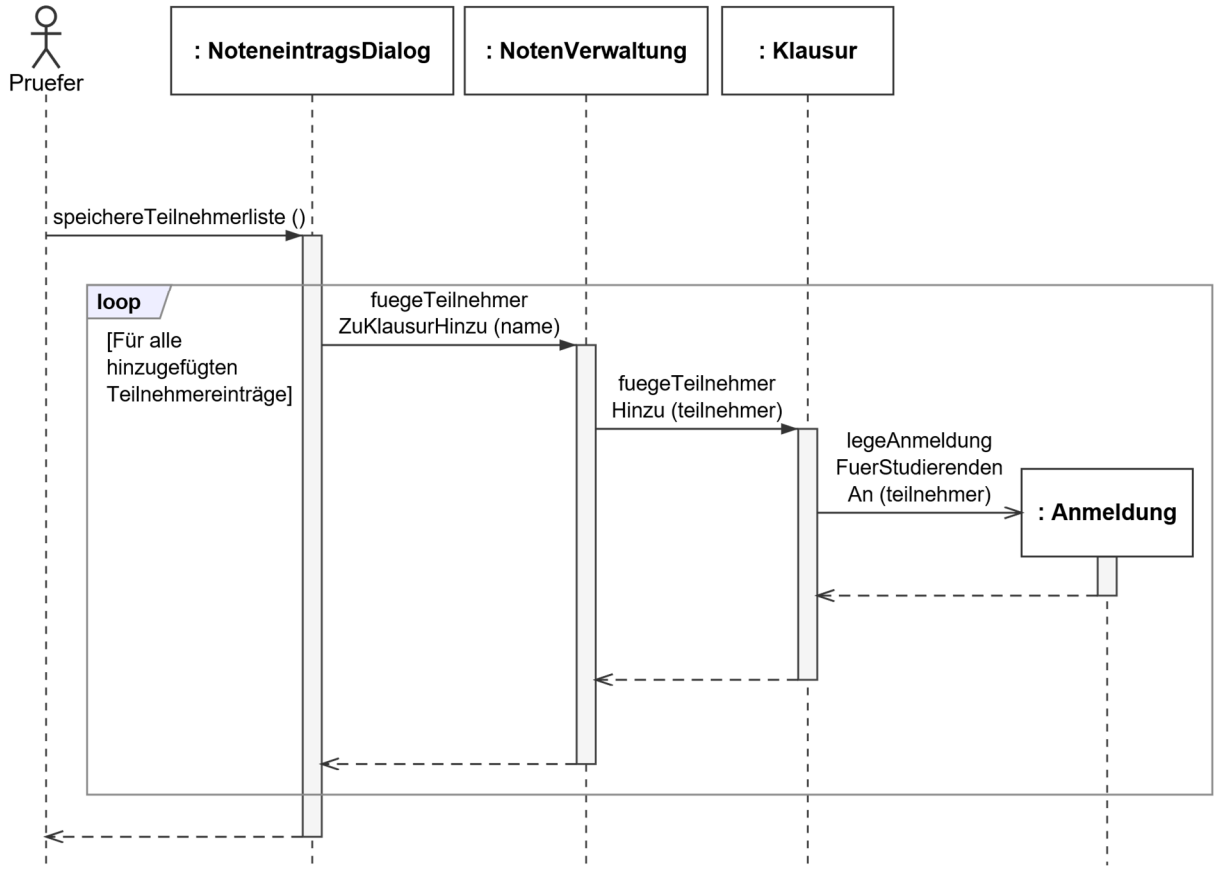


Abbildung 114: Sequenzdiagramm für das Hinzufügen von Teilnehmern

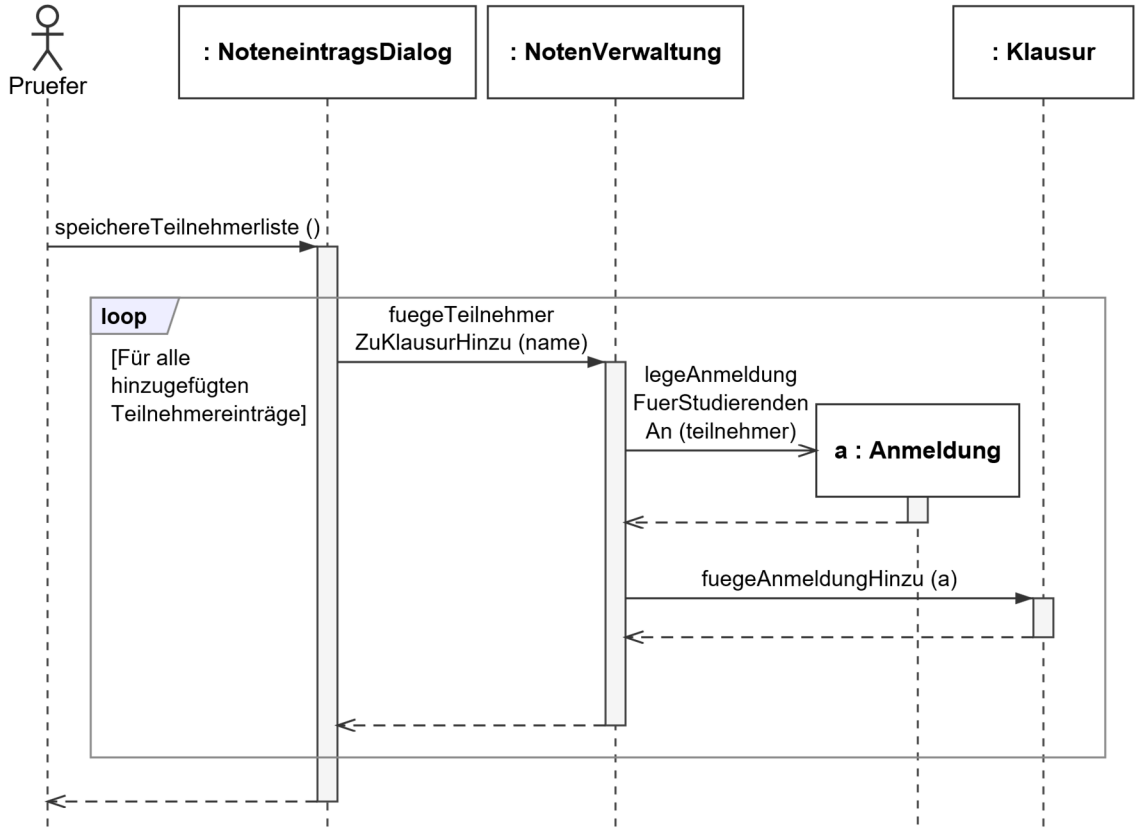


Abbildung 115: Weniger günstige Lösung mit NotenVerwaltung als allwissender Klasse

10.3 Verfeinerung des Klassendiagramms

Wenn man eine erste Version des Analyseklassendiagramms erstellt hat, verfeinert man es nach und nach. Mögliche Fragen, die einem dabei helfen können, sind z. B.:

- Welche Eigenschaften gehören zusammen und können daher zu einer Klasse zusammengefasst werden?
- Über welche Objekte des Anwendungsbereichs müssen Informationen gespeichert werden?
  Welche Klassen eignen sich dafür?
- Welche Klassen eignen sich zur Aufnahme bestimmter Operationen?
- Umfasst eine Klasse mehrere unterschiedliche Aspekte, die man besser in getrennte Klassen auslagert?
- Mit welchen anderen Objekten steht ein Objekt in Beziehung?
  Gilt die Beziehung immer?
  Mit wie vielen Objekten kann eine Beziehung bestehen?
  Hieraus werden Assoziationen entwickelt.
- Wie lassen sich die einzelnen Anforderungen mit den bisherigen Klassen abbilden?
  Können alle Use-Cases durchgeführt werden?
  Sind Erweiterungen notwendig?
- Gibt es Gemeinsamkeiten, die mit Hilfe von Vererbung abgebildet werden können?

10.4 Analysemuster

Im Folgenden werden einige Muster für die Modellierung typischer Fragestellungen vorgestellt, wie sie in der Analyse immer wieder vorkommen. Die Muster entstammen dem „Lehrbuch der Objektmodellierung“ von Balzert [Balz11]. Sie wurden z. T. leicht modifiziert.

10.4.1 Analysemuster Liste

Möchte man eine einfache Liste mehrerer Objekte modellieren, so benötigt man eine Klasse zur Repräsentation der Liste selbst und eine Klasse für die Listenelemente, die mit einer 1:*-Assoziation verbunden sind. Die Aggregation drückt aus, dass die Elemente Teil der Liste sind. Das Diagramm aus Abbildung 116 stellt die Teilnehmerliste einer Reise dar.

10.4.2 Analysemuster Typ-Exemplar

Das Beispiel aus Abbildung 117 stammt aus einer Bibliothek. Dort kann es mehrere Exemplare desselben Buchs geben. Sie alle haben denselben Titel, Autor und Verlag. Sie unterscheiden sich lediglich durch eine von der Bibliothek vergebene eindeutige Nummer – der Signatur, die auf dem Buchrücken aufgeklebt ist.

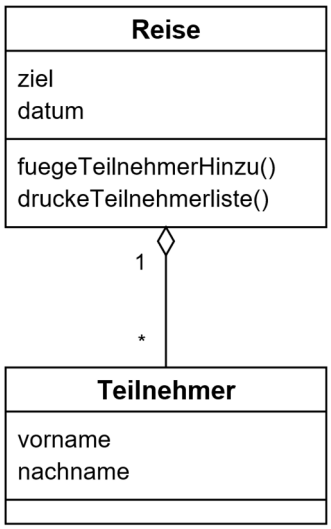


Abbildung 116: Analysemuster „Liste“

Die Gemeinsamkeiten des jeweiligen Buchs werden in der Klasse „Buchbeschreibung“ abgespeichert. Sie repräsentiert also einen Typ. Er beschreibt die identischen Eigenschaften aller Exemplare.

Für jedes einzelne physische Exemplar wird im Beispiel ein Objekt der Klasse Buchexemplar angelegt, das die Signatur enthält. Das Buchexemplar gehört immer zu exakt einer Buchbeschreibung. Umgekehrt kann es zu einer Buchbeschreibung beliebig viele Buchexemplare geben.

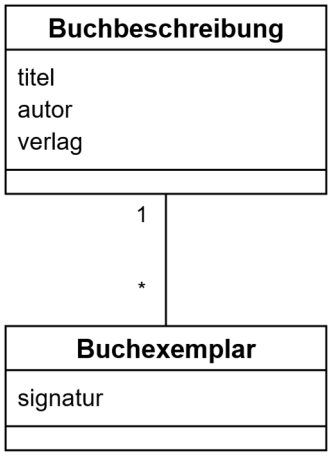


Abbildung 117: Analysemuster „Typ-Exemplar“

10.4.3 Analysemuster Historie

Oftmals ist es wichtig, dass man später noch nachvollziehen kann, was sich im Laufe der Zeit geändert hat. Bei der Klasse „*Angestellter*“ im oberen Teil der Abbildung 118 ist dies nicht möglich. Ändert sich die Tätigkeit des Angestellten, erhält das Attribut „*taetigkeit*“ einfach einen anderen Wert. Welchen Wert dieses Attribut früher hatte, kann man nicht mehr rekonstruieren.

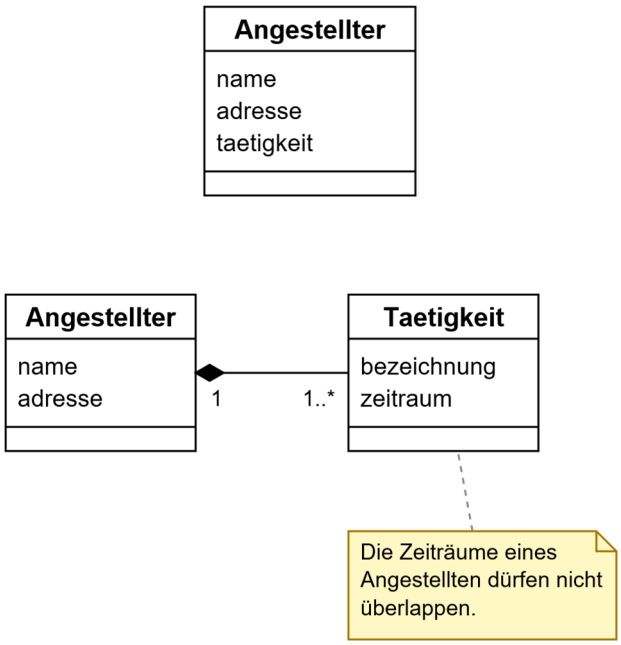


Abbildung 118: Klasse Angestellter ohne Historie (oben) und mit Historie (unten)

Möchte man dies tun, hilft einem die Einführung einer Historie für die Tätigkeiten. Im unteren Teil von Abbildung 118 ist die Tätigkeit kein einfaches Attribut mehr, sondern eine eigene Klasse. Die Komposition macht deutlich, dass es sich dabei dennoch um einen untrennbaren Bestandteil der Klasse Angestellter handelt. Aufgrund der Multiplizität 1..* kann einem Angestellten nun mehr als eine Tätigkeit zugeordnet sein. Damit man weiß, von wann bis wann ein Angestellter eine zugeordnete Tätigkeit ausgeführt hat, enthält die Tätigkeit zusätzlich noch das Attribut „*zeitraum*". Bei der Tätigkeit, die der Angestellte momentan ausführt, ist das Ende des Zeitraums noch offen.

Im Ursprungsdiagramm konnte ein Angestellter zu jedem Zeitpunkt immer nur eine Tätigkeit haben. Dies bedeutet im geänderten Diagramm, dass die Zeiträume der Tätigkeiten eines Angestellten nicht überlappen dürfen. Dies geht aus dem Modell nicht direkt hervor und muss daher als Zusatzbedingung formuliert werden. Sie wurde hier als Notiz ins Diagramm eingefügt.

Soll es auch möglich sein, dass ein Angestellter mehrere Tätigkeiten zur gleichen Zeit innehat (z. B. in Teilzeit), dann ist dies in der Variante mit der Historie ebenfalls möglich. Man muss dann nur die Zusatzbedingung der Überlappungsfreiheit streichen.

10.4.4 Analysemuster Gruppe und Gruppenhistorie

Das in der oberen Hälfte von Abbildung 119 dargestellte Muster einer Gruppe ist sehr einfach. Die Mitglieder einer Gruppe (hier: die Angestellten) werden über eine Assoziation mit der Gruppe (hier: die Abteilung) verbunden. Die Gruppe kann beliebig viele Mitglieder haben. Jeder Angestellte gehört im Beispiel zu genau einer Abteilung. Prinzipiell ist es aber auch möglich, zu mehreren Gruppen zu gehören, z. B. könnte ein Angestellter zu mehreren Projektteams gehören. Dafür muss dann eine andere Multiplizität als 1 gewählt werden, z. B. „*".

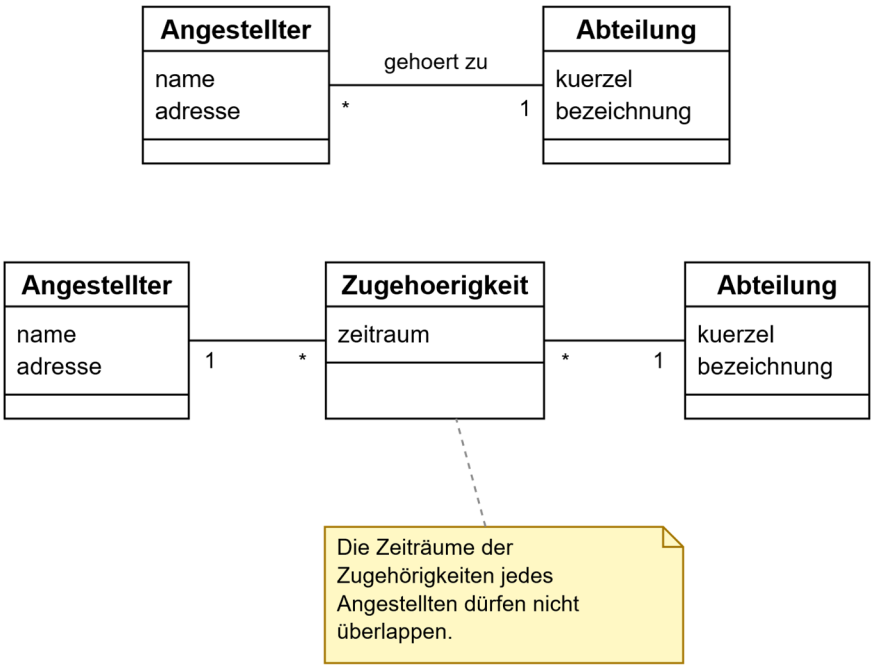


Abbildung 119: Gruppenzugehörigkeit ohne Historie (oben) und mit Historie (unten)

Der untere Teil der Abbildung zeigt die Gruppenhistorie. Hier sind die Muster „Gruppe" und „Historie" kombiniert worden. Ein Objekt der Klasse „*Zugehoerigkeit*" gibt an, von wann bis wann ein bestimmter Angestellter zu einer bestimmten Abteilung gehört hat bzw. derzeit noch gehört. Auch hier ist wieder eine Zusatzbedingung erforderlich, wenn man sicherstellen möchte, dass ein Angestellter zu einem Zeitpunkt immer nur zu einer Abteilung gehört.

10.4.5 Analysemuster Wechselnde Rollen

Auch das folgende Muster basiert u. a. auf dem Muster „Historie". Ausgangspunkt ist das Modell in Abbildung 120. Die abstrakte Klasse „*Registrierter Arzt*" hat zwei Unterklassen „*Niedergelassener Arzt*" und „*Angestellter Arzt*". Das bedeutet, ein niedergelassener Arzt ist ein registrierter Arzt, ebenso ist ein angestellter Arzt ein registrierter Arzt. Die abstrakte Oberklasse bildet die Gemeinsamkeiten von niedergelassenen und angestellten Ärzten ab. Im Beispiel sind dies die Attribute „*name*" und „*eintragungsnummer*".

Das Diagramm aus Abbildung 120 ist gut geeignet, solange es keine Wechsel gibt. Wenn ein niedergelassener Arzt zu einem angestellten Arzt wird (oder umgekehrt), dann muss ein komplett neues Objekt der anderen Klasse angelegt werden, die Werte der Attribute „*name*" und „*eintragungsnummer*" müssen kopiert werden, und das alte Objekt muss gelöscht oder zumindest aus der Liste der aktiven Arzt-Objekte entfernt werden. Das ist zwar möglich, aber nicht unbedingt elegant.

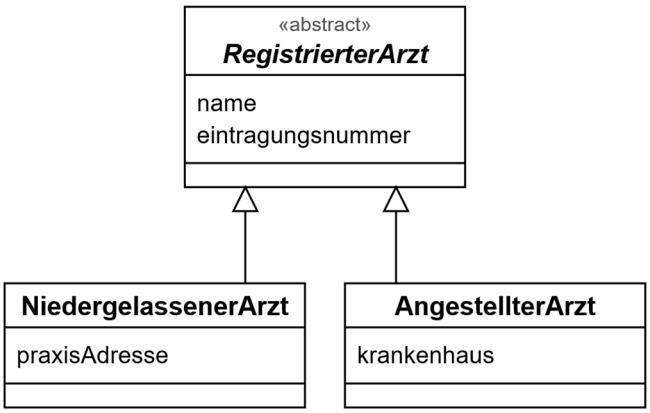


Abbildung 120: Eine Veränderung eines niedergelassenen Arztes zu einem angestellten Arzt ist bei dieser Lösung schwierig

Eine bessere Lösung zeigt Abbildung 121 Hier ist die Klasse „*Registrierter Arzt*" eine konkrete Klasse. Für jeden Arzt gibt es ein Objekt dieser Klasse, das sich bei einem Wechsel des Status auch nicht ändert. Die Tätigkeit als niedergelassener oder angestellter Arzt wird nun über die abstrakte Klasse „*Status*" mit den konkreten Unterklassen „*NiedergelasseneTätigkeit*" und „*AngestelltenTätigkeit*" abgebildet.

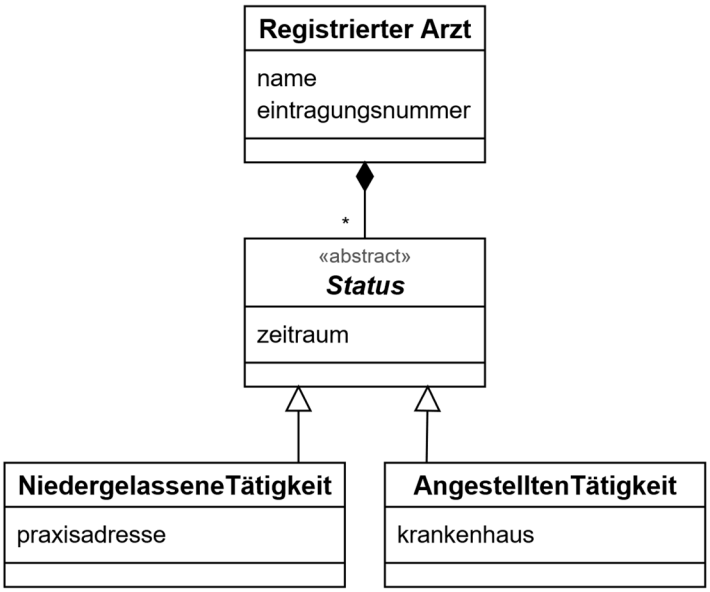


Abbildung 121: Mit dem Analysemuster „Wechselnde Rollen" sind Veränderungen der Tätigkeit leicht möglich – und später noch nachvollziehbar

Die Aussage, die mit diesem Diagramm ausgedrückt wird, ist also nicht mehr „ein niedergelassener Arzt ist ein registrierter Arzt", sondern „ein registrierter Arzt *kann* den Status ‚niedergelassene Tätigkeit' haben". Auch wenn beide Aussagen richtig sind, hat die Art der Modellierung unterschiedliche Konsequenzen. Ein und derselbe Sachverhalt kann durchaus verschieden modelliert werden. Ob man einen solchen Zusammenhang besser mit Hilfe von Vererbung oder mit Hilfe einer Komposition modelliert, hängt insbesondere davon ab, ob die betreffende Eigenschaft zumindest im Kontext des zu entwickelnden Systems unveränderlich ist, oder ob sie öfter einmal wechselt.

Das Muster „Historie“ findet sich hier in der Klasse „*Status*“ wieder. Mit Hilfe des Attributs „*zeitraum*“ kann man nachvollziehen, wann ein bestimmter Arzt als niedergelassener und wann als angestellter Arzt gearbeitet hat. Will man ausschließen, dass ein Arzt zur gleichen Zeit angestellt und niedergelassen ist, muss man auch hier eine Zusatzbedingung formulieren, dass die Zeiträume der Status-Objekte eines registrierten Arztes nicht überlappen dürfen.

10.4.6 Analysemuster Baugruppe

Die Zusammensetzung von Objekten aus anderen Objekten wird im Muster „Baugruppe“ mit Hilfe von Aggregationen dargestellt. Das Beispiel in Abbildung 122 wurde bereits in Kapitel 7.4 erläutert. Die beiden folgenden Analysemuster, „Stückliste“ und „Verwendungsnachweis“ betreffen ebenfalls die Zusammensetzung von Objekten aus anderen Objekten. Dort werden auch die Unterschiede der verschiedenen Muster diskutiert.

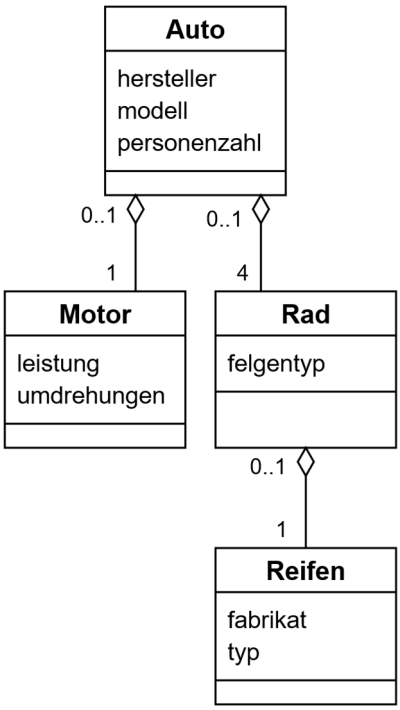


Abbildung 122: Analysemuster „Baugruppe“

10.4.7 Analysemuster Stückliste

Eine Stückliste dient als Vorschrift für den Zusammenbau von Produkten. In Abbildung 123 ist dargestellt, aus welchen anderen Produkttypen ein bestimmter Produkttyp zusammengesetzt ist. Wie der Buchtyp beim Muster „Typ-Exemplar“ repräsentiert die Klasse Produkttyp keine einzelnen Exemplare, sondern einen Typ. Bei einem Fahrzeug wäre das z. B. „Ford Focus“ oder „VW Golf“, nicht das einzelne Fahrzeug mit der Autonummer „ZW-X 45“.

Mit Hilfe der Klasse „Bestandteil“ wird dargestellt, wie viele Stück des jeweils enthaltenen Produkttyps in den übergeordneten Produkttyp eingebaut werden. Das Attribut „*menge*“ hätte beispielsweise den Wert 4, wenn der übergeordnete Produkttyp ein Fahrzeugtyp ist, und der untergeordnete Produkttyp ein bestimmter Rad-Typ. Damit kann man z. B. ausrechnen, wie

viele Stück einer bestimmten Sorte von Rädern benötigt werden, wenn man z. B. eintausend Exemplare eines Fahrzeugtyps herstellt.

Ein in einem anderen Produkttyp enthaltener Produkttyp kann selbst wieder weitere untergeordnete Produkttypen enthalten. Es lässt sich also eine mehrstufige Hierarchie aufbauen. Durch die Verwendung von Aggregationen wird sichergestellt, dass es sich um eine strenge Hierarchie handelt, denn natürlich kann ein übergeordneter Produkttyp nicht selbst wieder Teil eines untergeordneten Produkttyps sein.

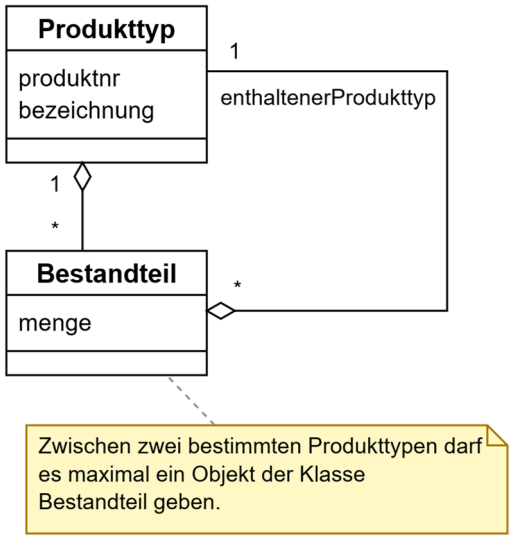


Abbildung 123: Analysemuster „Stückliste“

Ein Unterschied zur Baugruppe besteht darin, dass die Stückliste Produkttypen enthält und die Baugruppe einzelne Exemplare. Man kann beliebig neue Produkttypen anlegen und ihre Zusammensetzung festlegen. Dies ist mit der Baugruppe nicht möglich. Dort ist die Hierarchie durch die Klassenstruktur fest vorgegeben. Eine Änderung würde eine Änderung der Software selbst erfordern.

Ein Vorteil der Baugruppe ist, dass man spezifische Attribute und Methoden für die jeweiligen Produkttypen verwenden kann. Attribute wie „*leistung*“ oder „*felgentyp*“ wären in der Stückliste sehr unpassend, da die allermeisten Produkttypen ganz andere Eigenschaften als Leistung oder Felgentyp haben.

10.4.8 Analysemuster Verwendungsnachweis

Als dritte Möglichkeit zur Darstellung einer Produktstruktur ist in Abbildung 124 das Muster „Verwendungsnachweis“ abgebildet. Hier handelt es sich wie bei der Stückliste um eine generische Struktur, d. h. es ist nicht vorgegeben, mit welchen Typen von Produkten man es zu tun hat. Stattdessen kann man die Struktur für beliebige Produkte verwenden. Im Gegensatz zur Stückliste werden hier aber keine Produkttypen abgebildet, sondern Exemplare. Wie bei den Buchexemplaren im Muster „Typ-Exemplar“ handelt es sich um einzelne physische Exemplare, die sich z. B. durch eindeutige Seriennummern unterscheiden.

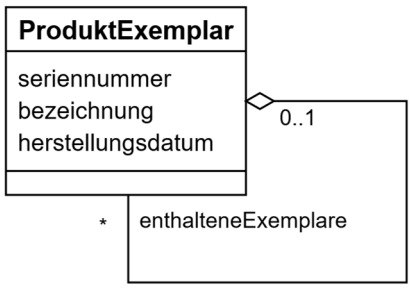


Abbildung 124: Analysemuster „Verwendungsnachweis“

Anders als in der Stückliste wird nicht dargestellt, wie ein Produkt eines bestimmten Typs prinzipiell zusammengesetzt ist, sondern aus welchen anderen Exemplaren ein bestimmtes Produktexemplar ganz konkret besteht. Es wird also nicht dargestellt, dass ein bestimmter Fahrzeugtyp vier Stück eines bestimmten Rad-Typs enthält. Stattdessen wird dargestellt, dass das Fahrzeug mit der Seriennummer 7874 die Räder mit den Seriennummern 4561, 4562, 4563 und 4564 enthält.

Auch diese Struktur ist hierarchisch aufgebaut. Jedes Produktexemplar kann beliebig viele andere Exemplare enthalten, die selbst wiederum andere Exemplare enthalten. Die strenge Hierarchie wird wiederum durch die Aggregation sichergestellt. Im Unterschied zur Baugruppe kann man auch mit diesem Muster beliebige Hierarchien von Produkten beliebigen Typs aufbauen.

Baugruppe, Stückliste und Verwendungsnachweis beziehen sich also auf ähnliche Sachverhalte, haben aber unterschiedliche Zwecke. Welches Muster sollte man in welchen Fällen verwenden?

- Verwende die Baugruppe,
  wenn der Typ der betrachteten Produkte eindeutig fest liegt und spezielle Attribute und Operationen für diese konkreten Typen benötigt werden (z. B. Auto-spezifische Attribute und Operationen).

- Verwende die Stückliste,
  wenn beliebige Produkttypen hinzugefügt werden sollen und die Zusammensetzung im Vordergrund steht, nicht aber die speziellen Eigenschaften des jeweiligen Typs.

- Verwende den Verwendungsnachweis,
  wenn einzelne Exemplare und ihre Zusammensetzung verwaltet werden sollen, und besondere Typ-spezifische Eigenschaften keine Rolle spielen.

11 Analysediagramm – Fallstudie Geräteverwaltung

In diesem Kapitel soll ein Analyse-Klassendiagramm für die Fallstudie einer Geräteverwaltung entwickelt werden, für die in Kapitel 4.15 ein Use-Case-Diagramm entwickelt wurde. Die folgenden Ausführungen beziehen sich auf die dort beschriebenen Anforderungen sowie die Diagramme und die erstellten Use-Case-Beschreibungen. Es empfiehlt sich, an dieser Stelle zunächst noch einmal Kapitel 4.15 zu lesen.

11.1 Räume und Mitarbeiter

Zunächst werden die Use-Cases „Raum bearbeiten", „Mitarbeiter bearbeiten" und „Mitarbeiter suchen" betrachtet. Für die Use-Cases wurden in Abbildung 125 die zwei Control-Klassen „*Raumverwaltung*" und „*Mitarbeiterverwaltung*" angelegt, wobei letztere für die beiden Use-Cases „Mitarbeiter bearbeiten" und „Mitarbeiter suchen" zuständig ist. Die Aufgaben der Methoden in den Control-Klassen gehen aus ihren Bezeichnungen hervor. Auf die Boundary-Klassen wird in dem Beispiel verzichtet, die Interaktion mit dem Benutzer bietet in dem Beispiel keine größeren Besonderheiten.

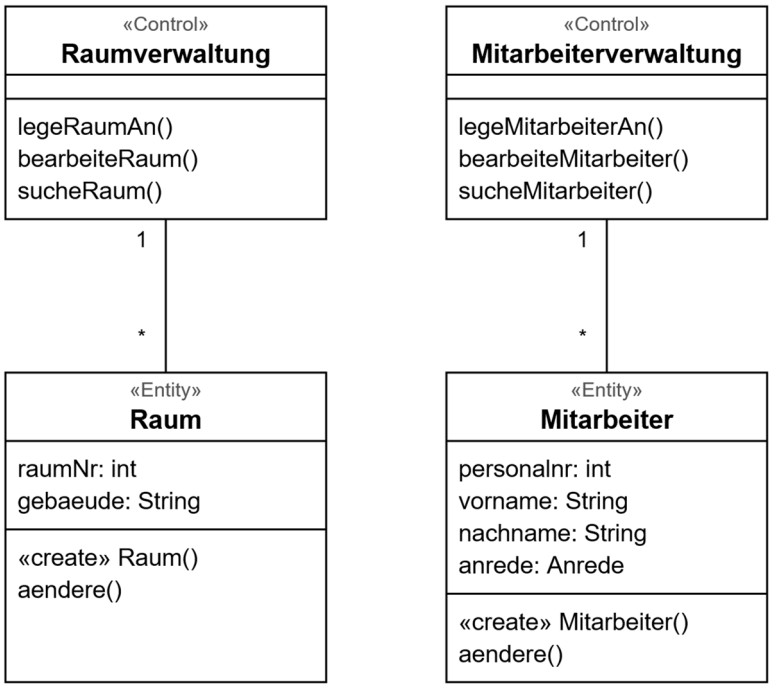


Abbildung 125: Räume und Mitarbeiter

Für die zu bearbeitenden Räume und Mitarbeiter wurden zwei Entity-Klassen definiert. Die Attribute ergeben sich aus den – hier nicht aufgeführten – Beschreibungen der Use-Cases. Zwar gibt man in der Analyse meist keine Datentypen an, doch stellt das hier verwendete Modellierungstool diese immer dar. Daher wurden einfach die Standardtypen des Tools verwendet (deren Namen sich z. T. in der Schreibweise von den Java-Datentypen unterscheiden). Im Entwurf werden dann die tatsächlich verwendeten Datentypen eingefügt. Für die Anrede kommt ein Aufzählungstyp zum Einsatz (Abbildung 126).

Abbildung 126: Aufzählungstyp Anrede

Die Methode „*aendere( )*" steht als Vertreter für die im Entwurf aufzunehmenden setter-Methoden. Sie werden von den „*bearbeite( )*"-Methoden der Control-Klassen aufgerufen. In den Use-Cases „Raum bearbeiten" und „Mitarbeiter bearbeiten" sollen auch neue Räume und Mitarbeiter angelegt werden. Hierzu dienen die „*legeAn( )*"-Methoden der Control-Klassen, die jeweils den Konstruktor der zugehörigen Entity-Klasse aufrufen.

Es gibt zwar keinen expliziten Use-Case für die Raum-Suche, dennoch wurde eine Such-Methode für Räume eingeführt, da man beim Bearbeiten eines Raumes diesen zunächst finden muss. Die Klassen „*Raumverwaltung*" und „*Mitarbeiterverwaltung*" verfügen jeweils über Listen aller betreffenden Raum- bzw. Mitarbeiter-Objekte. Daher lassen sich die Suchfunktionen sinnvoll in den Verwaltungsklassen unterbringen. Natürlich müssen die Suchfunktionen auch auf Attributwerte der Entity-Klassen zugreifen können. Hierfür werden im Entwurf die erforderlichen getter-Methoden ergänzt. In der Analyse geht man davon aus, dass man auf die benötigten Informationen zugreifen kann.

11.2 Geräte

Als nächstes werden die Use-Cases „Gerät anlegen", „Gerät bearbeiten" und „Gerät suchen" betrachtet. Für diese drei Use-Cases wurde in Abbildung 127 eine gemeinsame Control-Klasse „Geraeteverwaltung" angelegt. Zwar wurden in Kapitel 4.15 keine Beschreibungen der drei genannten Use-Cases erstellt, doch geht aus der dort gezeigten Beschreibung des Use-Case „Gerät verleihen" hervor, dass es Gerätetypen und zu jedem Gerätetyp einzelne Geräte, also Exemplare gibt. Dies entspricht dem Analysemuster „Typ-Exemplar" (Kapitel 10.4.2). Entsprechend wurden die Klassen „*Geraetetyp*" und „*Geraet*" angelegt. Ein Gerätetyp wäre zum Beispiel der Beamer „BM 7350" der Firma „Proinix". Eine Hochschule könnte dann z. B. fünf physische Geräte dieses Typs haben, die durch unterschiedliche Seriennummern eindeutig identifizierbar sind.

Zusätzlich wurde noch eine Klasse „*Kategorie*" eingeführt, um Gerätetypen besser einordnen zu können. Beispiele für Kategorien könnten sein „Beamer", „Kopierer", „Drucker", usw. Neben einer Suche, bei der Suchbegriffe eingegeben werden müssen, kann man dann auch über die Kategorien zu den enthaltenen Gerätetypen navigieren, indem man zunächst die gewünschte Kategorie auswählt und dann die hierzu verfügbaren Gerätetypen angezeigt bekommt.

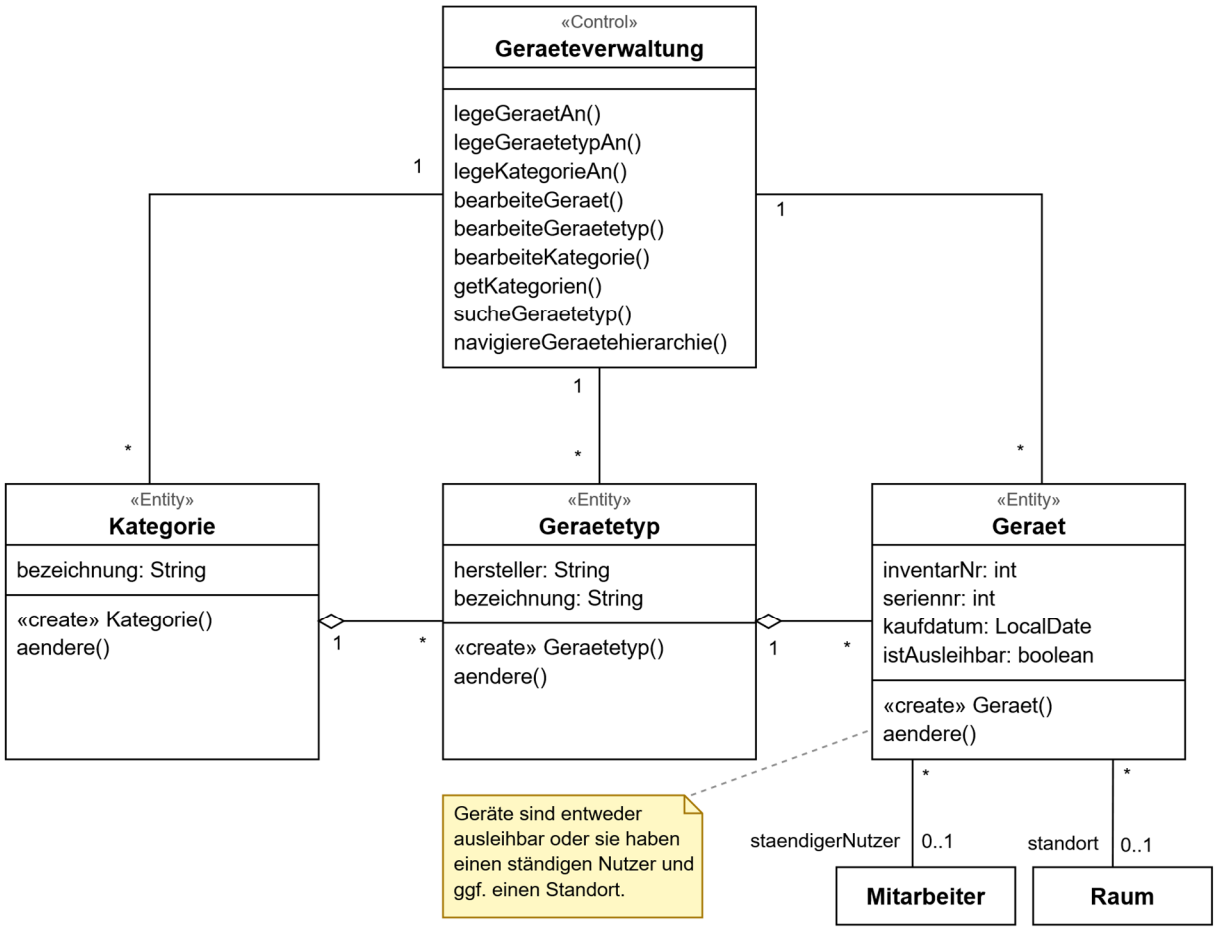


Abbildung 127: Geräte

Geräte können fest einem Mitarbeiter und einem Raum zugeordnet sein. Solche Geräte, die einen ständigen Nutzer und einen festen Standort haben, können nicht ausgeliehen werden. Bei ihnen muss das Attribut „*istAusleihbar*“ den Wert „*false*“ haben. Dieser Zusammenhang ist als Kommentar dokumentiert, da er ansonsten nicht aus dem Klassendiagramm hervorgeht. Die Details der Klassen Mitarbeiter und Raum sind aus Übersichtsgründen in diesem Diagramm weggelassen worden.

In der Klasse „*Geraeteverwaltung*“ ist für jede der verwalteten Entity-Klassen eine Methode zum Anlegen und eine zum Bearbeiten vorhanden. Diese Methoden sind auch dafür zuständig, Objekte einander zuzuordnen, wie z. B. Geräte zu Gerätetypen.

Schließlich gibt es noch eine Suchmethode für Gerätetypen, mit der man über die Eingabe eines Suchstrings z. B. nach Bezeichnung oder Hersteller suchen kann. „*NavigiereGeraetehierarchie( )*“ dient der bereits beschriebenen Navigation über die Kategorien zu den Gerätetypen und den zugehörigen Geräten. „*getKategorien( )*“ liefert als Grundlage für die Navigation zunächst einmal eine Liste aller vorhandenen Kategorien. Die weiterhin erforderlichen getter-Methoden zum Zugriff auf die untergeordneten Gerätetypen und Geräte sind aus Übersichtsgründen weg gelassen worden, wie dies in der Analyse oft gemacht wird.

11.3 Reservierungen

Zur Umsetzung der Use-Cases „Gerät reservieren“ (Tabelle 13) und „Reservierung bearbeiten“ sowie ihren Spezialisierung wurden in Abbildung 128 die Control-Klasse „*Reservierungsverwaltung*“ sowie die Entity-Klasse „*Reservierung*“ erstellt. Eine Reservierung bezieht sich auf einen Gerätetyp, nicht auf ein einzelnes Gerät. Schließlich ist es egal, welches von mehreren verfügbaren Geräten man bekommt, da die Geräte eines Typs alle dieselben Eigenschaften haben. Jede Reservierung ist außerdem dem Mitarbeiter zugeordnet, für den die Reservierung erfolgt ist.

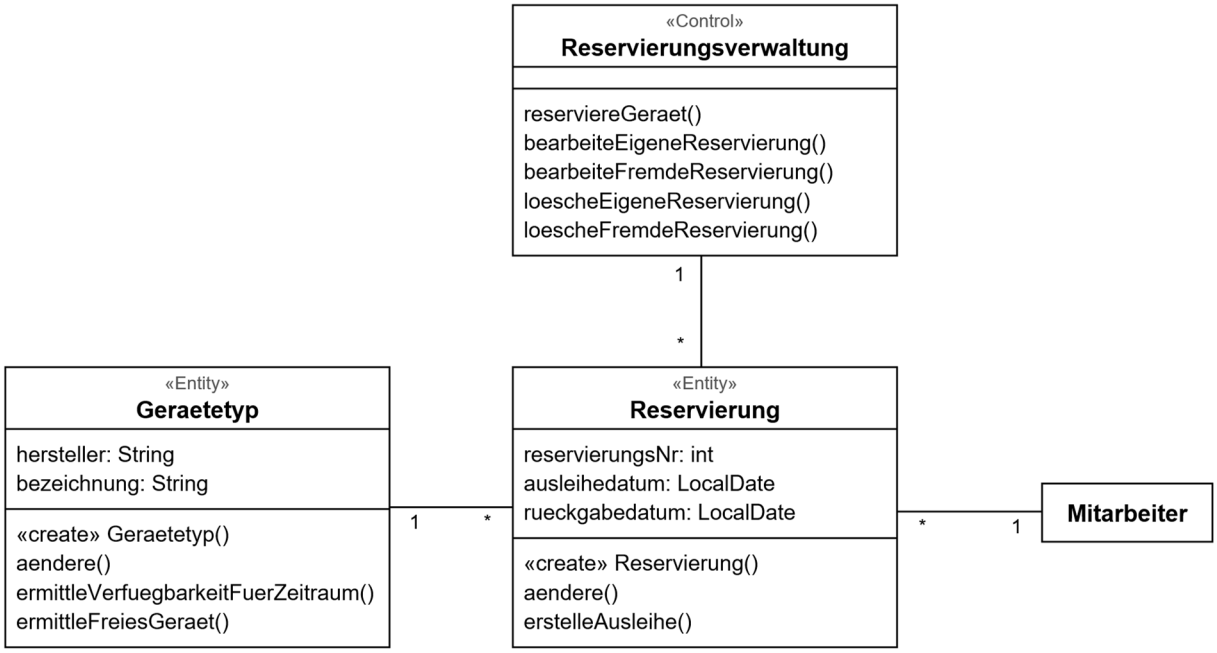


Abbildung 128: Reservierung

Damit man eine Reservierung erstellen kann, muss zunächst geprüft werden, ob im gewünschten Zeitraum überhaupt noch ein Gerät des betreffenden Typs verfügbar ist. Dies erledigt die Methode „*ermittleVerfuegbarkeitFuerZeitraum( )*“ der Klasse „*Gerätetyp*“. Sie muss die vorliegenden Reservierungen sowie die im nächsten Kapitel beschriebenen Ausleihen auswerten um festzustellen, ob die Reservierung möglich ist. Ansonsten darf kein Reservierungsobjekt angelegt werden.

Die weiteren Methoden der Klasse „*Reservierungsverwaltung*“ betreffen das Ändern und Löschen der eigenen Reservierungen eines Mitarbeiters im Use-Case „Eigene Reservierung bearbeiten“ sowie die Bearbeitung fremder Reservierungen durch einen Geräteverwalter im Use-Case „Fremde Reservierungen bearbeiten“. Es wurden keine eigenen Methoden für das Löschen von Reservierungen definiert. Das Löschen wird als Sonderfall des Bearbeitens von Reservierungen behandelt.

11.4 Ausleihe

Zur Umsetzung der Use-Cases „Gerät verleihen“ (Tabelle 14) und „Gerät zurücknehmen“ dienen die Control-Klasse „*Ausleiheverwaltung*“ und die Entity-Klasse „*Ausleihe*“ in Abbildung 129. Im Gegensatz zur Reservierung bezieht sich eine Ausleihe nicht auf einen Gerätetyp,

sondern auf ein konkretes Gerät, d. h. ein einzelnes Exemplar. Man muss ja nachvollziehen können, wer welches Exemplar hatte, z. B. wenn ein Gerät Beschädigungen aufweist. Eine Ausleihe ist immer einem Mitarbeiter zugeordnet.

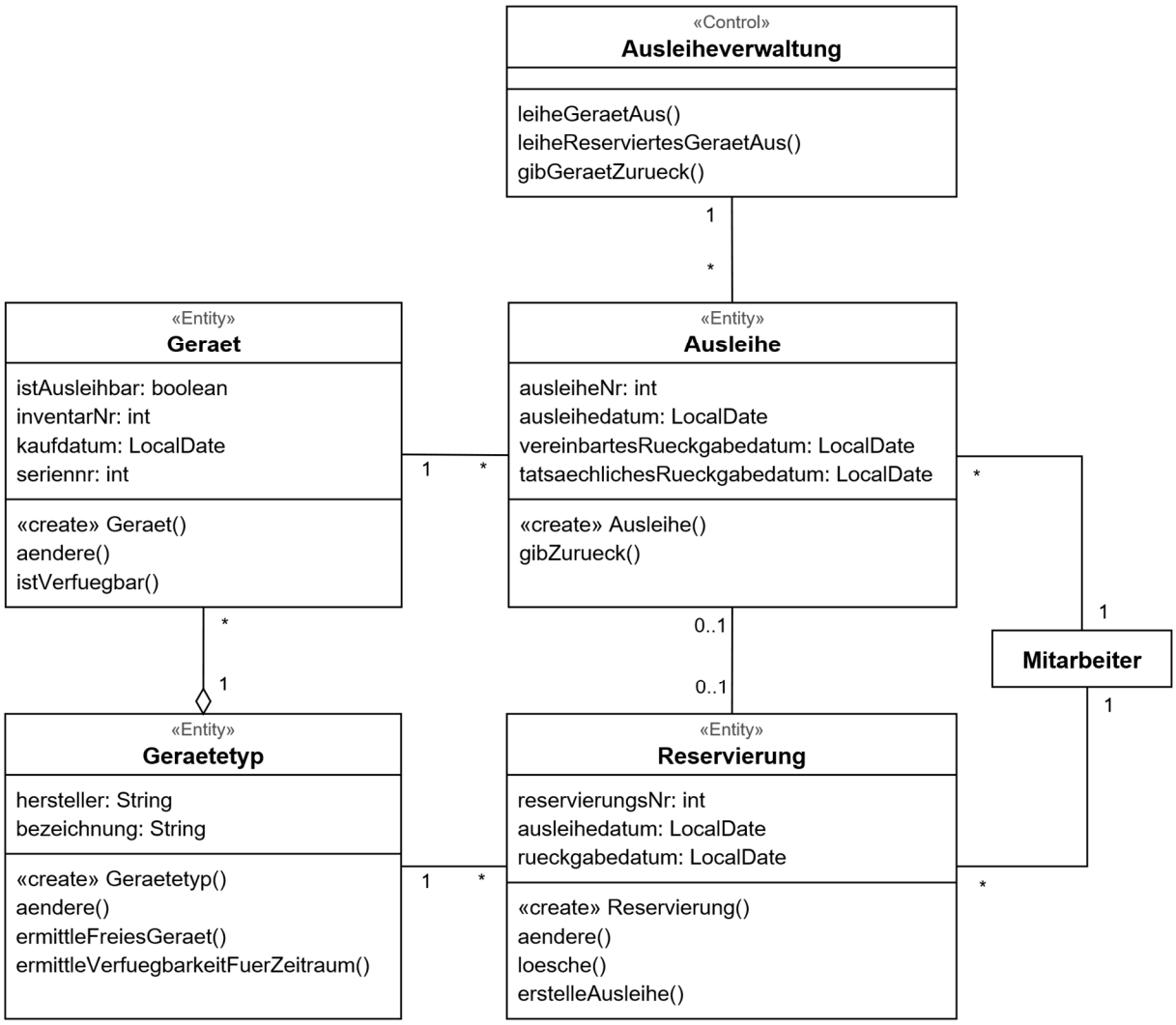


Abbildung 129: Ausleihe

Aus der Assoziation zwischen „Ausleihe“ und „Reservierung“ geht hervor, dass es sowohl Ausleihen gibt, die sich auf eine Reservierung beziehen, als auch solche, die dies nicht tun. Eine Ausleihe ohne Reservierung wird angelegt, wenn jemand ein Gerät ausleiht ohne es vorher reserviert zu haben. Hierzu dient die Methode „*leiheGeraetAus( )*“ der Control-Klasse. Sie ruft den Konstruktor der Ausleihe auf.

Zu einer Reservierung gibt es zunächst auch keine Ausleihe, diese wird erst angelegt, wenn das reservierte Gerät abgeholt wird. Hierfür ist die Methode „*leiheReserviertesGeraetAus( )*“ der Klasse „*Ausleiheverwaltung*“ zuständig, die wiederum „*erstelleAusleihe( )*“ von „Reservierung“ aufruft. Während die Attribute „*ausleihedatum*“ und „*rueckgabedatum*“ der Klasse Reservierung die geplanten Daten beinhalten, handelt es sich beim „*ausleihedatum*“ der Klasse „*Ausleihe*“ um das Datum, an dem das Gerät tatsächlich ausgeliehen wurde.

Zudem gibt es ein „*vereinbartesRueckgabedatum*“, zu dem das Gerät zurückgegeben werden soll und ein „*tatsaechlichesRueckgabedatum*“, das mit Hilfe der Methode „*gibZurueck( )*“ eingetragen wird, wenn das Gerät tatsächlich wieder abgegeben wird. Aufgerufen wird diese Methode von „*gibGeraetZurueck( )*“ der Control-Klasse.

Wenn ein Gerät ausgeliehen werden soll, muss erst einmal ein verfügbares Gerät gefunden werden. Hierzu dient die Methode „*ermittleFreiesGeraet( )*“ in der Klasse „*Geraetetyp*“, die die zugehörigen Geräte abfragt, ob sie gerade verliehen oder verfügbar sind. Hierzu verfügt die Klasse „*Geraet*“ über die Methode „*istVerfuegbar( )*“.

11.5 Verfügbarkeit ermitteln

Die meisten Methoden in dem Beispiel sind relativ einfach. Zumeist werden Objekte angelegt, miteinander verbunden und Attributwerte eingetragen. Etwas komplexer ist das Vorgehen zur Ermittlung der Verfügbarkeit in der Methode „*ermittleVerfuegbarkeitFuerZeitraum( )*“. Wenn man eine Reservierung anlegen möchte, muss im betreffenden Zeitraum ein Gerät des gewünschten Typs frei sein. Ansonsten muss das System es ablehnen, eine Reservierung zu erstellen. Auf Grundlage der vorgestellten Klassenstruktur wird folgendermaßen vorgegangen:

1. Es werden die Ausleihe-Objekte der ausleihbaren Geräte des gewünschten Typs untersucht:

   Wie viele Geräte sind momentan verfügbar oder werden zum Anfang des Reservierungszeitraums zurückgegeben sein?
   Hierzu dienen die Methoden „*istVerfuegbar( )*“ und „*ermittleAbWannVerfuegbar( )*“ der Klasse „Geraet“.

2. Es werden die Reservierungen des Typs überprüft:

   Wie viele Reservierungen, für die noch keine Ausleihen vorliegen, überschneiden sich mit dem gewünschten Reservierungszeitraum? Diese Zahl muss von der oben ermittelten Zahl noch abgezogen werden.

Möchte man ein Gerät ohne Reservierung ausleihen, muss dieselbe Prüfung durchgeführt werden, ansonsten könnte sich die Verleihdauer mit einer vorliegenden Reservierung überschneiden.

Man könnte sich auch vorstellen, die Verfügbarkeitsprüfung zu vereinfachen, indem man die bisher vorgestellten Strukturen und Funktionalitäten verändert. Hierfür gibt es beispielsweise folgende Alternativen:

1. Man könnte zu jeder Ausleihe eine Reservierung anlegen, auch wenn man das Gerät direkt mitnimmt. Dann könnte man die Verfügbarkeitsprüfung ausschließlich mit Hilfe der Reservierungen durchführen, ohne noch auf die Ausleihen zugreifen zu müssen.

Nachteil: Redundanz. Die Daten in Reservierungs- und Ausleihe-Objekt sind dieselben.

2. Man könnte die Reservierungen auch direkt einem konkreten Gerät zuordnen. Dann muss man bei der Verfügbarkeitsprüfung nur prüfen, ob man irgendein Gerät findet, das zu dem gewünschten Zeitraum frei ist.

   Nachteil: Soll jemand ein anderes Gerät desselben Typs erhalten (z. B. weil das andere kaputt ist und noch genug Geräte verfügbar sind), muss man die Zuordnung ändern.

3. Man könnte die Klassen Reservierung und Ausleihe auch zu einer einzigen Klasse zusammenfassen.

   Nachteil: Die Klasse wird komplexer und hat mehrere Zuständigkeiten, nämlich die Reservierung von Gerätetypen und die Ausleihe von einzelnen Geräten. Dies versucht man zu vermeiden, damit die Struktur übersichtlich bleibt und Änderungen möglichst einfach durchzuführen sind.

Oftmals gibt es wie hier mehrere Lösungsmöglichkeiten für eine Aufgabenstellung. Dann muss man sich unter Abwägung der Vor- und Nachteile für eine Alternative entscheiden.

11.6 Überblick über die Entity-Klassen

In Abbildung 130 sind noch einmal alle Entity-Klassen mit ihren Assoziationen im Überblick dargestellt.

Eine Erweiterung findet sich noch darin: Die Klasse „*Mitarbeiter*“ hat eine zusätzliche Methode „*zeigeNutzung( )*“, die einen Überblick über die aktuellen und früheren Reservierungen und Ausleihen eines Mitarbeiters liefert und damit den Use-Case „Gerätenutzung anzeigen“ und seine Spezialisierungen unterstützt. Wie andere benötigt auch diese Methode eine Reihe von getter-Methoden in verschiedenen Klassen, die im Analysediagramm aus Übersichtsgründen weggelassen werden. Noch nicht berücksichtigt wurde außerdem der Use-Case „Statistik erstellen“. Je nach gewünschten Statistiken benötigt man hierfür entsprechende Methoden für Gerätetypen, Räume u. ä.

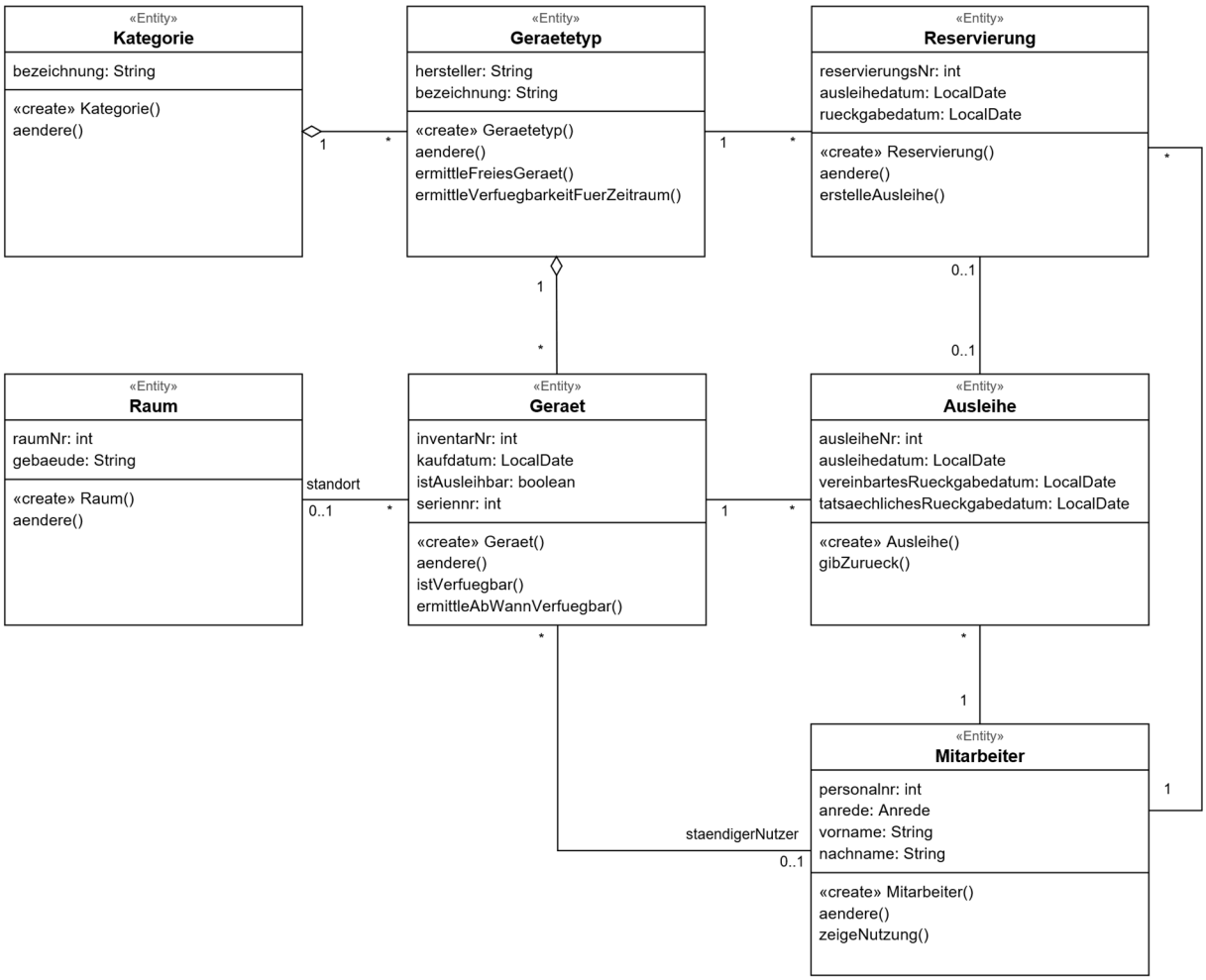


Abbildung 130: Gesamtdiagramm der Entity-Klassen

11.7 Paketdiagramm

Bei einer großen Zahl von Klassen ist es meist übersichtlicher, das Klassendiagramm in mehrere Pakete (Packages) aufzuteilen. Auch in Java werden ja Packages zur Strukturierung der Klassen verwendet. Das Klassendiagramm aus Abbildung 130 ist mit insgesamt sieben Entity-Klassen nicht so groß, dass man es unbedingt aufteilen müsste. Dennoch wurde es beispielhaft getan um das Paketdiagramm in Abbildung 131 zu erstellen.

Auf oberster Ebene gibt es die drei Pakete „*geraeteverwaltung*“, „*raumverwaltung*“ und „*mitarbeiterverwaltung*“. Das Paket „*geraeteverwaltung*“ ist noch einmal in die zwei Pakete „*geraetehierarchie*“ und „*geraeteverleih*“ unterteilt.

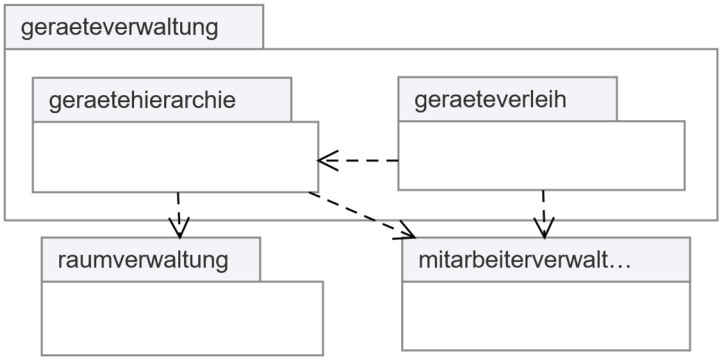


Abbildung 131: Paketstruktur in UML

Abbildung 132 zeigt die entsprechende Package-Struktur in Eclipse. In einer Java-Klasse wird das Paket, zu der die Klasse gehört, folgendermaßen angegeben.

```
package geraeteverwaltung.geraetehierarchie;
```

Bei den gestrichelten Pfeilen in Abbildung 131 handelt es sich um Abhängigkeitsbeziehungen. So ist beispielsweise das Paket „*geraeteverleih*“ abhängig von den Paketen „*geraetehierarchie*“ und „*mitarbeiterverwaltung*“. Eine Abhängigkeit zwischen zwei Paketen entsteht dadurch, dass ein Paket Klassen aus einem anderen Paket nutzt, referenziert oder deren Methoden aufruft. Dies hat insbesondere zur Folge, dass z. B. Änderungen im Paket „*geraetehierarchie*“ unter Umständen auch Änderungen im Paket „*geraeteverleih*“ erforderlich machen. In die andere Richtung gibt es keine Abhängigkeit, d. h. Änderungen im Paket „*geraeteverleih*“ sind ohne Auswirkungen auf andere Pakete.

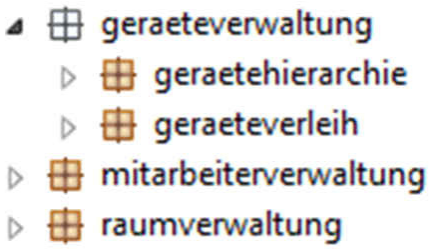


Abbildung 132: Java-Package-Struktur in Eclipse

Um ein System wartungs- und änderungsfreundlich zu gestalten, strebt man eine möglichst lose Kopplung zwischen unterschiedlichen Paketen an. Dies bedeutet, dass jedes Paket möglichst wenige Abhängigkeiten von anderen Paketen hat. Insbesondere versucht man bidirektionale Abhängigkeiten zu vermeiden. Das bedeutet, dass man sich genau überlegen muss, wie man die Klassen aufteilt, um zwischen den Paketen nur Abhängigkeiten in jeweils eine Richtung zu haben. Dies wird in Kapitel 14.2 im Zusammenhang mit der Systemarchitektur ausführlicher diskutiert.

Die Details eines Pakets können jeweils in einem eigenen Klassendiagramm dargestellt werden. Man kann aber auch die zu einem Paket gehörenden Klassen in das Paketsymbol einzeichnen, wie dies in Abbildung 133 für das Paket „*geraetehierarchie*“ getan wurde. Werden referenzierte Klassen aus anderen Paketen nicht ebenfalls innerhalb ihres Paketsymbols dargestellt, so wird der Paketname dem Klassennamen vorangestellt. Streng nach UML werden

Paket- und Klassennamen eigentlich durch zwei Doppelpunkte voneinander getrennt („raumverwaltung::Raum“), doch ist auch die in Java verwendete Trennung durch einen Punkt häufig zu sehen („raumverwaltung.Raum“).

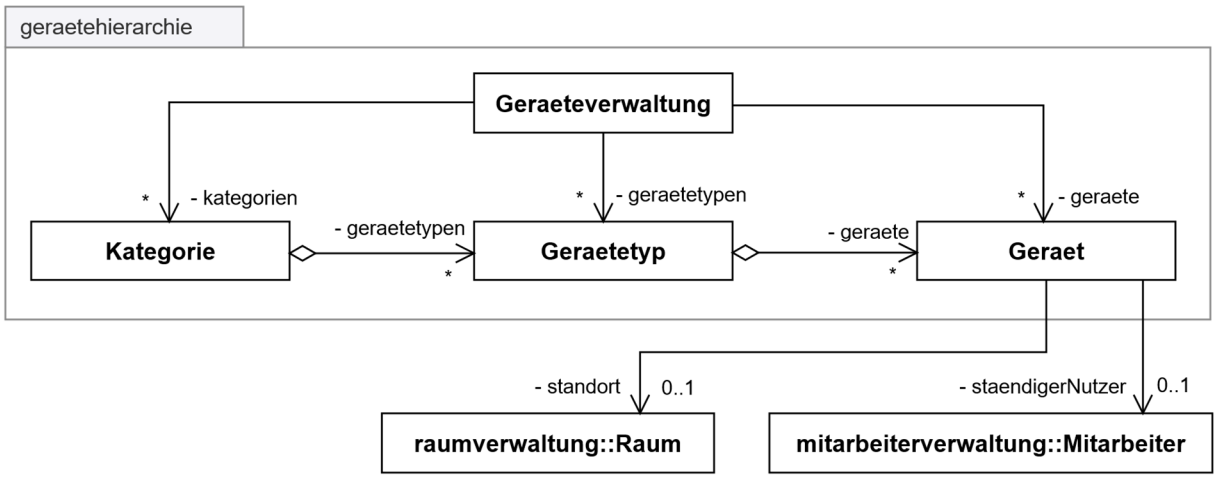


Abbildung 133: Die Klassen des Paketes „geraetehierarchie“ mit verbundenen Klassen aus anderen Paketen

12 Zustandsdiagramme

Zustandsdiagramme eignen sich zur Modellierung von Systemen, die auf Benutzereingaben oder andere externe Ereignisse warten und darauf reagieren. Beispiele hierfür sind Automaten oder grafische Benutzungsoberflächen. Man nennt solche Systeme auch reaktive Systeme. Man kann damit auch den Lebenszyklus von Objekten darstellen. Beispielsweise durchläuft eine Bestellung im Laufe ihres Lebens im Unternehmen verschiedene Zustände. Zunächst ist sie neu eingegangen, dann vielleicht in Bearbeitung, anschließend erledigt und später vom Kunden beglichen. Zustandsdiagramme werden nur dort eingesetzt, wo dies für das Verständnis hilfreich ist, oder wo das Verhalten eines Systems damit sinnvoll spezifiziert werden kann.

12.1 Zustandsdiagramm der Klasse Gerät

Als Beispiel wird die Klasse „*Geraet*" aus der Fallstudie betrachtet (Abbildung 134). Für ein neues Gerät muss zunächst ein Objekt dieser Klasse angelegt werden. Möglicherweise muss ein neues Gerät außerdem zunächst geprüft und freigegeben werden. Außerdem können Geräte kaputt gehen, dann müssen sie in Reparatur gegeben werden. Auch dies sollte im System erfasst werden, da ein Gerät, das sich in Reparatur befindet, nicht verliehen werden kann. Die genannten Informationen über ein Gerät können im Use-Case „Gerät bearbeiten" erfasst werden.

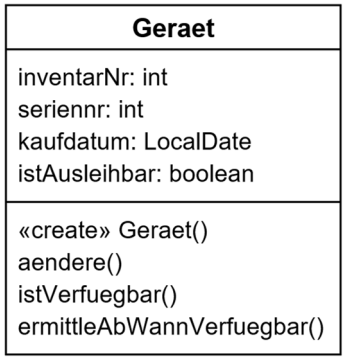


Abbildung 134: Klasse Gerät

Ein Gerät kann sich also im Laufe der Zeit in unterschiedlichen Zuständen befinden, z. B. im Zustand „neu", „bereit", oder „in Reparatur". Es sind gewisse Regeln einzuhalten, die sich auf die Zustände beziehen, z. B. darf ein defektes Gerät oder ein Gerät in Reparatur nicht verliehen werden. Ebenso darf ein neues Gerät nicht ohne Prüfung zum Verleih bereitgestellt werden.

Die möglichen Zustände und erlaubten Zustandsübergänge werden in einem Zustandsdiagramm dargestellt. Abbildung 135 zeigt das Zustandsdiagramm der Klasse „Geraet".

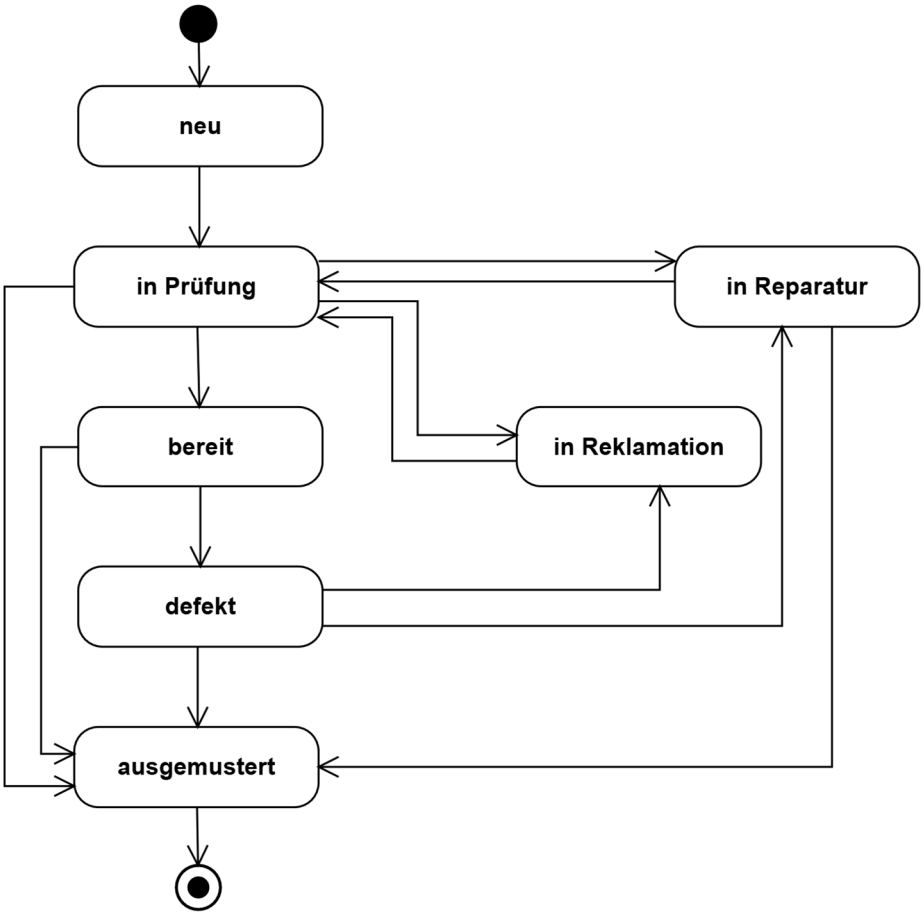


Abbildung 135: Zustandsdiagramm der Klasse Gerät

Durch den als ausgefüllten Kreis dargestellten Startknoten wird der Zustand „neu" als Startzustand definiert. Ein neu angelegtes Gerät befindet sich also zuerst in diesem Zustand. Wenn es sich darin befindet, gibt es nur eine Möglichkeit: Das Gerät muss geprüft werden. Dann befindet es sich im Zustand „in Prüfung". Hier gibt es nun mehrere mögliche Folgezustände, z. B. kann sich das Gerät – wenn es nicht in Ordnung ist – anschließend in Reparatur oder in Reklamation befinden, oder es kann ausgemustert sein (auch wenn das bei einem neuen Gerät eher ungewöhnlich wäre). Schließlich gibt es noch die Möglichkeit, dass das Gerät in Ordnung ist und daher in den Zustand „bereit" versetzt wird.

Die weiteren möglichen Zustandsübergänge lassen sich anhand des Diagramms gut nachvollziehen. Wichtig ist auch, dass es zwischen vielen Zuständen keine direkten Zustandsübergänge gibt. So kann man ein defektes Gerät nicht einfach wieder in den Zustand „bereit" versetzen. Stattdessen muss es erst in Reparatur oder in Reklamation gegeben werden und anschließend in Prüfung genommen werden. Vom Zustand „ausgemustert" gibt es lediglich einen Zustandsübergang zum Endknoten, d. h. hier ist der Lebenszyklus des Geräts beendet.

Zustände dauern in der Regel eine gewisse Zeit an. Zu jedem Zeitpunkt befindet sich ein Objekt in genau einem Zustand. Das Objekt wartet in dieser Zeit auf ein Ereignis. Tritt ein für das Objekt relevantes Ereignis ein, so reagiert es auf dieses Ereignis, führt irgendwelche Aktionen aus und wechselt gegebenenfalls in einen anderen Zustand.

Der Zustand eines Objektes wird durch seine aktuellen Attributwerte und Beziehungen definiert. Dabei werden nicht alle Attributwerte betrachtet, sondern nur die, die für das Verhalten relevant sind. So ist z. B. ein Attribut „*inventarNr*" nicht relevant für das Verhalten, Ein mögliches Attribut „*istDefekt*" hingegen schon. Es muss auf jeden Fall zu jedem Zeitpunkt eindeutig bestimmbar sein, in welchem Zustand sich ein Objekt momentan befindet.

Ein Zustandsdiagramm beschreibt einen sogenannten Zustandsautomaten (engl. „finite state machine"). Ein solcher Zustandsautomat kann – wie im hier betrachteten Beispiel – zu einer einzelnen Klasse gehören. Es ist aber auch möglich, dass er sich auf eine aus mehreren Klassen gebildete Struktur bezieht, auf eine komplette Komponente, oder auf ein ganzes System.

Ein Zustandsautomat besteht aus Zuständen und Zustandsübergängen, sogenannten Transitionen. Eine Transition ist atomar, sie benötigt keine Zeit (bzw. ist die Zeit so kurz, dass sie vernachlässigt werden kann), und sie kann nicht unterbrochen werden. Es sind nur die im Zustandsdiagramm eingezeichneten Transitionen erlaubt. Ausgelöst wird eine Transition immer von einem Ereignis. Welche Transition durchgeführt wird („feuert"), hängt vom aktuellen Zustand, dem Ereignis und gegebenenfalls von zutreffenden Bedingungen ab.

Es gibt verschiedene Arten von Ereignissen, z. B. eine eintreffende Nachricht (Methodenaufruf), ein eintreffendes Signal, das Ablaufen einer Zeitspanne, das Erreichen eines bestimmten Zeitpunkts, oder eine Bedingung, die wahr wird. Zudem kann man zusätzliche Bedingungen aufstellen, die überprüft werden, wenn ein Ereignis eintritt.

Übrigens betrifft die Forderung, dass ein Objekt oder ein System zu jedem Zeitpunkt immer nur genau einen Zustand haben darf, immer nur einen Zustandsautomaten. Ein System kann auch mehrere, voneinander unabhängige Zustandsautomaten haben. Beispielsweise könnte ein Schneeräumfahrzeug einerseits einen Zustandsautomaten für seine Schaufel haben (Zustände „oben" und „unten"), und andererseits einen für seine Fortbewegung (Zustände „vorwärts", „Stillstand", „rückwärts").

12.2 Implementierung von Zustandsdiagrammen

Es gibt keine Vorschriften, wie ein Zustandsdiagramm zu implementieren ist. Die Implementierung sollte folgendes sicherstellen:

- Die Abbildung der Zustände.
- Man muss herausfinden können, in welchem Zustand sich das Objekt befindet.
- Man muss die erlaubten Zustandsübergänge herbeiführen können.
- Es sollte verhindert werden, dass nicht modellierte Zustandsübergänge erfolgen.

Eine einfache Möglichkeit besteht darin, in der jeweiligen Klasse ein Attribut zu definieren, das als Zustandsvariable dient. In Abbildung 136 wurde hierfür das Attribut „*zustand*" eingeführt. Als Datentyp hierfür wurde der Aufzählungstyp „*Geraetezustand*" definiert, der die Zustände des Zustandsdiagramms aus Abbildung 135 als Werte enthält. Ein Aufzählungstyp ist besser als z. B. Integer-Werte. Zum einen ist im Code immer explizit zu sehen, um welchen

Zustand es sich handelt. Zum anderen können nicht versehentlich ungültige Werte eingetragen werden, da ein Aufzählungstyp nur die definierten Werte annehmen kann. Als Default-Wert erhält „*zustand*" den Wert „*NEU*". Damit erhält jedes neu angelegte Gerät zunächst diesen Anfangszustand, wie er im Zustandsdiagramm festgelegt ist.

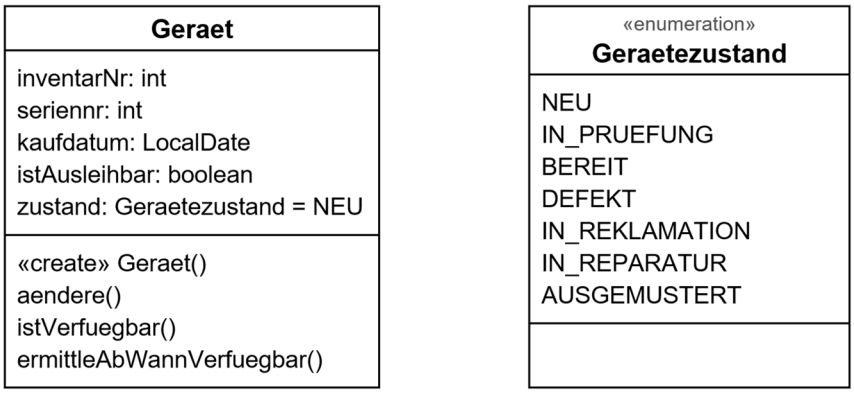


Abbildung 136: Implementierung mit Hilfe einer Zustandsvariablen

Bei einer Zustandsänderung wird der Wert des Attributs „*zustand*" geändert. Hierfür werden ein oder mehrere Methoden benötigt. In Abbildung 137 wurden mit „*nimmInPruefung( )*", „*stelleBereit( )*" usw. insgesamt sechs Methoden eingeführt, die jeweils einen der möglichen Zustände herstellen. Lediglich für den Zustand „*NEU*" gibt es keine Methode, da das Objekt sich nach dem Anlegen automatisch in diesem Zustand befindet, und es gemäß Zustandsdiagramm anschließend nicht mehr möglich ist, dorthin zurückzukehren. Die ebenfalls neu eingefügte Methode „*hatNochGarantie( )*" dient nicht zur Änderung eines Zustandes, sie wird später zur Überprüfung einer Bedingung benötigt.

Abbildung 137: Methoden für Zustandsübergänge

```
public void nimmInPruefung(){
	zustand = Geraetezustand.IN_PRUEFUNG;
}

public void stelleBereit(){
	zustand = Geraetezustand.BEREIT;
}
```

Listing 36: Die Methoden ändern den Wert der Zustandsvariablen

Listing 36 zeigt beispielhaft die zwei Methoden „*nimmInPruefung( )*“ und „*stelleBereit( )*“. Mit diesen und den weiteren Methoden lassen sich nun alle möglichen Zustandsänderungen durchführen. Allerdings wird noch nicht berücksichtigt, dass nur bestimmte Zustandsübergänge erlaubt sind, nämlich nur die im Zustandsdiagramm eingezeichneten.

Also muss in jeder Methode zunächst überprüft werden, ob es vom derzeitigen Zustand erlaubt ist, in den gewünschten Zustand zu wechseln. Betrachtet man beispielsweise den Zustand „*in Prüfung*“, so gibt es in Abbildung 135 drei eingehende Transitionen, die von den Zuständen „*in Prüfung*“, „*neu*“, „*in Reparatur*“ und „*in Reklamation*“ ausgehen. Also muss die Methode „*nimmInPruefung( )*“ zuerst feststellen, ob sich das Gerät in einem dieser möglichen Vorgängerzustände befindet. Nur dann darf es den Wert der Zustandsvariablen auf „*IN_PRUEFUNG*“ ändern. Ebenso darf die Methode „*stelleBereit*“ den Zustandsübergang nur durchführen, wenn sich das Gerät in Prüfung befindet. Die geänderten Methoden finden sich in Listing 37. Vergleichbare Überprüfungen müssen auch die anderen zustandsverändernden Methoden durchführen.

```
public void nimmInPruefung(){
	switch(zustand){
		case NEU:
		case IN_REPARATUR:
		case IN_REKLAMATION:
			zustand = Geraetezustand.IN_PRUEFUNG;
			break;
		default:
}

public void stelleBereit(){
	if(zustand==Geraetezustand.IN_PRUEFUNG){
		zustand = Geraetezustand.BEREIT;
	}
}
```

Listing 37: Nun werden die nicht zulässigen Zustandsübergänge verhindert

12.3 Angaben an Transitionen

Abbildung 138 zeigt mögliche Angaben an den Transitionen eines Zustandsdiagramms:

- Ereignis

  Das Ereignis löst den Zustandsübergang aus. Oft wird das Ereignis aber auch weggelassen, wenn es z. B. nur aus dem Aufruf der Methode zum Wechsel des Zustands besteht.

- Bedingung

  In manchen Fällen muss noch eine weitere Bedingung erfüllt sein, damit der Zustandsübergang erfolgt. Wenn das jeweilige Ereignis eintritt, wird überprüft, ob die Bedingung erfüllt ist. Nur dann wird der Zustandsübergang durchgeführt. Die Bedingung wird in eckigen Klammern angegeben.

- Aktion

  Hier wird eine Aktion angegeben, die während des Zustandswechsels durchgeführt wird, z. B. die ausgeführte Methode. Vor dem Namen der Aktion wird jeweils ein Schrägstrich eingetragen, um sie von dem Ereignis zu trennen.

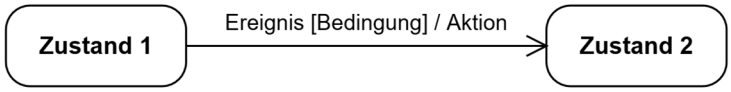


Abbildung 138: Angaben an Transitionen

In Abbildung 139 wurde das Zustandsdiagramm des Geräts aus Abbildung 135 um die Angaben der Bedingungen und Aktionen ergänzt. Als Aktionen sind hier jeweils die ausgeführten Methoden angegeben. Die meisten Ereignisse wurden weggelassen. Das Ereignis besteht meist nur darin, dass die betreffende Methode aufgerufen wird.

Lediglich bei den aus dem Zustand „*bereit*" ausgehenden Transitionen sind die Ereignisse „*Defekt aufgetreten*" bzw. „*Gerät veraltet*" angegeben. Hierbei handelt es sich um externe Ereignisse, die z. B. vom Geräteverwalter wahrgenommen werden, worauf er den Aufruf der betreffenden Methoden auslöst, indem er z. B. eine entsprechende Schalfläche der Benutzungsoberfläche drückt. Aus Sicht des Systems würde man ein solches Ereignis genauer als „*'Defekt aufgetreten' gedrückt*" bezeichnen. Die genaue Logik der Benutzungsoberfläche ist Gegenstand des Entwurfs. Aus fachlicher Sicht ist nur wichtig, dass das genannte Ereignis letztlich dazu führt, dass die Methode „*meldeDefekt( )*" ausgeführt wird.

Selbstverständlich könnte man auch für die anderen Zustandsübergänge Ereignisse anlegen, wie z. B. „*'Nimm in Prüfung' gedrückt*". Da diese jedoch keine nennenswerten Zusatzinformationen bringen, wurden nur die erwähnenswerten externen Ereignisse eingetragen. Letztlich muss der Modellierer entscheiden, welche Informationen wichtig sind, um die dargestellte Logik zu verstehen.

Vor dem Namen der Aktion ist auch dann ein Schrägstrich eingetragen, wenn davor weder ein Ereignis noch eine Bedingung angegeben ist. Hierdurch wird sofort deutlich, dass es sich um die Aktion handelt.

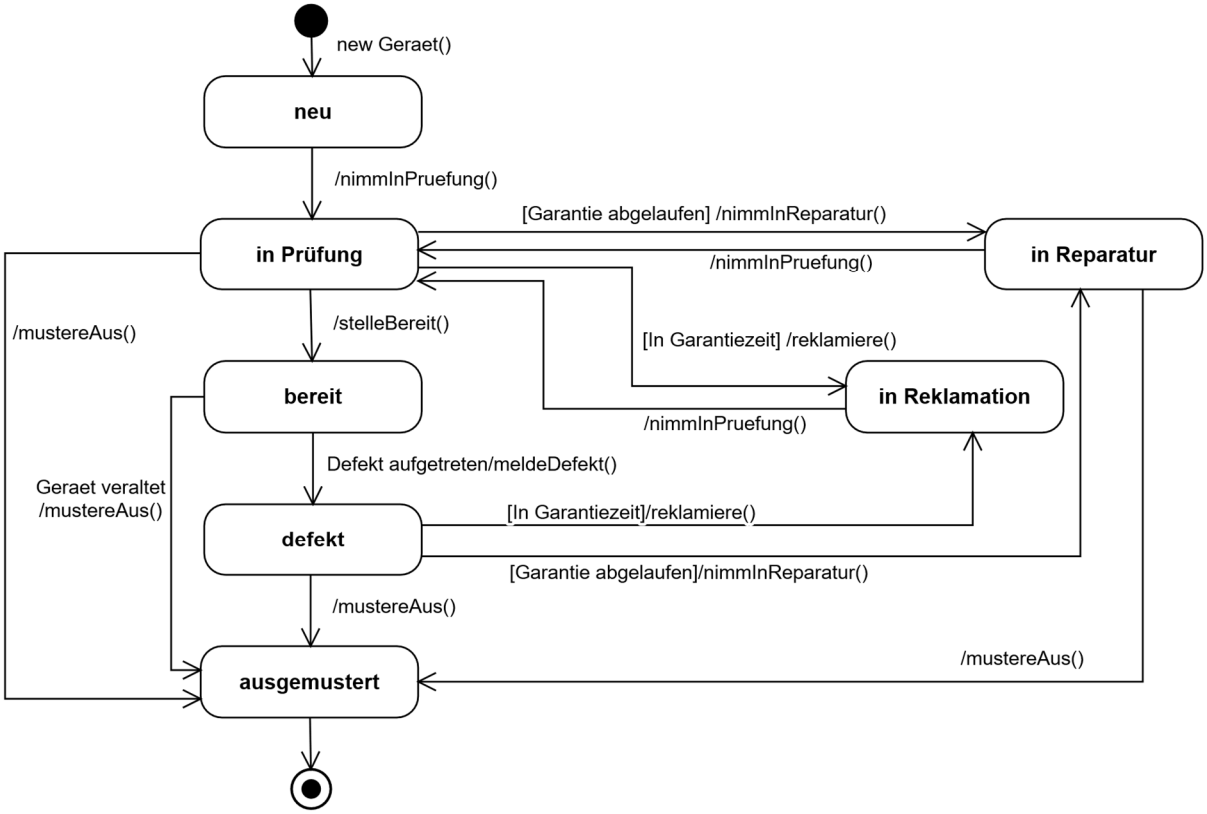


Abbildung 139: Zustandsdiagramm mit Beschriftungen

An den Transitionen zu den Zuständen „*in Reklamation*“ und „*in Reparatur*“ sind noch zusätzliche Bedingungen eingetragen. Befindet sich das Gerät beispielsweise im Zustand „*defekt*“ so darf das Ereignis „*Aufruf der Methode ‚reklamiere( )‘*“ nur dann dazu führen, dass in den Zustand „*in Reklamation*“ gewechselt wird, wenn sich das Gerät noch innerhalb der Garantiezeit befindet. Die Methode „*reklamiere( )*“ muss also nicht nur überprüfen, ob sich das Gerät momentan in einem zulässigen Vorgängerzustand von „*in Reklamation*“ befindet, sondern auch, ob die Bedingung „*in Garantiezeit*“ erfüllt ist. Nur wenn beides der Fall ist, darf sie den Wert der Zustandsvariablen auf „*IN_REKLAMATION*“ ändern.

12.4 Weitere Implementierungsmöglichkeiten

Im vorangehenden Abschnitt wurden die verschiedenen Zustände komplett durch eine einzige Zustandsvariable abgebildet. Das muss aber nicht unbedingt so sein. Es ist nur erforderlich, dass ein Objekt (oder auch ein ganzes System) sich zu jedem Zeitpunkt immer genau in einem Zustand befindet, und dass sich dieser Zustand eindeutig feststellen lässt. Die hierzu erforderlichen Informationen können aber durchaus über verschiedene Attribute und Objekte verteilt sein.

In Abbildung 140 hat die Klasse „*Geraet*“ keine explizite Zustandsvariable mehr. Stattdessen gibt es boolesche Attribute, die z. B. angeben, ob das Gerät defekt oder ausgemustert ist. Ob ein Objekt gerade in Prüfung oder in Reparatur ist, kann man herausfinden, indem man die zugeordneten Pruefung- bzw. Reparaturauftrag-Objekte auswertet. Gibt es eine zugeordnete Prüfung, die noch nicht abgeschlossen ist, so befindet sich das Gerät gerade in Prüfung. Eine

Prüfung ist abgeschlossen, wenn im Attribut „*endeDatum*“ ein Wert eingetragen ist. Die Methode „*istAbgeschlossen( )*“ überprüft dies.

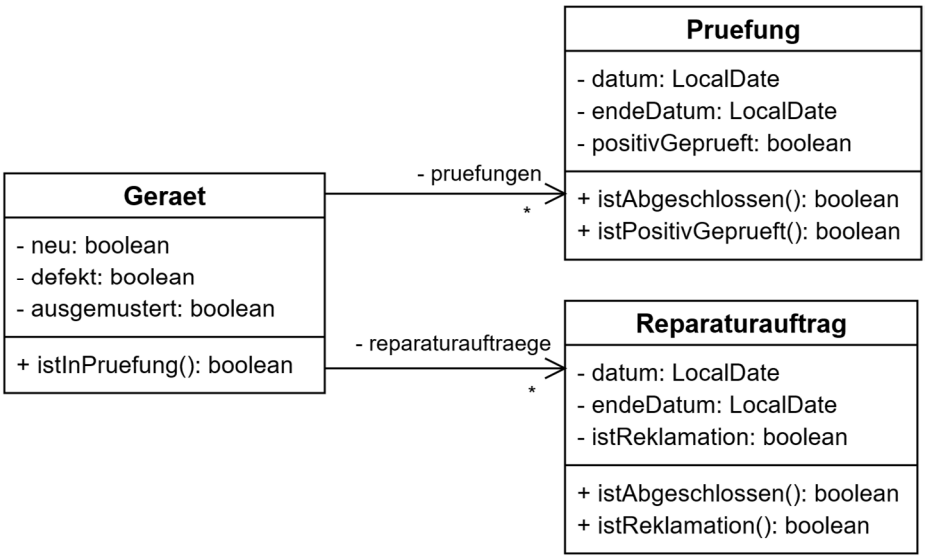


Abbildung 140: Ermittlung von Zuständen aus Kombinationen von Attributwerten und zugeordneten Objekten

Als Beispiel ist in Listing 38 die Implementierung der Methode „*istInPruefung( )*“ angegeben. Die for-Schleife geht solange alle zugeordneten Prüfungen durch, bis sie eine findet, die noch nicht abgeschlossen ist. Dann ist der am Ende zurückgegebene Wert der Variablen „*istInPruefung*“ „*true*“. Wird keine noch nicht abgeschlossene Prüfung gefunden, so behält er den Wert „*false*“. Andere Zustände können auf vergleichbare Weise ermittelt werden.

```
import java.util.ArrayList;

public class Geraet {
	private boolean neu;
	private boolean defekt;
	private boolean ausgemustert;
	private ArrayList<Pruefung> pruefungen = new ArrayList<Pruefung>();
	private ArrayList<Reparaturauftrag> reparaturauftraege =
				new ArrayList<Reparaturauftrag>();

	public boolean istInPruefung(){
		boolean istInPruefung = false;
		for (Pruefung p:pruefungen){
			if(!p.istAbgeschlossen()){
				istInPruefung = true;
				break;
			}
		}
		return istInPruefung;
	}

	// .. weitere Methoden zur Änderung von Attributwerten und
	// zur Ermittlung von Zuständen
}
```

Listing 38: Implementierung der Klasse Geraet aus Abbildung 140 (Ausschnitt)

Diese Implementierung hat gegenüber der Implementierung mit einer Zustandsvariablen den Nachteil, dass sie wesentlich schwieriger zu überblicken und fehlerträchtiger ist. Natürlich wird man die Variante aus Abbildung 140 nicht deshalb wählen, weil man ausschließlich das Zustandsdiagramm implementieren möchte. Klassen wie „*Pruefung*“ und „*Reparaturauftrag*“ würde man erstellen, um andere Aufgaben zu erledigen, wie z. B. das Erfassen von Prüfungsergebnissen, die Abwicklung der Reparatur usw. Unter Umständen entsteht dann jedoch schwer durchschaubares Verhalten, und unter ungünstigen Umständen können eigentlich unzulässige Kombinationen von Attributwerten entstehen, die zu schwer auffindbaren Fehlern führen. Hier kann ein Zustandsdiagramm ein nützliches Hilfsmittel sein. Gegebenenfalls kann es sich lohnen, zusätzlich eine Zustandsvariable einzuführen um sicherzustellen, dass die durch das Zustandsdiagramm festgelegten Regeln immer eingehalten werden.

In Kapitel 12.2 wurde zum Herbeiführen jedes einzelnen Zustandes eine eigene Methode genutzt. Natürlich können auch die Zustandswechsel ganz anders auf verschiedene Methoden aufgeteilt werden. So könnte man z. B. nur eine einzige Methode „*zustandWechseln( )*“ vorsehen, die das auslösende Ereignis als Parameter übergeben bekommt, den gegenwärtigen Zustand sowie einzuhaltende Bedingungen überprüft und den Wert der Zustandsvariablen entsprechend ändert.

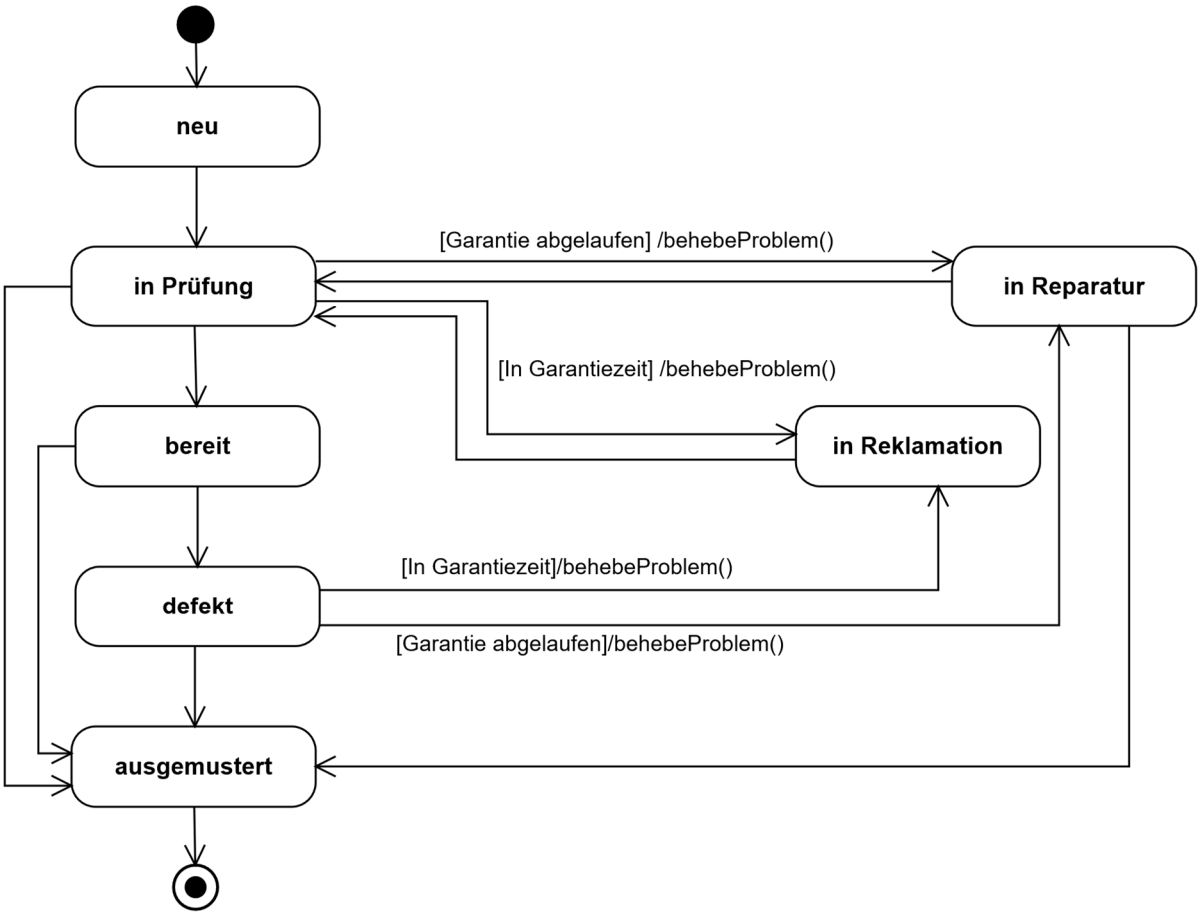


Abbildung 141: Methoden zum Herbeiführen von Zustandsübergängen

In Abbildung 141 wurden die Methoden „*reklamiere( )*“ und „*nimmInReparatur( )*“ zu einer Methode „*behebeProblem( )*“ zusammengefasst. Diese Methode überprüft zunächst, ob das Gerät entweder im Zustand „defekt“ oder „in Prüfung“ ist. Ist dies der Fall, wird als nächstes noch die Bedingung überprüft. Hat das Gerät noch Garantie, so wird es in den Zustand „*in Reklamation*“ überführt, ansonsten in den Zustand „in Reparatur“. Die anderen Transitionen wurden der Übersicht halber nicht beschriftet. An ihnen hat sich gegenüber Abbildung 139 nichts geändert.

```
public void behebeProblem(){
	switch(zustand){
		case DEFEKT:
		case IN_PRUEFUNG:
			if(hatNochGarantie){
				zustand = Geraetezustand.IN_REKLAMATION;
			} else {
				zustand = Geraetezustand.IN_REPARATUR;
			}
			break;
		default:
	}
}
```

Listing 39: Implementierung der Methode „behebeProblem( )“

Listing 39 zeigt die Implementierung dieser Methode in Java. Zur Überprüfung des Zustandes wurde hier eine switch-Anweisung verwendet. Da hinter „*case DEFEKT:*“ keine break-Anweisung folgt, werden auch beim Zustand „*defekt*“ die auf die nächste case-Anweisung folgenden Zeilen ausgeführt.

12.5 Zustandsdiagramme – weitere Konzepte

Im gerade betrachteten Beispiel gehen vom Zustand „defekt“ zwei Transitionen mit demselben auslösenden Ereignis (Aufruf der Methode „*behebeProblem( )*“) und derselben Aktion aus. Sie unterscheiden sich lediglich durch die Bedingung. Dies kann man alternativ wie in Abbildung 142 mit Hilfe einer Verzweigung darstellen. Das Ereignis startet zunächst die Transition, die aus dem Zustand „defekt“ herausführt. Anschließend folgt eine Verzweigung, und je nach erfüllter Bedingung wird als nächstes entweder der Zustand „in Reklamation“ oder „in Reparatur“ erreicht. Bei der anderen eingezeichneten Verzweigung passiert genau dasselbe, ausgehend vom Zustand „in Prüfung“.

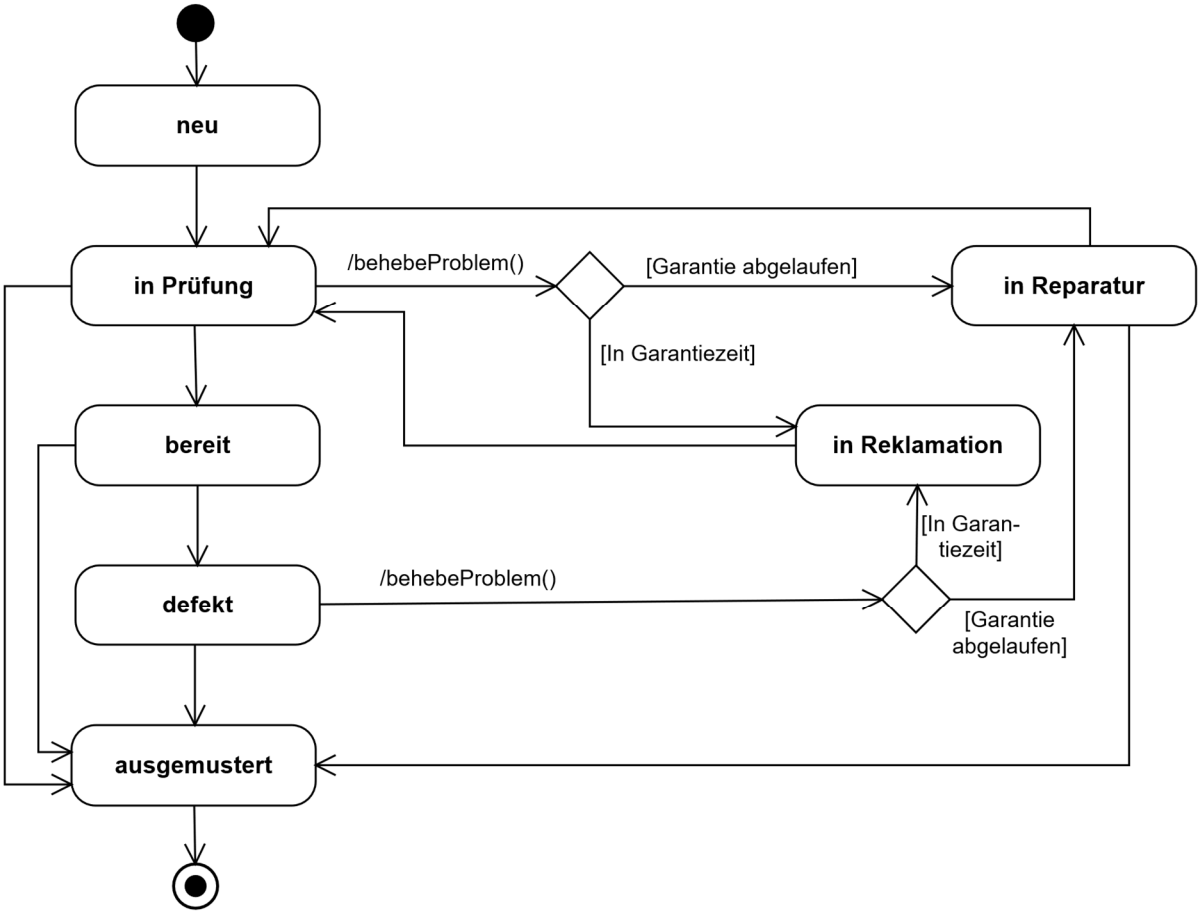


Abbildung 142: Verzweigungen

Bei einer solchen Verzweigung muss man natürlich sicherstellen, dass in jedem Fall immer genau eine Bedingung zutrifft. Würden einmal beide Bedingungen zutreffen, wüsste man nicht, welches der Folgezustand ist. Definitionsgemäß kann das Objekt ja immer nur einen Zustand haben. Das System wäre „nicht-deterministisch", d. h. es lässt sich nicht aufgrund der Zustände, Bedingungen und Ereignisse voraussagen, in welchen Zustand es sich begibt.

Dies gilt nicht nur bei einer Verzweigung, sondern auch, wenn mehrere Transitionen mit demselben Ereignis aus einem Zustand herausgehen. Abbildung 143 zeigt ein Beispiel für einen nicht-deterministischen Automaten. Wenn das System im Zustand 1 ist und das Ereignis 1 eintrifft, so muss aufgrund der Bedingung entschieden werden, zu welchem Folgezustand man gelangt. Für Werte der Variablen X zwischen 5 und 10 sind jedoch beide Bedingungen wahr, so dass der Folgezustand nicht eindeutig bestimmt werden kann.

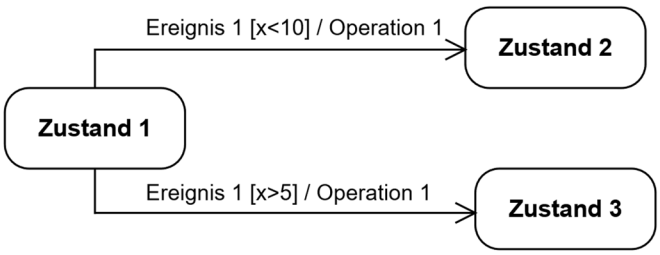


Abbildung 143: Nicht-deterministischer Automat

Ebenso muss man vermeiden, dass einmal keine Bedingung zutrifft. Lauteten die Bedingungen etwa „*x>20*“ und „*x<10*“, so wäre für Werte zwischen 10 und 20 gar kein Folgezustand bestimmt.

Bislang wurden Aktionen an den Transitionen angegeben. Zustände sind eher passiv. Das System wartet auf ein Ereignis. Daraufhin wird eine Aktion ausgeführt und ggf. ein anderer Zustand eingenommen. Anschließend wird wieder gewartet.

Es kann allerdings auch sein, dass eine Aktion während eines Zustandes durchgeführt wird. In Abbildung 144 geht es um die Zustände eines Industrie-Ofens. Nach dem Einschalten befindet er sich in der Aufwärmphase. Da das Aufwärmen längere Zeit dauert, kann man nicht direkt eine Transition vom Startknoten zu „bereit“ ziehen. In der Zeit zwischen dem Einschalten und dem Zustand „bereit“ befindet sich der Ofen eben in der Aufwärmphase. Wenn nun in dieser Aufwärmphase die ganze Zeit über etwas getan wird, wird dies innerhalb des Zustandssymbols notiert. In Abbildung 144 werden während der Aufwärmphase die Werte protokolliert. Der Aktionsbezeichnung wird „*do/*“ vorangestellt, da es sich um etwas handelt, das während des Zustandes getan wird.

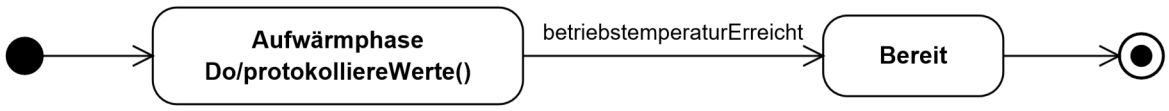


Abbildung 144: Aktion innerhalb eines Zustandes

Schließlich kann man auch noch angeben, dass bestimmte Aktionen beim Erreichen oder beim Verlassen eines Zustandes durchgeführt werden. So wird in Abbildung 145 jedes Mal, wenn „Zustand“ erreicht wird, „*operation1*“ ausgeführt. Wenn „Zustand“ verlassen wird, wird immer „*operation3*“ ausgeführt. Dies kann man wie im oberen Teil von Abbildung 145 darstellen, indem man die betreffende Aktion an jede eingehende bzw. ausgehende Transition schreibt. Alternativ kann man diese Aktionen wie im unteren Teil der Abbildung mit dem Präfix „*entry/*“ bzw. „*exit/*“ in das Zustandssymbol eintragen.

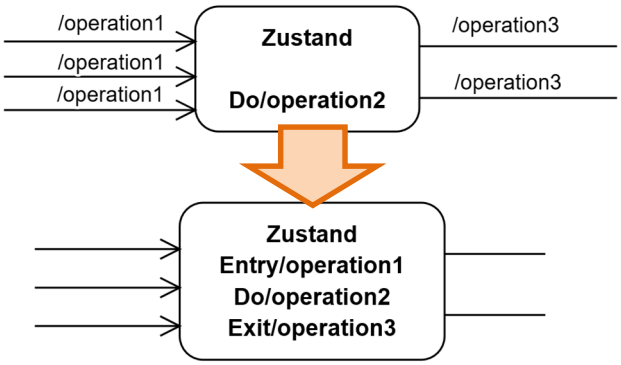


Abbildung 145: Aktionen beim Eintreten und Verlassen eines Zustandes

13 Oberflächenentwurf

In diesem Kapitel geht es um grundlegende Überlegungen, welche Funktionalitäten und Möglichkeiten die Benutzungsoberfläche des Systems haben muss, um das umzusetzen, was in Form von Use-Case-Diagrammen und Analyse-Klassendiagrammen dokumentiert wurde.

Es gibt sehr viele verschiedene Möglichkeiten, wie eine Oberfläche für ein und dasselbe System aussehen kann. So macht es beispielsweise einen Unterschied, ob es sich um ein Desktop-Programm, eine Web-Anwendung oder eine Smartphone-App handelt. Zudem können unterschiedliche GUI-Technologien und –Frameworks zum Einsatz kommen und verschiedene Stile, Navigationskonzepte usw. angewandt werden. Die konkrete Ausgestaltung im Sinne eines besonders ansprechenden Designs und möglichst guter Ergonomie wird hier nicht thematisiert. Diese Themen werden in anderen Lehrveranstaltungen behandelt.

Die im Folgenden dargestellten Vorgehensweisen und Entwürfe sind daher nicht als Vorgaben zu sehen, die man unverändert übernehmen sollte. Sie dienen lediglich dazu, grundlegende Fragen und Entscheidungen zu erläutern, mit denen erreicht wird, dass die Funktionalität der Oberfläche dem Analyse-Modell entspricht. Hierzu wird als Beispiel eine sehr einfache, funktionale Oberfläche einer webbasierten-Anwendung verwendet. Die Überlegungen, die beim Entwurf dieser einfachen Oberfläche angestellt werden, sind auch für ganz andere Oberflächen gültig. Schließlich muss es in jedem Fall möglich sein, die benötigten Daten einzugeben und die erforderlichen Funktionen auszuführen, ganz unabhängig von der Darstellung.

Um frühzeitig einen Eindruck von dem Oberflächenentwurf zu gewinnen, kann man ihn zunächst in Form von Wireframes („Drahtgitter“) skizzieren. Diese Skizzen – oftmals zunächst als Handskizzen ausgeführt – eignen sich für eine frühzeitige Abstimmung mit den Benutzern. Eine Skizze der Benutzungsoberfläche ist für viele Anwender wesentlich greifbarer als textuelle Beschreibungen oder Modelle.

Um die Logik der Aufrufstruktur darzustellen, d. h. aus welchen Dialogen heraus sich welche anderen Dialoge aufrufen lassen, kann man Zustandsdiagramme einsetzen. Ein Zustand besteht dann darin, dass ein bestimmter Dialog angezeigt wird. Mögliche Ereignisse sind Benutzerinteraktionen. Z. B. kann das Anklicken einer Schaltfläche dazu führen, dass eine Transition ausgeführt wird, die dann zum nächsten Zustand, d. h. dem Anzeigen eines anderen Dialogs, führt.

Oftmals werden beim Entwurf der Oberfläche und dem gedanklichen Durchspielen von Bedienungsabläufen noch vorhandene Probleme, Widersprüche und Lücken aufgedeckt. Auf diese Weise findet eine Validierung der Inhalte der Analyse-Modelle statt. Bei aufgedeckten Problemen müssen die Analyse-Modelle entsprechend angepasst werden.

Die fertig gestellten Oberflächen-Skizzen und Zustandsdiagramme dienen schließlich als Vorlage für die spätere Implementierung der Oberfläche. Beim anschließenden Test der

Oberfläche kann man die Transitionen des Zustandsdiagramms durchgehen und somit systematisch überprüfen, ob die Ablauflogik stimmt.

Welche Anforderungen lassen sich aus den Analyse-Modellen (Use-Case-Diagramm, Klassendiagramm usw.) für die Benutzungsoberfläche ableiten? Kurz gesagt muss Folgendes sichergestellt werden:

- Man muss alle benötigten Funktionalitäten und Inhalte erreichen können.
- Es müssen alle Use-Cases durchführbar sein.
- Dabei müssen die benötigten Objekte mit den zu pflegenden Attributen und Beziehungen angelegt und bearbeitet werden können.
- Es müssen alle Geschäftsregeln berücksichtigt werden. Z. B. darf man nur gültige Werte eingeben können.

13.1 Hauptnavigation

Um die einzelnen Funktionen ausführen zu können, muss man sie zunächst einmal finden. Hierfür braucht man eine geeignete Navigationsstruktur. Für die Hauptnavigation kann man eine hierarchische Struktur verwenden. Sie kann beispielsweise folgendermaßen gegliedert werden:

- Oberste Ebene:
  Hier können thematische Oberbegriffe für inhaltlich zusammengehörige Funktionen verwendet werden. Z. B. können alle Funktionen im Zusammenhang mit Kunden zusammengefasst werden.

- Zweite Ebene:
  Hier kann man aufrufbare Funktionen anordnen. Diese können den Use-Cases entsprechen. Es kann aber auch sinnvoll sein, Use-Cases aufzuteilen oder zusammenzufassen, wenn dies für die Bearbeitung sinnvoll ist.

- Dritte Ebene:
  Auf der dritten Ebene können sich die Einzel-Dialoge für die aufgerufenen Funktionen befinden.

Je nach Umfang und Komplexität kann die Navigationshierarchie auch mehr oder weniger Hierarchie-Ebenen enthalten.

In oberen Bereich von Abbildung 146 findet man eine solche hierarchische Navigationsstruktur für das Fallbeispiel der Geräteverwaltung. Das Hauptfenster enthält eine Menüleiste mit Einträgen für die verschiedenen Funktionsgruppen „Geräte“, „Ausleihe“, „Mitarbeiter“, „Räume“ und „Statistik“. Klickt man auf einen dieser Einträge, so wird in der Leiste darunter das betreffende Untermenü angezeigt. In Abbildung 146 ist „Geräte“ ausgewählt. Standardmäßig wird dann gleich die Ansicht des ersten Unterpunkts „Geräteübersicht“ dargestellt, die als Einstiegspunkt für viele Funktionen dient, z. B. zur Ansicht und zum Bearbeiten eines ausgewählten Geräts.

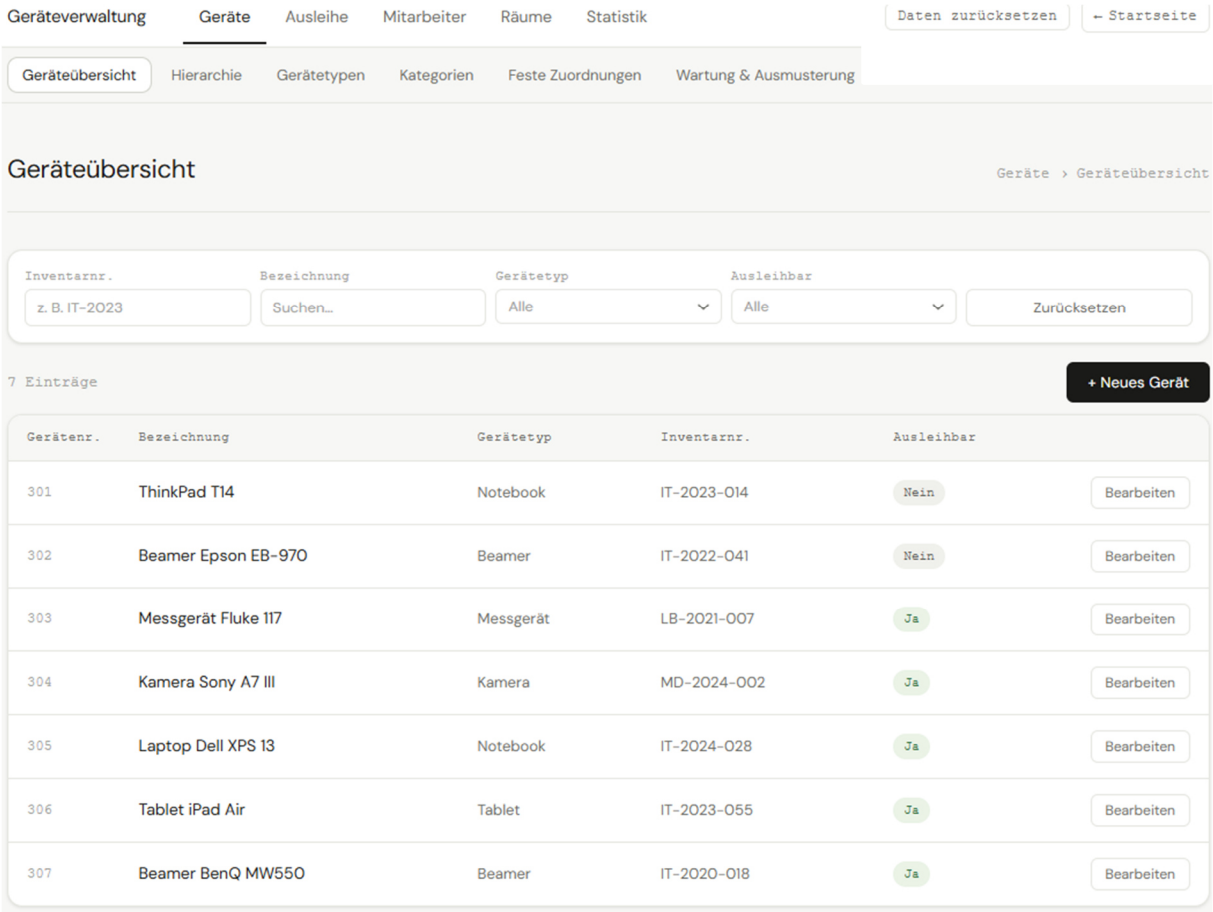


Abbildung 146: Hauptfenster mit Navigationsstruktur (zwei obere Leisten)

Abbildung 147: Untermenü für „Ausleihe“

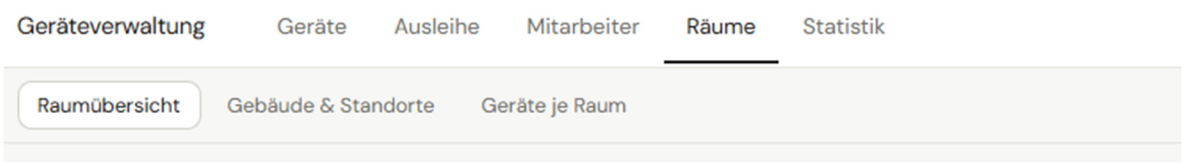


Abbildung 148: Untermenü für „Räume“

Abbildung 147 und Abbildung 148 enthalten die Untermenüs für die Haupteinträge „Ausleihe“ und „Räume“.

Beim Vergleich mit dem Use-Case-Diagramm (Abbildung 21) wird deutlich, dass die Einträge der zweiten Ebene ungefähr den Use-Cases entsprechen. Allerdings wurden manche Use-Cases noch einmal aufgeteilt. So gibt es im Use-Case-Diagramm nur einen Use-Case zum Anlegen und Bearbeiten von Geräten. In der Navigationsstruktur ist das Anlegen eines neuen Geräts ein eigener Punkt. Das Bearbeiten eines vorhandenen Geräts wird aus der Geräteübersicht heraus gestartet. Hierzu dienen die „Bearbeiten“-Buttons für die einzelnen Geräte. Hierdurch spart man sich eine separate Suche nach dem gewünschten Gerät.

Während bei den Use-Cases zunächst nur die grundlegende Funktionalität im Vordergrund steht, liegt der Schwerpunkt des Oberflächenentwurfs auf der Bedienbarkeit, weshalb es zwangsläufig zu Änderungen gegenüber den Ablaufbeschreibungen der Use-Cases kommt.

Im Use-Case-Diagramm sind verschiedene Akteure eingezeichnet, die jeweils andere Use-Cases durchführen können. In der Oberfläche sollte dies dazu führen, dass jeder Benutzer aufgrund seiner Berechtigungen unterschiedliche Funktionen angeboten bekommt. So müssten für einen gewöhnlichen Mitarbeiter unter „Verleih" nur die Möglichkeiten zum Reservieren eines Geräts und zum Bearbeiten seiner eigenen Reservierungen zur Verfügung stehen. Die anderen Einträge müssten ausgeblendet werden. Die Navigationsstruktur sollte also rollenspezifisch gefiltert werden. Oder aber es werden gleich komplett verschiedene Oberflächen für verschiedene Rollen erstellt.

Eine hierarchische Navigationsstruktur kann auch ganz anders dargestellt werden, z. B. mit Hilfe von Drop-Down-Menüs oder Form eines aufklappbaren Navigationsbaums.

13.2 Anlegen eines Objekts

Der Dialog zum Anlegen eines einzelnen Objekts enthält Eingabeelemente für die vom Benutzer zu pflegenden Attributwerte. Im Dialog „Mitarbeiter anlegen" in Abbildung 149 findet man für jedes Attribut der links daneben abgebildeten Klasse „Mitarbeiter" ein entsprechendes Eingabeelement. Bei „Vorname" und „Nachname" handelt es sich um einfache Textfelder, in die man die jeweiligen Strings einträgt.

Je nach Datentyp können auch andere Eingabeelemente sinnvoll sein. So handelt es beim Datentyp „Anrede" um einen – hier nicht im Klassendiagramm dargestellten – Aufzählungstyp, der die Werte „Frau" und „Herr" etc. annehmen kann. Der Benutzer muss dann zwischen diesen Werten wählen, was im Beispiel mit Hilfe einer Drop-Down-Liste realisiert wurde.

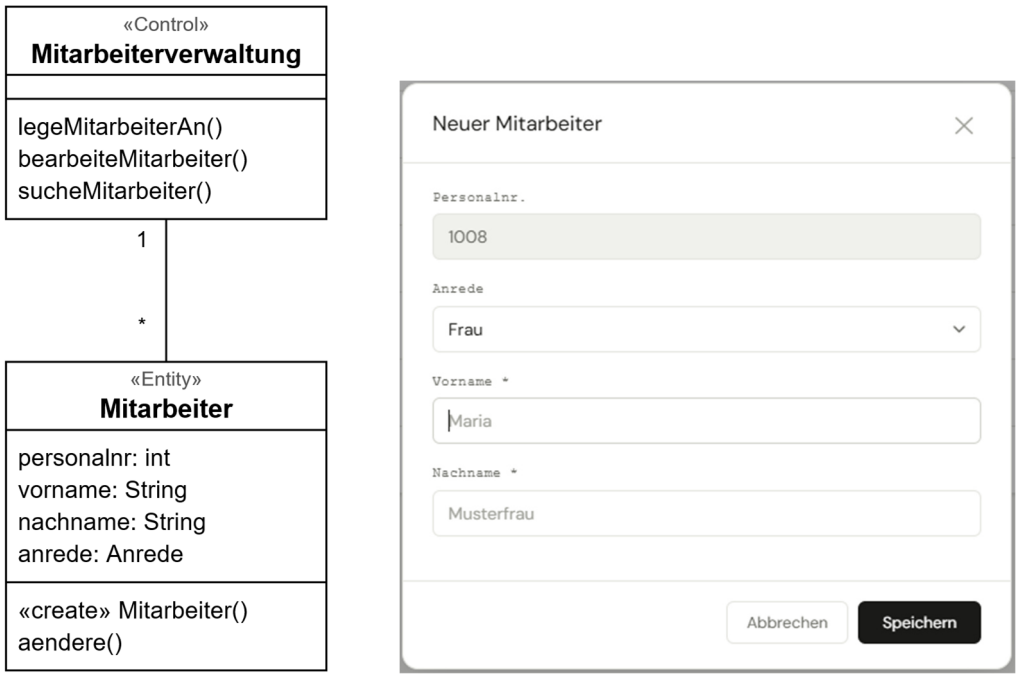


Abbildung 149: Neues Objekt anlegen

Nicht für jedes Attribut soll einfach ein Wert eingegeben werden. In der Regel sollte jeder Mitarbeiter eine andere Personalnummer erhalten. Dies erreicht man am einfachsten dadurch, dass die Personalnummer automatisch vergeben wird. Sie sollte im Dialog angezeigt werden, aber nicht geändert werden können. Das betreffende Textfeld ist daher in Abbildung 149 grau hinterlegt.

Bei der Frage nach der Vergabe und der Eindeutigkeit von Personalnummern handelt es sich um ein Beispiel für eine fachliche Frage, die mit dem Auftraggeber und Fachexperten zu klären und auch zu dokumentieren ist, z. B. in einem ergänzenden Text zum Klassendiagramm.

Es ist auch möglich, dass eine Klasse über Attribute verfügt, die für die Benutzer unwichtig sind oder beim Anlegen des Objekts noch keine Rolle spielen. Sie werden dann gar nicht in der Oberfläche angezeigt.

Die Entwicklung der Oberfläche kann nicht schematisch aus Use-Case-Beschreibungen und Klassendiagrammen abgeleitet werden. Sie erfordert vielmehr individuelle Überlegungen, welche Informationen die Benutzer jeweils in welcher Form angezeigt bekommen und welche Eingaben möglich sein sollen.

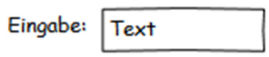
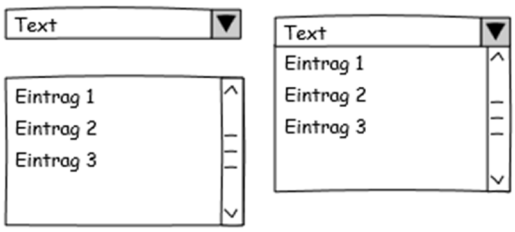
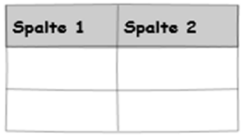
Inhalt	*Mögliche Steuerelemente*	*Darstellung*
Strings, Zahlen	Textfeld	
Aufzählungstypen	Drop Down-Liste, Listenfeld, Combobox	
Boolesche Werte	Checkbox, Radiobutton	
Zugeordnete Objekte mit mehreren Attributen	Tabelle	
Auslösen von Aktionen	Schaltfläche	
Und viele mehr …		

Tabelle 20: Beispiele für Steuerelemente

Die spätere Implementierung der Oberfläche soll dafür sorgen, dass das Drücken der Schaltfläche „Okay" zum Aufruf der Methode „*legeMitarbeiterAn( )*" des für die Mitarbeiter-Objekte zuständigen Objekts der Control-Klasse „*Mitarbeiterverwaltung*" führt. Diese ruft dann den Konstruktor der Klasse „*Mitarbeiter*" auf. Im Entwurf werden diese Methoden um entsprechende Parameter zur Weitergabe der eingegebenen Werte ergänzt. Hinzu kommen Eingabeüberprüfungen, die etwa dafür sorgen, dass verpflichtende Felder ausgefüllt sind, bevor ein Objekt angelegt wird. Spezielle Anforderungen, wie die automatische Personalnummernvergabe, müssen in Entwurf und Implementierung ebenfalls berücksichtigt werden.

Je nach Datentyp und möglichen Eingaben eignen sich unterschiedliche Steuerelemente. In Tabelle 20 sind für einige Inhalte typische Steuerelemente aufgeführt.

13.3 Finden eines Objekts

Damit man ein existierendes Objekt ansehen und bearbeiten kann, muss man es zunächst einmal finden. Hierfür gibt es verschiedene Möglichkeiten:

- Direkte Eingabe eines identifizierenden Attributs
  Dabei kann es sich z. B. um eine Personalnummer oder Rechnungsnummer handeln. Auf diese Weise lässt sich sehr schnell auf das gewünschte Objekt zugreifen, aber man muss die betreffende Nummer kennen.
- Auswahl aus einer Liste
  Dies eignet sich insbesondere bei einer überschaubaren Menge von Objekten. Wird die Liste zu lange, muss man zu lange scrollen um den gewünschten Eintrag zu finden. Hilfreich kann eine Einschränkung über Filter sein.
- Navigation über eine Hierarchie
  Beispielsweise kann man in einer Explorer-Baumstruktur eine Kategorie anklicken und die dazu gehörenden Unterkategorien ausklappen. Auf diese Weise kann man ggf. über mehrere Hierarchiestufen bis zum einzelnen Objekt-Eintrag navigieren.
- Suche
  Die Mitarbeiterliste in Abbildung 150 enthält ganz oben eine Suchleiste. Hier können für die einzelnen Attribute die gewünschten Werte eingegeben werden. Entsprechend werden in der Liste die zu den Suchwerten passenden Einträge angezeigt. Es brauchen nicht unbedingt die kompletten Werte eingetippt werden. Trägt man beispielsweise nur drei Buchstaben des Nachnamens ein, so werden diejenigen Mitarbeiter herausgesucht, deren Nachnamen diese Buchstabenkombination enthalten.

Die Handhabung großer Listen sollte erleichtert werden, z. B. indem nur jeweils zehn Einträge angezeigt werden und man zu den weiteren Einträgen blättern kann. Auch flexible Sortiermöglichkeiten können hilfreich sein.

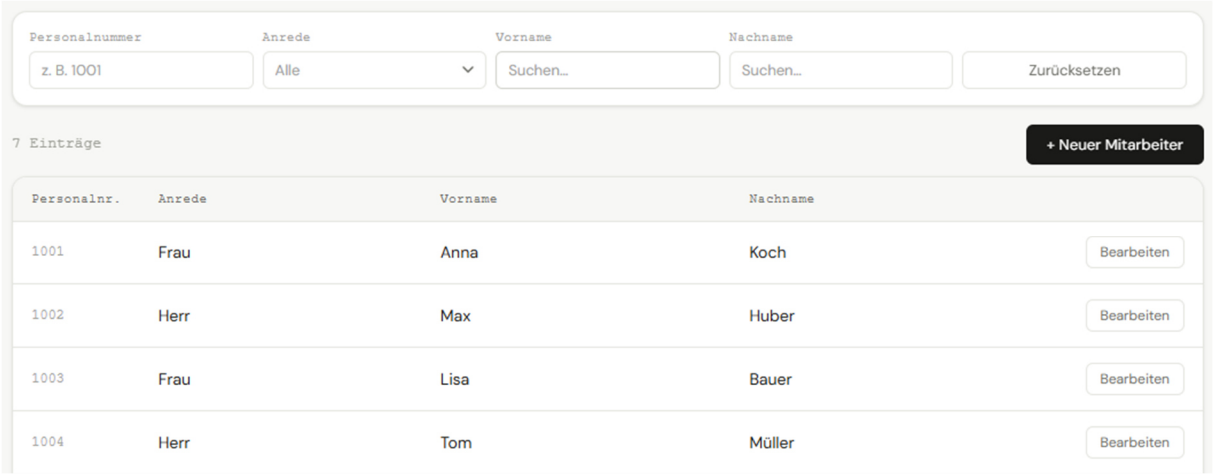


Abbildung 150: Suchleiste (oben in der Abbildung)

Ausgehend von der Liste der gefundenen Objekte sollte man ein gewünsches Objekt auswählen und bearbeiten können. Im vorliegenden Beispiel erfolgt dies über den „Bearbeiten"-Button der betreffenden Zeile. Eine andere Möglichkeit wäre es, über die rechte Maustaste ein Kontextmenü aufzurufen, das auch weitere Aktionen für das gewählte Objekt enthalten kann, wie z. B. Löschen oder Kopieren.

Im hier gezeigten Beispiel werden Funktionen wie das Anlegen und Bearbeiten von Objekten nicht direkt über die Hauptnavigation erreicht, sondern über die Übersichtsseite mit allen bzw. allen bei einer Suche gefundenen Objekte (vgl. Abbildung 146 und Abbildung 150). Die Entscheidung über die am Besten geeignete Platzierung solcher Funktionalitäten hängt mit einer möglichst guten Bedienbarkeit zusammen und soll hier nicht weiter thematisiert werden.

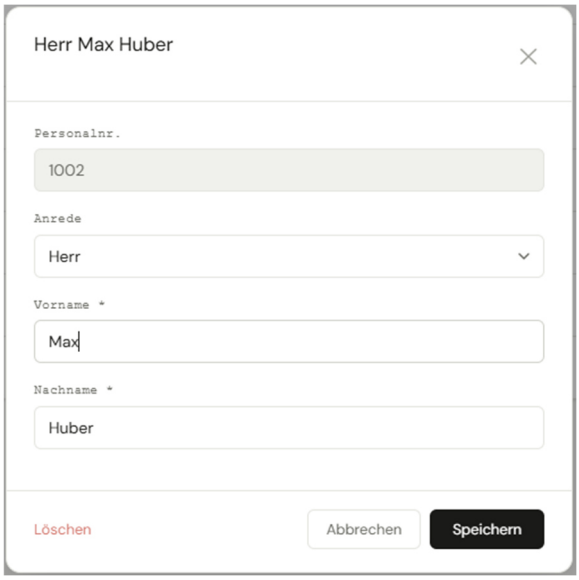


Abbildung 151: Objekt bearbeiten

13.4 Bearbeiten eines Objekts

Der Dialog zum Bearbeiten eines Objekts (Abbildung 151) sieht aus wie der Dialog zum Anlegen eines neuen Objekts (Abbildung 149), mit dem Unterschied, dass die Eingabeelemente bereits mit den Attributwerten des ausgewählten Objekts gefüllt sind. Sie können nun geändert werden. Mit der Schaltfläche „Speichern“ werden die geänderten Werte in das Objekt übernommen. Auch hier müssen ggf. wieder Eingabevalidierungen vorgenommen werden. Manchmal ist es auch nicht erlaubt, gewisse Werte noch einmal zu verändern. Dann werden sie nur angezeigt, können aber nicht bearbeitet werden.

13.5 Zuordnung von Objekten

Bislang wurde nur die Bearbeitung einzelner Objekte betrachtet. Meist stehen Objekte aber auch mit anderen Objekten in Beziehung. Die möglichen Beziehungen werden im Klassendiagramm als Assoziationen modelliert. Beim Anlegen oder Bearbeiten eines Objekts muss es daher auch möglich sein, ihm andere Objekte zuzuordnen.

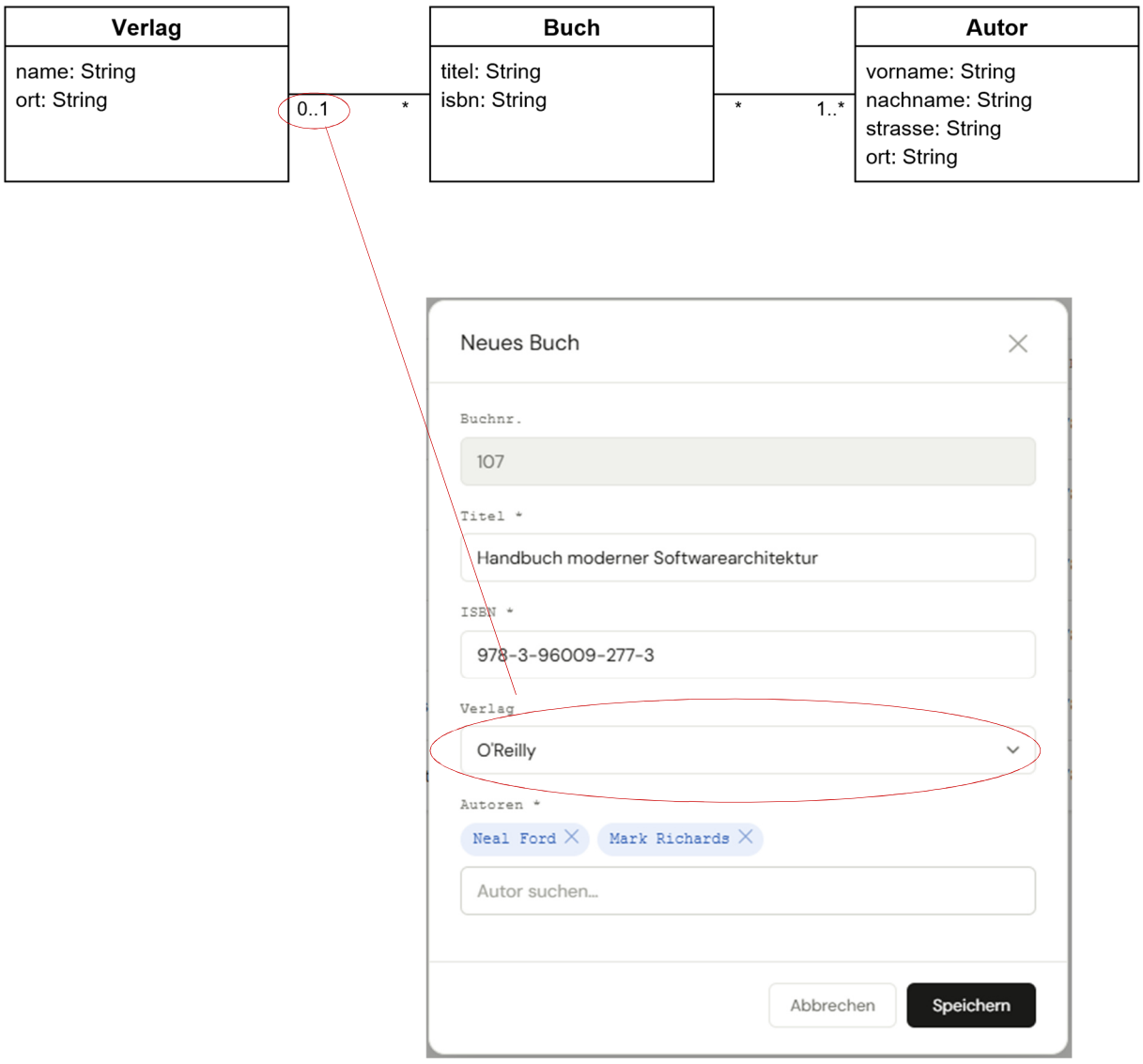


Abbildung 152: Zuordnung eines anderen Objekts – bei Multiplizität mit Obergrenze 1

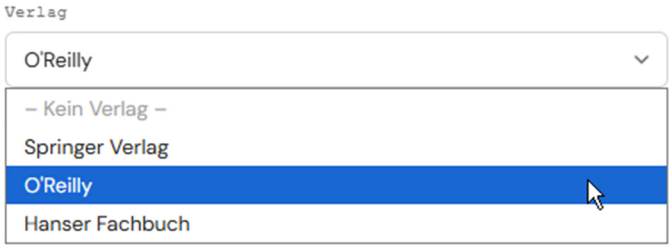


Abbildung 153: Auswahl des Verlags

In Abbildung 152 ist im Klassendiagramm modelliert, dass jedes Buch-Objekt mit einem Verlag-Objekt in Beziehung stehen kann. Es kann aber auch sein, dass ein Buch-Objekt kein Verlag-Objekt zugeordnet hat. Ein Buch kann beispielsweise auch im Selbstverlag erschienen sein.

Legt man ein Buch-Objekt an, sollte es daher möglich sein maximal ein Verlag-Objekt zuzuordnen. Der Dialog in Abbildung 152 enthält hierfür eine Drop-Down-Liste. In der Liste sind die Namen aller Verlagsobjekte enthalten, aus der der gewünschte Verlag ausgewählt wird (Abbildung 153). Die Liste muss auch den Eintrag „Kein Verlag“ oder einen leeren Eintrag enthalten, da die Untergrenze der Multiplizität null ist.

Eine Drop-Down-Liste ist nur dann sinnvoll, wenn die zu erwartende Zahl von Verlagen überschaubar ist. Durch eine Liste mit hunderten von Einträgen zu scrollen ist nicht besonders angenehm.

Muss man mit einer großen Zahl von Verlagen rechnen, sollte man eine andere Lösung wählen. Beispielsweise könnte man eine Schaltfläche bereitstellen, über die man einen separaten Dialog zur Auswahl eines Verlags erreicht. In diesem Dialog kann man dann beispielsweise durch die Verlagsliste blättern oder eine Suche durchführen. Nach Auswahl des gewünschten Verlags gelangt man dann wieder zurück zu „Buch anlegen“, wo der Name des gewählten Verlags angezeigt wird.

Neben einem Verlag können einem Buch gemäß dem Klassendiagramm ein oder mehrere Autoren zugeordnet sein. Da es mehr als einen Autor geben kann, wird eine Liste oder Tabelle benötigt, um die Autoren des Buchs in dem Dialog anzuzeigen. In dem Beispiel von Abbildung 154 werden sogenannte „Chips“ für die einzelnen Autoren verwendet. Mit „Autor suchen“ kann man weitere Autoren heraussuchen und hinzufügen (Abbildung 155). Mit dem „X“ in einem Chip lassen sich Autoren entfernen. Dabei wird das Autorenobjekt nicht gelöscht, sondern nur die Verbindung zwischen Buch und Autor getrennt.

Meist wird die Zahl der Autoren eines Buchs überschaubar sein. Ansonsten könnte man eine Tabelle mit Scrollbalken verwenden.

Zwar haben Autoren noch weitere Attribute als Vor- und Nachname, doch sollten im Sinne der Übersichtlichkeit nur die im Zusammenhang mit dem Buch benötigten Attribute angezeigt werden. Benötigt man die Adresse eines Autors, so kann man sich das Autorenobjekt ja gesondert ansehen.

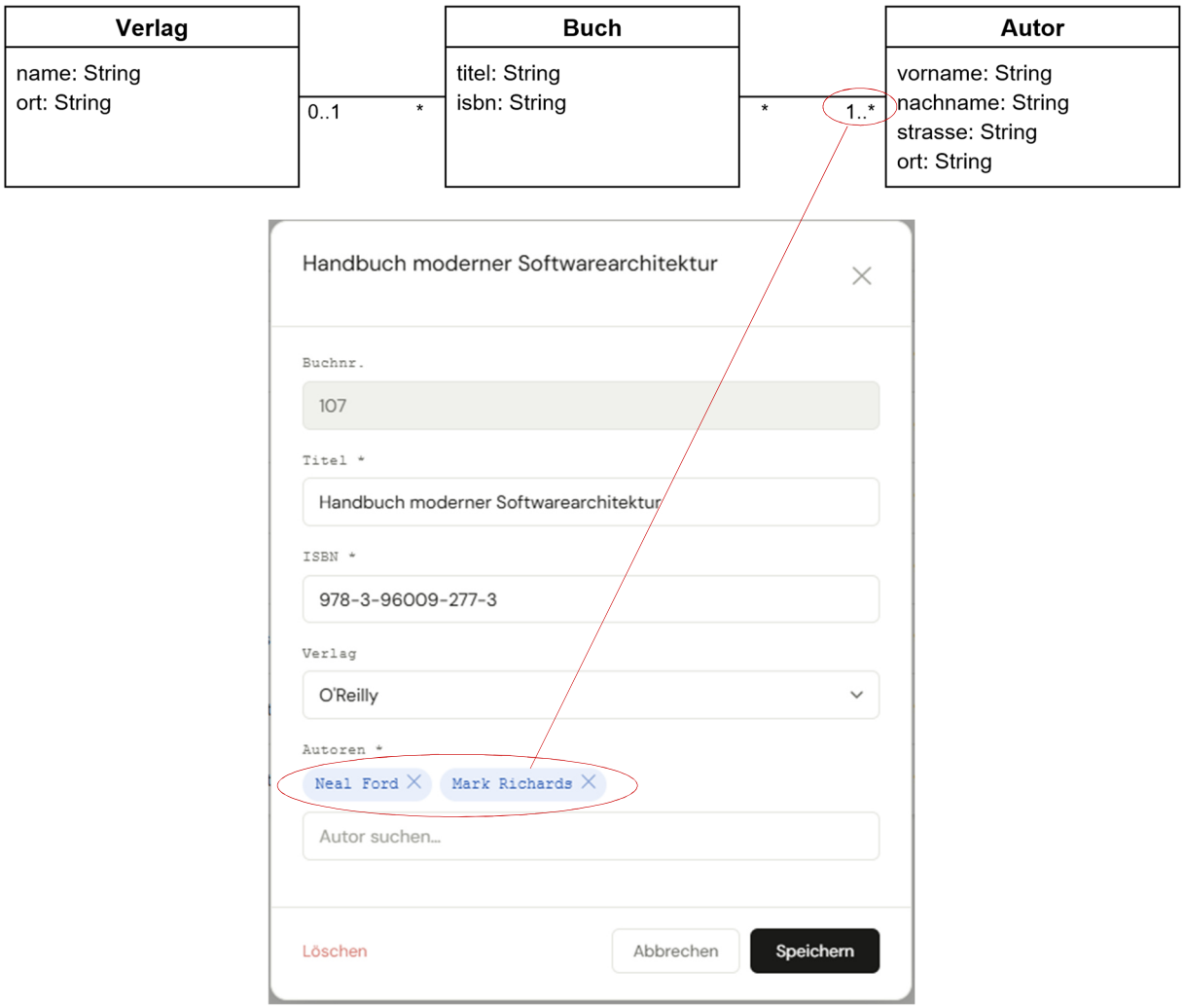


Abbildung 154: Zuordnung anderer Objekte – bei Multiplizität mit Obergrenze *

Abbildung 155: Suche und Auswahl von Autoren

Für jede Assoziation muss man überlegen, in welche Richtung man die betreffenden Objektzuordnungen in der Benutzungsoberfläche pflegen möchte. So kann man in dem Beispiel entweder im Buchdialog den Verlag zuordnen, oder im Verlagsdialog die Bücher. Welche dieser beiden Varianten man wählt, hängt von den zu unterstützenden Arbeitsabläufen ab. Die Use-Case-Beschreibungen können hierfür oftmals einen ersten Anhaltspunkt liefern.

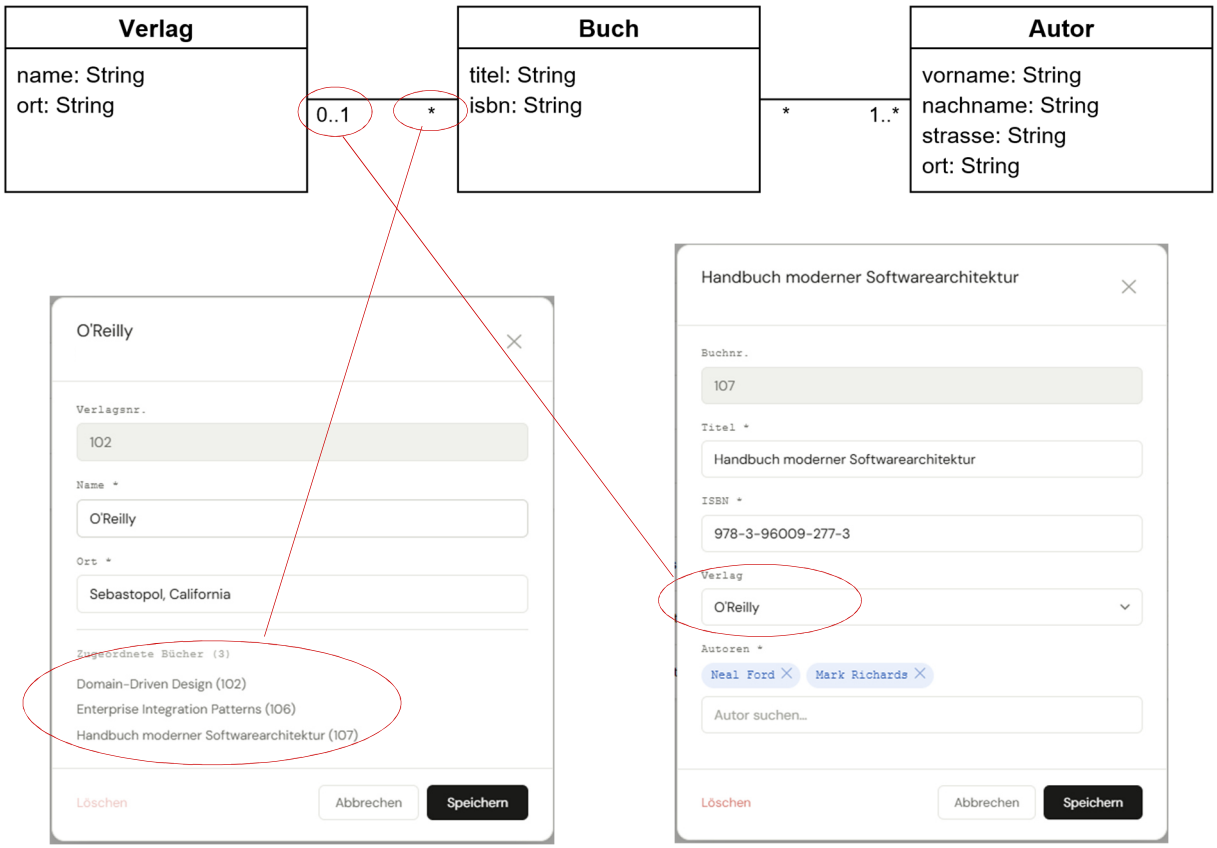


Abbildung 156: Navigierbarkeit auf der Oberfläche

Es ist selbstverständlich auch möglich, die Beziehungen in beide Richtungen anzusehen oder zu pflegen. So wurde in Abbildung 156 nicht nur im Buchdialog (rechts) eine Drop-Down-Liste für die Verlagsauswahl vorgesehen, sondern auch im Verlagsdialog (links) eine Liste der in dem Verlag erschienenen Bücher. In diesem Beispiel ist die Liste der Bücher nur lesbar. Die Verbindung zwischen Buch und Verlag kann somit nur über den Buch-Dialog hergestellt werden.

Prinzipiell wäre es aber auch möglich, im Verlags-Dialog Bücher zuzuordnen – ähnlich, wie die Autoren im Buch-Dialog hinzugefügt werden.

Können Zuordnungen auf beiden Seiten der Assoziation bearbeitet werden, so hat dies zur Folge, dass eine Änderung auf der einen Seite auch zu geänderten Einträgen im Dialog der anderen Seite führt. Man gewinnt auf diese Weise Flexibilität beim Pflegen der Beziehungen. Andererseits besteht die Gefahr, dass die verschiedenen Möglichkeiten von den Benutzern als verwirrend empfunden werden.

13.6 Umsetzung von Vererbungsstrukturen

Bei einer Generalisierungsbeziehung erben die Unterklassen die Eigenschaften der Oberklasse. Insbesondere verfügen die Objekte der Unterklasse zusätzlich über die in der Oberklasse definierten Attribute. Daher muss man in den Dialogen zum Anlegen und zum Bearbeiten eines Objekts der Unterklasse die Möglichkeit haben, auch die geerbten Attribute zu pflegen.

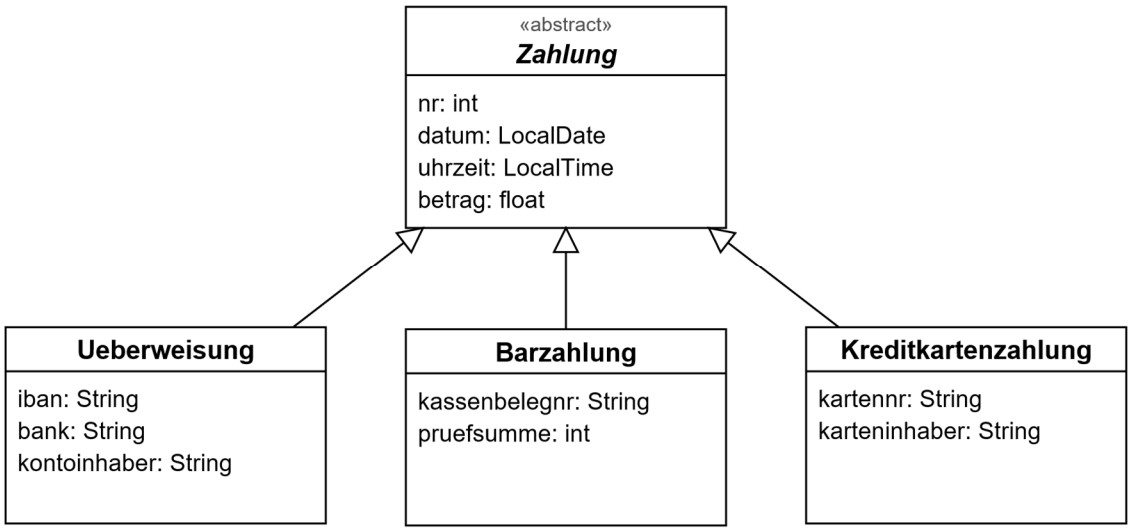


Abbildung 157: Klassendiagramm mit Vererbung

Als Beispiel wird das Klassendiagramm in Abbildung 157 betrachtet. Aufgrund der Vererbungsbeziehung müssen beispielsweis im Dialog zum Anlegen einer Barzahlung neben der Kassenbelegnummer und der Prüfsumme auch die Zahlungsnr, Datum und Uhrzeit sowie der Betrag eingegeben bzw. automatisch eingetragen werden.

Da die konkreten Unterklassen die Attribute von „Zahlung" erben, müssen die geerbten Zahlungsfelder in den drei Dialogen für Überweisungen, Barzahlungen und Kreditkartenzahlung vorhanden sein.

Anstatt drei individuelle Dialoge zu erstellen, kann man auch für alle Arten von Zahlungen einen einzigen, dynamisch anpassbaren Dialog erstellen. Darin sind zunächst die Felder aus der Oberklasse „Zahlung" enthalten. Zudem kann man die Art der Zahlung auswählen. Entsprechend passt sich der Dialog an, und es werden die Felder für die in der jeweiligen Unterklasse definierten Attribute eingeblendet.

So ist in Abbildung 158 links als Art der Zahlung „Barzahlung" ausgewählt, rechts „Überweisung". Entsprechend kann man im linken Dialog die Barzahlungsattribute „Kassenbelegnummer" und „Prüfsumme" eingeben, im rechten Dialog hingegen die Überweisungsattribute „IBAN", „Bank", und „Kontoinhaber".

In dem vorliegenden Beispiel wurden keine Methoden modelliert. Da Methoden ebenfalls vererbt werden, müssen die Dialoge der Unterklassen dann auch die entsprechenden Schaltflächen u. ä. für die gewünschten Aufrufe von Methoden aus den Oberklassen bereitstellen.

Da es sich bei den Objekten aller drei konkreten Klassen um Zahlungen handelt, kann es sinnvoll sein, sie an vielen Stellen einheitlich zu behandeln. So zeigt Abbildung 159 eine Liste von Zahlungen. Diese Liste nimmt Zahlungen jeglicher Art auf. Die Vererbungsbeziehung erleichtert die Implementierung einer solchen allgemeinen Liste. Zudem macht das Beispiel – ganz unabhängig von Implementierungsaspekten – den fachlichen Vererbungszusammenhang deutlich. Z. B. gilt: Eine Kreditkartenzahlung *ist eine* Zahlung, und daher kann sie in die Liste aller Zahlungen aufgenommen werden.

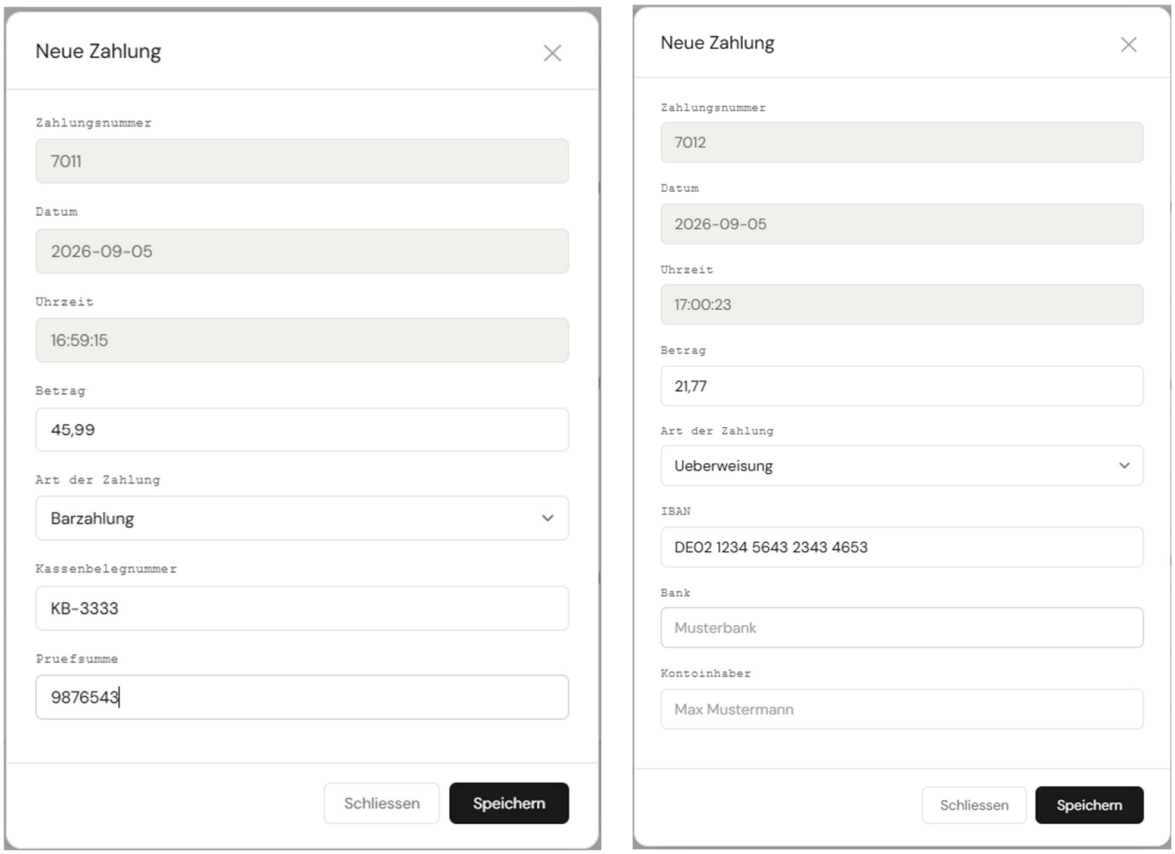


Abbildung 158: Dialoge zum Anlegen von Objekten der konkreten Unterklassen

Zahlungsnr.	Datum	Uhrzeit	Betrag	Art	
7001	2026-03-01	08:00:00	18,00 EUR	Barzahlung	Anzeigen
7002	2026-03-02	09:10:00	25,35 EUR	Kreditkartenzahlung	Anzeigen
7003	2026-03-03	10:20:00	32,70 EUR	Ueberweisung	Anzeigen
7004	2026-03-04	11:30:00	40,05 EUR	Barzahlung	Anzeigen

Abbildung 159: Die Liste mit Zahlungen kann unterschiedliche Typen von Zahlungen enthalten.

13.7 Fallbeispiel Geräteverwaltung

Im Folgenden sollen einige wichtige Dialoge für das Fallbeispiel der Geräteverwaltung skizziert werden. In Abbildung 160 ist noch einmal das Klassendiagramm mit den Entity-Klassen und ihren Assoziationen dargestellt.

Zunächst wird die Hierarchie aus Kategorie, Gerätetyp und Gerät betrachtet. Wie in Kapitel 11 erläutert, ist jedes Gerät, d. h. jedes einzelne Geräteexemplar, von einem bestimmten Gerätetyp. Gerätetypen sind wiederum in Kategorien eingeteilt.

Die Klasse Kategorie enthält lediglich das Attribut *„bezeichnung“*. Entsprechend kann der Dialog zum Anlegen bzw. zum Bearbeiten einer Kategorie recht einfach sein. In Abbildung 161 wird die Kategorie „Präsentation“ bearbeitet.

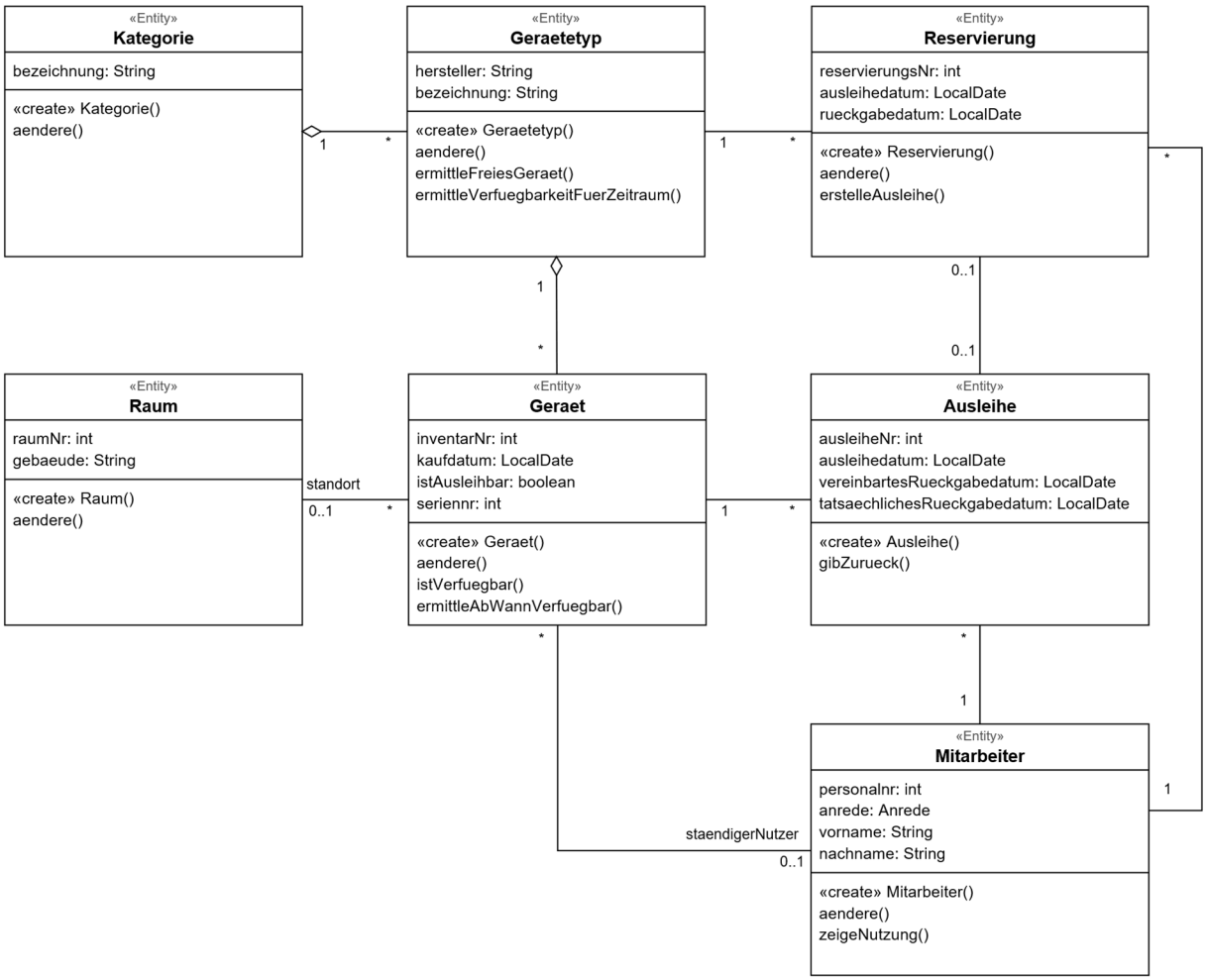


Abbildung 160: Entity-Klassen des Fallbeispiels zur Geräteverwaltung (Wiederholung von Abbildung 130)

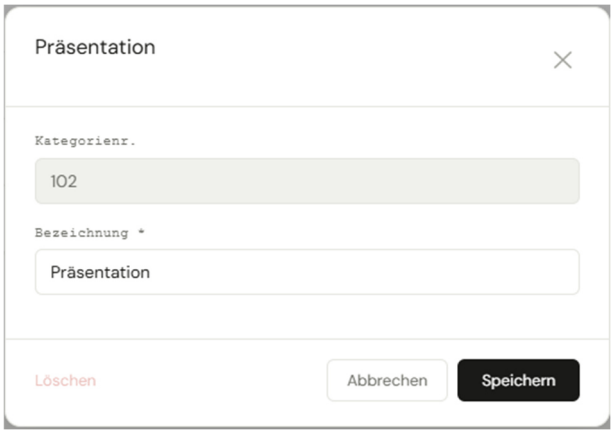


Abbildung 161: Kategorie bearbeiten

Der Dialog, mit dem man einen neuen Gerätetyp anlegen kann (Abbildung 162) enthält außer den Textfeldern für Bezeichnung und Hersteller eine Möglichkeit, aus den vorhandenen Kategorien eine auszuwählen (Abbildung 163).

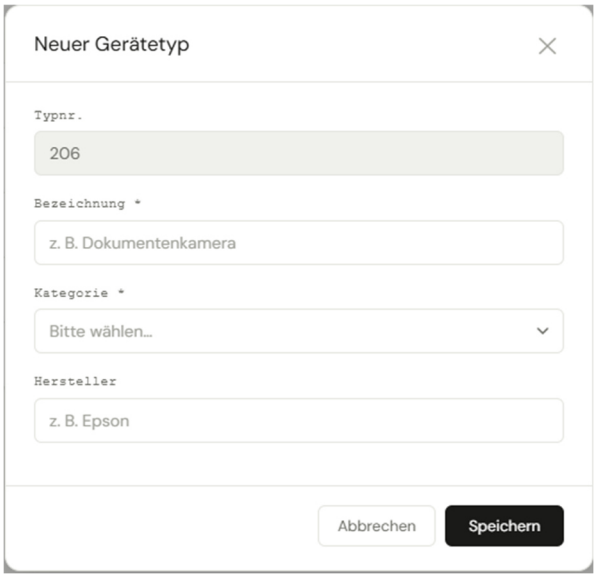


Abbildung 162: Gerätetyp anlegen

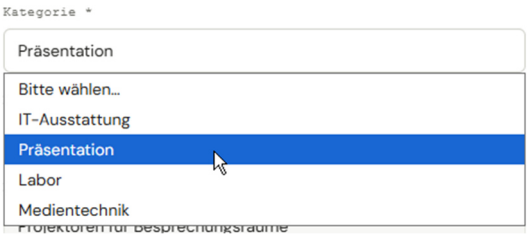


Abbildung 163: Kategorie auswählen (im Dialog aus Abbildung 162)

Abbildung 164 zeigt den Dialog zum Anlegen eines neuen Geräts. Da jedes Gerät zu einem Gerätetyp gehört, enthält er ein entsprechendes Auswahlfeld. Wenn man es mit großen Zahlen von Gerätetypen zu tun hat, ist es sinnvol, zunächst eine Auswahlmöglichkeit für die Kategorie anzubieten. Die Liste der auswählbaren Gerätetypen ist dann auf die selektierte Kategorie eingeschränkt.

Schließlich lässt sich noch auswählen, ob das Gerät ausleihbar ist. Nicht ausleihbare Geräte sind dauerhaft einem festen Standort und einem bestimmten Mitarbeiter zugeordnet. Da in Abbildung 164 das Häkchen bei „Gerät ist ausleihbar" nicht gesetzt ist, sind in den Dialog die Auswahlfelder für den Standort und den ständigen Nutzer eingeblendet.

Um die Auswahllisten für Standort und ständigen Nutzer mit Einträgen zu füllen, müssen Räume und Mitarbeiter angelegt sein. Auch hierfür werden entsprechende Dialoge benötigt.

Der Dialog zum Bearbeiten eines existierenden Geräts in Abbildung 165 entspricht dem zum Anlegen eines neuen Geräts. In diesem Beispiel ist das betreffende Gerät allerdings ausleihbar. Daher enthält der Dialog keine Auswahlfelder für Standort und ständigen Nutzer.

Durch das dynamische Ein- und Ausblenden von Standort und ständigem Nutzer in Abhängigkeit von dem Häkchen bei „Gerät ist ausleihbar" wird die Einhaltung der Zusatzbedingung aus Kapitel 11.2 sichergestellt: Ein Gerät ist entweder ausleihbar, oder es hat einen Standort und einen ständigen Nutzer (vgl. die Notiz in Abbildung 127).

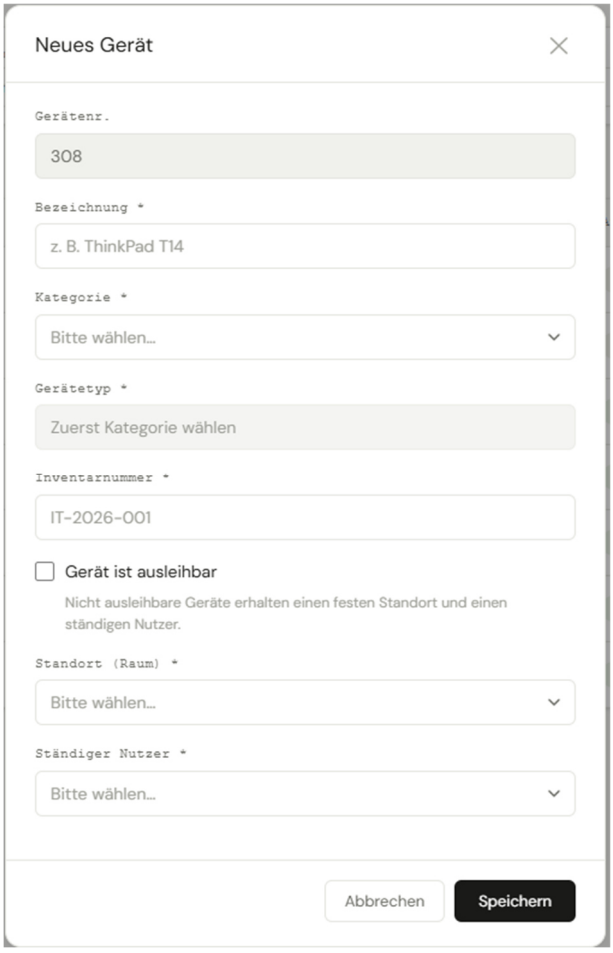


Abbildung 164: Gerät anlegen (nicht ausleihbares Gerät)

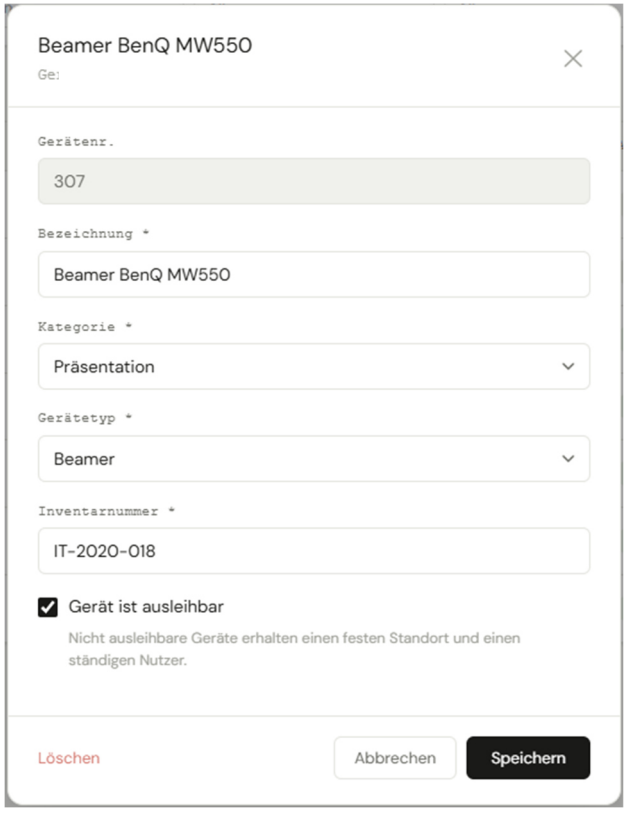


Abbildung 165: Gerät bearbeiten (ausleihbares Gerät)

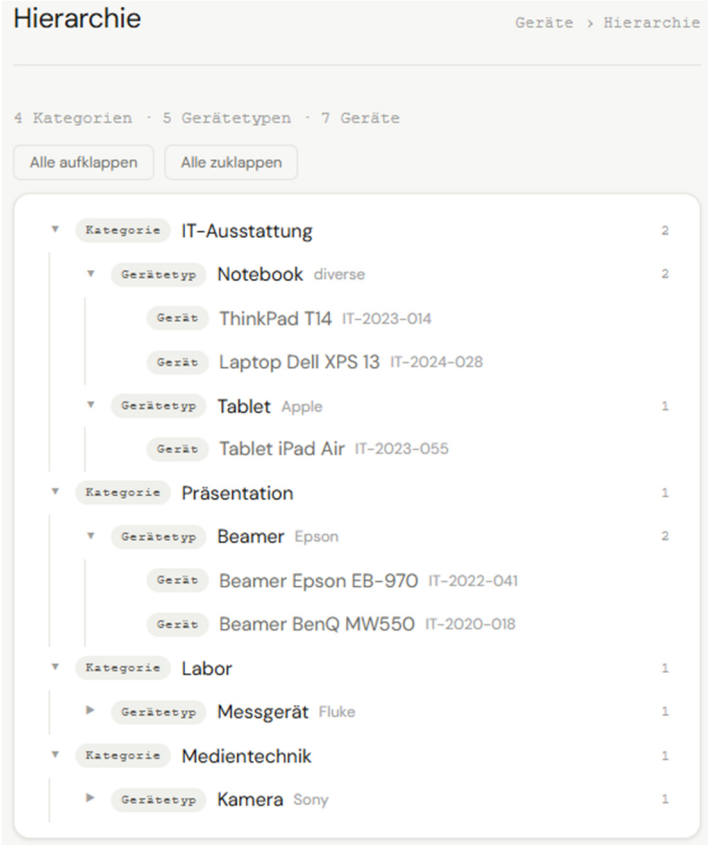


Abbildung 166: Alternative Darstellung der Gerätehierarchie

Die Aggregationsbeziehungen zwischen Kategorie, Gerätetyp und Gerät bilden eine dreistufige Hierarchie. Eine solche Hierarchie kann auch grafisch in Form einer Explorer-Struktur angezeigt und gepflegt werden. Die Darstellung in Abbildung 166 ist somit eine alternative Möglichkeit zur Anzeige und Pflege von Kategorien, Gerätetypen und Geräten. Unterbäume können nach Bedarf auf- und zugeklappt werden.

In dem gezeigten Beispiel handelt es sich bei der Explorer-Struktur zunächst nur um eine Anzeige der Hierarchie. Sie könnte jedoch dahingehend erweitert werden, dass auch eine Bearbeitung möglich ist. Etwa indem man auf einem Gerätetyp ein Kontextmenü öffnet, über das der Dialog zum Anlegen eines neuen Geräts aufgerufen wird.

Änderungen der Zuordnungen könnte man durch Verschieben von Elementen im Explorer-Baum erreichen. Über das Kontextmenü könnte sich auch ein Dialog öffnen lassen. mit dem man weitere Details des jeweils selektierten Elements bearbeiten kann.

Als letzter Dialog soll das Anlegen einer Reservierung betrachtet werden (Abbildung 167). Mitarbeiter und Gerätetyp können ausgewählt werden. Für das einfachere Ausfüllen von gewünschtem Ausleih- und Rückgabedatum kommen Date-Picker-Elemente zum Einsatz kommen, wo man die Daten aus einem kleinen aufklappbaren Kalender auswählt.

Klickt man auf „Verfügbarkeit prüfen“, so ermittelt das System, ob im gewünschten Zeitraum noch ein Gerät des gewünschten Typs vorhanden ist und ausgeliehen werden kann.

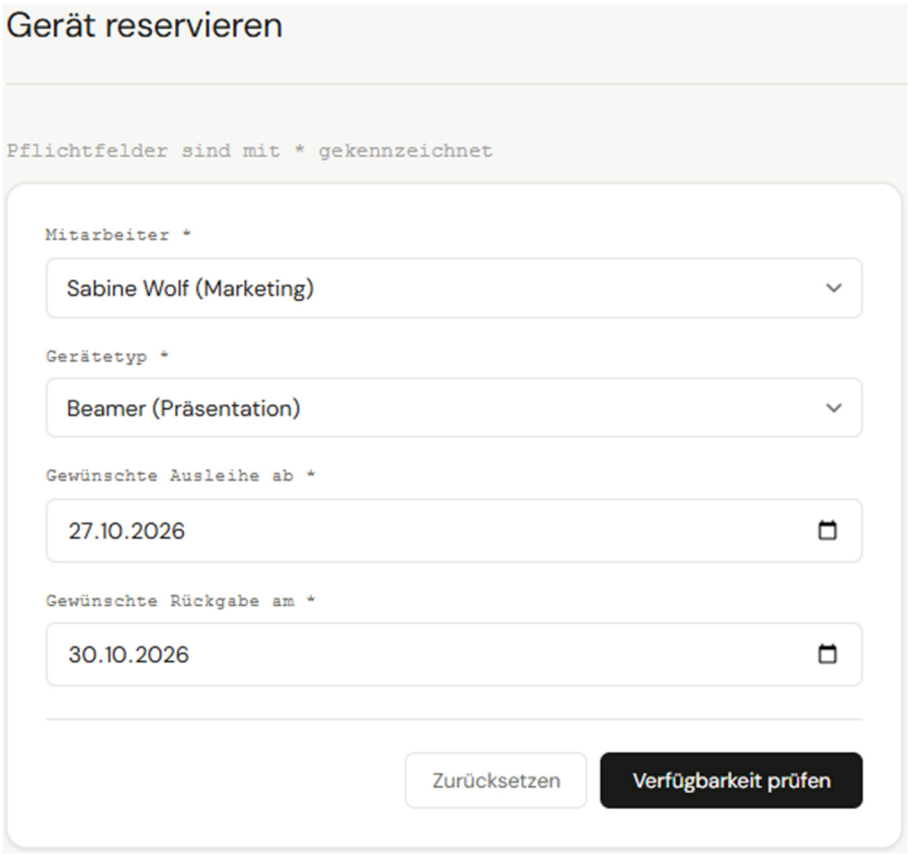


Abbildung 167: Gerätetyp reservieren

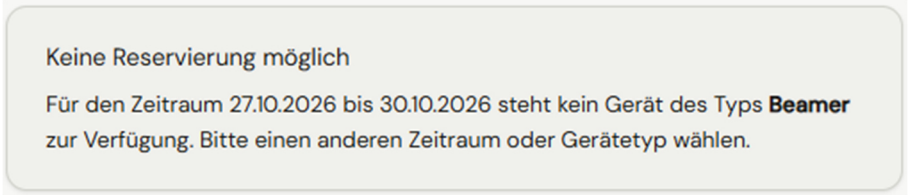


Abbildung 168: Mitteilung, dass die Reservierung nicht möglich ist

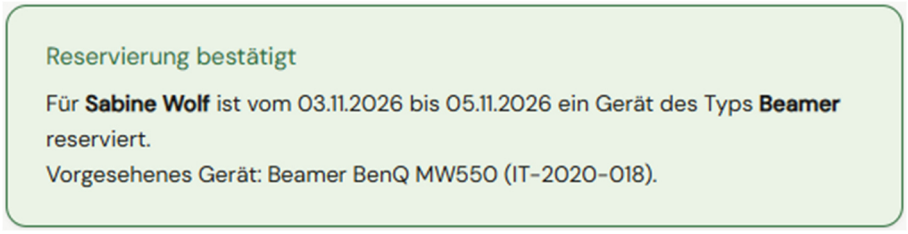


Abbildung 169: Bestätigung einer erfolgreichen Reservierung

Der Algorithmus hierfür wurde in Kapitel 11.5 beschrieben. Je nach Ergebnis der Verfügbarkeitsprüfung wird die Reservierung tatsächlich angelegt, oder der Benutzer erhält eine Misserfolgsmeldung und kann seine Eingaben ändern. Die betreffenden Meldungen finden sich in Abbildung 168 und Abbildung 169.

Die weiteren Dialoge werden hier nicht detailliert besprochen. Die Dialoge zum Anlegen und Bearbeiten von Mitarbeitern und Räumen sind recht einfach, da hier nur die entsprechenden Attributwerte eingetragen werden müssen. Personalnummern können wiederum vom System automatisch vergeben werden.

Der Dialog zum Anlegen einer Ausleihe ähnelt dem zum Anlegen einer Reservierung. Falls schon eine Reservierung vorliegt, können der Wert für das gewünschte Rückgabedatum und der Mitarbeiter übernommen und ggf. geändert werden. Als Ausleihdatum kann das System automatisch das aktuelle Datum eintragen. Schließlich muss noch das ausgeliehene Gerät eingetragen werden. Die Auswahlliste kann hierfür so gefiltert werden, dass nur die Geräte des reservierten Typs angezeigt werden, so dass das Auswählen des Geräts einfacher wird.

Außerdem wird ein Dialog für die Rückgabe eines Geräts benötigt. Wird die erfolgte Rückgabe darin bestätigt, kann das System das aktuelle Datum als tatsächliches Rückgabedatum eintragen.

14 Architektur

14.1 Begriff der Software-Architektur

Ab einer gewissen Größe ist es erforderlich, ein System in kleinere, überschaubarere Einheiten aufzuteilen. Die Architektur einer Software beschreibt, aus welchen einzelnen Teilen sie zusammengesetzt ist, welche Beziehungen zwischen diesen Teilen bestehen und über welche Schnittstellen sie zusammenwirken.

Eine derartige Zerlegung kann über mehrere Stufen erfolgen, indem man die einzelnen Teile selbst noch einmal in kleinere Teile untergliedert. Das System in Abbildung 170 besteht aus den Sub-Systemen 1, 2 und 3. Das Sub-System 3 ist selbst noch einmal in die beiden Bausteine 3.1 und 3.2 aufgeteilt.

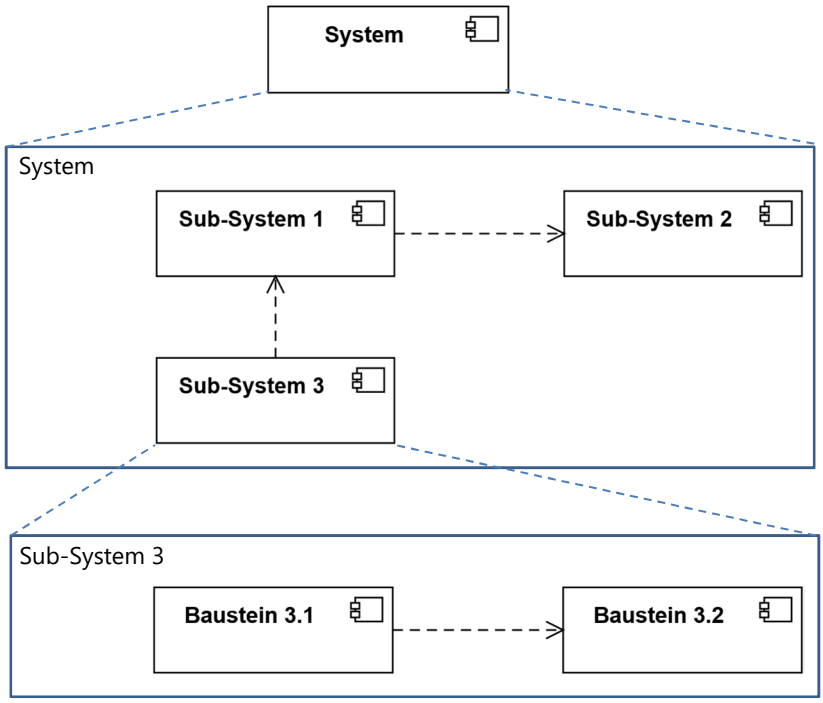


Abbildung 170: Zerlegung eines Systems

Die Bedeutungen der Begriffe „Sub-System“ und „Baustein“ sind nicht einheitlich definiert. Sie wurden hier gewählt um die gebildeten Einheiten nach ihren Hierarchieebenen zu unterscheiden.

Beim Entwurf einer geeigneten Architektur spielen die nicht-funktionalen Anforderungen eine wichtige Rolle, wie z. B. Geschwindigkeit, zu bewältigende Last oder Sicherheitsanforderungen.

Ist z. B. mit einer sehr großen Zahl von Benutzern und einem großen Datenvolumen zu rechnen, so muss das System skalierbar sein, d. h. man muss es leicht an eine größere Belastung anpassen können. Dies kann etwa dadurch geschehen, dass man die Software auf mehrere Server verteilt. Dies ist aber nur dann möglich, wenn man die Architektur von vornherein so

gestaltet, dass einzelne Komponenten des Gesamtsystems auf unterschiedlichen Rechnern installiert sein können.

Zu berücksichtigen sind außerdem Vorgaben des Kunden hinsichtlich der zu verwendenden Technologien, Plattformen, Programmiersprachen, Frameworks u. ä. Häufig liegt damit schon eine Reihe architektonischer Eigenschaften fest.

Da es sich bei der Architektur einer Software um ihren grundsätzlichen Aufbau handelt, sind Architektur-Entscheidungen sehr grundlegende Entscheidungen. Spätere Änderungen der Architektur sind zumeist sehr aufwändig.

Das ist ganz ähnlich wie beim Bau eines Hauses. Änderungen, die die grundlegende Architektur nicht betreffen, sind vergleichsweise einfach. So können etwa nicht tragende Trennwände versetzt werden. Möchte man hingegen zu einem bereits errichteten Gebäude nachträglich einen Keller hinzufügen, erfordert dies ggf. den Abriss und Neubau des Gebäudes. Genauso lassen sich in einem Softwaresystem, dessen Implementierung bereits weit gediehen ist, wietere Funktionalitäten hinzufügen oder Berechnungsalgorithmen verändern. Möchte man andererseits die Struktur verändern und die Komponenten ganz anders aufteilen, so muss die Software in großen Teilen neu geschrieben werden.

Aus diesem Grund sollte man sich zu Beginn der Entwicklung recht ausführliche Gedanken über die geeignete Architektur machen. Hilfreich kann ggf. die Implementierung eines Architektur-Prototypen sein, mit dem die zu verwendenden Technologien und das prinzipielle Zusammenspiel der verschiedenen Komponenten ausprobiert werden.

Stellt man trotz dieser ausführlichen Überlegungen während der Entwicklung fest, dass gewisse Architekturentscheidungen ungünstig waren, so sollte man in der Regel trotz des erforderlichen Aufwandes einen Umbau der Software vornehmen. Einen solchen „Umbau", bei dem die nach außen sichtbare Funktionalität des Systems nicht geändert wird, nennt man auch Refaktorisierung (englisch Refactoring).

Je früher dies geschieht, desto besser ist es, denn mit fortschreitender Entwicklung wird eine solche Refaktorisierung immer aufwändiger. Verzichtet man auf notwendige Architekturänderungen, so handelt man sich mittelfristig beträchtliche negative Konsequenzen ein. Beispielsweise gelingt es dann nicht – oder nur mit immensem Zusatzaufwand – wichtige nichtfunktionale Anforderungen zu erfüllen. Häufig leidet die Wartbarkeit, und der notwendige Aufwand für Fehlerbehebungen und Änderungen steigt drastisch, so dass für die Weiterentwicklung unter Umständen ein Mehrfaches der Zeit benötigt wird, die durch das Unterlassen der Refaktorisierung eingespart wurde.

Zerlegt man eine Software in einzelne Sub-Systeme, so kann dies nach unterschiedlichen Kriterien geschehen. Beispiele für solche Zerlegungskriterien sind:

- Zerlegung nach unterschiedlichen Tätigkeiten oder Prozessen

    - Z. B. könnte man eine betriebliche Software in Sub-Systeme für Tätigkeiten wie „Vermieten“, „Bestellen“ oder „Verkaufen“ aufteilen.
- Zerlegung nach unterschiedlichen Daten
    - Beispielsweise könnte es eigene Sub-Systeme für „Geräte“, „Produkte“ oder „Kunden“ geben.
- Zerlegung nach technischen Aspekten
    - Eine solche Aufteilung könnte Sub-Systeme für die Benutzungsoberfläche (GUI), die Datenhaltung, die Rechteverwaltung etc. vorsehen.

Diese Zerlegungskriterien können auch miteinander kombiniert werden. So kann auf oberster Ebene zunächst eine häufig anzutreffende Aufteilung in die technischen „Schichten“ Benutzungsoberfläche, Anwendungslogik und Datenbank erfolgen. Die einzelnen Schichten können dann selbst noch einmal nach anderen Kriterien unterteilt sein. Z. B. könnte die Anwendungslogik-Schicht nach Tätigkeiten untergliedert sein. Oder man hat auf oberster Ebene eine Aufteilung nach unterschiedlichen Daten vorgenommen, wobei jedes Sub-System wiederum in Benutzungsoberfläche, Anwendungslogik und Datenhaltung unterteilt ist.

14.2 Strukturierung mit Hilfe von Paketen

Zur Zerlegung von Softwaresystemen in kleinere Sub-Systeme können Pakete (Packages) verwendet werden. Abbildung 171 zeigt noch einmal das UML Paketdiagramm für das System zur Geräteverwaltung und die entsprechende Java-Paketstruktur in Eclipse.

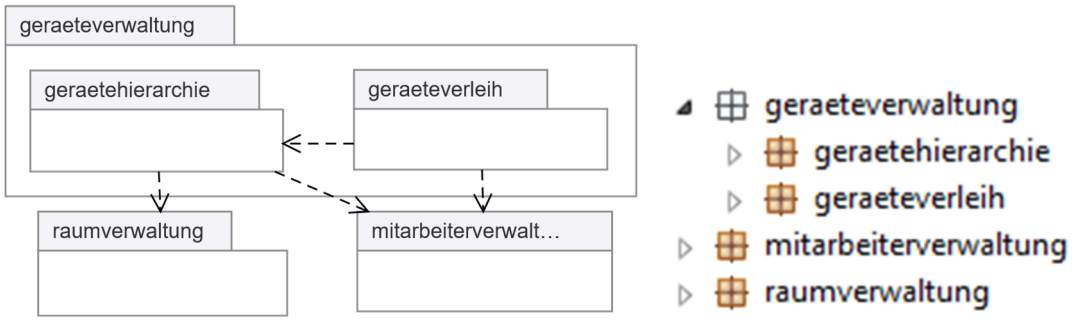


Abbildung 171: Paketstruktur in UML und Java-Package-Struktur in Eclipse (Wiederholung von Abbildung 131 und Abbildung 132)

Aus Architektursicht sind insbesondere auch die – als gestrichtelte Pfeile dargestellte – Abhängigkeiten interessant. Wie bereits in Kapitel 11.7 erläutert, wird eine möglichst lose Kopplung zwischen den einzelnen Paketen angestrebt. Man versucht ein System so zu zerlegen, dass möglichst wenige Abhängigkeiten zwischen den Sub-Systemen entstehen. Auf diese Weise lassen sich die einzelnen Sub-Systeme weitgehend unabhängig voneinander weiterentwickeln. Änderungen innerhalb eines einzelnen Sub-Systems führen dann nicht sofort dazu, dass gleichzeitig viele andere Sub-Systeme geändert werden müssen.

14.3 Lose Kopplung

In dem Detaildiagramm in Abbildung 172 wird deutlich, wodurch die Abhängigkeit des Pakets „*geraetehierarchie*“ von den Paketen „*raumverwaltung*“ und „*mitarbeiterverwaltung*“ entsteht. Die Klasse Geraet besitzt Assoziationen zu den Klassen „Raum“ aus dem Paket „*raumverwaltung*“ und der Klasse „*Mitarbeiter*“ aus dem Paket „*mitarbeiterverwaltung*“.

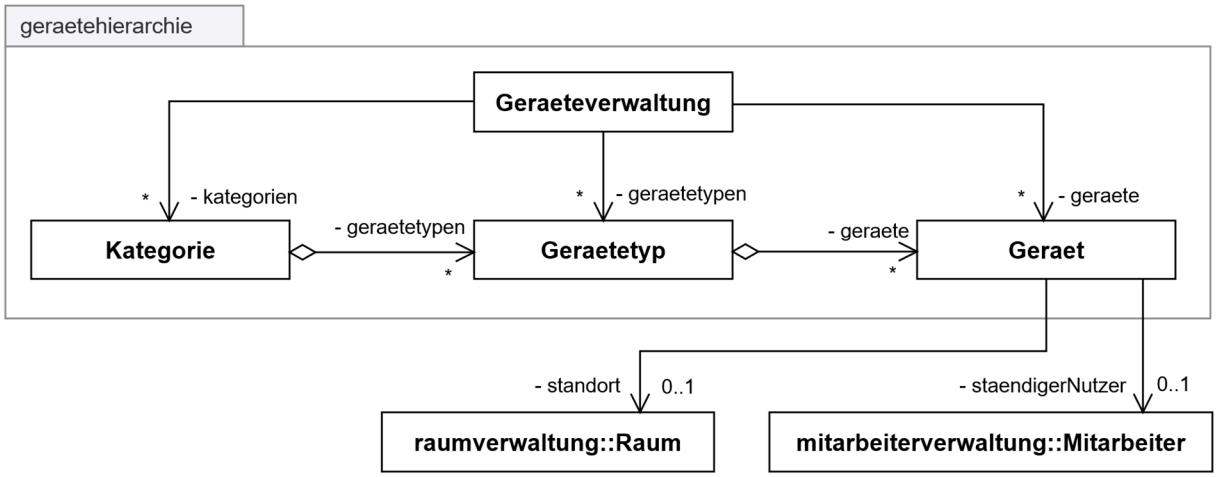


Abbildung 172: Paket „geraetehierarchie“ (Wiederholung von Abbildung 133)

Würde man in einem dieser beiden Pakete eine Änderung durchführen, so könnte dies auch das Paket „*geraetehierarchie*“ betreffen. Würde man beispielsweise den Namen der Klasse „*Raum*“ ändern, so müsste dies auch im Code der Klasse „*Geraet*“ geändert werden. Umgekehrt haben Änderungen im Paket „*geraetehierarchie*“ keine Auswirkungen auf die Pakete „*raumverwaltung*“ und „*mitarbeiterverwaltung*“.

Die Assoziationen zu „*Raum*“ und „*Mitarbeiter*“ sind nur in eine Richtung navigierbar. Wären sie in beide Richtungen navigierbar, so entstünden beidseitige Abhängigkeiten.

Mitarbeiter-Informationen können in vielen anderen Paketen von Bedeutung sein. Neben der Gerätehierarchie und dem Geräteverleih könnten bei einer Weiterentwicklung der Software auch Pakete für die Lohnbuchhaltung o. ä. hinzukommen. Hätte man nun wie in Abbildung 173 beidseitige Abhängigkeiten zwischen dem Paket „*mitarbeiterverwaltung*“ und all diesen anderen Paketen, so hätte man eine sehr enge Kopplung. Das Paket „*mitarbeiterverwaltung*“ müsste über alle anderen Pakete Bescheid wissen und wäre ggf. von deren Änderungen betroffen.

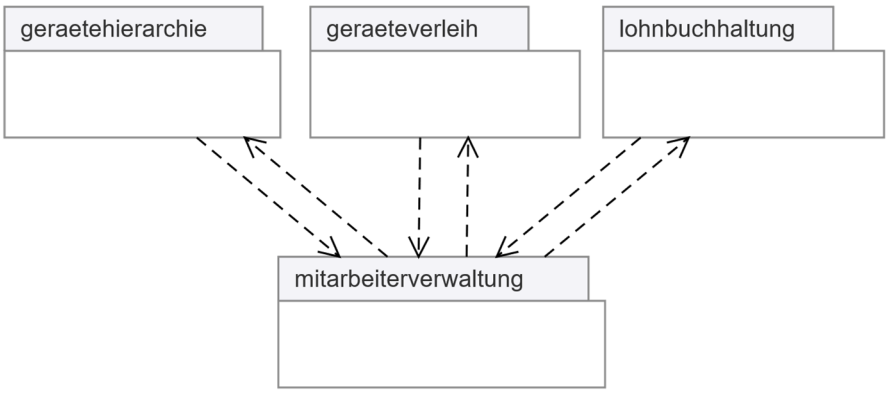


Abbildung 173: Enge Kopplung: mitarbeiterverwaltung muss über alle anderen Pakete Bescheid wissen und ist von deren Änderungen ggf. betroffen

Besteht die Abhängigkeit hingegen nur jeweils in die Richtung zum Paket „*mitarbeiterverwaltung*“, so muss das Paket „*mitarbeiterverwaltung*“ nichts von den anderen Paketen wissen, in denen Mitarbeiter-Informationen genutzt werden (Abbildung 174). Die anderen Pakete verwenden einfach das Paket „*mitarbeiterverwaltung*“. Ändert sich in diesen Paketen etwas, oder wird etwa ein weiteres Paket hinzugefügt, so braucht man im Paket „*mitarbeiterverwaltung*“ überhaupt nichts zu machen. Dies ist wesentlich wartungsfreundlicher als die beidseitigen Abhängigkeiten. Die Inhalte des Pakets „*mitarbeiterverwaltung*“ werden als eine Art Service zur Verfügung gestellt und können an vielen Stellen genutzt werden. Damit wird das Ziel eines hohen Grads an Wiederverwendung unterstützt.

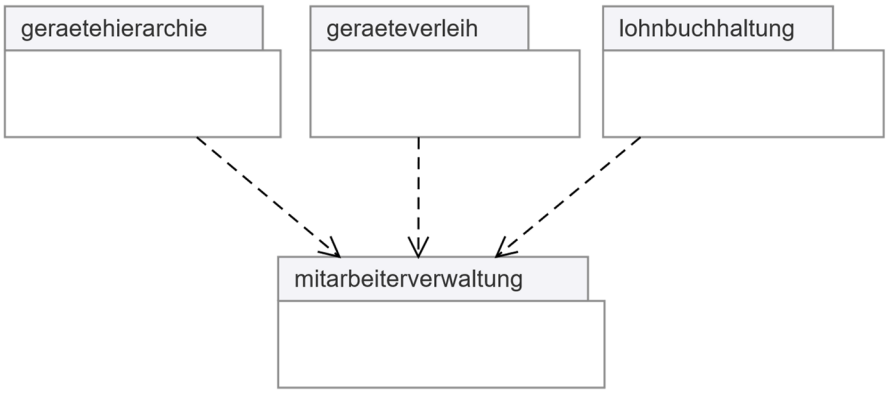


Abbildung 174: Lose Kopplung: mitarbeiterverwaltung muss nichts über die anderen Pakete wissen.

Auch zyklische Abhängigkeiten sind zu vermeiden, da es sich dabei ebenfalls um gegenseitige Abhängigkeiten handelt. So ist in Abbildung 175 Paket C von Paket B abhängig. Da Paket B von Paket A abhängig ist, das wiederum von Paket C abhängig ist, ist Paket B indirekt auch von Paket C abhängig.

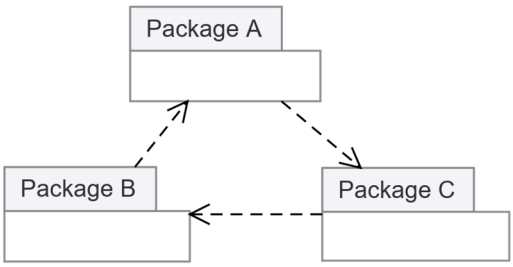


Abbildung 175: Zyklische Abhängigkeiten

Die Einschränkung auf unidirektionale Assoziationen zwischen Klassen unterschiedlicher Pakete hat auch Nachteile. Gemäß dem Modell aus Abbildung 172 kann man zu einem Gerät recht leicht auf den zugeordneten Raum und den zugeordneten Mitarbeiter zugreifen. Möchte man in Gegenrichtung navigieren und z. B. für einen bestimmten Mitarbeiter die zugeordneten Geräte herausfinden, so ist dies schwieriger, da das Mitarbeiter-Objekt über keine Liste mit Geräten verfügt.

Man muss daher alle Geräte durchgehen und überprüfen, ob ihnen das betreffende Mitarbeiterobjekt zugeordnet ist. Hierfür könnte man in der Klasse „Geraeteverwaltung" die folgende Methode implementieren:

```
getAlleGeraeteVonMitarbeiter(m:Mitarbeiter): List<Geraet>
```

Diese Methode bekommt das gewünschte Mitarbeiter-Objekt als Parameter übergeben und gibt eine Liste mit allen gefundenen Geräten zurück. Hierzu geht sie die Liste *„geraete"* durch und erfragt von jedem Geräte-Objekt, ob das gewünschte Mitarbeiter-Objekt als *„staendiger-Nutzer*" zugeordnet ist.

Während man zwischen den Paketen eine möglichst lose Kopplung anstrebt, d. h. einen möglichst geringen Grad an Abhängigkeiten, sollte innerhalb der Pakete eine möglichst hohe Kohäsion herrschen, d. h. die Klassen eines Pakets sollten möglichst eng miteinander zusammenhängen. Hat man in einem Paket Klassen, die nichts oder sehr wenig miteinander zu tun haben, so sollte man sich überlegen, ob man diese Klassen nicht eher in unterschiedlichen Paketen unterbringen sollte.

Hat man auf der anderen Seite Pakete mit sehr engen Abhängigkeiten, die sich auch nicht sinnvoll reduzieren lassen, so sollte man sich überlegen, ob man die eng voneinander abhängigen Klassen nicht in ein gemeinsames Paket legt.

Lose Kopplung (zwischen den Paketen) und hohe Kohäsion (im Innern der Pakete) sind zwei Seiten derselben Medaille.

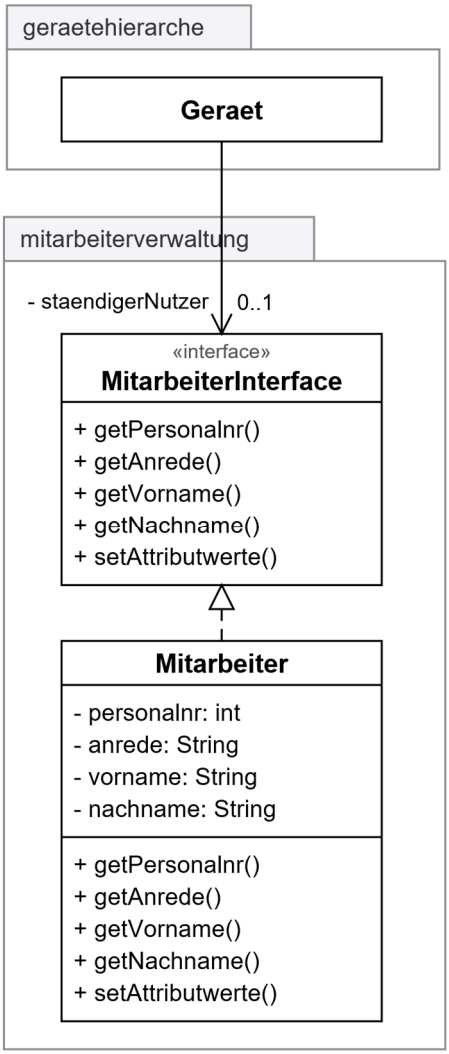


Abbildung 176: Lose Kopplung durch Interfaces

Zur weiteren Unterstützung der losen Kopplung verschiedener Pakete können Interfaces (Schnittstellen) eingesetzt werden. In Abbildung 176 hat die Klasse „Geraet" keine direkte Assoziation zur Klasse „Mitarbeiter" mehr. Stattdessen verweist die Assoziation auf das neu hinzugekommene Interface „MitarbeiterInterface". Die Klasse Mitarbeiter implementiert dieses Interface. „MitarbeiterInterface" enthält die Methodendeklarationen, die von außerhalb des Pakets aufgerufen werden dürfen.

Der Vorteil: Die Klasse „Geraet" muss nur noch das MitarbeiterInterface kennen. Wenn man sich einmal auf das Interface geeinigt hat, so können die Entwickler der beiden Pakete weitgehend unabhängig voneinander arbeiten. Die Entwickler des Pakets „*mitarbeiterverwaltung*" müssen dafür sorgen, dass die Methoden des Interface implementiert werden. Die Entwickler der Klasse „geraetehierarchie" nutzen die Methoden aus dem MitarbeiterInterface.

Das Interface stellt somit eine Art Vertrag zwischen den beiden Packages dar. Nur, wenn man feststellt, dass Änderungen des Interface erforderlich sind, sind erneut beiderseitige Abstimmungen notwendig. Daher sollte man sich möglichst genau überlegen, wie die Interfaces aussehen müssen, damit sie während der Entwicklung möglichst stabil bleiben.

In dem Beispiel stellt sich noch die Frage, wie denn ein Geraet-Objekt an ein konkretes Mitarbeiter-Objekt gelangt, das als ständiger Nutzer zugeordnet wird. Hierzu kann etwa eine Verwaltungsklasse im Package „mitarbeiterverwaltung“ eine entsprechende Suchmethode bereitstellen. In Kapitel 11.1 wurde eine Control-Klasse „Mitarbeiterverwaltung“ eingeführt, die diese Aufgabe übernehmen kann. Neben dem Mitarbeiterinterface muss also auch noch diese Control-Klasse nach außen sichtbar sein. Außer um zunächst an ein Objekt zu gelangen werden aber nur die Schnittstellen-Methoden verwendet.

14.4 Komponenten und Komponentendiagramme

Während Pakete vor allem der logischen und internen Strukturierung der Software dienen, handelt es sich bei Komponenten um Software-Bausteine, die in der Regel als physische Einheiten vorliegen (z. B. als jar-Dateien).

Dabei gibt es keinen festgelegten Zusammenhang zwischen Paketen und Komponenten. So kann eine Komponente ein oder mehrere Pakete umfassen, umgekehrt können die Inhalte eines Pakets auch auf mehrere Komponenten aufgeteilt werden.

Komponenten kommunizieren stets über definierte Schnittstellen miteinander. Abbildung 177 zeigt ein UML Komponentendiagramm mit zwei Komponenten. „Component 1“ stellt eine Schnittstelle „Interface 1“ zur Verfügung. „Component 2“ andererseits benötigt eine Komponente mit einer Schnittstelle vom Typ „Interface 1“. Das bedeutet, dass „Component 1“ und „Component 2“ zusammenpassen und gemeinsam eingesetzt werden können.

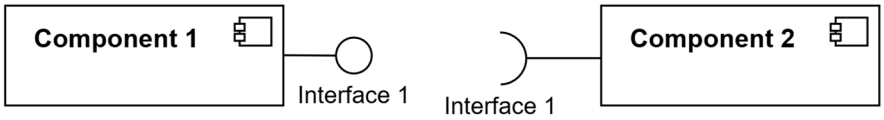


Abbildung 177: Komponenten mit bereitgestellter (links) und benötigter Schnittstelle (rechts)

Dieses Zusammenwirken lässt sich im Komponentendiagramm auch darstellen, indem die Symbole der zusammenpassenden bereitgestellten und benötigten Schnittstelle ineinander greifen oder „ankoppeln“ (Abbildung 178).

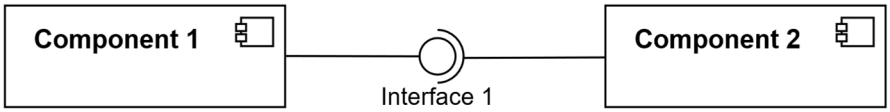


Abbildung 178: Darstellung mit „angekoppelter“ Schnittstelle

Das kleine Quadrat am Beginn der Verbindung zur Schnittstelle stellt einen sogenannten Port dar. Er könnte auch als langgezogenes Rechteck gezeichnet werden und mehrere Schnittstellen umfassen. Die Verwendung des Port-Symbols ist optional, d. h. man dürfte es in Abbildung 177 und Abbildung 178 auch weglassen.

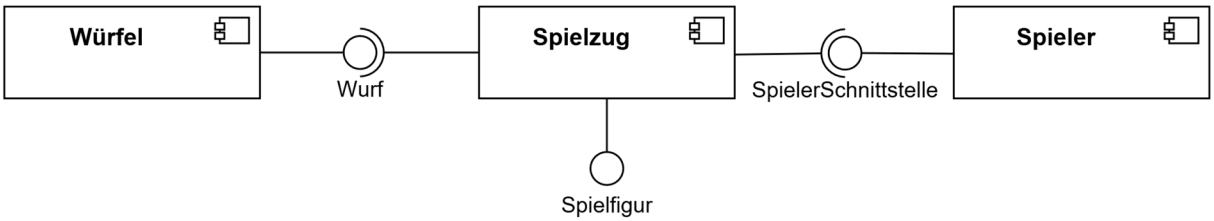


Abbildung 179: UML-Komponentendiagramm einer Spielsoftware (nach [SeGu06])

Abbildung 179 zeigt ein Beispiel aus einer Spielsoftware. Die Komponente „Würfel" stellt eine Schnittstelle „Wurf" zur Verfügung, die für die Verwaltung der Spielerdaten zuständige Komponente „Spieler" stellt eine Schnittstelle zur Verfügung, die ebenfalls „Spieler" heißt. Die Komponente „Spielzug" nutzt die Schnittstellen „Wurf" und „Spieler" und stellt selbst die Schnittstelle „Spielfigur" zur Verfügung.

In UML handelt es sich bei dem Interface intern um denselben Objekttyp wie ein Interface, das von einer Klasse implementiert werden kann (wie es z. B. in Abbildung 176 genutzt wurde). Die Umsetzung einer Komponenten-Schnittstelle muss aber nicht unbedingt genau durch ein Java-Interface erfolgen. Es sind vielmehr auch sehr komplexe Schnittstellen möglich, wie z. B. ein komplettes API (Application Programming Interface) mit sehr vielen Klassen. Umgekehrt sind auch sehr simple Schnittstellen möglich, wie z. B. einzelne statische Methoden.

Abbildung 180 zeigt das Beispiel aus Abbildung 178 in einer anderen Darstellung. Hier wurde die Schnittstelle mit dem Interface-Symbol aus dem Klassendiagramm modelliert. Die Bereitstellung der Schnittstelle durch „Component 1" wird mit einem Implementierungspfeil dargestellt. Die Nutzung dieser Schnittstelle durch „Component 2" wird durch einen Abhängigkeitspfeil mit dem Stereotyp «use» repräsentiert.

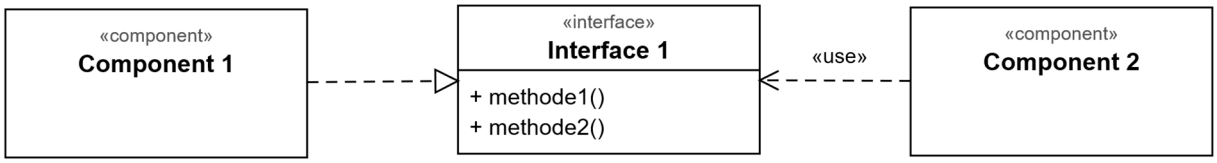


Abbildung 180: Andere Darstellung von Schnittstellen und Komponenten

Ein Vorteil dieser Darstellung besteht darin, dass die Methoden des Interface angezeigt werden können. Oftmals wird in Überblicksdiagrammen mit zahlreichen Komponenten und Schnittstellen die kompaktere Darstellung aus Abbildung 178 verwendet. Die Details der Schnittstellen können dann ggf. in separaten Diagrammen modelliert werden.

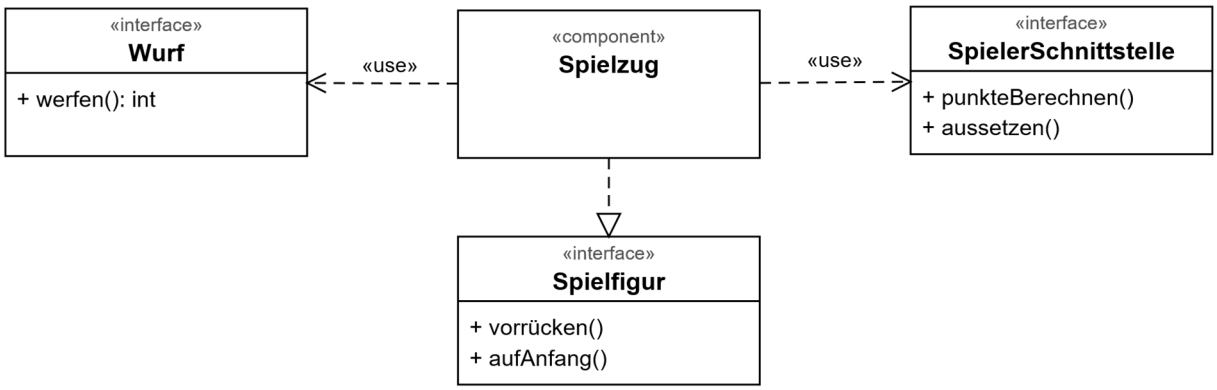


Abbildung 181: UML-Komponentendiagramm der Spielsoftware – alternative Darstellung (in Anlehnung an [SeGu06])

In Abbildung 181 wird die Komponente „Spielzug“ aus Abbildung 179 mit ihren benötigten und angebotenen Schnittstellen in der ausführlichen Darstellung gezeigt.

14.5 Modellierung der physischen Struktur

Soll die physische Struktur eine Software modelliert werden, so kann dies mit Hilfe sogenannter Artefakte geschehen (englisch Artifact). Ein Artefakt kann dabei jedes physische Objekt sein, im Zusammenhang mit Software handelt es sich in der Regel um Dateien, wie z. B. Konfigurationsdateien oder ausführbare Dateien, oder auch Bilder oder Dokumente. Ein Artefakt kann eine Komponente „manifestieren“, d. h. die Komponente liegt physisch in Form des betreffenden Artefakts vor.

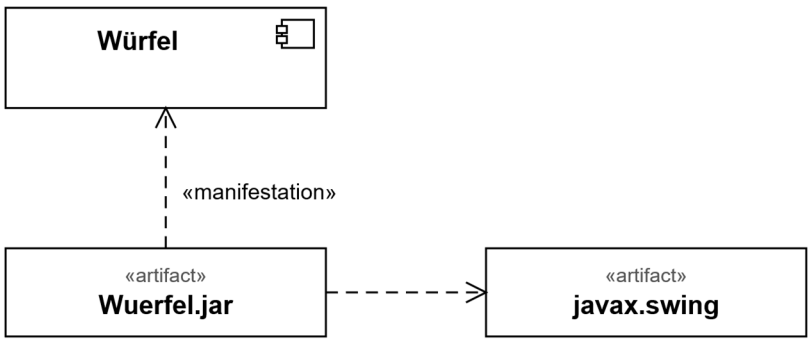


Abbildung 182: Darstellung der physischen Repräsentation (in Anlehnung an [SeGu06])

In Abbildung 182 manifestiert die Datei „Wuerfel.jar“ die Komponente „Würfel“. Zudem ist „Wuerfel.jar“ von der Java Swing-Bibliothek abhängig.

Soll die Hardware-Topologie und die Verteilung von Software auf verschiedene Computer modelliert werden, so kann man ein UML-Verteilungsdiagramm (engl. Deployment Diagram) erstellen. Hardware-Elemente, wie z. B. Server, Arbeitsplatzcomputer, Mobilgeräte, Router werden darin als „Nodes“ (Knoten) modelliert.

Ein Node wird in Form eines perspektivisch gezeichneten Quaders abgebildet. Zwischen Nodes können Kommunikationsbeziehungen existieren. Sie können mit Multiplizitäten versehen werden. Damit kann man beispielsweise modellieren, dass ein Server mit vielen Client-Rechnern in Verbindung steht. Schließlich lässt sich das Deployment, also die

Verteilung von Software-Artefakten auf verschiedene Nodes mit Hilfe von „deployment"-Beziehungen darstellen.

Abbildung 183 zeigt für eine verteilte Plattform für Gesellschaftsspiele, dass ein Spiele-Server mit mehreren Monopoly-Servern verbunden sein kann. Umgekehrt kann ein Monopoly-Server mit einem Spiele-Server verbunden sein, muss es aber nicht.

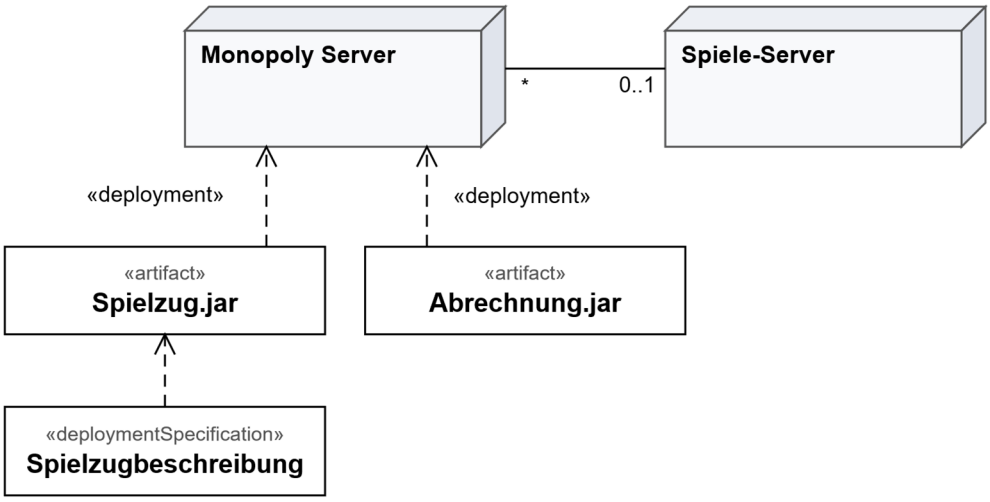


Abbildung 183: UML Verteilungsdiagramm (Deployment Diagram, in Anlehnung an [SeGu06])

Die beiden Artefakte „Spielzug.jar" und „Abrechnung.jar" werden auf dem Monopoly-Server installiert. Zudem gibt es noch ein weiteres Artefakt, die Spielzugbeschreibung, die von „Spielzug.jar" abhängig ist. Es handelt sich um eine „Deployment Specification", die nicht selbst auf einen Server aufgespielt wird. Darin wird beschrieben, was für das Aufspielen von „Spielzug.jar" auf den Monopoly-Server zu tun ist. Das Modell zeigt nur einen Ausschnitt, so sind z. B. die Artefakte für den Spiele-Server nicht dargestellt.

14.6 Architekturstile

Es wurde bereits erläutert, dass Systeme nach ganz unterschiedlichen Kriterien in Sub-Systeme aufgeteilt werden können. Zudem können die einzelnen Sub-Systeme auf diverse Art und Weise zusammenwirken. Entsprechend können ganz verschiedene Architekturstile entstehen.

So kann ein und dieselbe Aufgabe unter Umständen durch Systeme mit unterschiedlichen Architekturstilen auf ganz verschiedene Art und Weise gelöst werden.

Im Folgenden werden exemplarisch einige häufig verwendete Architekturstile besprochen:

- Pipes and Filters
- Repository („Shared Data")
- Objektorientiert
- Layered (Schichtenarchitekturen)

14.6.1 Pipes and Filters

Die Grundidee dieses Architekturstils: Daten wandern durch das System und werden dabei in mehreren Schritten bearbeitet. Es gibt zwei grundsätzliche Arten von Bausteinen: Pipes („Rohre“), die zur Weiterleitung von Daten dienen, und Filter, die die Daten verändern (Abbildung 184).

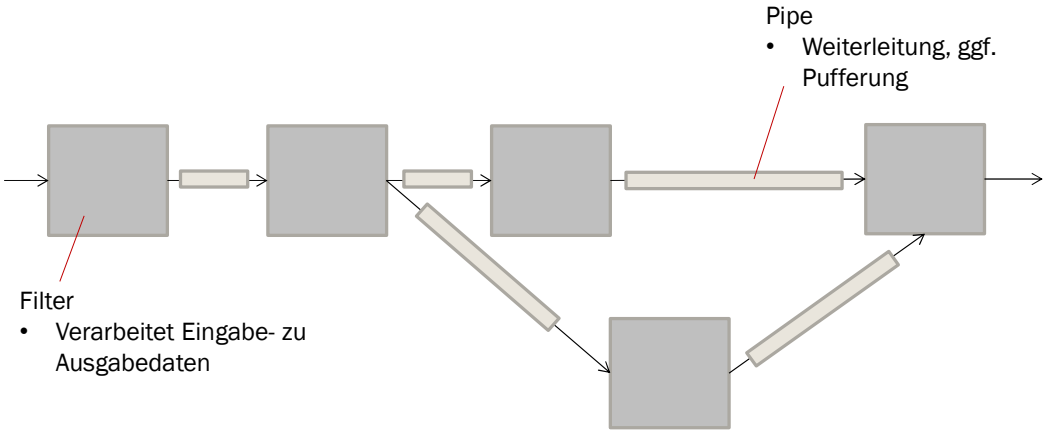


Abbildung 184: Architekturstil Pipes and Filters

Im einfachen Fall leitet eine Pipe einfach die Ausgangsdaten des einen Filters zum Eingang des nächsten Filters weiter. Sie kann aber auch weitere Aufgaben übernehmen, wie etwa das Puffern von Daten.

Anders als der Name vermuten lässt können Filter Daten nicht nur nach bestimmten Kriterien einschränken. Vielmehr kann es sich um beliebige Funktionen zur Verarbeitung von Daten handeln, also auch z. B. zur Transformation oder Anreicherung von Daten. Filter erhalten Eingabedaten über eingehende Pipes, verarbeiten diese zu Ausgabedaten, die über die ausgehenden Pipes weitergeleitet werden.

Ein typisches Beispiel für diese Muster findet sich in der Programmierung. So wird etwa ein C-Programm zunächst durch einen Präprozessor verarbeitet, anschließend kompiliert und schließlich durch einen Linker in ein ausführbares Programm überführt. Auch die Bildverarbeitung in einer Kamera funktioniert nach diesem Prinzip: Nach der Bilderfassung findet z. B. eine Farbkorrektur statt, anschließend werden Bildeffekte angewandt, und schließlich wird die Bilddatei komprimiert.

Die verarbeiteten Daten müssen nicht in Form von Dateien vorliegen, und es ist auch nicht immer erforderlich, dass Datenobjekte vor ihrer Weiterverarbeitung vollständig von der jeweiligen Vorstufe abgearbeitet wurden. Vielmehr können auch endlose Datenströme, wie z. B. Videostreams, gut mit dieser Architektur verarbeitet werden.

Ein weiterer Vorteil der Pipes- and Filters-Architektur besteht darin, dass es leicht möglich ist, das System zu verändern, indem Filter hinzugefügt, entfernt oder anders angeordnet werden.

Es gibt unterschiedliche Methoden, wie und in welcher Reihenfolge die einzelnen Filter oder Funktionen aufgerufen werden:

1. Durch eine zentrale Steuerung
2. Die einzelnen Funktionen rufen sich gegenseitig nacheinander auf
   - Jede Funktion ruft die nachfolgende auf („Push-Prinzip“), oder:
   - Jede Funktion fordert die Ergebnisse der vorangehenden an („Pull-Prinzip“)
3. Ereignisgesteuert („Implicit Invocation“)
   - Jede Funktion abonniert bestimmte Ereignisse, auf die sie reagiert.
     Der Abschluss einer Funktion kann dann selbst wieder ein Ereignis auslösen, auf das wiederum andere Funktionen reagieren usw.

14.6.2 Repository („Shared Data“)

Bei diesem Architekturstil arbeiten verschiedene Funktionen mit zentralen, gemeinsam genutzten Daten („Shared Data“). Dabei können die Funktionen auf die Daten zugreifen und sie auch verändern. Die geänderten Daten stehen damit sofort allen anderen Funktionen zur Verfügung.

Bekanntestes Beispiel für diesen Architekturstil sind Datenbanken. Hier liegen die Daten an einer zentralen Stelle und können über entsprechende Abfragen und Aktualisierungsoperationen gelesen und verändert werden.

Ein wesentlicher Vorteil dieser Architektur besteht darin, dass allen Funktionen stets dieselben, aktuellen und konsistenten Daten zur Verfügung stehen. Auch lassen sich Funktionen verhältnismäßig leicht hinzufügen oder verändern, ohne dass dies Auswirkungen auf die anderen Funktionen hat.

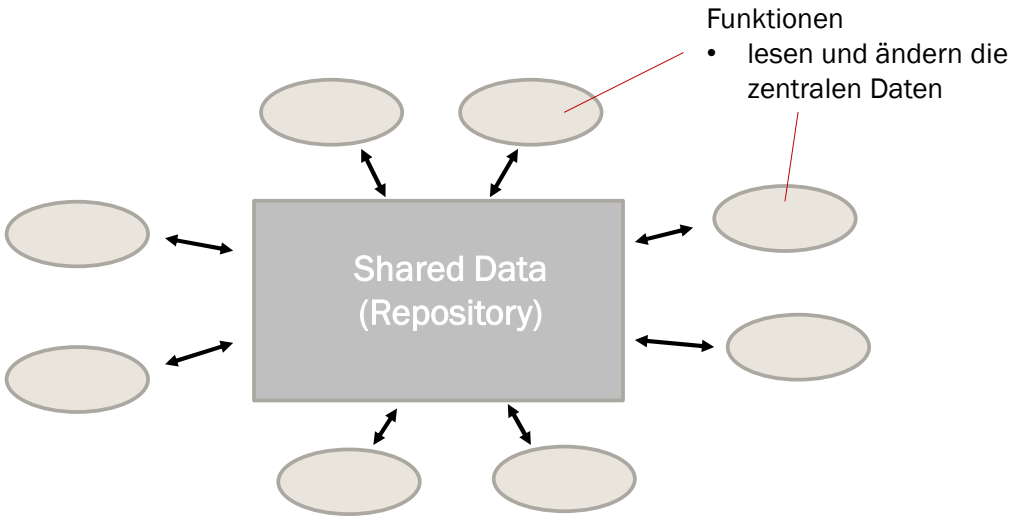


Abbildung 185: Architekturstil Shared Data (Repository)

Für die Steuerung der Funktionsaufrufe stehen prinzipiell dieselben Möglichkeiten zur Verfügung wie beim Architekturstil „Pipes and Filters“.

14.6.3 Objektorientiert

Das Prinzip der Objektorientierung ist aus der objektorientierten Programmierung bekannt. Hierbei sind Daten und Funktionen, die in den beiden vorangehenden Architekturstilen getrennt waren, in Objekten gekapselt. Es findet eine verteilte Verarbeitung in einem Objektgraphen statt. Diese Verarbeitung lässt sich sehr gut mit Sequenzdiagrammen visualisieren (Abbildung 186).

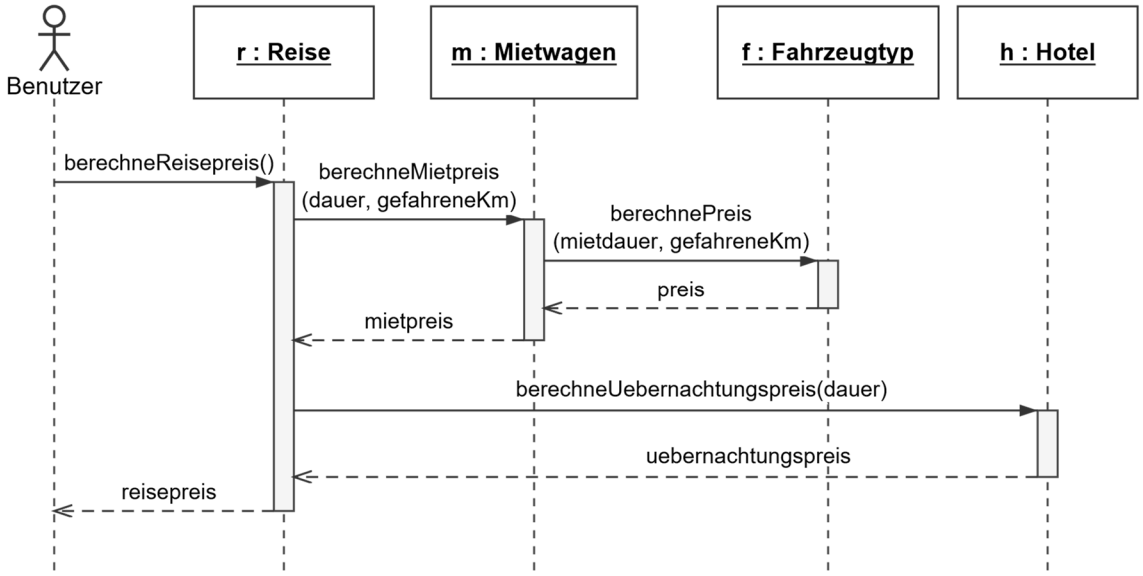


Abbildung 186: Objektorientierter Architekturstil (Wiederholung von Abbildung 105)

Jedes Objekt enthält Daten (Attribute) und die für ihre Bearbeitung benötigten Funktionen (Methoden). Bei komplexeren Funktionen rufen die Objekte gegenseitig die jeweils benötigten, einfacheren Funktionen auf. Es sind also sowohl die Verarbeitungsfunktionalität als auch die Steuerung entlang der Datenstrukturen verteilt.

Das objektorientierte Prinzip kann auch auf höherer Ebene eingesetzt werden. Bei den Objekten muss es sich nicht zwangsläufig um einzelne Java-Objekte o. ä. handeln, es können vielmehr auch grobgranulare, zusammengesetzte Objekte sein.

Die Verwendung einer objektorientierten Sprache wie Java legt natürlich nahe, zumindest auf der untersten Ebene den objektorientierten Architekturstil zu verwenden. Auf höherer Ebene können intern objektorientierte Bausteine jedoch auch entsprechend anderer Architekturstile zusammenarbeiten. Zum Beispiel können die Funktionen einer Pipes and Filters- oder einer Repository-Architektur intern objektorientiert aufgebaut sein.

Vorteile dieses Architekturstils sind insbesondere die Trennung von Verhalten und Implementierung, was u. a. die Austauschbarkeit und die separate Entwicklung unterschiedlicher Teile des Systems erleichtert, sowie eine recht gute Wiederverwendung.

14.6.4 Layered (Schichtenarchitekturen)

Eine Schichtenarchitektur umfasst mehrere Schichten (Layer). Jede Schicht bietet Funktionalitäten an, die von der nächsthöheren Schicht genutzt werden können. Umgekehrt ist dies

nicht möglich: Niedrigere Schichten dürfen nicht auf Inhalte oder Funktionen oberer Schichten zugreifen.

Zum Teil wird dies auch in Form von Schalenmodellen dargestellt, wie in Abbildung 187. Hier enthält die unterste Ebene (oder der innere Ken) allgemeine grundlegende Systemfunktionen. Die darüber liegende Schicht nutzt diese Kern-Funktionalitäten und bietet Basis-Dienste an. Die oberste Ebene – oder äußerste Schale – enthält schließlich die eigentliche, nützliche Anwendung. Sie kann auf die Basis-Dienste der darunter liegenden Schicht zugreifen.

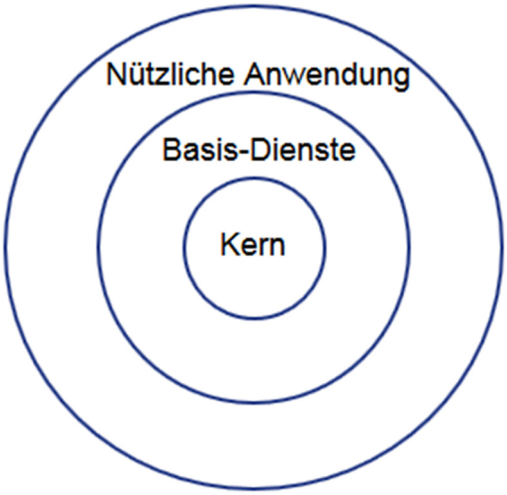


Abbildung 187: Schichten-Architektur

Ein Beispiel für diese Architektur findet sich in jedem Computer. Auf der untersten Ebene befindet sich die Hardware, darüber das Betriebssystem, darüber nützliche Anwendungen, wie z. B. eine Textverarbeitung. Hierbei greift die Textverarbeitung nicht direkt auf die Hardware zu, sondern nutzt entsprechende Befehle der direkt darunter liegenden Betriebssystem-Schicht. Und auch ein umgekehrter Zugriff findet nicht statt: So nutzt das Betriebssystem keine Funktionalitäten der Textverarbeitung.

Eine weitere, sehr verbreitete Schichtenarchitektur stellt das ISO/OSI-Modell der Datenübertragung dar. Es besteht aus insgesamt sieben Schichten, von der physischen Schicht bis hin zur Anwendungsschicht.

Auch sehr viele betriebliche Anwendungen und Web-Anwendungen nutzen typischerweise eine Schichtenarchitektur aus drei oder mehr Schichten. Sehr häufig ist eine Drei-Schichten-Architektur, die aus Oberfläche (Präsentation), Fachklassen (Anwendungslogik) und Datenhaltung besteht (Abbildung 188). Zum Teil werden diese Schichten noch weiter untergliedert, oder es werden noch zusätzliche Schichten eingezogen.

Ein wichtiger Vorteil besteht in der Unabhängigkeit und Austauschbarkeit der Schichten. So brauchen die Entwickler tieferer Schichten nichts über die darüber liegenden Schichten zu wissen, und für die Entwickler höherer Schichten ist die gesamte Komplexität aller darunter liegenden Schichten in der Schnittstelle der direkt darunter liegenden Schicht gekapselt.

In der Praxis wird gelegentlich nicht nur auf die direkt darunter liegende Schicht zugegriffen, sondern direkt auf eine tiefere Schicht, und manchmal findet auch eine direkte Kommunikation von unten nach oben statt. Dafür kann es im konkreten Fall gute Gründe geben, es stellt jedoch eine Verletzung des Prinzips der Schichtenarchitektur dar.

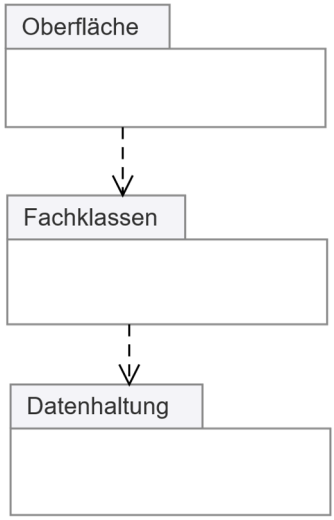


Abbildung 188: Drei-Schichten-Architektur

14.6.5 Nutzung von Architekturstilen

Bei den vorgestellten Architekturstilen handelt es sich nur um einige Beispiele, die häufig anzutreffen sind. Die Architekturstile können auf unterschiedlichen Detaillierungsstufen eingesetzt werden. So wurde bei der Vorstellung verschiedener Architekturstile recht allgemein von Funktionen oder Filtern gesprochen. Je nach Detaillierungsgrad kann es sich bei einer solchen Funktion beispielsweise um eine einzelne Methode handeln, aber auch um eine umfangreiche Komponente oder gar ein ganzes System.

Ferner lassen sich die Stile miteinander kombinieren, z. B. können die Filter einer Pipes- and Filters-Architektur selbst wiederum in Form einer Schichtenarchitektur realisiert werden, oder die einzelnen Schichten einer Schichtenarchitektur sind intern objektorientiert strukturiert.

15 Entwurf

Während es bei der Architektur eines Softwaresystems darum geht, aus welchen Komponenten und Schnittstellen sie aufgebaut ist, befasst sich der Entwurf damit, wie die Schnittstellen und Komponenten im Detail aussehen sollen. Häufig spricht man im Zusammenhang mit der Entwicklung der Architektur auch vom Grobentwurf und bei der detaillierten Ausarbeitung der Komponenten vom Feinentwurf. Im Folgenden wird der Begriff „Entwurf" im Sinne von Feinentwurf verwendet.

Auch bei eher klassischem, wasserfallartigem Vorgehen wird bei Entwurf und Implementierung oft eher iterativ vorgegangen. Es wird also nicht zuerst ein kompletter Entwurf sämtlicher technischer Details erstellt, der dann nur noch herunterprogrammiert werden muss. Stattdessen werden viele Entwurfsentscheidungen im Zuge der Implementierung getroffen, direkt umgesetzt und getestet, ggf. modifiziert, usw.

In diesem Kapitel liegt der Schwerpunkt vor allem auf dem Entwurf der Fachkonzeptschicht. Daneben sind aber viele andere technischen Aspekte zu entwerfen, wie z. B. die Datenbankzugriffsschicht oder die Verteilung des Systems. Diese Aspekte werden z. T. in eigenen Kapiteln erläutert, sind im Detail aber Gegenstand anderer Vorlesungen.

15.1 Entwurfsprinzipien

In der Praxis hat sich eine Reihe von Prinzipien als hilfreich für einen guten Entwurf bewährt. Hierbei handelt es sich nicht um feste, unbedingt einzuhaltende Vorgaben, sondern um grobe Orientierungslinien. Ob und wie man das jeweilige Prinzip anwendet, hängt von der konkreten Situation ab. Auch lassen sich nicht immer alle Prinzipien gleichzeitig anwenden. In Einzelfällen kommt man je nach berücksichtigten Prinzipien zu unterschiedlichen Lösungen. Dann muss man abwägen, welchem Prinzip man im jeweiligen Fall den Vorrang einräumen sollte.

Die meisten dieser Prinzipien lassen sich sowohl auf der Ebene des Fein- als auch des Grobentwurfs anwenden, d. h. sie sind auch für die Architektur relevant.

Einige wichtige Entwurfsprinzipien sind:

- **„KISS" – Keep it simple and stupid**
  Man sollte möglichst einfache und klare Implementierungen bevorzugen.

- **„YAGNI" – You ain't gonna need it**
  Man sollte keine Verallgemeinerungen für Dinge einführen, die man vielleicht einmal in Zukunft benötigen wird. Oft benötigt man sie dann doch nicht, und es wurden unnötige Aufwände und z. T. auch unnötige Komplexität verursacht.
  So sollte man z. B. nicht von vornherein Setter-Methoden für alle Attribute vorsehen.

- **Geringe Kopplung und hohe Kohäsion**
  Dieses Prinzip wurde bereits im Zusammenhang mit der Architektur besprochen. Es

gilt auf allen Detaillierungsebenen. Ebenso wie zwischen unterschiedlichen Paketen möglichst geringe Abhängigkeiten herrschen sollten, sollte z. B. auch zwischen unterschiedlichen Klassen wenig Abhängigkeiten existieren – dafür aber die Attribute und Methoden jeder Klasse einen engen Zusammenhang haben, d. h. eine hohe Kohäsion aufweisen.

- **Entwurf nach Verantwortlichkeiten**
  Jede Klasse sollte – ebenso wie jedes Paket und jede Komponente – möglichst einen einzelnen, klar definierten Aspekt abdecken. Wenn es schwer fällt, eine geeignete Benennung zu finden und diese dann aus mehreren Wörtern besteht, ist das ein Hinweis auf unklare Verantwortlichkeiten.

- **Trennung von fachlichen und technischen Aspekten**
  Dies hat den Vorteil, dass fachliche und technische Aspekte getrennt voneinander weiterentwickelt werden können. Kapselt man z. B. die technischen Aspekte des Datenbankzugriffs in eigene Klassen, fällt es wesentlich leichter, auf eine andere Datenbank-Version zu wechseln. Auf der anderen Seite wirken sich Änderungen von Kunden-Anforderungen meist nur auf die fachlichen Klassen aus.

- **Offen-Geschlossen-Prinzip**
  Der Entwurf sollte offen für Erweiterungen, aber geschlossen für Änderungen sein. Dies bedeutet, dass Erweiterungen leicht möglich sein sollten, dass es dafür aber möglichst wenig erforderlich sein sollte, den vorhandenen Code zu ändern.

Starke illustriert das Offen-Geschlossen-Prinzip an einem Beispiel [Star24]. Die Methode „*draw( )*“ in Listing 40 dient zum Zeichnen unterschiedlicher geometrischer Formen. Handelt es sich um einen Kreis, ruft die Mehode „*draw( )*“ die Methode „*drawCircle( )*“ auf. Bei einem Quadrat wird „*drawSquare( )*“ aufgerufen.

```
void draw(){
    if (type.equals("circle")
        drawCircle();
    else if (type.equals("square")
        drawSquare();
}
```

Listing 40: Methode „draw( )“ zum Zeichnen unterschiedlicher Formen (Quelle: [Star24])

Was muss nun passieren, wenn man das Programm erweitern möchte, etwa um Dreiecke? In diesem Fall muss der Code geändert werden. Bei der if-Abfrage muss der zusätzliche Fall in einem weiteren else-Teil berücksichtigt werden. Wenn der Typ der Form nicht nur an dieser einen Stelle, sondern auch noch an zahlreichen anderen Stellen im Code eine Rolle spielt, dann muss man den gesamten Code durchgehen um diese Stellen zu finden und anzupassen. Die Erweiterung um eine neue Form ist somit schwierig, und der bestehende Code muss an vielen Stellen geändert werden. Damit widerspricht Listing 40 dem Offen-Geschlossen-Prinzip.

Eine bessere Lösung zeigt Abbildung 189. Hier wurde ein Interface „*Form*“ mit einer Methode „*draw( )*“ definiert. Die Klassen „*Circle*“ und „*Square*“ implementieren dieses Interface. Ruft man „*draw( )*“ auf, so wird jeweils die „*draw( )*“-Methode aus der betreffenden Klasse ausgeführt. Möchte man jetzt Dreieck als weitere Form hinzufügen, so muss man lediglich die neue Klasse „*Triangle*“ hinzufügen, die ebenfalls das Interface „*Form*“ implementiert, und darin die Methode „*draw( )*“ implementieren. Die Erweiterung um eine neue Form ist somit recht einfach, und es muss kein vorhandener Code geändert werden. Somit entspricht die Lösung dem Offen-Geschlossen-Prinzip.

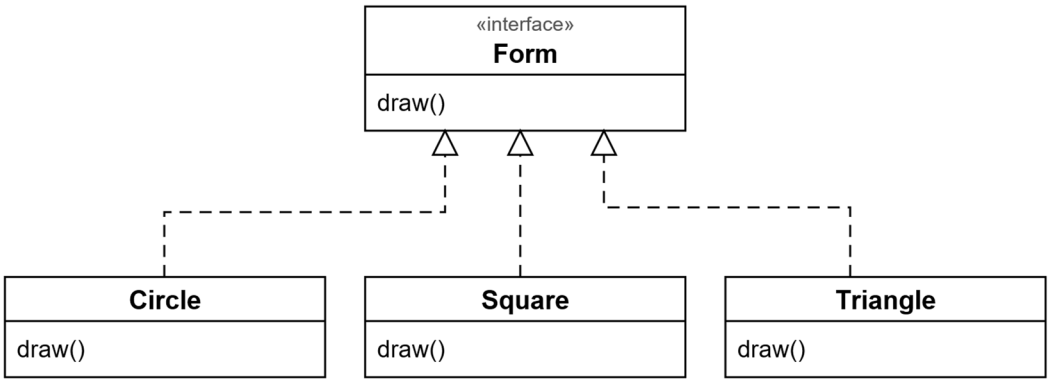


Abbildung 189: Bessere Lösung nach dem Offen-Geschlossen-Prinzip (nach [Star24]).

Auf der anderen Seite ist hier dieses Prinzip gegen die Prinzipien KISS (Keep it Simple and Stupid) und YAGNI (You ain’t gonna need it) abzuwägen. Spielt der Typ der Form vielleicht nur an ganz wenigen Stellen eine Rolle, so ist die Lösung aus Listing 40 auf jeden Fall deutlich einfacher. Und wenn man nicht vielleicht sowieso schon weiß, dass man künftig noch mit weiteren Formen rechnen muss, sagt einem das YAGNI-Prinzip, dass man lieber erst einmal auf diese zusätzliche Flexibilität verzichten sollte.

Stellt man während der Entwicklung dann aber plötzlich fest, dass doch immer noch neue Formen hinzukommen, ist es Zeit, den Code zu refaktorisieren und nach dem Offen-Geschlossen-Prinzip zu gestalten.

15.2 Entwurf der Fachkonzept-Schicht

Bei Anwendungen gemäß der typischen Drei-Schichten-Architektur aus Abbildung 188 befindet sich die eigentliche Anwendungslogik in der mittleren Fachklassen- oder Fachkonzept-Schicht. Ausgangspunkt für den Entwurf dieser Schicht ist das Analysemodell (vgl. Kapitel 10). Dieses wird nun zu einem Entwurfsmodell weiterentwickelt. Dabei kann es erforderlich sein, die Klassenstruktur zu modifizieren. Zudem sind die Assoziationen, Attribute und Methoden zu verfeinern, und die Objektverwaltung muss realisiert werden. Im Gegensatz zur Analyse geht es nun insbesondere auch darum, die nicht-funktionalen Anforderungen, wie Performance, Effizienz und Wartbarkeit zu berücksichtigen.

Die Klassenstruktur kann folgendermaßen modifiziert werden:

- **Zerlegen von Klassen**
  Umfangreiche, komplexe Klassen mit mehreren Zuständigkeiten werden in mehrere Klassen aufgeteilt.
- **Zusammenfassen von Klassen**
  Klassen mit starker Interaktion und hoher Kopplung werden zu gemeinsamen Klassen zusammengefasst.
- **Vererbung und Schnittstellen hinzufügen**
  Gemeinsame Eigenschaften und Verhalten können in Oberklassen bzw. Schnittstellen zusammengefasst werden. Hierdurch lassen sich die Wiederverwendbarkeit erhöhen und die Kopplung reduzieren.
- **Vererbung reduzieren**
  Wenn sich die verschiedenen Unterklassen kaum unterscheiden, kann man sie ggf. zusammenfassen.

15.2.1 Zusammenfassen von Klassen

Eine Möglichkeit zum Zusammenfassen von Klassen findet sich in dem Fallbeispiel zur Geräteverwaltung. Darin gibt es die Klassen „*Reservierung*" und „*Ausleihe*", die eine Reihe von Attributen sowie die Assoziation zum Mitarbeiter gemeinsam haben (vgl. Abbildung 190). Zudem repräsentieren sie einen ganz ähnlichen Sachverhalt, nämlich eine Geräteausleihe – zum einen eine geplante Ausleihe, zum anderen eine erfolgte Ausleihe.

Beim Ausleihen müsste man in das betreffende Reservierungsobjekt das tatsächliche Ausleihdatum eintragen und ein konkretes Gerät zuordnen. Für die Abwicklung der Rückgabe müssen in die zusammengefasste Klasse das tatsächliche Rückgabedatum und die „*gibZurueck( )*"-Methode der Klasse „*Ausleihe*" übernommen werden. Die Zusammenfassung der beiden Klassen würde auch die Verfügbarkeitsprüfung vereinfachen, da nur noch Objekte einer Klasse durchsucht werden müssen (vgl. Kapitel 11.5).

Gegen die Zusammenfassung der beiden Klassen spricht, dass die entstehende Klasse umfangreicher und komplexer wird. Weiter vorne wurde ausgeführt, dass jede Klasse möglichst nur für einen einzigen Aspekt verantwortlich sein soll. Die zusammengefasste Klasse wäre hingegen für zwei Aspekte verantwortlich: Reservierung und Ausleihe.

Auch die Benennung der entstehenden Klasse ist nicht ganz einfach, da es keinen naheliegenden Oberbegriff für Ausleihe und Reservierung gibt. Zusammengesetzte Namen wie „ReservierungUndAusleihe" sollten vermieden werden. Sie weisen auf eine mögliche Verletzung des Prinzips der einheitlichen Verantwortlichkeit hin.

Andererseits stehen die beiden Aspekte Reservierung und Ausleihe sehr eng miteinander in Beziehung, so dass man durchaus argumentieren kann, dass es sich um eine einzige Verantwortung handelt. Man kann eine Reservierung einfach als eine Ausleihe auffassen, die noch nicht begonnen hat. Dann vereinfacht sich auch die Namensgebung. Die zusammengefasste Klasse müsste folgerichtig „Ausleihe" heißen.

Es ergibt sich also keine ganz eindeutige Empfehlung für den vorliegenden Fall. Beide Varianten sind vertretbar.

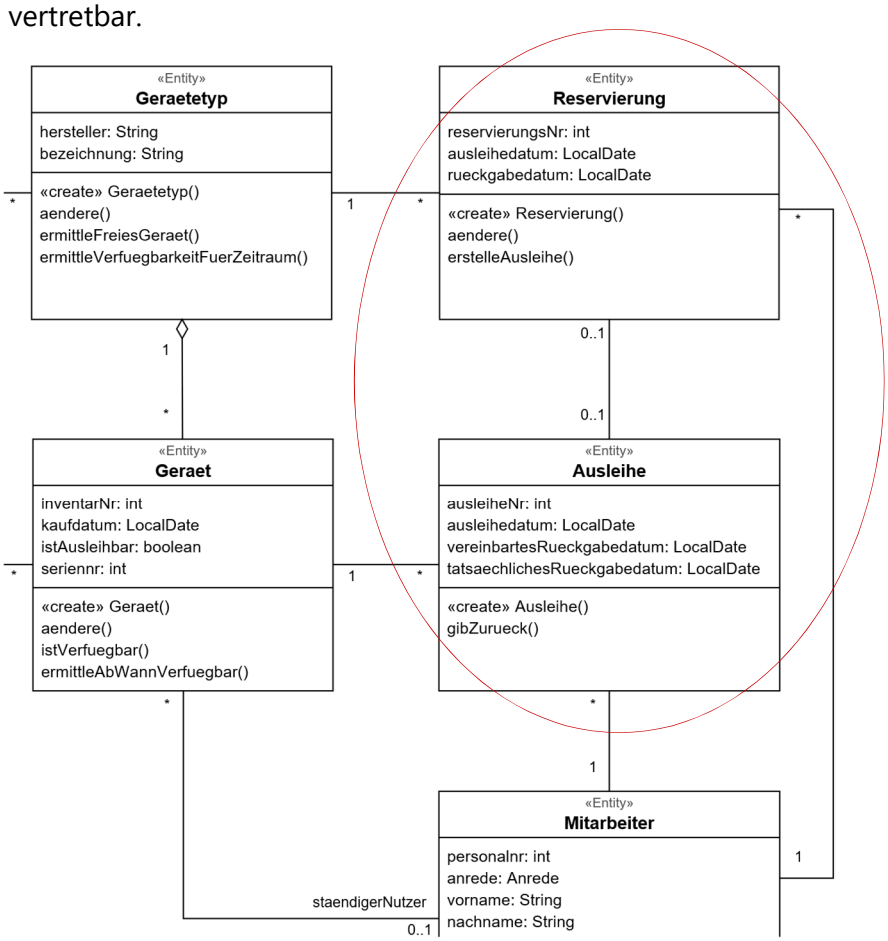


Abbildung 190: Mögliches Zusammenfassen von Klassen aus dem Fallbeispiel

15.2.2 Vererbung reduzieren

Ein Beispiel für die Reduzierung von Vererbung findet sich in [Balz11]. In Abbildung 191 findet sich eine abstrakte Klasse „*Lager*". Es gibt zwei Arten von Lagern: Offene Lager und Stellplatz-Lager. Entsprechend hat Lager die zwei konkreten Unterklassen „*OffenesLager*" und „*StellplatzLager*". Ein Stellplatz-Lager besteht aus vielen einzelnen Stellplätzen.

Eine Palette kann sich entweder in einem offenen Lager oder auf einem Stellplatz eines Stellplatz-Lagers befinden. Betrachtet man nur die Multiplizitäten, so könnte eine Palette zugleich einem offenen Lager als auch einem Stellplatz-Lager zugeordnet sein. Daher wurde eine Notiz hinzugefügt, dass dies ausgeschlossen ist.

Da die Klassen „*Offenes Lager*" und „*Stellplatz-Lager*" nicht so viele Unterschiede haben, stellt sich die Frage, ob wirklich insgesamt drei Klassen zur Repräsentation von Lagern erforderlich sind.

Das alternative Modell in Abbildung 192 kommt mit zwei Klassen weniger aus. Die abstrakte Klasse „*Lager*" mit ihren beiden Unterklassen wurde durch eine einzige Klasse „*Lager*" ersetzt. Diese Klasse ist nun konkret, d. h. es können Objekte davon gebildet werden.

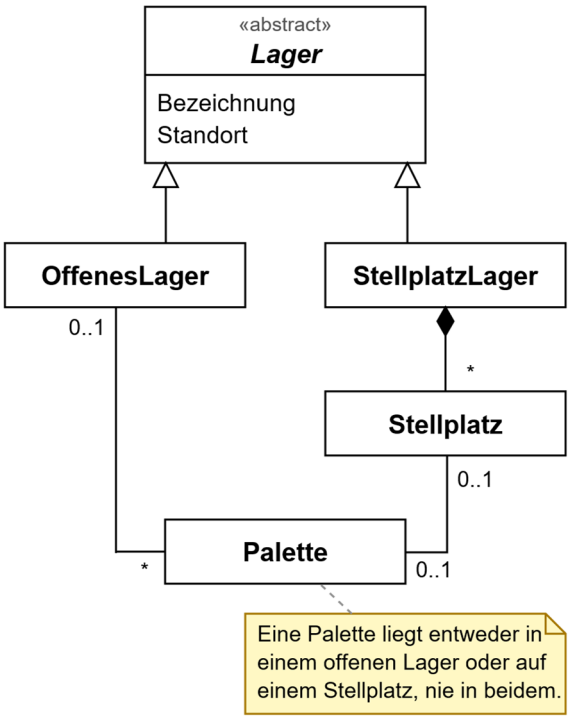


Abbildung 191: Beispiel mit Vererbung (nach [Balz11])

Ob es sich bei einem Lager um ein offenes Lager oder um ein Stellplatz-Lager handelt, wird nun durch ein Attribut „*Art*" gekennzeichnet. Jede Palette kann nun entweder direkt einem Lager oder einem Stellplatz-Lager zugeordnet sein. Um sicherzustellen, dass die direkte Zuordnung nur erfolgt, wenn das Attribut „*Art*" den Wert „*Offenes Lager*" enthält, wurde eine entsprechende Bedingung an der Assoziation notiert. Die andere Bedingung sorgt dafür, dass ein Lager nur dann Stellplätze enthalten kann, wenn es sich um ein Stellplatz-Lager handelt.

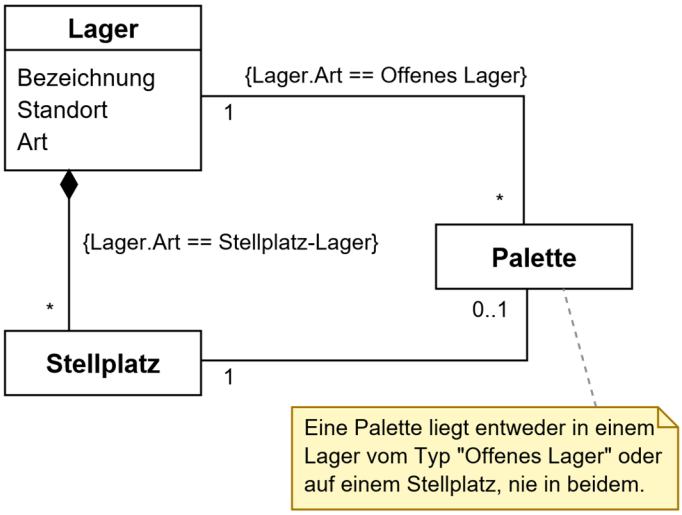


Abbildung 192: Das Lager-Beispiel ohne Vererbung (nach [Balz11]).

Dass man mit weniger Klassen auskommt, vereinfacht das Modell. Andererseits muss man beim Implementieren der Methoden für das Zuordnen von Paletten und das Anlegen von Stellplätzen die formulierten Bedingungen überprüfen. Im Ursprungsmodell wäre es von vornherein gar nicht möglich, eine Palette direkt einem Stellplatz-Lager zuzuordnen, da es keine direkte Assoziation zur Klasse Stellplatz-Lager gibt.

Das Lager-Beispiel verletzt das weiter vorne vorgestellte Offen-Geschlossen-Prinzip. Bei dem Beispiel mit den unterschiedlichen Formen in Kapitel 15.1 wurden anstelle eines Typ-Attributs unterschiedliche Klassen für die verschiedenen Formen eingeführt. Welche Lösung besser ist, hängt vom konkreten Fall ab. Wenn man sich ziemlich sicher sein kann, dass mit keinen weiteren Typen von Lagern zu rechnen ist, kann die Lösung aus Abbildung 192 sinnvoll sein. Können hingegen – wie im Fall der geometrischen Formen – weitere Typen hinzukommen, so hat das Offen-Geschlossen-Prinzip den in Kapitel 15.1 diskutierten Vorteil der einfachen Erweiterbarkeit ohne Änderungen im vorhandenen Code.

Ein ganz ähnlicher Fall findet sich in der Fallstudie der Geräteverwaltung. Der Kommentar in dem in Abbildung 193 gezeigten Modell-Ausschnitt besagt, dass bei einem Geräte-Objekt entweder das Attribut „*istAusleihbar*" den Wert „*true*" hat, oder dass es einem Mitarbeiter und ggf. einem Raum zugeordnet ist. Nicht explizit aufgeführt ist die weitere Bedingung, dass ein Geraet nur dann einer Ausleihe zugeordnet sein darf, wenn „*istAusleihbar*" den Wert „*true*" hat.

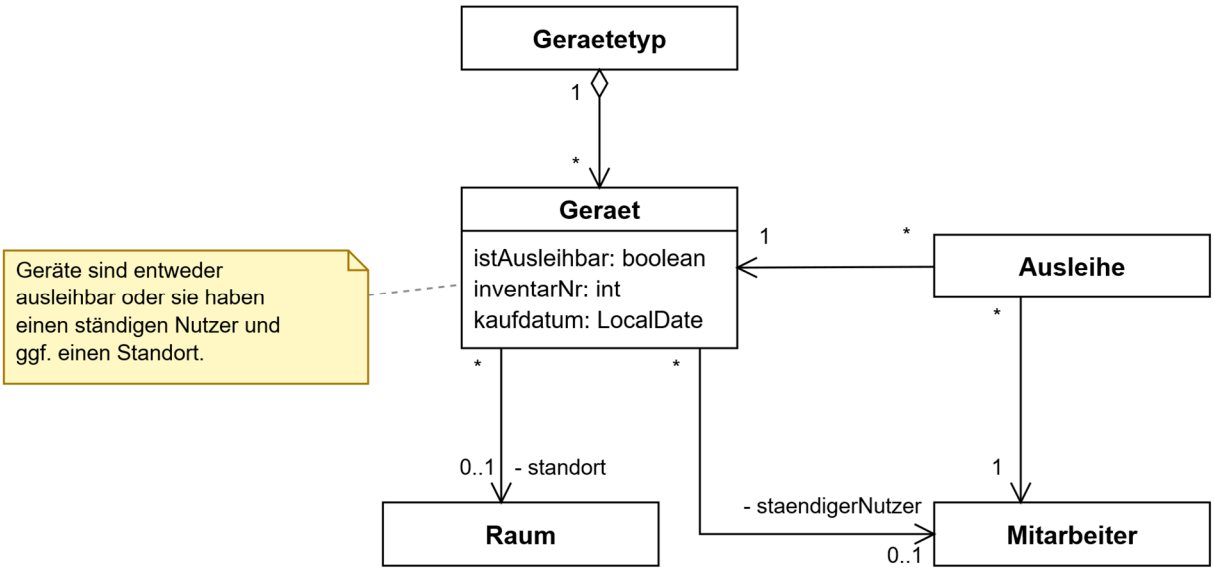


Abbildung 193: Verschieden Arten von Geräten im Fallbeispiel – ohne Vererbung

Auch dieser Sachverhalt lässt sich mit Hilfe von Vererbung darstellen. In Abbildung 194 ist die Klasse „*Geraet*" abstract. Sie hat die zwei konkreten Unterklassen „*FestZugeordnetesGeraet*" und „*AusleihbaresGeraet*". Ausleihen können nun nur Objekte der Klasse „*AusleihbaresGeraet*" zugeordnet haben, und nur Objekte der Klasse „*FestZugeordnetesGeraet*" können einen Mitarbeiter und ggf. einen Raum zugeordnet haben.

Da es ansonsten aber nicht viele Unterschiede zwischen den beiden Klassen gibt und auch nicht mit weiteren Geräte-Typen zu rechnen ist, kann das ursprüngliche Modell als einfachere Lösung ohne weiteres beibehalten werden.

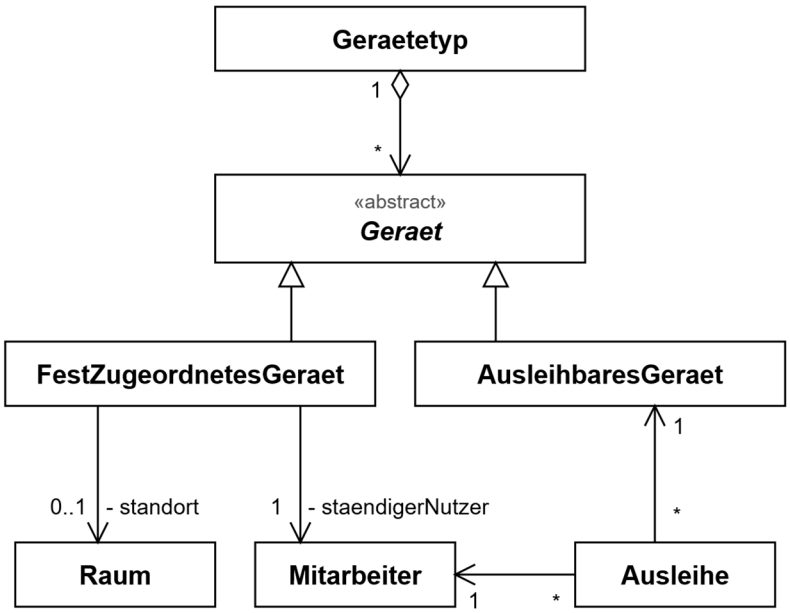


Abbildung 194: Verschieden Arten von Geräten im Fallbeispiel – mit Vererbung

15.2.3 Stereotypen

Schon mehrfach wurden sogenannte „Stereotypen" zur Klassifizierung von Klassen, Beziehungen oder Methoden verwendet. Dabei handelt es sich um ein UML-Konstrukt, mit dem Elemente von UML-Modellen nach beliebigen Kriterien eingeteilt und damit näher spezifiziert werden können. Die Bezeichnungen von Stereotypen werden in französische Anführungszeichen eingeschlossen: «*Bezeichnung Stereotyp*».

In der UML gibt es bereits eine Reihe vordefinierter Stereotypen, wie z. B. «*include*» oder «*extend*» für die Beziehungen in Use-Case-Diagrammen, «*Enumeration*» für Aufzählungstypen oder «*Create*» für Konstruktoren. Weitere Beispiele finden sich im Komponentendiagramm («*Artifact*», «*Manifestation*») und Verteilungsdiagramm («*Deployment*»).

Man kann aber auch eigene Stereotypen definieren. So wurden in Kapitel 10.2 die Klassen aus Analysemodellen mit Hilfe von Stereotypen als Boundary-, Control- oder Entity-Klassen gekennzeichnet. Dies ist zwar eine häufig verwendete Einteilung, aber nicht Teil des UML-Standards.

Im Entwurf werden oft weitere Stereotypen verwendet, mit denen Klassen entsprechend ihren Aufgaben eingeteilt werden, z. B. «*UI*» für Klassen, die das User Interface implementieren, oder «*Data Access Object*» für Klassen, die mit der Datenbank kommunizieren.

Abbildung 195 zeigt Beispiele für die Angabe von Stereotypen. Je nach Bedarf lassen sich beliebige weitere Stereotypen definieren. Insbesondere in großen Klassendiagrammen kann es hilfreich sein, geeignete Einteilungen zu treffen. Dabei sollte man innerhalb des Entwicklungsteams Vereinbarungen treffen, welche Stereotypen man einsetzt und was sie bedeuten.

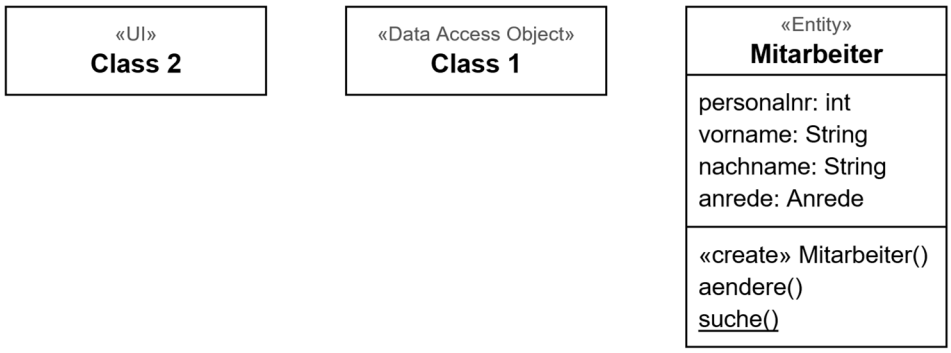


Abbildung 195: Beispiele für Stereotypen

15.2.4 Objektverwaltung

Zu den Aufgaben der Objektverwaltung gehören u. a.:

- Auflisten aller Objekte einer Klasse
- Suche über alle Objekte
- Sonstige benötigte Funktionen, die alle Objekte einer Klasse betreffen

In der Analyse kümmert man sich meist nicht explizit um die Objektverwaltung. Man geht davon aus, dass die hierfür benötigten Funktionen zur Verfügung stehen werden, wenn sie benötigt werden. Ggf. modelliert man benötigte Methoden als Klassenmethoden, wie z. B. die Methode „*suche( )*" in Abbildung 196.

Im Entwurf muss man sich nun konkret darum kümmern, wie die Objektverwaltung realisiert wird. So verfügt eine Klasse beispielsweise nicht von selbst über eine Liste all ihrer Objekte, die man als Grundlage für eine Suche verwenden könnte. Vielmehr muss man eine solche Liste selbst implementieren.

Eine Möglichkeit besteht darin, in der Klasse selbst ein statisches Attribut, d. h. ein Klassenattribut, vorzusehen, das die Liste der Objekte aufnimmt, wie dies in Abbildung 197 gemacht wurde. Die Methode „*suche( )*" ist hier ebenfalls als Klassenmethode modelliert. Für die konkrete Implementierung muss man dieser Methode noch geeignete Parameter hinzufügen, die die Suchkriterien aufnehmen können. Die Suchmethode kann dann die statische Liste „*alleMitarbeiter*" durchgehen und jedes Mitarbeiter-Objekt daraufhin überprüfen, ob es den Suchkriterien entspricht.

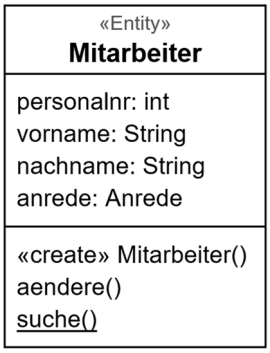


Abbildung 196: „suche( )" ist eine Aufgabe der Objektverwaltung

Beim Anlegen eines neuen Objekts muss dieses in die Liste „*alleMitarbeiter*“ eingetragen werden, damit es bei künftigen Suchen mitberücksichtigt wird.

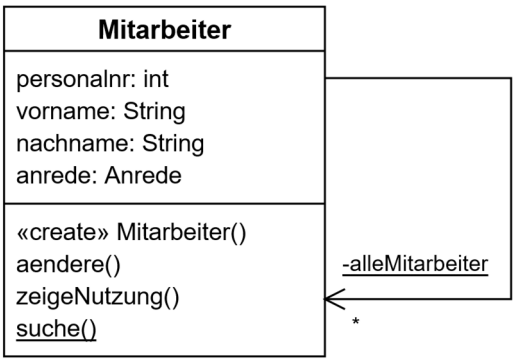


Abbildung 197: Realisierung der Objektverwaltung in der Klasse selbst

Anstatt die Objektverwaltung mit Hilfe statischer Attribute und Methoden in der betreffenden Klasse selbst zu implementieren, kann man auch eine separate Klasse definieren, die diese Aufgaben übernimmt. Von dieser Klasse sollte nur ein Objekt existieren. Dieses verfügt über eine Liste der verwalteten Objekte. Da dieses Objekt eine Art Behälter oder Container für die verwalteten Objekte darstellt, wird eine solche Verwaltungsklasse auch als Container-Klasse bezeichnet.

In Abbildung 198 wird die im Geräteverwaltungsbeispiel bereits existierende Control-Klasse „*Mitarbeiterverwaltung*“ als Container-Klasse verwendet. Man sieht auch, dass diese Klasse eine Methode zum Anlegen von Mitarbeitern enthält (auch als „Fabrik-Methode“ bezeichnet). Diese Methode ruft den Konstruktor von „*Mitarbeiter*“ auf und kümmert sich darum, dass das neu angelegte Mitarbeiter-Objekt in die Liste „*alleMitarbeiter*“ eingetragen wird.

Der Konstruktor von „Mitarbeiter“ sollte prinzipiell nur von der Methode „*legeMitarbeiterAn*“ aufgerufen werden. In Java lässt sich die Sichtbarkeit des Konstruktors allerdings nicht auf eine andere Klasse einschränken. Eine Einschränkung der Sichtbarkeit von „public“ auf „package“ würde zumindest den versehentlichen direkten Aufruf von außerhalb des Package verhindern.

Die Verlagerung der Objektverwaltung in eine eigene Container-Klasse entspricht dem Prinzip, dass jede Klasse nur eine Verantwortlichkeit haben sollte. Die Mitarbeiterklasse selbst ist in dem Beispiel für alles verantwortlich, was einen einzelnen Mitarbeiter betrifft, die Containerklasse „Mitarbeiterverwaltung“ ist hingegen für objektübergreifende Aufgaben zuständig.

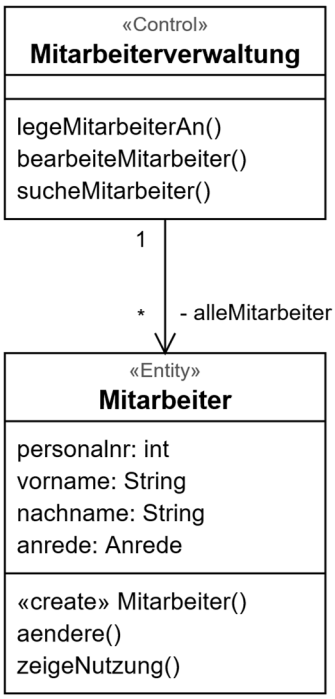


Abbildung 198: Containerklasse für die Objektverwaltung

15.2.5 Verfeinerung von Assoziationen

Wie Assoziationen prinzipiell implementiert werden können, wurde bereits in Kapitel 6 behandelt. Im Entwurf muss man unter anderem entscheiden, welche Navigationsrichtungen realisiert werden sollen, ob man redundante Assoziationen zur Optimierung von Zugriffspfaden einführt, und welche Datenstrukturen für Assoziationen mit der Multiplizität „*“ genutzt werden sollen.

Navigationsrichtungen

Bezüglich der Navigationsrichtungen ist insbesondere zu prüfen, welche Assoziationen unidirektional implementiert werden können. Unidirektionale Assoziationen sind nur in eine Richtung navigierbar, im Gegensatz zu bidirektionalen Assoziationen, die in beide Richtungen navigierbar sind. Unidirektionale Assoziationen sind einfacher zu implementieren, da nur eine der beiden beteiligten Klassen ein Attribut zur Speicherung der zugeordneten Objekte benötigt.

Insbesondere bei Assoziationen zwischen Klassen aus unterschiedlichen Paketen ist es wünschenswert, dass diese Assoziationen nur in eine Richtung navigierbar sind. Existieren bidirektionale Assoziationen zwischen zwei Paketen, so sind die beiden Pakete gegenseitig voneinander abhängig. Im Sinne einer losen Kopplung ist es aber wünschenswert, dass die Abhängigkeiten möglichst nur in eine Richtung bestehen (vgl. Kapitel 14.3).

Optimierung von Zugriffspfaden

Auch wenn man im Allgemeinen versucht, Redundanzen zu vermeiden, kann es gelegentlich hilfreich sein, redundante Assoziationen hinzuzufügen, um Zugriffspfade zu optimieren. Ein Beispiel, wo dies nützlich sein kann, findet sich in der Fallstudie zur Geräteverwaltung.

Darin gibt es die Control-Klasse „*Reservierungsverwaltung*", die auch als Container-Klasse für die Objekte der Klasse „*Reservierung*" dient (vgl. Abbildung 199). Sie verfügt daher über die Liste „*alleReservierungen*", in die jedes Reservierungsobjekt eingetragen wird.

Im Laufe der Zeit können sehr viele Reservierungsobjekte angelegt werden. Die meisten der entsprechenden Reservierungen liegen in der Vergangenheit und sind nicht mehr von besonderem Interesse. Eine Möglichkeit wäre es, diese Reservierungen zu löschen. Eventuell ist dies aber nicht gewünscht, weil man z. B. Auswertungen über Reservierungen der vergangenen Monate und Jahre durchführen möchte. Dann müssen auch die Reservierungen der Vergangenheit erhalten bleiben.

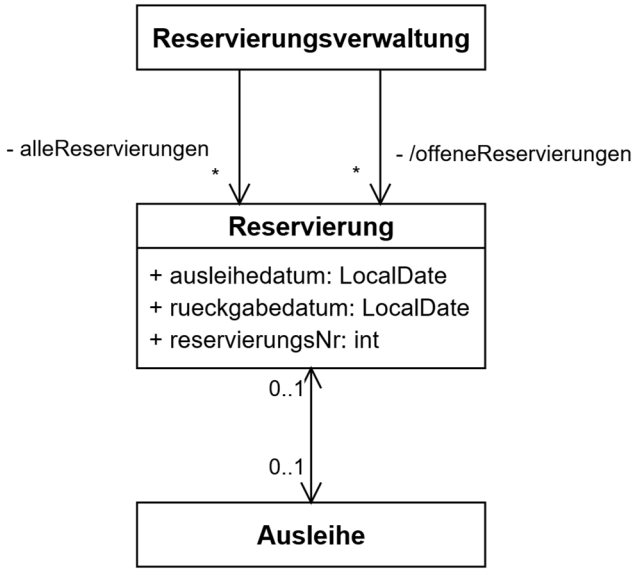


Abbildung 199: Redundante Assoziation für einfacheren Zugriff

Für viele Methoden sind aber nur die offenen Reservierungen interessant, bei denen zumindest das Rückgabedatum in der Zukunft liegt. Möchte man beispielsweise wissen, welche Geräte ein bestimmter Mitarbeiter reserviert hat, oder zu welchen Zeiten bestimmte Geräte verfügbar sind, sind dafür nur die offenen Reservierungen von Bedeutung.

Da nach einiger Zeit wesentlich mehr vergangene Reservierungen als offene Reservierungen im System vorhanden sind, muss jedes Mal die gesamte Liste der Reservierungen durchgegangen werden um für jede Reservierung zu überprüfen, ob sie noch offen ist. Das ist ein großer Aufwand.

Um sich diesen Aufwand zu ersparen, wurde in Abbildung 199 eine zweite Liste mit den offenen Reservierungen eingefügt. Sie enthält nur die Reservierungen, deren Rückgabedatum in der Zukunft liegt. Wird eine neue Reservierung angelegt, so wird sie zunächst in beide Listen

eingetragen. Ist das Rückgabedatum einer Reservierung abgelaufen, so muss sie aus der Liste der offenen Reservierungen entfernt werden. Hierfür muss man eine entsprechende Methode programmieren, die z. B. einmal täglich automatisch ausgeführt wird.

Es entfällt nun das mühsame Durchsuchen der großen Gesamtliste. Stattdessen können sämtliche Methoden, die nur die offenen Reservierungen benötigen, einfach auf die – wesentlich kleinere – Liste „*offeneReservierungen*" zugreifen.

In Abbildung 199 wurde die Assoziation durch einen Schrägstrich vor dem Rollennamen als abgeleitete Assoziation gekennzeichnet (vgl. Kapitel 7.1), d. h. man kann die durch diese Assoziation repräsentierte Information prinzipiell auch auf anderem Wege herausfinden – in diesem Fall durch das Durchsuchen der Liste „*alleReservierungen*".

Auswahl geeigneter Collections

Für die Implementierung von Assoziationen mit der Obergrenze „*" werden in Java zumeist Collections aus der Bibliothek „java.util" verwendet. In den vorangehenden Kapiteln wurde meist die „ArrayList" verwendet, die einem Array ähnelt, aber beliebig viele Elemente aufnehmen kann. Je nach gewünschten Eigenschaften kann es aber auch sinnvoll sein, andere Typen von Collections zu verwenden.

Mit Hilfe der Java Generics lassen sich Collections Typ-sicher machen So besagt die Angabe <String> hinter „ArrayList" in der zweiten Zeile von Listing 41, dass nur Objekte vom Typ „String" in die Liste a2 aufgenommen werden können. Hingegen können in die Liste a1 völlig beliebige Objekte eingefügt werden. Möchte man ein Objekt aus dieser Liste verwenden, so muss man erst einen Cast in den zutreffenden Typ durchführen.

Bei einer Assoziation möchte man sicherstellen, dass nur Objekte der verbundenen Klasse verwendet werden können. Daher sollte man immer Typ-sichere Collections verwenden.

```
ArrayList a1              = new ArrayList();
ArrayList<String> a2      = new ArrayList<String>();
ArrayList<Integer> a3     = new ArrayList<Integer>();
ArrayList<MeineKlasse> a4 = new ArrayList<MeineKlasse>();
Vector<String> v1         = new Vector<String>();
```

Listing 41: Beispiele für die Verwendung von Java Collections

Zu beachten ist auch, dass zu Beginn immer explizit ein Objekt der zu verwendeten Collection angelegt werden muss. Dies kann direkt bei der Deklaration der Attribute erfolgen, wie in Listing 41.

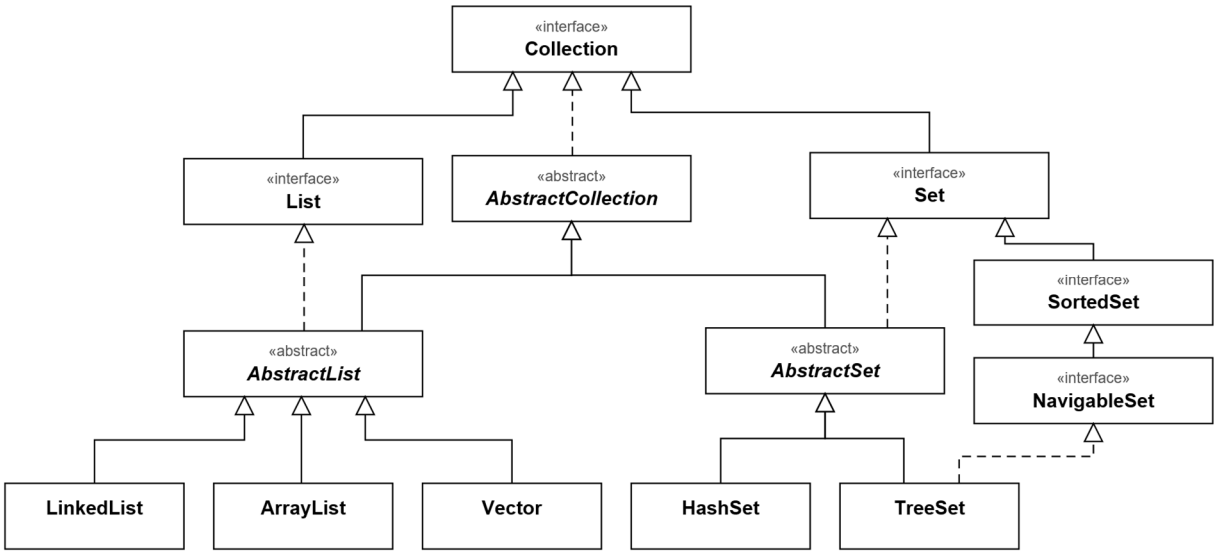


Abbildung 200: Java Collections-Hierarchie (Ausschnitt)

Abbildung 200 zeigt die Hierarchie der Java Collections als Klassendiagramm. Es enthält die folgenden Interfaces:

- Collection
  Dies ist das oberste Interface, hier sind einige Basisfunktionalitäten zum Einfügen von Elementen, Erzeugen eines Iterators u. ä. deklariert, über die alle Collections verfügen.

- List
  Collections, die dieses Interface implementieren, definieren eine Reihenfolge ihrer Elemente und ermöglichen den Zugriff darauf über einen Index. Doppelte Einträge sind in Listen erlaubt.

- Set
  In einem Set sind keine doppelten Einträge möglich.

- SortedSet
  Ein Set, dessen Elemente eine Reihenfolge haben.

- NavigableSet
  Hier werden einige Navigiermethoden deklariert.

Die abstrakten Klassen „*AbstractCollection*“, „*AbstractList*“ und „*AbstractSet*“ implementieren einige Basisfunktionen, wobei „*AbstractList*“ und „*AbstractSet*“ über die Eigenschaften der implementierten Interfaces „*List*“ bzw. „*Set*“ verfügen.

Als Unterklassen von „*AbstractList*“ stehen die folgenden konkreten Klassen zur Verfügung:

- LinkedList
  Hierbei handelt es sich um eine doppelt verkettete Liste. Sie bietet einen schnelleren wahlfreien Zugriff auf einzelne Elemente als die ArrayList.

- ArrayList
  Ein dynamisches Array. Im Vergleich zur LinkedList erfolgt das Einfügen und Löschen von Elementen schneller.

- Vector
  Ähnlich wie eine ArrayList. Hier sind alle Methoden synchronisiert, trotzdem ist nicht in allen Fällen Thread-Sicherheit garantiert.

„*AbstractSet*" hat die folgenden konkreten Unterklassen:

- HashSet
  Unsortierte Menge

- TreeSet
  SortierteMenge

Zur Auswahl der geeigneten Collections muss man sich überlegen, welche Eigenschaften man benötigt, wie z. B. eine Reihenfolge der Elemente oder die Möglichkeit doppelter Einträge. Zudem sind insbesondere bei großen Listen Speicherbedarf und Performance zur berücksichtigen. Je nach Größe und verwendeten Methoden eignen sich unterschiedliche Implementierungen.

Soll ein Zugriff über bestimmte Schlüssel erfolgen, so eignen sich anstelle der Collections Implementierungen des Interface „*Map*", z. B. „*HashMap*" oder „*TreeMap*".

Andere Implementierungsmöglichkeiten für Assoziationen

In den bisherigen Kapiteln wurden Assoziationen immer dadurch implementiert, dass mindestens eine der beiden Klassen über ein Attribut verfügt, das ein oder mehrere Elemente der anderen Klasse aufnehmen kann. Allerdings ist es keineswegs vorgegeben, wie Assoziationen implementiert werden müssen. Es muss lediglich möglich sein, Objekte der beteiligten Klassen entsprechend der Multiplizitäten miteinander in Beziehung zu setzen.

Gelegentlich kann es vorkommen, dass man Objekte von Klassen miteinander in Beziehung setzen möchte, dabei aber die beteiligten Klassen nicht verändern kann oder will. Beispielsweise kann es sich um Klassen aus Standardbibliotheken handeln, die auch an anderer Stelle verwendet werden und nicht verändert werden dürfen. In diesem Fall ist für die Assoziation eine andere Art der Implementierung nötig.

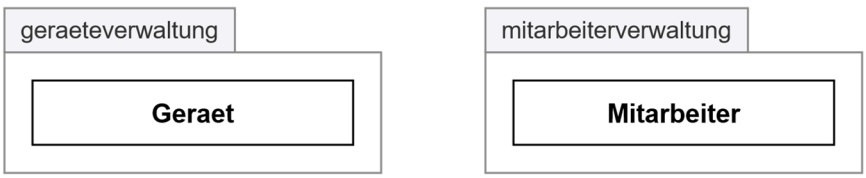


Abbildung 201: Es soll eine Assoziation implementiert werden ohne die beteiligten Klassen zu verändern

In Abbildung 201 sollen Geraet und Mitarbeiter miteinander verbunden werden, ohne Änderungen in den Paketen „geraeteverwaltung“ und „mitarbeiterverwaltung“ vorzunehmen.

Abbildung 202 zeigt eine mögliche Lösung. Es wird eine Klasse „*Geraetenutzung*“ eingeführt, wobei jedes Objekt dieser Klasse mit genau einem Gerät und genau einem Mitarbeiter verbunden ist. Ein Geraetenutzung-Objekt repräsentiert somit eine Zuordnung eines Geräts zu einem Mitarbeiter. Die Container-Klasse „*Geraetenutzungen*“, von der es nur ein Objekt geben darf, enthält eine Liste mit allen Geraetenutzung-Objekten. Soll nun ein Gerät einem Mitarbeiter zugeordnet werden, so muss man ein neues Geraetenutzung-Objekt anlegen, diesem das Geraet- und das Mitarbeiter-Objekt zuordnen, und es schließlich in die Liste „*geraetenutzungen*“ in der Container-Klasse „*Geraetenutzungen*“ eintragen.

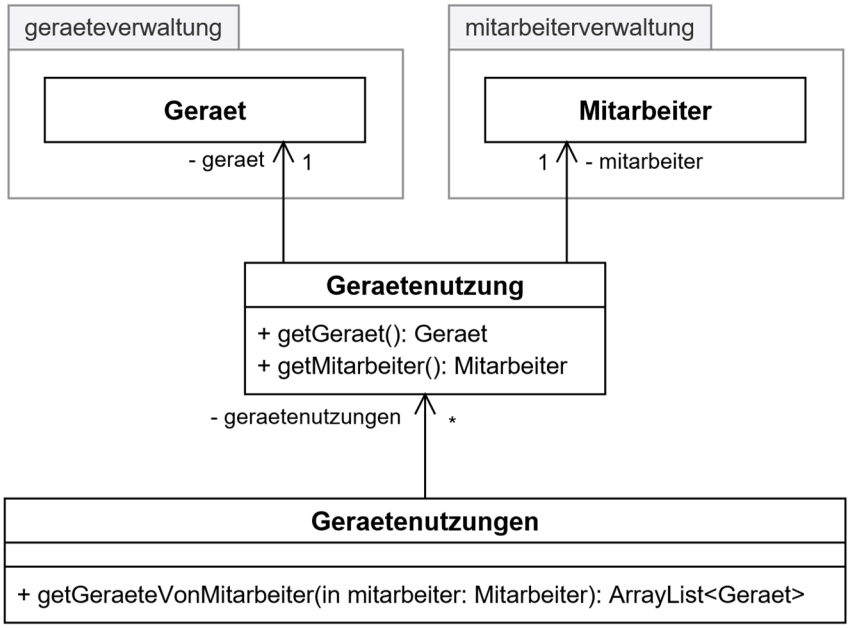


Abbildung 202: Implementierung einer Assoziation über Assoziationsobjekte

Über die Klasse „*Geraetenutzungen*“ lassen sich nun auch entsprechende Abfragen realisieren. Als Beispiel ist eine Methode „*getGeraeteVonMitarbeiter( )*“ eingetragen. Diese Methode bekommt das Mitarbeiter-Objekt als Parameter übergeben und gibt eine Liste mit den Geräten zurück, die diesem Mitarbeiter zugeordnet sind.

Hierzu muss die Methode alle Geraetenutzung-Objekte durchgehen und jeweils über die Methode „*getMitarbeiter( )*“ herausfinden, ob der gewünschte Mitarbeiter zugeordnet ist. Ist dies der Fall, so wird das zugeordnete Gerät in die Ergebnisliste eingetragen.

15.2.6 Verfeinerung von Attributen und Methoden

Auch die Attribute und Methoden sind im Vergleich zu den Analysemodellen weiter auszuarbeiten. Zu den festzulegenden Details gehören u. a. Sichtbarkeiten, Datentypen, Default-Werte, Parameter und Rückgabetypen.

In manchen Fällen kann es sinnvoll sein, abgeleitete Attribute zur Speicherung von Zwischenergebnissen einzuführen, so dass z. B. komplexe Berechnungen nicht ständig neu durchgeführt werden müssen.

Weiterhin werden zusätzlich benötigte Methoden eingefügt, wie z. B. getter- und setter-Methoden. Sehr komplexe Methoden sollten in mehrere einfache Methoden zerlegt werden.

Weitere Festlegungen und Überlegungen betreffen z. B. die Gültigkeitsbereiche der Datentypen, Gültigkeitsprüfungen für Parameter und die Definition von Exceptions zur Ausnahmebehandlung.

Als Beispiel wird die Verarbeitung von Geldbeträgen betrachtet, wie sie in vielen Programmen erforderlich ist. Die unreflektierte Nutzung der Datentypen float oder double mit Rundung der Berechnungsergebnisse führt aufgrund der Eigenschaften der numerischen Mathematik zu falschen Ergebnissen.

Besser ist die Nutzung von „*BigDecimal*" aus dem Paket java.math, wie in Listing 42. In der letzten Zeile wird eine String-Darstellung für Deutschland erzeugt – mit Komma und Eurozeichen.

```
import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Locale;


// ...

BigDecimal b = new BigDecimal("5712.12").add(new BigDecimal("345.31"));
String bString = NumberFormat.getCurrencyInstance(Locale.GERMANY).format(b);
```

Listing 42: Verwendung von Big Decimal

Für sehr umfangreiche Berechnungen ist BigDecimal möglicherweise nicht performant genug. In diesem Fall muss man sich andere Lösungen überlegen. Z. B. kann man Geldbeträge intern prinzipiell als Cent-Beträge abbilden und hierfür den Datentyp int verwenden.

16 Entwurfsmuster

In Kapitel 10.4 wurden einige Analysemuster zur Modellierung typischer, häufig auftretender Sachverhalte vorgestellt. Auch für den objektorientieren Entwurf existieren Muster mit bewährten Lösungen für häufige Problemstellungen.

Populär wurden diese durch das ursprünglich 1994 erschienene Buch „Design Patterns – Elements of Reusable Object-Oriented Software“ [GaHe14], das immer noch das maßgebliche Werk zu diesem Thema darstellt. Die darin beschriebenen Muster werden nach wie vor erfolgreich eingesetzt. Im Laufe der Zeit kamen weitere Muster hinzu. Im Internet lassen sich umfangreiche Sammlungen von Entwurfsmustern für verschiedenste Fragestelltungen finden. Im Folgenden werden einige allgemein anwendbare, häufig verwendete Muster besprochen.

16.1 Fassade

Das Fassaden-Muster dient der Unterstützung des Entwurfziels der losen Kopplung. So sollen z. B. einzelne Pakete möglichst lose miteinander gekoppelt sein (vgl. Kapitel 14.3).

Falls es sehr viele direkte Assoziationen oder Methodenaufrufe zu den einzelnen Klassen eines Pakets gibt, führt dies zu einer engen Kopplung (Abbildung 203). Diese Kopplung kann reduziert werden, indem man eine Fassadenklasse einführt, die für die komplette Kommunikation mit dem Paket zuständig ist (Abbildung 204). Von außerhalb des Pakets werden nur noch die Methoden der Fassadenklassen aufgerufen. Die Fassadenklasse-Methoden rufen dann ihrerseits die Methoden der Klassen innerhalb des Pakets auf. Falls diese Methoden Package-Sichtbarkeit erhalten, ist es gar nicht mehr möglich, sie direkt von außerhalb aufzurufen.

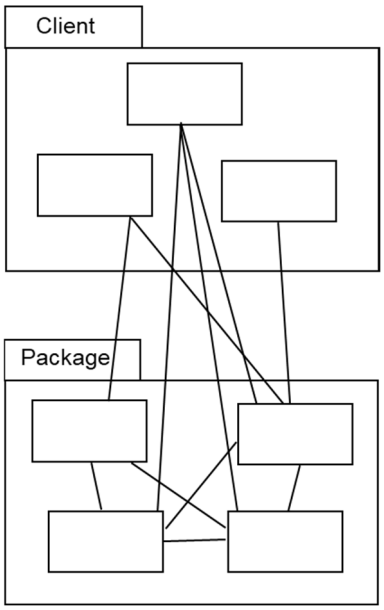


Abbildung 203: Enge Kopplung zweier Pakete

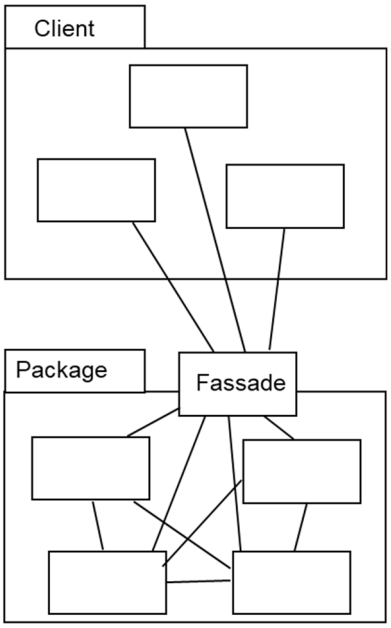


Abbildung 204: Entkopplung durch Fassadenklasse

Außerhalb des Pakets muss jetzt nur noch die Fassadenklasse mit ihren Methoden bekannt sein. Solange sich an der Fassadenklasse nichts ändert, haben Änderungen im Inneren des Pakets keine Auswirkungen auf andere Pakete.

Gemäß dem in Kapitel 10 eingeführten Boundary-Control-Entity-Schema gehen Anfragen an die Control-Klassen, die sie wiederum an die Entity-Klassen weiterleiten. Insofern können die Control-Klassen in der Fallstudie zur Geräteverwaltung als Fassaden-Klassen aufgefasst werden (Abbildung 205).

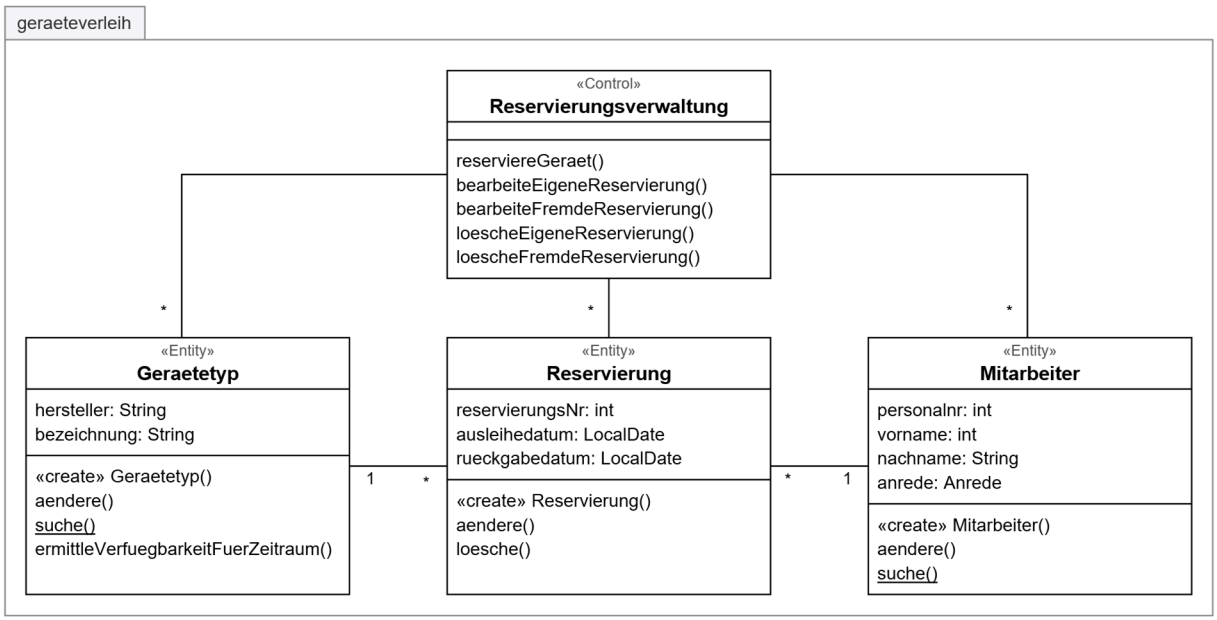


Abbildung 205: Die Control-Klasse „Reservierungsverwaltung“ stellt eine Fassadenklasse für das Paket „geraeteverleih“ dar.

16.2 Singleton

Ein Singleton ist eine Klasse, von der es nur ein Objekt geben soll. Dies ist z. B. für Container-Klassen gewünscht, deren einziges Objekt alle Objekte einer anderen Klasse verwaltet. Auch für zentrale Aufgaben, wie z. B. die Verwaltung der Datenbankverbindung, kann es sinnvoll sein, ein Singleton zu verwenden.

Singletons sollten allerdings nur dort eingesetzt werden, wo dies wirklich erforderlich ist. Ein Singleton entspricht einer globalen Variablen. Macht man alle möglichen Werte über Singletons systemweit bekannt, so widerspricht dies dem Geheimnisprinzip. In der Folge kann es zu unerwünschten Seiteneffekten kommen, zudem leidet die Änderbarkeit.

In Abbildung 206 wird gezeigt, wie man sicherstellen kann, dass es nur ein einziges Objekt von einer Klasse geben kann. Das statische Attribut „*einzigesExemplar*" enthält das einzige Exemplar, d. h. Objekt, der Klasse „*Beispiel*". „*x*" und „*y*" sind Beispiele für sonstige Attribute.

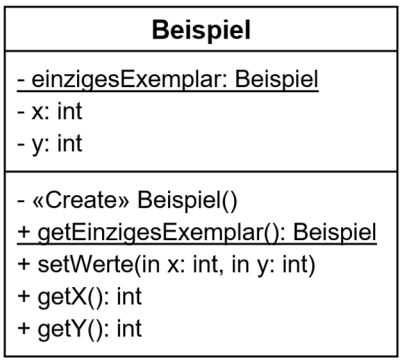


Abbildung 206: Singleton

Der Konstruktor ist private (erkennbar am Minuszeichen), damit man ihn nicht direkt von außerhalb der Klasse aufrufen kann. Sonst könnte man unerwünschterweise einfach mehrere Objekte der Klasse erstellen. Die statische Methode „*getEinzigesExemplar( )*" liefert das einzige existierende Objekt der Klasse zurück. „*setWerte( )*", „*getX( )*" und „*getY( )*" sind Beispiele für sonstige Methoden.

Für die Erzeugung des einzigen Objekts gibt es zwei Möglichkeiten. Zum einen kann es direkt bei der Deklaration der statischen Variablen instanziiert werden:

```
private static Beispiel einzigesExemplar = new Beispiel();
```

Meist reicht das aus. In manchen Fällen kann es nachteilig sein, dass Exceptions des Konstruktors nicht aufgefangen werden, und dass eventuell vom Konstruktor benötigte Objekte beim Programmstart noch nicht vorhanden sind.

Die zweite Möglichkeit besteht darin, das einzige Objekt der Klasse zu Beginn noch nicht zu instanziieren, sondern erst bei Bedarf. Bei der Deklaration des Attributs wird hier noch kein Objekt erzeugt:

```
private static Beispiel einzigesExemplar = null;
```

In diesem Fall muss die Methode „*getEinzigesExemplar( )*“ zunächst überprüfen, ob „*einzigesExemplar*“ schon ein Objekt enthält. Wenn nicht, erzeugt sie es durch Aufruf des Konstruktors. Falls das Singleton Thread-sicher sein soll, muss man die Zugriffe synchronisieren, sonst könnten unter Umständen mehrere Objekte angelegt werden.

16.3 Strategie

Das Strategie-Muster definiert eine Familie von Algorithmen, kapselt jeden einzelnen Algorithmus und macht sie austauschbar.

Es wird folgendes Beispiel betrachtet: In einem elektronischen Kalender sollen unter anderem die Feiertage angezeigt werden. Es soll möglich sein, ihn je nach Land so zu konfigurieren, dass jeweils die richtigen Feiertage angezeigt werden. Es soll auch leicht möglich sein, ihn für weitere Länder anzupassen.

Hierzu wird für die Ermittlung der Feiertage eine abstrakte Klasse oder ein Interface definiert. Für jedes Land wird eine konkrete Unterklasse angelegt. Je nach gewünschtem Land wird dem Kalender ein Objekt der zugehörigen Unterklasse zugeordnet. Dies ist in Abbildung 207 dargestellt. Die Methode „*konfiguriereFeiertage( )*“ dient dazu, einem Kalenderobjekt ein Feiertagsermittlungsobjekt des gewählten Landes zuzuordnen.

Je nachdem, was für ein konkretes Feiertagsermittlungsobjekt zugeordnet ist, wird in der Methode „*erstelleJahreskalender( )*“ die entsprechende Methode „*ermittleFeiertage( )*“ von „*FeiertagsermittlungDeutschland*“ oder „*FeiertagsermittlungOesterreich*“ ausgeführt. Entsprechende Klassen für weitere Länder lassen sich ohne Änderung vorhandenen Codes hinzufügen.

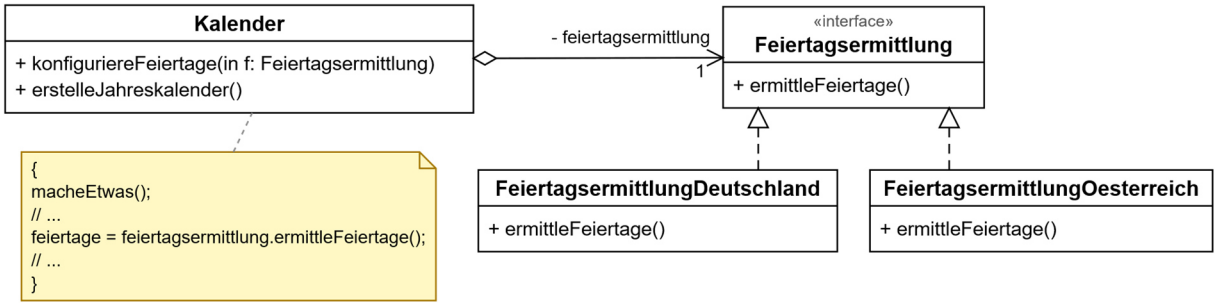


Abbildung 207: Strategiemuster

16.4 Schablonenmethode

Gelegentlich benötigt man in mehreren Klassen sehr ähnliche Methoden. Wären die Methoden genau gleich, könnte man eine gemeinsame Oberklasse definieren und die Methode dorthin verlagern.

Wenn es sich aber um Methoden wie in Abbildung 208 handelt, die neben vielen identischen auch einige unterschiedliche Teile haben, ist dies nicht möglich. Trotzdem möchte man die identischen Teile nicht redundant in jeder Klasse haben, sondern nur an einer Stelle programmieren.

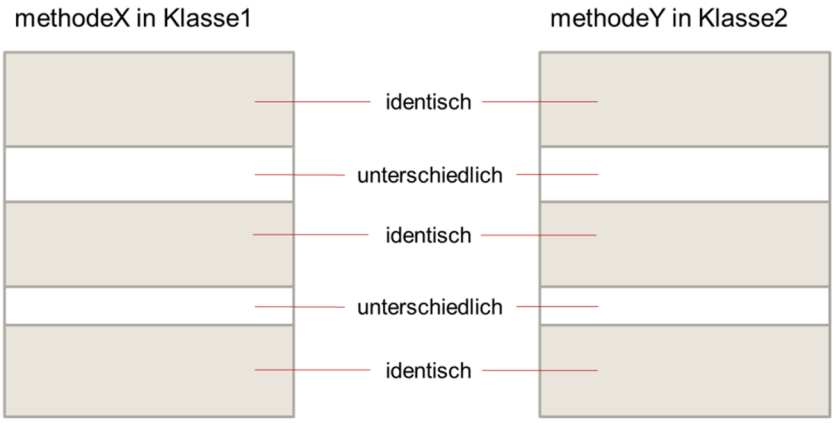


Abbildung 208: Zwei fast identische Methoden

Dies wird dadurch erreicht, dass man in einer abstrakten Oberklasse eine Schablonenmethode implementiert (Abbildung 209). Sie enthält die gemeinsamen Teile der verschiedenen Methoden.

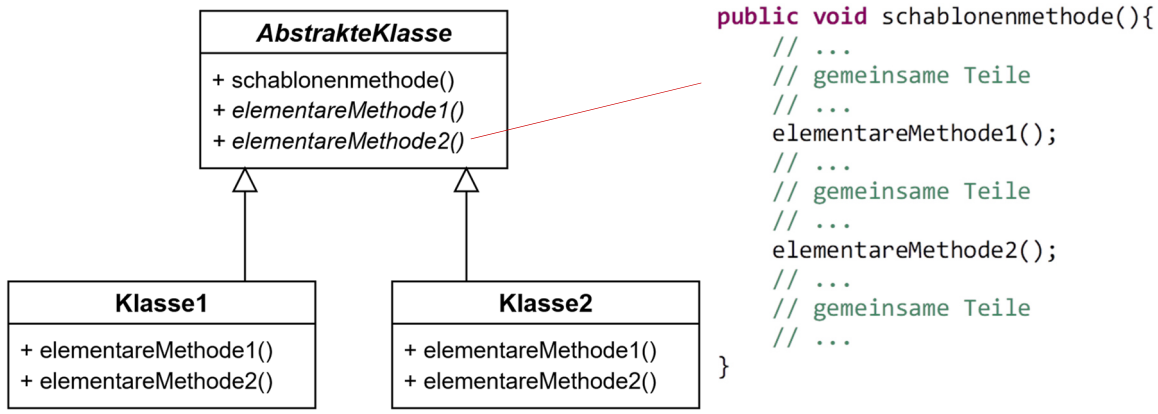


Abbildung 209: Schablonenmethode

An den Stellen, an denen es je nach Klasse Unterschiede in der Methode geben soll, werden abstrakte Methoden aufgerufen. Diese abstrakten Methoden müssen in den konkreten Unterklassen implementiert werden. Wird in dem Beispiel von Abbildung 209 die Methode *„schablonenmethode( )"* bei einem Objekt der *„Klasse1"* aufgerufen, so wird nach dem ersten gemeinsamen Teil die *„elementareMethode1( )"* aus der *„Klasse1"* aufgerufen. Anschließend folgt wieder ein gemeinsamer Teil, nach dem die *„elementareMethode2( )"* aus *„Klasse1"* ausgeführt wird.

Handelt es sich hingegen um ein Objekt der „*Klasse2*“, so werden an den entsprechenden Stellen die elementaren Methoden aus „*Klasse2*“ ausgeführt.

Die in der abstrakten Klasse implementierte „*schablonenmethode( )*“ stellt somit eine Schablone dar, bei der die zu ergänzenden „Lücken“ aus abstrakten Methoden bestehen. Sie werden beim Aufruf durch die entsprechenden Implementierungen der verwendeten konkreten Klasse ausgefüllt.

16.5 Fabrikmethode

Als Beispiel soll ein Framework für Anwendungen erstellt werden, die mehrere Dokumente anzeigen können. In diesem Framework gibt es eine abstrakte Klasse „*Dokument*“. Anwendungsentwickler können dieses Framework für ihre eigene Anwendung nutzen, indem sie Unterklassen von „*Dokument*“ erstellen.

In Abbildung 210 entwickelt ein Entwicklungsteam eine „*Anwendung1*“, die Dokumente einer Klasse „*Dokument1*“ verwendet. Ein anderes Team entwickelt eine „*Anwendung2*“. Darin werden Dokumente der Klasse „*Dokument2*“ genutzt. Beide nutzen das Framework, das ihnen eine Reihe von vordefinierten Methoden zum Umgang mit Dokumenten bereitstellt, so dass sie diese nicht selbst schreiben müssen.

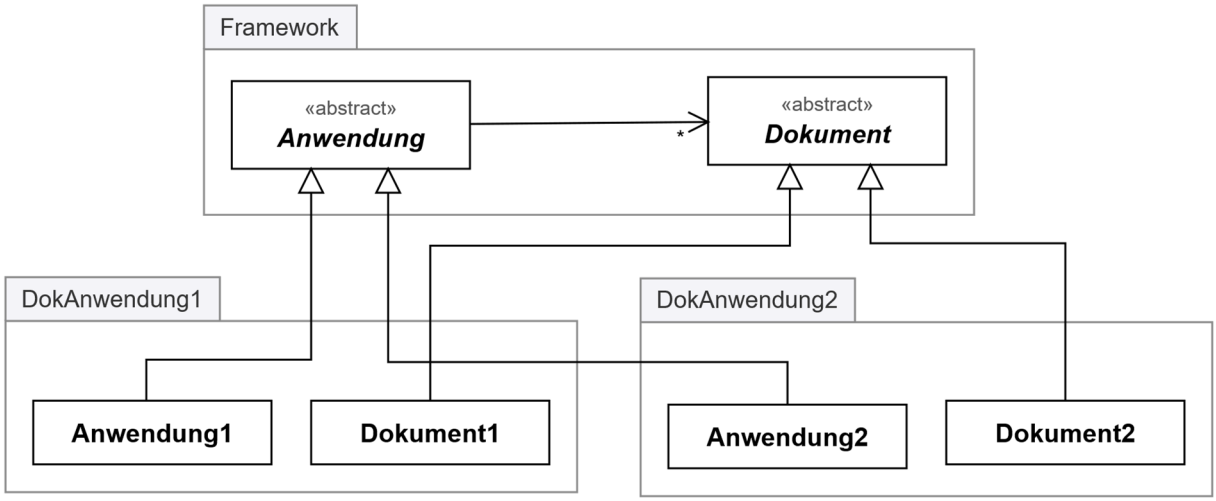


Abbildung 210: Fabrikmethode: Problemstellung

Das Framework soll auch die Möglichkeit bieten, neue Dokumente anzulegen. Dabei ergibt sich das Problem, dass die Klasse „*Dokument*“ abstrakt ist, weshalb von ihr keine Objekte angelegt werden können.

Das ist auch richtig so, denn letztlich werden ja Objekte der konkreten Klassen „*Dokument1*“ bzw. „*Dokument2*“ benötigt. Da diese im Framework aber nicht bekannt sind, weil sie erst später von den Nutzern des Frameworks definiert werden, kann die Framework-Klasse „*Anwendung*“ auch keine Objekte dieser konkreten Klassen anlegen.

Der Lösungansatz ist ähnlich wie bei der Schablonenmethode: Man verlagert die Unterklassen-spezifischen Aufgaben in abstrakte Methoden, die dann in den Unterklassen implementiert werden müssen. In diesem Fall erzeugen die betreffenden Methoden Objekte, weshalb sie Fabrikmethoden genannt werden.

In Abbildung 211 enthält die Klasse „Anwendung" die abstrakte Methode *„createDokument( )"*. Der Rückgabetyp ist *„Dokument"*. Die Unterklassen implementieren diese Methoden, indem sie mit Hilfe des Konstruktors ein Objekt der Klasse *„Dokument1"* bzw. *„Dokument2"* erzeugen und dieses zurückgeben. Da es sich um Unterklassen von *„Dokument"* handelt, sind sie mit dem Rückgabetyp *„Dokument"* kompatibel. Man kann also dort, wo als Rückgabe ein Objekt vom Typ *„Dokument"* erwartet wird, auch ein Objekt vom Typ *„Dokument1"* bzw. *„Dokument2"* zurückgeben.

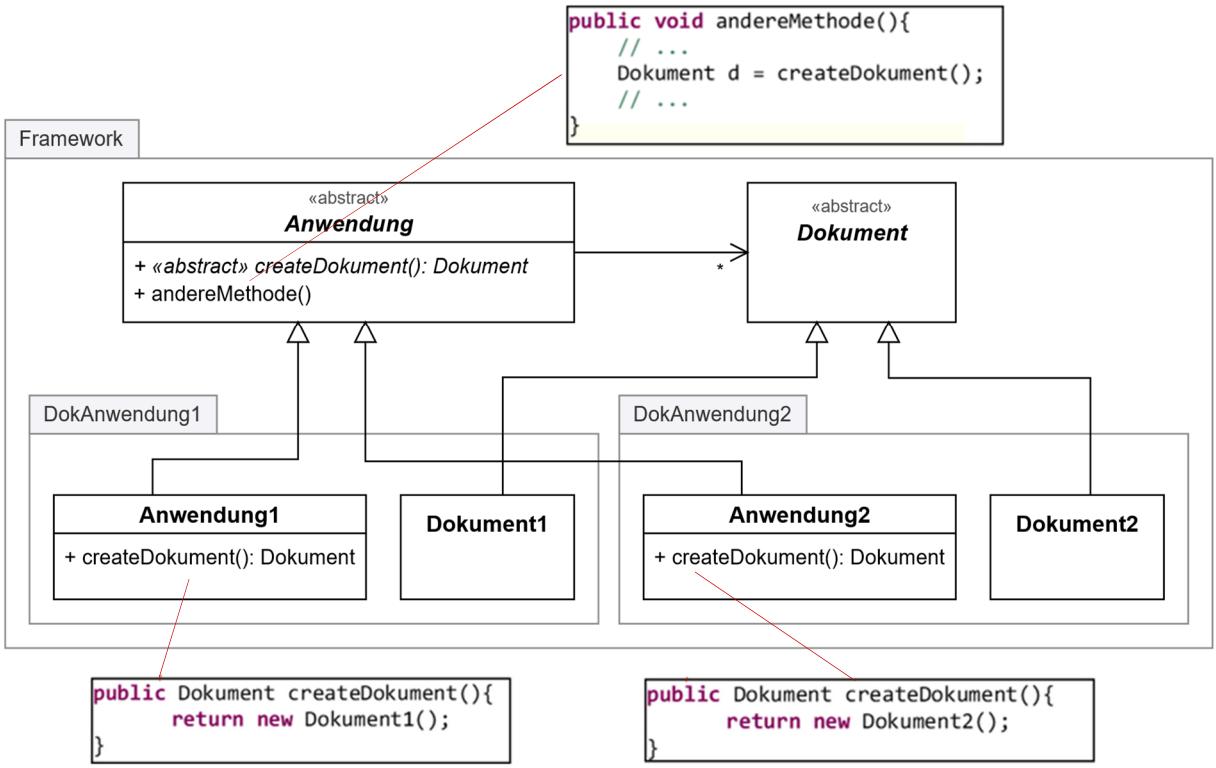


Abbildung 211: Fabrikmethode: Lösung

Wenn nun in einer anderen Methode der Klasse *„Anwendung"* ein Dokument erzeugt werden soll, ruft diese Methode die Fabrikmethode *„createDokument( )"* auf. Wird diese andere Methode nun z. B. bei einem Objekt der Klasse *„Anwendung1"* ausgeführt, so erzeugt *„createDokument( )"* ein Objekt vom Typ *„Dokument1"* und gib dieses zurück. Bei *„Anwendung2"* erfolgt dies in gleicher Weise, wobei ein Objekt vom Typ *„Dokument2"* zurückgegeben wird.

16.6 Kompositum

Bei diesem Muster geht es darum, eine Hierarche aufzubauen, wie etwa die Ordner-Hierarchie eines Dateisystems. In einem Ordner können sich Dateien befinden und weitere Ordner, die selbst wieder Ordner enthalten können, usw. Ordner 1 in Abbildung 212 enthält die

Ordner 2, 3, und 4. Rechts ist der Inhalt von Ordner 3 dargestellt: Die Ordner 5 und 6 und die Dateien „Datei1.txt“ und „Datei2.txt“.

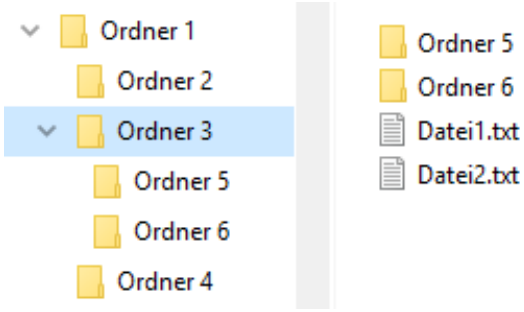


Abbildung 212: Hierarchische Ordnerstruktur

Ordner und Dateien haben vieles gemeinsam, z. B. Attribute wie Name oder Anlegedatum, und Methoden, z. B. zum Öffnen oder zum Anzeigen der Eigenschaften. Gemeinsam ist ihnen außerdem, dass sie beide in einen Ordner gelegt werden können. Das Kompositum-Muster ermöglicht es einem übergeordneten Ordner, beide Arten von Objekten gleich zu behandeln.

Dazu wird eine abstrakte Oberklasse für Dateien und Ordner angelegt, in Abbildung 213 *„Dateiobjekt“* genannt. Sie enthält die gemeinsamen Attribute und Methoden, sowie einige abstrakte Methoden, über die beide Klassen verfügen müssen, die aber anders implementiert sind. So soll *„printInfo( )“* jeweils auch Attributwerte ausgeben, über die die andere Klasse nicht verfügt.

Ein Kompositum ist wörtlich etwas Zusammengesetztes. In diesem Fall sind Ordner zusammengesetzt, und zwar aus Dateiobjekten (und damit aus Dateien und Ordnern). Dies ist in Form einer Aggregation zwischen Ordner und Dateiobjekt modelliert. Die Raute am Ende der Aggregation gibt an, dass der Ordner den enthaltenen Dateiobjekten übergeordnet ist. Ein Ordner kann beliebig viele Dateiobjekte enthalten. Umgekehrt kann jedes Dateiobjekt in einem Ordner liegen. Die Untergrenze null wurde gewählt, weil zumindest der alleroberste Ordner selbst nicht mehr in einem Ordner liegt.

Der Ordner muss sich nun nicht mehr darum kümmern, ob es sich bei den enthaltenen Objekten um Dateien oder Ordner handelt. Er kann einfach bei jedem enthaltenen Objekt die in der Klasse *„Dateiobjekt“* deklarierten Methoden aufrufen. Möchte man z. B. den kompletten Inhalt eines Ordners mitsamt der Hierarchie seiner Unterordner aufrufen, muss man einfach seine Methode *„printKomplettenInhalt( )“* aufrufen.

Aus dem dargestellten Code geht hervor, dass zunächst *„printInfo( )“* aufgerufen wird (zur Ausgabe des Namens und einiger Attributwerte des Ordners). In der anschließenden for-Schleife wird für alle enthaltenen Dateiobjekte wiederum *„printKomplettenInhalt( )“* aufgerufen.

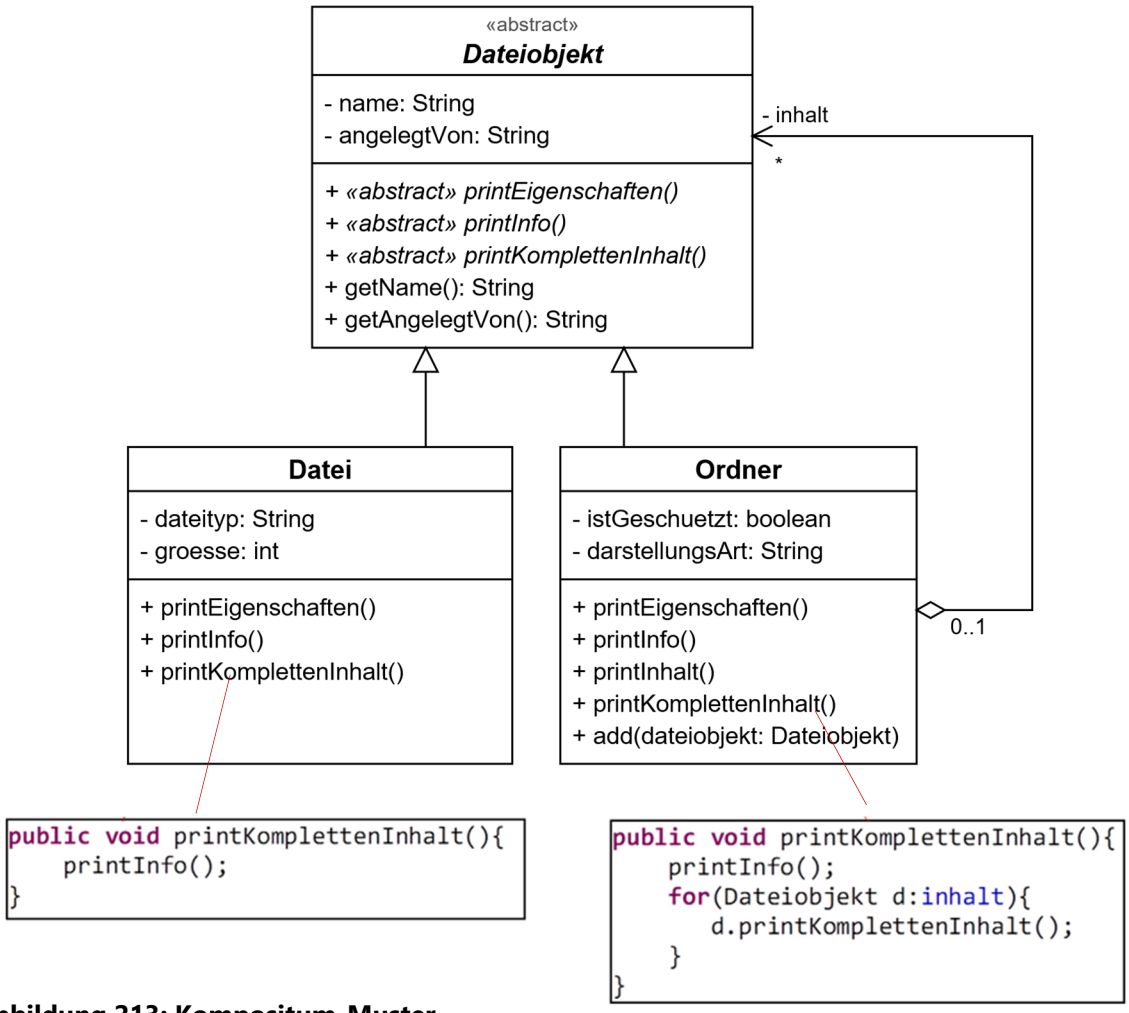


Abbildung 213: Kompositum-Muster

Handelt es sich bei einem solchen Dateiobjekt um eine Datei, so führt dies nur zum Aufruf von *„printInfo( )“*. Handelt es sich dagegen selbst wieder um einen Ordner, folgt dem Aufruf von *„printInfo( )“* erneut die for-Schleife mit dem Aufruf von *„printKomplettenInhalt( )“* für alle enthaltenen Dateiobjekte. Auf diese Weise wird rekursiv die gesamte Hierarchie aus Ordnern und Dateien ausgegeben.

Dieser Struktur lassen sich leicht weitere Objekt-Arten hinzufügen. Möchte man z. B. neben Ordnern und gewöhnlichen Dateien noch Verknüpfungs-Objekte verarbeiten, die selbst keinen Inhalt, sondern lediglich einen Link enthalten, so muss man lediglich eine weitere Unterklasse von „Dateiobjekt“ hinzufügen und die abstrakten Methoden implementieren. Die Methoden der Klasse *„Ordner“*, wie z. B. die oben beschriebene Methode *„printKomplettenInhalt( )“* funktioniert unverändert, wobei die neu hinzugekommenen Verknüpfungsobjekte automatisch mit berücksichtigt werden.

16.7 Dekorierer

Beim Dekorierer-Muster sollen einem Objekt zur Laufzeit neue Zuständigkeiten dynamisch zugeordnet werden können. Dabei schaltet man dem Objekt weitere Objekte mit demselben Typ wie die ursprüngliche Klasse vor.

Als Beispiel wird eine Klasse „*Modem*“ betrachtet (Abbildung 214). Sie verfügt über eine Methode „*send*“, der man einen String übergeben kann, der dann über eine Datenleitung versandt wird.

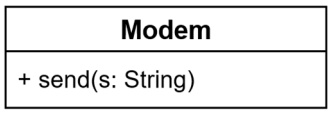


Abbildung 214: Objekte der Klasse „Modem“ sollen dynamisch um weitere Zuständigkeiten erweitert werden

Es soll nun möglich sein, ein Modem-Objekt um folgende Zuständigkeiten zu erweitern:

1. Die versandten Daten sollen in einer Datei protokolliert werden (Logging).
2. Die Aufrufe sollen gezählt werden.

Die hierfür erforderlichen Methoden sollen nicht direkt in der Klasse Modem implementiert werden, und es soll möglich sein, das Logging und das Zählen während der Laufzeit beliebig hinzuzufügen oder wegzunehmen.

Das Klassendiagramm des zur Lösung verwendeten Dekorierer-Musters findet sich in Abbildung 215. Darin arbeitet die sonstige Anwendung nicht direkt mit der Klasse „Modem“, sondern mit einem „*ModemInterface*“. Dieses Interface enhält einfach die Methode „*send( )*“, die in der Klasse „*Modem*“ ja bereits enthalten ist. Man muss in der Klasse „*Modem*“ lediglich noch angeben, dass sie dieses Interface implementiert.

Die abstrakte Klasse „*ModemDekorierer*“ implementiert ebenfalls das „*ModemInterface*“. Allerdings ist die „*send( )*“-Methode darin abstract, d. h. erst in den konkreten Unterklassen findet sich tatsächlicher Programmcode zu ihrer Implementierung. Zudem gibt es eine Assoziation zum „*ModemInterface*“. Jeder „*ModemDekorierer*“ dekoriert genau ein „*Modem-Interface*“. Dabei kann es sich tatsächlich um ein Modem handeln, oder aber selbst wieder um einen ModemDekorierer, d. h. ein Objekt einer Unterklasse von „*ModemDekorierer*“.

Die zusätzlich gewünschte Funktionalität wird nun in Form eben dieser Unterklassen von „*ModemDekorierer*“ bereitgestellt. So verfügt ein ZaehlDekorierer über ein Attribut zum Speichern der Zahl der Aufrufe. Zudem enthält es die von „*ModemDekorierer*“ geerbte Assoziation zu „*ModemInterface*“, d. h. ein ZaehlDekorierer dekoriert immer ein anderes Objekt einer Klasse, die das „*ModemInterface*“ implementiert.

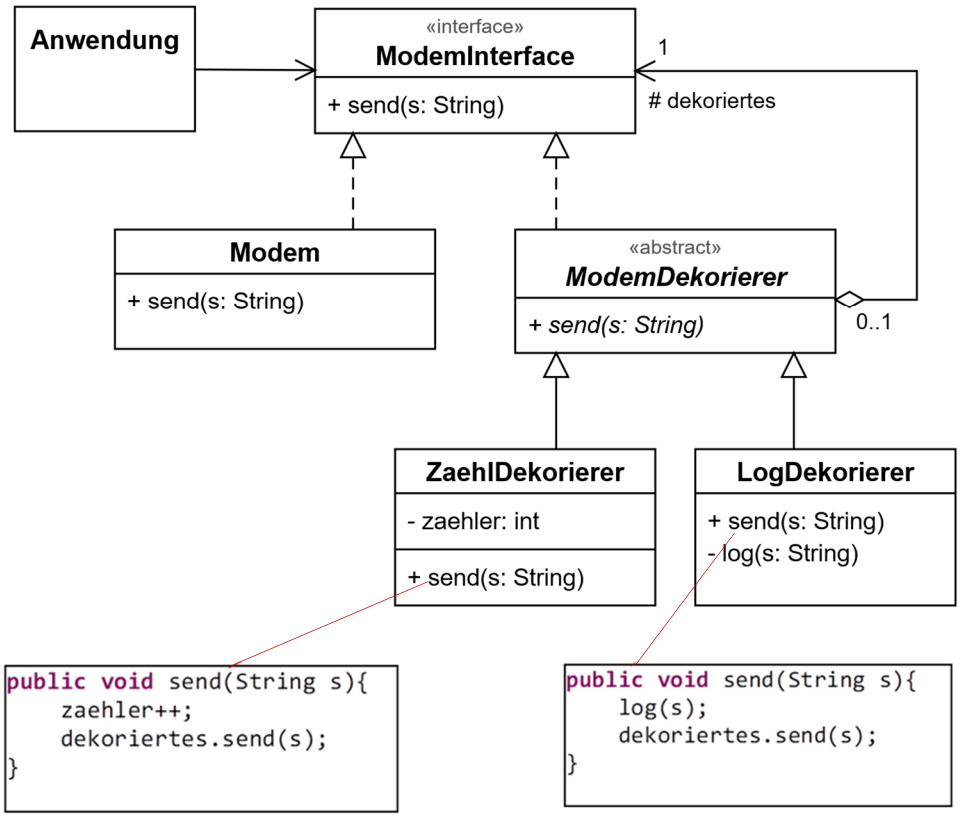


Abbildung 215: Dekorierer-Muster

Wird nun die Methode „*send( )*" eines „*ZaehlDekorierers*" aufgerufen, so wird zunächst der Zähler erhöht. Anschließend ruft diese Methode die „*send( )*"-Methode des dekorierten Objekts auf. Ein „*LogDekorierer*" arbeitet ebenso: Zunächst wird eine „*log( )*"-Methode aufgerufen, anschließend die Methode „*send( )*" des dekorierten Objekts.

Jeder Dekorierer führt also zunächst seine spezielle Zusatzaufgabe aus und leitet dann den Aufruf einfach an das dekorierte Objekt weiter.

Da das Modem und die verschiedenen Modem-Dekorierer alle vom Typ „*ModemInterface*" sind, kann die Anwendung nun entweder direkt mit einem Modem arbeiten, oder aber mit einem Dekorierer arbeiten, der direkt oder indirekt ein Modem dekoriert.

In Abbildung 216 hat die Anwendung direkt ein Modem zugeordnet. Sie ruft dessen „*send( )*"-Methode auf.

Abbildung 217 zeigt ein mit einem ZaehlDekorierer dekoriertes Modem. Der Anwendung ist nun der ZaehlDekorierer zugeordnet. Sie ruft dessen „*send( )*"-Methode auf. Diese Methode führt ihre spezielle Aufgabe aus (das Zählen der Aufrufe), und ruft dann die „*send( )*"-Methode des Modems auf.

In Abbildung 218 ist das Modem mit einem LogDekorierer und einem ZaehlDekorierer dekoriert, d. h. die Aufrufe werden sowohl gezählt als auch protokolliert.

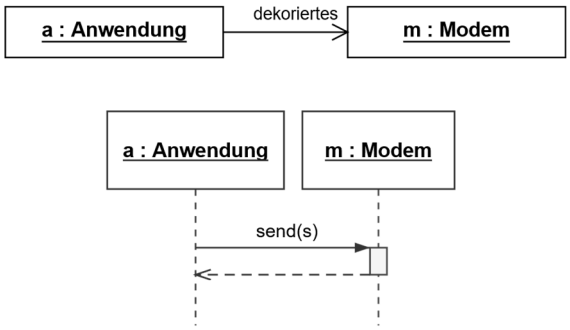


Abbildung 216: Direkter Aufruf der send-Methode beim Modem (oben die Objekte, unten der Aufruf im Sequenzdiagramm)

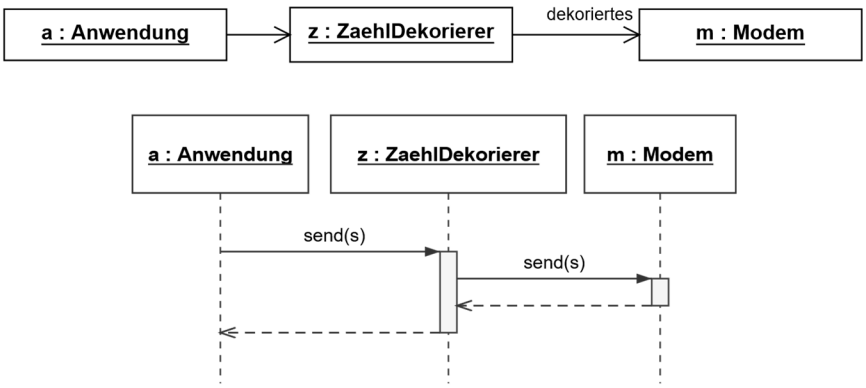


Abbildung 217: Hier ist das Modem mit einem ZaehlDekorierer dekoriert (oben die Objekte, unten der Aufruf im Sequenzdiagramm)

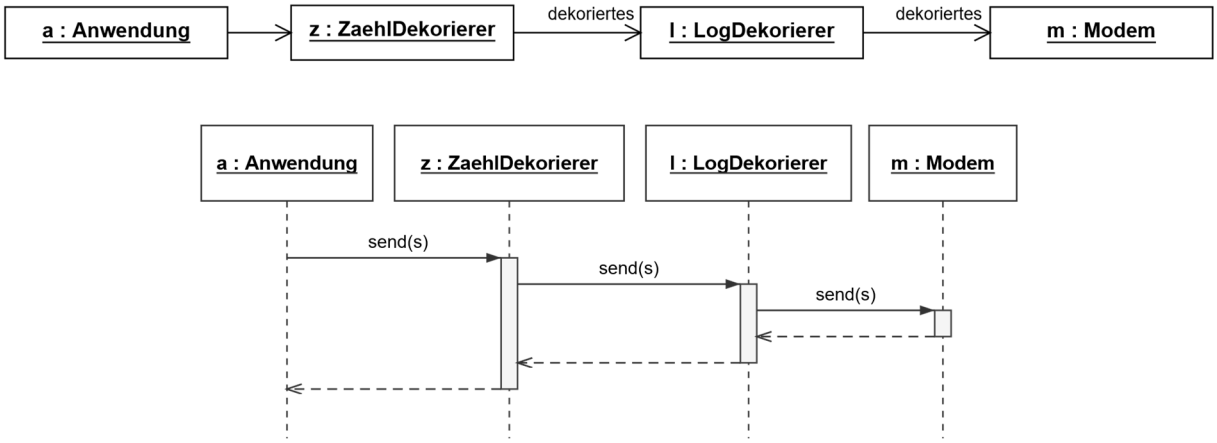


Abbildung 218: Verwendung von ZahelDekorierer und LogDekorierer (oben die Objekte, unten der Aufruf im Sequenzdiagramm)

Damit diese Funktionalitäten dynamisch zugeordnet und weggenommen werden können, sind noch entsprechende Methoden erforderlich, die die Zuordnung der beteiligten Objekte vornehmen.

16.8 Beobachter

Das Beobachter-Muster (Observer Pattern) wird häufig bei grafischen Oberflächen eingesetzt. Es dient dazu, bei Änderungen eines Objekts alle davon abhängigen Objekte zu benachrichtigen und zu aktualisieren.

Beispielsweise sollen die Daten eines Objekts zum einen als Tabelle, zum anderen in verschiedenen Grafiken dargestellt werden. Das Tabellenobjekt kennt die grafischen Darstellungen nicht. Trotzdem sollen Änderungen der Tabelle direkt die Grafiken aktualisieren. Zudem soll es auch leicht möglich sein, weitere grafische Darstellungen hinzuzufügen.

Der Lösungsansatz des Beobachter-Musters besteht darin, dass das Anwendungsobjekt (Subject) über eine Liste aller Beobachter (Observer), d. h. aller Anzeige-Objekte, verfügt und sie über Änderungen informiert. Als Reaktion synchronisiert sich jeder Beobachter mit dem Zustand des Subjekts.

Im Klassendiagramm in Abbildung 219 enthält die Klasse „*KonkretesSubjekt*" die anzuzeigenden Daten, vereinfacht als einfaches String-Attribut „*daten*" mit entsprechender setter- und getter-Methode dargestellt.

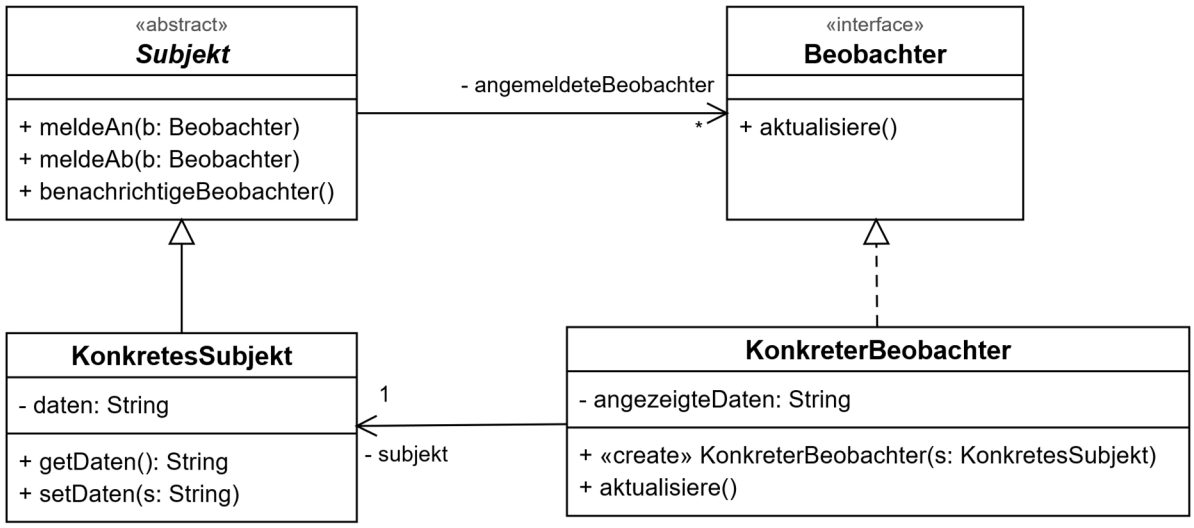


Abbildung 219: Beobachter-Muster

Objekte der Klasse „*KonkreterBeobachter*" dienen der Anzeige der Daten des konkreten Subjekts. Hierzu soll der Inhalt des Attributs „*angezeigteDaten*" mit dem Inhalt von „*daten*" des konkreten Subjekts synchronisiert werden.

„*KonkretesSubjekt*" ist eine Unterklasse der abstrakten Klasse „*Subjekt*", „*KonkreterBeobachter*" implementiert das Interface „*Beobachter*". „*Subjekt*" und „*Beobachter*" stellen den Mechanismus zum Hinzufügen und Abmelden von Beobachtern sowie zu ihrer Benachrichtigung zur Verfügung.

Hierzu verfügt die Klasse „*Subjekt*" über die Liste „*angemeldeteBeobachter*". Mit „*meldeAn( )*" und „*meldeAb( )*" können Beobachter dieser Liste hinzugefügt bzw. von der Liste entfernt werden. Die wichtigsten Methoden sind in Abbildung 220 abgebildet.

Ändern sich nun die Daten im konkreten Subjekt durch Aufruf von „*setDaten( )*“, so wird nicht nur der Wert des Attributs „*daten*“ geändert, sondern es wird zusätzlich noch die Methode „*benachrichtigeBeobachter( )*“ aufgerufen. Diese Methode ruft bei allen Beobachtern, die sich in der Liste „*angemeldeteBeobachter*“ befinden, die Methode „*aktualisiere( )*“ auf.

Die konkreten Beobachter sind nun selbst dafür zuständig, den betreffenden Wert vom konkreten Subjekt anzufordern und den Wert ihrer Anzeige zu aktualisieren. Hierzu verfügt jeder konkrete Beobachter über eine Assoziation zum beobachteten Subjekt. Die Methode „*aktualisiere( )*“ ruft die Methode „*getDaten( )*“ des beobachteten Subjekts auf und weist den Wert dem Attribut „*angezeigteDaten*“ zu. Nun stimmen die Werte von „*daten*“ im konkreten Subjekt und „*angezeigteDaten*“ im konkreten Beobachter überein.

Zusammengefasst führt jede Änderung der Subjekt-Daten dazu, dass das Subjekt alle Beobachter darüber informiert, dass eine Änderung stattgefunden hat, woraufhin die Beobachter sich mit dem Subjekt synchronisieren.

Das Beobachter-Muster wird wie erwähnt vielfach bei grafischen Benutzungsoberflächen eingesetzt. So basiert etwa das Event-Handling von Java FX auf diesem Muster. Die verschiedenen Controls erhalten „EventHandler“ zugeordnet . Dabei handelt es sich um Beobachter. Dabei ist dies im Vergleich zum einfachen Beobachter-Muster noch erweitert. So werden verschiedene Arten von Ereignissen unterschieden, z. B. ein Klick der linken oder der rechten Maustaste). Der Event-Handler reagiert darauf, indem er jeweils andere Methoden ausführt.

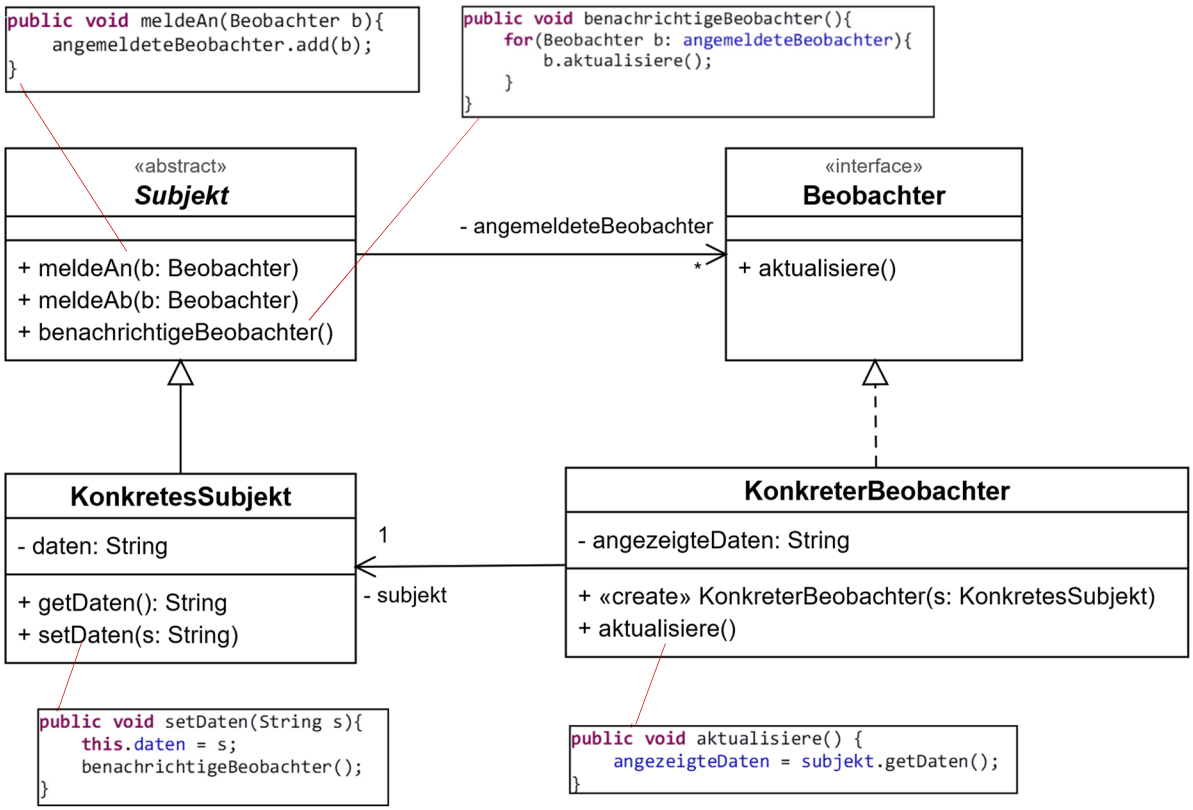


Abbildung 220: Beobachter-Muster mit Methoden

17 Persistenz

Unter Persistenz versteht man die dauerhafte Speicherung von Daten. Hierfür stehen verschiedene Möglichkeiten zur Verfügung. Im einfachsten Fall werden die Daten in flache Dateien geschrieben. In einem objektorientierten Programm müssen hierfür die Objektstrukturen serialisiert und in die Datei geschrieben werden, bzw. umgekehrt müssen die Daten aus der Datei ausgelesen und in Objekte umgewandelt werden.

Am häufigsten erfolgt die Speicherung in Datenbanken. In den vergangenen Jahren haben insbesondere zur Speicherung sehr großer Datenmengen verschiedene Typen von NoSQL-Datenbanken an Bedeutung gewonnen. „NoSQL" steht für „Not only SQL". Dazu gehören z. B. Key/Value-Speicher, spaltenorientierte, graphenorientierte oder dokumentenorientierte Datenbanken. Nach wie vor die größte Bedeutung aber haben die relationalen Datenbanken.

Im vorliegenden Kapitel wird das Zusammenspiel zwischen objektorientierter Software und relationalen Datenbanken besprochen. Ein grundlegendes Verständnis von relationalen Datenbanken wird in anderen Lehrveranstaltungen vermittelt und im Folgenden vorausgesetzt.

Um die Daten der Objekte eines objektorientierten Systems in einer relationalen Datenbank speichern zu können, sind insbesondere die folgenden beiden Aspekte wichtig:

1. Abbildung der Objektstrukturen auf Datenbanktabellen
2. Abspeichern von Objekten und Wiedergewinnen von Objekten aus den gespeicherten Daten

17.1 Abbildung der Objektstrukturen auf Datenbanktabellen

17.1.1 Abbildung einfacher Klassen

Im Standardfall wird jede Entity-Klasse auf eine Datenbanktabelle abgebildet (Abbildung 221). Dabei wird jedes Attribut zu einer Spalte in der Tabelle.

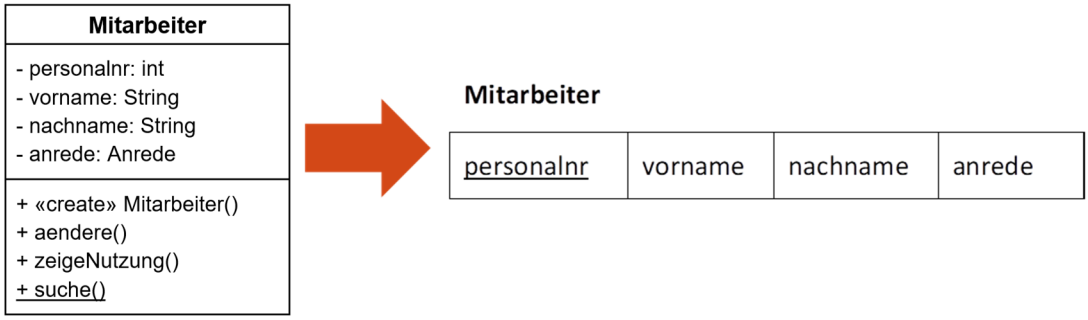


Abbildung 221: Abbildung einer Klasse auf eine Datenbanktabelle

Dies erscheint zunächst recht einfach, doch gilt es in der Praxis eine Reihe von Fragestellungen zu klären. Zum einen sind die Datentypen der objektorientierten Programmiersprachen in der Regel nicht identisch mit den Datentypen der Datenbanken. So können sich die erlaubten Wertebereiche unterscheiden. Ebenso ist die mögliche Länge von Zeichenketten zu berücksichtigen. So gibt es für Strings in Java keine Längenbeschränkung, wohingegen für

den Typ „Varchar“ in SQL eine Maximallänge angegeben werden muss. Ggf. sind geeignete Umwandlungen erforderlich. Zudem sollten Überprüfungen stattfinden, um Fehler und Angriffe durch SQL-Injection zu vermeiden.

In der Datenbank muss jeder Datensatz im Gegensatz zu einem Objekt einen eindeutigen Primärschlüssel haben. Daher muss man entscheiden, wie der Primärschlüssel erzeugt wird. Falls es ein identifizierendes Attribut gibt, für das sichergestellt wird, dass keine doppelten Werte auftreten, kann dieses Attribut zum Primärschlüssel in der Datenbanktabelle gemacht werden. In Abbildung 221 ist dies das Attribut „*personalnr*“, das deshalb in der Tabelle unterstrichen ist.

Liegt kein geeignetes Attribut mit eindeutigen Werten vor, so muss man in der Datenbanktabelle einen neuen Schlüssel einführen.

17.1.2 Umsetzung von 1:1-Assoziationen

Bei Assoziationen, deren Multiplizitäten auf beiden Seiten die Obergrenze 1 haben, muss für eines der beiden Objekte vermerkt werden, welches andere Objekt mit ihm in Beziehung steht. Hierfür muss in eine der beiden Tabellen der Primärschlüssel der anderen Tabelle als Fremdschlüssel eingetragen werden. So enthält Abbildung 222 die Tabelle „*Reservierung*“ die „*ausleihenr*“ als Fremdschlüssel. Hierüber lässt sich für jede Reservierung die zugeordnete Ausleihe herausfinden.

Der Untergrenze „0“ der Multiplizität wird dadurch Rechnung getragen, dass die „ausleihenr“ in der Reservierung auch leer sein kann. Die im objektorientierten Programm implementierten Navigationsrichtungen müssen nicht berücksichtigt werden, da über geeignete SQL-Abfragen sowohl zu jeder Reservierung die zugehörige Ausleihe als auch zu jeder Ausleihe die zugehörige Reservierung ermittelt werden kann.

17.1.3 Umsetzung von 1:*-Assoziationen

Hat bei einer Assoziation eine der Multiplizitäten die Obergrenze 1, die andere die Obergrenze *, so kann dies ebenfalls mit Hilfe einer Fremdschlüsselbeziehung abgebildet werden. In Abbildung 223 kann es zu einer Kategorie beliebig viele Gerätetypen geben, umgekehrt gehört aber jeder Gerätetyp zu genau einer Kategorie.

Möchte man zu einer Kategorie die zugehörigen Gerätetypen speichern, so benötigt man eine Liste. Dies kann man in der Datenbank nicht mit Hilfe einer einfachen Spalte abbilden. Man würde eine separate Tabelle benötigen. Wesentlich einfacher ist es, zum Gerätetyp die zugehörige Kategorie abzuspeichern, da er ja immer nur zu einer Kategorie gehört. Man muss also lediglich in der Tabelle „*Geraetetyp*“ die „*kategorienr*“ als Fremdschlüssel eintragen.

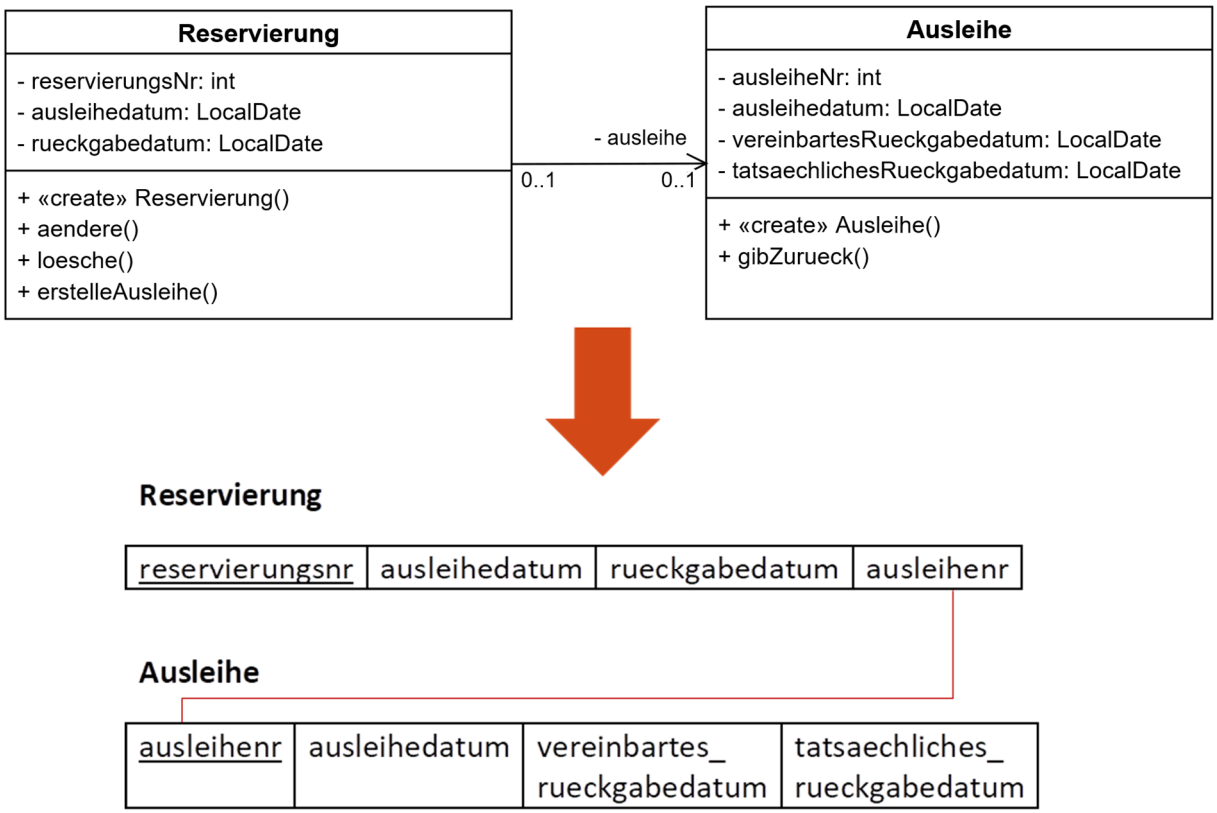


Abbildung 222: Umsetzung von 1:1-Assoziationen

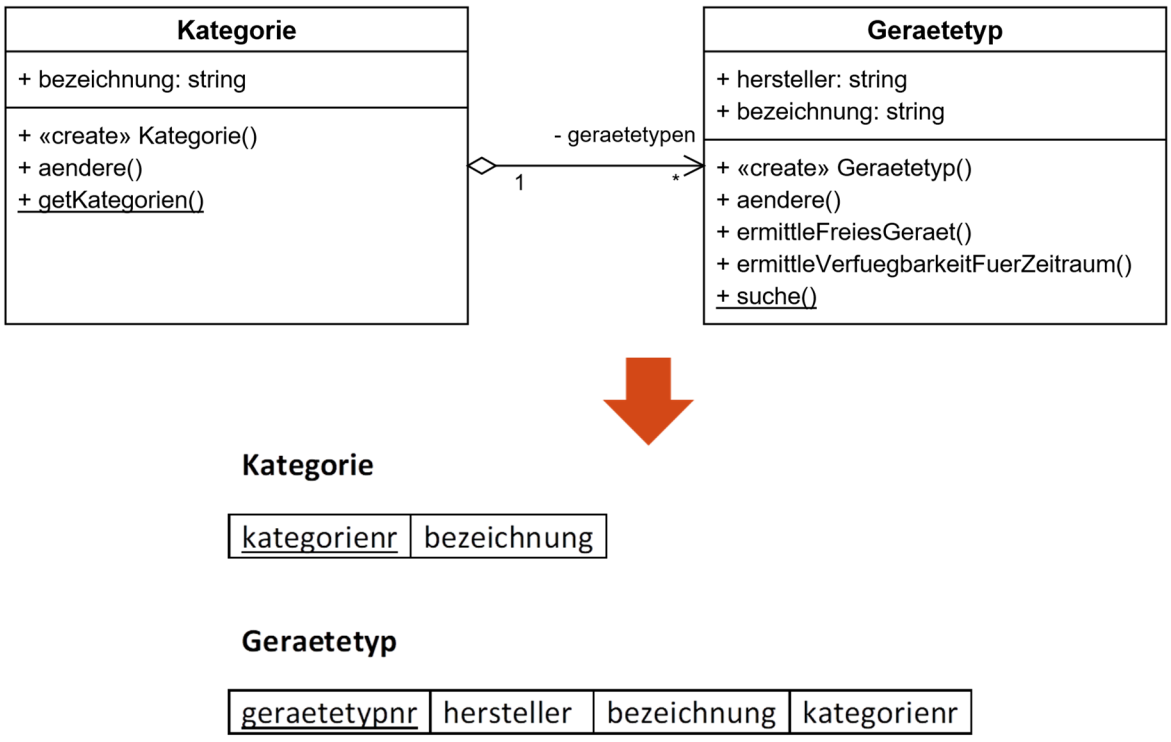


Abbildung 223: Umsetzung von 1:*-Assoziationen

Die zu einer Kategorie gehörenden Gerätetypen lassen sich wiederum leicht über eine SQL-Abfrage herausfinden.

Für die Tabellen „*Kategorie*“ und „*Geraetetyp*“ wurden hier übrigens neue Primärschlüssel eingeführt, da die zugrunde liegenden Klassen keine Attribute haben, die sich gut als Schlüssel eignen würden.

17.1.4 Umsetzung von *:*-Assoziationen

Hat eine Assoziation an beiden Enden Multiplizitäten mit Obergrenzen, die größer als 1 sind, so kann dies nicht mehr durch bloßes Hinzufügen eines Fremdschlüssels zu einer der beiden beteiligten Tabellen erledigt werden.

In dem Beispiel aus Abbildung 224 gehören zu jedem Projekt mehrere Mitarbeiter, und jeder Mitarbeiter kann an mehreren Projekten beteiligt sein. Die Lösung besteht darin, eine Zwischentabelle einzuführen, in der die Paarungen der einander zugeordneten Entitäten gespeichert werden. Im Beispiel enthält die Tabelle „*Projektmitarbeit*“ für jede Zuordnung eines Mitarbeiters zu einem Projekt einen Eintrag, der die „*personalnr*“ und die „*projektnr*“ als Fremdschlüssel enthält.

Der Primärschlüssel dieser Tabelle ist ein aus „*personalnr*“ und „*projektnr*“ zusammengesetzter Schlüssel. Jede Personalnummer kann ja mehrfach vorkommen – in Kombination mit unterschiedlichen Projektnummern. Ebenso kann jede Projektnummer mehrfach vorkommen, wobei jeweils eine andere Personalnummer zugeordnet ist.

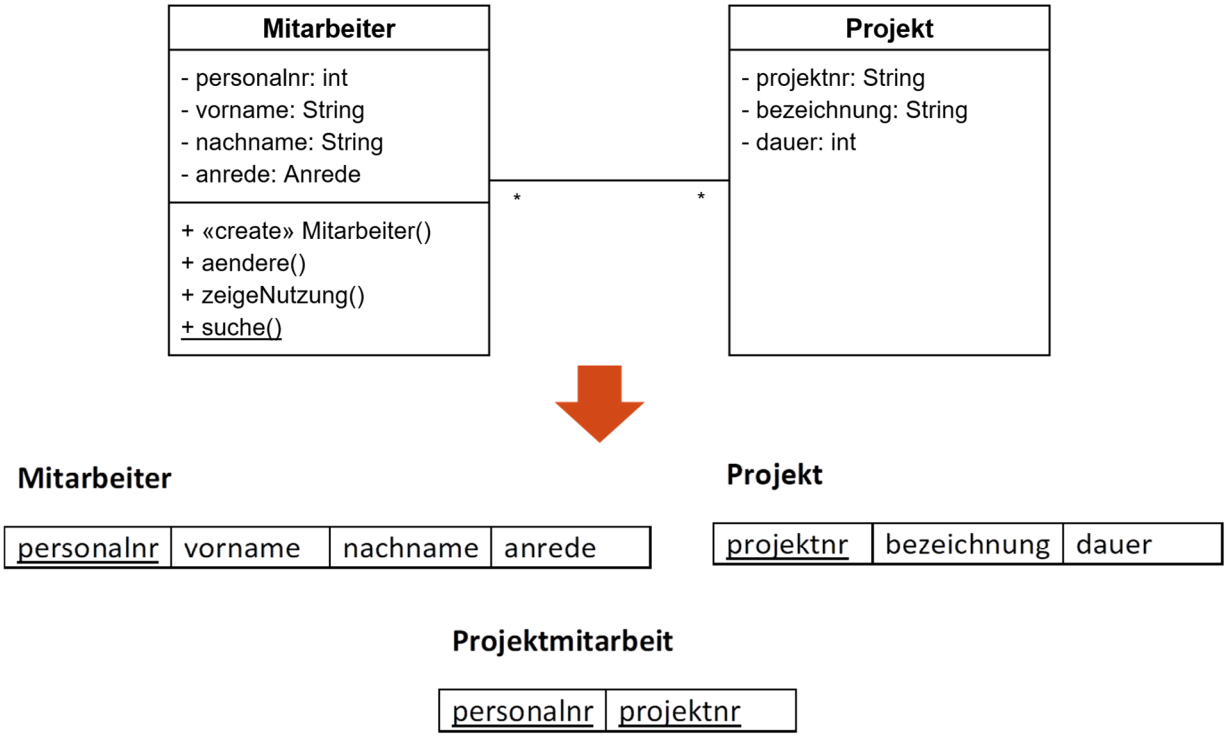


Abbildung 224: Umsetzung von *:*-Assoziationen

17.1.5 Umsetzung von Vererbungsbeziehungen

Zwar sieht der SQL-Standard die Möglichkeit vor, Typhierarchien zu definieren und damit Vererbung direkt umzusetzen, doch ist dies nicht einheitlich in allen Datenbanksystemen umgesetzt, weshalb es vielfach erforderlich ist, Vererbungsbeziehungen in gewöhnliche Tabellen umzusetzen.

Hierfür gibt es mehrere Möglichkeiten:

1. Eine Tabelle für jede konkrete Klasse
2. Eine Tabelle für die gesamte Vererbungshierarchie
3. Eine Tabelle für jede Klasse

Eine Tabelle für jede konkrete Klasse

Dabei wird die Vererbungsbeziehung aufgelöst. Für jede konkrete Klasse wird eine Tabelle angelegt, die alle Attribute dieser konkreten Klasse enthält, auch die von Oberklassen geerbten. So enthalten die Tabellen „*Buch*" und „*CD*" in Abbildung 225 auch die von „*Medium*" geerbten Attribute „*mediennr*", „*titel*" und „*urheber*".

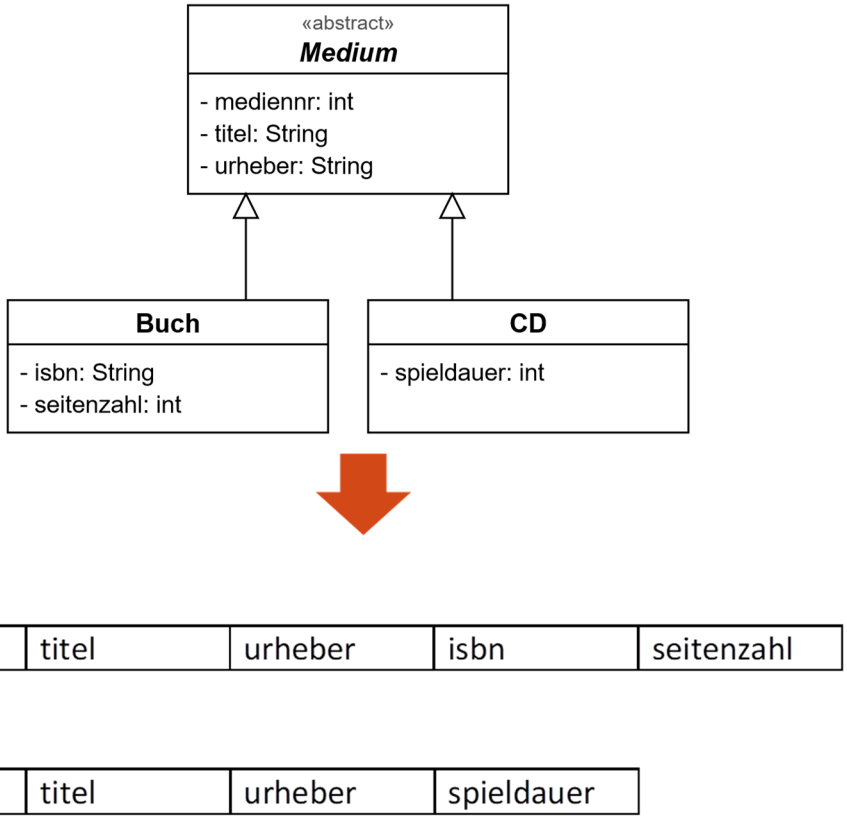


Abbildung 225: Umsetzung von Vererbungsbeziehungen – eine Tabelle pro konkrete Klasse

Bei dieser Lösung kann man recht einfach und schnell auf die Daten der kompletten Objekte zugreifen. Andererseits ist es z. B. mühsamer, eine Liste aller Medien zu generieren.

Eine Tabelle für die gesamte Vererbungshierarchie

Hierbei wird nur eine einzige Tabelle für die Wurzelklasse der Hierarchie angelegt. Dies wird in Abbildung 226 für das Beispiel aus Abbildung 225 gezeigt. Die Tabelle „*Medium*" enthält

hier alle Attribute, nicht nur aus der Klasse „*Medium*“, sondern auch aus den Unterklassen „*Buch*“ und „*CD*“.

Medium

mediennr	titel	urheber	isbn	seitenzahl	spieldauer	art

Abbildung 226: Umsetzung von Vererbungsbeziehungen – eine Tabelle für die gesamte Hierachie

In die Spalte „*art*“ wird eingetragen, um welche Medien-Art es sich handelt – Buch oder CD. Entsprechend sind bei jedem Eintrag auch nur diejenige Attribute mit Werten gefüllt, die zu der jeweiligen Medien-Art gehören.

Eine Tabelle für jede Klasse

Die dritte Möglichkeit besteht darin, für jede Klasse eine Tabelle anzulegen, also auch für abstrakte Oberklassen. So enthält Abbildung 227 auch eine Tabelle „*Medium*“, die nur die Attribute der abstrakten Klasse „*Medium*“ umfasst.

Medium

mediennr	titel	urheber

Buch

mediennr	isbn	seitenzahl

CD

mediennr	spieldauer

Abbildung 227: Umsetzung von Vererbungsbeziehungen – eine Tabelle für jede Klasse

Für jedes konkrete Objekt werden nun Einträge in zwei Tabellen vorgenommen. So gibt es für ein Buch einen Eintrag in der „*Buch*“-Tabelle und in der der „*Medium*“-Tabelle, wobei die Buch-spezifischen Attribute in der „*Buch*“-Tabelle gespeichert sind, die von „*Medium*“ geerbten Attribute hingegen in der Tabelle „*Medium*“.

Um alle Attribute eines Buches zu finden, ist nun eine Abfrage über zwei Tabellen nötig. Damit die beiden Einträge einander zugeordnet werden können, muss die Tabelle „*Buch*“ einen Fremdschlüssel zur Identifikation des zugehörigen Eintrags in der Tabelle „*Medium*“ enthalten. Im Beispiel ist der Primärschlüssel „*mediennr*“ zugleich der Fremdschlüssel, der die Verbindung zu „*Medium*“ herstellt.

Es wird hier davon ausgegangen, dass „*mediennr*“ eindeutige Werte über alle Medien hinweg hat. Dann kann dieses Attribut als Primärschlüssel sowohl für „*Medium*“, als auch für „*Buch*“ und „*CD*“ genutzt werden. Alternativ könnte man auch neue Schlüssel wie „*buchnr*“ und „*cdnr*“ einführen. Dann müsste man „*mediennr*“ zusätzlich als Fremdschlüssel in „*Buch*“ und „*CD*“ aufnehmen.

17.2 Abspeichern und Materialisieren von Objekten

Beim Abspeichern – auch Persistieren oder De-Materialisieren genannt – eines Objekts wird ein Datenbankeintrag mit den Werten der zu speichernden Attribute erzeugt. Falls das Objekt Beziehungen zu anderen Objekten hat, so müssen diese ebenfalls gespeichert werden, sofern sie noch nicht in der Datenbank sind.

Beim Materialisieren oder Instanziieren, d. h. dem Laden eines Objektes aus der Datenbank, wird ein Objekt der betreffenden Klasse angelegt, und die aus der Datenbank gelesenen Werte werden als Attributwerte eingetragen. Wenn das Objekt Beziehungen zu anderen Objekten hat, müssen diese ebenfalls materialisiert werden.

Falls man es mit sehr großen Objekten und Objektgraphen zu tun hat, ist es nicht unbedingt wünschenswert, immer sofort alles komplett aus der Datenbank zu lesen. Als Beispiel wird das Klassendiagramm in Abbildung 228 betrachtet. Materialisiert man ein Kundenobjekt, so muss man auch seine Bestellungen materialisieren, sowie für jede Bestellung alle Bestellpositionen und die darin referenzierten Produkte. Unter Umständen werden auf diese Weise Hunderte von Objekten materialisiert, obwohl man vielleicht nur auf den Namen des Kunden zugreifen will und seine Bestellungen im Moment gar nicht benötigt werden. Dies ist sehr ineffizient.

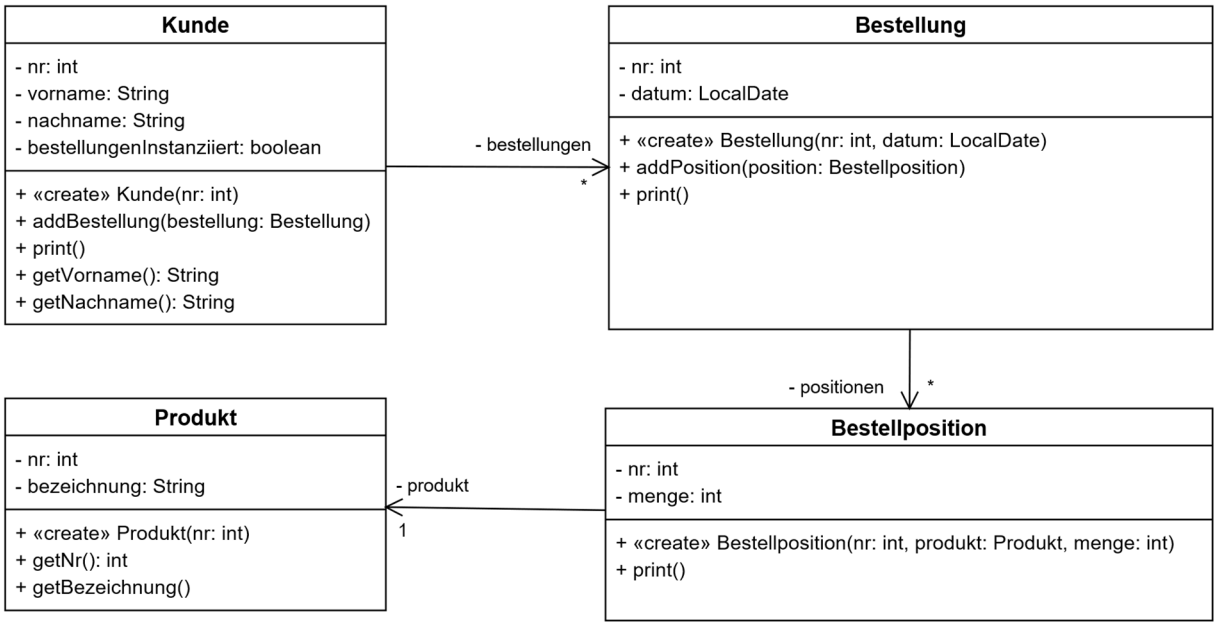


Abbildung 228: An einem Kundenobjekt hängt ein großer Objektgraph

17.3 Lazy Instantiation

Eine Lösungsmöglichkeit ist die Lazy Instantation, die „faule" Instanziierung. Dabei ist man insofern faul, als man immer das instanziiert, was momentan benötigt wird. Im Beispiel von Abbildung 228 wird zunächst nur das Kundenobjekt mit seinen Attributwerten materialisiert. Die zugehörigen Bestellungen werden erst einmal weggelassen. Vielleicht werden sie ja gar nicht gebraucht.

Erst wenn eine Methode aufgerufen wird, die Zugriff auf Bestellungen braucht, werden die Bestellungsobjekte ebenfalls materialisiert.

Im dem Beispiel soll es zusätzlich noch eine Klasse „*Kundenverwaltung*" zum Anlegen und Verwalten der Kundendaten geben, sowie eine Klasse „*Kundenspeicher*", die mit der Datenbank kommuniziert. Listing 43 zeigt die Methode der Klasse „*Kundenverwaltung*" zum Materialisieren des Kunden mit einer bestimmten Kundennummer. Es wird ein Kundenobjekt angelegt, mit der Kunden-Nr und den zugehörigen, aus der Datenbank ausgelesenen Werten für Vorname und Nachname. Da es sich um Lazy Instantiation handelt, werden die Bestellungen noch weggelassen.

```
public static Kunde leseKundeAus(int kundennr) {
      Kunde k = new Kunde(kundennr,
                  Kundenspeicher.leseVornameAus(kundennr),
                  Kundenspeicher.leseNachnameAus(kundennr));
      return k;
}
```

Listing 43: Lazy Instantiation der Klasse Kunde – ohne die zugehörigen Bestellungen

Methoden der Klasse „*Kunde*", die nur die direkten Attribute dieser Klasse verwenden, können nun ohne weiteres genutzt werden, wie z. B. „*getNachname( )*" in Listing 44.

```
public String getNachname() {
      return this.nachname;
}
```

Listing 44: Eine Methode der Klasse „Kunde", die nicht auf verbundene Objekte zugreift

Wird nun eine Methode aufgerufen, die Bestellungen benötigt, so müssen diese ggf. erst materialisiert werden. Die Methode „*print( )*" der Klasse Kunde in Listing 45 soll nicht nur Vor- und Nachname des Kunden ausgeben, sondern auch eine Bestellungsübersicht. Damit die for-Schleife über die Bestellungen funktioniert, müssen die Bestellungsobjekte vorhanden sein. Daher wird vorher die Methode „*instanziiereBestellungen( )*" augerufen. Diese Methode ist ebenfalls in Listing 45 enthalten.

In dieser Methode wird zunächst überprüft, ob die Bestellungen bereits instanziiert sind. Hierzu wird ein boolesches Attribut abgefragt, das zu Beginn den Defaultwert „*false*" hat. Sind die Bestellungen also noch nicht instanziiert, so werden sie nun aus der Datenbank ausgelesen und in das Listenattribut „*bestellungen*" des Kunden eingetragen. Anschließend erhält das Attribut „*bestellungenInstanziiert*" den Wert „*true*", damit die Bestellungen beim nächsten Mal nicht erneut instanziiert werden.

```
	public void print() {
		System.out.println(nr + " " + vorname + " " + nachname);
		instanziiereBestellungen();
		for(Bestellung b:bestellungen) {
			b.print();
		}
	}

	private void instanziiereBestellungen() {
		if(!bestellungenInstanziiert) {
			bestellungen.addAll(
			    Kundenverwaltung.leseBestellungenFuerKundeAus(this.nr)
			);
			bestellungenInstanziiert = true;
		}
	 }
```

Listing 45: Die Bestellungen werden bei Bedarf instanziiert

Für das Instanziieren der Bestellungen könnte man auch wieder Lazy Instantiation anwenden und die Bestellpositionen erst bei Bedarf instanziieren. Allerdings ist die Zahl der Bestellpositionen für eine einzelne Bestellung meist überschaubar. Außerdem möchte man, wenn man etwas mit einer Bestellung tut, in der Regel die Bestellpositionen sehen. Und wenn zu der Bestellung meistens sowieso die Positionen instanziiert werden, kann man dies auch sofort tun. Lazy Instantiation bringt dann keinen Vorteil.

In Abbildung 228 sind auf den nicht navigierbaren Seiten der Assoziationen keine Multiplizitäten eingetragen. Es ist aber naheliegend, dass jedes Produkt in mehreren Bestellpositionen (aus unterschiedlichen Bestellungen) referenziert wird. Hier gilt es zu verhindern, dass beim Instanziieren der Bestellungen (in deren Folge auch die Bestellpositionen und eben auch noch nicht instanziierte Produkte instanziiert werden müssen), ein und dasselbe Produkt mehrfach instanziiert wird. Das muss verhindert werden, indem beim Instanziieren jedes Produkt-Objekts zunächst geprüft wird, ob zu dem betreffenden Datensatz bereits ein Objekt instanziiert wurde. In diesem Fall soll keine erneute Instanziierung stattfinden, sondern es soll das vorhandene Objekt referenziert werden.

17.4 Proxy-Muster

Mit Hilfe des Proxy-Musters können ganz ähnliche Problemstellungen wie mit der Lazy Instantiation gelöst werden. Die Idee dabei ist, anstelle des eigentlichen Objekts zunächst ein Proxy, d. h. ein Stellvertreter-Objekt zu erzeugen, dass das eigentliche Objekt erst bei Bedarf materialisiert.

Als Beispiel wird eine Anwendung zur Textverarbeitung betrachtet, die mit einem Text-Objekt arbeiten kann (Abbildung 229). Der eigentliche Inhalt der Text-Objekte sei sehr groß, weil er z. B. sehr viele Bilder enthält. Daher soll er erst geladen werden, wenn er tatsächlich benötigt wird.

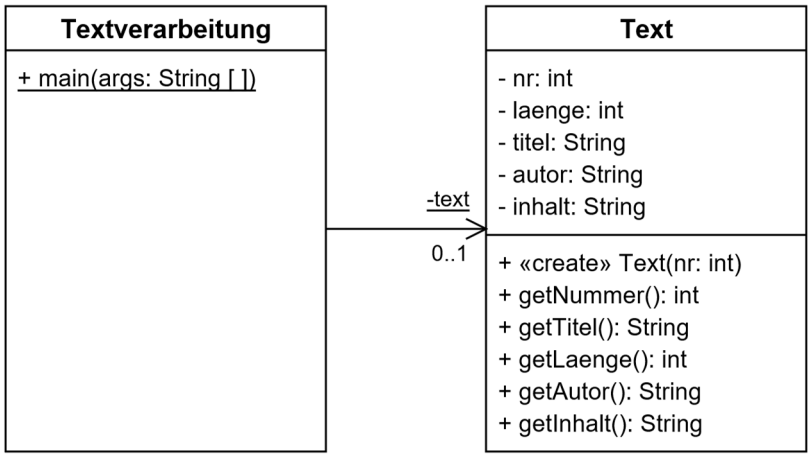


Abbildung 229: Ausgangssituation für das Proxy-Muster – die Textobjekte seien sehr groß.

Beim Proxy-Muster wird der eigentliche Text durch ein Proxy ersetzt, das die gleiche Schnittstelle wie das Original hat. In Abbildung 230 wird keine Schnittstelle, sondern eine abstrakte Klasse „*TextObjekt*" verwendet, die u. a. einige Attribute und konkrete Methoden enthält, die sowohl für das Original als auch für das Proxy benötigt werden.

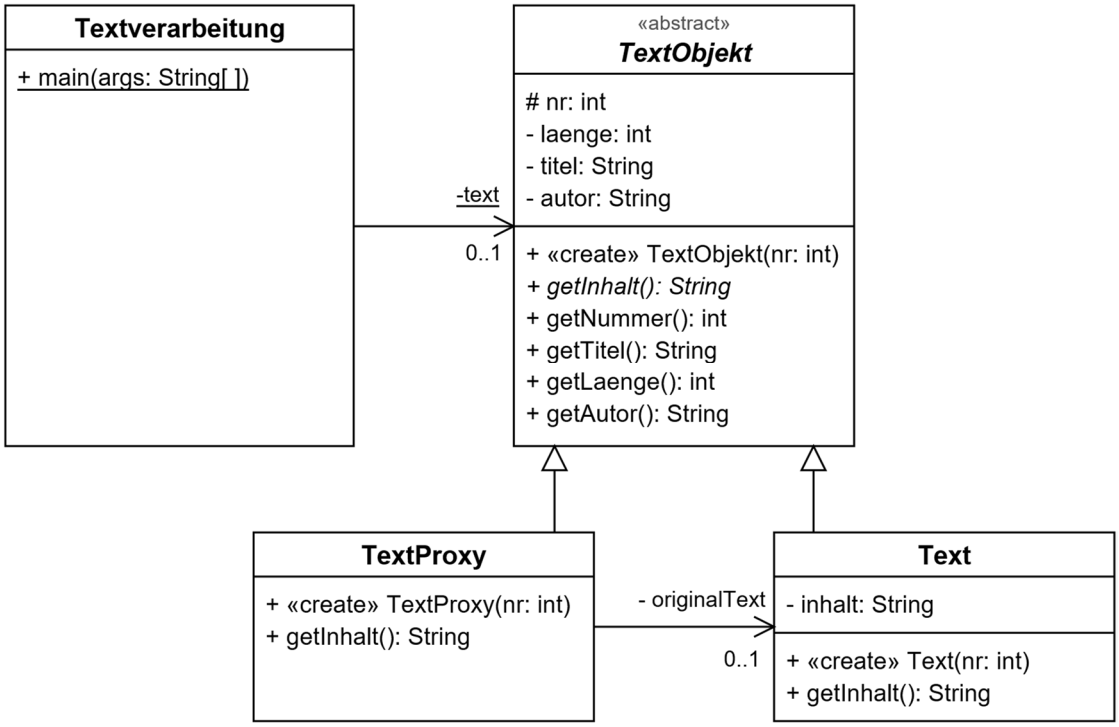


Abbildung 230: Proxy-Muster

Wenn ein Text für die Textverarbeitung geladen wird, dann wird zunächst ein Objekt der Klasse „*TextProxy*" erzeugt und an das Textverarbeitungs-Objekt übergeben. Das Textverarbeitungs-Objekt arbeitet mit dem Proxy wie mit dem Original. Es weiß gar nicht, dass es mit einem Proxy arbeitet.

Das Proxy vertritt nun das eigentliche Text-Objekt. Manche Anfragen kann es selbst beantworten. Werden Methoden wie „*getTitel( )*" oder „*getAutor( )*" aufgerufen, so führt das Proxy diese Methoden selbst aus, da es die betreffenden Attributwerte des Textes enthält. Was es

nicht selbst enthält, ist der eigentliche Inhalt. Wird nun die Methode „*getInhalt( )*" aufgerufen, so kümmert sich das Proxy darum, dass das Originalobjekt ggf. instanziiert wird, und leitet die Anfrage dann an dieses Originalobjekt weiter.

```
public class TextProxy extends TextObjekt{
	private Text originalText = null;

	public TextProxy(int nr) {
		super(nr);
	}

	public String getInhalt() {
		if(originalText == null) {
			originalText = Textverwaltung.leseKomplettenTextAus(nr);
		}
		return originalText.getInhalt();
	}

}
```

Listing 46: Das Original-Objekt wird erst instanziiert, wenn der komplette Inhalt benötigt wird

Listing 46 zeigt einen Ausschnitt aus der Klasse „*TextProxy*". Zunächst existiert noch kein Originaltext, das Attribut „*originalText*" enthält den Default-Wert „*null*". In „*getInhalt( )*" wird zunächst überprüft, ob dieser Wert immer noch null ist. In diesem Fall lässt es sich von der Klasse „*Textverwaltung*" das komplette Objekt des Originaltexts erzeugen. Anschließend ruft es die Methode „*getInhalt( )*" beim Originaltext auf.

Proxies können nicht nur als Platzhalter für Objekte eingesetzt werden, deren Erzeugung aufwändig ist. Sie können z. B. auch zusätzliche Funktionalitäten oder Schutzmechanismen anbieten, etwa zur Objektverwaltung oder zur Überprüfung von Zugriffsrechten. Bei verteilten Systemen kann man Remote Proxies einsetzen, d. h. auf dem lokalen Rechner vorhandene Vertreter für Objekte, die sich auf einem entfernten Rechner befinden.

17.5 Entwurf der Persistenzschicht

Die einfachste Möglichkeit ist, den Code für die Persistenz direkt in die Fachklasse zu schreiben. Dieser Code setzt die SQL-Befehle über eine entsprechende Schnittstelle ab (Abbildung 231). Im Zusammenhang mit Java wird meist JDBC (Java Database Connectivity) verwendet.

Allerdings entstehen hier eine enge Kopplung und eine hohe Abhängigkeit zwischen den fachlichen Komponenten und der Datenhaltung. Änderungen sind dabei sehr schwierig durchzuführen. Soll z. B. auf eine andere Datenbank mit einem etwas anderen SQL-Dialekt gewechselt werden, so ist der zu ändernde Code auf sehr viele Klassen verteilt.

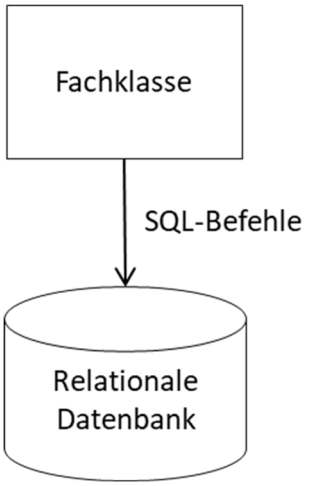


Abbildung 231: Einfacher Ansatz für Persistenz (nach [Star24])

Eine bessere Trennung wird durch die Einführung eigener Datenklassen erreicht (Abbildung 232). Dabei wird für jede Fachklasse eine Datenklasse angelegt, die sich um das Speichern und Materialisieren der Fachklassenobjekte kümmert. Änderungen der Datenhaltung betreffen nun nur noch die Datenklassen. Dennoch müssen sich die Entwickler immer noch selbst um die Details der Speicherung kümmern. Die Speicherung vernetzter Objektstrukturen ist nicht ganz einfach.

Abbildung 232: Einfacher Persistenzmechanismus mit Datenklassen (nach [Star24])

Eine höhere Flexibilität und Wartbarkeit wird durch eigene Persistenzschichten erreicht, die den Persistenzmechanismus möglichst komplett kapseln, verschiedene Persistenzmechanismen unterstützen und weitere Anforderungen umsetzen, wie z. B. die automatische Generierung von Primärschlüsseln, Lazy Instantiation, die Unterstützung von Transaktionen, usw.

Ein in Persistenzschichten eingesetztes Entwurfsmuster ist das Data Access Object (DAO). Die Übertragung der eigentlichen Daten erfolgt dabei mit Hilfe von Data Transfer Objects (DTO). Abbildung 233 zeigt die Zusammenhänge, Abbildung 234 den dynamischen Ablauf. Zunächst erzeugt das Business Object (Fachklassen-Objekt) ein DAO. Anschließend fordert es Daten vom DAO an („*getData( )*"). Dieses liest die Daten aus der persistenten Data Source, d. h. aus der Datenbank. Anschließend erzeugt es ein DTO und trägt die aus der Datenbank gelesenen Werte in dessen Attribute ein. Das DTO wird an das Business Object zurückgegeben, das dann die entsprechenden Werte daraus entnehmen kann.

Bei Änderungen von Datenwerten trägt es diese in das DTO ein („*setProperty( )*"). Schließlich beauftragt es das DAO mit der persistenten Speicherung der Änderungen („*setData( )*"). Das DAO liest die Werte aus dem DTO („*getProperty( )*") und trägt sie in die Datenbank ein („*setData( )*").

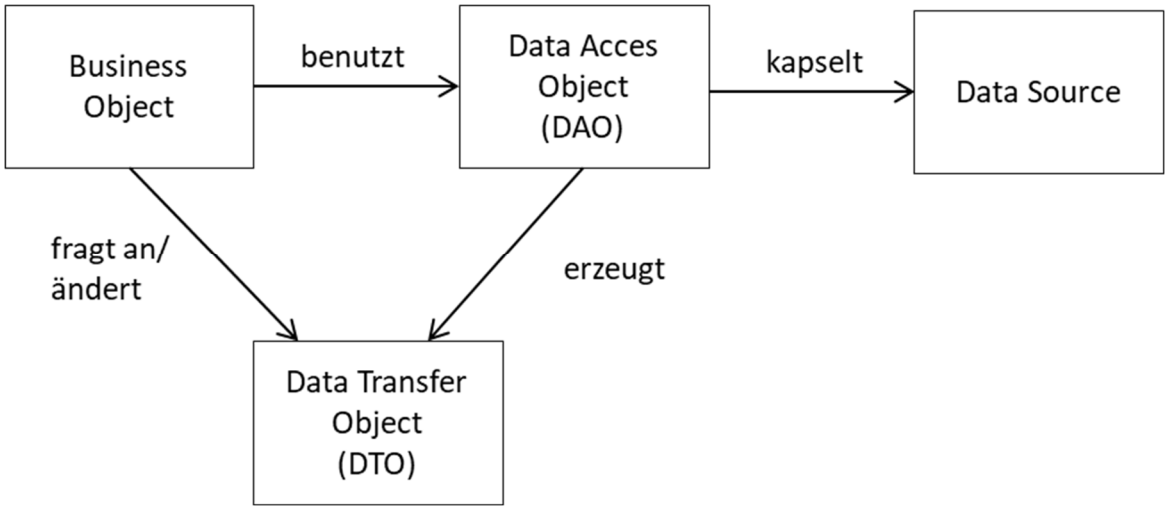


Abbildung 233: Data Access Object (DAO) Muster (nach [Star24])

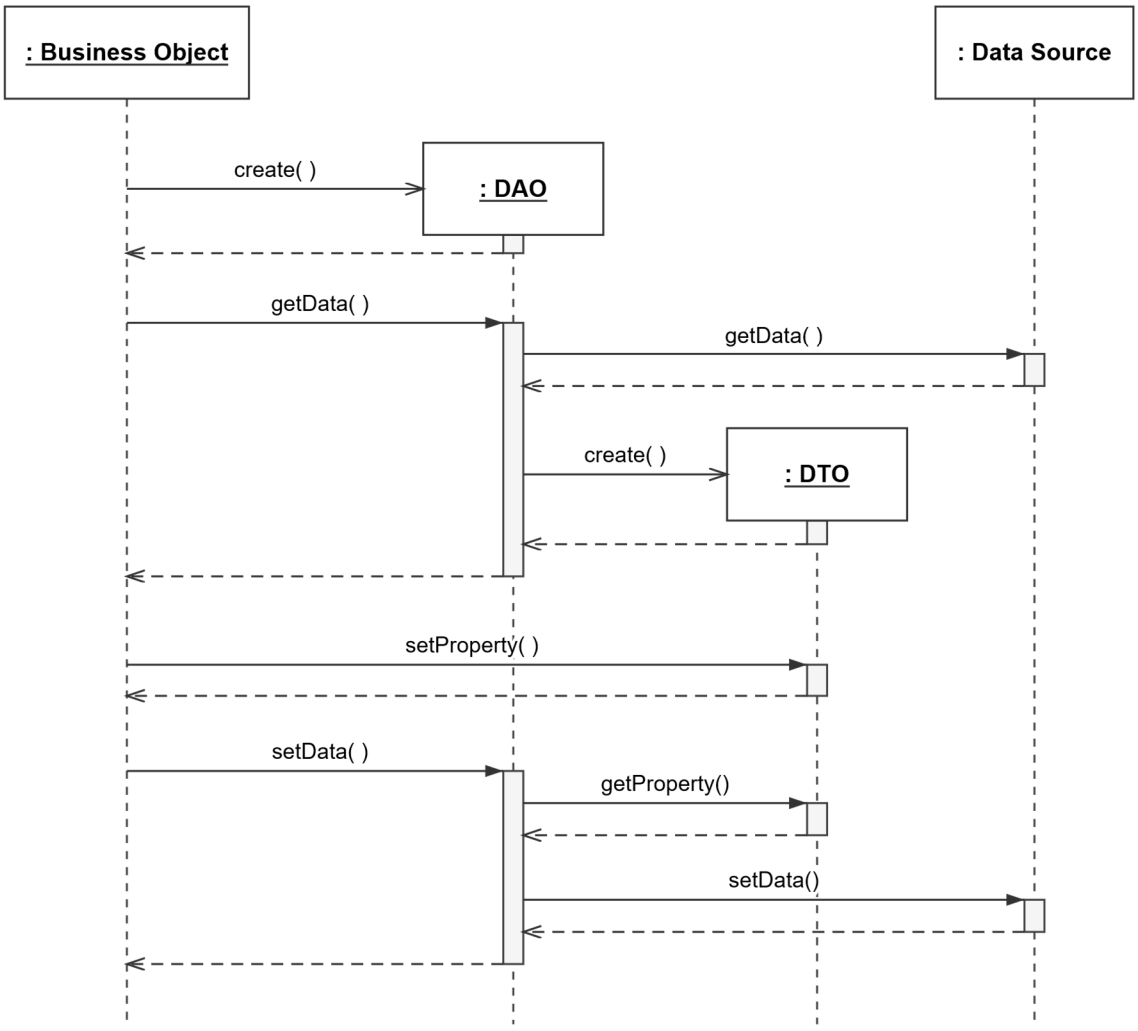


Abbildung 234: Ablauf bei Nutzung des DAO-Musters (nach [Star24])

Bei weitergehenden Anforderungen an die Persistenzschicht werden meist Persistenzframeworks eingesetzt. Mit JPA (Java Persistence API) steht eine Spezifikation für die Persistierung

von Java-Objekten in relationalen Datenbanken zur Verfügung. Hierzu existieren verschiedene Implementierungen, wie z. B. Apache Open JPA, Hibernate und EclipseLink.

JPA sieht die Defintion eines objekt-relationalen Mapping von sogenannten POJOs („Plain Old Java Objects“) mit Hilfe von Annotationen im Java Code oder in einer separaten XML-Datei vor.

Weiterhin ist eine Abfragesprache definiert. Die Implementierungen ermöglichen zumeist auch die Erzeugung des Datenbank-Schemas.

18 Testen

Beim Testen führt man Software aus, um festzustellen, ob sie das tut, was sie soll – und um Fehler zu finden, d. h. Abweichungen vom Sollverhalten.

Testen ist aber längst nicht die einzige Tätigkeit, die sich mit der Qualität von Software befasst. Zum Software-Qualitätsmanagement gehört eine Vielzahl konstruktiver und analytischer Maßnahmen.

Konstruktive Maßnahmen tragen dazu bei, von Anfang an eine möglichst hohe Qualität zu erreichen. Hierzu gehören etwa die Nutzung einheitlicher Styleguides, die Verwendung geeigneter Entwicklungswerkzeuge, die Anwendung geeigneter Entwurfsprinzipien, wie z. B. lose Kopplung, die Nutzung von Entwurfsmustern, und vieles mehr.

Analytische Maßnahmen dienen dazu, die in der Softwareentwicklung erstellten Artefakte (wie Entwurfsdokumente, Quellcode oder ausführbare Programme) zu überprüfen. Dabei sucht man nach vorhandenen Problemen und Fehlern, damit man sie anschließend beheben kann.

Hierfür gibt es eine Reihe von Verfahren, wie z. B. Code-Reviews oder die Ermittlung qualitätsbezogener Kennzahlen. Eine solche Kennzahl könnte etwa die Code-Komplexität durch tiefe Verschachtelung bewerten. Eine zu hohe Code-Komplexität ist ungünstig, da sie leicht zu Fehlern führt. Neben diesen statischen Verfahren spielt vor allem das Testen als dynamisches Verfahren eine wichtige Rolle in jedem Softwareprojekt.

18.1 Grundlagen

18.1.1 Fehler

Wenn man von Fehlern spricht, lassen sich die folgenden Aspekte unterscheiden [Vige10]:

- **Fehlhandlung (engl. Error)**
  Eine Fehlhandlung ist es beispielsweise, wenn ein Entwickler einen Algorithmus falsch implementiert, oder wenn ein Anwender ein System falsch bedient.

- **Fehlerzustand (engl. Fault)**
  Beispiele wären ein inkorrektes Teilprogramm, das bei der Ausführung zu einer Fehlerwirkung führt, oder der Zustand einer Komponente, die unter hoher Belastung die Funktionsfähigkeit beeinträchtigt.

- **Fehlerwirkung (eng. Failure)**
  Hierbei handelt es sich um die Wirkung eines Fehlerzustandes, die nach außen in Erscheinung tritt, beispielsweise indem ein von einem Programm berechneter Ist-Wert nicht mit dem Soll-Wert übereinstimmt.

Nicht jeder Fehlerzustand führt auch immer zu einer Fehlerwirkung. Beispielsweise kann ein Sortieralgorithmus einer amerikanischen Software jahrelang problemlos funktionieren. Erst als die Software auch nach Deutschland verkauft wird, treten plötzlich falsche Sortierungen auf, da bis dahin kaum jemals Umlaute in den Datensätzen verwendet wurden. Ziel eines sorgfältigen Tests sollte es sein, von Vornherein auch solche Szenarien mit zu überprüfen.

18.1.2 Test-Stufen

Es gehört zur alltäglichen Arbeit von Software-Entwicklern, den gerade geschriebenen Code auszuprobieren, also zu testen. Wenn man eine neue Methode geschrieben hat, ruft man diese mit einigen Beispielwerten auf und prüft, ob sie so arbeitet wie beabsichtigt. Hierbei spricht man von einem Unit-Test, d. h. es wird eine einzelne Einheit („Unit“) getestet.

Im einfachsten Fall handelt es sich bei einer solchen Einheit um eine Methode. Daneben testet man aber auch das Zusammenspiel über verschiedene Klassen hinweg, und auch die Schnittstellen kompletter Komponenten.

Schließlich müssen auch das Zusammenspiel der verschiedenen Komponenten und das Gesamtsystem getestet werden.

Man unterscheidet die folgenden Test-Stufen:

- **Komponenten- oder Unit-Test**
  Test einzelner Methoden, Klassen oder Komponenten.
- **Integrationstest**
  Verbinden bereits getesteter Komponenten zu größeren Teilsystemen, deren Zusammenspiel getestet wird.
- **Systemtest**
  Test des kompletten Gesamtsystems.
- **Abnahmetest**
  Grundlage für die Abnahme durch den Kunden.

18.1.3 Unterschiedliche Sichten auf das Testobjekt

Weiterhin kann man unterscheiden, ob nur die Außensicht auf das Testobjekt betrachtet wird, oder auch seine interne Struktur:

- **White-Box-Test**
  Es wird die interne Struktur der Methoden betrachtet. Hierdurch kann man die Tests beispielsweise so gestalten, dass dabei alle Verzweigungen durchlaufen und somit auch alle Anweisungen ausgeführt werden.
- **Gray-Box-Test**
  Hierbei wird das Zusammenwirken der Komponenten betrachtet. So kann man z. B. dafür sorgen, dass bei den Tests auch alle Schnittstellen zwischen den Komponenten

aufgerufen werden. Das Innere der Komponenten wird aber nicht weiter berücksichtigt.

- **Black-Box-Test**
  Man behandelt das Testobjekt als schwarzen Kasten, dessen interne Struktur überhaupt nicht berücksichtigt wird. Es wird lediglich überprüft, ob bestimmte Eingaben zu den gewünschten Ausgaben führen.

18.1.4 Weitere Begriffe

Wie wird festgelegt, was ein „richtiges Ergebnis" oder ein „erwünschtes Verhalten" eigentlich ist? Zum einen kann man überprüfen, ob das Testobjekt das tut, was in einer schriftlichen Spezifikation festgehalten wurde (z. B. dem Pflichtenheft). In diesem Fall spricht man von **Verifikation**. Dies macht man insbesondere bei Unit- und Integrationstests.

Im Gegensatz dazu überprüft man bei System- und Abnahmetests auch, ob das Testobjekt den Kundenanforderungen tatsächlich entspricht. Damit werden auch die nicht explizit formulierten Wünsche, Anforderungen und Vorstellungen berücksichtigt. In diesem Fall spricht man von **Validierung**.

Wenn man nach dem Testen die gefundenen Fehler behebt, muss anschließend erneut getestet werden. Zum einen muss man überprüfen, ob die Fehler tatsächlich erfolgreich behoben wurden. Zum anderen sollten aber auch die bereits vorher erfolgreichen Tests wiederholt werden. Schließlich ist es möglich, dass man bei der Fehlerbehebung versehentlich neue Fehler erzeugt hat, die dafür sorgen, dass vorher fehlerfreie Funktionen nun plötzlich nicht mehr korrekt arbeiten. Die Wiederholung der bereits durchgeführten Tests nennt man **Regressionstests**. Die oft mehrfache Wiederholung einer großen Zahl von Tests ist sehr aufwändig, daher ist eine möglichst weitgehende Automatisierung des Testens wünschenswert.

Grundlage für das Durchführen eines Tests ist der sogenannte **Testfall**. Darin werden der Testablauf und die Testdaten beschrieben. Der Testablauf umfasst die beim Test durchzuführenden Schritte. Zu den Testdaten gehören die Eingabedaten und die erwarteten Resultate.

18.2 Testfallexplosion

Eine grundsätzliche Problematik beim Testen besteht darin, dass man nicht alle möglichen Fälle testen kann. Dies ist schon bei sehr kleinen Methoden der Fall. Das wird anhand der in Listing 47 aufgeführten Methode illustriert.

Die Methode „*funktion( )*" hat insgesamt 7 Eingabeparameter. Möchte man die Methode vollständig testen, so muss man geeignete Testfälle aufstellen, mit jeweils unterschiedlichen Kombinationen der Werte für a bis g. Dabei ist es auf jeden Fall erforderlich, sämtliche Anweisungen zu durchlaufen. Wird eine Anweisung bei den Tests nicht durchlaufen, so würde ein Fehler darin nicht gefunden. Also wird u. a. ein Testfall benötigt, bei dem a>b ist, und ein anderer, bei dem a<=b ist. Nun genügt es aber nicht, nur alle Anweisungen zu durchlaufen.

Es könnte z. B. im Block „Anweisungen 2“ eine Variable einen Wert zugewiesen bekommen, der in „Anweisungen 4“ ein Problem verursacht, in „Anweisungen 5“ jedoch nicht. Also sollte man auch sicherstellen, dass sämtliche möglichen Pfade durchlaufen werden.

```
public void funktion(double a, double b, double c,
              double d, double e,  double f, double g){
    for(int i=0; !fertig && i<20; i++){
        if(a>b){
            // ... Anweisungen1
        } else {
            if (c<d){
                if(d==e){
                    // ... Anweisungen2
                } else {
                    // ... Anweisungen3
                }
            } else {
                if(g>f){
                    // ... Anweisungen4
                } else {
                    // ... Anweisungen5
                }
            }
        }
    }
}
```

Listing 47: Zu testende Methode mit einigen Verzweigungen

Zudem enthält die Methode eine for-Schleife, die – je nach Wert der Variablen „*fertig*“ – bis zu zwanzig Mal durchlaufen werden kann. Nun könnte es Fehler geben, die beim neunmaligen Durchlaufen der Schleife auftauchen, aber nicht beim vierzehnmaligen Durchlaufen der Schleife. Also müsste man Testfälle aufstellen, bei denen die Schleife einmal, zweimal, dreimal usw. durchlaufen wird. Und dabei müsste man noch jeweils die Durchläufe aller Kombinationen von Pfaden durch die if-Anweisungen miteinander kombinieren.

Zur näheren Betrachtung stellt man einen Kontrollflussgraphen der zu testenden Methode auf. Hierbei werden die Anweisungen aus dem Code als Knoten dargestellt, die entsprechend ihrer Reihenfolge unter Berücksichtigung von Verweigungen und Schleifen miteinander verbunden werden. Aufeinanderfolgende Anweisungen ohne zwischendurch ein- oder ausgehende Kanten können dabei zu einem Knoten zusammengefasst werden.

Der Kontrollflussgraph für die Methode aus Listing 47 wird in Abbildung 235 gezeigt. Die Knoten wurden nummeriert um im Text auf die Nummern verweisen zu können.

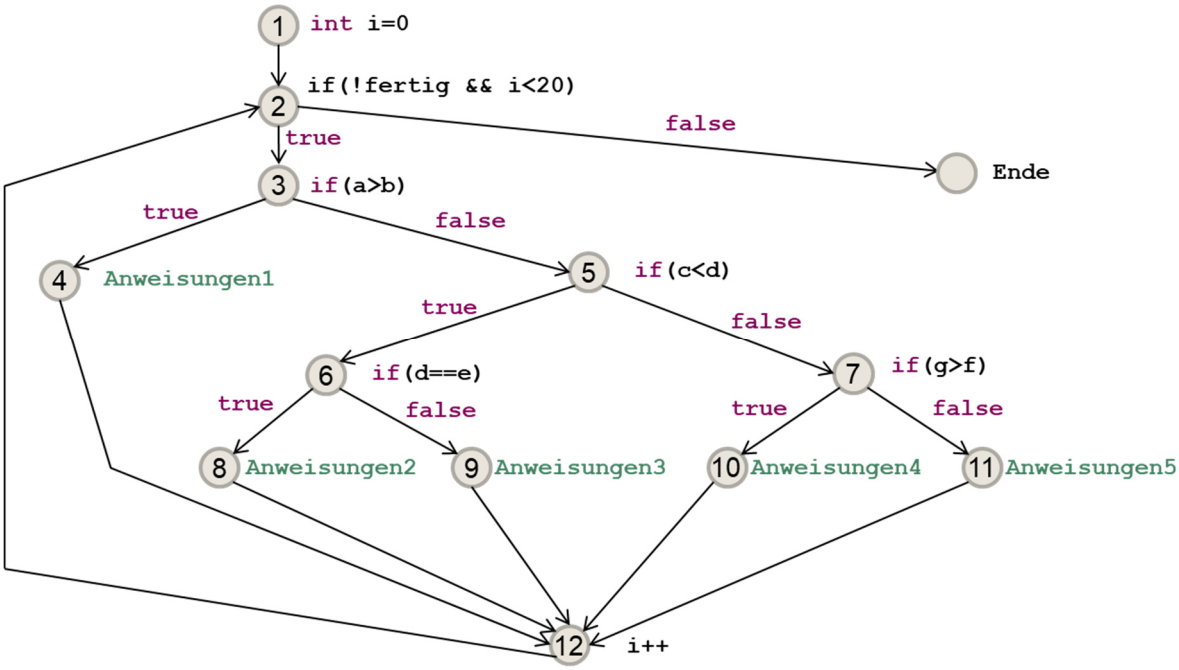


Abbildung 235: Kontrollflussgraph der Methode aus Listing 47

Nun kann man sich überlegen, wieviele mögliche Pfade es durch diesen Kontrollflussgraphen gibt:

- Bei einmaligem Durchlaufen der Schleife: 5
  Pro Testfall kann ja nur einer der Knoten 4, 8, 9, 10 oder 11 durchlaufen werden.
- Bei zweimaligem Durchlaufen der Schleife: $5 \times 5 = 5^2$
  Es wird jeder der fünf Fälle des ersten Durchlaufs mit jedem der fünf Fälle im zweiten Durchlauf kombiniert.
- Bei dreimaligem Durchlaufen der Schleife: $5 \times 5 \times 5 = 5^3$
- Bei zwanzigmaligem Durchlaufen der Schleife: 5^{20}
- Da die Schleife einmal oder zweimal oder dreimal usw. durchlaufen werden kann, müssen die Zahlen für diese unterschiedlichen Fälle addiert werden:
  $5 + 5^2 + 5^3 + \ldots + 5^{20}$
- Es ergibt sich eine Gesamtzahl von ca. 100 Billionen Testfällen.

Würde man diesen Test manuell durchführen und bräuchte man dazu fünf Minuten pro Testfall, so wäre man ca. 1 Milliarde Jahre beschäftigt. Bei einer automatisierten Testdurchführung mit 6 Mikrosekunden pro Testfall würden etwa 20 Jahre benötigt.

Es ist also bereits bei dieser kleinen Methode unmöglich, alle Pfade zu testen. Umso mehr gilt dies für eine größere Software mit vielen Zeilen Code.

Da es also unmöglich ist, einen vollständigen Test durchzuführen, sollte man die Testfälle so auswählen, dass man möglichst große Chancen hat, eventuell vorhandene Fehler zu finden. Hierzu dienen die im Folgenden beschriebenen Verfahren der Äquivalenzklassenbildung und des Einsatzes von Code-Überdeckungsmaßen.

18.3 Äquivalenzklassenbildung

Die Äquivalenzklassenbildung ist ein Black-Box-Verfahren, d. h. man betrachtet die innere Struktur des Testobjekts nicht.

Es wird die Menge aller möglichen Eingabewerte in Äquivalenzklassen unterteilt. Diese Einteilung erfolgt so, dass sich das Testobjekt für alle Eingabewerte innerhalb einer Äquivalenzklasse gleich verhalten müsste. Die Testfälle werden dann so gebildet, dass aus jeder Äquivalenzklasse nur einige Repräsentanten getestet werden.

Als Beispiel wird die mathematische Betragsfunktion betrachtet. Sie gibt für positive Werte und Null den Wert selbst zurück, für negative Werte ändert sie das Vorzeichen.

Hier lassen sich zwei Äquivalenzklassen unterscheiden: Die positiven Werte (inkl. Null) und die negativen Werte. Möchte man eine Methode testen, die den Betrag ermitteln soll, so sollte man Repräsentanten jeder der beiden Äquivalenzklassen testen, d. h. ein oder mehrere positive und ein oder mehrere negative Werte.

Möchte man etwa nur zwei Testfälle verwenden, so ist es wesentlich sinnvoller, die Werte -17 und +84 als Eingaben zu verwenden, als z. B. die Werte 33 und 57. Wenn die Methode für den einen positiven Wert das richtige Ergebnis zurückgibt, ist es wahrscheinlich, dass sie dies für andere positive Werte ebenfalls tut, da diese von der Definition der Betragsfunktion her gleich behandelt werden. Da negative Werte anders behandelt werden, ist es wesentlich wahrscheinlicher, dass die Vorzeichenumkehr nicht oder falsch durchgeführt wird.

Neben Äquivalenzklassen für gültige Eingaben sollte man auch Äquivalenzklassen für ungültige Eingaben bilden, um zu überprüfen, wie das Testobjekt damit umgeht. Interessant sind insbesondere auch die Prüfung der Grenzen von Äquivalenzklassen und Werte in der Nähe von Grenzen, da diese fehlerträchtig sind. Nicht zuletzt sollten auch sehr große und sehr kleine Werte getestet werden um Wertebereichsüberschreitungen zu überprüfen.

18.3.1 Beispiel zur Äquivalenzklassenbildung

Im Folgenden wird die Äquivalenzklassenbildung an einem Beispiel erläutert (entnommen [Kleu26]). Hierbei soll folgender Konstruktor einer Klasse „Studierender“ getestet werden:

```
public Studierender(String name, int geburtsjahr, Fachbereich fachbereich)
```

Dabei sollen folgende Gültigkeitsregeln gelten:

- Der Name darf nicht leer sein.

- Das Geburtsjahr muss zwischen 1900 und 2010 liegen (1900 und 2010 sind mit eingeschlossen).
- Der Fachbereich enstammt der folgenden Aufzählung: IMST, BWL, AING.

Es lassen sich nun folgende Äquivalenzklassen aufstellen:

Ä1 Name nicht leer (gültig)
Ä2 Name leer (ungültig)
Ä3 Geburtsjahr < 1900 (ungültig)
Ä4 1900 ≤ Geburtsjahr ≤ 2010 (gültig)
Ä5 Geburtsjahr > 2010 (ungültig)
Ä6 Fachbereich IMST (gültig)
Ä7 Fachbereich BWL (gültig)
Ä8 Fachbereich AING (gültig)

Beim Aufstellen der Testfälle wählt man nun konkrete Werte aus diesen Äquivalenzklassen, z. B. „Schmidt" für Ä1 und „1987" für Ä4.

Da der Konstruktor drei Parameter benötigt, muss jeder Testfall eine Kombination dieser drei Parameter enthalten. Gültige Werte können miteinander kombiniert werden. Beim Testen ungültiger Werte müssen die anderen Werte gültig sein, da man sonst nicht weiß, ob jeder der ungültigen Werte für sich alleine bereits einen Abbruch erzeugt (z. B. durch eine Exception).

Die Äquivalenzklassen können nun beispielsweise mit den in Tabelle 21 aufgeführten Testfällen getestet werden. Testfall 1 fasst drei gültige Äquivalenzklassen zusammen. Bei den Testfällen 4, 5 und 6 ist jeweils ein Wert ungültig, die anderen zwei gültig. Die ungültigen Werte sind markiert.

Testnr	**1**	**2**	**3**	**4**	**5**	**6**
Geprüfte Äquivalenz-klassen	Ä1, Ä4, Ä6	Ä7	Ä8	Ä2	Ä3	Ä5
Name	„Meyer"	„Schmidt"	„Schulz"	„"	„Becker"	„Maurer"
Geburtsjahr	1987	1989	1985	1980	1892	2012
Fachbereich	IMST	BWL	AING	IMST	BWL	AING
Ergebnis	ok	ok	ok	Ab-bruch	Ab-bruch	Ab-bruch

Tabelle 21: Testfälle für den Konstruktor „Studierender"

Insbesondere bei Zahlen sollten nicht nur Werte irgendwo in der Mitte des Wertebereichs getestet werden, sondern auch die Grenzen. Oft treten Fehler gerade an Grenzen von Wertebereichen auf, z. B. wenn statt dem Vergleichsoperator „<=" versehentlich „<" verwendet wurde.

In dem Beispiel betrifft das die Jahreszahlen. Hier sind die Grenzwerte:

Ä3O	Obergrenze für Ä3:	1899
Ä4U	Untergrenze für Ä4:	1900
Ä4O	Obergrenze für Ä4:	2010
Ä5U	Untergrenze für Ä5:	2011

Sie können mit den zusätzlichen Testfällen in Tabelle 22 getestet werden.

Testnr	7	8	9	10
Geprüfte Äquivalenz-klassen	Ä4U	Ä4O	Ä3O	Ä5U
Name	„Meyer“	„Schmidt“	„Schulz“	„Schneider“
Geburtsjahr	1900	2010	1899	2011
Fachbereich	IMST	BWL	AING	IMST
Ergebnis	ok	ok	Abbruch	Abbruch

Tabelle 22: Weitere Testfälle für die Grenzwertanalyse

18.3.2 Abhängigkeiten der Parameter

In vielen Fällen können sich die Parameter gegenseitig beeinflussen. So soll die folgende Methode den größten von drei Werten zurückgeben:

```
public static int max(int x, int y, int z)
```

Als Werte sind beliebige ganzzahlige Werte zulässig, d. h. es gibt pro Parameter eigentlich nur eine Äquivalenzklasse. Allerdings können sich die Werte gegenseitig beeinflussen. Dies wird deutlich, wenn man die fehlerhafte Implementierung aus Listing 48 betrachtet.

```
public static int max(int x, int y, int z) {
	int max = 0;
	if (x > z)
		max = x;
	if (y > x)
		max = y;
	if (z > y)
		max = z;
	return max;
}
```

Listing 48: Fehlerhafte Implementierung der Methode „max“ (Quelle: [Kleu26])

Testet man die Methode mit den folgenden drei Testfällen, so wird der Maximalwert jeweils richtig zurückgegeben und der Fehler aus Listing 48 wird nicht gefunden:

```
max(1, 2, 3)
max(3, 2, 1)
max(2, 3, 1)
```

Hingegen tritt die Fehlerwirkung bei den folgenden beiden Testfällen auf:

```
max(3, 1, 2);
max(3, 3, 3);
```

Eigentlich müsste man alle unterschiedlichen Kombinationen testen, bei denen x=y, x>y, x<y, x=z, x<z, x>z, y=z, y>z, y<z. Bei größeren Beispielen führt auch dies wieder zu Testfallexplosionen.

18.3.3 Äquivalenzklassen und Objektorientierung

In der objektorientierten Programmierung hängen die Ergebnisse einer Methode oft auch von Attributwerten ab. Solche Attributwerte sind bei der Testfallerstellung als zusätzliche Parameter zu berücksichtigen.

Beispielsweise soll in der Methode „*einkaufssummePruefen( )*" in Listing 49 geprüft werden, ob eine Bestellung ohne weitere Liquiditätsprüfung genehmigt wird. Für Kunden, die bereits geprüft sind, soll die Bestellung immer möglich sein, für sonstige Kunden muss ab 500 € noch eine Prüfung stattfinden. Beim Testen der Methode muss neben der Bestellsumme auch der Wert des Attributs „*istGeprueft*" berücksichtigt werden.

```
public class Zahlungspruefung {
	private boolean istGeprueft;

	public boolean einkaufssummePruefen(int wert){
		boolean ergebnis = false;
		if(istGeprueft || wert<500){
			ergebnis = true;
		}
		return ergebnis;
	}

	public void setGeprueft(boolean geprueft){
		this.istGeprueft = geprueft;
	}
}
```

Listing 49: Das Ergebnisse von „einkaufssummePruefen" hängt vom Wert des Attributs „istGeprueft" ab (Quelle: [Kleu26])

18.4 Code-Überdeckung beim Testen

Im Folgenden wird betrachtet, welche Teile des Codes in welchen Kombinationen mit Hilfe verschiedener Testfälle erreicht werden. Hierzu muss man sich den Code ansehen. Es handelt sich also um White-Box-Tests.

Als Beispiel wird die Methode „*bonus( )*" aus Listing 50 verwendet. Sie enthält die zwei Input-Parameter „*score*" und „*alt*" und gibt eine Zahl zurück.

Als Grundlage für die Bestimmung verschiedener Überdeckungsmaße wird zunächst der Kontrollflussgraph der Methode erstellt (Abbildung 236).

```
public int bonus(int score, int alt) {
	int ergebnis = 0;
	if (score > 2600)
		ergebnis = 20;
	else
		ergebnis = 5;
	if (alt > 42 & score > 900)
		ergebnis += 10;
	return ergebnis;
}
```

Listing 50: Zu testende Methode zur Bonusberechnung (Quelle: [Kleu26])

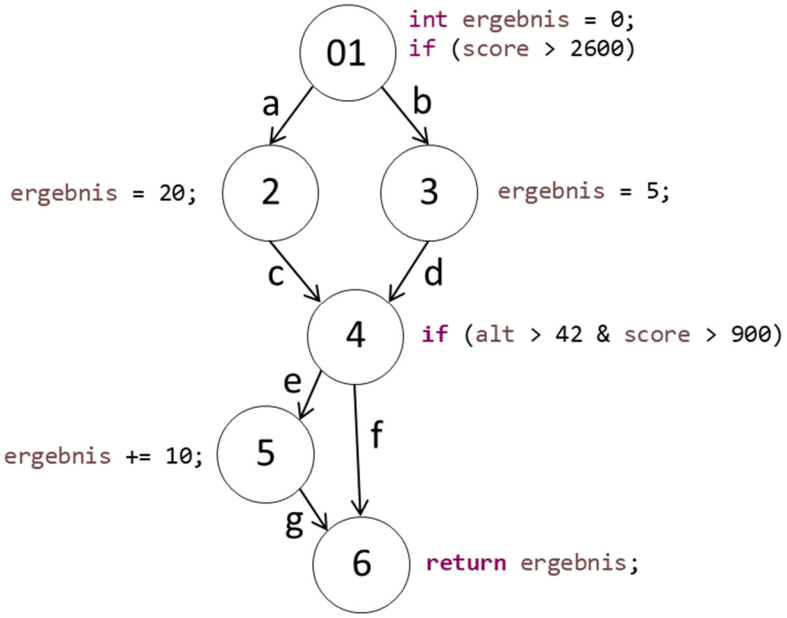


Abbildung 236: Kontrollflussgraph der Methode aus Listing 50 (nach [Kleu26])

18.4.1 Anweisungsüberdeckung

Die Anweisungsüberdeckung gibt den Anteil der durch die Testfälle ausgeführten Anweisungsknoten an der Gesamtzahl der Anweisungsknoten an.

So erreicht man mit dem Testfall

T1 (score=2601, alt=43)

die Knoten 01, 2, 4, 5 und 6 und somit 5 von 6 Knoten. Die Anweisungsüberdeckung beträgt somit 5/6.

Da sich das Erreichen der Knoten 2 und 3 gegenseitig ausschließt, werden insgesamt zwei Testfälle benötigt, um eine hundertprozentige Anweisungsüberdeckung zu erreichen, beispielsweise die folgenden beiden Testfälle:

T1 (score=2601, alt=43) – erreichte Knoten: 01, 2, 4, 5, 6
T2 (score=2600, alt=88) – erreichte Knoten: 01, 3, 4, 5, 6

18.4.2 Zweigüberdeckung

Die Zweigüberdeckung gibt den Anteil der besuchten Kanten an der Gesamtzahl der Kanten an.

Im obigen Beispiel erreicht man mit dem Testfall T1 die Kanten a, c, e und g. Das sind vier von sieben Kanten. Die Zweigüberdeckung beträgt also 4/7.

Nimmt man noch den Testfall T2 hinzu, so erreicht dieser die Kanten b, d, e und g. Zusammen werden damit 6 der 7 Kanten erreicht, die Überdeckung ist damit 6/7.

Die Kante f wird bislang nicht erreicht. Hierzu benötigt man entweder einen dritten Testfall, oder man ersetzt T2 durch den Testfall

T3 (score=900, alt=43)

Dieser erreicht die Kanten b, d und f. Mit T1 und T3 erreicht man somit eine Zweigüberdeckung von 100% (und im Übrigen auch eine hundertprozentige Anweisungsüberdeckung).

Die Zweigüberdeckung unterscheidet sich von der Anweisungsüberdeckung dadurch, dass auch sämtliche leeren else-Zweige durchlaufen werden. Hierdurch können tendenziell mehr Fehler gefunden werden, da die Anweisung, die alternativ zu dem leeren else-Zweig durchgeführt wird, möglicherweise Werte so ändert, dass in einer späteren Anweisung kein Fehler auftritt, wohingegen die unveränderten Werte zum Fehler führen.

Dies wird an dem Codeausschnitt in Listing 51 deutlich. Um eine vollständige Anweisungsüberdeckung zu erreichen, genügt es, den Fall „*bedingung==true*" zu prüfen. Hierbei tritt kein Fehler auf. Eine vollständige Zweigüberdeckung verlangt zusätzlich, den Fall „*bedingung==false*" zu testen, bei dem die Anweisung „*b=1*" übersprungen wird. Da b nun den ursprünglichen Wert „0" behält, führt die letzte Zeile aufgrund der Division durch Null zu einer Exception.

```
int b = 0;
if(bedingung) {
    b = 1;
}
c = 5 / b;
```

Listing 51: Hier muss der leere else-Zweig überprüft werden, um den Fehler zu finden

18.4.3 Pfadüberdeckung

Bei einer Pfadüberdeckung muss jeder Pfad, d. h. jede mögliche Kombination von Zweigen durchlaufen werden. Im Beispiel aus Abbildung 236 gibt es zwei Verzweigungen mit je zwei ausgehenden Zweigen. Hier muss nun jeder an der oberen Verzweigung gewählte Zweig mit jedem an der unteren Verzweigung gewählten Zweig kombiniert werden. Es ergeben sich insgesamt also 2 x 2 = 4 mögliche Pfade:

1. Kanten a – c – e – g
2. Kanten b – d – e – g

3. Kanten b – d – f
4. Kanten a – c – f

Mit den oben aufgestellten Testfällen T1, T2 und T3 werden bereits die ersten drei Fälle abgedeckt. Für den vierten Pfad wird noch ein weiterer Testfall benötigt, z. B.:

T4 (score=3000, alt=30)

Dass man mittels Pfadüberdeckung Fehler finden kann, die man mit einer reinen Zweigüberdeckung unter Umständen nicht findet, lässt sich wieder an einem einfachen Beispiel zeigen. Testet man die Methode in Listing 52 beispielsweise mit den Wertepaaren (x=2, y=-1) und (-3, y=5), so erreicht man damit eine vollständige Zweigüberdeckung, findet aber den Fehler nicht.

Verwendet man hingegen alle vier, für eine Pfadüberdeckung erforderlichen Testfälle, so wird auch der Fall x>0 und y>0 geprüft, der in der Zeile „*return 1 / a*" zu einer Division durch Null und damit zu einer Exception führt.

```
public static int pfadueberdeckung(int x, int y) {
	int a;
	if(x>0) {
		a = 0;
	} else {
		a = 1;
	}
	if(y>0) {
		return 1 / a;
	} else
		return a;
}
```

Listing 52: Mit reiner Zweigüberdeckung wird der Fehler unter Umständen nicht gefunden

Dass in den hier gezeigten Beispielen immer eine Division durch Null als Fehler verwendet wird, liegt daran, dass es sich um einen einfachen, ohne weitere Erklärungen nachvollziehbaren Fehler handelt. Das Prinzip gilt für alle Arten von Fehlern, also z. B. auch einfach für eine fehlerhafte Berechnung eines Wertes.

Noch nicht betrachtet wurden Schleifen. Schleifen können unter Umständen sehr oft durchlaufen werden, auch steht die Zahl der Schleifendurchläufe häufig nicht vorher fest. Meist fordert man, dass eine gewisse Mindestzahl von Schleifen durchlaufen wird.

Nicht immer sind auch alle Pfade erreichbar. Taucht etwa die Bedingung „a>0" an mehreren Verzweigungen auf, so ist es natürlich nicht möglich, an der ersten Verzweigung die Kante für den Fall „a>0" zu wählen und diese mit der Kante für den Fall „a<=0" an der zweiten Verzweigung zu kombinieren.

Eine vollständige Pfadüberdeckung ist meist sehr aufwändig und daher in der Praxis meist nicht realisierbar – wie auch das oben besprochene Beispiel zur Testfallexplosion zeigt.

18.4.4 Bedingungsüberdeckung

Durch die bisher betrachteten Überdeckungsarten können unter Umständen wichtige Situationen bezüglich der Variablenwerte übersehen werden. So können beispielsweise bei der Anweisung „*if (a || b)*" beide ausgehenden Zweige erreicht werden ohne dass die Variable b jemals den Wert „*true*" erhält (indem man a einmal den Wert „true" und einmal den Wert „false" zuweist). Ein Fehler, der nur dann auftritt, wenn b true ist, würde dann nicht gefunden.

Für die verschiedenen Arten der Bedingungsüberdeckung werden daher die im Programmcode verwendeten Bedingungen näher analysiert. Hierbei zerlegt man zusammengesetzte Bedingungen in atomare Bedingungen, die sich nicht weiter zerlegen lassen, wie z. B. „*x>=5*".

Auch negierte atomare Bedingungen gelten als atomar. Ein Beispiel ist „*!(x>=5)*". Dies entspricht der atomaren Bedingung „*x<5*".

Als Beispiel wird der folgende Ausdruck betrachtet:

```
( (x==1) || (x>10) ) && ( (y<5) || (y>=40) )
```

Er lässt sich in folgende atomare Bedingungen zerlegen:

- `x==1`
- `x>10`
- `y<5`
- `y>=40`

Einfache Bedingungsüberdeckung

Die einfache Bedingungsüberdeckung wird erreicht, wenn jede im Programm vorkommende atomare Bedingung einmal zu true und einmal zu false ausgewertet wird.

Die bereits betrachtete Methode zur Bonusberechnung (Listing 53) enthält drei atomare Bedingungen:

1. `score > 2600`
2. `alt > 42`
3. `score > 900`

Die einfache Bedingungsüberdeckung lässt sich beispielsweise mit den folgenden beiden Testfällen erreichen

T1 (score=2601, alt=30)
T2 (score=800, alt=44)

Bei T1 sind die erste und dritte Bedingung wahr und die zweite falsch. Bei T1 ist die zweite Bedingung wahr, die erste und dritte dagegen falsch.

An dem Beispiel wird deutlich, dass die einfache Bedingungsüberdeckung nicht zugleich die Anweisungs- oder Zweigüberdeckung sicherstellt, denn bei keinem der beiden Testfälle wird die Anweisung „*ergebnis += 10*" erreicht.

```
public int bonus(int score, int alt) {
	int ergebnis = 0;
	if (score > 2600)
		ergebnis = 20;
	else
		ergebnis = 5;
	if (alt > 42 & score > 900)
		ergebnis += 10;
	return ergebnis;
}
```

Listing 53: Zu testende Methode zur Bonusberechnung (Quelle: [Kleu26], Wiederholung von Listing 50)

Minimale Mehrfachbedingungsüberdeckung

Bei der minimalen Mehrfachbedingungsüberdeckung werden alle Teilbedingungen betrachtet. Neben den atomaren Bedingungen sind dies alle zusammengesetzten Ausdrücke. Für den bereits betrachteten Ausdruck

```
( (x==1) || (x>10) ) && ( (y<5) || (y>=40) )
```

sind dies:

- `x==1`
- `x>10`
- `y<5`
- `y>=40`
- `(x==1) || (x>10)`
- `(y<5) || (y>=40)`
- `( (x==1) || (x>10) ) && ( (y<5) || (y>=40) )`

Nun muss jede Teilbedingung einmal wahr und einmal falsch sein.

Die minimale Mehrfachbedingungsüberdeckung beinhaltet die Zweig- und damit auch die Anweisungsüberdeckung.

Für das Beispiel aus Listing 55 kommt zu den drei oben aufgeführen atomaren Bedingungen

1. `score > 2600`
2. `alt > 42`
3. `score > 900`

noch die folgende zusammengesetzte Bedingung hinzu:

4. `alt > 42 & score > 900`

Um jede der vier Bedingungen einmal wahr und einmal falsch werden zu lassen, kann man beispielsweise folgende zwei Testfälle verwenden:

T1 (score=2601, alt=45)
T2 (score=800, alt=30)

Bei T1 sind alle Bedingungen wahr, bei T2 alle falsch.

Auch bei den Bedingungsüberdeckungen kann man weitergehende Überdeckungsmaße aufstellen, wie etwa sämtliche Kombinationen aus wahr und falsch ausgewerteten atomaren Bedingungen. Ähnlich wie bei der vollständigen Pfadüberdeckung führt auch dies schnell zu Testfallexplosionen.

Erreichen der Bedingungen

Bei der Prüfung der Bedingungsüberdeckungen ist zu beachten, dass die zu prüfenden Bedingungen auch erreicht werden müssen, da sie ansonsten bei den Tests nicht ausgewertet werden. So wird in Listing 54 die Zeile *„if (b>0 && c>0) {"* nur erreicht, wenn a größer als null ist.

```
public void m(int a, int b, int c) {
      if(a>0) {
            if (b>0 && c>0) {
                  // Anweisungen 1
            } else {
                  // Anweisungen 2
            }
      } else {
            // Anweisungen 3
      }
}
```

Listing 54: Die Bedingungen in Zeile 3 werden nur erreicht, wenn a>0 ist

Die enthaltenen elementaren Bedingungen sind:

1. a>0
2. b>0
3. c>0

Mit den folgenden beiden Testfällen werden alle drei Bedingungen einmal wahr und einmal falsch:

T1(a=1, b=1, c=1)
T2(a=-1, b=-1, c=-1)

Allerdings wird die Zeile *„if (b>0 && c>0) {"* in Testfall T2 gar nicht erreicht. Somit werden die Fälle, dass b>0 und c>0 falsch sind, nicht geprüft. Daher wird hier noch ein dritter Testfall benötigt um tatsächlich eine einfache Bedingungsüberdeckung zu erreichen, z. B.:

T3(a=1, b=-1, c=-1)

18.4.5 Datenflussorientierte Überdeckungen

Häufig hängen Fehler davon ab, welche Werte die Variablen haben. Zusätzlich zur Betrachtung des Kontrollflusses und der Bedingungen kann daher die Berücksichtigung des Datenflusses hilfreich sein.

Für die Methode *„mach( )"* in Listing 55 erreicht man mit den beiden Testfällen

T1(a=true, b=true)
T2(a=false, b=false)

die vollständige Zweigüberdeckung und die minimale Mehrfachbedingungsüberdeckung. Trotzdem werden die Fehler nicht entdeckt. Sie hängen nicht rein vom Kontrollfluss ab, sondern von dem Wert der Variablen x.

```
public static int mach(boolean a, boolean b) {
	int x = 0;
	if (a)
		x = 2;
	else
		x = 3;
	if (b)
		return (6 / (x - 3));
	else
		return (6 / (x - 2));
}
```

Listing 55: Hier müssen die Zusammenhänge bzgl. der Werte von „x" berücksichtigt werden.

Man kann verschiedene datenflussorientierte Überdeckungsmaße formulieren, z. B.:

- Es müssen alle Pfade von einer Definition zu ihrer Nutzung in Entscheidungen getestet werden.
- Es müssen alle Pfade von einer Definition zu ihrer Nutzung in Berechnungen getestet werden.
- Es müssen alle Pfade von einer Definition zu ihrer Nutzung getestet werden (sowohl in Entscheidungen als auch Berechnungen).

Abbildung 237 zeigt den Kontrollflussgraphen der Methode „mach( )". Dabei ist jeweils angegeben, wo Definitionen und wo Nutzungen von Variablen erfolgen.

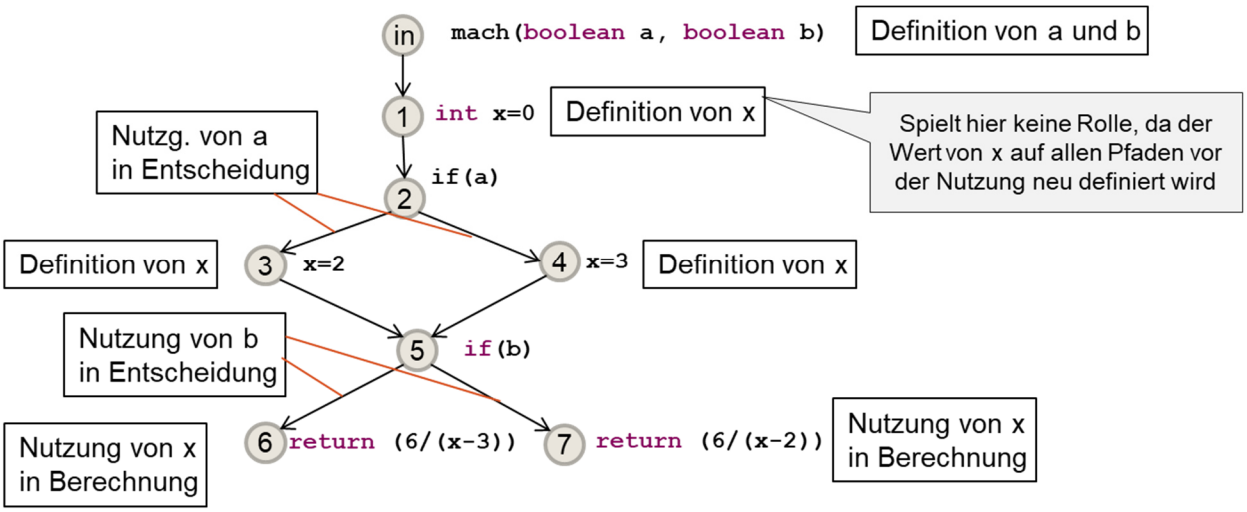


Abbildung 237: Kontrollflussgraph mit Angabe der Definition und der Nutzung von Variablen

Testet man hier alle Pfade von der Definition von x zur Nutzung von x, so werden die Fehler gefunden, da neben den mittels T1 und T2 getesten Pfaden

in-1-2-3-5-6
in-1-2-4-5-7

auch die Pfade

in-1-2-3-5-7
in-1-2-4-5-6

getestet werden müssen. Hierzu dienen die beiden weiteren Testfälle:

T3(a=true, b=false)
T4(a=false, b=true)

Hiermit werden die Fehler gefunden (auch hier jeweils eine Division durch Null).

18.5 Testplanung und -durchführung

Testen ist mit zum Teil erheblichem Aufwand verbunden. Zudem kann man – wie oben beschrieben – nicht alles testen. Von daher muss man sich überlegen, wie und was man im Laufe des Projektes testen möchte, wie man den Testablauf organisiert und wie man mit den Testergebnissen umgeht.

Es sind insbesondere die folgenden Punkte zu klären:

1. Testobjekte und Leistungsmerkmale
2. Teststrategie
3. Testabläufe
4. Test-Ende-Kriterien
5. Testdokumentation

Je nach Umfang des Projekts werden diese Punkte in einem eigenen Dokument festgehalten, dem Testplan. Sie werden im Folgenden näher beschrieben.

18.5.1 Testobjekte und Leistungsmerkmale

Zunächst ist festzulegen, was alles getestet werden soll, wie z. B.

- Klassen
- Komponenten
- Schnittstellen
- Oberflächen
- Abläufe
- Datenbank
- Installationsroutinen
- Build-Files
- Zusammenspiel mit Drittsystemen

Dabei ist auch zu entscheiden, welche Merkmale konkret getestet werden sollen. Die Funktionalität gemäß Anforderungen soll in der Regel möglichst vollständig getestet werden. Zudem sollten die spezifizierten Qualitätsmerkmale getestet werden, wie z. B. Performance, zu bewältigende Datenmengen oder Sicherheitsmerkmale.

Falls es Merkmale gibt, die aus technischen Gründen oder wegen des zu hohen Aufwandes, nicht getestet werden, sollte dies ebenfalls beschrieben werden. Beispielsweise kann festgelegt werden, dass Schnittstellen, die vom Kunden bereitgestellt werden, nicht gesondert getestet werden.

18.5.2 Teststrategie

Hier werden die grundsätzlichen Testziele sowie die Vorgehensweise und Reihenfolge beim Testen beschrieben. Dabei wird auch festgelegt, mit welchen Testmethoden die verschiedenen Merkmale auf den einzelnen Teststufen getestet werden. Beispiele für solche Festlegungen:

- Unit-Tests der einzelnen Klassen und Methoden durch die Entwickler
- Automatisierte Tests für alle öffentlichen Methoden
- Manuell durchgeführte Tests für jeden Use-Case
- Integrationstest mit Top Down- oder Bottom Up-Integration
- Lasttest und Performance-Test nach Integration
- Regressionstests aller automatisierten Tests nach jedem Build
- Stichprobenhafte Regressionstests aller anderen Tests nach größeren Änderungen

Das Testen der nicht-funktionalen Anforderungen ist meist wesentlich vielfältiger und schwieriger als das Testen der funktionalen Anforderungen. Um beispielsweise einen realistischen Lasttest durchführen zu können, muss man erst einmal ermitteln, mit welcher Last überhaupt zu rechnen ist. Die Software muss in einer Hardwareumgebung installiert werden, die der späteren produktiven Umgebung entspricht, und es muss die entsprechende Last erzeugt werden, z. B. mit Hilfe spezieller Softwarewerkzeuge für Lasttests.

Beispiele für Tests nicht-funktionaler Anforderungen:

- Lasttest
- Performance-Test
- Volumen-/Massentest
- Stresstest
    - Was passiert bei Überlastung?
- Test der Sicherheit
    - Z. B. gegen unberechtigten Systemzugang
- Test der Stabilität/Zuverlässigkeit
    - Im Dauerbetrieb
- Test auf Robustheit
    - Gegenüber Fehlbedienung, Hardwareausfall, Wiederanlauf, …

- Test auf Kompabilität/Datenkonversion
- Test unterschiedlicher Konfigurationen
    - Z. B. Sprachen, Betriebssysteme
- Test auf Benutzungsfreundlichkeit
- Prüfung der Dokumentation
- Prüfung auf Änderbarkeit/Wartbarkeit

Für Integrationstests müssen geeignete Integrationsstrategien festgelegt werden. Möchte man das Zusammenspiel von Komponenten testen ohne dass bereits alle Komponenten fertiggestellt sind, so stellt sich das Problem, dass fehlende Komponenten ggf. durch entsprechende Testtreiber oder aber Stubs (Platzhalter) ersetzt werden müssen.

Existiert etwa noch keine grafische Oberfläche, so verwendet man Testtreiber, die die zu testenden Komponenten aufrufen und mit Daten versorgen. Existiert andererseits beispielsweise noch keine Datenhaltung, so kann man einen Platzhalter programmieren, der zu speichernde Daten entgegennimmt oder Suchanfragen mit Beispielwerten beantwortet.

Man kann folgende Integrationsstrategien unterscheiden:

- **Top-Down-Integration**
  Der Test beginnt mit der „obersten" Komponente, die selbst nicht von anderen Komponenten aufgerufen wird. Untergeordnete Komponenten werden zunächst durch Platzhalter ersetzt und später nach und nach gegen die richtigen Komponenten ausgetauscht. Hierdurch braucht man keine Testtreiber, muss aber Platzhalter programmieren.
- **Bottom-Up-Integration**
  Man beginnt mit elementaren Komponenten, die nach und nach zu größeren Komponenten zusammengesetzt werden. Bei dieser Strategie braucht man keine Platzhalter, aber viele Testtreiber.
- **Ad-Hoc-Integration**
  Dabei handelt es sich um eine Integration in der (zufälligen) Reihenfolge der Fertigstellung. Dies kann zu Zeitgewinn führen, aber man braucht sowohl Platzhalter als auch Treiber.
- **Big-Bang-Integration**
  Die Integration erfolgt erst ganz am Ende, wenn alle Komponenten fertiggestellt sind. In diesem Fall werden weder Testtreiber noch Platzhalter benötigt. Dennoch sollte man die Big-Bang-Integration vermeiden, da ein großer Testaufwand am Ende des Projektes entsteht. Fehler treten dann geballt auf und sind recht schwer zu lokalisieren.

Insbesondere Unit- und Integrationstests werden während der Entwicklung fortlaufend durchgeführt. Für einen umfangreichen Systemtest wird häufig vor Fertigstellung eines Release eine eigene Testphase vorgesehen. Hierfür erfolgt ein Entwicklungsstopp, d. h. es findet

keine Weiterentwicklung der auszuliefernden Software mehr statt. Stattdessen werden ausführliche Tests durchgeführt. Die gefundenen Fehler werden anschließend behoben, danach erfolgen weitere Tests usw., bis die definierten Test-Ende-Kriterien (siehe unten) erreicht sind. In den Fehlerbehebungsphasen dürfen die Entwickler ausschließlich die Fehler beseitigen, aber keine neue Funktionalität entwickeln, weil dabei neue Fehler entstehen könnten. Durch diese Vorgehensweise erreicht man in der Regel eine stetig sinkende Fehlerzahl, so dass schließlich eine hinreichende Qualität für die Auslieferung der Software erreicht wird.

18.5.3 Testabläufe

Hier werden die verschiedenen Aufgaben zur Vorbereitung, Durchführung und Auswertung der Tests beschrieben. Hinzu kommt die Festlegung der benötigten Mitarbeiter und Qualifikationen und der notwendigen Testinfrastruktur, wie Hardware, Testwerkzeuge u. ä. Nicht zuletzt müssen die Verantwortlichkeiten für die diversen Testaufgaben definiert werden.

18.5.4 Test-Ende-Kriterien

Test-Ende-Kritierien legen fest, wann man mit dem Testen fertig ist. Beispielsweise könnten in einem Projekt folgende Kriterien definiert werden:

- 100% aller öffentlichen Methoden getestet,
- 90% Zweigüberdeckung in allen Klassentests,
- 100% aller Use-Cases getestet,
- Keine Fehler der Klasse 1 (Systemabsturz, Datenverlust) und Klasse 2 (wesentliche Funktion ist fehlerhaft) mehr bekannt, und
- 90% der gefundenen Fehler der Klasse 3 sind behoben

Falls Tests zu schlecht ausfallen, werden sie abgebrochen und die getesteten Software-Elemente zur Überarbeitung zurückgegeben. Auch werden oft entsprechende Abbruch-Kriterien festgelegt, ebenso wie Kriterien für die Fortsetzung der Tests.

In den obigen Kriterien wurde auf verschiedene Fehlerklassen Bezug genommen. Da die beim Testen gefundenen Fehler unterschiedlich schwere Auswirkungen haben, teilt man sie meist in Klassen ein. Aus [SpLi12] stammt das folgende Beispiel einer fünfstufigen Skala zur Fehlerklassifikation:

Klasse 1	Systemabsturz mit ggf. Datenverlust. Das Testobjekt ist in dieser Form nicht einsetzbar.
Klasse 2	Wesentliche Funktion ist fehlerhaft; Anforderung nicht beachtet oder falsch umgesetzt. Das Testobjekt ist nur mit großen Einschränkungen einsetzbar.
Klasse 3	Funktionale Abweichung bzw. Einschränkung („normaler Fehler“); Anforderung fehlerhaft oder nur teilweise umgesetzt. Das Testobjekt kann mit Einschränkungen genutzt werden.

Klasse 4 Geringfügige Abweichung. Das Testobjekt kann ohne Einschränkung genutzt werden.

Klasse 5 Schönheitsfehler (z. B. Rechtschreibfehler oder Mangel im Maskenlayout). Das Testobjekt kann ohne Einschränkung genutzt werden.

Je nach Fehlerklasse sind unterschiedliche Konsequenzen erforderlich. So sollte ein Produkt, das noch Fehler der Klasse 1 oder Klasse 2 enthält, nicht ausgeliefert werden. Treten solche Fehler während des Einsatzes beim Kunden auf, so sollten direkt entsprechende Bugfixes erstellt werden. Bei Fehlern der Klasse 4 oder 5 ist es hingegen akzeptabel, wenn sie erst später, in einer regulär geplanten neuen Version der Software behoben werden.

18.5.5 Testdokumentation

Zur Testdokumentation gehören die Spezifikation der Testfälle und die Dokumentation der Ergebnisse. Für manuell durchzuführende Tests werden die durchzuführenden Schritte, die zu tätigenden Eingaben und die erwarteten Ausgaben in einer Testfallspezifikation beschrieben. Abbildung 238 zeigt ein mögliches Schema für eine solche Testfallspezifikation. Die Inhalte orientieren sich an [ReRe18]. Eine gute Grundlage zum Aufstellen solcher Testfallspezifikationen bilden die Beschreibungen der Use-Cases (Kapitel 3), da dort die durchzuführenden Schritte und das erwartete Verhalten des Systems beschrieben sind.

Zu den Test-Schritten, Eingaben und erwarteten Ergebnissen kommen noch weitere Festlegungen, wie z. B. in welcher Hard- und Softwareumgebung der Test durchgeführt wird oder welche Voraussetzungen vorliegen müssen. Beispielsweise könnte eine Voraussetzung sein, dass bestimmte Testdatensätze in der Datenbank vorhanden sind, die beim Test genutzt werden.

Nr	
Bezeichnung	
Verantwortlicher	
Durchzuführen von (Rolle)	
Umgebung	
Voraussetzungen	
Zu testende Funktion	
Schritte (inkl. Eingaben)	**Erwartetes Ergebnis**
1.	
2.	
3.	
4.	
5.	
Qualitätsmerkmale und Testkriterien	
Schnittstellen zu anderen Tests und Arbeitspaketen	
Anmerkungen	

Abbildung 238: Schema für eine Testfallspezifikation

Beim manuellen Durchführen eines Tests wird in der Regel ein Testprotokoll angefertigt, z. B. nach einem Schema wie in Abbildung 239. Die Struktur basiert im Wesentlichen auf der Testfallspezifikation. Dort ist festgelegt, was beim Test gemacht werden soll und welche Ergebnisse erwartet werden. Im Testprotokoll wird ergänzend vermerkt, was tatsächlich gemacht wurde und welche Ergebnisse tatsächlich beobachtet wurden. In die grau hinterlegten leeren Feldern in Abbildung 239 werden die Inhalte der Testfallspezifikation eingetragen, die Protokoll-Eintragungen erfolgen in den weißen Feldern.

Entsprechen die Ergebnisse den erwarteten Ergebnissen, so kann dies in dem Formular einfach abgehakt werden. Lediglich bei Abweichungen vom erwarteten Ergebnis ist genau zu beschreiben, welches Ergebnis tatsächlich beobachtet wurde.

Ein solches Testprotokoll ermöglicht es, die Entstehung des Fehlers zu reproduzieren, was eine wichtige Grundlage für die Fehlerbehebung ist.

Testfall Nr		**Protokoll Nr**	
Bezeichnung		**Datum**	
Verantwortlicher **Durchzuführen von (Rolle)**		**Uhrzeit** **Tester** **Weitere Mitwirkende**	
Umgebung		**Tatsächlich vorhandene Umgebung**	
Voraussetzungen		**Tatsächlich vorhandene Voraussetzungen**	
Zu testende Funktion			
Schritte	**Erwartetes Ergebnis**	**Tatsächliches Ergebnis**	**Genaue Fehlerbeschreibung, wenn nicht OK**
1.		☐OK ☐Nicht OK	
2. 3.		☐OK ☐Nicht OK ☐OK ☐Nicht OK	
4.		☐OK ☐Nicht OK	
5.		☐OK ☐Nicht OK	
		☐OK ☐Nicht OK	
Qualitätsmerkmale und Testkriterien		**Tatsächlich beobachtete Merkmale und Kriterien**	
Schnittstellen zu anderen Tests und Arbeitspaketen **Anmerkungen**		**Anmerkungen zu Schnittstellen zu anderen Tests und Arbeitspaketen** **Anmerkungen**	
		Änderungen an der Testumgebung aufgrund der Tests inkl. erforderliche Wiederherstellungsaktionen	
		Unterschrift	

Abbildung 239: Schema für ein Testprotokoll

Schließlich sind die gefundenen Fehler zu sammeln und an die für die Fehlerbehebung zuständigen Entwickler weiterzuleiten. Außerdem muss nachverfolgt werden, ob und wann die Fehler behoben werden.

Im einfachsten Fall erstellt man hierzu in einem Tabellenkalkulationssystem eine Fehlerstatusliste. Darin werden alle gefundenen Fehler mit den wichtigsten Angaben aufgelistet, wie z. B.

- Eindeutige Fehlernummer
- Kurze Beschreibung
- Fehlerklasse
- Melder des Fehlers
- Zuständiger Bearbeiter
- Status (z. B. offen, in Arbeit, behoben, validiert)

Anhand einer solchen Liste lässt sich leicht nachvollziehen, welche Fehler gefunden wurden, welche noch behoben müssen usw.

In größeren Projekten kommen oft spezielle Bug Tracking-Systeme zum Einsatz, die u. a. eine aktive Weiterleitung der Meldungen an die Bearbeiter durchführen, verschiedene Auswertungen und Statistiken erstellen können und zahlreiche Konfigurationsmöglichkeiten enthalten. Neben kommerziellen Programmen gibt es auch eine Reihe von Open Source Bug Tracking-Systemen, z. B. „Bugzilla" (www.bugzilla.org).

18.6 Testautomatisierung

Manuelle Tests sind aufwändig und fehleranfällig. Insbesondere Regressionstests, also die Wiederholung von Tests nach Änderungen, lassen sich manuell nicht in größerem Umfang durchführen. Daher versucht man das Testen möglichst weitgehend zu automatisieren. Hierfür gibt es eine Reihe von Testwerkzeugen und –frameworks. Im Folgenden wird die Arbeit mit JUnit beschrieben, einem weit verbreiteten Testframework für Java. Vergleichbare Frameworks gibt es auch für andere gängige Programmiersprachen.

JUnit ist Open Source. In einigen Entwicklungsumgebungen, wie z. B. Eclipse, ist es bereits integriert und braucht daher nicht mehr separat installiert zu werden. Die Beispiele verwenden die Version JUnit 5.

18.6.1 JUnit-Tests schreiben

Die Tests für JUnit werden in Java geschrieben. Listing 56 zeigt ein Beispiel einer Test-Klasse mit zwei Testfällen.

Die zentralen JUnit-Klassen liegen im Paket org.junit.jupiter.api. Im Beispiel werden die benötigten Klassen Assertions und Test importiert. Testklassen werden häufig so benannt wie die zu testende Klasse, wobei „*Test*" angehängt wird. Es wird hier also eine Klasse „*Mitarbeiter*" getestet.

Bei den eigentlichen Testfällen handelt es sich um Methoden, die mit der Annotation „*@Test*" versehen werden. Meist benennt man die Methode mit einem Namen, der mit „*test*" beginnt und häufig den Namen der zu testenden Methode enthält.

In der Methode „*testKonstruktor( )*“ wird zunächst ein neues Mitarbeiter-Objekt angelegt. Es soll nun getestet werden, ob die Methoden „*getVorname( )*“ und „*getNachname( )*“ der Klasse Mitarbeiter die Strings zurückgeben, die dem Konstruktor übergeben wurden.

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

public class MitarbeiterTest {

  @Test
  public void testKonstruktor(){
    Mitarbeiter m = new Mitarbeiter("Ute","Mai");
    Assertions.assertTrue(m.getVorname().equals("Ute"), "Gleicher Vorname?");
    Assertions.assertTrue(m.getNachname().equals("Mai"), "Gleicher Nachname?");
  }

  @Test
  public void testEindeutigeId(){
    Assertions.assertTrue(
        new Mitarbeiter("Ute","Mai").getId()
        != new Mitarbeiter("Ute","Mai").getId(),
        "IDs unterschiedlich?");
  }
}
```

Listing 56: Beispiel für JUnit-Tests (Nach: [Kleu26])

Hierzu verwendet man sogenannte Assertions (Zusicherungen). Mit „*assertTrue( )*“ wird überprüft, ob ein boolescher Ausdruck den Wert „*true*“ hat. Bei der ersten Zusicherung wird also geprüft, ob „*m.getVorname( ).equals("Ute")*“ den Wert „*true*“ zurückliefert. Ist dies der Fall, so ist die Zusicherung erfüllt und dieser Teil des Tests erfolgreich. Der Test enthält aber noch eine zweite Assertion, mit der der Nachname überprüft wird. Erst, wenn beide Zusicherungen erfüllt sind, ist der gesamte Test „*testKonstruktor( )*“ erfolgreich.

Bei dem zweiten Parameter von „*assertTrue( )*“ handelt es sich um einen String. Wenn die Zusicherung nicht erfüllt ist, wird dieser String zusammen mit dem erwarteten Ergebnis („*true*“) und dem tatsächlichen Ergebnis („*false*“) ausgegeben. Hierdurch weiß man, an welcher Stelle der Test fehlgeschlagen ist. Man kann diesen String aber auch weglassen.

Ein Test kann wie in dem Beispiel mehrere Assertions enthalten. Sobald eine fehlschlägt, werden die anderen Assertions allerdings nicht mehr ausgeführt. Daher kann es sinnvoll sein, nur eine Assertion pro Test zu verwenden.

Mit dem zweiten Testfall „*testEindeutigeId( )*“ wird überprüft, ob zwei neu angelegte Mitarbeiterobjekte mit demselben Vor- und Nachnamen trotzdem eine unterschiedliche ID haben (also einen unterschiedlichen Identifizierer, wie z. B. eine Personalnummer).

Bei „*assertTrue( )*“ handelt es sich um eine statische Methode der Klasse „Assertions“. Um diese Klasse nicht jedesmal angeben zu müssen, kann man auch einen statischen Import der Methoden von „Assert“ durchführen, wie dies in der ersten Zeile von Listing 57 gemacht wurde.

Dann muss man beim Aufruf nicht mehr *„Assertions.assertTrue( )“* schreiben, sondern kann direkt *„assertTrue( )“* verwenden.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class MitarbeiterTest {

  @Test
  public void testKonstruktor(){
    Mitarbeiter m = new Mitarbeiter("Ute","Mai");
    assertTrue(m.getVorname().equals("Ute"), "Gleicher Vorname?");
    assertTrue(m.getNachname().equals("Mai"), "Gleicher Nachname?");
  }
}
```

Listing 57: Durch den statischen Import kann die assertTrue-Methode ohne Angabe des Klassennamens aufgerufen werden (Nach [Kleu26])

18.6.2 JUnit-Tests ausführen

Den Test kann man nun ausführen, allerdings nicht als gewöhnliche Java-Anwendung, sondern als JUnit-Test. In Eclipse wählt man dazu „Run As – JUnit Test“ aus. Im Idealfall erhält man als Ergebnis einen grünen Balken, der angibt, dass alle Tests erfolgreich ausgeführt wurden (Abbildung 240).

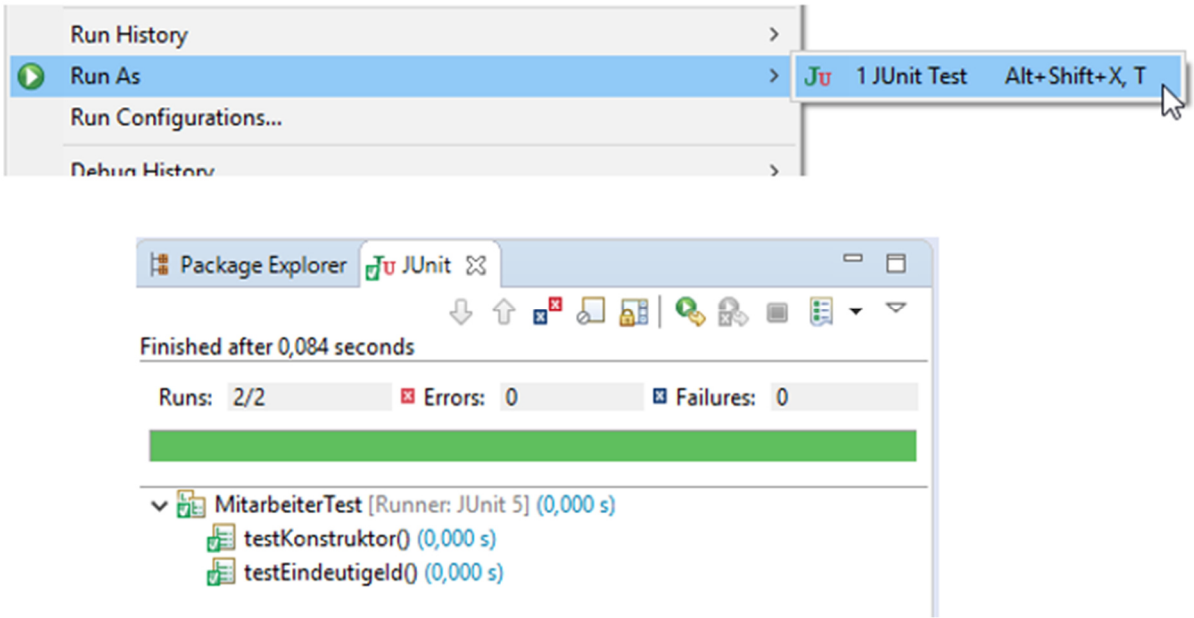


Abbildung 240: JUnit – Testausführung in Eclipse

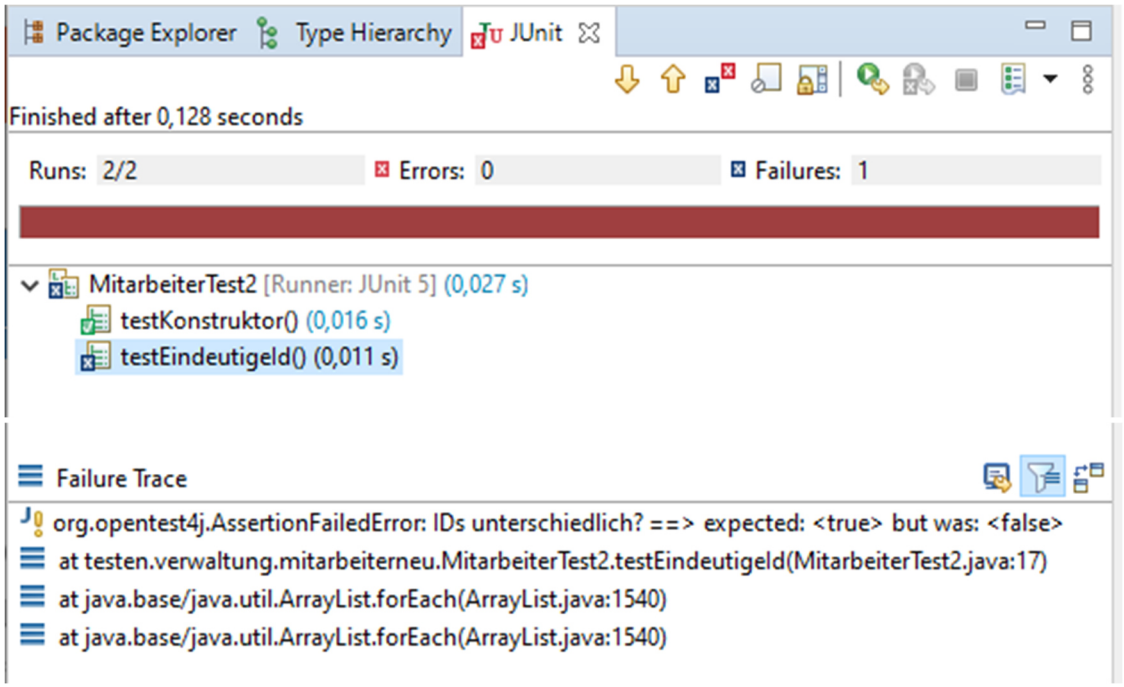


Abbildung 241: JUnit – Anzeige bei Fehlschlag

Schlagen hingegen ein oder mehrere Tests fehl, so erhält man einen roten Balken wie in Abbildung 241. Zudem wird angegeben, wieviele „Failures" (Fehlschläge) aufgetreten sind (rechts oben in der Abbildung).

In der Liste darunter sind die fehlgeschlagenen Tests mit einem speziellen Icon markiert. In dem dargestellten Beispiel ist der Testfall *„testEindeutigeId( )"* fehlgeschlagen. Selektiert man den fehlgeschlagenen Test, so werden im unteren Bereich des Fensters weitere Informationen zum Fehlschlag angezeigt. Insbesondere wird dort auch der bei der Assertion als zweiter Parameter angegebenen String ausgegeben (hier: *„IDs unterschiedlich?"*). Außerdem ist angegeben, dass der Fehlschlag in Zeile 17 der Testklasse *„MitarbeiterTest2.java"* aufgetreten ist.

18.6.3 Assert-Methoden

Neben der bereits verwendeten Methode *„assertTrue( )"* gibt es noch eine Reihe weiterer assert-Methoden, wie z. B.

```
assertArrayEquals(Object[] expected, Object[] actual)
assertEquals(Object expected, Object actual)
assertEquals(double expected, double actual, double delta)
assertFalse(boolean condition)
assertNotNull(Object object)
assertNull(Object object)
assertSame(Object expected, Object actual)
assertTrue(boolean condition)
fail()
```

Alle diese Methoden gibt es auch noch als Variante mit einem String-Parameter für eine Ausgabe beim Scheitern des Tests.

Anstelle von „*assertTrue( )*“ für den Ausdruck „*m.getVorname().equals("Ute")*“ hätte man im obigen Beispiel auch „*assertEquals(m.getVorname(), "Ute")*“ verwenden können. Der Vorteil von „*assertEquals( )*“ ist, dass bei einem Fehlschlag der erwartete und der tatsächliche Wert ausgegeben werden.

Die Variante von „*assertEquals( )*“ für double-Werte enthält einen dritten Parameter „*delta*“. Da Berechnungen von Fließkommazahlen aufgrund von Rundungsfehlern meist nicht exakt den gleichen Wert liefern, kann man mit diesem Parameter angeben, bis zu welcher Differenz zwei Werte noch als gleich angesehen werden sollen.

Die Methode „*fail( )*“ dient dazu, einen Test garantiert scheitern zu lassen. Sie kann beispielsweise in eine Zeile eingefügt werden, die beim Test eigentlich nicht erreicht werden soll, z. B. weil vorher eine Exception geworfen werden sollte.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class MitarbeiterTest3 {
  Mitarbeiter m1;

  @BeforeEach
  public void setUp() throws Exception {
    m1 = new Mitarbeiter("Uwe","Mey");
    m1.addFachgebiet(Fachgebiet.ANALYSE);
    m1.addFachgebiet(Fachgebiet.C);
    m1.addFachgebiet(Fachgebiet.JAVA);
  }

  @Test
  public void testHatFaehigkeit1(){
    assertTrue(
        m1.hatFachgebiet(Fachgebiet.C),
        "Faehigkeit vorhanden?");
  }

  @Test
  public void testHatFaehigkeit2(){
    assertFalse(
        m1.hatFachgebiet(Fachgebiet.TEST),
        "Faehigkeit vorhanden?");
  }
}
```

Listing 58: Text-Fixture (Nach [Kleu26])

18.6.4 Test-Fixtures

Oftmals werden für Tests bestimmte Voraussetzungen benötigt. So müssen vorher verschiedene Objekte angelegt und Daten initialisert werden. Möchte man dies nicht für jeden Test einzeln neu machen, so kann man dies in einer mit der Annotation „*@BeforeEach*“ gekennzeichneten Methode tun. Man spricht auch von Test-Fixtures. Das englische Wort Fixture bedeutet Ausstattung. Es geht also darum, die erforderliche Ausstattung zur Durchführung der Tests bereitzustellen.

In Listing 58 wird in der Methode „*setUp( )*" ein neues Mitarbeiter-Objekt angelegt und in einem Attribut gespeichert. Anschließend werden dem Mitarbeiter-Objekt drei Fachgebiete zugeordnet. Diese mit „*@BeforeEach*" annotierte Methode wird automatisch vor jedem der beiden Tests durchgeführt, so dass in den Tests das mit den entsprechenden Fachgebieten ausgestattete Mitarbeiterobjekt zur Verfügung steht.

Da die Tests nichts an dem Mitarbeiterobjekt verändern, ist die erneute Ausführung vor dem zweiten Test eigentlich gar nicht nötig. Es würde genügen, „*setUp( )*" nur einmal, am Anfang auszuführen. Dies erreicht man mit der Annotation „*@BeforeAll*", wobei es sich um eine statische Methode handeln muss, die das Objekt dann in einem statischen Attribut speichert.

Insgesamt stehen folgende Annotationen für Text-Fixtures zur Verfügung:

@BeforeEach	Die Methode wird vor jedem einzelnen Test ausgeführt, z. B. für das Anlegen von Objekten oder Datensätzen in einer Datenbank.
@AfterEach	Die Methode wird nach jedem einzelnen Test ausgeführt, z. B. um Änderungen in der Datenbank rückgängig zu machen, damit sie für den nächsten Test wieder unverändert zur Verfügung stehen.
@BeforeAll	Die mit @BeforeAll annotierte statische Methode wird einmalig ganz am Anfang durchgeführt, also nicht vor jedem Test wiederholt. Objekte und Daten, die für alle Tests zur Verfügung stehen sollen, können in statischen Attributen gespeichert werden. Die Werte gewöhnlicher Instanzattribute der Testklasse stehen hingegen nur innerhalb des jeweiligen Tests zur Verfügung, da bei jedem Test ein neues Objekt angelegt wird.
@AfterAll	Die statische Methode wird einmalig ganz am Ende durchgeführt, z. B. um Änderungen rückgängig zu machen.

18.6.5 Exceptions in Tests

Es sollte auch getestet werden, ob Methoden bei auftretenden Ausnahmen die erwarteten Exceptions werfen. Führt man gewöhnliche JUnit-Tests aus, und es tritt eine Exception auf, so wird diese als Error gemeldet – im Gegensatz zu einem Failure, der ein nicht erwartetes Ergebnis anzeigt. Zwar kann man anhand von Errors erkennen, ob die richtige Exception geworfen wurde, doch muss man bei mehreren Errors jeden einzelnen anklicken und überprüfen, ob es sich tatsächlich um die erwartete Exception handelt.

Komfortabler ist es, wenn man angibt, welche Exception erwartet wird. So überprüft „*assertThrows( )*" in Listing 59 ob bei Ausführung dieses Testfalls eine Exception vom Typ „*IllegalArgumentException*" geworfen wird. Ist dies dann tatsächlich der Fall, so wird der Test als erfolgreich gemeldet. Sollte hingegen keine Exception geworfen werden, bekommt man ein Failure angezeigt.

```
@Test
public void testAddFaehigkeit(){
  assertThrows(IllegalArgumentException.class, () -> {
    m1.addFachgebiet(Fachgebiet.TEST);
  });
}
```

Listing 59: Testen, ob die gewünschte Exception geworfen wird (Nach [Kleu26])

Möchte man nicht nur den Typ der geworfenen Exception überprüfen, sondern auch den enthaltenen Text, so kann man dies wie in Listing 60 machen. Nach der ersten Prüfung mittels „*AssertThrows()*" findet eine zweite Prüfung statt, bei der mit Hilfe von „assertEquals( )" der von der Exception gelieferte Text mit dem erwarteten Text verglichen wird.

```
@Test
public void testAddFaehigkeit2(){
  Throwable exception = assertThrows(IllegalArgumentException.class, () -> {
      m1.addFachgebiet(Fachgebiet.TEST);
  });
  assertEquals(exception.getMessage(), "Maximal 3 Fachgebiete");
}
```

Listing 60: Auffangen und überprüfen der Exception im Test

18.6.6 Fazit JUnit

Mit Hilfe von JUnit lassen sich vor allem Unit-Tests gut unterstützen. Es ist damit zunächst ein wichtiges Hilfsmittel für jeden einzelnen Entwickler bei seiner täglichen Arbeit. Für neu geschriebene Methoden lassen sich schnell geeignete Tests schreiben, die sich später auch beliebig wiederholen lassen.

Hiermit lässt sich auch der Ansatz des „Test Driven Development" gut unterstützen. Dabei werden zuerst die Tests geschrieben und erst anschließend die eigentlichen Methoden, die sukzessive erweitert werden, bis sie schließlich alle definierten Tests bestehen.

Durch den Einsatz von Fixtures kann man auch Integrations- und Systemtests automatisieren.

Die Benutzungsoberfläche einer Software lässt sich nicht mit JUnit automatisiert testen. Hierfür gibt es spezielle Tools. Die Automatisierung von Oberflächentests ist jedoch prinzipiell aufwändiger, da z. B. die verschiedenen Eingabeelemente eindeutig identifiziert und bei der Testdurchführung in der richtigen Reihenfolge mit Eingaben versehen bzw. angeklickt werden müssen, o. ä. Daher spielen beim Testen von Oberflächen manuelle Tests nach wie vor eine große Rolle, zumal neben der Funktionalität zugleich auch die Bedienbarkeit getestet werden kann.

19 Quellenverzeichnis

[Balz09] Balzert, H.: Lehrbuch der Softwaretechnik. Basiskonzepte und Requirements Engineering.
3. Auflage. Spektrum Akademischer Verlag, Heidelberg 2009.

[Balz11] Balzert, H.: Lehrbuch der Objektmodellierung: Analyse und Entwurf mit der UML2.
2. Auflage. Spektrum Akademischer Verlag, Heidelberg 2011.

[BoFa14] Bourque, P.; Fairley, R. (Hrsg.):, Guide to the Software Engineering Body of Knowledge Version 3.0. IEEE Computer Society, 2014.

[Cand15] McCandless, D.: Codebases. Millions of lines of code. V09. 24.9.2015.
http://www.informationisbeautiful.net/visualizations/million-lines-of-code/
(abgerufen am 27.7.2026).

[Zörn25] Zörner, S.: Softwarearchitektur en passant. 2025
https://www.dokchess.de (abgerufen am 27.7.2026).

[Eber22] Ebert, C.: Systematisches Requirements Engineering.
7. Auflage. dpunkt, Heidelberg 2022.

[GaHe14] Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J.: Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software.
Neue Übersetzung. mitp, Frechen 2014.

[Goll11] Goll, J.: Methoden und Architekturen der Softwaretechnik.
Vieweg-Teubner, Wiesbaden 2011.

[Gull11] Gulla, J.: Seven Reasons Why Information Technology Projects Fail.
Präsentation. SHARE Conference, Orlando 11.8.2011.

[HaWo15] Hastie, S.; Wojewoda S.: Standish Group 2015 Chaos Report - Q&A with Jennifer Lynch. InfoQ 4.10.2015. https://www.infoq.com/articles/standish-chaos-2015
(abgerufen am 27.7.2026).

[Kleu26] Kleuker, S: Qualitätssicherung durch Softwaretests. 3. Auflage.
Springer Vieweg, Wiesbaden 2026.

[OMG13] Object Management Group (Hrsg.): Business Process Model and Notation (BPMN). Version 2.0.2. Needham, MA 2013
https://www.omg.org/spec/BPMN/2.0.2/PDF (abgerufen am 30.8.2026).

[OMG17] Object Management Group (Hrsg.): OMG Unified Modeling Language (OMG UML). Version 2.5.1. Needham, MA 2015
http://www.omg.org/spec/UML/2.5.1/PDF/ (abgerufen am 27.6.2026).

[ReRe18] Reiss, M.; Reiss, G.: Praxisbuch IT-Dokumentation.
3. Auflage. Hanser, München 2018.

[SeGu06] Seemann, J.; von Gudenberg, J.: Software-Entwurf mit UML 2.
2. Auflage. Springer, Berlin Heidelberg New York 2006.

[Star24] Starke, G.: Effektive Software-Architekturen.
10. Auflage. Hanser, München 2024.

[Tiem18] Tiemeyer, E. (Hrsg.): Handbuch IT-Projektmanagement.
3. Auflage. Hanser, München 2018.

[Vect23] Requirements Engineering und Produktmanagement Vorlagen.
Vector Consulting Services GmbH, Stuttgart 2023
https://consulting.vector.com/de/de/download/requirements-engineering-und-produktmanagement-vorlagen (Angabe von Name und E-Mail erforderlich, abgerufen am 27.7.2026).

[Vige10] Vigenschow, U.: Testen von Software und Embedded Systems.
2. Auflage. dpunkt, Heidelberg 2010.

[Wird22] Wirdemann, R.: Scrum mit User Stories.
4. Auflage. Hanser, München 2022.

[Zörn21] Zörner, S.: Softwarearchitekturen dokumentieren und kommunizieren.
3. Auflage. Hanser, München 2021.